Artificial Neural Networks and Deep Learning - Notes - v0.8.0

260236

October 2026

Preface

Every theory section in these notes has been taken from two sources:

- Ian Goodfellow and Yoshua Bengio and Aaron Courville, Deep Learning, MIT Press. [3]
- Course slides. [11]

About:

GitHub repository

These notes are an unofficial resource and shouldn't replace the course material or any other book on artificial neural networks and deep learning. It is not made for commercial purposes. I've made the following notes to help me improve my knowledge and maybe it can be helpful for everyone.

As I have highlighted, a student should choose the teacher's material or a book on the topic. These notes can only be a helpful material.

During the Artificial Neural Networks and Deep Learning course, me and my other three colleagues Alberto Ondei, Abdullah Javed and Filippo Barbari, we created two projects:

Time Series Classification. Deep Neural Netowrk (TCN + BiLSTM with Attention) model on multivariate time-series classification.

Image Classification. Histopathology image classification to predict molecular subtypes.

Contents

1 Introduction to Deep Learning **8**
- 1.1 Machine Learning Foundations . . . 8
  - 1.1.1 Machine Learning Paradigms . . . 11
    - 1.1.1.1 Supervised Learning . . . 12
    - 1.1.1.2 Unsupervised Learning . . . 15
    - 1.1.1.3 Reinforcement Learning . . . 20
- 1.2 Towards Deep Learning . . . 23
- 1.3 Modern Pattern Recognition (Pre-DL) . . . 25
- 1.4 What is Deep Learning after all? . . . 27
- 1.5 What's Behind Deep Learning? . . . 32
- 1.6 Summary . . . 34

2 From Perceptrons to FNNs **35**
- 2.1 Historical Context . . . 35
- 2.2 The Perceptron . . . 41
  - 2.2.1 Who Invented It? . . . 41
  - 2.2.2 Mathematical Model & Logical Operations . . . 43
  - 2.2.3 Hebbian Learning Rule . . . 47
  - 2.2.4 Perceptron as Linear Classifier . . . 51
  - 2.2.5 Boolean Operators & Linear Separability . . . 54
- 2.3 Feed-Forward Neural Networks (FNNs) . . . 57
  - 2.3.1 Architecture . . . 57
  - 2.3.2 Activation Functions . . . 60
    - 2.3.2.1 Linear . . . 62
    - 2.3.2.2 Sigmoid . . . 64
    - 2.3.2.3 Hyperbolic Tangent (tanh) . . . 67
  - 2.3.3 Output Layer . . . 71
    - 2.3.3.1 Regression . . . 72
    - 2.3.3.2 Binary Classification . . . 76
    - 2.3.3.3 Multi-Class Classification . . . 80
  - 2.3.4 Neural Networks as Universal Approximators . . . 84
- 2.4 Learning and Optimization . . . 86
  - 2.4.1 Supervised Learning and Training Dataset . . . 87
  - 2.4.2 Error Minimization and Loss Function (SSE) . . . 90
  - 2.4.3 Gradient Descent Basics . . . 94
  - 2.4.4 Backpropagation (Conceptual Introduction) . . . 100
- 2.5 Maximum Likelihood Estimation (MLE) . . . 107
- 2.6 Perceptron Learning Algorithm . . . 114
- 2.7 Summary . . . 122

3 Neural Networks and Overfitting **123**
- 3.1 Universal Approximation Theorem . . . 123
- 3.2 Model Complexity . . . 128
- 3.3 Measuring Generalization . . . 132
- 3.4 Terminology Clarifications . . . 134
- 3.5 Cross-Validation Techniques . . . 136
  - 3.5.1 Hold-Out Validation . . . 137
  - 3.5.2 Leave-One-Out Cross-Validation (LOOCV) . . . 139

3.5.3 K-Fold Cross-Validation . . . 142
3.5.4 Nested Cross-Validation . . . 145
3.6 Preventing Overfitting . . . 150
3.6.1 Early Stopping . . . 152
3.6.2 Hyperparameter Tuning . . . 155
3.6.3 Weight Decay (L2 Regularization) . . . 164
3.6.4 Dropout (Stochastic Regularization) . . . 173
3.7 Tips & Tricks . . . 176
3.7.1 Activation Function Saturation . . . 177
3.7.2 ReLU and Variants . . . 180
3.7.3 Weight Initialization . . . 184
3.7.4 Batch Normalization . . . 188
3.7.5 Mini-Batch Training . . . 193
3.7.6 Learning Rate Scheduling . . . 197

4 Recurrent Neural Networks (RNNs) 200
4.1 Sequence Modeling . . . 201
4.2 Memoryless Models . . . 205
4.2.1 Autoregressive (AR) Models . . . 205
4.2.2 Feed-Forward Extensions: TDNNs . . . 207
4.3 Models with Memory . . . 210
4.3.1 Hidden State Dynamics and Outputs . . . 210
4.3.2 Linear Dynamical Systems and the Kalman Filter . . . 212
4.3.3 Hidden Markov Models (HMMs) . . . 219
4.3.4 Comparison to Deterministic Recurrent Systems . . . 230
4.4 Definition . . . 232
4.4.1 What is a RNN? . . . 232
4.4.2 Nonlinear Update Equations with Weights . . . 237
4.4.3 Universal Computation Capability (Hava Siegelmann) . . . 240
4.5 Backpropagation Through Time (BPTT) . . . 242
4.5.1 RNN unrolling over U time steps . . . 245
4.5.2 Shared weights across time . . . 250
4.5.3 Training Algorithm Steps . . . 254
4.5.4 Vanishing and Exploding Gradients Limitation . . . 256
4.5.5 Dealing with Gradient Problems . . . 262
4.6 Long Short-Term Memory Network (LSTM) . . . 264
4.6.1 Architecture . . . 264
4.6.2 Gates . . . 268
4.6.3 Lightweight Alternative: Gated Recurrent Unit (GRU) . . . 271
4.6.4 Networks . . . 275
4.6.5 Multi-layer LSTM . . . 279
4.6.6 Bidirectional LSTM (BiLSTM) Networks . . . 282
4.6.7 Practical Tips: Initialization & Conditioning . . . 285
4.7 Sequential Data Problems . . . 288
4.8 Sequence-to-Sequence Learning . . . 293
4.8.1 Probabilistic Formulation . . . 294
4.8.2 Conditional Language Models . . . 296
4.8.3 Encoder-Decoder Framework . . . 299
4.8.4 Training Sequence-to-Sequence Models . . . 301
4.8.5 Special Tokens & Dataset Batch Preparation . . . 305

4.8.6 Decoding: Greedy Search and Beam Search 307

5 Unsupervised Learning: Word Embedding 309
5.1 Introduction 309
5.1.1 Recall Machine Learning Paradigms 309
5.1.2 Neural Autoencoders 312
5.1.3 Applications of Autoencoders 317
5.2 Motivation for Word Embeddings 319
5.2.1 The Limits of Traditional Word Representations 319
5.2.2 Word Similarity Ignorance 322
5.2.3 What is a Word Embedding? 325
5.2.4 Distributed Representations 329
5.3 Language Modeling 331
5.3.1 The Language Modeling Problem 331
5.3.2 N-gram Language Models 334
5.3.3 The Curse of Dimensionality in N-gram Models 339
5.4 Neural Language Models 342
5.4.1 Bengio et al. (2003) 342
5.4.2 Architecture of the Neural Language Model 346
5.4.3 Training the Neural Language Model 352
5.4.4 Results and Impact 358
5.5 Word2Vec 360
5.5.1 From Neural Language Models to Word2Vec 360
5.5.2 Word2Vec Architectures 362
5.5.3 Continuous Bag-of-Words (CBOW) 367
5.5.4 CBOW Training and Weight Updates 370
5.5.5 Word2Vec Facts 374
5.6 Semantic Regularities in Embedding Spaces 376
5.6.1 Semantic Relationships 376
5.6.2 Linear Structure of Embedding Spaces 379
5.7 Applications of Word Embeddings 381
5.7.1 Information Retrieval 381
5.7.2 Document Classification and Similarity 386
5.7.3 Sentiment Analysis 388
5.8 GloVe: Global Vectors for Word Representation 390
5.8.1 Motivation behind GloVe 390
5.8.2 Global Co-occurrence Statistics 393
5.8.3 GloVe Training Objective 396
5.8.4 Comparison with Word2Vec 399

6 Image Classification 402
6.1 Computer Vision and Digital Images 402
6.1.1 What is Computer Vision? 402
6.1.2 Examples of Visual Recognition Tasks 404
6.1.3 Digital Images as Arrays 406
6.1.4 RGB Images and Videos 408
6.2 Local Spatial Transformations and Correlation 410
6.2.1 Local Neighbourhoods 410
6.2.2 Spatial Filters 414
6.2.3 Correlation as a Local Linear Operation 415

6.2.4 Correlation as Template Matching 419
6.2.5 Correlation on RGB Images 422
6.3 Image Classification with Linear Classifiers 424
6.3.1 The Image Classification Problem 424
6.3.2 Feeding Images to Neural Networks 427
6.3.3 One-layer Neural Network for Images 431
6.3.4 Training the Linear Classifier 435
6.4 Interpreting Linear Classifiers on Images 437
6.4.1 Geometric Interpretation 437
6.4.2 Image-based Interpretation 440
6.4.3 Linear Classifier as Template Matching 444
6.5 Why Image Classification is Difficult 447
6.5.1 Dimensionality 447
6.5.2 Label Ambiguity 450
6.5.3 Transformations 451
6.5.4 Inter-Class Variability 455
6.5.5 Perceptual Similarity vs Pixel Similarity 456
6.6 Nearest Neighbour Classifiers for Images 458
6.6.1 Pixel-wise Distances 458
6.6.2 k-NN for Images 460
6.6.3 Why Pixel Distance is Not Perceptual Distance 463
6.6.4 CIFAR-10 and t-SNE Interpretation 466

7 Convolutional Neural Networks (CNNs) 470
7.1 From Hand-Crafted to Data-Driven Features 471
7.1.1 The Feature-Extraction Perspective 471
7.1.2 Hand-Crafted Features for Image Classification 474
7.1.3 Advantages and Limitations of Hand-Crafted Features . . 480
7.1.4 Data-Driven Feature Learning 485
7.2 From Image Filtering to CNNs 491
7.2.1 Correlation as a Local Linear Operation 491
7.2.2 Examples of Linear Image Filters 498
7.3 The Typical CNN Architecture 505
7.4 Convolutional Layers 512
7.4.1 Convolution over Multi-Channel Inputs 512
7.4.2 Multiple Filters and Output Feature Maps 515
7.4.3 General Formula for a Convolutional Layer 517
7.4.4 Convolutional-Layer Hyperparameters 520
7.5 CNN Arithmetic and Parameter Counting 523
7.5.1 One Input Channel and One Filter 523
7.5.2 Multiple Filters 525
7.5.3 Multiple Input Channels 527
7.5.4 Counting Convolutional Parameters 530
7.5.5 CNN-Arithmetic Recap 532
7.6 Important Details of Convolution 534
7.6.1 Correlation vs. Mathematical Convolution 534
7.6.2 CNN Convolution vs. RGB Image Filtering 538
7.6.3 Padding 540
7.6.4 Stride 544
7.6.5 General Output-Size Formula 546

7.7 Activation Functions . . . 547
7.7.1 Why CNNs Need Nonlinearities . . . 547
7.7.2 ReLU and Leaky ReLU . . . 549
7.7.3 Hyperbolic Tangent and Other Activations . . . 551
7.8 Pooling Layers . . . 553
7.8.1 Max Pooling . . . 553
7.8.2 Pooling Size and Stride . . . 556
7.9 Dense Layers and the Complete CNN . . . 559
7.9.1 Flattening and Dense Layers . . . 559
7.9.2 Following Shapes Through a CNN . . . 562
7.9.3 Feature Extraction Network and Classifier . . . 565
7.10 CNNs in Action . . . 569
7.11 LeNet-5: The First Famous CNN . . . 574
7.11.1 Historical Motivation . . . 574
7.11.2 LeNet-5 Architecture . . . 575
7.11.3 LeNet-5 Implementation . . . 579
7.11.4 LeNet-5 Performance . . . 584
7.11.5 CNN vs Direct MLP Parameter Count . . . 585
7.12 Latent Representations Learned by CNNs . . . 588
7.12.1 The CNN Latent Representation . . . 588
7.12.2 Visualizing CNN Features with t-SNE . . . 590
7.13 Image Retrieval and Classification in Latent Space . . . 592
7.13.1 Content-Based Image Retrieval . . . 592
7.13.2 One-Nearest-Neighbour Classification . . . 595

8 CNN Anatomy, Transfer Learning, and Data Scarcity 598
8.1 CNNs as Structured Neural Networks . . . 598
8.1.1 Convolution as a Linear Operation . . . 598
8.1.2 Sparse Connectivity and Weight Sharing . . . 601
8.1.3 Translation Invariance as an Inductive Bias . . . 605
8.1.4 Convolution-as-Fully Connected Summary . . . 607
8.2 The Receptive Field . . . 609
8.2.1 Definition of Receptive Field . . . 609
8.2.2 Receptive Field Growth with Depth . . . 613
8.2.3 Effect of Pooling and Stride . . . 617
8.3 How CNN Representations Change with Depth . . . 622
8.4 Training Convolutional Neural Networks . . . 625
8.4.1 Backpropagation Through CNNs . . . 625
8.4.2 Backpropagation Through Max Pooling . . . 640

9 Exams 644
9.1 2026 . . . 644
9.1.1 July 17 . . . 644

Index 675

1 Introduction to Deep Learning

1.1 Machine Learning Foundations

Humans and animals learn from experience. Computers, too, can improve performance when exposed to more data or feedback. But how do we formally define "learning" in a way that's precise enough for a engineering course? Tom Mitchell[1], in 1997, proposed a now-classic definition:

Definition 1: Task, Experience, Performance

A computer program is said to learn from experience **E** with respect to some class of tasks **T** and a performance measure **P**, if its performance at tasks in **T**, as measured by **P**, improves with experience **E**.

- **Task (T)**: what the program is supposed to do. For example, classification (spam vs not spam), regression (predict house prices) or game playing (chess).
- **Experience (E)**: the data the algorithm is exposed to. For example, training set of labeled emails (spam vs ham), past games played by an agent, sensor data from a robot.
- **Performance measure (P)**: the metric used to evaluate progress. For example, classification accuracy (F1 score), mean square error for regression, total reward in reinforcement learning.

A system "learns" if, after seeing more data or interacting more with the environment, its **measured performance improves**.

Example 1: Definition in Action

Some scenarios:

1. **Email Spam Classifier**
   - **T (task)**: Classify emails as spam.
   - **E (experience)**: Training dataset of emails labeled as spam.
   - **P (performance measure)**: Accuracy on unseen emails.

   If accuracy improves as the classifier sees more labeled data, then computer program learning.

2. **Self-Driving Car**
   - **T**: Driving from A to B safely.
   - **E**: Millions of hours of driving footage + sensor readings.
   - **P**: Fewer accidents per mile, shorter trip times.

   If the car improves after more data, it has learned.

[1] Tom Mitchell is a *pioneer of machine learning*, both as a scientist and as an educator. His 1997 textbook, and especially that concise definition, shaped how an entire generation of students and researches understand Machine Learning (ML).

3. **Chess Playing Agent**
   - **T**: Win games.
   - **E**: Past games played against itself or others.
   - **P**: Win rate.

   More games, better play, computer program learning.

This definition matters because is **broad and general** (covers supervised, unsupervised, and reinforcement learning), it stresses **measurable improvement** (no improvement, no learning), and highlights the **central role of data** (E) and evaluation (P).

Why Mitchell's definition doesn't mentions "Machine Learning" explicitly

1. **It's meant to be general**. Mitchell wasn't defining *what ML is as a field*, but rather *what it means for a program to learn*. He avoided vague terms like "machine learning" or "artificial intelligence" and instead described the *process*:
   - A program improves at a **Task (T)**;
   - Thanks to **Experience (E)**;
   - As measured by **Performance (P)**.
2. **Machine Learning = building such programs**. So instead of saying "*Machine Learning is when...*", he framed it as: "*a computer program is said to learn if...*". That's why his definition became the **canonical operational definition of Machine Learning**.
3. **It links directly to practice**. The definition is testable: we can check if a system improves with experience. This is much stronger than a philosophical definition like "*machine learning is making computers intelligent*".

Example 2: Analogy

Think of physics. Newton didn't define "physics". He defined *laws of motion* and *gravity*. From those definitions, physics as a discipline could build itself consistently.
Similarly, Mitchell didn't define "Machine Learning" as a whole discipline. He defined **what it means for a program to learn**. The field then said: "Machine Learning is the study of programs that satisfy this definition".

Mitchell's definition tells us ML is **not about hardcoding solutions**, but about **improving performance with data-driven experience**, measurable by a task-specific metric.

Why we start with Tom Mitchell's definition

1. **Machine Learning is broad and fuzzy**. People use "learning", "AI", "intelligence" loosely. By giving a **formal, authoritative definition** at the beginning, the course sets a *clear baseline*: what do we mean by *learning*? How do we recognize it in a program?
2. **It frames the whole course**. Everything we explain later, supervised learning, neural networks, deep learning, must fit inside this triplet (Task, Experience, Performance). For example:
   - Neural Network training? It's about improving P on T given more E.
   - Reinforcement learning? Same template, different E and P.
3. **It's rigorous but simple**. Unlike philosophical definitions of intelligente, Mitchell's version is **operational**: it tells us *how to test if learning is happening*. It works as a **scientific foundation**, "*if we can't measure performance improvement, we can't claim the program learned*".
4. **It avoids confusion later**. If we started with supervised learning or deep learning right away, we'd lack the general umbrella. With this definition first, we can always check: "*what is our T? what is our E? what is our P?*".

Mathematical View

Formally, suppose we have:

- Dataset $D = \{(x_i, t_i)\}_{i=1}^{N}$ (inputs + targets).
- A model $f_\theta(x)$ with parameters θ.
- A loss function $L(f_\theta(x), t)$ that measures errors (P).

Learning means finding θ^* that minimizes the expected loss:

$$\theta^* = \arg\min_\theta \ \mathbb{E}_{(x,t)\sim E}\left[L\left(f_\theta(x), t\right)\right]$$

This equation will be explained more thoroughly in the following sections.

1.1.1 Machine Learning Paradigms

When Tom Mitchell gave us the **triplet (T, E, P)**, he provided a general definition of learning. But in practice, machine learning problems usually fall into a few **big paradigms**; categories defined by *what kind of data (experience) we provide* and *what kind of task we want solved*. These paradigms are like **different ways of framing the learning problem**:

1. **Supervised Learning**: we give the algorithm examples of input and desired output. The goal is learn to map new inputs to outputs.
2. **Unsupervised Learning**: we only give input data, no labels. The goal is discover hidden structures or representations.
3. **Reinforcement Learning**: we don't provide explicit labels. The system interacts with an environment, receives **rewards or penalties**, and learns a strategy (policy) to maximize reward over time.

These paradigms are important because are the **building blocks of the field**. Almost any ML problem can be described belonging to (or combining) these three. They differ mainly in the **nature of the data (E)** and the **type of feedback (P)** available. Understanding them helps in choosing the right algorithms and models for a problem.

Example 3: Analogy

Imagine teaching three kinds of students:

- **Supervised Learning student**: we show them math problems *with answers*, and they learn how to solve similar ones.
- **Unsupervised Learning student**: we give them a pile of problems *without answers*, and they try to find patterns (like grouping similar problems together).
- **Reinforcement Learning student**: we give them a puzzle game. They don't know the rules, but they learn through *trial and error* because we give them rewards when they succeed.

1.1.1.1 Supervised Learning

Supervised Learning is like learning *with a teacher*:

- The algorithm is given **examples of inputs and their correct outputs (labels)**.
- The goal is to learn a **mapping function** that predicts the correct output for new, unseen inputs.

Formally:

- Training dataset:

$$D = \{(x_1, t_1), (x_2, t_2), \ldots, (x_N, t_N)\}$$

  Where x_i are inputs and t_i are targets.
- Model: $f_\theta(x) \approx t$.
- Learning: choose parameters θ that minimize a loss function measuring error.

In other words, **Supervised Learning** is a type of machine learning where the algorithm is trained on a labeled dataset, meaning each training example includes both the input data and the correct output. And the goal is to learn a function that maps inputs to outputs, in order to make predictions on new, unseen data.

Types of Supervised Learning

In supervised learning we always have:

- **Inputs** x (features).
- **Outputs** t (labels/targets).
- A **model** $f_\theta(x)$ that learns a mapping from inputs to outputs.

The distinction between **classification** and **regression** depends on the **nature of the output**.

- **Classification**: Predict a **discrete class label**. The output space is a finite set of categories. For example:
  - Binary: $\{0, 1\}$, e.g. spam vs not spam.
  - Multi-class: $\{1, \ldots, K\}$, e.g. digits 0-9.

  From a mathematical point of view:

$$f_\theta(x) : \mathcal{X} \to \{1, 2, \ldots, K\}$$

> **Example 4: Cars vs Motorcycles**
>
> Use the classic triplet:

- **Task (T)**: distinguish between two categories (binary classification).
- **Experience (E)**: dataset of images labeled "car" or "motorcycle".
- **Performance (P)**: accuracy (percentage of correct predictions).

Pipeline (how supervised learning was traditionally done before deep learning):

- **Feature Extraction (Hand-Crafted Features)**. Raw data (like an image, sound, or text) is often too complex to give directly to a simple model. Traditionally, humans designed *rules* or *functions* to extract **features** from raw data.
  - Example (images): count edges, corners, textures, or wheel shapes.
  - Example (text): word frequencies, presence of certain keywords.
  - Example (audio): pitch, energy, Mel-frequency coefficients (MFCCs).

  These features are **manually engineered** to capture the most important aspects of the problem. The output is a vector of numbers (feature vector) that represents each example. This step is about "*what information to feed into the model*".

  In this example, hand-crafted features are:
  - Extract "number of circular shapes" (wheels);
  - Extract "dominant color";
  - Extract "edge orientation histograms".

  The photo is now a vector like $[2, 0.6, 0.8]$
- **Learning a Model (Classifier)**. Once we have feature vectors, we train a **machine learning model** that learns to map those features to outputs (labels or numbers). The model **learns decision boundaries** (for classification) or **functions** (for regression) that separate categories or fit numeric values. This is the **actual learning step**: the algorithm adjusts its parameters from the data.

  In this example, the classifier could be a Support Vector Machine (SVM) model, which learns as follows: if "number of wheels ≈ 2" then is a motorcycle; if "number of wheels ≈ 4" then is a car.

- **Regression**: Predict a **continuous value**. The output space is the set

of real numbers ($\mathbb{R}$). From a mathematical point of view:

$$f_\theta(x) : \mathcal{X} \to \mathbb{R}$$

Example 5: Price Prediction

Use the classic triplet:

- **Task (T)**: predict a **continuous value** instead of a discrete label.
- **Experience (E)**: dataset of houses (features: size, location, rooms) with their selling prices.
- **Performance (P)**: Mean Squared Error (MSE), Mean Absolute Error (MAE), or R^2 score.

Pipeline:

- **Hand-crafted features**: e.g., number of rooms, square meters, distance to city center.
- **Learned regressor**: a model that predicts a continuous output.

In simple terms, if our labels are:

- Categories, it's **classification**.
- Numbers, it's **regression**.

Why Deep Learning Changed This

In **deep learning**, feature extraction and learning are **not separated anymore**. Neural networks **learn features automatically from raw data** (pixels, sound waves, text). So the pipeline becomes **one end-to-end step**: input raw data → neural network → prediction.

More resources about Supervised Learning can be found in the notes for the Applied Statistics course:



1.1.1.2 Unsupervised Learning

Unsupervised Learning is like learning *without a teacher*:

- We only provide the algorithm with **inputs** $x_1, x_2, \ldots, x_N$.
- There are **no labels/targets** telling the algorithm the "correct answer".
- The goal is to **discover hidden structures** or **representations** in the data.

Formally:

- Dataset:
$$D = \{x_1, x_2, \ldots, x_N\}, \quad x_i \in \mathbb{R}^d$$
- Task: find structure in D, e.g., groups, manifolds, lower-dimension embeddings.
- Performance measure: less obvious (since no labels). It can be internal measures (compact clusters, variance explained) or extrinsic measures (utility in downstream tasks).

The most intuitive unsupervised task: Clustering

In supervised learning, we had "car vs motorcycle", categories are known. In unsupervised, no labels are given. The simplest question becomes: "*can we group the data into natural categories, even if we don't know their names?*". That's exactly what clustering does. **Clustering** is the process of grouping data points into **clusters** such that:

- Points in the same cluster are **similar** to each other.
- Points in different clusters are **dissimilar**.

Clustering uses a **similarity measure**, such as Euclidean distance. The algorithm groups data into clusters that minimize within-cluster distance and maximize between-cluster distance. Some common algorithms include:

- **Hierarchical Clustering**. Build a tree of clusters by progressively merging or splitting. Exists two approach: Agglomerative Clustering (Bottom-Up) or Divisive Clustering (Top-Down).

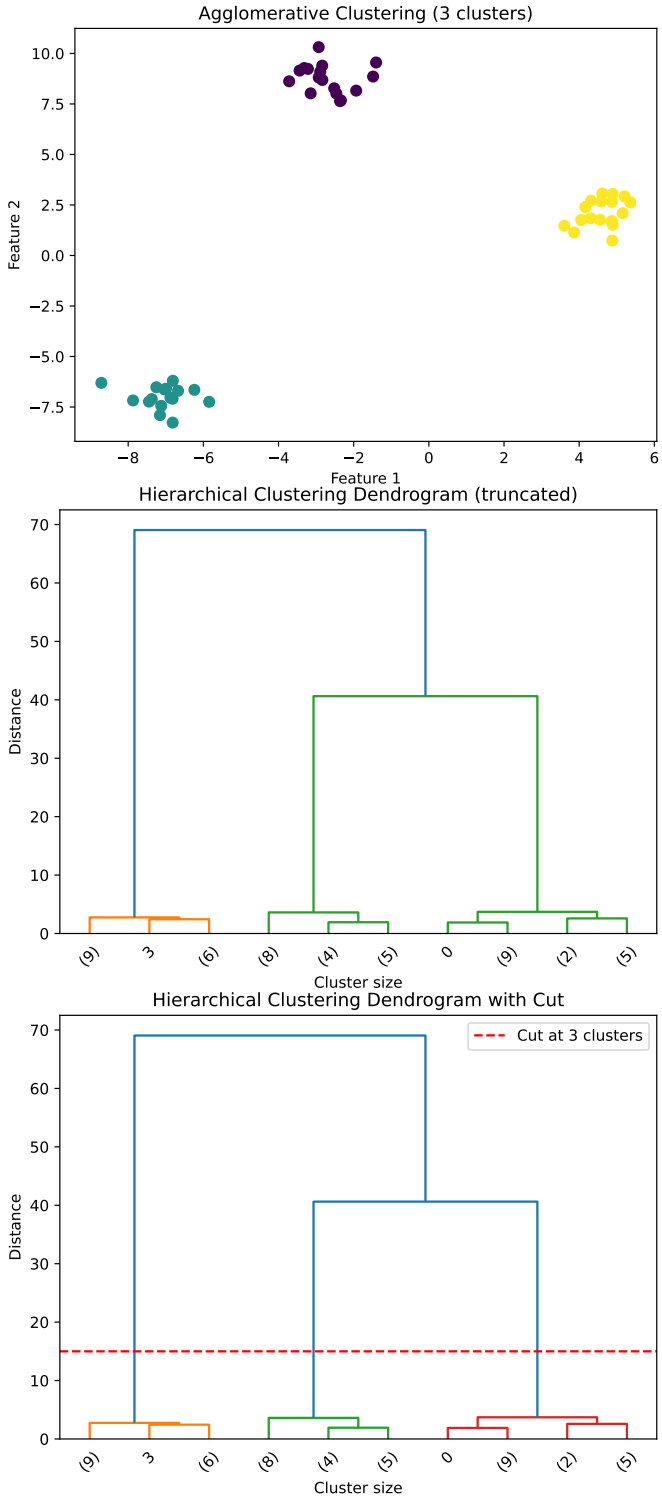


Figure 1: Agglomerative Clustering (top plot), Dendogram (mid plot) and Dendogram with cut (bottom plot).

About Figure 1, page 16. In the Agglomerative Clustering result, each dot is a **data point** (here we generated 50 synthetic points). The algorithm grouped them into **3 clusters**. We can see points within each cluster are **close together** in space. Also, the clusters are **well separated**, this is why hierarchical clustering works well here. The Dendogram shows the **hierarchical merging process**:

- At the **bottom**, each point starts as its own cluster.
- Going **upwards**, clusters that are close together are merged.
- The **height of each merge** (y-axis = distance) indicates how far apart the clusters were when merged.
- At the **top**, all points are eventually merged into a single cluster.

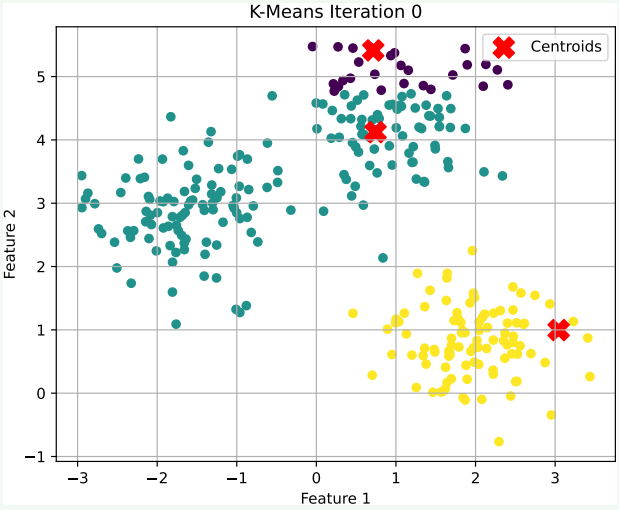
In the last figure, we "cut" the dendogram horizontally at a certain height (distance threshold), and we obtain a chosen number of clusters (here, 3). Everything **below the line** remains as separate clusters. Everything **above the line** (higher merges) is ignored. In the Dendogram, cutting at ≈ 15 gives **3 vertical "branches" crossing the red line**. Each branch corresponds to one cluster. These branches include **all 3 groups of points**.

- **K-Means**. Choose k clusters; assign points to the nearest cluster centroid; and update centroids until convergence.

Example 6: K-Means, taken from the Applied Statistics course

Below is a simple run of the K-means algorithm on a random dataset.

- Iteration 0 - **Initialization**

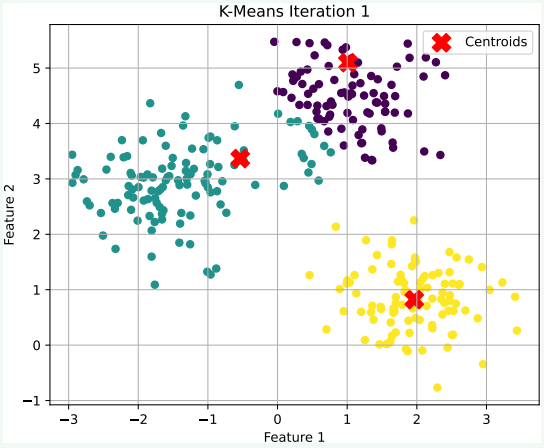


This is the starting point of the K-Means algorithm. **Three centroids are randomly placed in the feature space**.

At this point, no data points are assigned to clusters yet, or all are assumed to be uncolored/unclustered. The positions of the centroids will strongly influence how the algorithm proceeds.

The goal here is to start with some guesses. The next step will use these centroids to form the initial clusters.

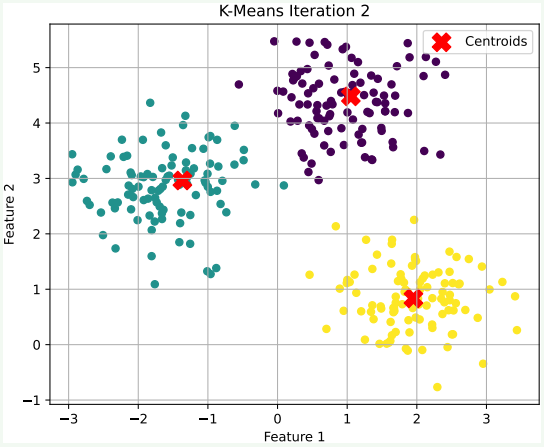
– Iteration 1 - **First Assignment and Update**



Each data point is assigned to the closest centroid, forming the first version of the clusters. New centroids are computed by taking the average of the points in each cluster. We can already see structure forming in the data, as points begin grouping around centroids.

This step is the first real clustering, and centroids begin to move toward dense regions of data.

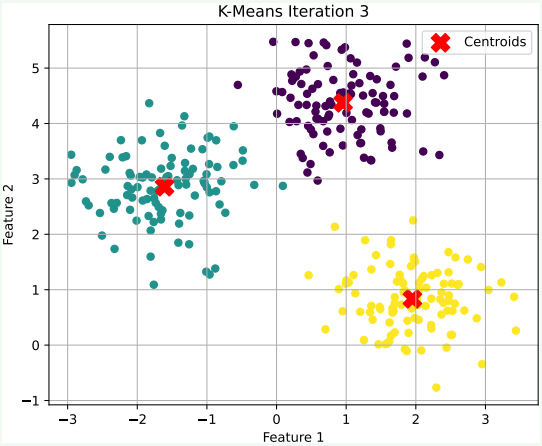
– Iteration 2 - **Re-Assignment and Refinement**



Clusters are recomputed based on updated centroids. Many points remain in the same clusters, but some may shift to a new cluster if a centroid has moved. Centroids continue moving closer to the center of their respective groups.

The algorithm is now refining the clusters and reducing the total distance from points to centroids.

– Iteration 3 - **Further Convergence**



At iteration 3, the K-Means algorithm reached convergence. The centroids no longer moved, and no points changed cluster.This means:

* The algorithm has found a locally optimal solution.
* Further iterations would not improve or change the clustering.
* The final configuration is considered the result of the algorithm.

In practice, this is how K-Means stops: it checks whether the centroids remain unchanged, and if so, it terminates automatically.

More resources about Unsupervised Learning and Clustering can be found in the notes for the Applied Statistics course:

1.1.1.3 Reinforcement Learning

Reinforcement Learning (RL) is like *learning by trial and error*. An **agent** interacts with an **environment** by taking **actions** and receiving **rewards** or **punishments**. The goal of the agent is to learn a policy that maximizes the cumulative reward over time.

At each step, the agent:

1. **Observes a state** s_t from the environment.
2. **Selects an action** a_t based on its current policy $\pi(a_t \mid s_t)$.
3. **Receives a reward** r_t and a **new state** s_{t+1}.

The agent's goal is to learn a **policy** $\pi(a \mid s)$ that maximizes the expected cumulative reward. Unlike supervised learning, no teacher gives the right answer; the agent learns from the **consequences** of its actions.

What is an Agent?

An **agent** is an *entity* that **makes decisions and takes actions in an environment to achieve a specific goal**. In reinforcement learning, the agent learns to optimize its behavior based on feedback from the environment.

With ***entity***, we mean anything that can perceive its environment through sensors and act upon that environment through actuators.

Example 7: Robot Navigation

For example, consider a robot navigating a maze. The robot (agent) perceives its surroundings (state), decides to move left or right (action), and receives feedback (reward) based on whether it gets closer to the exit or hits a wall. The robot's goal is to learn a strategy (policy) that maximizes its chances of reaching the exit while avoiding obstacles.

In simple terms, the robot through cameras and sensors perceives the maze (environment), decides its next move (action), and learns from the outcomes (rewards) to improve its navigation strategy (policy).

In summary:

- **Agent**: The robot.
- **Environment**: The maze.
- **State**: The robot's current position in the maze.
- **Action**: Moving left, right, forward, or backward.
- **Reward**: Positive reward for reaching the exit, negative reward for hitting a wall.
- **Policy**: The strategy the robot uses to decide its next move based on its current state.

The agent's **primary objective** is to **learn a policy that maximizes the cumulative reward** it receives over time by interacting with the environment.

Formalization of Reinforcement Learning

Reinforcement learning problems are often modeled using **Markov Decision Processes (MDPs)**. An MDP is defined by:

- **Task (T)**: learn a policy $\pi(a \mid s)$ mapping states to actions. In other words, the task is to find the best action to take in each state to maximize cumulative reward.
- **Experience (E)**: consists of sequences of states, actions, and rewards obtained by interacting with the environment.
- **Performance Measure (P)**: expected return (sum of discounted rewards):

$$P = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

  Where $\gamma \in [0, 1]$ is the discount factor that determines the importance of future rewards.

Key Concepts in Reinforcement Learning

The goal of this section is to introduce the Reinforcement Learning paradigm and its key concepts. These concepts will be covered in more detail in later sections. However, here are some of those concepts:

- **Exploration vs. Exploitation**: The dilemma of choosing between exploring new actions to discover their effects (***exploration***) and exploiting known actions that yield high rewards (***exploitation***).

  Why a dilemma? Because if the agent only exploits known actions, it may miss out on potentially better actions. Conversely, if it only explores, it may not accumulate enough reward.

- **Reward Signal**: The feedback received from the environment after taking an action, used to evaluate the action's effectiveness. It could be sparse or dense:
  - **Sparse Reward**: Rewards are infrequent, making it challenging for the agent to learn. For example, in a game, the agent might only receive a reward upon winning or losing.
  - **Dense Reward**: Rewards are given frequently, providing more immediate feedback. For example, in a driving simulation, the agent might receive small rewards for staying on the road and penalties for going off-road.

- **Delayed reward**: The reward for an action may not be immediate, making it challenging to associate actions with their long-term consequences. For example, in a chess game, a move may not yield an immediate reward but could lead to a win several moves later. The agent must learn to evaluate actions based on their long-term impact rather than immediate outcomes. This requires the agent to consider future rewards when making decisions.

⚖ RL vs. Supervised Learning

Reinforcement learning differs from supervised learning in several key ways:

Aspect	Supervised Learning	Reinforcement Learning
Data	Fixed labeled dataset (in-out pairs)	No labels; agent generates data by acting
Feedback	Correct answer for each example	Rewards (possibly delayed, sparse)
Goal	Minimize error (classification/regression)	Maximize cumulative reward
Typical methods	Regression, SVM, Neural Nets	Q-learning, Policy Gradients, Actor-Critic

⚠ Challenges of Reinforcement Learning

Reinforcement learning presents several challenges:

- **Exploration**: need to try enough actions to discover good strategies.
- **Delayed Feedback**: rewards may not be immediate, complicating reward assignment.
- **Sample inefficiency**: often requires millions of trials to learn effective policies.
- **Stability**: training can be unstable with neural nets.

Despite these challenges, RL has achieved remarkable success in various domains, including game playing, robotics, and autonomous systems.

In summary, reinforcement learning is a powerful paradigm for training agents to **make decisions in complex environments** by **learning from the consequences** of their actions. RL is distinct from supervised learning in its approach to data, feedback, and goals, making it suitable for a wide range of applications where direct supervision is not feasible.

1.2 Towards Deep Learning

This course, and this notes, focuses **mostly on Supervised Learning**, with some unsupervised learning concepts and techniques. ***Why?***

- Supervised Learning is the most widely used paradigm in practice (e.g., image classification, speech recognition, etc.);
- Many deep learning application (image recognition, NLP, etc.) are supervised tasks;
- Unsupervised learning will be touched when needed (e.g., representation learning, generative models, etc.);

Deep Learning is not a new paradigm, it's a **new approach** with supervised/unsupervised learning.

What about Deep Learning? Iris Flower Example

The Iris flower dataset is a classic dataset in machine learning, often used for classification tasks. It consists of 150 samples of iris flowers, each with four features: sepal length, sepal width, petal length, and petal width. The goal is to classify the flowers into three species: Iris setosa, Iris versicolor, and Iris virginica.

- **Traditional Machine Learning Approach:**
  - Extract "good features" from the raw data (e.g., petal length and width);
  - Train a classifier (e.g., decision tree) on these features;
- **Deep Learning Approach:**
  - Learn both **features** and **classifier** simultaneously from the raw data;

For example:

1. If **features are simple** (e.g., petal length and sepal width), then the classification task is **easy**, and a simple model (e.g., decision tree) can achieve high accuracy;
2. If **features are complex** (e.g., raw pixel values of flower images), then the classification task is **hard**, and the traditional approach **struggles to extract meaningful features**;
3. If **impossible to know** which features matter, then handcrafted features are **not enough**, and we need a model that can **learn features** from the data itself (e.g., a deep neural network).
4. Deep Learning learns features **directly from raw data**, making it suitable for complex tasks where feature engineering is challenging or infeasible (hierarchical representations).

⚖ Feature Engineering vs. Learned Features

- **Feature Engineering (Traditional ML)**:
  - Feature Engineering is the process of **using domain knowledge to extract features from raw data** that make machine learning algorithms work. It needs human experts to design and select features that are relevant to the task.
  - **Problem:** requires domain expertise, time-consuming, and may not capture all relevant information. It is often brittle and not transferable to new tasks or domains.
- **Learned Features (Deep Learning)**:
  - Learned Features are features that are **automatically learned by the model** from the raw data during training.
  - Layers learn progressively:
    * Lower layers learn simple patterns (e.g., edges, corners);
    * Middle layers learn more complex patterns (e.g., eyes, wheels);
    * Higher layers learn high-level concepts (e.g., faces, cars).
  - **Advantage:** optimized for the task at hand, can capture complex patterns, and are transferable to new tasks or domains. It requires less manual effort and often generalizes better to unseen data.

1.3 Modern Pattern Recognition (Pre-DL)

Before the rise of deep learning, modern pattern recognition techniques were primarily based on traditional machine learning algorithms and statistical methods. These techniques focused on feature extraction, dimensionality reduction, and classification using various algorithms.

Speech Recognition (early 1990s-2011)

Speech recognition systems used a **multi-stage pipeline** approach, which included:

- **Low-level features**: extracted from the raw audio waveform, such as MFCCs (Mel-Frequency Cepstral Coefficients), a compact representation of the spectral properties of the audio signal.
- **Mid-level features**: built by grouping/encoding low-level features over short time windows, capturing temporal dynamics. For example Mixture of Gaussians (MoG) used to model acoustic units (phonemes).
- **Classifier (high-level features)**: used to map mid-level features to words or phrases. Common classifiers included Hidden Markov Models (HMMs) combined with Gaussian Mixture Models (GMMs) to decode sequences of acoustic units into words.

This pipeline worked decently but was very **hand-crafted** and success depended heavily on the quality of feature engineering.

Object Recognition (2006-2012)

Computer vision systems followed a similar multi-stage pipeline approach:

- **Low-level features**: detect edges, corners, gradients using methods like SIFT (Scale-Invariant Feature Transform) or HOG (Histogram of Oriented Gradients).
- **Mid-level features**: combine low-level descriptors into higher-level "visual words". For example, clustering with k-means to create a codebook of visual words, and Sparse Coding to represent images as sparse combinations of these words.
- **Classifier (high-level features)**: train SVMs (Support Vector Machines) or Random Forests to classify images based on mid-level features.

Again, this approach was heavily reliant on hand-crafted features and required significant domain expertise to design effective features. However, before 2012, these methods were the state-of-the-art in many computer vision tasks.

General Pipeline (Pre-DL Pattern Recognition)

The general pattern recognition pipeline before deep learning can be summarized as follows:

1. **Low-level features**: raw signal transformation (e.g., edges, frequencies).

2. **Mid-level features**: encode or cluster low-level descriptors (e.g., visual words, acoustic units).
3. **Classifier (high-level features)**: learns categories from hand-designed representations.

Limitations

- **Domain expertise required**: designing MFCCs, SIFT, HOG, etc. required significant knowledge of the specific domain (speech, vision).
- **Task specific**: features built for one task often did not generalize well to others (e.g., MFCCs don't work well for images).
- **Brittleness**: sensitive to noise, illumination, scaling, speaker accents, etc.
- **Limited expressiveness**: as dataset grew, hand-crafted pipelines saturated in accuracy.

Before deep learning, pattern recognition was a **multi-stage pipeline** heavily **reliant on hand-crafted features and domain expertise**. While effective for its time, it had significant limitations in scalability, generalization, and robustness that deep learning would later address.

1.4 What is Deep Learning after all?

After showing the historical context, what Machine Learning is, the three paradigms and how pre-DL pattern recognition worked, we can finally answer the question:

Now that we know what ML does, what makes Deep Learning *different* from classic ML?

We will take our time answering this question. First, we need to understand the meanings of "features" and "classifiers".

What are "features"?

Features are **numerical representations of the raw data** that capture something meaningful for the task.

Type of Data	Raw data example	Example of features
Images	Pixels (RGB values)	Edges, corners, textures
Audio	Waveform (amplitude over time)	Pitch, frequency spectrum, MFCCs
Text	Words or sentences	Word counts, syntactic structure

In **classical ML**, these features were **manually designed** by humans; engineers decided *what* was important and *how to compute it*. For example:

Input image → extract edges manually → feed into SVM classifier

So we had:

Handcrafted Features → Learned Classifier

Where "handcrafted" means "coded by humans". So, before Deep Learning, the **feature extraction** and the **classifier** were two separate stages in the pipeline, and humans designed the first stage. This approach worked, but only if the human correctly guessed *what features matter* for the task.

What does "Learned Features" mean?

Deep Learning says: "*Stop handcrafting features; let the machine learn them automatically, layer by layer, together with the final classifier*".

In **Deep Learning**, the model itself learns how to transform raw data into useful internal representations. Each layer of a neural network acts as a **feature extractor** that learns automatically *what patterns matter*:

- First layers: detect edges, colors, or simple shapes.
- Intermediate layers: detect object parts (e.g., eyes, wheels, leaves).
- Deep layers: detect abstract categories (e.g., "face", "car", "flower").

So instead of telling the machine *what to look for*, we let it **discover patterns directly from data**. This is the "**learned features**" part.

What does "Learned Classifier" mean?

After features are extracted (automatically or manually), the model still needs to **make a decision**: classify, predict, or generate.

- In traditional ML, this is the final **classifier** stage (e.g., SVM, logistic regression, random forest).
- In Deep Learning, the **last few layers** of the network act as that classifier, they map high-level learned features to output labels.

So, both parts, the *feature extractor* and the *decision function*, are **learned jointly** through backpropagation.

$$\text{Raw Data} \xrightarrow[\text{learned weights}]{\text{Feature Extractor}} \text{Representations} \xrightarrow[\text{learned weights}]{\text{Classifier}} \text{Predictions}$$

So DL uses a single model to learn both **features** and **classifier** together: Learned Features + Learned Classifier. The model not only learns *how to decide* but also *how to see the world*, both are learned from data.

So, "What is Deep Learning after all?"

Deep Learning is **not just a new algorithm**, it's a new way of *approaching representation learning*. If we had to answer in one line: "Deep Learning is the automatic learning of hierarchical data representations and decision functions directly from raw data". That's why it's so powerful: it *adapts* to the data and the task, without relying on human intuition about features.

Deepening: Why Not Everything Is Deep Learning

Deep Learning is *powerful*, but it's not a silver bullet, it's not *free*. It's the best tool **when** we have: large amounts of diverse data, high compute, a task based on perception or pattern recognition. Otherwise, **traditional ML or statistical models** can be simpler, faster, and just as effective.

- **Deep Learning needs a *lot of data*.** Deep models have **millions (sometimes billions)** of parameters. They only generalize well when trained on **massive labeled datasets** (e.g., ImageNet: 14M images). If we have small data, like 300 samples from an industrial machine, a deep model will likely **overfit** and perform worse than simpler methods. In other words, Deep Learning shines when there is **data abundance**, but struggles in **data scarcity**.
- **Deep Learning needs a *lot of computation*.** Training is computationally heavy, requiring specialized hardware: GPUs, TPUs, clusters, or cloud computing. Classic ML (SVMs, Decision Trees, Random Forests) can run on a laptop. Deep nets require weeks of GPU training, hyperparameter tuning, and energy cost. So, if the

task doesn't justify the cost, simpler ML is more efficient.

- **Deep models are *black boxes.*** We can rarely explain *why* a deep network made a decision. For critical systems (healthcare, law, finance, safety) we need **interpretability** and **traceability**. Simpler models like linear regression or decision trees are **transparent**, easy to justify in front of regulators or domain experts. For example, a hospital won't risk a deep net saying "tumor" without being able to explain which features caused that prediction.
- **Deep models are *hard to train and tune.*** Choosing architecture (layers, neurons, learning rate, dropout, etc.) is an art. Training can **diverge** or **get stuck** (vanishing gradients, overfitting, exploding losses). We often need extensive experimentation and deep knowledge of optimization tricks. So, not every team or project can afford the expertise and trial cycles DL requires.
- **Deep Learning doesn't always fit the problem.** Some tasks simply:
  1. Have **structured or tabular data** (e.g., bank records, tabular logs). Here, traditional ML (XGBoost, Random Forests) often outperforms DL.
  2. Require **symbolic reasoning** or **logic**, not pattern recognition. Here, DL struggles to capture rules and relationships that classical AI or rule-based systems handle better.
  3. Need **causal inference**, not just correlations. DL finds patterns but doesn't understand cause-effect relationships, which are crucial in many scientific and policy domains. Let's think about a real-world example: predicting disease spread based on interventions (lockdowns, vaccinations) requires understanding causality, not just correlations in data (not just "if X happens, Y follows", but "if we do X, Y will change").
- **Deep Learning needs *good data.*** DL is extremely sensitive to: label noise (wrong annotations ruin learning); biases in the dataset (can reproduce or amplify them); distribution shifts (fails badly if test data differ from training). Traditional methods often handle noise and small variations more robustly. So, "garbage in $\rightarrow$ garbage out" is even more true with DL.
- **Deep Learning doesn't mean *understanding.*** DL recognizes **patterns**, not meaning. It can detect a cat, but it doesn't *know* what a cat is. It can predict outcomes, but not always *why* they happen. That's why current research explores **hybrid systems** combining DL with: symbolic reasoning (neuro-symbolic AI), knowledge graphs, logic and interpretability layers.

Deepening: ChatGPT, LLaMA & Modern AI Models - What Are They?

ChatGPT, LLaMA, Gemini, Claude, etc. are all based on a specific kind of **Deep Neural Network** called a **Transformer**, introduced in 2017 by Vaswani et al. ("Attention is All You Need"). So, fundamentally:

$$\text{ChatGPT, LLaMA, Gemini, etc.} \in \text{Deep Learning}$$

They are not "beyond" DL, they are its **current frontier**.

What kind of Deep Learning model? They belong to the family of **Large Language Models (LLMs)**.

- **Architecture**: Transformer (a type of deep neural network specialized for sequences and attention).
- **Learning paradigm**: mainly *self-supervised learning*, a subform of unsupervised learning.
- **Objective**: predict the next word (token) given the previous ones.

Mathematically:

$$P\left(w_t \mid w_1, w_2, \ldots, w_{t-1}\right)$$

"Given this context, what's the next most probable word?". That's the only thing it learns. Everything else (reasoning, style, facts) *emerges* from learning this next-token distribution on vast text corpora.

Why are they still called "Deep Learning"? They perfectly fit the definition we discussed earlier: "Deep Learning is the learning data representation and decision functions directly from data".

- They learn **representations** of words, sentences, and even concepts automatically.
- They have **layers upon layers** (up to 100+ in GPT-4).
- They **don't rely on hand-crafted linguistic features** (no human tells them grammar rules).
- They learn everything **directly from raw text data** (syntax, semantics, even reasoning patterns).

So they exemplify:

$$\begin{gathered}\text{Learned Features (embeddings)} + \\ \text{Learned Classifier (next word predictor)}\end{gathered}$$

But at **massive scale**, with **billions of parameters** and trained on **terabytes of text**. This scale is what enables their surprising capabilities.

What makes them *different* from earlier Deep Learning. Traditional DL (e.g., CNNs, RNNs) had strong **task specialization**: CNNs for vision, RNNs for sequences, LSTMs for time series. Instead, Transformers with LLMs changed the game because they are **general-purpose learners**:

- They can handle language, code, images, audio, even multimodal data.
- Their **attention mechanism** learns relationships between all parts of the input simultaneously.

They are sometimes called: "Foundation Models", because they can be *fine-tuned* for many downstream tasks (translation, summarization, question answering, etc.).

Why do they feel intelligent? When we train on *massive data* (trillions of words) and *huge models* (hundreds of billions of parameters), the model starts showing **emergent behaviors**:

- Understanding context, humor, and nuance.
- Performing reasoning and arithmetic.
- Generating coherent, creative text.
- Translating languages fluently.
- Writing code snippets.

But still, it's pattern prediction. There is **no explicit symbolic reasoning** or understanding; it's just learned statistical structure at enormous scale. So we say: "They are **Deep Learning models**, trained on **massive dataset**, showing **emergent intelligence**".

1.5 What's Behind Deep Learning?

If the concept of neural networks exists since the 1950s, ***why did Deep Learning explode only after 2012?*** This is a natural question that comes *after* we've seen what Deep Learning is. To answer this question, we show two perspectives: the **MIT view** and **The Economist view**.

The MIT view: Computational Power

According to MIT and many early researchers, Deep Learning became possible only when **computational resources** caught up with the theory. It means that the mathematics and algorithms (backpropagation, perceptrons, convolutional nets) existed for decades, but **training deep networks** requires enormous computation:

- Millions of matrix multiplications.
- Thousands of gradient updates per sample.
- Gigantic datasets.

Before 2010, this was impractical. Around 2011-2012, **GPUs** (Graphics Processing Units) changed everything:

- They made large-scale matrix computations thousands of times faster.
- Deep learning frameworks (Theano, TensorFlow, PyTorch) made GPU computing accessible.
- Hardware parallelism allowed training networks with **hundreds of layers** instead of 3-4.

From the MIT perspective: Deep Learning became possible because **we finally had the computational power to train deep models**.

The Economist view: Big Data

In 2012, *The Economist* (yes, the famous magazine) proposed a different, and equally valid, explanation: "Deep Learning exploded because the world finally generated **enough data** to feed it". It means that the Internet, social media, smartphones, sensors, and cloud storage created **massive labeled datasets**:

- ImageNet (over 14 million labeled images).
- YouTube (millions of labeled videos).
- Text from web, Wikipedia, books, perfect for LLM pretraining.

Deep neural networks thrive on data volume: they don't generalize well with few examples. The more data, the better they learn **hierarchical representations**. So from the Economist perspective: "Deep Learning rose because **we finally had Big Data**, the fuel it needs to work".

The Real View: Both Matter

In reality, both perspectives are correct and complementary. Deep Learning's success is due to the **synergy of computational power and big data**:

- Before 2010, algorithms existed but computing was too slow and data too scarce. Then neural networks were limited to shallow architectures and small datasets.
- Around 2012, hardware (GPUs, TPUs, distributed training) made computation feasible. Simultaneously, the explosion of digital data provided the massive labeled datasets needed.

This combination triggered the **Deep Learning revolution**. The turning point was **ImageNet 2012**, where Krizhevsky, Sutskever, and Hinton demonstrated that a deep convolutional network (AlexNet) could drastically outperform traditional methods on image classification. [8] This success was possible only because:

- They used two NVIDIA GPUs to train a deep network with millions of parameters.
- They trained on the large ImageNet dataset with 1.2M labeled images.

The result was an error rate of 15%, compared to 26% for the best traditional method. This landmark event showcased the **power of deep learning when both computational resources and big data are available**.

Read the original paper by Krizhevsky et al. (2012)



1.6 Summary

Everything we've seen, supervised, unsupervised, or reinforcement learning, ultimately depends on **how we represent data**. In traditional ML, features are *hand-crafted*. In Deep Learning, features are *learned automatically* through hierarchical representations. The revolution of Deep Learning wasn't new math, it was learning **what matters** in the data instead of coding it by hand.

$$\text{Success of ML} \Rightarrow \text{Success of its feature representation}$$

Deep networks just made the **representation learning** automatic and scalable.

Deep Learning = Learning Data Representation from Data

Deep Learning is not a specific architecture (like CNN, RNN, or Transformers) or algorithm. It's the **paradigm** where:

1. Input → raw data (e.g., pixels, text, audio)
2. Model → multiple non-linear layers learning internal representations.
3. Output → desired prediction/task.
4. Learning → end-to-end optimization of all layers together.

So instead of:

$$\text{Human designs features} \rightarrow \text{Model learns mapping}$$

We now have:

$$\text{Model learns both features and mapping} \rightarrow \text{directly from data}$$

This is the essence of Deep Learning: **learning hierarchical representations directly from raw data**.

"Which data?" - The key question of the course

This is the **transition line** to the rest of the course (notes). Now that we know *what* Deep Learning is, the next question is *what data we use and how*. Different data types define the upcoming sections:

Data Type	Upcoming Section
Tabular / numerical	Perceptrons & Feed-Forward NNs
Images	Convolutional Neural Networks (CNNs)
Sequential (text, time series)	Recurrent Neural Networks (RNNs) & Transformers
Unlabeled data	Autoencoders & Word Embeddings

So this question of "which data?" becomes the **roadmap** for the rest of the course (notes).

2 From Perceptrons to FNNs

2.1 Historical Context

When Artificial Intelligence first emerged as a field in the 1940s and 1950s, researchers were fascinated by the idea of creating machines that could *think*, *adapt*, and *learn* as the human brain does. At that time, traditional computers were already capable of executing precise, deterministic instructions with incredible speed. However, these **early machines lacked flexibility**: they could not interpret noisy or ambiguous input, nor could they modify their behavior from experience.

This limitation led scientists to look beyond the rigid Von Neumann architecture[2] and toward the **brain** as an alternative computational paradigm. The human brain, with its billions of interconnected neurons, represented a radically different kind of machine: **massively parallel**, **distributed**, **redundant**, and **fault-tolerant**. Each neuron is *simple*, yet together they form a system capable of extraordinary complexity and adaptability.

From this inspiration arose the idea of **neural networks**: mathematical models built from simple interconnected units that imitate, in a highly abstract way, the behavior of biological neurons. Interestingly, neural networks are not a recent invention of the deep learning era: they have existed since the birth of AI itself. In fact, the phrase "*Deep Learning is not AI, nor Machine Learning*" is often used pedagogically to emphasize that **deep learning is only a specific family of techniques within machine learning, which itself is a subfield of artificial intelligence**. Neural networks therefore belong to a broader historical research tradition that predates modern deep learning.

In summary, the reason researchers in the 1940s and 1950s looked "beyond Von Neumann" was that they sought to create machines that could **learn from experience** and **adapt to new situations**, capabilities that traditional computers lacked:

- **1940s motivation**: classic computers were excellent at precise and fast arithmetic, but researchers wanted systems that could **interact with noisy data**, operate in **parallel**, be **fault-tolerant**, and **adapt**.
- **Brain as a computational model**: the brain offers a radically different architecture that is massively parallel, distributed, redundant system. These properties are an appealing template for computation, which inspired artificial neurons and later full neural networks.

The inception of AI

In the years immediately following the Second World War, a new scientific dream began to take shape: the idea that intelligence could be recreated in a machine. Early pioneers such as **Alan Turing**, **John von Neumann**, **Warren McCulloch**, and **Walter Pitts** laid the foundations of what would soon

[2] The sequential model where computation and memory are separated

be called *Artificial Intelligence*. Computers had just proven they could follow precise instructions and perform huge calculations at incredible speed, yet these machines were nothing more than rigid automata: they obeyed every command literally, unable to perceive, reason, or learn.

The emerging field of AI was born from the desire to bridge that gap, to make machines that could **adapt**, **generalize from experience**, and **interact intelligently** with the world. The 1940s and 1950s were therefore an era of conceptual excitement: *could the brain's mechanisms be modeled mathematically and implemented in hardware or software?* The earliest experiments sought to replicate the nervous system's structure, creating computational units that mimicked neurons and synapses. These units could, in principle, activate or remain silent depending on the inputs they received, a primitive form of reasoning.

At this stage, AI and neural networks were almost inseparable: **building an intelligent machine was often seen as building an artificial brain**. In the following decades, this original vision split into two major traditions. The first was the *symbolic* approach, based on explicit rules, logic, and hand-crafted representations. The second was the *connectionist* approach, where systems learn from data through networks of simple computational units. Although the connectionist line remained in the background for many years, it later reemerged as the foundation of what we now call **Deep Learning**.

From Von Neumann Machines to Brain-Inspired Models

In the 1940s, the **Von Neumann architecture** defined what we still call a *classical computer*: a machine with a central processor (CPU) that executes instructions stored in memory, step by step, following a deterministic sequence. This design is extremely powerful for arithmetic and logic, but it has key limitations when the goal is to emulate intelligence.

A Von Neumann computer is **serial**, **rigid**, and **exact**: it does exactly what it's told, line by line. Intelligence, however, requires something different, the ability to handle **noisy or incomplete data**, **recover from errors**, **adapt to change**, and **operate in parallel** on many signals at once. The human brain, in contrast, is a **massively parallel** and **distributed** system made of roughly 10^{11} neurons, each connected to thousands of others through 10^{14} to 10^{15} synapses.

This comparison motivated the idea of a **computational model inspired by the brain**. Instead of a single central processor, the brain uses huge numbers of simple processing units (neurons) working together. Each neuron performs a small, nonlinear operation, but their collective behavior gives rise to perception, reasoning, and learning.

Researchers realized that if intelligence in humans comes from these interactions, perhaps machines could become intelligent by simulating **networks of artificial neurons**, each following simple rules, but collectively capable of complex, adaptive computation.

In short:

- Von Neumann: deterministic, sequential, rigid.
- Brain-inspired: parallel, adaptive, fault-tolerant.

This shift marks the conceptual birth of **neural networks** as a new computational paradigm.

Neural Networks in the Early AI Era

The idea of using the **human brain as a model for computation** comes from how extraordinary its complexity and efficiency are. A typical adult brain contains around **100 billion neurons** (10^{11}), and each neuron is connected to roughly **7'000** others, forming an estimated 10^{14} - 5×10^{14} **synapses**, even reaching 10^{15} in a three-year-old child.

Despite being slow compared to digital processors (neurons fire in milliseconds, not nanoseconds), the brain's power lies in its **massive parallelism** and **redundancy**. Each **neuron is a simple processing element**, but together they create a distributed, nonlinear, and fault-tolerant system capable of perception, reasoning, adaptation, and learning; functions that no single algorithmic machine of the 1940s could perform.

From a computational viewpoint, this means:

- **Processing is distributed**: no central control; intelligence arises from interactions.
- **Information is encoded collectively**: a concept survives even if some neurons fail.
- **Parallelism ensures speed and robustness**: thousands of operations occur simultaneously.
- **Adaptivity**: synaptic strengths (connections) change with experience, enabling learning.

These characteristics inspired the **first attempts to formalize "neurons" mathematically**, giving rise to the **perceptron** and to the field of *artificial neural networks*. The perceptron is, in essence, a simplified abstraction of how a biological neuron integrates inputs, applies a threshold, and produces an output. An idea that we'll explore in the following section.

⚖ What about the computation of biological versus artificial neurons?

🧠 In a **biological neuron**, information is transmitted through **electrochemical signals**:

- The **dendrites** receive inputs from other neurons through *synapses*.
- Each input can be **excitatory** (it increases activation) or **inhibitory** (it decreases activation).
- The neuron **integrates** all these signals in the **cell body (soma)**.
- When the total accumulated signal exceeds a **threshold**, the neuron **fires**, sending an output through its **axon** to other neurons.

Although this process is complex and involves various biochemical mechanisms, it can be summarized as:

collect inputs → integrate → compare with threshold → fire

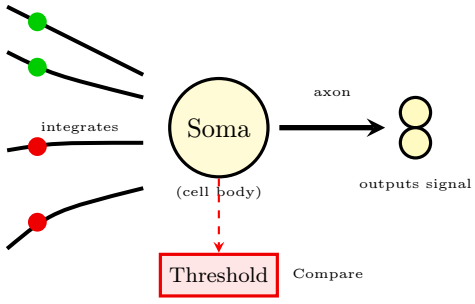


Figure 2: **Biological neuron structure**. The *dendrites* receive *excitatory* (green) and *inhibitory* (red) signals through synapses, which are then integrated in the *soma* (cell body). If the total accumulated signal exceeds a threshold, the neuron *fires*, sending an output through the *axon* to other neurons.

⚙ But how to model this computationally? In the **artificial version**, we simplify this biological process into a mathematical model:

$$h_j(x, w, b) = f\left(\sum_{i=1}^{I} w_i x_i - b\right) = f\left(w^T x\right)$$

Where:

- x_i are the **input values** (analogous to signals received by dendrites). They are like the neurotransmitter signals that a biological neuron receives from other neurons.
- w_i are the **weights** (analogous to synaptic strengths). They represent how strongly each input influences the neuron's activation.
- b is the **bias** (analogous to the threshold). It determines the level of input required for the neuron to activate.

- $f(\cdot)$ is the **activation function** (analogous to the firing mechanism). It decides whether the neuron fires based on the integrated input.

Each artificial neuron thus performs three main steps:

1. **Weighted sum** of its inputs (integration): $\sum_{i=1}^{I} w_i x_i$.
2. **Subtracts the bias** (thresholding): $\sum_{i=1}^{I} w_i x_i - b$.
3. **Applies the activation function** (firing decision): $f\left(\sum_{i=1}^{I} w_i x_i - b\right)$.

Definition 1: Artificial Neuron

An **Artificial Neuron** is a **mathematical model** inspired by the way a biological neuron works (page 38). It's the **basic computation unit** of a neural network.

While a real neuron collects electrical signals from thousands of connections (synapses) and "fires" if the total signal passes a threshold, an artificial neuron does the same thing, but with numbers.

Formally, it takes several inputs $(x_1, x_2, \ldots, x_I)$, multiplies each by a **weight** w_i, sums them, adds a **bias** b, and passes the result through an **activation function** $f(\cdot)$:

$$h_j(x, w, b) = f\left(\sum_{i=1}^{I} w_i x_i - b\right) = f\left(w^T x\right) \tag{1}$$

Where:

- **Inputs** (x_i): the signals coming from other neurons or from data (e.g., pixel values).
- **Weights** (w_i): how strong each input connection is (analogous to synaptic strength).
- **Bias** (b): shifts the activation threshold up or down.
- **Activation function** (f): decides whether the neuron "fires" (outputs a strong signal) or stays quiet.

In essence, the pipeline of an artificial neuron is:

Weighted sum → Threshold/Bias → Nonlinear activation → Output

Definition 2: Bias

The **Bias** is an additional parameter in an artificial neuron that allows

the activation function f to be shifted horizontally, providing the model with the ability to represent patterns that do not pass through the origin.

Mathematically, it appears as the constant term b in the neuron's activation equation (see page 39):

$$a = w^T x + b$$

The bias represents the **intrinsic tendency of a neuron to activate**, even in the absence of input. It acts like a tunable threshold that controls *when* the neuron fires.

Think of the bias as the neuron's **default tendency to fire**, it decides *how easy or hard* it is for the neuron to activate:

- A **large positive bias** $\rightarrow$ neuron tends to fire even with small input.
- A **large negative bias** $\rightarrow$ neuron needs strong evidence (large input sum) to fire.

In other words, the bias *shifts the activation threshold* left or right along the input axis, allowing the neuron to learn more complex decision boundaries.

Imagine a simple rule: "*if the weighted sum of our inputs is greater than 0, we output 1*". Now suppose all our inputs are zero ($x_1 = x_2 = 0$). If we want the neuron to still fire in that case, we need a **bias** to "push" it over the threshold. Bias gives the neuron a *baseline activity*, like saying: "*even if there's not input, we are slightly inclined to fire*".

So, an **artificial neuron** mimics the logical essence of a biological one: a small computing unit that combines multiple inputs into one output, depending on the learned connection strengths (weights) and a bias term. This is the foundation of the **perceptron**, the first neural network model, the topic of the next section.

2.2 The Perceptron

2.2.1 Who Invented It?

Once researchers realized that the brain could be viewed as a network of simple processing units, the next natural step was to formalize this idea into an actual **computational model**, what we now call a **neural network**.

Definition 3: Neural Network

A **Neural Network** is simply a **collection of artificial neurons** (page 39) connected by weighted links. Each neuron:

- Receives inputs,
- Computes a weighted sum,
- Applies an activation function,
- And produces an output that becomes the input for the next neuron.

Through these connections, the network forms a structure capable of **transforming input data into meaningful outputs**, a function approximator that *learns* by adjusting its weights.

The very first implementations appeared in the 1940s-1960s, with three major milestones:

McCulloch & Pitts (1943). They proposed the **Threshold Logic Unit (TLU)**, the first mathematical model of a neuron. Each unit:

- Received multiple binary inputs,
- Multiplied them by fixed weights,
- Summed them up,
- Compared the sum to a threshold,
- Output 1 if the threshold was exceeded, 0 otherwise.

They proved that a network of such units could represent **any logical function**, meaning it could, in theory, "compute" anything if properly wired.

Frank Rosenblatt (1957). He built the first **trainable model**, the **Perceptron**. Rosenblatt's perceptron could automatically **learn** the correct weights from examples using an update rule based on errors. His prototype was implemented in hardware:

- The weights were stored as adjustable electrical components (potentiometers),
- Electric motors updated them during learning. This was the first step from theoretical neuroscience to **machine learning**.

- **Bernard Widrow (1960)**. He developed the **ADALINE (Adaptive Linear Neuron)** and later the **MADALINE (Multiple ADALINE network)**. Widrow's key idea was to express the threshold as a **bias term**, simplifying the equations and making it easier to train models using gradient-based optimization, a cornerstone of modern networks.

Together, these models represent the **first generation of neural networks**: simple, linear systems inspired by the brain but operating with mathematics and electricity. They laid the groundwork for the more complex architectures that would follow, leading to the deep learning revolution we see today.

2.2.2 Mathematical Model & Logical Operations

The **Perceptron** is the **simplest neural network**, a single neuron that transforms multiple input signals into one output through a weighted sum and a thresholding function.

Formally, given inputs:

$$x = [x_1, x_2, \ldots, x_I]$$

And weights:

$$w = [w_1, w_2, \ldots, w_I]$$

The perceptron computes the quantity:

$$a = \sum_{i=1}^{I} w_i x_i + b = w^T x + b \tag{2}$$

Where:

- x_i are the input features,
- w_i are the learnable connection weights,
- b is the **bias** (representing the firing threshold).

Then, this **activation** a passes through a **step function** (also called **threshold** or **activation function**) to produce the final output y:

$$y = \begin{cases} 1 & \text{if } a > 0 \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

In some conventions, the output can also be -1 or $+1$ instead of 0 and 1, depending on how the data is encoded.

Sometimes, we include the bias directly as a weight w_0 associated with a fixed input $x_0 = 1$, rewriting the equations as:

$$y = f\left(w_0 x_0 + w_1 x_1 + \ldots + w_I x_I\right) = f\left(w^T x\right) \tag{4}$$

This makes formulas simpler and more uniform for training algorithms (compact vector notation).

Interpretation of the Perceptron math

The perceptron divides the input space into two regions separated by a **decision boundary** (a hyperplane[3]). If the weighted sum of inputs exceeds the threshold, the neuron "fires" (outputs 1); otherwise, it stays silent (outputs 0). Thus,

[3] A **hyperplane** is a **generalization of a line or a plane** to any number of dimensions. It's the mathematical way to describe a *flat surface* that separates space into two parts. In 1D, a hyperplane is just a point that splits the line into two halves; in 2D, it's a line that divides the plane into two regions, one where the perceptron outputs 1 and the other where it outputs 0; in 3D, it's a plane that separates space into two halves. In higher dimensions, it remains a flat subspace that partitions the input space.

the perceptron acts as a **linear classifier**: it determines which side of the hyperplane the input vector lies on.

We can express the **exact set of points where the neuron is undecided** (the **Decision Boundary Equation**) by setting the activation a to zero:

$$w^T x + b = 0 \quad \text{(decision boundary equation)} \tag{5}$$

What it can actually do: Logical Operations

Now, if the perceptron is a computational unit, *what kind of computations can it perform?* To answer that, we need simple, well-defined **functions** to test it on. The most basic functions are the **logical operations** used in Boolean algebra. Logical operations (like AND, OR, NOT) are perfect because:

- They have **binary inputs** (0 or 1), exactly like neuron activations.
- They produce **binary outputs** (true or false), like the perceptron's step function.
- They let us see immediately whether the neuron can separate input cases correctly.

Logical operations are the **first experiments** that show the perceptron's power as a *linear classifier*.

When the perceptron can reproduce logic operations like AND or OR, it proves that:

1. A single neuron can implement **decision-making**.
2. The model is capable of **classification** (separating inputs into categories).
3. We can assign **geometric meaning** (a hyperplane dividing true/false examples).

Example 1: Logical OR ($\vee$)

x_1	x_2	$y = x_1 \vee x_2$
0	0	0
0	1	1
1	0	1
1	1	1

We want the perceptron to output **1** if *any* input is 1. A possible set of parameters is:

$$w_1 = 1, \quad w_2 = 1, \quad b = -0.5$$

This gives us the activation function:

$$a = w_1 x_1 + w_2 x_2 + b = x_1 + x_2 - 0.5$$

Or equivalently:

$$y = \begin{cases} 1 & \text{if } x_1 + x_2 - 0.5 > 0 \\ 0 & \text{otherwise} \end{cases}$$

So each neuron computes:

$$y = f\left(w_1 x_1 + w_2 x_2 + b\right) = f\left(1 \cdot x_1 + 1 \cdot x_2 - 0.5\right)$$

Checking all input combinations:

- For $(0,0)$: $a = 0 + 0 - 0.5 = -0.5 \Rightarrow y = 0$
- For $(0,1)$: $a = 0 + 1 - 0.5 = 0.5 \Rightarrow y = 1$
- For $(1,0)$: $a = 1 + 0 - 0.5 = 0.5 \Rightarrow y = 1$
- For $(1,1)$: $a = 1 + 1 - 0.5 = 1.5 \Rightarrow y = 1$

Thus, the perceptron correctly implements the OR function.

Example 2: Logical AND ($\wedge$)

x_1	x_2	$y = x_1 \wedge x_2$
0	0	0
0	1	0
1	0	0
1	1	1

We want the perceptron to output **1** only if *both* inputs are 1. A possible set of parameters is:

$$w_1 = 1, \quad w_2 = 1, \quad b = -1.5$$

This gives us the activation function:

$$a = w_1 x_1 + w_2 x_2 + b = x_1 + x_2 - 1.5$$

Or equivalently:

$$y = \begin{cases} 1 & \text{if } x_1 + x_2 - 1.5 > 0 \\ 0 & \text{otherwise} \end{cases}$$

So each neuron computes:

$$y = f\left(w_1 x_1 + w_2 x_2 + b\right) = f\left(1 \cdot x_1 + 1 \cdot x_2 - 1.5\right)$$

Checking all input combinations:

- For $(0,0)$: $a = 0 + 0 - 1.5 = -1.5 \Rightarrow y = 0$
- For $(0,1)$: $a = 0 + 1 - 1.5 = -0.5 \Rightarrow y = 0$
- For $(1,0)$: $a = 1 + 0 - 1.5 = -0.5 \Rightarrow y = 0$

- For $(1, 1)$: $a = 1 + 1 - 1.5 = 0.5 \Rightarrow y = 1$

Thus, the perceptron correctly implements the AND function. However, we can see that other weight/bias combinations could achieve the same result. For example:

$$w_1 = 1.5, \quad w_2 = 1.5, \quad b = -2.0$$

In both examples, the perceptron defines a **line (in 2D)** that separates input combinations giving output 1 from those giving output 0. For OR, the line lies closer to the origin, since only $(0, 0)$ should give 0; for AND, the line lies further away, since only $(1, 1)$ should give 1. So, by adjusting weights and bias, the perceptron can learn to classify inputs according to these logical rules. However, it's clear that **manually setting weights and biases for complex tasks is impractical**. This brings us to the next important topic: ***how can it learn those weights automatically instead of us setting them by hand?***

2.2.3 Hebbian Learning Rule

Now that we understand what the Perceptron does and who invented it, let's explore **how it learns** from data. When the first artificial neurons were proposed, researchers wanted them not just to compute, but to **learn from experience**, as biological neurons do. The earliest and most influential idea for this was the **Hebbian Learning Rule**, introduced by psychologist **Donald Hebb** in 1949.

The biological intuition

Donald Hebb was a psychologist, not a mathematician. In 1949, he was trying to explain **how the brain learns from experience**, without having explicit "teachers" or formulas. He observed that, in biological brains, learning seems to happen **through association**. That's the origin of his famous sentence:

"*Cells that fire together, wire together.*"

This means that if **two neurons** are **active at the same time** (one sending a signal and the other firing) then the **connection** (synapse) between them should **become stronger**. Over time, the brain reinforces useful associations automatically.

In other words, if neuron A consistently helps activate neuron B, the connection from A to B should be strengthened. This principle is thought to underlie learning and memory formation in the brain.

The Artificial Version: Mathematical Formulation

Now, we translate this biological intuition into a mathematical rule that can be applied to the Perceptron. In artificial neurons, "firing" means *output* is active (e.g., output is 1). So if both input and output are active at the same time, that's equivalent to "they fired together". The Hebbian learning rule says:

- **Increase** the weight of connections that are active when the neuron fires.
- **Decrease** or leave unchanged the connections that are inactive or misaligned.

To translate this into a mathematical rule for a Perceptron, we **express the weight update** as follows:

$$\Delta w_i = \eta \cdot x_i \cdot t \tag{6}$$

- If $x_i > 0$ (input is active) and $t > 0$ (target output is active), both are active, then Δw_i is positive, so the weight w_i **increases**, the connection strengthens.
- If $x_i > 0$ (input is active) but $t \leq 0$ (target output is inactive), mismatch, then Δw_i is zero or negative, so the weight w_i **decreases** or remains the same, the connection weakens.
- If $x_i \leq 0$ (input is inactive), regardless of t, then no update occurs since Δw_i is zero, the connection remains unchanged.

Where:

- η is the **learning rate**, a small positive constant that controls how much the weights are adjusted during each update. It ensures that learning is gradual and stable. To make an analogy, think of η as the **speed limit** on a road: it prevents the learning process from speeding ahead too quickly and potentially crashing (i.e., diverging).
- x_i is the ith **input value** to the Perceptron.
- t is the **target output** (desired response) for the given input.
- Δw_i is the **change in weight** for the ith input. This change is added to the current weight w_i to get the new weight.

The full update rule becomes:

$$w_i^{(k+1)} = w_i^{(k)} + \Delta w_i = w_i^{(k)} + \eta \cdot x_i \cdot t \tag{7}$$

This is the **Weight Update Rule**. It tells us *how to modify* each connection w_i after seeing one training example. Conceptually, at each learning step (each training example):

1. **Take the current weights** $w_i^{(k)}$.
2. **Compute** how much they should change $\Delta w_i = \eta \cdot x_i \cdot t$.
3. **Add that change** to get the new weights $w_i^{(k+1)}$.

How it works

1. **Initialize** all weights w_i to small random values (or zeros).
2. **Set** the learning rate η to a small positive value (e.g., 0.01).
3. For each **training example** (x, t):
   - **Compute the Perceptron's output** y using the current weights:
   $$y = f\left(w^T x\right)$$
   - **Compare with the target** t.
     - If $y = t$, the output y matches the target t, the neuron is already correct, so **no weight update is needed** since the association is already learned.
     - If $y \neq t$, the output y does not match the target t, the neuron is incorrect, and we need to **update the weights** to strengthen the association. This is done using the Hebbian learning rule:
     $$w_i^{(k+1)} = w_i^{(k)} + \eta \cdot x_i \cdot t$$
     This can be explained in informal steps:
       * For each weight w_i, compute the change $\Delta w_i = \eta \cdot x_i \cdot t$
       * Update the weight: $w_i \leftarrow w_i + \Delta w_i$
4. Repeat until all examples are correctly classified or a stopping criterion is met (e.g., a maximum number of iterations).

Example 3: Hebbian Learning Rule

Let's say we're learning a simple OR function with two inputs x_1 and x_2. The target outputs t for the four possible input combinations are:

- $x = [0, 0] \rightarrow t = 0$
- $x = [1, 0] \rightarrow t = 1$
- $x = [0, 1] \rightarrow t = 1$
- $x = [1, 1] \rightarrow t = 1$

We do not include a bias term in this example for simplicity. We'll use a step activation function:

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

The algorithm proceeds as follows:

1. **Initialize weights** $w_1 = 0.0$, $w_2 = 0.0$ and learning rate $\eta = 0.1$.
2. **First training example** $x = [0, 0]$, $t = 0$:
   - **Compute output**: $y = f(0.0 \cdot 0 + 0.0 \cdot 0) = f(0) = 0$
   - Output matches target, so **no weight update needed**.
3. **Second training example** $x = [0, 1]$, $t = 1$:
   - **Compute output**: $y = f(0.0 \cdot 0 + 0.0 \cdot 1) = f(0) = 0$
   - Output does not match target, so we **update weights**:

$$\begin{aligned} \Delta w_1 &= 0.1 \cdot 0 \cdot 1 = 0.0 \\ \Delta w_2 &= 0.1 \cdot 1 \cdot 1 = 0.1 \\ w_1 &\leftarrow 0.0 + 0.0 = 0.0 \\ w_2 &\leftarrow 0.0 + 0.1 = 0.1 \end{aligned}$$

4. **Third training example** $x = [1, 0]$, $t = 1$:
   - **Compute output**: $y = f(0.0 \cdot 1 + 0.1 \cdot 0) = f(0) = 0$
   - Output does not match target, so we **update weights**:

$$\begin{aligned} \Delta w_1 &= 0.1 \cdot 1 \cdot 1 = 0.1 \\ \Delta w_2 &= 0.1 \cdot 0 \cdot 1 = 0.0 \\ w_1 &\leftarrow 0.0 + 0.1 = 0.1 \\ w_2 &\leftarrow 0.1 + 0.0 = 0.1 \end{aligned}$$

5. **Fourth training example** $x = [1, 1]$, $t = 1$:
   - **Compute output**: $y = f(0.1 \cdot 1 + 0.1 \cdot 1) = f(0.2) = 1$
   - Output matches target, so **no weight update needed**.

After one pass through the training data, the weights are $w_1 = 0.1$ and $w_2 = 0.1$. Repeating this process over multiple epochs will further refine the weights until the Perceptron correctly models the OR function.

Should the bias be updated if the output doesn't match the target?

In the Hebbian learning rule, the **bias term can also be updated similarly to the weights**. The bias can be treated as a weight connected to an input that is always 1. Therefore, if the output does not match the target, the bias should also be updated to help correct the output. The update rule for the bias b would be:

$$\Delta b = \eta \cdot x_0 \cdot t = \eta \cdot 1 \cdot t = \eta \cdot t \qquad x_0 = 1 \tag{8}$$

So, if the **output** is **incorrect**, the bias would be adjusted by adding Δb to the current bias value:

$$b^{(k+1)} = b^{(k)} + \Delta b = b^{(k)} + \eta \cdot t \tag{9}$$

This adjustment helps shift the activation threshold of the Perceptron, making it more likely to produce the correct output in future iterations.

In other words, we can think of the **bias** as a *special weight* w_0 that connects to a *constant input* $x_0 = 1$. This trick lets us treat the bias **exactly the same** as all the other weights in the update rule. Therefore, the neuron computes:

$$y = f\left(w_0 \cdot 1 + w_1 x_1 + w_2 x_2 + \dots\right)$$

And the update rule applies uniformly to **every** w_i, including w_0 (the bias):

$$w_i^{(k+1)} = w_i^{(k)} + \eta \cdot x_i \cdot t \quad \text{for } i = 0, 1, 2, \dots$$

Thus, at every iteration:

- The **normal weights** $(w_1, w_2, \dots)$ adapt based on the input features and target output.
- The **bias** b (also considered a weight, like w_0) is updated to help the Perceptron better fit the data. It adapts based on the target output t alone, since its associated input is always 1 (i.e., $x_0 = 1$).

The bias learns to **adjust the overall tendency** of the neuron to fire. If the network often needs to output 1 (positive target), the bias weight increases, making it easier for the neuron to activate. Conversely, if the network often needs to output 0 (negative target), the bias weight decreases, making it harder for the neuron to activate. This dynamic adjustment of the bias is crucial for the Perceptron to learn effectively from data.

2.2.4 Perceptron as Linear Classifier

A **classifier** is a model that assigns input data points to one of several classes. In the case of the perceptron, it classifies input vectors into two classes based on a linear decision boundary.

A **linear classifier** is a type of classifier that makes its decisions based on a linear combination of the input features. In poor words, it makes a decision by checking on which side of a *line* (in 2D), *plane* (in 3D), or *hyperplane* (in higher dimensions) the input data point lies.

The perceptron computes:

$$a = w^T x + b$$

where w is the weight vector, x is the input vector, and b is the bias term, and decides:

$$y = \begin{cases} 1 & \text{if } a > 0 \\ 0 & \text{if } a \leq 0 \end{cases} \tag{10}$$

So the **decision** happens depending on the *sign* of a: positive values lead to class 1, while zero or negative values lead to class 0.

The **Decision Boundary** is the exact set of points where the model is **undecided**, where it switches from one class to the other. That happens precisely when the condition changes sign from negative to positive. The "border" between those two cases is when the activation a equals **zero**. Formally, this occurs when:

$$w^T x + b = 0$$

That's where the perceptron's decision flips, and therefore it's the **boundary line (or hyperplane)**. This boundary divides the input space into two halves:

- Points where $w^T x + b > 0$ are classified as class 1.
- Points where $w^T x + b < 0$ are classified as class 0.
- Points where $w^T x + b = 0$ lie exactly on the decision boundary.

Wait, why is zero special? In the above equation (10), the perceptron outputs 0 when $a = 0$. Why is it called the decision boundary? In theory, the **boundary**:

$$w^T x + b = 0$$

Is **not assigned to any class**, it's the **limit** between them. Exactly on the boundary ($a = 0$), the model *is indifferent*, because **geometrically** that point is the **separator**, not really part of any region (see Figure 3, page 52, to visualize this concept). However, in practice, the $\leq$ sign in the perceptron decision rule is just a **tie-breaking rule**, otherwise we wouldn't know what to output when $a = 0$. But for geometry and theory, we're interested in **where the switch happens**, so we call the exact set of points the **decision boundary**.

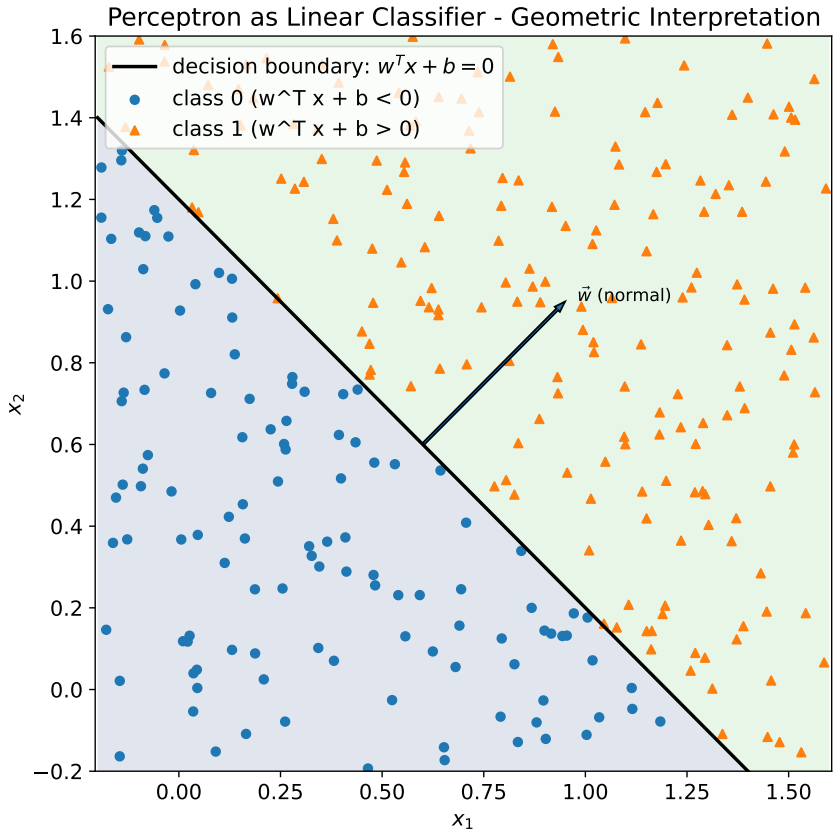


Figure 3: A 2D example of a perceptron as a linear classifier. The line represents the decision boundary where $w^T x + b = 0$. Points on one side of the line are classified as class 1 (green area, orange triangles, everything that satisfies $w^T x + b > 0$), while points on the other side are classified as class 0 (blue, $w^T x + b < 0$). The arrow indicates the **normal vector** $\vec{w}$, which is perpendicular to the decision boundary and points towards the class-1 side. The normal vector $\vec{w}$ points in the direction where the perceptron output increases.

Concept	**Meaning**
w	Defines the *direction* of the separating hyperplane.
b	Shifts the hyperplane from the origin.
$w^T x + b = 0$	Equation of the decision boundary (hyperplane).
$w^T x + b > 0$	Region classified as class 1.
$w^T x + b < 0$	Region classified as class 0.
$\vec{w}$	Normal vector to the decision boundary, indicating the direction of increasing output.
Limitation	Can only classify linearly separable data.

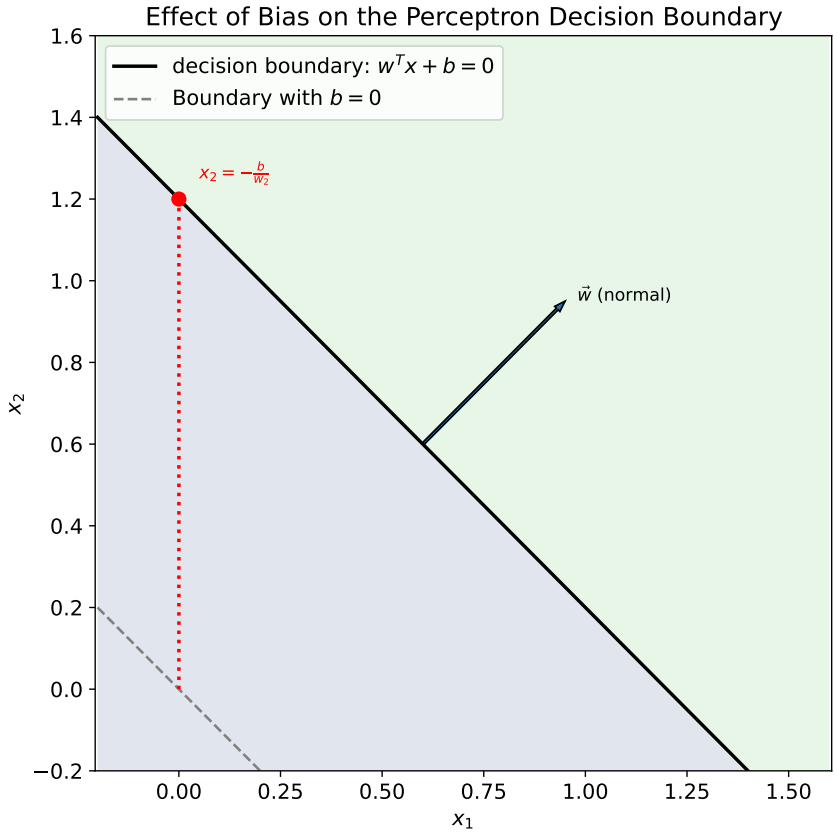


Figure 4: Geometric interpretation of the bias in a perceptron. The solid black line shows the decision boundary $w^T x + b = 0$ for $b = -1.2$, while the dashed gray line represents the case $b = 0$. The red dotted segment highlights the vertical shift of the intercept caused by the bias. The normal vector $\vec{w}$ is perpendicular to the boundary and points toward the region where the neuron output is 1 ($w^T x + b > 0$).

If the bias is negative, why does the boundary shift upwards? Imagine $w = [1, 1]$. Then $w^T x + b = x_1 + x_2 + b$. Without bias ($b = 0$), the boundary is:

$$x_1 + x_2 = 0$$

Is the **line through the origin** at a 45-degree angle. Now, if we **add** $b = -1.2$, the boundary becomes:

$$x_1 + x_2 - 1.2 = 0 \quad \Rightarrow \quad x_1 + x_2 = 1.2$$

This line is **shifted upwards** because for any given x_1, x_2 must be larger to satisfy the equation. Thus, a **negative bias** shifts the decision boundary **upwards**, while a **positive bias** would shift it **downwards**. In this case, for x_2 direction, the bias effectively **increases** the threshold that x_2 must reach to cross the boundary:

$$x_2 = -x_1 + 1.2$$

2.2.5 Boolean Operators & Linear Separability

Once we've seen that a perceptron can learn **logical functions** (like AND, OR), the next natural question is:

"*Can it learn **all** possible logical operators?*"

Short answer: **No**. And understanding why leads to the crucial idea of **linear separability**: the key limitation of the perceptron model.

Let's summarize the four fundamental binary logical functions (i.e., functions with two binary inputs and one binary output):

Operator	**Output** $= 1$ **when...**	**Linearly separable?**
AND	both inputs are 1	✔ Yes
OR	at least one input is 1	✔ Yes
NAND	at least one input is 0	✔ Yes
NOR	both inputs are 0	✔ Yes
XOR	exactly one input is 1	✖ No
XNOR	both inputs are the same	✖ No

Note that the first four operators (AND, OR, NAND, NOR) are all **linearly separable**, while the last two (XOR, XNOR) are **not**. But what does "linearly separable" mean in this context?

The game changer: *Linear Separability*

Definition 4: Linearly Separable

A dataset is **Linearly Separable** if there **exists** a straight line (in 2D), plane (in 3D), or **hyperplane** (in higher dimensions) that **perfectly divides** the **two classes of data points**. That is, all points of one class lie on one side, and all points of the other class lies on the opposite side.

Formally, given a dataset with two classes, it is linearly separable if there exist weights $w_1, w_2, \ldots, w_n$ and a bias b such that for every data point $(x_1, x_2, \ldots, x_n)$:

$$\begin{cases} w_1x_1 + w_2x_2 + \ldots + w_nx_n + b > 0 & \text{if the point belongs to Class 1} \\ w_1x_1 + w_2x_2 + \ldots + w_nx_n + b < 0 & \text{if the point belongs to Class 2} \end{cases}$$

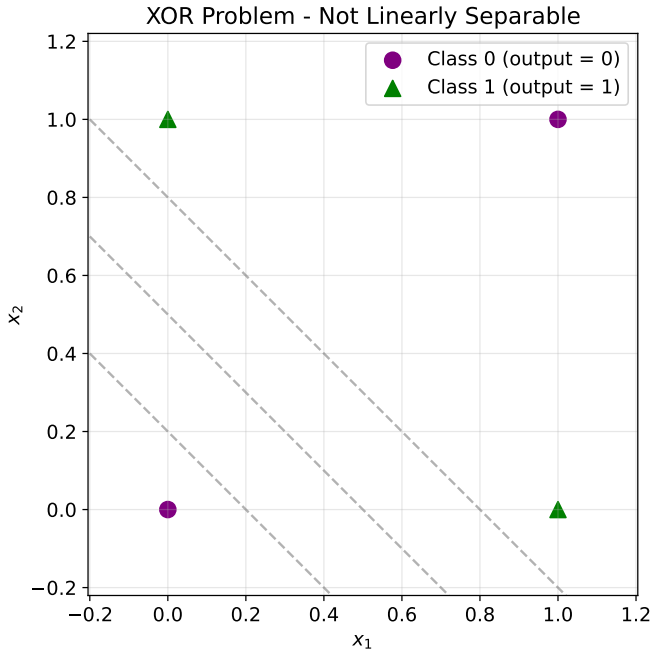
If such weights and bias exist, the dataset is linearly separable. Otherwise, no single perceptron can solve it (i.e., classify it correctly).

⚠ The XOR problem - The classic example of non-linear separability

Until now, we've seen that perceptrons can learn linearly separable functions like AND and OR. However, the linear separability limitation becomes evident when we consider some logical functions, such as XOR (exclusive OR). A little reminder of the XOR truth table:

x_1	x_2	$\text{XOR}(x_1, x_2)$
0	0	0
0	1	1
1	0	1
1	1	0

The XOR function outputs 1 only when exactly one of its inputs is 1. If we plot the input-output pairs of the XOR function on a 2D plane, we get the following points:



Here, the points are arranged in an "X" pattern:

- Class 1 points are at (0, 1) and (1, 0) (opposite corners).
- Class 0 points are at (0, 0) and (1, 1) (remaining corners).

No single straight line can separate the Class 1 points from the Class 0 points. We'd need *two lines* forming a region (a non-linear boundary). Hence, the XOR function is **not linearly separable**, and a single-layer perceptron cannot learn it.

In summary, the perceptron can only create **linear decision boundaries**, so:

- ✅ It perfectly models **linearly separable** problems (like AND, OR, simple threshold rules).
- ❌ If fails for **non-linearly separable** problems (like XOR, parity, circle-vs-ring, etc.).

This realization in the 1960s led to what's often called the "**AI winter**," as researchers recognized the limitations of single-layer perceptrons. However, this challenge also paved the way for the development of **multi-layer neural networks** (and backpropagation), which can overcome these limitations by creating complex, non-linear decision boundaries, combining multiple perceptrons in layers.

2.3 Feed-Forward Neural Networks (FNNs)

2.3.1 Architecture

After discovering that a single perceptron can only draw **one straight boundary**, researchers realized that we need **multiple layers** of neurons to build **non-linear decision surfaces**. That's how **Feed-Forward Neural Networks (FNNs)** were born.

Definition 5: Feed-Forward Neural Networks (FNNs)

A **Feed-Forward Neural Network (FNN)** is an artificial neural network where information **flows in one direction only**, from the input layer, through any hidden layers, to the output layer. There are no cycles or loops in the network.

$$x \rightarrow \text{Layer 1} \rightarrow \text{Layer 2} \rightarrow \ldots \rightarrow \text{Output Layer}$$

Each layer receives signals from the previous layer, processes them using weighted connections and activation functions, and passes the output to the next layer.

Structure of a Feed-Forward Network

An FNN is composed of:

1. **Input Layer**: one neuron per input feature (e.g., pixel value, sensor reading). Does not perform computation, it simply distributes inputs to the next layer.
2. **Hidden Layer(s)**: Contain neurons that each compute:

   $$a_j = f\left(w_j^T x + b_j\right)$$

   where w_j are the weights, b_j is the bias, x is the input vector from the previous layer, and f is the nonlinear activation function (sigmoid, tanh, ReLu, etc.). The index j identifies the specific neuron in the hidden layer. Each neuron learns different **intermediate features** of the data.
3. **Output Layer**: Produces the network's final result. Activation depends on the task, we mean for classification we often use **softmax** or **sigmoid**, while for regression we use a **linear** activation.

The connections between neurons are **weighted**:

- Each neuron in layer l is connected to **all** neurons in the previous layer $l-1$. This is called a **fully connected** or **dense** layer.
- Every connection has its own **weight**, which is learned during training. Every neuron also has a own **bias** term.
- During training, all these weights and biases are adjusted to minimize the difference between the predicted output and the actual target values.

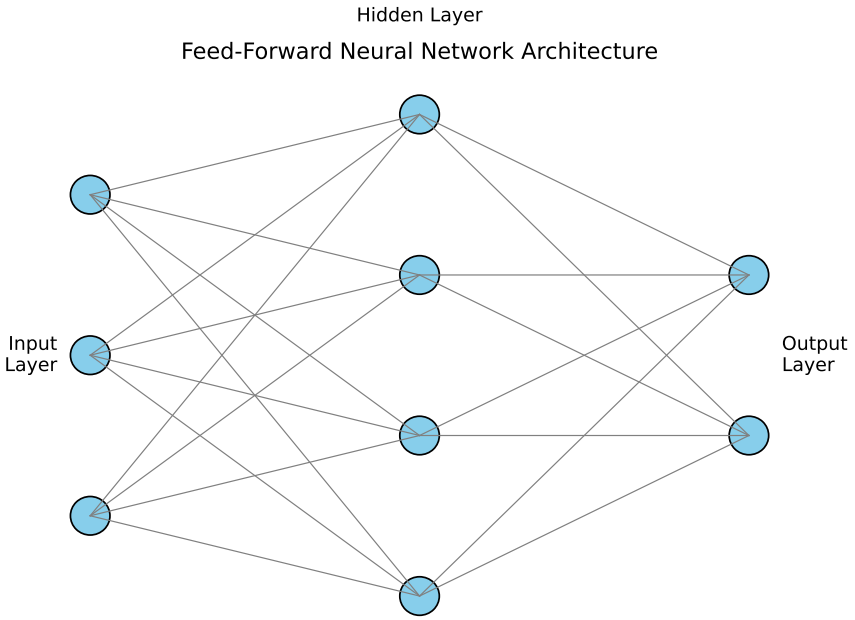


Figure 5: Architecture of a simple feed-forward neural network. Each layer is fully connected to the next one. Signals flow in one direction (input → hidden → output) without feedback connections.

FNNs can be represented as graphs based on this architecture:

- **Nodes** represent neurons.
- **Edges** represent weighted connections between neurons.

This graph representation helps visualize the network's structure and understand how information propagates through it.

Mathematically, for a layer l:

$$\begin{cases} a^{(l)} = f\left(W^{(l)}a^{(l-1)} + b^{(l)}\right) & \text{for hidden layers} \\ x^{(l)} = f\left(a^{(l)}\right) & \text{for output layer} \end{cases}$$

where:

- $a^{(l)}$ is the activation vector of layer l.
- $W^{(l)}$ is the weight matrix connecting layer $l-1$ to layer l.
- $b^{(l)}$ is the bias vector for layer l.
- f is the activation function.
- $x^{(l)}$ is the final output of the network.

This formalism allows us to **compute the output of the network given an input vector by sequentially applying these transformations layer by layer**.

How FNNs learn hierarchical features

Adding layers lets the network learn **hierarchical representations**:

- The first layers capture **simple patterns** (e.g., edges in images).
- Deeper layers combine these simple patterns into **more complex features** (e.g., shapes, objects).

This ability to build abstractions through depth is the essence of **Deep Learning**.

2.3.2 Activation Functions

Every neuron computes a **weighted sum** of its inputs and bias, called **Net Input** or **Activation Potential**:

$$a = w^T x + b \tag{11}$$

Up to this point, everything is **linear**. If we stopped here and used $y = a$ as the output, the neuron would just perform a *linear transformation*. So, we need to add some **non-linearity** to the neuron's output (a sort of *magic ingredient*) to allow the network to learn complex patterns. Without it, the entire neural network would collapse into a single linear operation: a big matrix multiplication. Remember that we come from the perceptron, which used a step function as activation and complex patterns, like XOR, could not be learned without non-linearity.

So we introduce the concept of **Activation Function**. A neuron takes the net input a computed above, and then applies an **activation function** $f(a)$ to produce its final output:

$$y = f(a)$$

The **Activation Function** defines **how the neuron "fires"**, i.e., how it transforms the raw input signal into an output that will be passed to the next layer.

Why do we really need activation functions?

Activation functions are what give neural networks their power and flexibility. Three main reasons why they are essential:

1. **To *break linearity***. Without it, the entire neural network would collapse into a single linear operation. Mathematically:

   $$\text{If } g(a) = a, \text{ then } g(W^{(2)} g(W^{(1)} x)) = W^{(2)} W^{(1)} x$$

   This is *still linear*, no matter how many layers we stack. We'd just have one big matrix multiplication. So, without $g(\cdot)$, the network couldn't model *curves*, *XOR*, *images*, *language*, or anything nonlinear.

2. **To allow *complex decision boundaries***. Think of a perceptron. With only linear operations, it can only separate data using a straight line (or plane, or hyperplane). By inserting a nonlinear $g(a)$, hidden neurons can **bend** the space (combine multiple lines to form curved boundaries). For example, a simple linear neuron can only separate classes with a single line, while nonlinear activations (like sigmoid or ReLU) allow the network to create complex, curved, multi-region separations.

3. **To *control output range***. Activation functions can also **squash** values:

Function	Formula	Output Range	Typical Use
Sigmoid	$\frac{1}{1+e^{-a}}$	$(0, 1)$	Binary classification output layers.
Tanh	$\frac{e^{a}-e^{-a}}{e^{a}+e^{-a}}$	$(-1, 1)$	Hidden layers in some networks.
Linear	a	$(-\infty, +\infty)$	Regression output layers.

That's why we use sigmoid/tanh in hidden layers or output layers for classification (to get probabilities between 0 and 1), while linear activations are used in regression tasks (to allow any real-valued output).

Exist many different activation functions, each with its own characteristics and use cases. The choice of activation function can significantly impact the performance and capabilities of a neural network. In the following, we will explore some of the most commonly used activation functions in neural networks.

In the next sections, we will mention the derivative result and the range of each activation function:

- During training, neural networks learn by **minimizing a loss function**, and this requires **backpropagation**, which is based entirely on **derivatives**. We will explain backpropagation later, but for now, here is a brief overview of how it works: (1) each neuron has parameters w_i (weights) and b (bias); (2) to adjust them, we compute how the **loss** changes if we slightly change each parameter; (3) mathematically, that's done through **gradients**, the derivatives of the loss with respect to the weights. When we apply the **chain rule** to compute these gradients, we get something like:

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial a} \cdot \frac{\partial a}{\partial w_i}$$

  Where $\frac{\partial y}{\partial a}$ is the derivative of the activation function $f(a)$. Therefore, having an activation function: **too small** $(= 0)$, means gradients vanish and the learning stops; **too large** gradients can cause instability. Hence, the derivative of the activation function is crucial for effective learning.

- The **range** of an activation determines what kind of outputs each neuron can produce, and this affects: (1) **how the next layer receives data**, and **how easy it is to train** the network. For example, if an activation function outputs values in a limited range (like between 0 and 1), it can help keep the network's outputs stable and prevent extreme values that could lead to numerical issues during training.

2.3.2.1 Linear

The **Linear Activation Function** is the simplest activation function, defined as:

$$f(a) = a \tag{12}$$

That means the neuron's output equals its input; there's no distortion or thresholding. So the neuron is just a **weighted sum** followed by nothing.

Intuitive interpretation

If all neurons in a network use $f(a) = a$, then every layer just performs a **linear transformation** of the input. Stacking **multiple linear layers doesn't add any expressive power**; the entire **network can be reduced to a single linear transformation**. In general, for a network with n layers, each represented by a weight matrix W_i, the overall transformation is:

$$f\left(W_n\left(W_{n-1}\left(\ldots W_2\left(W_1 x\right)\ldots\right)\right)\right) = \left(W_n W_{n-1}\ldots W_2 W_1\right) x$$

However, if the network only uses linear activation functions, then it simplifies to:

$$f\left(W_2\left(W_1 x\right)\right) = \left(W_2 W_1\right) x$$

Therefore, linear activation functions are rarely used in practice for hidden layers, as they cannot capture complex patterns in data. In other words, a purely linear network cannot learn anything more complex than a straight boundary; it is basically a big matrix multiplication because all the layers collapse into one.

Property	Description
Formula	$f(a) = a$
Derivative	$f'(a) = 1$
Range	$(-\infty, +\infty)$
Nonlinear?	✖
Typical use	Regression output layers (not hidden neurons)

When to use it

Even though a **linear activation** is useless inside hidden layers (because it doesn't add nonlinearity), it's still **important at the output layer** of certain models:

- **Regression problems**: when we want a real-valued output (like predicting house prices), a linear activation allows the network to produce any value in the range $(-\infty, +\infty)$. However, the linear activation is applied only at the output layer, while hidden layers use nonlinear activations to capture complex patterns.
- **Autoencoders or embedding layers**: sometimes the linear activation helps maintain continuous representations of data.

In summary, the **linear activation** keeps the output proportional to the input. It's mathematically simple and differentiable, but **does not allow the network to model nonlinear relationships**. Hence, not used in hidden layers.

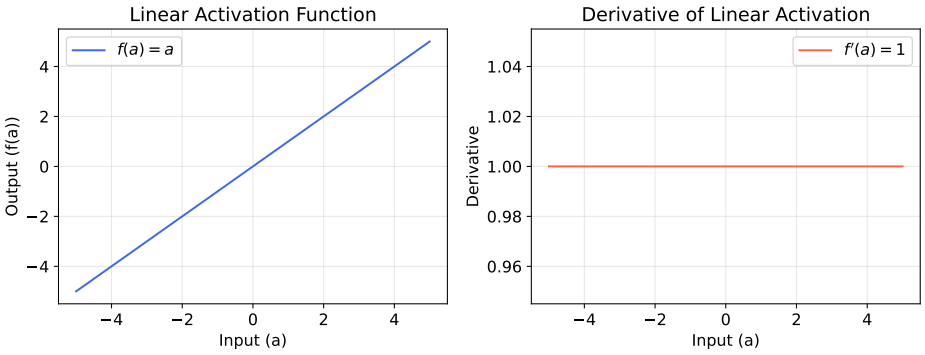


Figure 6: Linear activation $f(a) = a$ and its derivative. The function is the identity (a straight line, showing that the neuron outputs exactly what it receives), and its constant derivative $f'(a) = 1$ allows perfect gradient flow. However, being linear, it adds no expressive power to the network.

2.3.2.2 Sigmoid

The **Sigmoid Activation Function** (or **Logistic Activation Function**) is defined as:

$$f(a) = \frac{1}{1 + e^{-a}} \tag{13}$$

It "**squashes**" any real-valued **input** a **into a range** between **0 and 1**.

The Sigmoid converts its input into something that looks like a **smooth threshold**:

- ⇈ Large positive inputs a produce outputs close to 1;
- ⇊ Large negative inputs a produce outputs close to 0;
- ≈ Inputs a close to 0 produce outputs close to 0.5

It's often described as giving a "**firing probability**" to a neuron, mimicking how biological neurons activate gradually rather than with a hard step.

Graphically, the Sigmoid function is a smooth **S-shaped** curve (sigmoidal). It's **continuous** and **differentiable everywhere**. It has a gentle slope around 0 and saturates near the extremes (0 or 1).

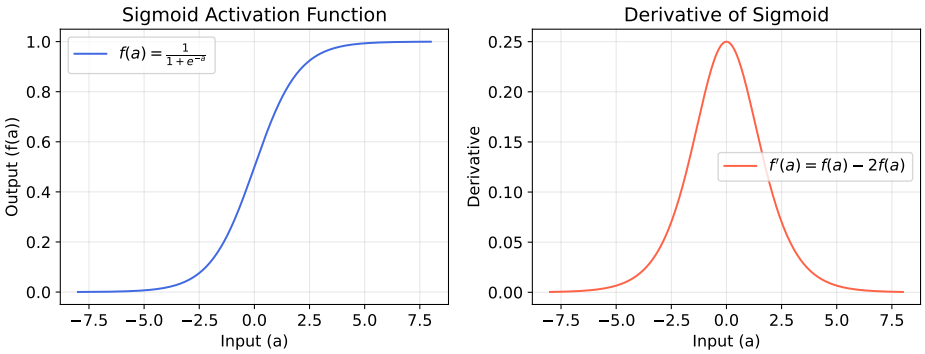


Figure 7: Sigmoid Activation Function and its derivative. The sigmoid introduces smooth nonlinearity and maps inputs into $(0, 1)$, but its derivative vanishes for large inputs, causing slow learning in deep networks.

The **derivative** tells us how sensitive the neuron's output is to changes in its input. For the Sigmoid function, the derivative is given by:

$$f'(a) = f(a) - 2f(a) = f(a) \cdot [1 - f(a)] \tag{14}$$

This means:

- When $f(a) \approx 0.5$, the derivative is maximized at 0.25, allowing for significant weight updates during training (**neuron is responsive**).
- When $f(a) \approx 0$ or $f(a) \approx 1$, the derivative approaches 0, leading to very small weight updates (**neuron is saturated** and gradients vanish).

  This **vanish gradient problem** makes deep networks with Sigmoid activations **hard to train**, as gradients become very small in earlier layers

during backpropagation. In other words, the Sigmoid function can cause **slow learning** in deep networks due to its saturating behavior at extreme input values.

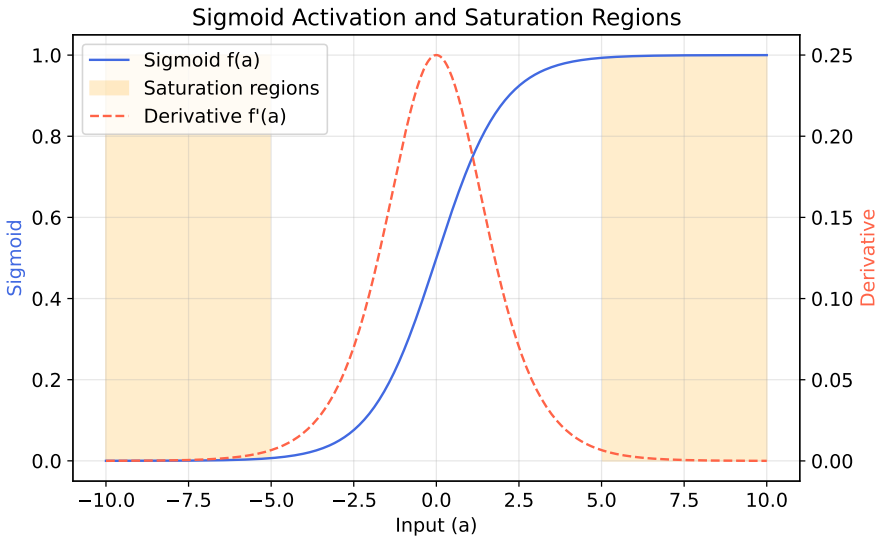


Figure 8: Sigmoid activation and saturation regions.

In figure 8 we can see the **vanish gradient problem** in action: when neurons saturate, their gradients vanish, making it hard for the network to learn from data during training.

- The blue curve shows the **sigmoid activation** $f(a)$. That smooth **S-shaped curve** (in blue) represents:

$$f(a) = \frac{1}{1 + e^{-a}}$$

  In the center (around $a = 0$), the output is about 0.5, and the curve is **steepest**; for large positive $a > 5$, the curve **flattens near 1**; for large negative $a < -5$, it **flattens near 0**. Those flat tails are the **saturation regions** (highlighted in orange). These regions mean that when the neuron receives very strong positive or negative inputs, its output doesn't change much anymore; it has reached its "max" or "min" activation.

- The **orange shaded regions** highlight the parts of the sigmoid where the output is **almost constant**:
  - On the left (for $a < -5$), the shaded area fills between the sigmoid and 1, where the output is very close to 0 (saturated low);
  - On the right (for $a > 5$), the shaded area fills between the sigmoid and 0, where the output is very close to 1 (saturated high).

  In those regions:

$$\frac{\partial f}{\partial a} = f'(a) \approx 0$$

So the neuron has **stopped responding**: even big changes in a cause almost no change in the output $f(a)$.

- The red dashed curve shows the **derivative** $f'(a)$ (plotted on the **right y-axis** with scale 0 to 0.25):

$$f'(a) = f(a) \cdot (1 - f(a))$$

The derivative is only significant in a **small central region** (roughly between -3 and 3), where it reaches a maximum of 0.25 at $a = 0$. Outside this range, the derivative **drops to near zero**, indicating that the neuron is **saturated** and **not learning effectively**. Notice how the derivative vanishes precisely in the orange saturation regions.

Property	**Value / Meaning**
Formula	$f(a) = \dfrac{1}{1+e^{-a}}$
Range	$(0, 1)$
Derivative	$f'(a) = f(a) \cdot (1 - f(a))$
Output interpretation	Probability or "firing strength".
Pros	Smooth, differentiable, bounded output, probabilistic interpretation.
Cons	Vanishing gradients for large a, outputs not zero-centered, computationally expensive.

When to use it

- **Output layer of binary classification** networks, where outputs represent probabilities:

$$\mathbb{P}(y = 1 \mid \mathbf{x}) = f(w^T \mathbf{x} + b)$$

- Historically used in hidden layers (in early networks), but now often replaced by ReLU or its variants due to vanishing gradient issues.

In summary, the Sigmoid activation function is essentially a **soft version** of the perceptron's step function:

$$\text{step: } f(a) = \begin{cases} 1 & \text{if } a \geq 0 \\ 0 & \text{if } a < 0 \end{cases} \quad \longrightarrow \quad \text{sigmoid: } f(a) = \frac{1}{1+e^{-a}}$$

So the sigmoid allowed neural networks to become **differentiable**, which made **gradient-based learning (backpropagation)** possible. However, its tendency to **saturate** and cause **vanishing gradients** has led to the adoption of alternative activation functions (like ReLU) in modern deep learning architectures.

2.3.2.3 Hyperbolic Tangent (tanh)

The **Hyperbolic Tangent (tanh) Activation Function**, commonly known as **tanh**, is defined as:

$$f(a) = \tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}} \tag{15}$$

And its derivative is:

$$f'(a) = 1 - \tanh^2(a) = 1 - f^2(a) \tag{16}$$

The tanh function maps input values to an output range between -1 and 1. It is a scaled version of the sigmoid function, centered around zero.

The tanh activation function looks **very similar to the sigmoid**, but it is **symmetric around zero**: outputs range from -1 to 1, which helps in centering the data and can lead to faster convergence during training. This makes it **zero-centered**, which is a big advantage.

- When the input is zero ($a = 0$), the output is also zero ($f(0) = 0$).
- For large positive inputs, the output approaches 1 ($f(a) \to 1$ as $a \to +\infty$).
- For large negative inputs, the output approaches -1 ($f(a) \to -1$ as $a \to -\infty$).

That means hidden neurons can have both positive and negative activations, which helps later layers learn faster because the data stays **balanced** around zero.

Why it's better than sigmoid

- **Range**: The tanh function outputs values between -1 and 1, while the sigmoid function outputs values between 0 and 1. This means that tanh is zero-centered, which can help with convergence during training.
- **Gradient around zero**: The derivative of the tanh function is ≈ 1 around zero, while the derivative of the sigmoid function is ≈ 0.25 around zero. This means that the tanh function has a steeper gradient[4] around zero, which can help with learning.
- **Saturates?** Both functions can saturate for large positive or negative inputs, leading to the vanishing gradient problem. However, because tanh is zero-centered, it can help mitigate this issue to some extent.
- **Training speed**: In practice, models using the tanh activation function often converge faster than those using the sigmoid function, especially in deep networks.

[4] A "steeper gradient" means that small changes in the input lead to larger changes in the output, which can help the model learn more effectively.

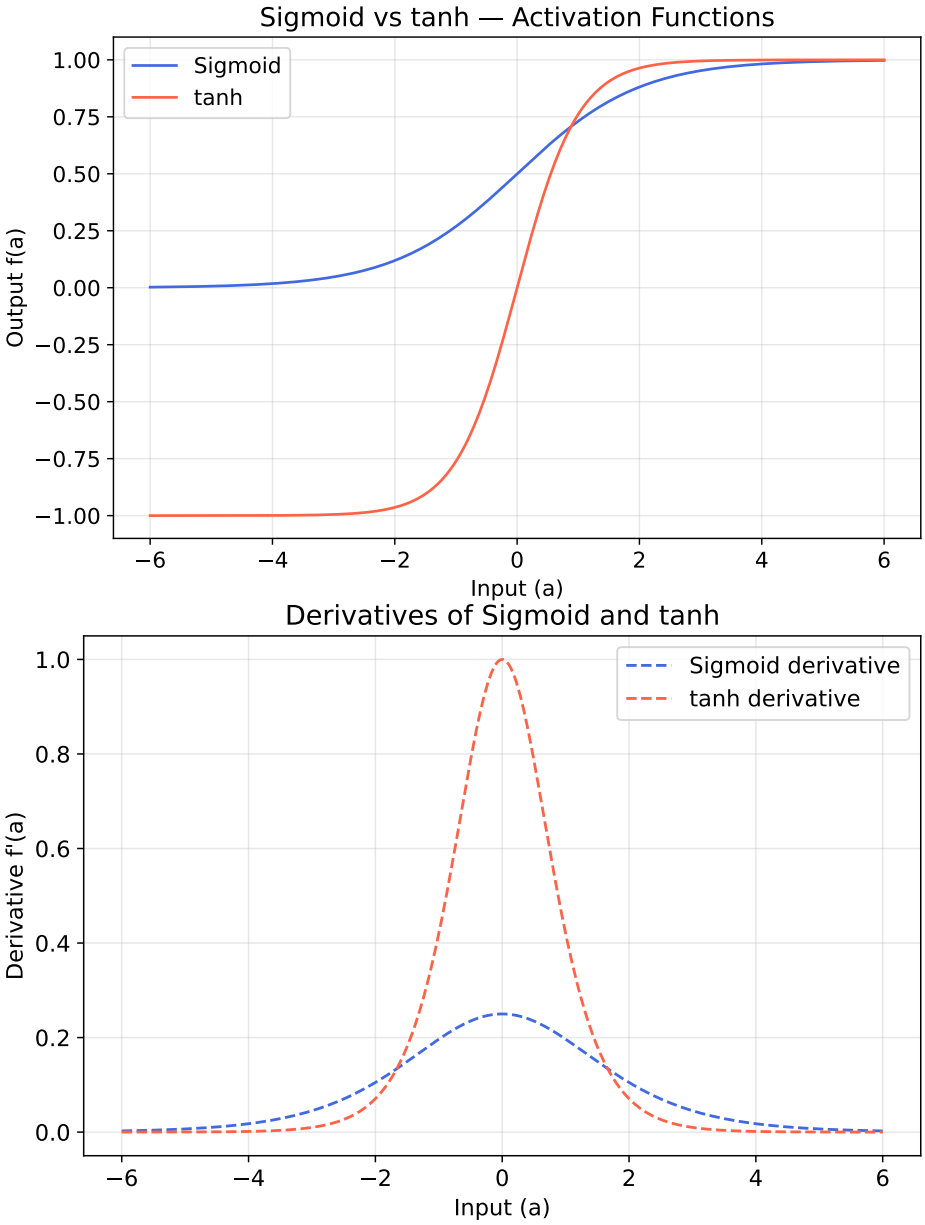


Figure 9: Hyperbolic Tangent (tanh) Activation Function and its Derivative.

Deepening: Why tanh can lead to faster convergence than sigmoid?

The **main reason** tanh can lead to faster convergence than sigmoid is that **tanh is zero-centered**. Now, *why does being zero-centered help?* The reason is related to how gradients work during backpropagation, i.e., how the model learns from data. We will explain the backprop technique in detail in future sections. For now, consider a neuron:

$$z = w^T x + b$$

During gradient descent (i.e., the learning process), we compute:

$$\frac{\partial L}{\partial w} = \delta x \quad \text{where} \quad \delta = \frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \cdot f'(z)$$

That means the weight update depends on the product of δ and x. If x (the input) **is always positive** (sigmoid case), gradients tend to have the **same sign**, and the weights are updated **mostly in the same direction**. In contrast, if x can be **positive or negative** (tanh case), gradients can have **different signs**, allowing for more balanced updates to the weights.

Furthermore, the **derivative of tanh is larger around zero** compared to sigmoid, which means that **when the input is near zero, the model can learn more effectively** because the gradients are stronger (i.e., the model can make larger updates to the weights, because the output changes more significantly with small changes in input).

In summary, models using **tanh** often converge faster than those using **sigmoid** because thanh outputs are **zero-centered**, which leads to more balanced gradients and more efficient optimization. Additionally, the derivative of tanh can reach **1**, while the sigmoid derivative is at most **0.25**, allowing for stronger gradients and faster learning when the input is near zero.

The **zero-centered** output means activations can cancel each other out more easily, so the network doesn't get a constant "positive bias" in its gradient (a problem with sigmoid). This often leads to **faster convergence** during training.

When to use tanh

The tanh activation function is often preferred over the sigmoid function in hidden layers of neural networks, especially when the data is centered around zero. It is particularly useful in scenarios where:

- The **input** data is **normalized** to have a **mean of zero**.
- The **model requires faster convergence** during training.
- The **network is deep**, and the benefits of zero-centered activations help mitigate issues like vanishing gradients.

However, it's important to note that while tanh can be advantageous in many situations, it **still suffers from the vanishing gradient problem for very large or very small input values**. Therefore, in very deep networks, other activation functions like ReLU (Rectified Linear Unit) are often preferred.

2.3.3 Output Layer

The **output layer** is the *last* layer of the network, the one that produces the model's **final prediction**. Up to this point, the **hidden layers** have been learning to extract useful features (patterns, relationships, hierarchies). But the **output layer** translates all of that internal representation into the final, human-meaningful result. For example:

- A **continuous number** (e.g., house price) in **regression tasks** (e.g., predicting a numerical value).
- Or a **class label** (e.g., cat vs. dog) in **classification tasks** (e.g., categorizing images).

So, the **choice of activation function** in the output layer depends on the **type of output we want**. Exist several options:

- For **regression tasks**, where we want to predict a continuous value, we often use a **linear activation function** (or no activation function at all) in the output layer. This allows the network to produce a wide range of values.
- For **binary classification tasks**, where we want to classify inputs into two classes, we typically use the **sigmoid activation function** in the output layer. This squashes the output to a value between 0 and 1, which can be interpreted as a probability.
- For **multi-class classification tasks**, where we want to classify inputs into more than two classes, we often use the **softmax activation function** in the output layer. This produces a probability distribution over the classes, ensuring that the sum of the outputs equals 1.

The **design of the output layer** is crucial because it directly affects how well the network can perform its intended task. Choosing the appropriate activation function and structure for the output layer ensures that the network's predictions are meaningful and useful for the specific problem at hand.

2.3.3.1 Regression

In **regression problems**, we want the network to predict **a real-valued quantity**, something that can take *any* number, positive or negative. For example, predicting the price of a house based on its features (size, location, number of rooms, etc.) is a regression task. In this case, the **output layer** of the neural network typically consists of a **single neuron** that produces a **continuous output**, not categorical labels (we want a number, not a class like "expensive" or "cheap").

The output function

For regression tasks, we don't want to limit or distort the network's output. Therefore, the last layer simply uses a **linear activation** (page 62):

$$f(a) = a \quad \text{or equivalently} \quad y = w^T x + b$$

This means the output neuron just returns the raw weighted sum of its inputs, no squashing or thresholding.

If we used a **sigmoid** or **tanh** activation in the output layer, the output would be forced into $(0, 1)$ or $(-1, 1)$ ranges, respectively. This would be problematic for regression tasks where the target variable can take on a wide range of values. For example, if we're predicting house prices, we want the output to be able to represent any price, not just values between 0 and 1 (e.g., a house could cost $\$250,000$, which is far outside the range of a sigmoid output, or a temperature could be -10 degrees Celsius, which is outside the range of tanh). The **linear** activation allows any real number to be output, making it suitable for regression tasks.

Typical network setup for regression

A typical neural network for regression tasks has the following structure:

Component	Example
Hidden layers	Several, with nonlinear activations (e.g., ReLU, tanh).
Output layer	One neuron (for single output) with **linear activation**.
Loss function	**Mean Squared Error (MSE)** or **Mean Absolute Error (MAE)**.

Deepening: Mean Squared Error (MSE)

When our network predicts continuous values (like prices, temperatures, voltages, etc.), we need a way to measure **how far the predictions are from the real targets**. That's what a **loss function** does: it quantifies the prediction error. The **Mean Squared Error (MSE)** is the most

common one for regression tasks:

$$\text{MSE} = \frac{1}{N} \cdot \sum_{i=1}^{N} (y_i - t_i)^2 \tag{17}$$

Where:

- N is the number of data points (samples).
- y_i is the predicted value for the i-th sample.
- t_i is the true (target) value for the i-th sample.

The term $(y_i - t_i)^2$ is the **error** (difference between prediction and truth), and we **square** it to make all errors positive (avoid cancellation) and to penalize **larger mistakes more strongly** (e.g., an error of 10 counts 100 times more than an error of 1). Then we **average** over all samples to get the mean error per prediction.

So MSE measures how "spread out" our predictions are around the true values. A **lower MSE** means our model is doing a better job at predicting the continuous target variable.

Example 4: Example of MSE calculation

Suppose we have the following regression model:

Sample	t_i	y_i	$y_i - t_i$	Squared Error
1	2	3	+1	1
2	5	4	−1	1
3	6	8	+2	4
4	3	2	−1	1

To compute the MSE:

$$\text{MSE} = \frac{1}{4} \cdot (1 + 1 + 4 + 1) = \frac{1}{4} \cdot 7 = 1.75$$

So the Mean Squared Error for this model is 1.75, indicating the average squared difference between the predicted and true values.

About derivations, it is important to note that MSE is **differentiable**, which is crucial for training neural networks using gradient-based optimization methods (we will cover this in detail later). The derivative of MSE with respect to the predictions y_i is:

$$\frac{\partial \,\text{MSE}}{\partial \, y_i} = \frac{2}{N} \cdot (y_i - t_i) \tag{18}$$

So, the weight updates are proportional to how wrong each prediction is. It means, large errors produce larger gradients, leading to bigger

adjustments in the weights during training, which helps the model learn more effectively.

In summary, MSE tells us **how far off our predictions are on average**. It's like saying "*how wrong am I, squared and averaged?*" The squaring heavily punishes big mistakes, making MSE ideal when we care about precision in regression tasks.

Deepening: Mean Absolute Error (MAE)

The **Mean Absolute Error** measures the **average absolute distance** between predicted and true values:

$$\text{MAE} = \frac{1}{N} \cdot \sum_{i=1}^{N} |y_i - t_i| \tag{19}$$

Where:

- N is the number of data points (samples).
- y_i is the predicted value for the i-th sample.
- t_i is the true (target) value for the i-th sample.

Unlike MSE, which *squares* the difference, MAE simply takes the **absolute value** of the error. That means:

- Every error contributes proportionally to its magnitude (no squaring).
- Large errors don't explode quadratically, they contribute linearly.

So MAE measures the **average size of the mistakes**, regardless of direction.

Example 5: Example of MAE calculation

Using the same predictions as before (from Example on page 73), to compute the MAE:

$$\text{MAE} = \frac{1}{4} \cdot (1 + 1 + 2 + 1) = \frac{1}{4} \cdot 5 = 1.25$$

So the Mean Absolute Error for this model is 1.25, indicating the average absolute difference between the predicted and true values. Compared to MSE, MAE gives a more direct sense of the average error magnitude without squaring.

Regarding derivations, the MAE is **not differentiable** at points where the prediction equals the target (i.e., $y_i = t_i$) because of the absolute

value function. However, we can use the **subgradient** for optimization:

$$\frac{\partial |x|}{\partial x} = \begin{cases} +1 & \text{if } x > 0 \\ -1 & \text{if } x < 0 \\ \text{undefined (but taken as 0)} & \text{if } x = 0 \end{cases} \tag{20}$$

So gradient updates from MAE are **constant in magnitude**. They don't depend on how far the prediction is from the truth. That's why MAE can converge slower but more robustly, especially in the presence of outliers (which can heavily skew MSE).

In summary, MAE tells us **how many units off my predictions are on average**, while MSE punishes larger errors more severely:

- **MSE** tries to **minimize variance**, forces the model to avoid large mistakes aggressively.
- **MAE** tries to **minimize average error**, focuses on overall robustness.

If our data has **outliers** (e.g., occasional very wrong samples), MAE is less distorted by them because it doesn't exaggerate their impact.

2.3.3.2 Binary Classification

In **binary classification**, the task is to decide **two possible outcomes**, for example *spam* vs *not spam* in email filtering, or *disease* vs *no disease* in medical diagnosis. The **output** of the network is typically a single neuron that produces a value between 0 and 1, representing the **probability** of one of the classes:

$$\mathcal{P}(y = 1 \mid x) \in [0, 1]$$

That is, "*how likely is this input to belong to class 1?*" The other class's probability can be derived as:

$$\mathcal{P}(y = 0 \mid x) = 1 - \mathcal{P}(y = 1 \mid x)$$

The output function

At the output layer, we typically have:

- **1 neuron**, because we only need one value (the probability of class 1).
- The **activation function** is usually the **sigmoid function** (or sometimes the **tanh function**), which maps any real-valued number into the range (0, 1), making it suitable for probability estimation.

The **sigmoid function** is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Or the **tanh function**:

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

Despite tanh outputting values in the range $(-1, 1)$, it can be scaled to (0, 1) for probability interpretation:

- $f(a) > 0 \Rightarrow$ class 1
- $f(a) < 0 \Rightarrow$ class 0

It's sometimes preferred due to its zero-centered output, which can help with optimization. However, the **sigmoid** function is **more commonly used** in binary classification tasks.

Typical network setup for binary classification

Component	Example
Hidden layers	Several, with nonlinear activations (e.g., ReLU, tanh).
Output layer	One neuron (for single output) with **sigmoid activation**.
Loss function	**Binary Cross-Entropy (log loss)**.

Deepening: Binary Cross-Entropy (BCE, Log Loss)

The goal in **binary classification** is to predict the *probability* that an input belongs to class 1, given by our network's sigmoid output:

$$\hat{y} = f(a) = \frac{1}{1 + e^{-a}} \quad \in (0, 1)$$

The true label t is:

- 1 if the sample belongs to class 1,
- 0 if it belongs to class 0.

The **Binary Cross-Entropy (BCE, Log Loss)** loss function measures the difference between the predicted probabilities $\hat{y}$ and the true labels t. It is defined as:

$$\text{BCE}\,(t, \hat{y}) = L = -\frac{1}{N} \cdot \sum_{i=1}^{N} \left[t_i \cdot \ln\left(\hat{y}_i\right) + (1 - t_i) \cdot \ln\left(1 - \hat{y}_i\right) \right] \tag{21}$$

Let's understand each term:

- N is the number of samples in the dataset.
- t_i is the true label for sample i (0 or 1, since it's binary classification).
- $\hat{y}_i$ is the predicted probability for sample i (output of the sigmoid).
- $\ln\left(\hat{y}_i\right)$ **penalizes** the model when it **predicts a low probability for the true class** (when $t_i = 1$).
- $\ln\left(1 - \hat{y}_i\right)$ **penalizes** the model when it **predicts a high probability for the false class** (when $t_i = 0$).
- If the true label is 1 ($t_i = 1$), the loss simplifies to $-\ln\left(\hat{y}_i\right)$. The model is **penalized** when it predicts a **small** probability for class 1.
- If the true label is 0 ($t_i = 0$), the loss simplifies to $-\ln\left(1 - \hat{y}_i\right)$. The model is **penalized** when it predicts a **large** probability for class 1.

So, **the closer the prediction is to the truth, the smaller the loss**.

Example 6: BCE Calculation Example

Let's consider a simple example with 4 samples:

Sample	True Label (t)	Predicted Probability ($\hat{y}$)
1	1	0.9
2	0	0.2
3	1	0.4
4	0	0.6

Now, we calculate the BCE loss for each sample:

- Sample 1: $[1 \cdot \ln(0.9) + (1-1) \cdot \ln(1-0.9)] = \ln(0.9) \approx -0.105$
- Sample 2: $[0 \cdot \ln(0.2) + (1-0) \cdot \ln(1-0.2)] = \ln(0.8) \approx -0.223$
- Sample 3: $[1 \cdot \ln(0.4) + (1-1) \cdot \ln(1-0.4)] = \ln(0.4) \approx -0.916$
- Sample 4: $[0 \cdot \ln(0.6) + (1-0) \cdot \ln(1-0.6)] = \ln(0.4) \approx -0.916$

Finally, we compute the average BCE loss over all samples:

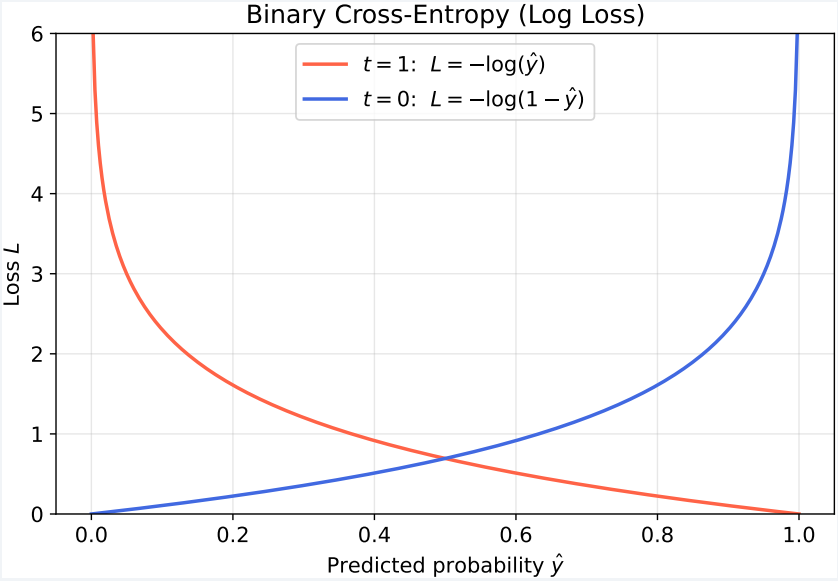
$$L = -\frac{1}{4}(-0.105 - 0.223 - 0.916 - 0.916) = -\frac{-2.16}{4} = 0.54$$

So, the BCE loss for this example is approximately **0.54**. This indicates that the model's predictions are not very accurate, as a lower loss value indicates better performance.

Cross-Entropy comes from **information theory**. It measures the **difference between two probability distributions**:

- The true distributions of the labels (0 or 1).
- The predicted distributions from the model (probabilities between 0 and 1).

Minimizing BCE is equivalent to **maximizing the likelihood** of our data under the model's predictions. So we're training the network to output probabilities that match the true labels as closely as possible.



The loss is **asymmetric**: wrong confident predictions get punished exponentially.

- Red curve ($t = 1$): Loss is low when predicted probability $\hat{y}$ is close to 1 (correct and confident), and high when $\hat{y}$ is close to 0 (incorrect).
- Blue curve ($t = 0$): Loss is low when predicted probability $\hat{y}$ is close to 0 (correct and confident), and high when $\hat{y}$ is close to 1 (incorrect).

Finally, BCE is **differentiable**, which is essential for training neural networks using gradient-based optimization methods. Its derivative with respect to a (the input to the sigmoid) is:

$$\frac{\partial L}{\partial a} = \hat{y} - t = f(a) - t \tag{22}$$

This derivative is used in backpropagation to update the network's weights during training.

In summary, **Binary Cross-Entropy** is the standard loss function for binary classification tasks in neural networks, effectively measuring the discrepancy between predicted probabilities and true binary labels, and guiding the training process to improve model performance. In simple words, it asks: "*how surprise would I be if the model's predicted probability were true?*" The less surprised (closer to 1 for correct class), the smaller the loss; the more surprised (model confident but wrong), the larger the penalty.

2.3.3.3 Multi-Class Classification

When we want the network to choose **one label among many**, for example to classify images of handwritten digits (0, 1, 2, . . . , 9), we need the model to output a **vector of probabilities**, one for each possible class. For example, if the model is 70% sure that the image is a 3, 20% sure it is an 8, and 10% sure it is a 5, the output vector should be:

$$\hat{y} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0.7 \\ 0 \\ 0.1 \\ 0 \\ 0.2 \\ 0 \\ 0 \end{bmatrix}$$

Where each entry corresponds to the predicted probability of each class (from 0 to 9). To achieve this, exists two main techniques: **one-hot encoding** for the labels and the **softmax activation function** for the output layer.

How we represent targets: One-Hot Encoding

One-Hot Encoding is a simple way to represent **categorical variables** (things that take one of several discrete values, like *color*, *day of week*, or *class label*) in a numerical format that a neural network can understand.

The problem it solves. Neural networks work only with **numbers**, not words or symbols. So if our categories are, for example, *cat*, *dog*, and *bird*, we can't feed them directly into the network. We must convert each category into a **numeric vector**.

How it works. The naïve approach would be to assign each category a unique integer (e.g., *cat* = 0, *dog* = 1, *bird* = 2). But this is **misleading**, because the network would think that *bird* (2) is somehow "bigger" or "twice" a "dog" (1). That numerical relationship is meaningless and these categories have no inherent order. Instead, we create a **binary vector** for each class (category), where:

- The position corresponding to that class is set to 1 ("**hot**").
- All other positions are set to 0 ("**cold**").

So for our example with three categories, the one-hot encoded vectors would be:

- *cat* $\rightarrow [1, 0, 0]$
- *dog* $\rightarrow [0, 1, 0]$
- *bird* $\rightarrow [0, 0, 1]$

Each vector is called **one-hot vector** because exactly **one element is "hot"** (1) and all others are "cold" (0).

How we get probabilities: Softmax Activation Function

The **Softmax Activation Function** takes a vector of arbitrary real numbers (called *logits*) and turns it into a **probability distribution**, i.e. a vector of positive numbers that **sum to 1**:

$$\text{softmax}(a_i) = \frac{e^{a_i}}{\sum_{j=1}^{K} e^{a_j}} \tag{23}$$

Where:

- a_i is the i-th **element of the input vector** (logits).
- K is the total **number of classes**.
- e is the base of the natural logarithm (neperian constant).

Intuition. Each neuron in the output layer produces a **score**: a real number that can be positive, negative, or large. Softmax converts these scores into **relative probabilities** that express how confident the network is about each class:

- Large $a_i \to$ large $e^{a_i} \to$ **high** probability.
- Small $a_i \to$ small $e^{a_i} \to$ **low** probability.

The exponential function e^{a_i} magnifies differences between scores, so the biggest score gets *much more weight*, but every class still receives a small share.

Example 7

Suppose the output layer of a neural network produces the following logits for a 3-class classification problem:

$$a = \begin{bmatrix} 2.0 \\ 1.0 \\ 0.1 \end{bmatrix}$$

To convert these logits into probabilities using the softmax function, we first compute the exponentials:

$$e^a = \begin{bmatrix} e^{2.0} \\ e^{1.0} \\ e^{0.1} \end{bmatrix} \approx \begin{bmatrix} 7.389 \\ 2.718 \\ 1.105 \end{bmatrix}$$

Next, we sum these exponentials:

$$S = 7.389 + 2.718 + 1.105 \approx 11.212$$

Finally, we compute the softmax probabilities:

$$\text{softmax}(a) = \begin{bmatrix} \frac{7.389}{11.212} \\ \frac{2.718}{11.212} \\ \frac{1.105}{11.212} \end{bmatrix} \approx \begin{bmatrix} 0.659 \\ 0.242 \\ 0.099 \end{bmatrix}$$

Thus, the output probabilities for the three classes are approximately 65.9%, 24.2%, and 9.9%, respectively. The network is most confident that the input belongs to class 1.

Softmax acts like a "**competition**" between neurons:

- Each output neuron tries to **"win"** by having the highest score.
- The exponentials amplify the differences, making the highest score dominate.
- The normalization (dividing by the sum) ensures all probabilities add up to 1.

This is why it's called "softmax": it produces a **soft** version of the **maximum** function, where the highest score gets the most weight, but all classes still receive some probability (unlike a hard max which would assign 100% to the highest and 0% to all others).

Putting it all together

In a **K-class classification** problem, the network's final layer has:

- **K output neurons**, one for each class.
- Each neuron produces a **logit** (a_i) an unnormalized score.
- The **softmax function** converts these logits into a **probability distribution** over the K classes:

$$\hat{y}_i = \text{softmax}(a_i) = \frac{e^{a_i}}{\sum_{j=1}^{K} e^{a_j}}$$

So the network outputs a probability distribution over classes, all y_i are between 0 and 1, and they sum to 1. However, to train the network effectively, we also need a suitable loss function that works well with this setup (about training, we will discuss it later, but for now, let's focus on the loss function). This is where **Categorical Cross-Entropy (CCE)** comes into play.

The **Categorical Cross-Entropy (CCE)** loss function measures how close the predicted probability distribution $\hat{y}$ is to the true one-hot distribution $\mathbf{t}$:

$$\text{CCE}(\mathbf{t}, \hat{y}) = -\sum_{i=1}^{K} t_i \cdot \log(\hat{y}_i) \tag{24}$$

Because only the true class has $t_i = 1$ (all others are 0), this simplifies to:

$$\text{CCE}(\mathbf{t}, \hat{y}) = -\log(\hat{y}_c) \tag{25}$$

Where:

- c is the **index of the true class**.
- y_c is the **predicted probability for the true class**.

This means CCE penalizes the model when it assigns a low probability to the true class, **encouraging** it to **predict higher probabilities for the correct class** during training.

Why Softmax and CCE work well together? The combination of Softmax and CCE is powerful because:

- **Softmax produces a valid probability distribution**, which is exactly what CCE needs to compute the loss.
- **CCE** focuses the learning on **maximizing the probability of the true class**, which aligns perfectly with the goal of classification tasks.
- The **gradients** computed from CCE with respect to the logits are well-behaved (very simple), making training more stable and efficient. The drivative of CCE combined with Softmax is the following:

$$\frac{\partial\ (\text{CCE})}{\partial\, a_i} = \hat{y}_i - t_i \tag{26}$$

  This means the gradient is simply the difference between the **predicted probability** and the **true label**, which is **easy to compute** and interpret. As we will see later, this makes backpropagation very efficient and stable.

This synergy makes Softmax + CCE the **standard choice for multi-class classification problems in neural networks**. It is a generalization of the Sigmoid + BCE setup used for binary classification, extending the same principles to handle multiple classes effectively.

2.3.4 Neural Networks as Universal Approximators

In 1989, Kurt Hornik, Maxwell Stinchcombe, and Halbert White published a seminal paper titled "Multilayer Feedforward Networks are Universal Approximators" [5]. This groundbreaking work established that Feed-Forward Neural Networks (FNNs) with at least one hidden layer and non-linear activation functions can approximate any continuous function on compact subsets of $\mathbb{R}^n$ to any desired degree of accuracy, given sufficient neurons in the hidden layer:

> "*A single hidden layer feed-forward neural network with S-shaped activation functions can approximate any measurable function to any desired degree of accuracy on a compact set.*"

This theorem establishes the *theoretical power* of neural networks: given enough hidden neurons, an FNN can approximate **any continuous function** $f(x)$ over a bounded input domain. Let's define this more formally.

Theorem 1 (Universal Approximation Theorem)**.** *Let $K \subset \mathbb{R}^n$ be a compact set, and let:*

$$f : K \to \mathbb{R}$$

Be a continuous function. Then, for every $\varepsilon > 0$, there exist:

- *a Feed-Forward* ***Neural Network with a single hidden layer****,*
- *a* ***finite*** *number of* ***hidden neurons*** *J,*
- *weights $w_{ji}^{(1)}, w_j^{(2)} \in \mathbb{R}$,*
- *biases $b_j \in \mathbb{R}$,*

Such that for every $x \in K$ (for all inputs in the compact set):

$$\left| f(x) - \sum_{j=1}^{J} w_j^{(2)} \, \sigma\left(\sum_{i=1}^{n} w_{ji}^{(1)} x_i + b_j \right) \right| < \varepsilon \tag{27}$$

Where:

- *$w_{ji}^{(1)}$ are the* ***weights*** *from* ***input layer to hidden layer****.*
- *b_j are the* ***biases*** *of the* ***hidden layer*** *neurons.*
- *$\sum_{i=1}^{n} w_{ji}^{(1)} x_i + b_j$ is the* ***input to the hidden layer*** *neuron j.*
- *$w_j^{(2)}$ are the* ***weights from hidden layer to output layer****.*
- *$\sigma(\cdot)$ is the* ***non-linear activation function*** *applied at* ***hidden*** *layer neurons. For example, a common choice is the* ***sigmoid activation function*** *(page 64):*

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

- *The output of the neural network is the weighted sum of the activated hidden neurons, which approximates $f(x)$ within ε for all $x \in K$ (i.e., the neural network can get arbitrarily close to $f(x)$).*

In simpler terms, any continuous function can be represented by a neural network with just **one hidden layer**, if that layer has enough neurons and uses a non-linear activation function.

Why it works (intuition)

Each hidden neuron with non-linear activation acts like a **basis function**. By combining enough of these basis functions (hidden neurons), the neural network can approximate complex functions by adjusting the weights and biases.[5]

Imagine we have an unknown function $f(x) = \sin(x)$. And we want our neural network to **learn** this function. So, in other words, we want our neural network to approximate $f(x)$ as closely as possible ($\hat{f}(x) \approx f(x)$). Let's take a **single neuron** with a non-linear activation function (e.g., sigmoid):

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

If we plot this, it looks like an S-shaped curve (page 64):

- Almost 0 for large negative inputs.
- Almost 1 for large positive inputs.
- Smoothly transitions between 0 and 1 around input 0.

When we apply this neuron to a **linear combination of** x:

$$\sigma(w \cdot x + b)$$

We get a *shifted and stretched S-curve* along the x-axis. Now imagine we have **many hidden neurons**, each with their own weights and biases:

$$\hat{f}(x) = \sum_{j=1}^{J} w_j^{(2)} \cdot \sigma(w_j^{(1)} \cdot x + b_j)$$

Each neuron produces its own "bump" or "S-step" at a different location. When we **add them together**, those bumps **stack up and blend**, creating any curve shape we want. And that's the whole trick! Just like adding sinusoids can approximate any periodic signal (Fourier series), adding non-linear S-shaped functions can approximate any continuous curve. Note that non-linearity is crucial. If we used only linear activations and stacked them, the result would still be a linear function. This would collapse the network's expressive power.

Important note

The Universal Approximation Theorem guarantees that a neural network **can** approximate any continuous function, but it **does not tell us how to find the right weights and biases to do so**. In practice, training a neural network to approximate a specific function requires effective optimization algorithms (like gradient descent) and sufficient training data. Additionally, while a single hidden layer is theoretically sufficient, deeper networks (with more hidden layers) often learn more efficiently and generalize better in practice.

[5] Similar to how sine and cosine functions can approximate any waveform in Fourier series (i.e., any periodic function can be approximated by summing sines and cosines), the non-linear activations in hidden neurons serve as building blocks to construct complex functions.

2.4 Learning and Optimization

Let's retrace what we've built so far step by step:

1. We started with the **historical context**, understanding why we want machines to "learn" like brains, transitioning from symbolic AI to data-driven learning.

2. Next, we explored the **Perceptron**, the simplest computational neuron, which introduced us to linear decision boundaries and Hebbian learning.

3. However, we also learned about the **limitations of the Perceptron**, particularly its inability to solve non-linear problems like XOR.

4. To overcome these limitations, we delved into **Feed-Forward Neural Networks (FNNs)**, discovering how multi-layer networks with hidden layers and nonlinear activations can model complex functions.

5. Finally, we touched on the **Universal Approximation Theorem**, which assures us that even a single hidden layer is theoretically sufficient to approximate any function (though in practice, deeper networks often perform better).

Now we know **what** the architecture can represent. But we haven't yet learned **how** to *find the right weights* that make it represent what we want. And that's *exactly* why this section begins.

After defining the structure of a neural network, we must **teach it** to perform a task, such as classifying images or predicting values. This teaching process is called **learning** or **training** (or simply *learning by optimization*). Think of the journey like this:

$$\text{Architecture} \rightarrow \text{Function Space} \rightarrow \text{Optimization} \rightarrow \text{Learning}$$

We've defined the **function space** (what kinds of functions the network can represent), and now, we must **search inside that space** for the specific function that matches our data. This search is done through **optimization algorithms** that adjust the network's weights based on the data we provide. So, in this section, we'll answer three big questions:

1. ***How does a neural networks learn?*** (page 87) By comparing predictions with known targets (supervised learning).

2. ***How do we measure "how wrong" it is?*** (page 90) Through *loss functions* (some of which we have already encountered in the output layer design).

3. ***How do we improve it?*** (page 94) Through *optimization algorithms* like gradient descent and (later) backpropagation.

2.4.1 Supervised Learning and Training Dataset

This section introduces the **formal setup** of *Supervised Learning* in the context of neural networks. It defines **what data to use**, **what we want the network to learn**, and **how we measure learning success**. It's the *conceptual skeleton* that the later mathematical tools (loss, gradient descent, backpropagation) will stand on.

What is Supervised Learning?

Supervised Learning is a **machine learning paradigm** where the algorithm learns **from examples that include both the input and the correct output**. In other words, it learns **under supervision** from labeled data.

The basic idea is simple. We give the model a set of **training examples**:

$$\mathcal{D} = \{(x_1, t_1), (x_2, t_2), \ldots, (x_N, t_N)\}$$

Formally, a **dataset** is:

$$\mathcal{D} = \left\{ (x_i, t_i) \right\}_{i=1}^{N} \tag{28}$$

Where:

- x_i is the **input data** (e.g., an image, temperature readings, pixels, sensor values, etc.).
- t_i is the **target output** (the *label* or *ground truth* we want the model to predict).
- N is the total number of training examples.

The model (in our case, a neural network) tries to learn a **function** $f(\cdot)$ such that:

$$f(x_i) \approx t_i \quad \text{for all } i = 1, 2, \ldots, N$$

For all examples in the training set. In other words, to find a function $f(x)$ that not only fits the training data but also **generalizes** well to unseen data (i.e., it can correctly predict outputs for new inputs not in the training set). Formally, a **model** is a function:

$$g(x; w) \text{ with parameters } w \tag{29}$$

Where:

- $g(\cdot; w)$ is the model (neural network) with parameters w (weights and biases).
- The goal is to find the optimal parameters w^* such that:

$$w^* = \arg\min_w E(w) \tag{30}$$

  Where $E(w)$ is a **loss function** that measures how far predictions $g(x_i; w)$ are from the true targets t_i across the training set.

The method is called **supervised** because the learning process is **guided**: each input x comes with the **correct answer** t. The network uses it to know whether it was right or wrong, and to adjust its weights accordingly.

Independently by the paradigm used (supervised, unsupervised, reinforcement learning), with **Training** we mean the **process of adjusting weights w so that the network reproduces the mapping between inputs and outputs seen in the data**. This is done by **minimizing a loss function** that quantifies the difference between the network's predictions and the true targets in the training dataset.

Neural Networks as a Parametric Model

In general, a neural network can be written as a **parametric function**:

$$y\left(x; w\right) = g\left(x, w\right) \tag{31}$$

Where:

- x is the input features (data).
- w is the set of parameters (weights and biases in all layers) of the network.
- $y\left(x; w\right)$ is the output of the network (the prediction for input x given parameters w).
- ; indicates that y depends on both x and w.
- $g(\cdot, \cdot)$ represents the entire computation performed by the neural network (all layers, activations, etc.). See above equation 29.

We want $y\left(x_n; w\right)$ to be as close as possible to the target t_n for each training example (x_n, t_n) in the dataset $\mathcal{D}$. This is achieved by **optimizing the parameters w** to minimize a **loss function** that measures the discrepancy between predictions and targets across all training examples. Formally, find parameters w^* that minimize a loss function $E(w)$:

$$w^* = \arg\min_w E\left(w\right)$$

Where $E(w)$ quantifies the error between $y\left(x_n; w\right)$ and t_n for all training examples. The loss function measures how wrong the model is across all training examples:

$$E\left(w\right) = \sum_{n=1}^{N} \ell\left(t_n, y\left(x_n; w\right)\right) \tag{32}$$

Where $\ell(\cdot, \cdot)$ is a loss function that quantifies the error for a single example (e.g., Mean Squared Error for regression, Cross-Entropy Loss for classification).

The power of this setup

This framework allows us to covers a wide range of tasks:

- **Regression**: Predicting continuous values (e.g., house prices, temperature).
- **Classification**: Assigning inputs to discrete categories (e.g., spam vs. not spam, image recognition).
- **Function Approximation**: Learning complex mappings from inputs to outputs.

By defining the problem in terms of inputs, targets, and a loss function, we can apply various optimization algorithms (like gradient descent) to train the neural network effectively. So, we must care only about:

- The **structure of the network** (layers, activation functions).
- The **choice of loss function** (depends on the task).
- The **optimization algorithm** to minimize the loss.

This abstraction makes neural networks a versatile tool for many machine learning problems.

Example 8: Analogy

Imagine a student (the network) learning to solve math exercises.

- **Inputs** x_n are the exercises given to the student by the teacher.
- **Targets** t_n are the correct answers provided by the solution book.
- **Network** $g(x, w)$ is the student's method of solving the exercises, which depends on their current knowledge (parameters w).
- **Loss function** $E(w)$ measures how many answers the student got wrong compared to the solution book.
- **Optimization** (training) is the process of the student studying and adjusting their methods (updating w) to minimize mistakes on future exercises.

Over time, with enough practice (training examples), the student improves their ability to solve new exercises correctly (generalization). So training means making fewer mistakes over time by adjusting reasoning (weights).

2.4.2 Error Minimization and Loss Function (SSE)

In order to teach a neural network, we need a way to **measure how incorrect it is**. This measurement is called the **error (or loss) function**. Once defined, we can **minimize it** by adjusting the weights. This process is the essence of learning.

Definition 6: Error Function

Given a **training set**:

$$\mathcal{D} = \left\{ (x_n, t_n) \right\}_{n=1}^{N} \tag{33}$$

And the network's predictions:

$$y_n = g\left(x_n; w\right) \tag{34}$$

The **Error Function** $E(w)$ measures the total discrepancy between all predictions and their true targets:

$$E(w) = \sum_{n=1}^{N} \text{Loss}(t_n, y_n) \tag{35}$$

Where $\text{Loss}\left(\cdot\right)$ is the per-sample difference between the predicted and actual value (i.e., the loss function). The error function aggregates these losses across the entire training set to give a single scalar value representing how well the model is performing overall. The goal of learning is to find the weights w that minimize this error function, thus improving the model's predictions (formally, Equation 39 on page 91).

Definition 7: Loss Function

A **Loss Function** (sometimes called **error** or **Cost Function**) is a **mathematical function that quantifies how wrong a model's predictions are** compared to the true (target) values. In other words:

$$\text{Loss}(t, y) = \text{scalar measure of discrepancy between } t \text{ and } y \tag{36}$$

Where t is the true target value, and y is the model's prediction. The loss function **outputs a single number representing how bad the prediction is**; lower values indicate better predictions. During training/learning, the network tries to **minimize** this loss by adjusting its weights, to reduce its mistakes.

It is strictly related to the **Error Function** $E(w)$, which aggregates the loss over the entire training set to give a total measure of how well the model is performing. Indeed, for a single training example (x_n, t_n), we have:

$$L_n = \text{Loss}\left(t_n, g\left(x_n; w\right)\right) \tag{37}$$

Where L_n is the loss for sample n, t_n is the true target, and $g(x_n; w)$ is the model's prediction for input x_n with weights w. And the total error function over all N samples is:

$$E(w) = \sum_{n=1}^{N} L_n = \sum_{n=1}^{N} \text{Loss}\left(t_n, g\left(x_n; w\right)\right)$$

So, the **loss function measures the error for one sample**, while the **Error Function sums these losses over the entire dataset** to give a total error measure.

The simplest and most classic choice: SSE

Exists many choices for the loss function. The simplest and most classic is the **Sum of Squared Errors (SSE)**. In early neural networks (and still often in regression problems), the **Sum of Squared Errors (SSE)** was the standard loss function:

$$E(w) = \sum_{n=1}^{N} \text{Loss}(t_n, y_n) = \sum_{n=1}^{N} \left[t_n - g\left(x_n; w\right)\right]^2 \tag{38}$$

- t_n is the true target for sample n.
- $g(x_n; w)$ is the network's prediction for input x_n.

Similar to the Mean Squared Error (MSE, page 73), the SSE squares the differences to makes all errors **positive** (so under- and over-predictions both count) and **emphasizes large errors** (penalizes them more heavily). Also, squaring makes the error function **differentiable**, which is crucial for optimization algorithms like gradient descent.

Minimizing the SSE means finding the weights w that make the network's predictions as close as possible to the true targets across the entire training set. Formally, **learning becomes finding the set of weights w^* that minimize the total error**:

$$w^* = \arg\min_{w} E(w) \tag{39}$$

Each weight w_i acts like a small knob controlling part of the network's behavior. We tweak these knobs slightly so that the network's predictions move closer to the true outputs. When all knobs are adjusted such that $E(w)$ is as small as possible, the network has **learned** the function.

Geometric Interpretation

Minimizing the error means **finding a point in parameter space w where the error surface $E(w)$ reaches its minimum**. We can visualize $E(w)$ as a **landscape**: valleys represent low error (good predictions), and hills represent high error (bad predictions). The learning process is like navigating this landscape to find the lowest valley, which corresponds to the optimal weights w^* that minimize the error.

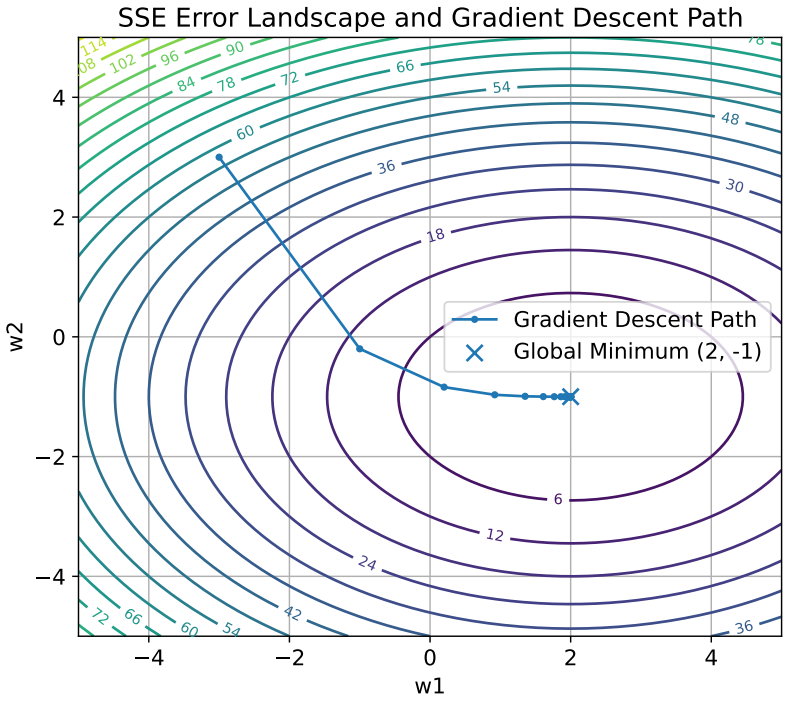


Figure 10: How the **Sum of Squared Errors (SSE)** behaves as a function of the **model parameters (weights)**, and how **gradient descent** moves step by step toward the minimum error point. The x-axis w_1 and y-axis w_2 represent two weights (parameters) of the model/ Each contour line shows **all combinations of weights** (w_1, w_2) that produce the **same total error** $E(w_1, w_2)$. The loss function we use:

$$E\left(w_1, w_2\right) = \left(w_1 - 2\right)^2 + 2\left(w_2 + 1\right)^2$$

Those small points connected by a line represent the **path that gradient descent follows** over time. Starting from an initial guess, each step moves downhill toward lower error, reaching the minimum point where the error is lowest (point $(2, -1)$ in this case). In a real network, the number of weights isn't just two but can be thousands or millions, making the error surface a high-dimensional landscape. We can't visualize that directly, but this contour map is an **analogy** to help understand how optimization algorithms like gradient descent navigate the error surface to find the best weights.

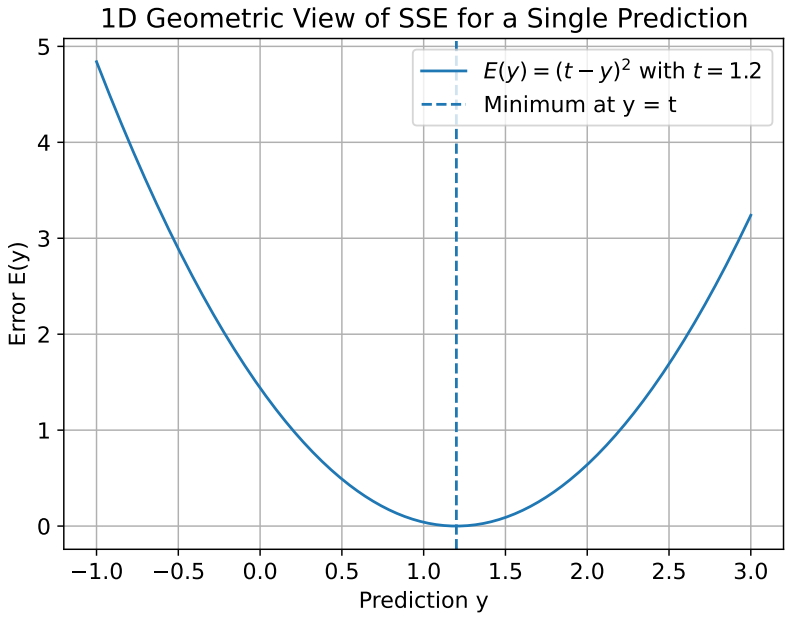


Figure 11: The **error function** for **a single training example** using the **Sum of Squared Errors (SSE)** loss: $E(y) = (t - y)^2$, where t is the true target and y is the model's prediction. The x-axis represents the model's prediction y (all possible output values the model could predict for this input sample), and the y-axis shows the corresponding error $E(y)$ (how wrong the model would be for each possible prediction). This graph shows **how the error changes** as the model output moves away from the correct answer. It is a **parabola**, because the error grows quadratically as we move away from the correct value. At $y = t$, the error is zero (the model predicts perfectly) because $E(t) = (t - t)^2 = 0$. As y moves away from t, the difference grows, and squaring makes it **positive and larger**. The dashed vertical line marks the **minimum point** where the prediction perfectly matches the target ($y = t$, zero error).

Relation to Statistical Foundations

The squared error has a nice **probabilistic interpretation**. If we assume that the target values t_n are generated by a model with **Gaussian noise**:

$$t_n = g(x_n; w) + \epsilon_n, \quad \epsilon_n \sim \mathcal{N}(0, \sigma^2)$$

Then minimizing the sum of squared errors (SSE) is equivalent to **Maximum Likelihood Estimation (MLE)** of the weights w. In other words, by minimizing SSE, we are finding the weights that make the observed data most probable under the assumed Gaussian noise model. So the SSE is not arbitrary choice; it has a solid statistical basis when the noise in the data is Gaussian. In the section Maximum Likelihood Estimation (MLE) (page 107), we will explore this connection in more detail.

2.4.3 Gradient Descent Basics

We've defined the learning goal (page 91):

$$w^* = \arg\min_w E(w)$$

Where $E(w)$ is our **error (or loss) function**, for instance the Sum of Squared Errors (SSE, Equation 38 on page 91):

$$E(w) = \sum_{i=1}^{N} (t_n - g(x_n; w))^2$$

The idea of **gradient descent** is to find this minimum *iteratively*, by moving the parameters w step by step in the direction that **reduces** the error.

The Concept of the Gradient and the Key Idea of Gradient Descent

Let's start simple. For a function of one variable $E(w)$, the **derivative** $\frac{\partial E}{\partial w}$ at a point w tells us the slope of the function:

- If positive $\rightarrow$ $E(w)$ increases as w increases.
- If negative $\rightarrow$ $E(w)$ decreases as w increases.
- If zero $\rightarrow$ $E(w)$ is flat (local minimum or maximum).

In higher dimensions (e.g., multiple weights $w_1, w_2, \ldots, w_d$), we generalize the derivative to a **vector** of partial derivatives:

$$\nabla E(w) = \begin{bmatrix} \frac{\partial E}{\partial w_1} \\ \frac{\partial E}{\partial w_2} \\ \vdots \\ \frac{\partial E}{\partial w_d} \end{bmatrix}$$

This vector, the **gradient** of E at w, points in the direction of **steepest ascent** of the function (the **direction** in which $E(w)$ **increases the most rapidly**).

So, if we want to **minimize** $E(w)$, we should move in the direction of **steepest descent**, which is the **opposite direction** of the gradient, i.e., $-\nabla E(w)$. That's why it's called **gradient descent**!

> **Definition 8: Gradient Descent**
>
> **Gradient Descent** is an **iterative optimization algorithm** used to find the set of parameters (weights) that **minimize a loss function**.

In simple words, it is the process by which a neural network *learns* by **repeatedly adjusting its weights in the direction that reduces the loss the most**.

Formally, let w be the vector of weights, and $E(w)$ be the loss function. The **gradient** with respect to the weights is given by $\nabla E(w)$:

$$\nabla E(w) = \begin{bmatrix} \frac{\partial E}{\partial w_1} \\ \frac{\partial E}{\partial w_2} \\ \vdots \\ \frac{\partial E}{\partial w_k} \end{bmatrix} \tag{40}$$

This vector points in the **direction of steepest increase** of $E(w)$. To *minimize* the loss, we go in the **opposite direction** of the gradient. That gives the update rule for the weights, known as **Core Learning Rule**:

$$w_{k+1} = w_k - \eta \cdot \nabla E(w_k) \tag{41}$$

Where:

- w_k is the weight vector at iteration k.
- $\nabla E(w_k)$ is the gradient (slope) of the error function at w_k.
- η (eta) is the **learning rate** (step size), a small positive scalar that controls the step size.
- w_{k+1} is the updated weight vector after taking a step in the direction of steepest descent.

It is called Core Learning Rule because it is the **fundamental principle underlying how neural networks learn from data by adjusting their weights to minimize error**. Everything that comes next (like backpropagation) are all *variants or extensions* of this exact rule. They all keep this same pattern:

new param = old param−(some learning rate)×(some form of gradient)

The only difference is *how* the gradient or learning rate is computed or adjusted. It is a sort of **DNA of learning** in neural networks.

The Neural Network Case

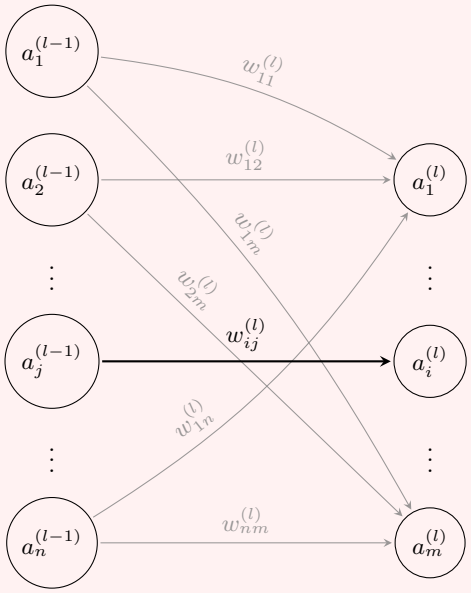
The previous definition is **general**, and it doesn't care whether w is one number, a list or a tensor of weights inside a neural network. However, sice we are in the context of neural networks, let's introduce some notation specific to them.

In a neural network, w isn't just one parameter vector (one weight vector), but rather a **collection of all weights and biases across all layers**:

$$w = \left\{ w_{ij}^{(l+1)}, w_{ij}^{(l+2)}, \ldots, w_{ij}^{(L)}, b_i^{(l+1)}, b_i^{(l+2)}, \ldots, b_i^{(L)} \right\} \tag{42}$$

Where:

- L is the **total number of layers in the network**.
- Each $w_{ij}^{(l)}$ is the weight connecting neuron j in layer $l-1$ to neuron i in layer l:



- Each $b_i^{(l)}$ is the bias for neuron i in layer l.

The **gradient** $\nabla E(w)$ then contains the partial derivatives of the loss function with respect to **each individual weight and bias** in the network:

$$\nabla E(w) = \left\{ \frac{\partial E}{\partial w_{ij}^{(l)}}, \frac{\partial E}{\partial b_i^{(l)}} \right\} \tag{43}$$

The **core learning rule** still applies, but now we update **each weight and bias** individually:

$$w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} - \eta \frac{\partial E}{\partial w_{ij}^{(l)}} \quad \text{and} \quad b_i^{(l)} \leftarrow b_i^{(l)} - \eta \frac{\partial E}{\partial b_i^{(l)}} \tag{44}$$

This is how **gradient descent** is applied in the context of training neural networks.

How it works (intuitively)

Intuitively, gradient descent works as follows:

1. Start from an **initial guess** for the weights w_0 (often random).
2. Compute the **gradient** $\nabla E(w_0)$ of the loss function at the current weights:

$$\nabla E(w_0) = \begin{bmatrix} \frac{\partial E}{\partial w_1} \\ \frac{\partial E}{\partial w_2} \\ \vdots \\ \frac{\partial E}{\partial w_d} \end{bmatrix}_{w=w_0}$$

3. Do a step **against** that gradient to update the weights using the **core learning rule**:

$$w_{k+1} = w_k - \eta \cdot \nabla E(w_k) \quad \Rightarrow \quad w_1 = w_0 - \eta \cdot \nabla E(w_0)$$

   Then, the new weights w_1 should yield a **lower loss** $E(w_1) < E(w_0)$. If we were on a neural network, we would update **all weights and biases** similarly:

$$w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} - \eta \cdot \frac{\partial E}{\partial w_{ij}^{(l)}} \quad , \quad b_i^{(l)} \leftarrow b_i^{(l)} - \eta \cdot \frac{\partial E}{\partial b_i^{(l)}}$$

4. Repeat until: the gradient becomes small (close to zero), or we reach a maximum number of iterations.

Convergence Speed and False Positives. The speed at which gradient descent converges to the minimum depends on:

- The **shape** of the loss function (e.g., steepness, curvature). It strongly affects how gradient descent behaves:
  - **Convex surface** (like a bowl): Gradient descent always converges there (since there is a **single global minimum**). However, if η is too large, it may oscillate around the minimum.
  - **Non-convex surface** (like many hills and valleys): There may be **multiple local minima** and **saddle points**. Gradient descent might:
    - Get stuck in a **local minimum** (the derivative is zero, but it is not the global optimum).
    - Oscillate in flat regions.
    - Move very slowly along narrow valleys.
- The **learning rate** η.

  The **learning rate** η is crucial: it controls **how big a step** we take each iteration.

  - η too **small** → **Learning is very slow** because we take tiny steps (many iterations needed).
  - η too **large** → **May overshoot the minimum** and even diverge (loss increases instead of decreasing).
  - Choosing a good η is often done via **experimentation** or using techniques. However, if η is chosen well, gradient descent can efficiently find a good set of weights that minimize the loss function.

  Don't worry if this seems abstract now; we'll later learn that **adaptive optimizers** (like ***momentum***, ***Adam***, etc.) are more robust ways to handle this.

- The **initial weights** w_0. In **convex** functions (like a parabola), there is a global minimum, and *gradient descent* will converge to it. In **non-convex** functions (like many neural network loss landscapes), there may be multiple local minima, and gradient descent may converge to one of them depending on the starting point. So in practice, we often run gradient descent multiple times with different initial weights to find a good solution.

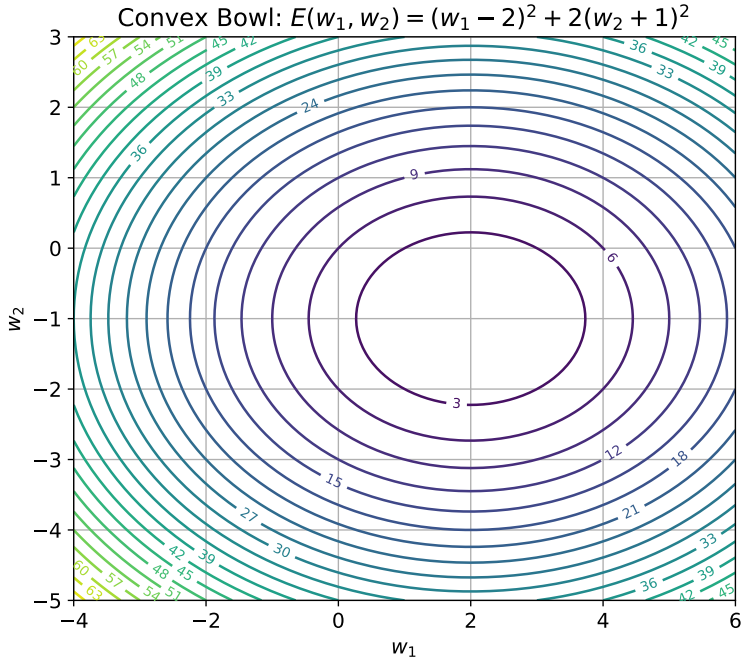


Figure 12: Example of a **convex quadratic surface** (bowl-shaped). The contour lines (ellipses) represent levels of constant error $E(w_1, w_2)$. There is a **single global minimum** at the center of the bowl ($w_1^* = 2, w_2^* = -1$), and outer ellipses represent higher error values. This is the ideal scenario for gradient descent, as it will always converge to the global minimum regardless of the starting point (similar to Figure 10, page 92).

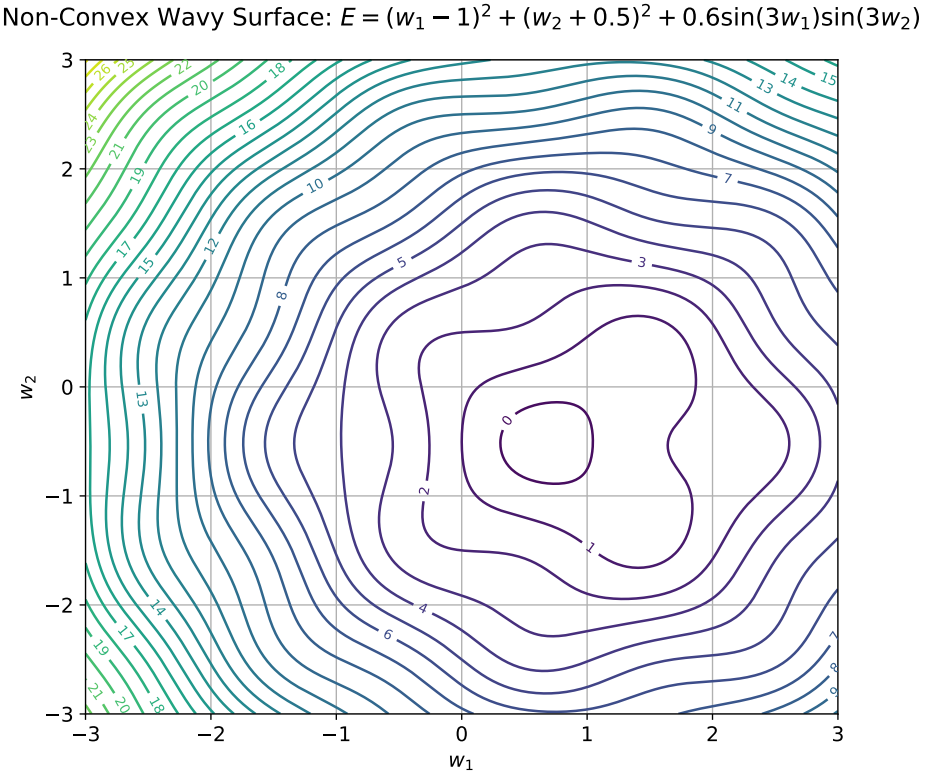


Figure 13: Example of a **non-convex surface** with multiple local minima and saddle points. The loss function $E(w_1, w_2)$ has two components: (1) the **quadratic bowl** that pushes weights toward the global minimum, and (2) the **sinusoidal term** that introduces *oscillations* in the surface. Those oscillations create **small waves** (bumps and dips). Each dip (a small valley) is a *local minimum*, and each bump is a *local maximum* or ridge. This is what happens in **non-linear models** like deep neural networks, where the composition of many layers and activations makes the error surface very complex (and non-convex). There are thousands or millions of such local minima, making optimization challenging. Gradient descent may get trapped in one of these local minima instead of finding the global minimum. However, in practice, many local minima have **similar performance**, so this isn't always bad.

2.4.4 Backpropagation (Conceptual Introduction)

Why Backpropagation exists?

The **problem** we face is **computational**:

- Gradient descent (page 94) at each iteration requires the computation of **all partial derivatives** of the loss function E with respect to **all weights and biases** in the network. Formally, we need to compute (introduced in section 2.4.3, page 96):

$$\nabla E(w) = \left\{ \frac{\partial E}{\partial w_{ij}^{(l)}}, \frac{\partial E}{\partial b_i^{(l)}} \right\}$$

- In a network with thousands of weights, each weight influences the output *indirectly* through multiple layers.

Computing those derivatives by brute force (finite differences or manual application of the chain rule) would be **computationally expensive** and **inefficient**. We need a more efficient way to compute these gradients.

Backpropagation Concept

Backpropagation (or **backward propagation of errors**) is an efficient **algorithm** used to **compute the gradients** of the loss function with respect to all weights and biases **in a neural network**. It leverages the **chain rule of calculus** in a systematic, efficient way to reuse computations and propagate errors backward through the network. It is analogous to hashmaps in programming, where intermediate results are stored and reused to avoid redundant calculations.

Remark: The Chain Rule

The **Chain Rule** is a fundamental rule in calculus used to compute the derivative of a composite function.
Suppose we have a **function inside another function**:

$$y = f\left(g(x)\right)$$

We want to know how y changes when we slightly change x. In other words, we want to find the derivative $\frac{\partial y}{\partial x}$. The chain rule tells us that we can break this down into two parts:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial g} \cdot \frac{\partial g}{\partial x} \tag{45}$$

This means that to find out how y changes with respect to x, we first find out how y changes with respect to g (the inner function), and then multiply that by how g changes with respect to x. It's literally "follow the chain" of dependencies.
Let's see a quick example. Let's say:

$$y = f\left(g(x)\right) = (2x+3)^2$$

We can identify:

- Inner function: $g(x) = 2x + 3$
- Outer function: $f(g) = g^2$

Now, we compute the derivatives:

- $\frac{\partial f}{\partial g}\left(g^2\right) = 2g$
- $\frac{\partial g}{\partial x}\left(2x + 3\right) = 2$

Now, applying the chain rule:

$$\frac{\partial y}{\partial x} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial x} = 2g \cdot 2 = 4g$$

Finally, substituting back $g(x)$:

$$\frac{\partial y}{\partial x} = 4(2x + 3) = 8x + 12$$

Conceptually, Backpropagation works during the training phase of a neural network. **During training phase**, there are **two complementary flows** of information:

- **Forward Pass** (Input $\rightarrow$ Output): The purpose is to **measure how good the current weights are at predicting the target outputs**. The input data is passed through the network layer by layer to compute the output. During this phase, the activations of each neuron are computed and stored for later use. Formally, compute predictions $y(x; w)$ for input x and weights w, and save intermediate activations $a^{(l)}$ for each layer l. Here, $a^{(l)}$ represents the activations at layer l:

  $$a^{(l)} = f\left(W^{(l)}a^{(l-1)} + b^{(l)}\right) = f(z^{(l)})$$

  where f is the activation function, $W^{(l)}$ are the weights, and $b^{(l)}$ are the biases at layer l.

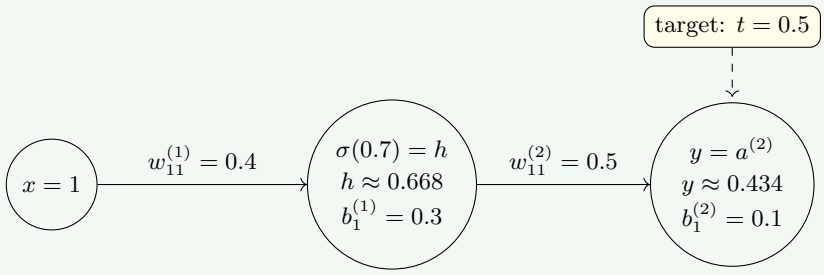
- **Backward Pass** (Output $\rightarrow$ Input): The purpose is to **know how to change weights to reduce the loss**. The error (the difference between the predicted output and the actual target) is propagated backward through the network. During this phase, the gradients of the loss function with respect to each weight and bias are computed using the chain rule, utilizing the stored activations from the forward pass. Formally, compute gradients $\frac{\partial E}{\partial w_{ij}^{(l)}}$ and $\frac{\partial E}{\partial b_i^{(l)}}$ for all weights and biases using the chain rule. The stored activations $a^{(l)}$ are used to compute these gradients efficiently.

Hence the name **backpropagation**: the error is propagated backward through the network to compute gradients efficiently. In the next pages, we will see a

detailed example of how backpropagation works step by step. Formally deriving the backpropagation equations will be covered in later sections.

Example 9: Numerical Example of Backpropagation Concept

We'll train a **1-hidden-layer neural network** to learn the function $y = x$ for one input sample $x = 1$ and target $t = 0.5$. Visually, the network looks like this:



The network architecture is as follows:

- Input layer: 1 neuron, with notation $a^{(0)} = x = 1$.
- Hidden layer: 1 neuron, with a sigmoid activation function:

$$g^{(1)}(a) = \sigma(a) = \frac{1}{1 + e^{-a}}$$

- Output layer: 1 neuron, with notation $y = w_2 \cdot h + b_2$. Where y is the output, w_2 is the weight connecting hidden to output layer, and b_2 is the bias of the output layer. The activation function is linear:

$$g^{(2)}(a) = a$$

The weights and biases are initialized as follows (randomly chosen for this example):

- $w_{11}^{(1)} = 0.4$ (weight from input to hidden layer)
- $b_1^{(1)} = 0.3$ (bias of hidden layer)
- $w_{11}^{(2)} = 0.5$ (weight from hidden to output layer)
- $b_1^{(2)} = 0.1$ (bias of output layer)

The learning rate η is set to 0.1.

Backpropagation Step 1: Forward Pass. In this step, we compute the activations of the hidden and output layers given the input $a^{(0)} = x = 1$.

1. **Compute hidden layer activation** (net input). The neuron has the index $i = 1$ in layer $l = 1$ since it's the first hidden layer (equation 11, page 60):

$$a_1^{(1)} = w_{11}^{(1)} \cdot a^{(0)} + b_1^{(1)} = 0.4 \cdot 1 + 0.3 = 0.7$$

The $a_1^{(1)}$ calculated above is the weighted sum before activation. Now, this value is passed through the sigmoid activation function:

$$h = g^{(1)}\left(a_1^{(1)}\right) = g^{(1)}(0.7) = \sigma(0.7) = \frac{1}{1+e^{-0.7}} \approx 0.668$$

2. **Compute output layer activation**. Again, using equation 11 (page 60), we compute the net input to the output neuron:

$$a_1^{(2)} = w_{11}^{(2)} \cdot h + b_1^{(2)} = 0.5 \cdot 0.668 + 0.1 \approx 0.434$$

But now, Since the output layer uses a linear activation function, the output is:

$$y = g^{(2)}(a_1^{(2)}) = a_1^{(2)} \approx 0.434$$

3. **Compute the error**. The predicted output from the forward pass is $y \approx 0.434$. The error can now be computed using the target $t = 0.5$. For example, using Mean Squared Error (MSE):

$$E = \frac{1}{2}(t-y)^2 = \frac{1}{2}(0.5 - 0.434)^2 \approx 0.0022$$

This error quantifies how far the network's prediction is from the target.

At the end of this step, we have stored (cached) some intermediate values needed for the backward pass:

- **Input activation**: $a^{(0)} = 1$
- **Linear combo to hidden layer**: $a_1^{(1)} = w_{11}^{(1)} \cdot a^{(0)} + b_1^{(1)} = 0.7$
- **Hidden layer activation**: $h = g^{(1)}\left(a_1^{(1)}\right) = \sigma(0.7) \approx 0.668$
- **Linear combo to output layer**: $a_1^{(2)} = w_{11}^{(2)} \cdot h + b_1^{(2)} \approx 0.434$
- **Output activation (identity)**: $y = g^{(2)}\left(a_1^{(2)}\right) \approx 0.434$
- **Error**: $E \approx 0.0022$

Backpropagation Step 2: Backward Pass. In this step, we compute the gradients of the loss function with respect to each weight and bias using backpropagation. We use the Mean Squared Error (MSE, page 72) loss function:

$$E = \frac{1}{2}(t-y)^2$$

Where t is the target output ($t = 0.5$) and y is the predicted output from the forward pass ($y \approx 0.434$). We will compute the gradients layer by layer, starting from the output layer and moving backward to the hidden layer. Also, we will **denote the error term for each layer as** $\delta^{(l)}$ and we propagate it backward through the network (using the cached values from the forward pass!). Simply put, we need to find $\nabla E(w)$:

$$\nabla E(w) = \left\{ \frac{\partial E}{\partial w_{ij}^{(l)}}, \frac{\partial E}{\partial b_i^{(l)}} \right\} = \left\{ \frac{\partial E}{\partial w_{11}^{(2)}}, \frac{\partial E}{\partial b_1^{(2)}}, \frac{\partial E}{\partial w_{11}^{(1)}}, \frac{\partial E}{\partial b_1^{(1)}} \right\}$$

Where:

$$\frac{\partial E}{\partial w_{ij}^{(l)}} = \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial a_i^{(l)}} \cdot \frac{\partial a_i^{(l)}}{\partial w_{ij}^{(l)}} \qquad \frac{\partial E}{\partial b_i^{(l)}} = \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial a_i^{(l)}} \cdot \frac{\partial a_i^{(l)}}{\partial b_i^{(l)}}$$

However, to avoid recomputing terms, we define the error term $\delta^{(l)}$ for layer l as:

$$\delta^{(l)} = \frac{\partial E}{\partial a_i^{(l)}} = \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial a_i^{(l)}}$$

This allows us to express the gradients more compactly:

$$\frac{\partial E}{\partial w_{ij}^{(l)}} = \delta^{(l)} \cdot a_j^{(l-1)} \qquad \frac{\partial E}{\partial b_i^{(l)}} = \delta^{(l)}$$

These equations may be unfamiliar now, but they will make sense in future sections when we derive them formally. For now, let's treat them as given formulas for computing the gradients using the error terms $\delta^{(l)}$. Now, we compute $\delta^{(l)}$ for each layer starting from the output layer and moving backward to the hidden layer. The steps are as follows:

1. **Compute the error at the output layer**:

$$\begin{aligned} \frac{\partial E}{\partial b_i^{(l)}} = \frac{\partial E}{\partial b_1^{(2)}} = \delta^{(2)} &= \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial a_1^{(2)}} \\ &= \frac{\partial}{\partial y} \left(\frac{1}{2}(t-y)^2 \right) \cdot 1 \\ &= -(t-y) \cdot 1 \\ &= -(0.5 - 0.434) = -0.066 \end{aligned}$$

   It represents how much the output y deviates from the target t. Here we used the output activation function value y from the **cached values**.

2. **Compute gradients for weights and biases in the output layer**:

$$\underbrace{\frac{\partial E}{\partial w_{11}^{(2)}}}_{\frac{\partial E}{\partial w_{ij}^{(l)}}} = \delta^{(2)} \cdot \underbrace{a_1^{(1)}}_{a_j^{(l-1)}} = \delta^{(2)} \cdot h \approx -0.066 \cdot 0.668 \approx -0.044$$

It shows how much the weight $w_{11}^{(2)}$ should be adjusted to reduce the error. It will be used to update the weight during the optimization step. Again, we used the hidden layer activation ($h \approx 0.668$) from the **cached values**. Now, also compute the gradient for the bias that is obtained directly from the previously step (also **cached**):

$$\frac{\partial E}{\partial b_i^{(l)}} = \frac{\partial E}{\partial b_1^{(2)}} = \delta^{(2)} \approx -0.066$$

3. Compute the error at the hidden layer:

$$\delta^{(1)} = \delta^{(2)} \cdot w_{11}^{(2)} \cdot g'^{(1)}\left(a_1^{(1)}\right)$$

where $g'^{(1)}(a)$ is the derivative of the sigmoid function:

$$g'^{(1)}(a) = \sigma(a) \cdot (1 - \sigma(a))$$

Thus,

$$g'^{(1)}(0.7) = \sigma(0.7) \cdot (1 - \sigma(0.7)) \approx 0.668 \cdot (1 - 0.668) \approx 0.222$$

Where we used the **cached values** of the activation of the hidden layer $\sigma(0.7) \approx 0.668$ from the forward pass. Therefore,

$$\delta^{(1)} \approx -0.066 \cdot 0.5 \cdot 0.222 \approx -0.0073$$

4. Compute gradients for weights and biases in the hidden layer:

$$\frac{\partial E}{\partial w_{11}^{(1)}} = \delta^{(1)} \cdot a^{(0)} \approx -0.0073 \cdot 1 \approx -0.0073$$

$$\frac{\partial E}{\partial b_1^{(1)}} = \delta^{(1)} \approx -0.0073$$

Step 3: Update Weights and Biases. Finally, we update the weights and biases using the computed gradients and the learning rate $\eta = 0.1$ (all the gradients are **cached** from the backward pass):

- Update weight from hidden to output layer:

$$w_{11}^{(2)} \leftarrow w_{11}^{(2)} - \eta \cdot \frac{\partial E}{\partial w_{11}^{(2)}} \approx 0.5 - 0.1 \cdot (-0.044) \approx 0.5044$$

- Update bias of output layer:

$$b_1^{(2)} \leftarrow b_1^{(2)} - \eta \cdot \frac{\partial E}{\partial b_1^{(2)}} \approx 0.1 - 0.1 \cdot (-0.066) \approx 0.1066$$

- Update weight from input to hidden layer:

$$w_{11}^{(1)} \leftarrow w_{11}^{(1)} - \eta \cdot \frac{\partial E}{\partial w_{11}^{(1)}} \approx 0.4 - 0.1 \cdot (-0.0073) \approx 0.40073$$

- Update bias of hidden layer:

$$b_1^{(1)} \leftarrow b_1^{(1)} - \eta \cdot \frac{\partial E}{\partial b_1^{(1)}} \approx 0.3 - 0.1 \cdot (-0.0073) \approx 0.30073$$

After this single iteration of forward and backward passes, the weights and biases have been updated to better approximate the target function $y = x$ for the input $x = 1$. Repeating this process over many iterations and samples will allow the network to learn the desired mapping.

2.5 Maximum Likelihood Estimation (MLE)

In the previous sections, we discussed how to *compute* gradients (backpropagation) and *optimize* weights (gradient descent) in neural networks. But **why** do we minimize a loss function in the first place? What's the **statistical justification** for this approach? Is there a **probabilistic interpretation** of learning in neural networks? And why is it important?

We've already seen that when we train a neural network, we *minimize a loss function* $E(w)$ (page 91). MLE gives a **statistical justification** for that loss: it's equivalent to **maximizing the probability** of observing our data given the model parameters. In other words, **MLE turns learning into a probability problem**.

Definition 9: Maximum Likelihood Estimation (MLE)

Let our model have parameters θ (or w in neural networks), and our dataset be:

$$\mathcal{D} = \{x_1, x_2, \ldots, x_N\}$$

We assume that each data point is drawn from a probability distribution that depends on those parameters:

$$p\left(x_n \mid \theta\right)$$

If all samples are **i.i.d.** (independent and identically distributed) then the probability of the entire dataset is:

$$p\left(\mathcal{D} \mid \theta\right) = \prod_{n=1}^{N} p\left(x_n \mid \theta\right)$$

This is called the **likelihood function**:

$$L\left(\theta\right) = p\left(\mathcal{D} \mid \theta\right) = \prod_{n=1}^{N} p\left(x_n \mid \theta\right) \tag{46}$$

Then the **Maximum Likelihood Estimation (MLE)** is the value of θ that maximizes this likelihood:

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} L\left(\theta\right) = \arg\max_{\theta} p\left(\mathcal{D} \mid \theta\right) \tag{47}$$

In practice, we often maximize the **log-likelihood** instead:

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} \log L\left(\theta\right) = \arg\max_{\theta} \log p\left(\mathcal{D} \mid \theta\right) \tag{48}$$

Because the logarithm is a monotonic function, maximizing the log-likelihood is equivalent to maximizing the likelihood itself. Also, the logarithm allows us to turn products into sums, which are easier to work with.

What does i.i.d. mean?

The acronym **i.i.d.** stands for *independent and identically distributed*. It's a **fundamental assumption** in statistics that simplifies how we model data. When we say that our samples are i.i.d., we mean two things:

1. **Independent**. Each observation (data point) does **not depend** on the others. Formally:

$$p(x_1, x_2, \ldots, x_N) = \prod_{n=1}^{N} p(x_n)$$

   So, knowing x_1 tells us nothing about x_2, and so on. Each is separate, independent draw from the underlying process. For example, if we toss a coin 10 times, each toss is independent of the others, assuming the coin is fair and the tosses don't influence each other.

2. **Identically Distributed**. All samples come from the **same probability distribution**. Formally:

$$x_n \sim p(x \mid \theta) \quad \forall n$$

   This means they are generated by the **same model parameters** (θ) (e.g., same mean and variance in a Gaussian). For example, each coin toss has the same probability ($P(\text{Head}) = 0.5$).

So, when we say:

$$x_1, x_2, \ldots, x_N \overset{\text{are i.i.d.}}{\sim} p(x \mid \theta)$$

We mean that each sample x_n is an independent draw from the **same** distribution underlying probability parameterized by θ. This is the **standard assumption** when deriving the Likelihood:

$$p(\mathcal{D} \mid \theta) = \prod_{n=1}^{N} p(x_n \mid \theta)$$

Without the i.i.d. assumption, we couldn't factor the joint probability into a product of individual probabilities, making the analysis much more complex.

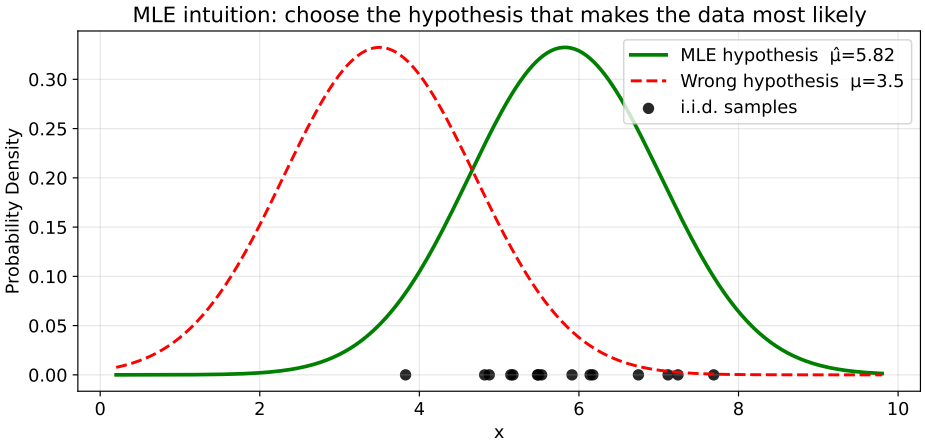
Core Idea of MLE

The core idea of MLE is to find the parameter values that make the **observed data most probable** under our assumed statistical model. By maximizing the likelihood function, we identify the parameters that best explain the data we have collected. Let's illustrate this with a simple example.

Imagine a **Gaussian distribution** (bell curve) centered around some unknown true mean $\hat{\mu}$. Each black point below the curve represents one **sample** (observation): $x_1, x_2, \ldots, x_N$. These samples:

- Come from the **same distribution** $\mathcal{N}(\mu, \sigma^2)$, so they are **identically distributed**. It means they share the same mean μ and variance σ^2.
- Are drawn **independently** from each other, so they are **independent**.

If we were to repeat the sampling many times, we'd get different sets of samples, but all coming from *the same bell curve*.



Now, MLE asks: "*which value of μ would make these observed points most likely under the Gaussian model?*" If we shift the curve too far left or right, the points no longer lie near its center, reducing their likelihood. The "best" μ is the one centering the curve on the data cloud, called $\hat{\mu}_{\text{MLE}}$. This $\hat{\mu}_{\text{MLE}}$ is the **Maximum Likelihood Estimate** of the mean.

Let's derive the MLE for the mean of a Gaussian distribution step-by-step.

1. **Model assumption**. We assume our data points are i.i.d. samples from a Gaussian distribution:

$$x_1, x_2, \ldots, x_N \sim \mathcal{N}(\mu, \sigma^2)$$

   Where σ^2 is known, and μ is the parameter we want to estimate. Each point has probability density:

$$p\left(x_n \mid \mu, \sigma^2\right) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_n - \mu)^2}{2\sigma^2}\right)$$

   The **probability density** is a function that describes the likelihood of a random variable taking on a specific value. For continuous variables, it indicates how dense the probability is around that value.

2. **Likelihood of the dataset**. The likelihood of the entire dataset, assuming i.i.d. samples, is:

$$L\left(\mu\right) = p\left(\mathcal{D} \mid \mu, \sigma^2\right) = \prod_{n=1}^{N} p\left(x_n \mid \mu, \sigma^2\right)$$

3. **Log-likelihood**. We take the logarithm of the likelihood to simplify calculations:

$$\log \ell\left(\mu\right) = -\frac{N}{2} \log\left(2\pi\sigma^2\right) - \frac{1}{2\sigma^2} \sum_{n=1}^{N} \left(x_n - \mu\right)^2$$

   Derivative with respect to μ. We differentiate the log-likelihood with respect to μ because we want to find the value of μ that maximizes it (and

the gradient tells us where the maximum is):

$$\begin{aligned}\frac{\partial \ell}{\partial \mu} &= -\frac{1}{2\sigma^2} \cdot \frac{\partial}{\partial \mu} \sum_{n=1}^{N} (x_n - \mu)^2 \\ &= -\frac{1}{2\sigma^2} \cdot \sum_{n=1}^{N} 2\,(\mu - x_n) \\ &\Rightarrow \frac{1}{\sigma^2} \cdot \sum_{n=1}^{N} (x_n - \mu)\end{aligned}$$

4. **Set derivative to zero**. To find the maximum, we set the derivative equal to zero and solve for μ, because at the maximum point, the slope of the function is zero:

$$\begin{aligned}0 &= \frac{1}{\sigma^2} \cdot \sum_{n=1}^{N} (x_n - \mu) \\ &\Rightarrow \sum_{n=1}^{N} x_n - N\mu = 0 \\ &\Rightarrow N\mu = \sum_{n=1}^{N} x_n \\ &\Rightarrow \mu = \frac{1}{N} \cdot \sum_{n=1}^{N} x_n\end{aligned}$$

   Thus, the MLE for the mean of a Gaussian distribution is the **sample mean**:

$$\hat{\mu}_{\text{MLE}} = \frac{1}{N} \cdot \sum_{n=1}^{N} x_n \tag{49}$$

This derivation shows how MLE provides a principled way to estimate model parameters by maximizing the likelihood of the observed data. In this case, it leads us to the intuitive result that the best estimate for the mean of a Gaussian is simply the average of the observed samples. In simple terms, Gaussian is the geometric intuition behind the MLE of the mean.

Applying MLE to Neural Networks

In neural networks we do something that looks like this:

$$\text{Find } w^* = \arg\min_{w} E\,(w)$$

Where $E(w)$ is the **loss function** (e.g., Mean Squared Error, Binary Cross-Entropy). MLE gives the **statistical justification** for this optimization: minimizing these loses is equivalent to maximizing the likelihood of the observed data under the model defined by the neural network with parameters w. So, **MLE defines *what* loss we should minimize**, and **gradient descent** plus **backpropagation** define ***how* to minimize it**.

1. **Assume a probabilistic model for our targets**. We assume each training example (x_n, t_n) was generated by an unknown process:

$$t_n \sim p\left(t_n \mid x_n, w\right)$$

Where:

- x_n is the input (features).
- t_n is the target (label).
- w are the model parameters (weights of the neural network).
- $p\left(t_n \mid x_n, w\right)$ is the probability of observing target t_n given input x_n and model parameters w.

2. **Apply the Maximum Likelihood principle**. We want to find weights that make our observed dataset as probable as possible:

$$\hat{w}_{\text{MLE}} = \arg\max_w p\left(\mathcal{D} \mid w\right) = \arg\max_w \prod_{n=1}^{N} p\left(t_n \mid x_n, w\right)$$

Taking the logarithm:

$$\hat{w}_{\text{MLE}} = \arg\max_w \sum_{n=1}^{N} \log\left(p\left(t_n \mid x_n, w\right)\right)$$

Or equivalently:

$$\hat{w}_{\text{MLE}} = \arg\min_w E(w)$$

Where:

$$E(w) = -\sum_{n=1}^{N} \log\left(p\left(t_n \mid x_n, w\right)\right)$$

Is the **negative log-likelihood loss**, in neural networks often called simply the **loss function** (used in training).

3. **Depending on the probabilistic assumptions, we get different loss functions**. For example:

- For Regression tasks, the probabilistic model is often Gaussian:

$$t_n \sim \mathcal{N}\left(y\left(x_n, w\right), \sigma^2\right)$$

With logarithm of the likelihood:

$$-\frac{1}{2\sigma^2} \cdot \sum_{n=1}^{N} \left(t_n - y\left(x_n, w\right)\right)^2$$

This leads to the **Mean Squared Error (MSE)** loss (page 73):

$$E(w) = \frac{1}{N} \cdot \sum_{n=1}^{N} \left(t_n - y\left(x_n, w\right)\right)^2 \qquad \underbrace{w^* = \arg\min_w E(w)}_{\text{optimize weights}}$$

- For Binary Classification tasks, the probabilistic model is often Bernoulli:

$$t_n \sim \text{Bernoulli}\left(y\left(x_n, w\right)\right)$$

With logarithm of the likelihood:

$$\sum_{n=1}^{N} \left[t_n \log y + (1 - t_n) \log\left(1 - y\right)\right]$$

This leads to the **Binary Cross-Entropy (BCE)** loss (page 77):

$$E(w) = -\frac{1}{N} \cdot \sum_{n=1}^{N} \left[t_n \cdot \ln\left(y\left(x_n, w\right)\right) + (1 - t_n) \cdot \ln\left(1 - y\left(x_n, w\right)\right)\right]$$

$$\underbrace{w^* = \arg\min_{w} E(w)}_{\text{optimize weights}}$$

- For Multi-Class Classification tasks, the probabilistic model is often Categorical:

$$t_n \sim \text{Categorical}\left(\text{softmax}(y\left(x_n, w\right))\right)$$

With logarithm of the likelihood:

$$\sum_{n=1}^{N} \sum_{c=1}^{C} t_{n,c} \log y_c$$

This leads to the **Categorical Cross-Entropy (CCE)** loss (page 83):

$$E(w) = -\frac{1}{N} \cdot \sum_{n=1}^{N} \sum_{c=1}^{C} t_{n,c} \cdot \ln\left(y_c\left(x_n, w\right)\right)$$

$$\underbrace{w^* = \arg\min_{w} E(w)}_{\text{optimize weights}}$$

So if we assume **Gaussian noise**, the MLE leads to MSE; if we assume **Bernoulli labels**, it leads to binary cross-entropy; and if we assume **Categorical labels**, it leads to categorical cross-entropy. This shows how **the choice of loss function is directly tied to our probabilistic assumptions about the data**.

In summary, MLE provides a **statistical foundation** for training neural networks by showing that minimizing common loss functions is equivalent to maximizing the likelihood of the observed data under appropriate probabilistic models. This connection helps us understand **why** we use certain loss functions and guides us in choosing the right one based on the nature of our data and task.

> **Deepening: How to Choose the Error Function?**
>
> Now that we understand where loss functions come from via MLE, the

next question is: **How do we choose the right loss function for a given problem?** The choice depends on the **type of task** (regression vs. classification) and the **underlying probabilistic assumptions** about the data.

1. **Linear**. If our model is linear and data are separable, we can use **Perceptron Loss**. This loss focuses on maximizing the margin between classes.

2. **Regression Tasks**. If we're predicting continuous values, we often assume Gaussian noise in the targets. This leads us to use the **Mean Squared Error (MSE)** loss, which corresponds to maximizing the likelihood under a Gaussian model.

3. **Binary Classification Tasks**. If we're classifying inputs into two classes, we typically model the targets as Bernoulli-distributed. This results in using the **Binary Cross-Entropy (BCE)** loss, which maximizes the likelihood under a Bernoulli model.

4. **Multi-Class Classification Tasks**. For problems with more than two classes, we assume a Categorical distribution for the targets. This leads us to use the **Categorical Cross-Entropy (CCE)** loss, which maximizes the likelihood under a Categorical model.

In practice, it's essential to align our choice of loss function with our assumptions about the data generation process. This ensures that our training procedure is statistically sound and that we're optimizing for the most appropriate objective given our specific problem.

2.6 Perceptron Learning Algorithm

After understanding how modern feed-forward neural networks are trained (through **gradient descent**, **backpropagation**, and **MLE**), we now return to the **first learning rule ever proposed** for neural models: the **Perceptron Learning Algorithm** (PLA), introduced by Frank Rosenblatt in 1957.

Why study this old algorithm after learning about modern techniques?

Because it is actually the **ancestor** of the entire modern training framework we just saw (gradient descent, error minimization, MLE, backpropagation). In simple terms, backpropagation didn't appear out of nowhere: it is a generalization of the Perceptron rule. Rosenblatt's 1957 perceptron was the **first algorithm** that *learned from data autonomously*. It introduced three ideas that survive today in modern deep learning:

1. **Weighted sum of inputs → decision**. The perceptron **computes a weighted sum of its inputs and applies a threshold to decide its output** (0 or 1). This is the basis of all neural networks. We have already formalized this as:
$$a = w^T x + b$$

2. **Error-based correction**. The perceptron **updates its weights based on the error it makes on each training example**. This is the core idea of learning from data, which we have seen in modern networks through loss functions and gradient descent.

3. **Iterative improvement**. The perceptron learning **algorithm iteratively adjusts its weights over multiple passes through the training data**, refining its decision boundary. This iterative process is fundamental to modern training algorithms, including stochastic gradient descent.

Every deep learning model, even a transformer, still rests on these same principles.

Remark: What is an algebraic hyperplane?

In $\mathbb{R}^d$ a (linear) **Hyperplane** is the set of points $x \in \mathbb{R}^d$ that satisfy a single linear equation of the form:

$$w^T x + w_0 = 0$$

Where:

- $w \in \mathbb{R}^d$ is the **normal vector** (perpendicular to the hyperplane).
- w_0 is the **bias/offset** shifting the hyperplane from the origin (or b in our notation).
- The **signed distance** of any point x from the hyperplane is given

by:

$$d(x, \Pi) = \frac{w^T x + w_0}{\|w\|}$$

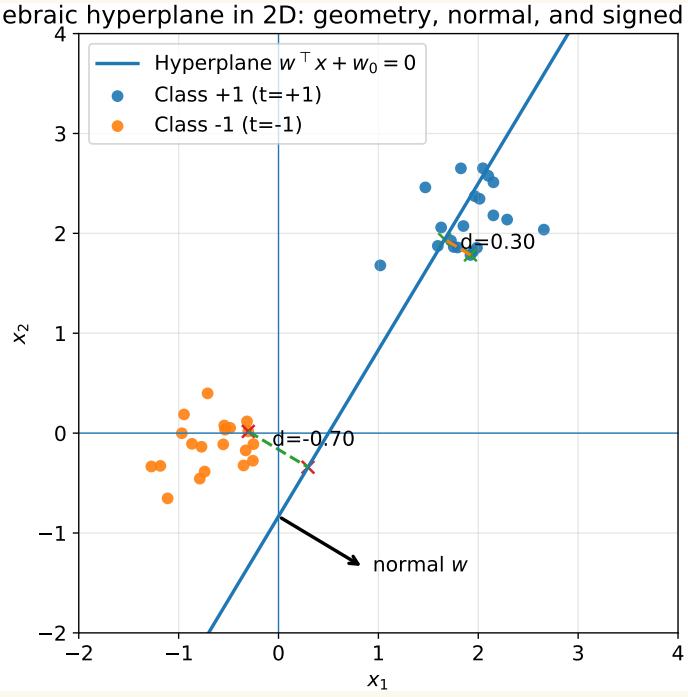
Its **sign** tells the side of the hyperplane; its **magnitude** is the perpendicular distance.

For a linear classifier (perceptron), we assign class by the sign:

$$\text{class}(x) = \text{sign}\left(w^T x + w_0\right)$$

In 2D, a hyperplane is a line (see below). In the following figure, the hyperplane separates two classes of points in $\mathbb{R}^2$. Two example points are projected perpendicularly onto the hyperplane to illustrate their distances.

Algebraic hyperplane in 2D: geometry, normal, and signed distance



The perceptron learning algorithm is designed for **binary classification tasks** where the data is **linearly separable**. It iteratively adjusts the weights of the perceptron based on the errors it makes on the training data, aiming to find a hyperplane that separates the two classes. So, we want to derive an **error function** that expresses how *wrong* the perceptron is, such that we can later minimize it using a gradient-based rule.

1. **Start from the *algebraic hyperplane***. For a linear classifier, the decision boundary is a **hyperplane**:

$$w^T x + w_0 = 0$$

Where:

- w is the normal vector to the plane (its orientation).
- w_0 is the bias term (offset from the origin).
- x is an input vector (sample).

Every point x_i can lie:

- **Above the plane** if $w^T x_i + w_0 > 0$ (positive side, class $+1$).
- **Below the plane** if $w^T x_i + w_0 < 0$ (negative side, class -1).
- **On the plane** if $w^T x_i + w_0 = 0$ (neutral).

The value calculated by $w^T x_i + w_0$ is called the **activation**, **algebraic distance** or **net input** (page 60):

$$a_i = w^T x_i + w_0$$

2. **Encode class labels as $t_i \in \{+1, -1\}$**

   - If a sample belongs to the positive class, set $t_i = +1$.
   - If it belongs to the negative class, set $t_i = -1$.

   Then the **predicted class** (the output of the perceptron) is given by:

   $$\hat{y}_i = \text{sign}\,(a_i) = \text{sign}\left(w^T x_i + w_0\right)$$

3. **Determine whether a sample is correct or misclassified**. We multiply the predicted *algebraic distance* by the true label t_i:

   $$t_i \cdot a_i \Rightarrow t_i \cdot \left(w^T x_i + w_0\right)$$

   This product indicates correctness:

Case	Expression	Meaning
Correctly classified	$t_i \left(w^T x_i + w_0\right) > 0$	Sample lies on the correct side of the hyperplane.
Misclassified	$t_i \left(w^T x_i + w_0\right) < 0$	Sample lies on the wrong side.
Exactly on the boundary	$t_i \left(w^T x_i + w_0\right) = 0$	Neutral (rare case).

   So this single product already *encodes correctness* of the classification.

4. **Define the *set of misclassified samples***. We now define the set of misclassified samples M as:

   $$M = \left\{i \mid t_i \left(w^T x_i + w_0\right) < 0\right\}$$

   These are the indices of samples that are on the wrong side of the hyperplane (i.e., misclassified).

5. **Measure "how wrong" those samples are**. Each misclassified point x_i with $i \in M$ lies at a **signed distance** from the hyperplane given by:

$$d_i = \frac{w^T x_i + w_0}{\|w\|}$$

Since the point is misclassified, $t_i \cdot \left(w^T x_i + w_0\right) < 0$, so the signed distance is **negative** (i.e., it's on the wrong side). To simplify, we can just use the **activation** $a_i = w^T x_i + w_0$ as a measure of error (ignoring the normalization by $\|w\|$). This simplification isn't a problem because it only scales the error, not its direction.

6. **Build an error function using those distances**. To quantify the total error made by the perceptron, we sum the negative activations of all misclassified samples. So, we want a scalar function $D\left(w, w_0\right)$ that:

   - Is **positive** when there are misclassifications.
   - Is **zero** when all samples are correctly classified.
   - Increases when misclassified points are further from the decision boundary.

   Simply summing the negative activations of misclassified points achieves this:

$$D\left(w, w_0\right) = -\sum_{i \in M} t_i \cdot \left(w^T x_i + w_0\right)$$

   Where:

   - $w^T x_i + w_0$ is the algebraic distance (activation) of sample i from the hyperplane.
   - t_i is the true label ($+1$ or -1) that flips the sign for misclassified points (flipping it makes it negative when misclassified).
   - The negative sign in front ensures that D is positive when there are misclassifications (since $t_i \cdot \left(w^T x_i + w_0\right) < 0$ for misclassified points).
   - The sum over $i \in M$ aggregates the errors from all misclassified samples.

   So this function is **large when many points are misclassified or far away**, and **zero when all points are correctly classified**.

7. **Objective: Minimize this error function**. The perceptron learning algorithm aims to find weights w and bias w_0 that minimize the error function $D\left(w, w_0\right)$. Formally:

$$\min_{w, w_0} D\left(w, w_0\right) = \min_{w, w_0} \left(-\sum_{i \in M} t_i \cdot \left(w^T x_i + w_0\right) \right)$$

   Minimizing D will push $t_i \cdot \left(w^T x_i + w_0\right)$ to be **positive** for all samples, meaning all points will be **correctly classified**.

Now that we have defined the error function $D(w, w_0)$ that quantifies how wrong the perceptron is, we can proceed to derive the **Perceptron Learning Algorithm** by finding a way to update the weights w and bias w_0 to minimize this error. This will involve computing the gradients of D with respect to w and w_0, and using these gradients to iteratively adjust the parameters in the direction that reduces the error. In simple terms, we have developed the recipe (the error function) and now we will derive the cooking instructions (the learning rule).

1. **Compute the gradient**. We'll compute the **partial derivatives** with respect to w and w_0:

$$\begin{aligned}\frac{\partial D}{\partial w} &= -\sum_{i \in M} t_i \cdot \frac{\partial \left(w^T x_i + w_0\right)}{\partial w} \\ &= -\sum_{i \in M} t_i \cdot x_i\end{aligned}$$

$$\begin{aligned}\frac{\partial D}{\partial w_0} &= -\sum_{i \in M} t_i \cdot \frac{\partial \left(w^T x_i + w_0\right)}{\partial w_0} \\ &= -\sum_{i \in M} t_i \cdot 1\end{aligned}$$

2. **Apply gradient descent**. Gradient descent updates parameters **in the direction opposite to the gradient** to minimize the error:

$$w^{(\text{new})} = w^{(\text{old})} - \eta \cdot \frac{\partial D}{\partial w}$$

$$w_0^{(\text{new})} = w_0^{(\text{old})} - \eta \cdot \frac{\partial D}{\partial w_0}$$

Where η is the learning rate (a small positive constant). Substituting the partial derivatives calculated earlier:

$$w^{(\text{new})} = w^{(\text{old})} - \eta \cdot \left(-\sum_{i \in M} t_i \cdot x_i\right) = w^{(\text{old})} + \eta \cdot \sum_{i \in M} t_i \cdot x_i$$

$$w_0^{(\text{new})} = w_0^{(\text{old})} - \eta \cdot \left(-\sum_{i \in M} t_i\right) = w_0^{(\text{old})} + \eta \cdot \sum_{i \in M} t_i$$

Trying to optimize over all misclassified points at once. Here, we are happy to see that the updates **add contributions from all misclassified points**, pushing the weights and bias in the direction that reduces their error (i.e., moves *all* misclassified points to the correct side of the hyperplane $\sum_{i \in M} t_i \cdot x_i$ and $\sum_{i \in M} t_i$).

However, this approach can be computationally expensive for large datasets, since it requires summing over all misclassified points at each update. In practice, it is more convenient to perform **incremental updates**, correcting the model using one misclassified sample at a time. This

leads to a **stochastic** version of the update rule, where the parameters are adjusted immediately after observing a misclassified example instead of aggregating all errors (i.e., we can update the weights and bias after processing each misclassified point, rather than waiting to compute the sum over all misclassified points). The resulting update rule is computationally simpler and allows the model to adapt continuously during the training pass through the data.

3. **Make it *stochastic***. The word "stochastic" means "*involving randomness or uncertainty*". It comes from the Greek "*stochastikos*", meaning "*able to guess*" or "*randomly determined*". So a **stochastic process** is one that includes random variables or random choices; instead, a **deterministic process** always behaves the same way for the same inputs (no randomness). In neural networks, **stochastic** means that we don't **compute the gradient** using *all* training data at once, but rather we approximate it **using a *subset*** (or even a single sample). This introduces a bit of *random noise* in each update, but makes learning faster and more dynamic.

   In perceptron learning algorithm, computing the whole sum over all misclassified points M can be expensive and inefficient, especially for large datasets. Instead, we can update **one misclassified point at a time**:

$$w^{(\text{new})} = w^{(\text{old})} + \eta \cdot t_i \cdot x_i$$
$$w_0^{(\text{new})} = w_0^{(\text{old})} + \eta \cdot t_i$$

   - $t_i \cdot x_i$ are the points in the direction that will move x_i to the correct side of the hyperplane.
   - $\eta > 0$ is the **learning rate** (small step size).

   This approach is called **Stochastic Gradient Descent (SGD)** (or **Batch Gradient Descent** with batch size 1, where the **batch size is the number of samples used to compute the gradient at each step**) because we use a single (or a few) random samples to approximate the gradient, rather than the full dataset.

   This makes the learning process more efficient and allows the perceptron to adapt quickly to new data because each update is based on the most recent misclassification and not the entire dataset at once. See the Figure 14 for an illustration of stochastic vs full-batch updates (page 120).

4. **Interpret geometrically**. We can interpret the updates geometrically:

   - **Case 1: Point correctly classified**. If $t_i \cdot \left(w^T x_i + w_0\right) > 0$, no update is made.
   - **Case 2: Point misclassified**. If $t_i \cdot \left(w^T x_i + w_0\right) < 0$, then update using:

$$w \leftarrow w + \eta \cdot t_i \cdot x_i$$
$$w_0 \leftarrow w_0 + \eta \cdot t_i$$

     This pushes the hyperplane **towards the misclassified point**, reducing its error $D\left(w, w_0\right)$.

Intuitively:

- If $t_i = +1$ (positive class), the point is misclassified, so we **add** x_i to w to move the hyperplane closer to w.
- If $t_i = -1$ (negative class) and the point is misclassified, we **subtract** x_i from w to move the hyperplane away from w.

5. **Repeat until convergence**. The **Perceptron Convergence Theorem** states that if the training data is linearly separable (i.e., there exists a hyperplane that can perfectly separate the two classes), the perceptron learning algorithm will converge to a solution (a hyperplane that separates the classes) in a finite number of steps. However, if the data is not linearly separable, the algorithm will never converge and will keep updating indefinitely. In practice, we can set a maximum number of iterations or a convergence criterion to stop the algorithm if it doesn't find a solution.

   Theorem 2 (Perceptron Convergence Theorem)**.** *If the training data is linearly separable, the perceptron learning algorithm will converge to a solution in a finite number of steps. If the data is not linearly separable, the algorithm will not converge.*

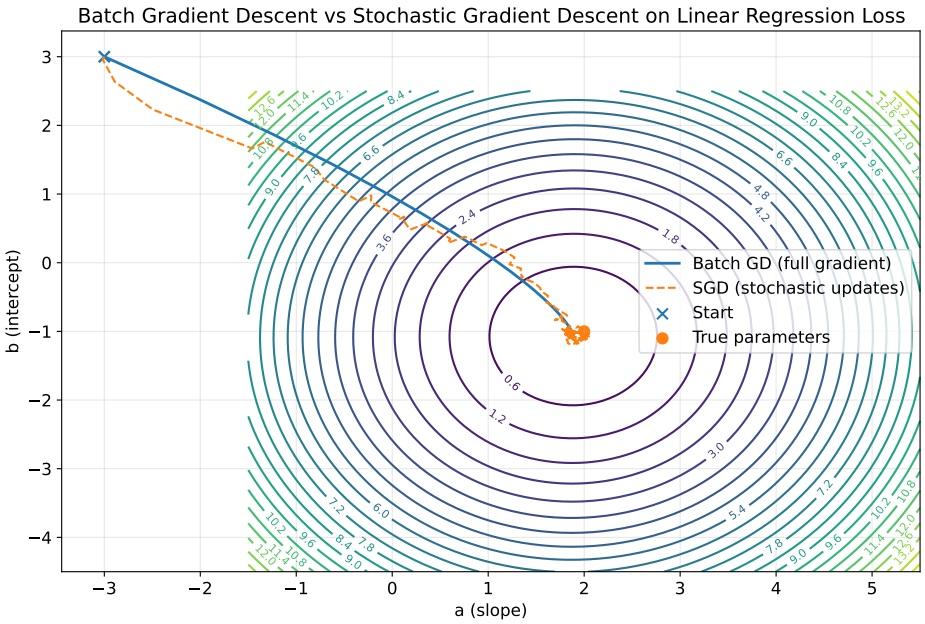


Figure 14: An illustration example of stochastic updates vs full-batch updates. The contours are the **MSE loss** for a simple linear regression in parameter space (a, b). **Batch Gradient Descent** follows a smooth path (solid line) using the **full dataset gradient** each step, while **Stochastic Gradient Descent** (dashed line) takes a more erratic path using **individual sample gradients**, leading to faster but noisier convergence.

```
// Init
Initialize w, w_0 = small random values
Set learning rate η > 0
// Training loop
while not converged:
    for each training sample (x_i, t_i):
        // Compute activation
        a_i = w^T x_i + w_0
        // If misclassified update weights and bias
        if t_i · a_i ≤ 0:
            w ← w + η · t_i · x_i
            w_0 ← w_0 + η · t_i
```

Listing 1: Perceptron Learning Algorithm

The above pseudo-code summarizes the Perceptron Learning Algorithm:

1. **Initialization**. We start by initializing the weights w and bias w_0 to small random values. This randomness helps break symmetry and allows the algorithm to explore different solutions.

2. **Training loop**. We repeat the training process until convergence (or a maximum number of iterations):

   - For each training sample (x_i, t_i), we compute the activation a_i.
   - We check if the sample is misclassified by evaluating $t_i \cdot a_i$. If it's less than or equal to zero, it means the sample is on the wrong side of the hyperplane, and we perform an update to the weights and bias to move the decision boundary closer to the misclassified point.

2.7 Summary

This section starts with the **simplest neuron** (the perceptron, page 41) and ends with the **first learning algorithm** (the Perceptron Learning Rule), showing how these early ideas evolved into the modern **feed-forward networks** that can learn complex, non-linear patterns.

1. **Where we started: the Perceptron model** (page 41). At the beginning of the chapter, we see historical context: the perceptron as the first trainable neural network model, invented by Frank Rosenblatt in 1957. It introduced the idea of adjusting weights based on errors to learn from data:

   $$y = \text{sign}\left(w^T x + b\right)$$

   This was the **first artificial neuron**, a linear classifier with a hard threshold activation function. However, it could only solve linearly separable problems and had limitations (e.g., XOR problem).

2. **Multilayer networks (FNNs)**. To overcome this limitation, we introduced **hidden layers**, **differentiable activations** (sigmoid, tanh), and **continuous outputs**. Now the model can approximate **any continuous function**, not just linear boundaries. This is what transforms the perceptron into a **Feed-Forward Neural Network (FNN)**.

3. **How do we train these networks?** That lead to:

   - **Gradient descent**: an optimization algorithm to minimize the loss function by iteratively updating weights in the direction of the steepest descent.
   - **Backpropagation**: an efficient way to compute gradients for all weights in the network using the chain rule of calculus (a sort of cache mechanism to avoid redundant calculations).
   - **Maximum Likelihood Estimation (MLE)**: a statistical framework to derive loss functions (e.g., cross-entropy for classification, mean squared error for regression) based on the likelihood of the observed data given the model parameters.

   We saw how modern NNs *learn*, layer by layer, using data and gradients.

4. **Return to the perceptron learning algorithm**. Once we understood *how modern networks learn*, we revisited the **Perceptron Learning Algorithm** as a simple case of these principles. It uses a Stochastic Gradient Descent (SGD) approach to update weights based on individual training examples.

3 Neural Networks and Overfitting

3.1 Universal Approximation Theorem

During the 1980s, researchers were trying to **understand the theoretical power** of neural networks. Before this period, many scientists were skeptical about the capabilities of neural networks: "*could neural networks really learn any kind of relationship between inputs and outputs, or were they limited to simple functions?*". In 1989-1991, a series of papers by:

- **George Cybenko (1989)**: *"Approximation by superpositions of a sigmoidal function"* [2];
- **Kurt Hornik (1991)**: *"Approximation Capabilities of Multilayer Feedforward Networks"* [4].

Proved rigorously that: even a **single hidden layer** feed-forward neural network with enough neurons and a non-linear activation (like sigmoid or tanh) can approximate **any continuous function** on a compact domain of $\mathbb{R}^n$ to any desired degree of accuracy. This result gave **mathematical legitimacy** to neural networks, showing that they are not just *pattern machines*, but theoretically *universal function approximators*.

Formal Statement

Let's formalize the theorem.

Theorem 3 (**Universal Approximation Theorem**)**.** *Let $f : \mathbb{R}^n \to \mathbb{R}$ be a* ***continuous*** *function defined on a* ***compact set*** *$K \subset \mathbb{R}^n$.*

Then, for any smaller number $\varepsilon > 0$, there exists a neural network function $F(x)$ of the form:

$$F(x) = \sum_{j=1}^{m} \alpha_j \cdot g\left(w_j^T x + b_j\right) \tag{50}$$

Such that:

$$|F(x) - f(x)| < \varepsilon \quad \forall x \in K \tag{51}$$

Where:

- *$g(\cdot)$ is a* ***nonlinear, continuous, bounded activation function*** *(e.g., sigmoid, tanh);*
- *α_j, w_j, b_j are the network parameters (α_j are the* ***output layer weights****, w_j are the* ***input weights for the hidden layer****, and b_j are the* ***biases****).*
- *m is the number of neurons in the hidden layer.*
- *ε is the desired approximation accuracy.*

In other words, with ***enough hidden neurons****, a simple feedforward neural network can represent* ***any function****, no matter how complex.*

Intuition

The Universal Approximation Theorem tells us that **neural networks are incredibly powerful function approximators**. Even with just a **single hidden layer**, they can learn to represent **any continuous function** to an arbitrary degree of accuracy, as long as we provide enough neurons. This is because the non-linear activation functions allow the network to **combine simple building blocks** (the outputs of individual neurons) into **complex structures** that can capture intricate patterns in the data. However, it's important to note that while the theorem guarantees the existence of such a network, it does not provide a practical way to find the right architecture or parameters, nor does it address issues like **overfitting** or **generalization** to unseen data.

Geometric Intuition. Geometrically, each neuron in the hidden layer defines a **non-linear "bump"** or "ridge" in the input space. By combining enough of these bumps (weighted by α_j), the network can **shape its output** to follow any desired surface. Visually, if we had a 2D function $f(x_1, x_2)$, each hidden neuron adds a small deformation to the surface. Stacking many of them yields a highly flexible model:

$$f(x_1, x_2) \approx \sum_{j=1}^{m} \alpha_j \cdot g\left(w_{j1} \cdot x_1 + w_{j2} \cdot x_2 + b_j\right)$$

So, neural networks are **universal sculptors** of mathematical functions, capable of molding their outputs to fit any continuous shape we desire.

Practical Implications

The theorem tells us **existence**, not **constructability** (it says a perfect network *exists*, but not how to find its weights efficiently). In practice, we rely on **training algorithms (like backpropagation)**, which may get stuck in local minima or underfit/overfit the data. Also, even though a single layer is theoretically enough, **deep networks** (many layers) cam approximate the same function with *fewer neurons* and *more efficient representations*. This is why *deep learning* became the dominant paradigm.

Ockham's Razor and Model Simplicity

The idea dates back to **William of Ockham (c. 1285-1349)**, a Franciscan friar and philosopher who formulated a logical and methodological principle still fundamental to science and machine learning "*Entia non sunt multiplicanda praeter necessitatem*" (entities must not be multiplied beyond necessity). In essence, prefer the **simplest explanation** that fits the data. This became known as **Ockham's Razor**, where "razor" metaphorically represents the act of **cutting away unnecessary complexity**.

In the context of Neural Networks and Deep Learning, Ockham's Razor suggests that when building models, we should **favor simpler architectures** that adequately capture the underlying patterns in the data without introducing unnecessary complexity. In other words, among all models that can explain the

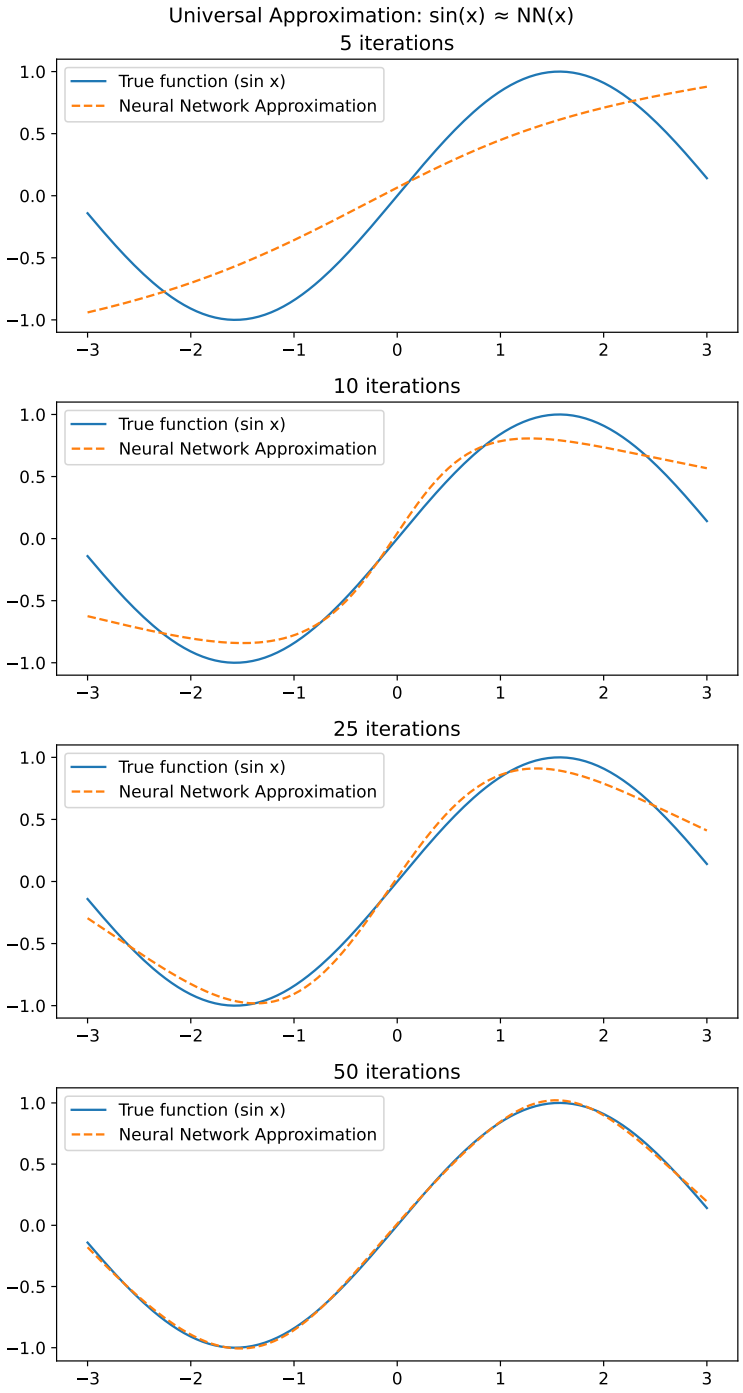


Figure 15: This grid of four plots illustrates how a simple 1-hidden layer network (with 20 neurons) can approximate a nonlinear function, such as a sine wave. This simple 1-hidden layer neural network learns to approximate $\sin(x)$ even though we never told it what "sine" is. Also, this example shows how increasing the number of iterations allows the network to better fit the target function.

training data, choose the one with the **lowest complexity** that still generalizes well to unseen data. Because neural networks are universal approximators, they can model **almost anything**, including true underlying patterns and random noise. But this flexibility is dangerous: a too-powerful model may **memorize** the training data rather than **learn patterns**, leading to **overfitting**. However, a model that is too simple may **underfit**, failing to capture important structures in the data. Thus, Ockham's Razor guides us to find a **balance** between simplicity and complexity, aiming for models that are just complex enough to capture the true patterns without overfitting.

Remark: What is Overfitting?

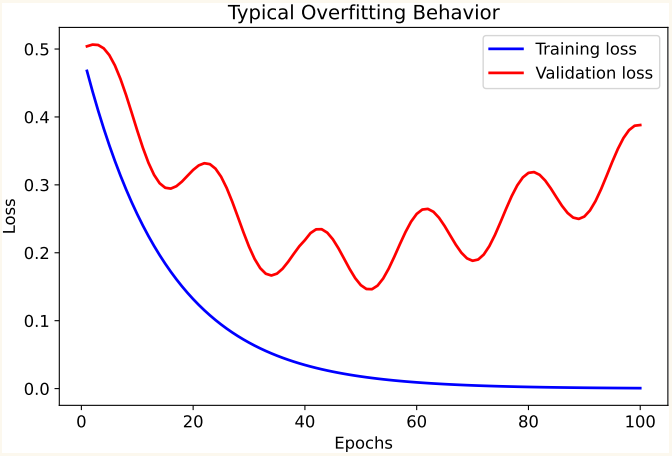
Overfitting occurs when a **model learns** not just the **true patterns** in the data, **but also the random noise**. It's like memorizing answer for an exam instead of understanding the subject; we do well on the questions we saw before (training data), but fail on new ones (test data). Imagine fitting a curve to data points:

Fit Type	Description
Underfitting	The model is too simple (e.g., a straight line when the pattern is curved), it misses important structure.
Good Fit	The model captures the true pattern without being too complex.
Overfitting	The model bends and twists to go exactly through every training point, even if those points contain random noise.

Our model is overfitting when we observe the following:

- **Training error**: very *low*
- **Test error**: *high*

The model performs **too well** on the training set because it's **memorized it**, not generalized it. Usually, we can spot overfitting by plotting training and test errors over time:



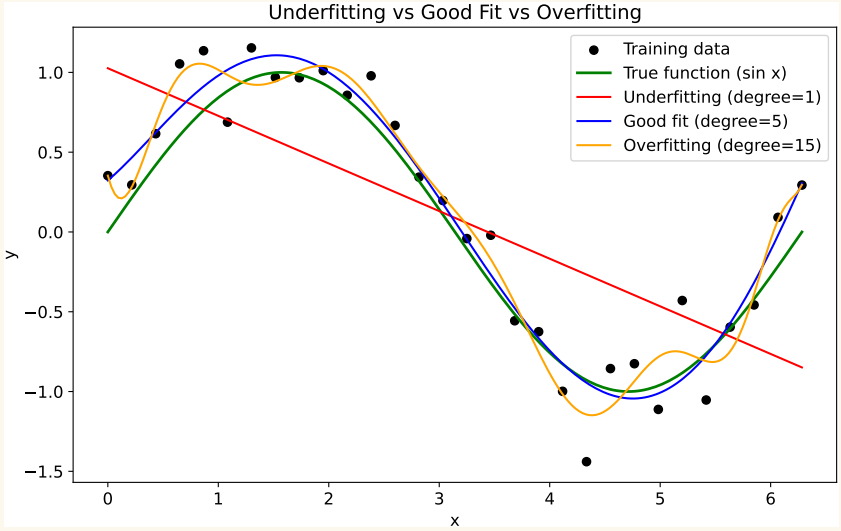
Remark: What is Underfitting?

Underfitting occurs when a model is **too simple** to capture the underlying structure or patterns of the data. It performs poorly not only on unseen (test) data but **also on the training data**. Formally, if $f(x)$ is the true function we want to learn, and $\hat{f}$ is our model's prediction, underfitting means that:

$$\hat{f}(x) \text{ cannot approximate } f(x) \text{ even on the training set}$$

So, the **training error remains high**, and naturally the **test error** will be high as well.

We can think of underfitting as using a **too rigid model** for a complex relationship. For example, trying to fit a **straight line** to data that follows a **sinusoidal** curve, or training a neural network with **too few neurons/layers**, so it can't represent the non-linearities of the data. The model simply **doesn't have enough capacity** (parameters, complexity, expressiveness) to learn the data's structure.



In a neural network context, underfitting can occur when:

- It has **too few neurons** or **layers** to represent the data completely.
- The **training time** is too short → not enough gradient updates to learn the patterns.
- The **learning rate** is too high → never converges properly.
- Strong **regularization** (e.g., large weight decay, dropout) limits learning capacity.
- Features are **not informative** (bad processing, missing normalization).

3.2 Model Complexity

Machine learning, and neural networks in particular, are based on an **inductive assumption**: "if a model performs well on a **large and representative set of training examples**, then it will also perform well on **unseen examples** drawn from the same distribution". This is called **Inductive Hypothesis** (or **Inductive Bias**) because it assumes that the patterns learned from the training data will generalize to new data.

In other words, we **trust that patterns learned from the training data generalize** to future data, as long as the data are **independent and identically distributed (i.i.d.)** (page 108), and the model captures the **true underlying structure** rather than random noise.

Formally, let

$$\hat{E}_{\text{train}} = \frac{1}{N} \sum_{i=1}^{N} \left(f(x_i; w) - t_i \right)^2$$

be the *empirical training error* computed on the training dataset (i.e., the average squared error over the training examples), and let

$$E = \mathbb{E}_{(x,t)\sim D} \left[\left(f(x; w) - t \right)^2 \right]$$

be the *expected (true) error* over the underlying data distribution D.

The **inductive hypothesis** assumes that, if the training data are **i.i.d.** samples from D and the model doesn't overfit, then:

$$\hat{E}_{\text{train}} \approx E$$

In other words, the empirical training error $\hat{E}_{\text{train}}$ is a good estimate of the true error E. Since the test set is also drawn from the same distribution D, its error provides an estimate of E. Therefore, under these assumptions:

$$E_{\text{test}} \approx E \approx \hat{E}_{\text{train}}$$

In simple terms, if the model is trained on a large and representative dataset, and the data are i.i.d., then the training error should be a good proxy for the test error, allowing us to **trust that good performance on the training set will translate to good performance on unseen data**.

⚠ If these assumptions break (e.g., due to *high model complexity*, *small dataset*, or *distribution shift*), then this approximation fails and the model does not generalize well.

? How can we quantify model complexity?

The **complexity** of a neural network is determined by several factors, including:

- The **number of parameters** (weights and biases) in the network: More parameters generally mean higher complexity.

- The **depth** of the network (number of layers): Deeper networks can capture more complex patterns.
- The **nonlinearities** introduced by activation functions: More complex activation functions can increase the model's capacity to learn intricate patterns.
- The **regularization** applied. Regularization is a technique used to reduce model complexity and prevent overfitting.

As complexity increases, the model's **capacity** to fit the data grows.

Not too complex, not too simple: how to find the right balance?

Finding the right model complexity is crucial for good generalization. This involves balancing:

- **Underfitting**: When the model is too simple to capture the underlying patterns in the data, leading to high bias and poor performance on both training and test data.
- **Overfitting**: When the model is too complex and captures noise in the training data, leading to low training error but high test error (high variance).

This balance is often referred to as the **Bias-Variance Trade-off**, a mathematical intuition that helps explain the relationship between model complexity, bias, and variance:

$$E_{\text{test}} = E\left[\left(y - \hat{f}(x)\right)^2\right] = \underbrace{(\text{Bias}^2)}_{\text{Systematic error}} + \underbrace{\text{Variance}}_{\text{Sensitivity error}} + \underbrace{\text{Noise}}_{\text{Irreducible error}} \tag{52}$$

Where:

- E_{test} is the expected test error.
- y is the true output.
- $\hat{f}(x)$ is the model's prediction.
- **Bias** is the systematic error introduced by approximating a real-world problem with a simplified model. It measures **how far our model is from the true function** on average:

  $$\text{Bias}(x) = \mathbb{E}\left[\hat{f}(x)\right] - f(x)$$

  High bias can cause the model to miss relevant relations between features and target outputs (underfitting, learning **too little**). For example, a linear model trying to fit a highly nonlinear relationship will have high bias.

- **Variance** is the sensitivity error due to fluctuations in the training data. It measures **how much our model changes if we change the dataset**:

$$\text{Var}(x) = \mathbb{E}\left[\left(\hat{f}(x) - \mathbb{E}\left[\hat{f}(x)\right]\right)^2\right]$$

  High variance can cause the model to model the random noise in the training data rather than the intended outputs (overfitting, learning **too much / too specifically**). For example, a very deep neural network with many parameters may fit the training data perfectly but perform poorly on unseen data.

- **Noise** is the irreducible error inherent in the data itself, which cannot be eliminated by any model.

  For example, measurement errors or inherent randomness in the data generation process contribute to noise.

The goal is to find a model complexity that minimizes the total error, which is the sum of bias and variance, paying attention to underfitting (high bias, low variance) and overfitting (low bias, high variance). In practice, the **inductive hypothesis is what makes machine learning possible**, **model complexity decides whether this hypothesis *holds* or *breaks***, and the **bias-variance trade-off provides a framework to understand and manage this balance**.

Model Complexity	Bias	Variance
Low (Underfitting)	High	Low
Optimal	Balanced	Balanced
High (Overfitting)	Low	High

Table 1: This table summarizes the relationship between model complexity, bias, and variance. As model complexity increases, bias tends to decrease while variance tends to increase. The optimal model complexity is where bias and variance are balanced, minimizing the total error.

Definition 1: Inductive Hypothesis

The **Inductive Hypothesis** (or **inductive bias**) is the fundamental assumption that **a model that performs well on the training data will also perform well on unseen data**, proved that both come from the **same underlying distribution**.

In mathematical terms:

$$E_{\text{test}} \approx E_{\text{train}} \quad \text{if} \quad D_{\text{train}} \sim D_{\text{test}} \tag{53}$$

Where E_{train} and E_{test} are the expected training and test errors, respectively, and D_{train} and D_{test} are the training and test data distributions; the $\sim$ symbol indicates that both datasets are drawn from the same distribution.

This assumption underlies *all* machine learning: without it, no matter how low the training error is, we would have no reason to believe the model will generalize to new data.

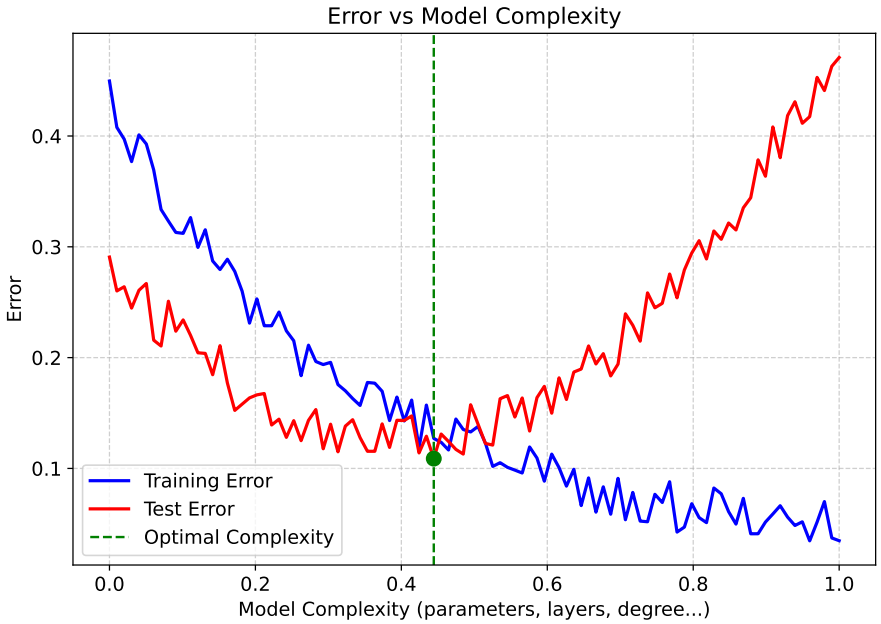


Figure 16: This plot illustrates the **Model Complexity vs. Error Curve**, showing how **training** and **test** errors evolve as **model complexity** increases, giving the classic U-shaped curve for test error. Initially, as model complexity increases, both training and test errors decrease, indicating better fit to the data. However, beyond a certain point (the optimal complexity), the test error starts to increase due to overfitting, while the training error continues to decrease. The optimal model complexity is where the test error is minimized, balancing bias and variance effectively.

3.3 Measuring Generalization

When we train a neural network, we typically monitor the **training loss** (i.e., how well the model predicts the training data). However, a **low training error** does *not* necessarily mean that our model is generalizing well to unseen data.

⚠ **The model has *seen* the training data.** The model's parameters were **directly optimized** to minimize that same error. So it's like asking a student to re-solve the same exercises used in the exam preparation. Success there tells us *nothing* about their ability to handle new ones.

$$E_{\text{train}} = \text{Empirical Risk (on known data)}$$

$$E_{\text{test}} = \text{True Risk (on unseen data)}$$

⚠ **Optimism bias.** Since the model *learned from* those points, the estimate of performance on that data is **optimistically biased**, it always looks better than reality.

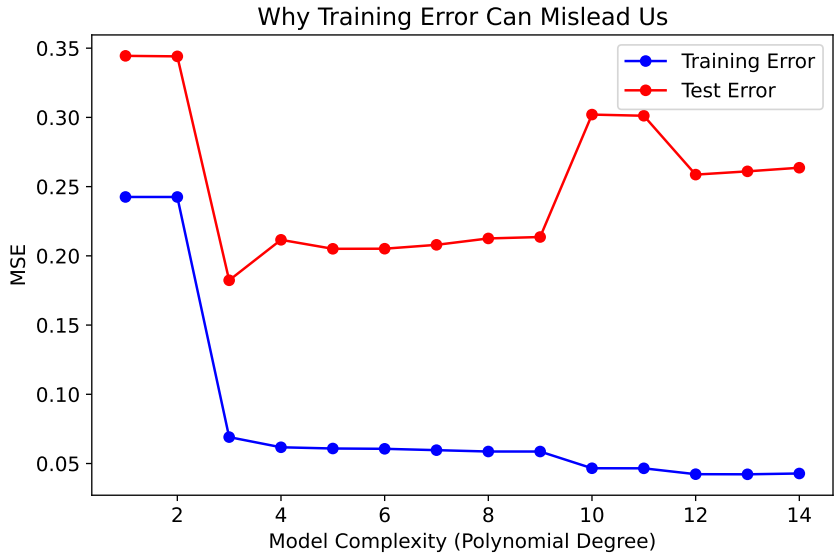
$$E_{\text{train}} \leq E_{\text{test}}$$

Always true for flexible models that can overfit the training data. The more complex the model, the stronger the bias (the model can fit noise, artificially lowering training error E_{train}).

⚠ **Example.** Imagine fitting a very flexible polynomial to noisy data:

- With degree 1 (linear), training error is high (underfitting).
- With degree 15, training error goes to zero, but the curve oscillates widly. Test error on unseen data is huge (overfitting).

That's why we cannot rely on training error E_{train} to assess generalization. It **doesn't measure generalization**, only memorization of training data.



How to measure generalization?

To correctly **measure generalization**, we must evaluate on data the model **has never seen during training**. This is done by **dataset splitting**. We divide the available dataset D into disjoint subsets:

Set	Purpose	Size
Training set	Learn model parameters (weights, biases).	During training.
Validation set	Tune hyperparameters, monitor overfitting.	During training.
Test set	Asses final generalization performance.	After training.

The goal is:

$$\begin{aligned} \text{Training data} &\rightarrow \text{Model fitting} \\ \text{Validation data} &\rightarrow \text{Model selection} \\ \text{Test data} &\rightarrow \text{Model assessment} \end{aligned}$$

This ensures that the **test error** E_{test} is an **unbiased estimate** of the model's true generalization performance on unseen data.

Okay, but how to split the data?

The most common approach is **Random Subsampling** (or Hold-Out Method):

1. Randomly **shuffle** the **dataset** D.
2. Randomly **split** our **dataset** into three disjoint subsets, for example:
   - 70% for training,
   - 15% for validation,
   - 15% for testing.
3. **Train** on the training set.
4. **Tune** on the validation set.
5. **Evaluate** once on the test set.

Because of the randomness in sampling, we often repeat the split several times (with different seeds) and **average** the results to reduce bias. This process is, of course, automated using libraries like `scikit-learn` (Python), not done manually.

Use Stratified Sampling. Imagine we have a dataset for binary classification with 90% of class A and 10% of class B. That means 90% of our data are Class A and only 10% are Class B (**class imbalance**). If we randomly split our dataset into training and test sets (say, 80% training, 20% test), there's a **risk** that one of the splits contains *almost no examples of Class B*. To avoid this, we use **Stratified Sampling**, which ensures that **each subset (train, validation and test) preserves the same class properties as the original dataset**. In our example, both training and test sets would have approximately 90% of Class A and 10% of Class B, maintaining the class distribution.

3.4 Terminology Clarifications

In the context of neural networks and deep learning, certain terms are often used interchangeably or may have nuanced meanings depending on the context. Here are some clarifications on commonly used terminology:

- **Training Dataset**. Let's start from the whole thing we have available to us. The **training dataset** (sometimes called the *available dataset*) is the complete collection of samples we can access for building and evaluating our model. It contains all our labeled examples:

  $$\mathcal{D} = \{(x_i, t_i)\}_{i=1}^{N}$$

  But we will **not** train our model on all of them at once. We'll split them into smaller subsets with different purposes.

- **Training Set**. The **training set** is the portion of the data used to **fit the model parameters** (i.e., to adjust the weights and biases so that the network learns patterns). It is used during backpropagation and gradient descent. The loss computed on this set is called the **training loss** and drives weight updates. Performance on this set tells us if the model is *learning*, but **not if it generalizes well**.

  $$E_{\text{train}} = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \left(t_i - f\left(x_i; w\right)\right)^2$$

  We *can* monitor the training loss over epochs to see if the model is converging, but it's not sufficient to decide if the model is "good" (we'll soon use validation for that).

- **Validation Set**. The **validation set** is used to **evaluate and tune** the model *during* training, without directly affecting the weights. It's like a "preview" of how the model will perform on new data. The main purposes of the validation set are:

  - Selecting **hyperparameters** (like learning rate, number of neurons, regularization, dropout rate, etc.).
  - Performing **early stopping** (detecting overfitting by monitoring validation loss).
  - Comparing different model architectures.

  We typically compute a **validation loss** or **validation accuracy** after each epoch:

  $$E_{\text{val}} = \frac{1}{N_{\text{val}}} \sum_{i=1}^{N_{\text{val}}} \left(t_i - f\left(x_i; w\right)\right)^2$$

  When validation error starts increasing while training error decreases, it's a sign of **overfitting**.

- **Test Set**. The **test set** is used *only once*, at the very end, to obtain an **unbiased estimate of the models generalization performance**. It acts as a *simulation of the real world*: the model has never seen these

samples during training or validation. No gradient updates or hyperparameter tuning should be done based on test set performance. Its purpose is **final assessment** only.

$$E_{\text{test}} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \left(t_i - f\left(x_i; w\right)\right)^2$$

After we've looked at the test performance, we should not go back and tune hyperparameters, otherwise the test set stops being "unseen" data, and our estimate becomes optimistically biased.

- **Golden Rule**: Always keep the test set completely separate until the very end. Use training and validation sets for model development, and only use the test set for final evaluation. So, never use validation/test data to update model weights. Otherwise, we risk **data leakage**.

Definition 2: Data Leakage

Data Leakage occurs when **information from outside the training dataset is used to create the model**, leading to overly optimistic performance estimates. This can happen if:

- The test set is accidentally used during training or hyperparameter tuning.
- Features that won't be available in real-world scenarios are included in the training data.
- The same data points are present in both training and test sets.

⚠ Data leakage can cause a model to perform well on the test set but fail in real-world applications, as it has effectively "cheated" by learning from information it shouldn't have access to.

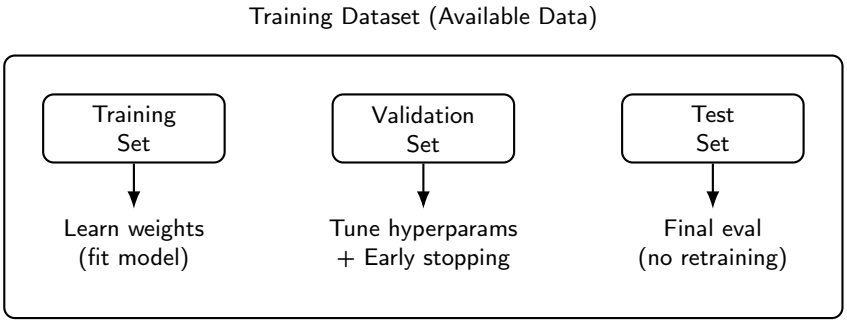


Figure 17: Visualization of the relationships between training, validation, and test sets within the overall training dataset.

3.5 Cross-Validation Techniques

Cross-Validation is the use of the **training dataset** to both: train the model (*parameter fitting* and *model selection*), and **estimate its error on new data**. In other words, we don't need a separate external test dataset for every trial, we can reuse the available data intelligently (e.g., part for training, part for checking generalization).

Main Idea. Instead of holding out a single fixed portion (like in the simple *hold-out* method), cross-validation systematically **rotates** which samples are used for training and which for validation. Each sample eventually acts as validation data once and training data many times. Thus, we can:

- **Train multiple models** on different training subsets;
- **Validate** each on the complementary subset;
- **Average the validation errors** to get a more robust estimate of the model's true performance on unseen data.

It is especially useful when the dataset is limited, as it maximizes the use of available data for both training and validation. However, other techniques (like hold-out, LOOCV, K-Fold) are specific implementations of cross-validation with different trade-offs in terms of bias, variance, and computational cost. In the following, we explore some of these techniques.

3.5.1 Hold-Out Validation

The **Hold-Out Validation** (or **Hold-Out Method**) is the simplest form of cross-validation. It consists of dividing the available dataset into **two or three subsets**, each serving a specific role in training, tuning, and evaluating the model. This method provides a first, practical way to test the **inductive hypothesis**, that a model performing well on unseen data will also generalize to future examples (page 128).

How does it work?

In the typical **three-way split**, we start from the **training dataset**, i.e., all the data we can use to develop the model, we then split it into disjoint subsets:

- **Training Set**: This subset is used to **parameter fitting**, i.e., to train the model by adjusting its weights based on the input-output pairs.
- **Validation Set**: This subset is used to **model selection**, i.e., to evaluate different model configurations (e.g., architectures, hyperparameters) and select the best one based on its performance on this set.
- **Test Set** (hold-out): This subset is used to **model assessment**, i.e., to provide an unbiased evaluation of the final model's performance on unseen data after training and validation are complete.

The steps are as follows:

1. **Divide** the available dataset into training and validation (and possibly test) sets.
2. **Train** the neural networks using only the training set.
3. **Validate** periodically on the validation set to monitor generalization:
    - Tune hyperparameters (e.g., learning rate, architecture) based on validation performance.
    - Apply *early stopping* if validation error starts to increase (indicating overfitting).
4. **Assess** the final model on the hold-out test set (data not seen during training) to estimate its true generalization error.

The hold-out validation gives an estimate of how well the model performs on *unseen data* by simulating future inputs using the reserved portion of the dataset. It's essentially a "**miniature deployment test**" performed before real-world use.

Risks and Limitations

While hold-out validation is straightforward and easy to implement, it has some limitations. The main risk is that **hold-out validation can be biased** depending on how the data are split:

1. **Non-representative sampling**
   - The validation set may not accurately reflect the data distribution.
   - The estimated generalization error may be too optimistic or too pessimistic.
2. **Small datasets**
   - If we hold out too many samples, there are too few left for training.
   - If we hold out too few, the validation estimate becomes noisy and unreliable.
3. **Unbalanced classes (classification case)**
   - If the classes are not represented equally in the training and validation sets, the model may not learn to generalize well across all classes.

   ✔ **Solution:** use **stratified sampling** (page 133) to maintain class proportions in each subset.
4. **Single split variability**
   - A different random split can yield a different result.
   - The error estimate depends too much on "which data" ended up in the validation set.

   ✔ **Solution:** Later, we will see more robust techniques (like **K-Fold Cross-Validation**) that mitigate this issue by averaging results over multiple splits.

When to use Hold-Out Validation?

Hold-out validation is most appropriate when:

- ✔ When the **dataset is large enough** to afford separate training, validation and test sets without sacrificing training data. With large enough we mean at least a few thousand samples.
- ✔ When we want **fast model evaluation** without the computational overhead of more complex cross-validation methods (only one training phase).
- ✔ When small fluctuations in the validation split are not expected to significantly affect the model selection process.

3.5.2 Leave-One-Out Cross-Validation (LOOCV)

Leave-One-Out Cross-Validation (LOOCV) is a special case of *K-Fold Cross-Validation* (next section) where the number of folds K is equal to the number of samples N in the dataset. It means that each data point acts **once** as validation data, and $N-1$ times as part of the training set.

How does it work?

1. Given a dataset with N samples:

$$\mathcal{D} = \{(x_1, t_1), (x_2, t_2), \ldots, (x_N, t_N)\}$$

2. For each sample $i = 1, 2, \ldots, N$:

   - Train the model on all samples except the i-th one:

$$\mathcal{D}_{\text{train}}^{(i)} = \mathcal{D} \setminus \{(x_i, t_i)\}$$

   - Validate (test) the model on the i-th sample:

$$\mathcal{D}_{\text{val}}^{(i)} = \{(x_i, t_i)\}$$

3. Collect all validation errors E_i from each iteration, and compute the **average error**:

$$\hat{E}_{\text{LOOCV}} = \frac{1}{N} \cdot \sum_{i=1}^{N} E_i$$

   This average error $\hat{E}_{\text{LOOCV}}$ gives an **almost unbiased estimate** of the model's generalization error.

```
Input:
    // Dataset composed of N samples,
    // where each sample is a pair of input and target output
    Dataset 𝒟 = {(x_1, t_1), (x_2, t_2), ..., (x_N, t_N)}
    Learning algorithm 𝒜
    Error function ℒ(t̂, t)

Initialize total error E ← 0

// For each sample in the dataset
for i = 1 to N:
    // Create validation set using the i-th sample
    𝒟_val ← {(x_i, t_i)}
    // Create training set by excluding the i-th sample
    𝒟_train ← 𝒟 \ {(x_i, t_i)}

    Train the model:
        // Train a model f^(i) using the learning algorithm 𝒜
        // on the training set 𝒟_train
        f^(i) ← 𝒜(𝒟_train)

```

```
    Compute prediction on validation sample:
        // Predict the output for the validation sample x_i
        // using the trained model f^(i)
        t̂_i ← f^(i)(x_i)

    Compute validation error:
        // Compute the error E_i for the validation sample
        // using the error function L
        E_i ← L(t̂_i, t_i)

    Accumulate error:
        E ← E + E_i

Compute average LOOCV error:
    // The final estimate of the generalization error is
    // the average of the validation errors
    Ê_LOOCV ← (1/N) E

Output:
    Ê_LOOCV
```

Listing 2: Leave-One-Out Cross-Validation Algorithm

⚠ Risks and Limitations

While LOOCV maximizes data usage for training and provides a nearly unbiased error estimate, it has some drawbacks:

1. **Computational Cost**. We must train the model N times (once for each sample), which can be **prohibitively expensive** for large datasets or deep neural networks.
2. **High variance in models**. Each training set differs by only one sample, so the models can be **very similar**. This can lead to high variance in the validation errors, making the average error estimate less stable.
3. **Not practical for deep learning**. Neural networks often require thousands of training iterations to converge, making LOOCV impractical for large-scale problems.

When to use LOOCV?

LOOCV has an **unbiased estimate** (each data point serves as validation once, so every sample influences the estimate equally) and an **efficient use of data** (almost all samples are used for training in each iteration, ideal when data are scarce). Thanks to these properties, LOOCV is particularly useful when:

- ✔ When N is **small** (e.g., a few hundred samples or less).
- ✔ When we want an **almost unbiased** generalization error estimate.
- ✔ When computation time is **not a concern** (e.g., simple models).

LOOCV approximates the *true expected error* E_{test} and it's nearly unbiased because each sample plays both roles, training and validation, but it tends to have **high variance** because each training set is almost identical, leading to similar models. However, for modern deep learning, it's mostly of **theoretical interest**, a conceptual benchmark rather than a practical tool.

3.5.3 K-Fold Cross-Validation

K-Fold Cross-Validation divides the available dataset into K equally (or nearly equally) sized subsets, called *folds*. The model is trained and validated K times, each time using a different fold as the **validation set**, and the remaining $K-1$ folds as the **training set**. After completing all K rounds, the K validation errors are **averaged** to estimate the model's overall generalization performance.

How does it work?

1. Given a dataset with N samples:

$$\mathcal{D} = \{(x_1, t_1), (x_2, t_2), \ldots, (x_N, t_N)\}$$

2. Split the dataset $\mathcal{D}$ into K *folds* (disjoint subsets):

$$\mathcal{D} = \mathcal{D}_1 \cup \mathcal{D}_2 \cup \ldots \cup \mathcal{D}_K$$

   Where each fold $\mathcal{D}_k$ contains approximately $\frac{N}{K}$ samples:

$$|\mathcal{D}_k| \approx \frac{N}{K}, \quad \text{for } k = 1, 2, \ldots, K$$

3. For each fold $k = 1, 2, \ldots, K$:

   - Train the model on the other $K-1$ folds (i.e., all folds except the k-th one, mathematically $\mathcal{D} \setminus \mathcal{D}_k$):

$$\mathcal{D}_{\text{train}}^{(k)} = \bigcup_{\substack{j=1 \\ j \neq k}}^{K} \mathcal{D}_j$$

   - Validate (test) the model on the k-th fold:

$$\mathcal{D}_{\text{val}}^{(k)} = \mathcal{D}_k$$

   - Compute the validation error $\hat{e}_k$ on the validation set $\mathcal{D}_{\text{val}}^{(k)}$.

4. Collect all validation errors E_k from each iteration, and compute the **average error**:

$$\hat{E}_{\text{K-Fold}} = \frac{1}{K} \cdot \sum_{k=1}^{K} \hat{e}_k$$

   This average error $\hat{E}_{\text{K-Fold}}$ provides an estimate of the model's generalization error.

```
Input:
    Dataset 𝒟 = {(x_1, t_1), ..., (x_N, t_N)}
    Number of folds K
    Learning algorithm 𝒜
```

```
    Error function $\mathcal{L}(\hat{t}, t)$

Split $\mathcal{D}$ into $K$ disjoint folds:
    $\mathcal{D} = \mathcal{D}_1 \cup \cdots \cup \mathcal{D}_K$

Initialize total error $E \leftarrow 0$

for $k = 1$ to $K$:
    // Create validation set using the $k$-th fold
    $\mathcal{D}_{\text{val}} \leftarrow \mathcal{D}_k$
    // Create training set by excluding the $k$-th sample
    $\mathcal{D}_{\text{train}} \leftarrow \mathcal{D} \setminus \mathcal{D}_k$

    Train the model:
        $f^{(k)} \leftarrow \mathcal{A}(\mathcal{D}_{\text{train}})$

    Compute validation error on fold:
        // Average error over all samples
        // in the validation fold
        $E_k \leftarrow \frac{1}{|\mathcal{D}_k|} \sum_{(x_i, t_i) \in \mathcal{D}_k} \mathcal{L}\left(f^{(k)}(x_i), t_i\right)$

    Accumulate error:
        $E \leftarrow E + E_k$

Compute average K-Fold error:
    // Average the accumulated error over all folds
    $\hat{E}_{\text{K-Fold}} \leftarrow \frac{1}{K} E$

Output:
    $\hat{E}_{\text{K-Fold}}$
```

Listing 3: K-Fold Cross-Validation Algorithm

Differently from LOOCV, K-Fold Cross-Validation allows us to choose a smaller K:

- **5-Fold Cross-Validation** is the most commonly used value, balancing bias and variance in the error estimate.
- **10-Fold Cross-Validation** is also popular, especially in scenarios where more data is available, providing a slightly lower bias at the cost of increased computational time.
- **Stratified K($K = N$)-Fold Cross-Validation** reduces to LOOCV, where each fold contains exactly one sample. Unbiased but computationally expensive.
- **2 or 3-Fold Cross-Validation** can be used for very large datasets where computational efficiency is a concern, but may lead to higher variance in the error estimate.

In general, the K-Fold Cross-Validation provides a trade-off between bias and variance in the error estimate, as well as computational cost. Small K (e.g., 2 or 3) leads to higher bias but lower variance, while large K (e.g., LOOCV) leads to lower bias but higher variance and computational cost.

⚠ Limitations and ✔ Advantages

Compared to Hold-Out, it uses the entire dataset more efficiently because each sample is used for validation once and for training $K - 1$ times (lower bias). Also, **compared to LOOCV**, it is much cheaper computationally (only K trainings instead of N).

- ✔ **Efficient data usage**: all samples contribute to both training and validation.
- ✔ **Reduced bias**: the averaged approximates the expected generalization error better than a single split.
- ✔ **Stability**: more reliable than hold-out because the specific data division matter less.
- ✖ **Computational cost**: the model is trained K times, still heavier than a single hold-out validation. For deep neural networks, this can be impractical.
- ✖ **Data leakage risk**: all processing (normalization, scaling, etc.) must be **recomputed inside each fold**, otherwise information from the validation folds can leak into the training folds.
- ✖ **Variance in small datasets**: if K is too small, each fold may not represent the full data distribution well.

3.5.4 Nested Cross-Validation

When we train a neural network (or any machine learning model), we actually perform **two separate activities**:

- **Model selection**: choose the *best configuration* (e.g., hyperparameters, architecture) based on performance on a validation set.
- **Model assessment**: estimate the *true generalization ability* of the final chosen model on unseen data.

⚠ But here lies a **problem**: in most cross-validation setups, **we reuse the same data for both tasks**. That means we **peek** at our test data while tuning our model, leading to **optimistic bias** in our performance estimates.

❓ Why that's a problem (hidden optimism)? Let's say we're testing 10 different neuronal network architectures and we run **5-fold cross-validation** ($K = 5$) to see which performs best.

1. We choose the one with the lowest average validation error across the 5 folds.
2. Then we report that same average validation error as "the model's performance".

⚠ But this is **optimistically biased**, because:

- We **used** the validation data to pick the best model.
- Therefore, the validation score no longer represents unseen data, because the model (and our choice) are **tuned** to those particular folds we used (even if indirectly).

So we get a number that looks great, but it's **too optimistic**, it doesn't reflect how the model would perform on truly unseen data.

✅ There is no more optimism with Nested Cross-Validation

Nested Cross-Validation is a technique that **separates model selection from model assessment** by introducing **two layers of cross-validation loops**:

- An **outer loop** for **model assessment** (unseen test data). It evaluates *how good that chosen configuration* really is on truly unseen data.
- An **inner loop** for **model selection** (tuning hyperparameters). It decides *which configuration* performs best (uses only training data of that outer fold).

So now, when we report the average outer test error, it reflects **real generalization**, not the performance on data we optimized for.

Example 1: Nested Cross-Validation Analogy

Think of it like preparing for an exam:

- We take **practice tests** to decide the best study method (this is the **inner loop, model selection**).
- Then, we take the **final exam** to measure how well our preparation really worked (this is the **outer loop, model assessment**).

If we report our *practice test* scores as our *final exam* result, we'll be unrealistically confident, exactly what happens without nested cross-validation.

How does Nested Cross-Validation work?

1. **Outer Loop (Model Assessment)**:
   - Split the entire dataset into K **outer folds**:
   $$\mathcal{D} = \{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_K\}$$
   - For each outer fold k:
     - Reserve fold $\mathcal{D}_k^{\text{outer}}$ for **testing**.
     - Use the remaining $K-1$ folds as the **training set** for model selection in the inner loop:
     $$\mathcal{D}_{\text{train}}^{\text{outer}} = \bigcup_{\substack{j=1 \\ j \neq k}}^{K} \mathcal{D}_j$$
2. **Inner Loop (Model Selection)**:
   - Inside that outer training data $\mathcal{D}_{\text{train}}^{\text{outer}}$, perform another K-fold cross-validation, called M**-fold cross-validation**:
   $$\mathcal{D}_{\text{train}}^{\text{outer}} = \left\{\mathcal{D}_1^{\text{inner}}, \mathcal{D}_2^{\text{inner}}, \ldots, \mathcal{D}_M^{\text{inner}}\right\}$$
   To **tune hyperparameters** (e.g., learning rate, number of layers, etc.). The letter M is used to distinguish it from the outer loop's K.
   - Select the configuration θ_k^* that gives the **lowest inner validation error averaged** over the M inner folds:
   $$\theta_k^* = \arg\min_{\theta} \left(\frac{1}{M} \cdot \sum_{m=1}^{M} \text{Error}\left(\mathcal{D}_m^{\text{inner, val}}; \theta\right) \right)$$
3. **Model Evaluation**:
   - Retrain the model on **all inner folds combined** using the selected configuration θ_k^*:
   $$\mathcal{D}_{\text{train}}^{\text{inner}} = \bigcup_{m=1}^{M} \mathcal{D}_m^{\text{inner}}$$

- Evaluate it on the **outer test fold** $\mathcal{D}_k^{\text{outer}}$ to get the test error for that outer fold:

$$\text{Test Error}_k = \text{Error}\left(\mathcal{D}_k^{\text{outer}}; \theta_k^*\right)$$

And store the test error for that outer fold:

$$e_k = \text{Test Error}_k$$

4. **Average over outer folds**. After completing all K outer folds, compute the overall performance estimate:

$$\text{Overall Test Error} = \hat{E}_{\text{nested}} = \frac{1}{K}\sum_{k=1}^{K} e_k$$

This provides a **nearly unbiased** estimate of the model's true generalization performance, because the test data in each outer fold was never used during model selection in the inner loop.

```
Input:
    Dataset 𝒟 = {(x_1, t_1), ..., (x_N, t_N)}
    Number of outer folds K
    Number of inner folds M
    // Create set of candidate configurations
    // (e.g., hyperparameters, architectures)
    Set of candidate configurations Θ
    Learning algorithm 𝒜
    Error function ℒ(t̂, t)

Split 𝒟 into K disjoint outer folds:
    𝒟 = 𝒟_1^outer ∪ ··· ∪ 𝒟_K^outer

Initialize total outer error E ← 0

// For each outer fold
for k = 1 to K:
    // Create outer test set using the k-th fold
    𝒟_test^(k) ← 𝒟_k^outer
    // Create outer training set using the remaining folds
    𝒟_train^(k) ← 𝒟 \ 𝒟_k^outer

    Split 𝒟_train^(k) into M disjoint inner folds:
        𝒟_train^(k) = 𝒟_1^inner ∪ ··· ∪ 𝒟_M^inner

    Initialize best inner error E_best ← +∞
    Initialize best configuration θ_k^* ← None

    for each configuration θ ∈ Θ:
        E_inner ← 0

        // For each inner fold
        for m = 1 to M:
            // Create inner validation set
```

```
            // using the m-th fold
            D_val^(m) ← D_m^inner
            // Create inner training set
            // using the remaining folds
            D_train^(m) ← D_train^(k) \ D_m^inner

            Train inner model:
                // Train model f_θ^(m) on D_train^(m)
                // with configuration θ
                f_θ^(m) ← A(D_train^(m), θ)

            Compute inner validation error:
                // Compute error e_m on D_val^(m)
                // using model f_θ^(m)
                e_m ← 1/|D_val^(m)| Σ_{(x_i,t_i)∈D_val^(m)} L(f_θ^(m)(x_i), t_i)

            // Accumulate inner error for this configuration
            E_inner ← E_inner + e_m

        Compute average inner error:
            Ê_θ ← 1/M E_inner

        // Update best configuration if this one is better
        if Ê_θ < E_best:
            E_best ← Ê_θ
            θ*_k ← θ

    Retrain model on all outer training data with best
    configuration:
        f^(k) ← A(D_train^(k), θ*_k)

    Compute outer test error:
        e_k ← 1/|D_test^(k)| Σ_{(x_i,t_i)∈D_test^(k)} L(f^(k)(x_i), t_i)

    Accumulate outer error:
        E ← E + e_k

Compute average nested CV error:
    Ê_nested ← 1/K E

Output:
    Ê_nested
```

Listing 4: Nested Cross-Validation Algorithm

The key idea is that the **inner loop chooses the best hyperparameters**, while the **outer loop evaluates the whole selection procedure on unseen data**.

⚠ Limitations and ✅ Advantages

- ✔ Provides a **less optimistically biased estimate** of the generalization error, especially when the dataset is small and model selection is complex.
- ✔ Ensures a clear **separation between selection and assessment**.
- ✔ Uses **all data efficiently** across different folds.
- ✘ **Computationally expensive**: For each outer fold K, a full inner cross-validation with M folds must be repeated for all candidate configurations Θ, followed by a final retraining on the whole outer training set. This can lead to a total of $K \times M \times |\Theta|$ model trainings, which is often prohibitive for large datasets or complex models.
- ✘ **Complex implementation**: Careful bookkeeping is required to manage data splits and model configurations across nested loops. In other words, it's easy to make mistakes if not implemented carefully.

Some typical choices are $K = 5$ for the outer loop and $M = 3$ for the inner loop, balancing computational cost and reliable estimates. For more accurate estimates, $K = 10$ and $M = 5$ can be used, but at a higher computational cost. For small datasets, nested cross-validation is particularly beneficial to avoid overfitting during model selection, and a common choice is $K = 10$ and $M = 5$.

3.6 Preventing Overfitting

So far we've seen:

- **Overfitting**: when the model fits both the signal *and* the noise in the training data, leading to poor generalization on unseen data.
- **Underfitting**: when the model is too simple to capture the underlying patterns in the data, resulting in poor performance on both training and test sets.
- **Cross-Validation**: how to detect when a model generalizes poorly using techniques like Hold-Out Validation, Leave-One-Out Cross-Validation (LOOCV), K-Fold Cross-Validation, and Nested Cross-Validation.

But detecting overfitting is only half the battle. The next crucial step is to implement strategies to **prevent or limit overfitting during training** and enhance the model's ability to generalize well to new, unseen data. This section presents the **three main families of solutions** that modern deep learning uses to *control complexity* and *improve generalization*.

The Core Problem: Neural networks have enormous representational power, by the *Universal Approximation Theorem* (page 123), they can approximate any continuous function. However, if they have **too many parameters**, and **too little data** (or data with noise), they will **memorize** rather than **learn**. Preventing overfitting is thus about **introducing constraints or checks** that force the model to extract **essential structure** from the data, not incidental details.

Overview of Prevention Techniques. In this section, we will explore three main strategies to prevent overfitting in neural networks:

1. **Training control**: **Early Stopping technique** (page 152). This method stops training when validation error starts increasing.
2. **Model complexity control**: **Regularization (Weight Decay, L2) technique** (page 164). This method adds a penalty when weights grow too large, because large weights often indicate memorization.
3. **Randomization / Model averaging**: **Dropout technique** (page 173). This method randomly deactivates neurons during training, forcing the network to learn redundant representations that generalize better.

Each of these reduce the **effective capacity** of the network in a different way. Either by limiting how long it trains, how larger weights can grow, or how tightly neurons depend on each other.

How these techniques relate to the Training Curve

Similar to the curve of **Model Complexity vs. Error** shown in Figure 16, these techniques aim to find the optimal point where the model is complex enough to capture the underlying patterns in the data, but not so complex that it overfits. The **Training vs. Validation Error Curve** plot (shown in Figure 18) illustrates how training and validation errors evolve during training. The goal of these techniques is to keep the model in the region where both training and validation errors are low, avoiding the point where validation error starts to increase due to overfitting.

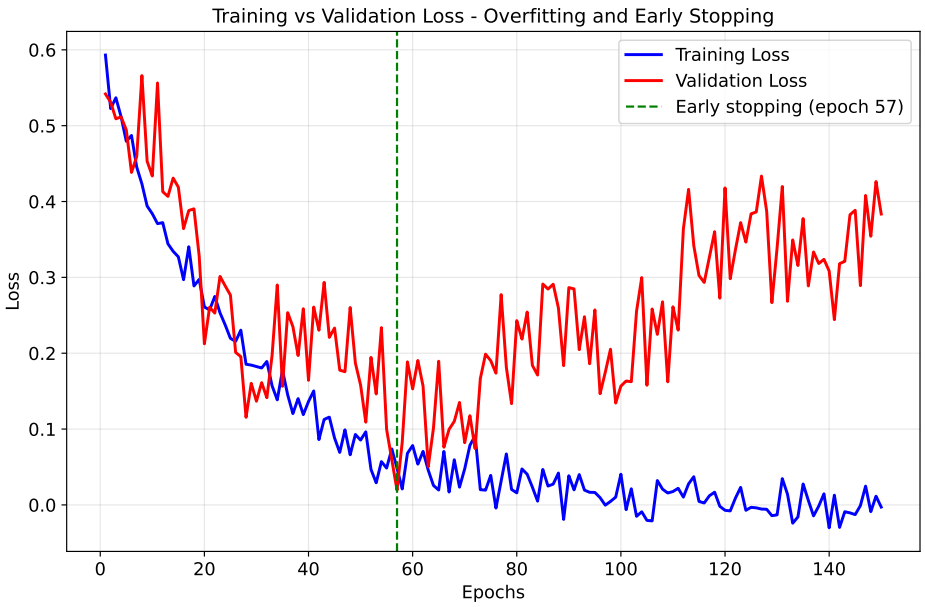


Figure 18: This plot illustrates the **Training vs. Validation Error Curve** during the training of a neural network. Initially, both training and validation errors decrease as the model learns from the data. However, after a certain point (epoch 57), the validation error starts to increase while the training error continues to decrease, indicating that the model is beginning to overfit the training data. The optimal stopping point is where the validation error is minimized, which can be achieved using techniques like Early Stopping.

Furthermore, all **these techniques operationalize the Ockham's Razor principle** we saw earlier: "prefer the simplest model that explains the data well" (page 124). By constraining the model's capacity in various ways, we encourage it to focus on the most salient features of the data, leading to better generalization and performance on unseen data.

3.6.1 Early Stopping

During training, the **training error** keeps decreasing as the model learns to fit the training data better and better. However, at some point, the **validation error** starts to increase again, indicating that the model is beginning to overfit the training data (as we saw in Figure 18 on page 151). That's the signature of **overfitting**: the model is still improving on the training data, but getting worse on unseen data.

The first and simplest method to prevent overfitting is **early stopping**. The idea is very straightforward: we stop the training process **right before** the validation error starts to rise.

Monitoring Validation Error

To apply early stopping, we need to monitor both:

- The **training error/loss** $E_{\text{train}}(k)$ (measures how well the network fits the training data).
- The **validation error/loss** $E_{\text{val}}(k)$ (measures how well it generalizes to unseen data).

Where k is the current training iteration (or epoch). So, during training, at each iteration (epoch) k, we look for the **epoch k_{ES}** (Early Stopping iteration) that minimizes the validation error:

$$E_{\text{val}}(k) \text{ is minimal } \implies k = k_{ES}$$

Once we find k_{ES}, we stop training and use the model parameters (weights and biases) from that iteration for our final model.

Stopping Criteria (iteration k_{ES})

Definition 3: Early Stopping

Early Stopping is a regularization technique that prevents overfitting by **monitoring the validation error during training** and **halting the learning process when the error begins to increase**.

The stopping iteration k_{ES} is defined as the epoch where the validation error $E_{\text{val}}(k)$ is minimized. Formally:

$$k_{ES} = \underset{k}{\text{argmin}}\, E_{\text{val}}(k) \tag{54}$$

It marks the point of **best generalization**; beyond it, the network starts fitting noise rather than signal.

By interrupting training at k_{ES}, the model parameters remain close to their optimal generalizing values, providing an **online estimate of the true generalization error**. With "*online estimate*", we mean that

we can assess the model's performance on unseen data without needing a separate test set at this stage; "*online*" refers to the fact that this evaluation happens during the training process itself.

However, simply stopping at the first sign of validation error increase can be problematic due to random fluctuations (noise) in the validation curve. So, in practice, we implement a more robust strategy. We usually use a **patience window** to avoid stopping too early due to random noise in the validation curve. The **Patience Window** (or **Patience Parameter**) defines **how many epochs** the training process should **wait after the last improvement in validation error** before deciding to stop. In other words, we don't stop immediately when $E_{\text{val}}(k)$ increases once; instead, we wait for a few epochs to see if it improves again (since small fluctuations can occur due to noise).

Example 2: Early Stopping with Patience Window

Imagine this simplified validation loss curve during training (table values):

Epoch	Validation Loss	Improvement?
...	...	...
10	0.40	✔ improvement
11	0.38	✔ improvement
12	0.37	✔ improvement
13	0.375	✘ slightly worse
14	0.373	✔ improvement
15	0.376	✘ worse again
16	0.379	✘ worse
17	0.382	✘ worse

Suppose we set a **patience window of 2 epochs**, the algorithm will stop after **epoch 17**, because it waited for 2 epochs after the last improvement (epoch 14) and saw no new decrease in validation loss. If we had set a patience of 4 epochs, it would have waited until epoch 19 before stopping.

Be careful: it's a heuristic! Early stopping with a *patience window* is **not mathematically guaranteed** to find the optimal epoch k_{ES}; we might stop a few epochs **too early** or **too late**, depending on noise, random initialization, learning rate, and other hyperparameters. In formal terms, it doesn't *guarantee* the true optimum, but it approximates a **local minimum in the generalization curve**, which is what we really want in practice. The local minimum corresponds to a model that generalizes well without overfitting, even if it's not the absolute best possible model.

Online Estimation of Generalization Error

The Early Stopping method can be viewed as an **online estimation of the true generalization error**, because we continuously track the validation loss as the network learns, and the shape of that curve tells us when the model begins to overfit. So, we use validation data to **approximate the true test error dynamically** without needing to retrain multiple times. Thus, early stopping is like a **built-in regularizer**: it doesn't change the loss function, but limits training to the "sweet spot" where generalization is maximal. Where "online" means that this estimation happens during the training process itself, rather than after training is complete.

Limitations and Advantages

- ✔ **Simple & efficient**: no modification to loss or architecture
- ✔ **Automatic**: modern frameworks can monitor and stop automatically.
- ✔ **Improves generalization** without additional parameters.
- ✘ Requires a **validation set**, which reduces training data slightly.
- ✘ Works best when validation loss is smooth (less noisy).
- ✘ The "patience" value and monitoring metric must be chosen carefully.

3.6.2 Hyperparameter Tuning

After understanding **Early Stopping** (which stops training at the right moment), we now focus on **choosing the right model itself**; that is, deciding *how big*, *how deep*, *how fast*, and *how regularized* the network should be. This process is called **Hyperparameter Tuning**, and it's a **crucial step to control overfitting and improve generalization**.

Definition 4: Hyperparameter Tuning

Hyperparameter Tuning is the process of **selecting the best model configuration**, that is, the combination of hyperparameters (such as the number of layers, neurons, learning rate, regularization, strength, etc.). That yields the **lowest validation error** and thus the best generalization ability.

Unlike **parameters** (weights and biases), which are learned automatically during training, **hyperparameters** control *how* learning occurs and *how complex* the model can be.

The optimal configuration:

$$\theta^* = \arg\min_{\theta_i} E_{\text{val}}(\theta_i) \tag{55}$$

is chosen by **comparing validation errors** across candidate models, often within a cross-validation framework to reduce bias.

Parameters vs. Hyperparameters

The definition above highlights the distinction between **parameters** and **hyperparameters**:

- **Parameters**: These are the internal **weights and biases** of the neural network that are **learned during training through optimization algorithms** like gradient descent. They directly influence the model's predictions, so they determine the function learned by the network.
- **Hyperparameters**: These are **external configurations set before training begins**. They include choices like the:
  - **Number of layers**: This determines the depth of the network. More layers can capture more complex patterns but may also lead to overfitting.
  - **Number of neurons per layer**: This controls the width of the network. More neurons can model more complex functions but increase the risk of overfitting.
  - **Learning rate**: This is a crucial hyperparameter that controls how much to change the model in response to the estimated error each time the model weights are updated.

- **Batch size**: This defines the number of training examples utilized in one iteration. Smaller batch sizes can provide a more accurate estimate of the gradient but take longer to train. We have seen this in the **Stochastic Gradient Descent** (*batch gradient descent*, item 2, page 118)
- **Regularization strength** γ: This controls the amount of regularization applied to the model to prevent overfitting. Higher values impose a stronger penalty on large weights. We will see this in future sections.
- **Dropout rate**: This defines the fraction of neurons to drop during training to prevent overfitting. A higher dropout rate means more neurons are ignored during each training iteration. Also this will be covered in future sections.

Hyperparameters are set **manually** by the user or **automatically** through hyperparameter optimization techniques (e.g., grid search, random search, Bayesian optimization). They control *how* the model learns and *how complex* it can become, thus affecting its ability to generalize to unseen data.

Why is Hyperparameter tuning important? And why does it matter for overfitting?

Hyperparameter tuning is essential for several reasons, especially in the context of overfitting. It determines the **model's capacity to learn from data** and its ability to generalize to unseen examples.

Hyperparameter	**Low Value**	**High Value**
Number of layers/neurons	Underfitting (too simple)	Overfitting (too complex)
Learning rate	Slow convergence	Unstable or overfitting
Regularization strength γ	Weak penalty, overfit	Too strong, underfit
Dropout rate	Too little regularization	Too much, underfit

Table 2: Impact of Hyperparameter Values on Model Performance.

In geneal, **too low values** of hyperparameters can lead to **underfitting**, where the model is too simple to capture the underlying patterns in the data. Conversely, **too high values** can lead to **overfitting**, where the model learns the training data too well, including its noise and outliers, resulting in poor generalization to new data. So tuning them correctly is essential to reach the **sweet spot** of generalization.

Hyperparameter Tuning Algorithm

The **Hyperparameter Tuning Algorithm** is not a real algorithm per se, but rather a **conceptual framework** for selecting the best hyperparameters based on validation performance. The input are:

- A **set Θ of possible hyperparameter configurations**:

$$\Theta = \{\theta_1, \theta_2, \ldots, \theta_M\}$$

  Where each θ_i is a specific combination of hyperparameter values (e.g., number of layers, learning rate, regularization strength, etc.), and M is the total number of configurations to evaluate.

- A **training dataset** $\mathcal{D}_{\text{train}}$ used to train the model for each hyperparameter configuration.

- A **validation dataset** $\mathcal{D}_{\text{val}}$ used to evaluate the model's performance for each configuration. Here we assume that our dataset is large enough to be split into training and validation sets. If not, cross-validation techniques can be employed to make the most of limited data (hold-out method page 137, k-fold page 142 or leave-one-out cross-validation page 139, or nested cross-validation page 145).

The procedure is as follows:

1. **Define candidate configurations**. Choose which hyperparameter combinations to evaluate. Each configuration θ_i should specify values for all relevant hyperparameters. For example, number of layers, number of neurons per layer, learning rate, regularization strength, etc.

2. **For each configuration $\theta_i \in \Theta$**:

   (a) **Train** a neural network using $\mathcal{D}_{\text{train}}$ with the hyperparameters specified by θ_i. This involves initializing the model, performing forward and backward passes, and updating weights according to the chosen optimization algorithm (we will see this in future sections).

   (b) **Evaluate** its performance on the **validation set** $\mathcal{D}_{\text{val}}$ to compute the validation error $E_{\text{val}}(\theta_i)$. This error metric could be mean squared error, cross-entropy loss, accuracy, etc., depending on the task. If using cross-validation, average the validation errors across all folds to get a robust estimate.

3. **Compare validation errors**. Identify the configuration that gives the **lowest validation error**:

$$\theta^* = \arg\min_{\theta_i \in \Theta} E_{\text{val}}(\theta_i)$$

4. **Select the best model**. Keep the model trained with the optimal hyperparameters θ^*, or retrain it from scratch using the **union of training** and **validation data** with the chosen hyperparameters to maximize the data available for learning.

5. **(optional) Final test**. Evaluate the chosen model on a **separate test set** $\mathcal{D}_{\text{test}}$ to estimate its generalization performance on unseen data.

The output of this procedure is the **optimal hyperparameter configuration** θ^* that minimizes the validation error, along with the **trained model** that can be used for predictions on new data, and an **estimate of its generalization performance** (from validation or test set).

How to choose candidate hyperparameter configurations?

In practice, candidate hyperparameter configurations are not chosen manually because this would be too time-consuming and inefficient. Instead, we use **automatic search methods** to explore the hyperparameter space effectively. In general, we define a **search space** for each hyperparameter and then **use an algorithm to sample configurations from this space**. The goal is to find the configuration that minimizes the validation error (if possible) without having to evaluate every possible combination.

```
Input:
    Training dataset D_train
    Validation dataset D_val
    Set of candidate configurations Θ = {θ_1, θ_2, ..., θ_M}
    Learning algorithm A
    Validation error function E_val(·)

Initialize best error E_best ← +∞
Initialize best configuration θ* ← None
Initialize best model f* ← None

for each configuration θ_i ∈ Θ:
    Train model:
        f_i ← A(D_train, θ_i)

    Evaluate on validation set:
        e_i ← E_val(f_i, D_val)

    if e_i < E_best:
        E_best ← e_i
        θ* ← θ_i
        f* ← f_i

// (Optional) It is optional because the model f* trained
// with θ* on D_train may already be good enough.
// However, if we want to maximize the data used for
// training, we can retrain the model with the
// best hyperparameters
Retrain final model on D_train ∪ D_val using θ*

Output:
    Best configuration θ*
    Best validation error E_best
    Best model f*
```

Listing 5: Generic Hyperparameter Tuning Algorithm

Specifically, there are three classical methods for hyperparameter tuning:

1. **Grid Search** (Exhaustive Search on a Discrete Grid). This method evaluates the model on **all possible combinations** of a predefined set of hyperparameter values. For example, we define a **grid** for each hyperparameter:

   $$\eta \in \{0.001, 0.01, 0.1\} \qquad \gamma \in \{0.01, 0.1, 1.0\} \qquad \text{layers} \in \{2, 3, 4\} \qquad \dots$$

   Then the algorithm trains a model for **every combination** of these values:

   $$\Theta = \{(\eta, \gamma, \text{layers}, \dots) \mid \eta \in \{\dots\}, \gamma \in \{\dots\}, \text{layers} \in \{\dots\}, \dots\}$$

   After training and evaluating each configuration, the one with the lowest validation error is selected (minimum E_{val}).

   ✔ **Advantages**

   - ✔ Simple and systematic.
   - ✔ Parallelizable and easy to implement.

   ⚠ **Limitations**

   - ✘ Computationally expensive (combinations grow exponentially with more hyperparameters). Also, parallelizable only if enough resources are available.
   - ✘ Many evaluations wasted on unimportant regions of the hyperparameter space.
   - ✘ Works well only with a *few* hyperparameters or *coarse grids*.

```
Input:
    Training dataset 𝒟_train
    Validation dataset 𝒟_val
    Discrete candidate values for each hyperparameter
    θ_1, ..., θ_p
    Learning algorithm 𝒜
    Validation error function E_val(·)

Construct the Cartesian product of all hyperparameter
    values:
    Θ ← θ_1 × θ_2 × ··· × θ_p

Initialize best error E_best ← +∞
Initialize best configuration θ* ← None

for each configuration θ ∈ Θ:
    Train model:
        f_θ ← 𝒜(𝒟_train, θ)

    Evaluate on validation set:
        e_θ ← E_val(f_θ, 𝒟_val)

    if e_θ < E_best:
        E_best ← e_θ
```

```
            θ* ← θ

Output:
    Best configuration θ*
    Best validation error E_best
```

Listing 6: Grid Search Algorithm

2. **Random Search** (Stochastic Sampling of Hyperparameter Space). Instead of evaluating all combinations, this method **randomly samples** hyperparameter configurations from specified distributions. For example, we define ranges or distributions for each hyperparameter:

$$\eta \sim \text{Uniform}(0.001, 0.1)$$
$$\gamma \sim \text{LogUniform}(0.01, 1.0)$$
$$\text{layers} \sim \text{DiscreteUniform}\{2, 3, 4\}$$
$$\dots$$

Then the algorithm samples N configurations randomly:

$$\Theta = \{\theta_1, \theta_2, \dots, \theta_N\}$$

After training and evaluating each sampled configuration, the one with the lowest validation error is selected. This method is **much more efficient than grid search**, because typically only a few hyperparameters significantly impact performance.

✔ **Advantages**

- ✔ Covers the search space more efficiently.
- ✔ Works better for high-dimensional spaces.
- ✔ Can easily add or extend hyperparameters.

⚠ **Limitations**

- ✘ Still blind, don't use information from previous trials.
- ✘ May miss the best configuration by chance.

```
Input:
    Training dataset D_train
    Validation dataset D_val
    Hyperparameter distributions P(θ) // e.g. log-uniform
    Number of trials N
    Learning algorithm A
    Validation error function E_val(·)

Initialize best error E_best ← +∞
Initialize best configuration θ* ← None

for i = 1 to N:
    Sample configuration:
        θ_i ~ P(θ)
```

```
    Train model:
        f_i ← A(D_train, θ_i)

    Evaluate on validation set:
        e_i ← E_val(f_i, D_val)

    if e_i < E_best:
        E_best ← e_i
        θ* ← θ_i

Output:
    Best configuration θ*
    Best validation error E_best
```

Listing 7: Random Search

3. **Bayesian Optimization** (Probabilistic Model-Based Search, Learning from Past Trials). The idea is simple yet powerful. Model the function:

$$f(\theta) = E_{\text{val}}(\theta)$$

As an **unknown function** (black box) and use **probabilistic reasoning** to decide which hyperparameters to test next. The steps are:

(a) Use previous evaluations to build a **probabilistic surrogate model** (e.g. a Gaussian Process, Tree Parzen Estimator, etc.) that approximates the relationship between hyperparameters and validation error ($f(\theta)$).

(b) Define an **acquisition function** (e.g. Expected Improvement, Upper Confidence Bound, etc.) that quantifies the potential benefit of evaluating a new configuration based on the surrogate model. It balances **exploration** (trying uncertain areas) and **exploitation** (focusing on promising areas).

(c) Choose the next configuration θ_{next} to evaluate by maximizing the acquisition function (i.e., the configuration that is expected to yield the most improvement).

(d) Now, train the model with θ_{next}, evaluate its validation error $E_{\text{val}}(\theta_{\text{next}})$, and update the surrogate model with this new data point.

(e) Repeat steps 2-4 until a stopping criterion is met (e.g., a maximum number of evaluations or convergence).

This method is not a naive search; it **learns from past evaluations** to make informed decisions about which hyperparameters to test next, leading to more efficient optimization.

✔ **Advantages**

- ✔ **Much fewer evaluations** needed for good performance.
- ✔ Adapts search intelligently (guided by prior results).
- ✔ Suitable for **expensive models** (deep networks, large datasets).

Limitations

- ✗ Implementation complexity (requires probabilistic modeling).
- ✗ Needs a meaningful continuous search space (discrete hyperparameters can be tricky).
- ✗ Slower per iteration (but far fewer iterations needed).

```
Input:
    Training dataset $\mathcal{D}_{\text{train}}$
    Validation dataset $\mathcal{D}_{\text{val}}$
    Search space $\Theta$ for hyperparameters
    Learning algorithm $\mathcal{A}$
    Validation error function $E_{\text{val}}(\cdot)$
    Initial number of random evaluations $N_{\text{init}}$
    Maximum number of iterations $T$

Initialize observation set $\mathcal{H} \leftarrow \emptyset$

// For each initial random configuration,
// evaluate and add to observations
for $i = 1$ to $N_{\text{init}}$:
    Sample initial configuration $\theta_i$ randomly from $\Theta$
    Train model:
        $f_i \leftarrow \mathcal{A}(\mathcal{D}_{\text{train}}, \theta_i)$
    Evaluate validation error:
        $e_i \leftarrow E_{\text{val}}(f_i, \mathcal{D}_{\text{val}})$
    Add observation:
        // The observation is simply the pair of
        // hyperparameters and their corresponding
        // validation error
        $\mathcal{H} \leftarrow \mathcal{H} \cup \{(\theta_i, e_i)\}$

// Start Bayesian optimization loop.
// The loop starts from $N_{\text{init}} + 1$ because we have already
// evaluated $N_{\text{init}}$ random configurations.
// We continue until we reach the maximum number
// of iter. $T$
for $t = N_{\text{init}} + 1$ to $T$:
    // Create a (surrogate) model
    // from the observations $\mathcal{H}$,
    // which is a set of pairs
    // (configuration, validation error)
    Fit surrogate model on $\mathcal{H}$

    // The acquisition function $a(\theta \mid \mathcal{H})$ uses the surrogate
    // model to estimate the potential improvement of
    // evaluating a new configuration $\theta$ based
    // on the observed data $\mathcal{H}$. It balances:
    //  1. exploration (trying configurations with high
    //     uncertainty)
    //  2. exploitation (focusing on configurations that
    //     are likely to perform well).
    Define acquisition function $a(\theta \mid \mathcal{H})$

    Choose next configuration:
        // Choose the configuration $\theta_t$ that maximizes
```

```
        // the acquisition function, which means it is
        // expected to yield the most improvement
        // based on the surrogate model
        // and past observations.
        θ_t ← arg max_{θ∈Θ} a(θ | H)

    Train model:
        f_t ← A(D_train, θ_t)

    Evaluate validation error:
        e_t ← E_val(f_t, D_val)

    Update observation set:
        // Add the new configuration and its validation
        // error to the observation set H,
        // which will be used to update the surrogate
        // model in the next iteration.
        H ← H ∪ {(θ_t, e_t)}

Select best observed configuration:
    // Once the optimization loop is complete,
    // we select the configuration with the lowest
    // validation error from the observation set H.
    θ* ← arg min_{(θ,e)∈H} e

Output:
    Best configuration θ*
    History of evaluated configurations H
```

Listing 8: Bayesian Optimization for Hyperparameter Tuning

These methods follow the same underlying goal: **minimize the validation error** by efficiently exploring the hyperparameter space

$$\theta^* = \underset{\theta_i \in \Theta}{\arg\min}\, E_{\text{val}}\left(\theta_i\right)$$

But they differ in ***how* they select candidate configurations** to evaluate, balancing exploration and exploitation in different ways.

3.6.3 Weight Decay (L2 Regularization)

The **Weight Decay** (or **L2 Regularization**) is a widely used technique to prevent overfitting in neural networks by **adding a penalty term to the loss function that discourages large weights**. This method helps to keep the model simpler and more generalizable by constraining the magnitude of the weights.

The key idea behind weight decay is to penalize overly large weights to prevent the network from overfitting training noise. When we train a neural network, we minimize a **loss function** (e.g. MSE regression or cross-entropy for classification):

$$E_{\text{train}}(w) = \frac{1}{N} \cdot \sum_{i=1}^{N} \mathcal{L}\left(y_i, f\left(x_i; w\right)\right)$$

If we let the optimization freely minimize this error, the network might use **very large weights** to fit every detail, creating a complex, oscillating decision surface that produces low training error but poor generalization to unseen data (i.e., **overfitting**). To counter this, we add a **penalty on the magnitude of weights** to the loss function:

$$E_{\text{reg}}(w) = E_{\text{train}}(w) + \underbrace{\frac{\gamma}{2} \cdot \sum_{q} w_q^2}_{\text{L2 penalty}}$$

- w_q is the q-th weight in the network.
- $\gamma > 0$ is the regularization parameter that controls the strength of the penalty (sometimes called **weight decay coefficient**). A larger γ encourages smaller weights, leading to a simpler model (more bias, less variance), while a smaller γ allows for more complex models (less bias, more variance).

This makes the optimization prefer **small, smooth weights**, producing smoother mappings from inputs to outputs, which helps in generalization.

In other words, the optimizer now minimizes both **error** and **weight energy**. This discourages the model from relying too heavily on any single feature or neuron, promoting a more distributed representation that is less likely to overfit the training data (the network learns gentler transformations, **simpler hypotheses**).

Definition 5: Weight Decay (L2 Regularization)

Weight Decay, or **L2 Regularization**, is a technique that prevents overfitting by adding a **penalty on the magnitude of the network's weights** to the loss function. The regularized loss function is:

$$E_{\text{reg}}(w) = E_{\text{train}}(w) + \frac{\gamma}{2} \cdot \sum_{q} w_q^2 \tag{56}$$

Where:

- $E_{\text{train}}(w)$ is the original training loss (e.g., MSE or cross-entropy).
- w_q are the weights of the neural network.
- $\gamma > 0$ is the regularization parameter controlling the strength of the penalty.

By discouraging large weights, the model learns smoother mappings and achieves better generalization, effectively limiting its complexity.

⚖ Relation to Early Stopping

Early Stopping limited training time to prevent overfitting (page 152), while **weight decay directly limits the magnitude of parameters**. Both methods implement the Ockham's Razor principle (page 124) by **favoring simpler models that generalize better**, but act at different stages:

- **Early Stopping stops training** when validation error starts to rise, so before weights can grow too large.
- **Weight Decay continuously penalizes** large weights during training, keeping them small throughout the process.

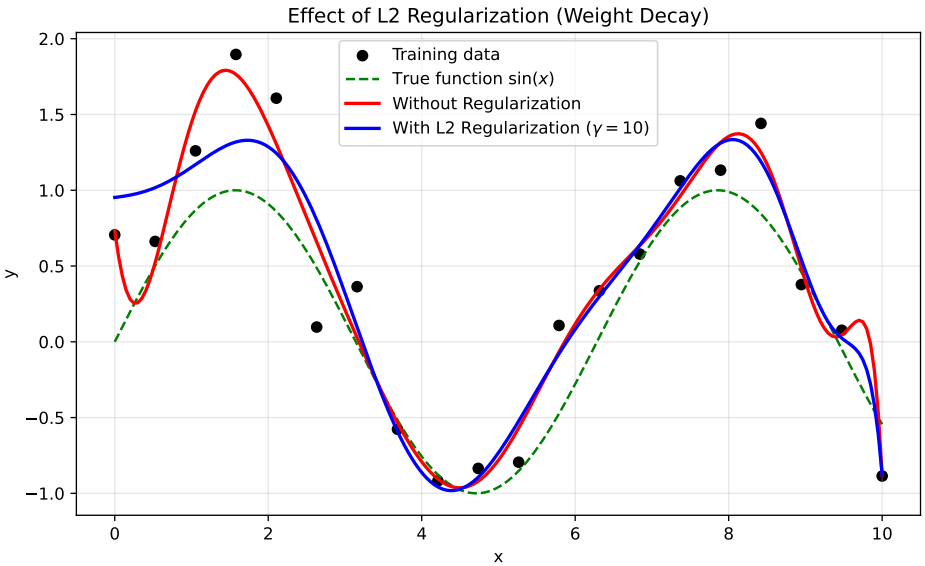


Figure 19: A simulation of polynomial regression with two different fits: one without regularization (large, oscillating weights, overfitting the data) and one with L2 regularization (smaller weights, smoother fit). The L2 regularized model generalizes better to unseen data.

Bayesian Interpretation (MAP vs MLE)

When we train a neural network, we're finding parameters w that best explain the data $D = \{x_i, y_i\}$, where x_i are inputs and y_i are outputs. There are two main approaches to estimate these parameters:

- **Maximum Likelihood Estimation (MLE)** (page 107): choose the parameters w that **maximize the probability of the data given the model**:

  $$w_{\text{MLE}} = \arg\max_w P(D \mid w)$$

  Or equivalently, minimize the negative log-likelihood (which corresponds to minimizing the training error $E_{\text{train}}(w)$):

  $$w_{\text{MLE}} = \arg\min_w -\log P(D \mid w) \equiv \arg\min_w E_{\text{train}}(w)$$

  That's just the **training loss** we usually minimize (like MSE or cross-entropy).

  ✔ MLE fits the data as best as possible.

  ✖ But it can lead to overfitting, especially with complex models and limited data. This is because MLE doesn't penalize complex parameters.

- **Maximum A Posteriori Estimation (MAP)**: instead of trusting data blindly, we also include **prior beliefs** about what weights are likely. Let's explain this step by step. When we use **MLE**, we're saying: "*we don't know anything about the parameters w; just find the ones that make the data as likely as possible*". That's pure **data fitting**. But what if we *do* have priori knowledge? For example, we might believe that weights shouldn't be too large (to avoid overfitting), and most weights should be closer to zero (the network should be simple). Then we can express this belief **probabilistically**, by giving each possible value of w a **prior probability** $P(w)$ that reflects **how plausible we think that value is before seeing any data**. Here, MAP combines this belief with the observed data.

  We want the **posterior probability** of the weights given the data:

  $$P(w \mid D) = \frac{P(D \mid w)\, P(w)}{P(D)}$$

  - $P(D \mid w)$ is the **likelihood** of the data given weights (same as in MLE).
  - $P(w)$ is the **prior** probability of weights (our belief about weights before seeing data).
  - $P(w \mid D)$ is the **posterior** probability of weights given the data (what we believe about weights after seeing data).
  - $P(D)$ is the evidence (normalizing constant).

The **Maximum A Posteriori (MAP)** estimation means: choose the weights w that **maximize the posterior probability** $P(W \mid D)$:

$$w_{\text{MAP}} = \arg\max_w P(w \mid D)$$

Using Bayes' rule, we can rewrite this as:

$$w_{\text{MAP}} = \arg\max_w \frac{P(D \mid w)\, P(w)}{P(D)} = \arg\max_w P(D \mid w)\, P(w)$$

Since $P(D)$ is constant with respect to w, we can ignore it in the optimization. Taking logs, we get:

$$w_{\text{MAP}} = \arg\max_w \left[\log P(D \mid w) + \log P(w)\right]$$

Where the **first term** fits the data (like MLE), and the **second term** adds a *regularization effect* (our prior belief about weights).

To **connect this to weight decay**, we need to choose a specific prior $P(w)$. A common choice is a **Gaussian prior** centered at zero:

$$P(w) = \prod_q \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{w_q^2}{2\sigma^2}\right)$$

This prior says that we believe weights are likely to be small (close to zero), with variance σ^2 controlling how strongly we believe this. Taking the log of this prior gives:

$$\log P(w) = -\sum_q \frac{w_q^2}{2\sigma^2} + \text{constant}$$

Ignoring constants, the MAP objective becomes:

$$w_{\text{MAP}} = \arg\max_w \left[\log P(D \mid w) - \sum_q \frac{w_q^2}{2\sigma^2}\right]$$

Or equivalently, minimizing the negative log-posterior:

$$w_{\text{MAP}} = \arg\min_w \left[-\log P(D \mid w) + \sum_q \frac{w_q^2}{2\sigma^2}\right]$$

This is exactly the same as minimizing the regularized loss function with weight decay:

$$E_{\text{reg}}(w) = E_{\text{train}}(w) + \frac{\gamma}{2}\sum_q w_q^2$$

Where $\gamma = \dfrac{1}{\sigma^2}$. Thus, **weight decay can be interpreted as MAP estimation with a Gaussian prior on weights**. This Bayesian perspective shows that weight decay not only helps prevent overfitting but also incorporates prior beliefs about model simplicity into the learning process.

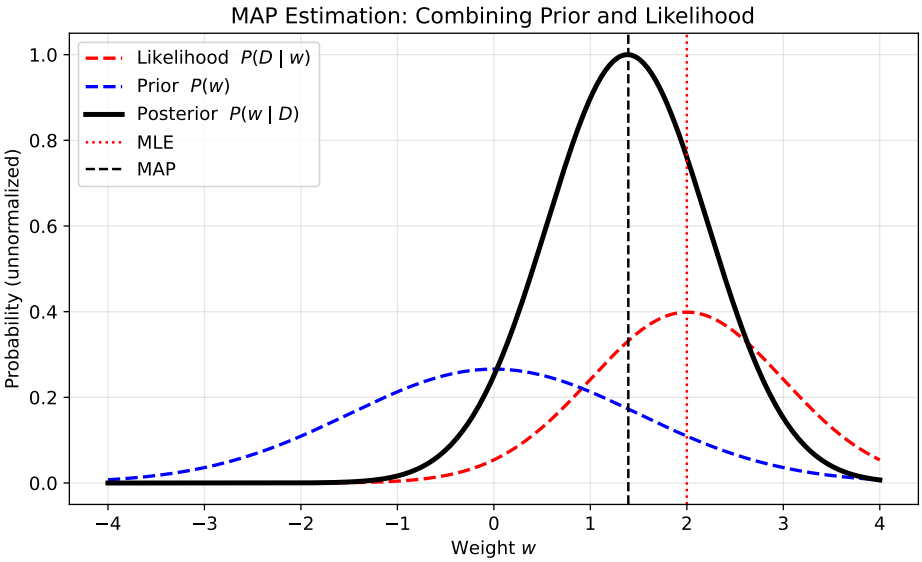


Figure 20: Bayesian interpretation of Weight Decay (L2 Regularization): MLE focuses solely on fitting the data, while MAP incorporates prior beliefs about weights, leading to more generalizable models. We have simulated a one-dimensional weight w: the **likelihood** $P(D \mid w)$ is peaked around some data-fitted value (like MLE); the **prior** $P(w)$ is a Gaussian centered at zero (we believe small weights are more likely); the **posterior** $P(w \mid D)$ combines (product) both, resulting in a peak that balances data fit and weight size.

Gaussian Priori Explanation

What is a prior over weights? When we train a model, we want to find good values for all weights w_q. If we follow a **Bayesian approach**, we say: each weight w_q is a random variable, and before seeing data we have a belief about how likely different values are. This belief is encoded in a **prior distribution**.

Choosing a Gaussian Prior. A natural, simple choice for this prior is a **Gaussian (normal) distribution** centered at zero:

$$P(w_q) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{w_q^2}{2\sigma^2}\right)$$

Intuitively:

- The mean is zero, so small weights are more probable than large ones.
- The variance σ^2 controls how strongly we believe this:
  - A small σ^2 means we strongly believe weights should be close to zero (less flexibility).
  - A large σ^2 means we allow for larger weights (more flexibility).

Assuming independence between weights, the joint prior over all weights is:

$$P(w) = \prod_q P(w_q) = \prod_q \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{w_q^2}{2\sigma^2}\right)$$

So the prior over all weights is a multivariate Gaussian with diagonal covariance. Taking the log of this prior gives:

$$\log P(w) = \sum_q \log P(w_q) = -\frac{1}{2\sigma^2} \sum_q w_q^2 + \text{constant}$$

Combine with likelihood (training error.) The MAP objective (as derived earlier) is:

$$w_{\text{MAP}} = \arg\max_w \left[\log P(D \mid w) + \log P(w)\right]$$

Equivalently, minimizing the negative log-posterior:

$$E_{\text{reg}}(w) = -\log P(D \mid w) - \log P(w)$$

Replace $-\log P(D \mid w)$ with the **training loss** $E_{\text{train}}(w)$, and substitute the log prior:

$$E_{\text{reg}}(w) = E_{\text{train}}(w) + \frac{1}{2\sigma^2} \sum_q w_q^2 + \text{constant}$$

Define $\gamma = \frac{1}{\sigma^2}$, we get:

$$E_{\text{reg}}(w) = E_{\text{train}}(w) + \frac{\gamma}{2} \sum_q w_q^2$$

This is exactly the weight decay regularization term! Thus, assuming a Gaussian prior on weights leads directly to L2 regularization in the MAP framework.

Deepening: What is a Multivariate Gaussian with Diagonal Covariance?

If we have a **single weight** w_1, we can describe our belief about it as a **univariate Gaussian**:

$$P(w_1) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(w_1 - 0)^2}{2\sigma^2}\right)$$

That's just the classic bell curve centered at zero, spreading depending on σ^2.

Now suppose we have **two weights** w_1 and w_2. We could model their joint belief as a **2D Gaussian distribution**:

$$P(w_1, w_2) = \frac{1}{2\pi\,|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}\begin{bmatrix} w_1 \\ w_2 \end{bmatrix}^T \Sigma^{-1} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}\right)$$

Where Σ is the **covariance matrix**. This tells us **how the two weights vary together**.

The covariance matrix Σ looks like this:

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{bmatrix}$$

Where:

- σ_1^2 and σ_2^2 are the variances of w_1 and w_2.
- ρ is the correlation coefficient between w_1 and w_2.
  - If $\rho = 0$, the weights are **independent** (no correlation). This means changing w_1 doesn't affect w_2.
  - If $\rho \neq 0$, the weights are **correlated** (changing one affects the other). This means if w_1 increases, w_2 might also tend to increase (if $\rho > 0$) or decrease (if $\rho < 0$).

When we assume the weights are **independent**, we set $\rho = 0$. This simplifies the **covariance matrix to a diagonal matrix**, because the off-diagonal terms (which represent correlations) become zero:

$$\Sigma = \begin{bmatrix} \sigma_1^2 & 0 & 0 & \dots \\ 0 & \sigma_2^2 & 0 & \vdots \\ 0 & 0 & \sigma_3^2 & \vdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

Then the multivariate Gaussian **factorizes** into independent 1D Gaussians for each weight:

$$P(w) = \prod_q \frac{1}{\sqrt{2\pi\sigma_q^2}} \exp\left(-\frac{w_q^2}{2\sigma_q^2}\right)$$

Each weight has its own Gaussian prior, and they don't influence each other. That's exactly what we assumed: each w_q has its own $\mathcal{N}\left(0, \sigma^2\right)$ prior, and all weights are independent.

Finally, when the covariance is diagonal ($\rho = 0$ and equal for all weights), we get:

$$\Sigma = \sigma^2 I$$

Where I is the identity matrix. This means that the log prior simplifies to:

$$-\log P\left(w\right) = \frac{1}{2\sigma^2} w^T w = \frac{1}{2\sigma^2} \sum_q w_q^2$$

Which is exactly the L2 regularization term we use in weight decay! So the assumption of a *multivariate Gaussian with diagonal covariance* is what mathematically justifies the standard **weight decay formula**.

How do we choose the right regularization strength γ?

Different values of γ change how much we penalize large weights:

- $\gamma = 0$ means no regularization (just MLE, prone to overfitting).
- γ small means weak regularization (allows larger weights, more complex models).
- γ large means strong regularization (forces weights to be small, simpler models).

So we must find the **optimal** $\gamma = \gamma^*$ that gives the **best generalization**. The idea is to split our data into:

- A **training set** for fitting the weights w.
- A **validation set** for evaluating performance for each candidate γ.

Practically:

1. Choose a range of candidate γ values, for example:

$$\left|10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10\right|$$

2. For each candidate γ:
   - Train the neural network with regularized loss:

$$E_{\text{reg}}(w) = E_{\text{train}}(w) + \frac{\gamma}{2} \sum_q w_q^2$$

   - Compute validation error on the validation set:

$$E_{\text{val}}(w) = E_{\text{val}}(w) + \frac{\gamma}{2} \sum_q w_q^2$$

3. Pick the γ^* that gives the lowest validation error:

$$\gamma^* = \arg\min_{\gamma} E_{\text{val}} \tag{57}$$

4. Finally, retrain the network **on all data** using the selected γ^* to get the final model.

This cross-validation approach ensures we select a regularization strength that balances fitting the training data well while maintaining good generalization to unseen data.

3.6.4 Dropout (Stochastic Regularization)

Even with weight decay and early stopping, large networks can still **overfit** because **neurons learn to co-adapt** too much. For example, neuron A might learn to rely on neuron B being active to make its predictions. If B overfits, then A will also overfit. We need a way to **force independence** among neurons, to make the network robust to missing or noisy signals.

The Idea. During training, we **randomly "drop" neurons** (i.e., set their output to 0) with a certain probability p, independently at each training iteration.

- Each forward pass uses a **different random subnetwork**.
- The model can't rely on any specific neuron always being present, so it must learn **redundant representations**.

Without dropout, the activation of neuron j in layer l is:

$$h_j^{(l)} = g\left(\sum_i w_{ij}^{(l)} h_i^{(l-1)} + b_j^{(l)}\right)$$

- $h_i^{(l-1)}$ are the activations from the previous layer.
- $w_{ij}^{(l)}$ are the weights connecting neuron i in layer $l-1$ to neuron j in layer l.
- $b_j^{(l)}$ is the bias term for neuron j in layer l.
- $g(\cdot)$ is the activation function (e.g., tanh, sigmoid).

Dropout introduces a random **mask** $m_j^{(l)}$ for each neuron, sampled from a **Bernoulli distribution** with parameter p (the probability of **keeping** the neuron). Formally:

$$m_j^{(l)} \sim \text{Bernoulli}(p) \Rightarrow m_j^{(l)} = \begin{cases} 1 & \text{with probability } p \text{ neuron is kept} \\ 0 & \text{with probability } 1-p \text{ neuron is dropped} \end{cases} \tag{58}$$

Then we define the **new (masked) activation** as:

$$\tilde{h}_j^{(l)} = m_j^{(l)} \cdot h_j^{(l)} = m_j^{(l)} \cdot g\left(\sum_i w_{ij}^{(l)} h_i^{(l-1)} + b_j^{(l)}\right) \tag{59}$$

- $h_j^{(l)}$ is the activation of neuron j in layer l.
- $m_j^{(l)}$ is a dropout mask (binary mask, 1 = keep neuron, 0 = drop neuron).
- p is the probability of **keeping** a neuron (typically $p = 0.5$ for hidden layers, $p = 0.8 - 0.9$ for input layer).
- $\tilde{h}_j^{(l)}$ is the **post-dropout activation** of neuron j in layer l.

Definition 6: Dropout

Dropout is a regularization technique where, *during training*, **each neuron is randomly dropped** (set to zero) with probability $1-p$, independently of other neurons. This **prevents co-adaptation of neurons** and encourages the network to learn robust features.

Formally, for neuron j in layer l:

$$m_j^{(l)} \sim \text{Bernoulli}(p) \quad \Rightarrow \quad m_j^{(l)} = \begin{cases} 1 & \text{with probability } p \\ 0 & \text{with probability } 1-p \end{cases}$$

$$\tilde{h}_j^{(l)} = m_j^{(l)} \cdot h_j^{(l)}$$

Where $\tilde{h}_j^{(l)}$ is the post-dropout activation.

What happens during training vs testing

During **training**, we apply dropout as described above, randomly dropping neurons with probability $1-p$. This forces the network to learn robust features that do not rely on any specific neuron. Formally, during training:

$$\tilde{h}_j^{(l)} = m_j^{(l)} \cdot h_j^{(l)}$$

During **testing**, dropout is disabled, so we use the **full network**, meaning that all neurons are active. However, during training each neuron was active only with probability p, so its **expected contribution** was smaller. To keep the scale of activations consistent between training and testing, we multiply each activation (equivalently, the outgoing weights) by p:

$$h_j^{(l,\text{test})} = p \cdot h_j^{(l)}$$

This is called the **Weight Scaling Rule**.

Definition 7: Weight Scaling Rule

In the **classical dropout formulation**, during testing we do **not** drop any neuron. Instead, we use the full network and **scale each activation by the keep probability** p:

$$h_j^{(l,\text{test})} = p \cdot h_j^{(l)} \tag{60}$$

Where $h_j^{(l)}$ is the activation of neuron j in layer l before scaling, and $h_j^{(l,\text{test})}$ is the scaled activation used during testing. Equivalently, one may view this as scaling the outgoing weights of that layer by p.

This is called the **Weight Scaling Rule**. Its purpose is to keep the **expected activation at test time equal to the expected activation**

during training. Indeed, during training:

$$\tilde{h}_j^{(l)} = m_j^{(l)} \cdot h_j^{(l)}, \qquad m_j^{(l)} \sim \text{Bernoulli}(p) \tag{61}$$

so:

$$\mathbb{E}\left[\tilde{h}_j^{(l)}\right] = \mathbb{E}\left[m_j^{(l)}\right] \cdot h_j^{(l)} = p \cdot h_j^{(l)} \tag{62}$$

Therefore, multiplying the full activation by p at test time preserves the same expected scale seen during training.

Remark: Inverted Dropout in Practice

Modern deep learning libraries usually implement **inverted dropout** instead of the classical weight scaling rule. In inverted dropout:

- During training, kept activations are scaled by $\frac{1}{p}$:

$$\tilde{h}_j^{(l)} = \frac{m_j^{(l)}}{p} \cdot h_j^{(l)}$$

- During testing, no scaling is needed:

$$h_j^{(l,\text{test})} = h_j^{(l)}$$

This is more convenient in practice because the network behaves normally at inference time.

Example 3: Dropout in a Simple Neural Network

Suppose a layer has 5 neurons with activations:

$$h^{(l)} = [0.4, 0.7, 0.3, 0.1, 0.9]$$

And we set dropout probability $p = 0.6$ (i.e., 60% chance to keep each neuron), and we randomly sample masks:

$$m^{(l)} = [1, 0, 1, 0, 1]$$

During training, the post-dropout activations are:

$$\tilde{h}^{(l)} = m^{(l)} \cdot h^{(l)} = [0.4, 0, 0.3, 0, 0.9]$$

During testing, we use the full activations scaled by p (each neuron's activation is multiplied by 0.6):

$$h^{(l)} \leftarrow p \cdot h^{(l)} = [0.24, 0.42, 0.18, 0.06, 0.54]$$

3.7 Tips & Tricks

Once we know the loss function, the optimization rule (gradient descent/backpropagation), and the regularization techniques (weight decay, dropout, etc.), we still face a **practical problem**: "*even if our model and data are correct, why does training sometimes fail, diverge, or get stuck?*". This last section focuses on the **engineering side** of deep learning, the small design decisions that make or break our model's ability to learn effectively.

In this section, we will cover six practical "tricks" to improve neural network training:

1. **Activation Function Saturation** (page 177): Sigmoid and Tanh saturate for large inputs, then gradients vanish (go to zero). We'll see the **zero-gradient problem** and why modern networks use non-saturating functions.

2. **ReLu and Variants** (page 180): the most widely used activation today. Simple, fast, avoids saturation, but comes with its own issues (dead neurons). Variants like **Leaky ReLu**, **ELU**, and **GELU** help mitigate these problems.

3. **Weight Initialization** (page 184): setting initial weights correctly is *critical* for stable gradient propagation. The **Xavier (Glorot)** and **He** initializations balance signal variance between layers to avoid vanishing/exploding gradients.

4. **Batch Normalization** (page 188): normalizes intermediate activations to keep each layer's input distribution stable across training. This speeds up convergence and allows for higher learning rates.

5. **Mini-Batch Training** (page 193): instead of computing gradients on the full dataset or single sample, we use small batches for better convergence and generalization. It's the practical compromise between **SGD** (Stochastic Gradient Descent) and **full-batch gradient descent**.

6. **Learning Rate Scheduling** (page 197): the learning rate controls the "step size" in optimization. Adaptive or decaying schedules (e.g., **Momentum**, **RMSProp**, **Adam**) ensure stable and efficient learning.

3.7.1 Activation Function Saturation

Problem: During backpropagation (page 100), the **gradient** must flow backward through all layers. If it becomes **very small** (≈ 0) in some layer, early layers **stop learning**. This is known as the **Vanishing Gradient Problem** (or **Zero-Gradient Problem**) and it happens when the **activation function saturates**.

What does "saturation" mean?

An **activation function** takes a neuron's input (weighted sum $z = w^T x + b$) and produces a non-linear output $g(z)$. For instance:

- The **sigmoid** function $g(z) = \frac{1}{1+e^{-z}}$
- The **tanh** function $g(z) = \tanh(z)$

Now, both are **S-shaped** (see Figure 7, page 64), they flatten for large positive or negative z values:

z range	sigmoid $g(z)$	tanh $g(z)$	Derivative $g'(z)$
Very negative	≈ 0	≈ -1	≈ 0
Near zero	≈ 0.5	≈ 0	≈ 0.25 (sigmoid), ≈ 1 (tanh)
Very positive	≈ 1	≈ 1	≈ 0

When $|z|$ is large, the **output saturates** (flattens) and the **derivative** $g'(z)$ becomes **very small** (≈ 0).

And why zero gradient is a disaster?

Let's review the backpropagation. In any neural network trained with gradient descent, each parameter (weight or bias) is updated according to the **core learning rule** (Equation 44, page 96):

$$w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} - \eta \cdot \frac{\partial E}{\partial w_{ij}^{(l)}} \quad \text{and} \quad b_i^{(l)} \leftarrow b_i^{(l)} - \eta \cdot \frac{\partial E}{\partial b_i^{(l)}}$$

Where:

- η is the learning rate.
- $\frac{\partial E}{\partial w_{ij}^{(l)}}$ and $\frac{\partial E}{\partial b_i^{(l)}}$ are the gradients of the loss E with respect to the weights and biases.

Each derivative is computed using the **chain rule** in backpropagation. For a neuron i in layer l, we have:

$$\frac{\partial E}{\partial w_{ij}^{(l)}} = \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial a_i^{(l)}} \cdot \frac{\partial a_i^{(l)}}{\partial w_{ij}^{(l)}} \qquad \frac{\partial E}{\partial b_i^{(l)}} = \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial a_i^{(l)}} \cdot \frac{\partial a_i^{(l)}}{\partial b_i^{(l)}}$$

Where:

- y is the output of the network.
- $a_i^{(l)} = g\left(z_i^{(l)}\right)$ is the activation of neuron i in layer l.
- $z_i^{(l)} = \sum_k w_{ik}^{(l)} a_k^{(l-1)} + b_i^{(l)}$ is the weighted input to neuron i in layer l.

The critical term here is $\frac{\partial y}{\partial a_i^{(l)}}$, which involves the derivative of the activation function $g'\left(z_i^{(l)}\right)$. If the activation function **saturates**, its derivative becomes almost zero: $g'\left(z_i^{(l)}\right) \approx 0$. This leads to:

$$\frac{\partial E}{\partial w_{ij}^{(l)}} \approx 0 \quad \text{and} \quad \frac{\partial E}{\partial b_i^{(l)}} \approx 0$$

As a result, the weight and bias updates become negligible:

$$w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} - \eta \cdot 0 = w_{ij}^{(l)} \quad \text{and} \quad b_i^{(l)} \leftarrow b_i^{(l)} - \eta \cdot 0 = b_i^{(l)}$$

So the weight and bias remain unchanged, effectively **halting learning** in that layer (neuron effectively **stops learning**).

Also, the **problem is not local**, it **propagates backward**! For a neuron in layer $l-1$, its gradient depends on the next layer:

$$\delta_j^{(l-1)} = g'(z_j^{(l-1)}) \cdot \sum_i w_{ij}^{(l)} \delta_i^{(l)}$$

If the next layer has small derivatives ($g'(z_j^{(l-1)}) \approx 0$) due to saturation, then $\delta_j^{(l-1)}$ also becomes very small, causing the same issue in layer $l-1$. This **cascading effect** can lead to **vanishing gradients** throughout the network, because **gradients shrink exponentially** as they are propagated backward through multiple layers with saturated activations.

Solutions?

In future sections, we will explore various strategies to mitigate the vanishing gradient problem caused by activation function saturation, including:

- Use activation functions that do not saturate easily, such as **ReLU** (page 180).
- **Properly initialize weights** to keep activations in the non-saturated region (page 184).
- Use normalization techniques like **Batch Normalization** to maintain stable activation distributions (page 188).
- Employ architectures like **Residual Networks** that facilitate gradient flow.

Deepening: Exploding Gradient Problem

Not all problems with gradients are about them becoming too small. Sometimes, they can become excessively large, leading to instability during training.
The **Exploding Gradient Problem** is the opposite of the vanishing gradient problem. It occurs when **gradients grow exponentially during backpropagation and become numerically unstable** (very large values), causing very large updates to the weights and divergence of the training process.

How does it happen? It typically occurs when:

- Weights are initialized with very large values.
- The network is very deep, and the product of derivatives during backpropagation leads to large gradients.

If each layer amplifies the gradient by a factor greater than 1, the gradients can grow exponentially as they are propagated backward through the layers. For example, if each layer multiplies the gradient by a factor of 2, after n layers, the gradient will be multiplied by 2^n, leading to extremely large values. With only 50 layers, this results in a factor of $2^{50} \approx 1.13 \times 10^{15}$, which is numerically unstable. So a tiny gradient (e.g., 10^{-10}) can become a huge value (e.g., 10^5) after backpropagating through 50 layers.

In practice: Symptoms we might see during training include:

- The loss suddenly becomes **NaN or Inf** after a few epochs.
- The network weights contain extremely large values.
- The gradient norm is enormous (hundreds or thousands).
- Training oscillates wildly or diverges completely.

This often happens in **deep fully connected networks** with bad initialization or **RNNs (Recurrent Neural Networks)** because gradients are multiplied through time steps.

Solutions? In future sections, we will explore strategies to mitigate the exploding gradient problem, including:

- **Gradient Clipping:** Limit the maximum value of gradients during backpropagation.
- **Proper Weight Initialization:** Use techniques like Xavier or He initialization to keep gradients stable (page 184).
- **Use of Normalization Layers:** Such as Batch Normalization to stabilize activations and gradients (page 188).
- **Architectural Changes:** Such as using LSTM or GRU units in RNNs to control gradient flow.

3.7.2 ReLU and Variants

Definition 8: ReLU Activation Function

The **Rectified Linear Unit (ReLU)** is defined as:

$$g(z) = \max(0, z) \tag{63}$$

And its derivative:

$$g'(z) = \begin{cases} 1, & \text{if } z > 0 \\ 0, & \text{if } z \leq 0 \end{cases} \tag{64}$$

So it's a **piecewise linear** function that outputs the input directly if it is positive; otherwise, it outputs zero. No exponential, no sigmoid, just a straight cutoff at zero.

Why does it work? The powerful simplicity of the ReLU

The ReLU activation function has become the default choice for many neural network architectures due to its simplicity and effectiveness. Here are some reasons why ReLU works so well:

- **Non-saturating**: for $z > 0$, the derivative is constant (1), which helps mitigate the vanishing gradient problem that plagues sigmoid and tanh activations (page 177).
- **Computationally cheap**: ReLU is simply a thresholding at zero, which is computationally efficient compared to sigmoid or tanh functions that require expensive exponentials.
- **Sparse activation**: In practice, many neurons output zero, leading to a sparse representation that can be beneficial for learning and generalization.
- **Fast convergence**: Empirically, networks using ReLU tend to converge faster during training compared to those using traditional activation functions.
- **Biological motivation**: ReLU is inspired by the behavior of real neurons, which are either activated (firing) or not (silent), resembling the ReLU activation pattern.

So is it the perfect activation function? Not quite...

Despite its advantages, the **biggest drawback** of ReLU is the **"dead ReLU" problem**. The **Dead ReLU Problem**, or **Dying ReLU Problem**, occurs when a **significant portion of neurons in a neural network become inactive and only output zero for any input**. This happens when the weights are updated in such a way that the input to the ReLU activation function is always negative, causing the neuron to output zero and effectively "die".

When a neuron dies, it stops learning because its gradient is zero (since the derivative of ReLU is zero for inputs less than or equal to zero). This can lead to a situation where a large number of neurons in the network are inactive, reducing the model's capacity to learn complex patterns in the data.

In simple terms, if a neuron's input z becomes **negative for all samples**, its output will always be zero. Then, the gradient will also be zero, and the neuron will be marked as dead (i.e., it will no longer contribute to learning). This often occurs due to: (1) large negative biases, (2) too high of a learning rate, or (3) poor weight initialization (with "*poor*" we mean that many neurons start in the negative region). Hence the need for **ReLU variants** that maintain some gradient even for negative inputs, allowing the network to continue learning and preventing neurons from dying.

ReLU Variants to the Rescue

To address the dead ReLU problem, several variants of the ReLU activation function have been proposed. Here the four most popular ones:

1. **Leaky ReLU**: a variant that **allows a small slope**, called α, **for negative inputs** (since the standard ReLU has a slope of 0 for negative inputs). It is defined as:

$$g(z) = \begin{cases} z, & \text{if } z > 0 \\ \alpha z, & \text{if } z \leq 0 \end{cases} \tag{65}$$

   where α is a small constant (e.g., 0.01). This creates a small, non-zero gradient when the unit is inactive, which helps to keep the neuron active during training. It is the simplest and most naïve attempt to solve the dead ReLU problem.

   Pros
   - ✔ Fixes dead neurons by allowing a small gradient when $z \leq 0$.
   - ✔ Keeps non-saturating property for positive inputs.

   Cons
   - ✘ Slight computational overhead compared to standard ReLU (but still cheap).
   - ✘ The choice of α is arbitrary and may require tuning.
   - ✘ Still linear for negative inputs, which may not capture complex patterns.

2. **Parametric ReLU (PReLU)**: similar to Leaky ReLU, but instead of using a fixed small slope α for negative inputs, **PReLU learns the slope parameter** during training. It is defined as:

$$g(z) = \begin{cases} z, & \text{if } z > 0 \\ az, & \text{if } z \leq 0 \end{cases} \tag{66}$$

   where a is a learnable parameter. This allows the model to adaptively learn the best slope for negative inputs based on the data. Useful when different neurons may benefit from different slopes.

Pros

- ✔ Learns the slope for negative inputs, potentially improving performance.
- ✔ Retains non-saturating property for positive inputs.

Cons

- ✘ Introduces additional parameters to learn, increasing model complexity.
- ✘ Slightly more computationally expensive than standard ReLU.

3. **Exponential Linear Unit (ELU)**: a smoother variant that **exponentially approaches a negative value** for negative inputs, defined as:

$$g(z) = \begin{cases} z, & \text{if } z > 0 \\ \alpha\left(e^z - 1\right), & \text{if } z \leq 0 \end{cases} \tag{67}$$

where α is a positive constant that controls the value to which ELU saturates for negative inputs. ELU has a smooth curve for negative inputs, which can help with learning (see Figure 21 page 183).

Pros

- ✔ Smooth gradient for negative inputs (no sharp corner at zero).
- ✔ Keeps mean activations close to zero, better convergence.
- ✔ Reduces bias shift, improving learning dynamics.

Cons

- ✘ More computationally expensive due to the exponential function.
- ✘ The choice of α may require tuning.

4. **Scaled Exponential Linear Unit (SELU)**: a **self-normalizing** activation function that **scales the output** to maintain a mean of zero and unit variance. It is defined as:

$$g(z) = \lambda \cdot \begin{cases} z, & \text{if } z > 0 \\ \alpha\left(e^z - 1\right), & \text{if } z \leq 0 \end{cases} \tag{68}$$

where λ and α are **predefined constants** (typically $\lambda \approx 1.0507$ and $\alpha \approx 1.6733$). Introduced with **Self-Normalizing Neural Networks** (Klambauer et al., 2017), SELU is designed to **keep the activations normalized throughout the network**, which can lead to faster convergence and improved performance.

Pros

- ✔ Self-normalizing properties help maintain stable activations.
- ✔ Reduces the need for batch normalization (since activations are kept normalized).
- ✔ Improves convergence speed and performance.

Cons

- ✘ More computationally expensive due to the exponential function.
- ✘ Requires careful weight initialization to maintain self-normalizing properties.

Activation	Formula	$g'(z<0)$	$g'(z>0)$	Notes
ReLU	$\max(0,z)$	0	1	Fast, simple, may die.
Leaky ReLU	$\max(\alpha z, z)$	α	1	Prevents dead neurons.
PReLU	$\max(az, z)$	a (learned)	1	Adaptive slope.
ELU	$\begin{cases} z, & z>0 \\ \alpha(e^z-1), & z\le 0 \end{cases}$	αe^z	1	Smooth, mean close to 0.
SELU	$\lambda \cdot \begin{cases} z, & z>0 \\ \alpha(e^z-1), & z\le 0 \end{cases}$	$\lambda\alpha e^z$	λ	Self-normalizing.

Table 3: Summary of ReLU, Leaky ReLU, PReLU, ELU, and SELU.

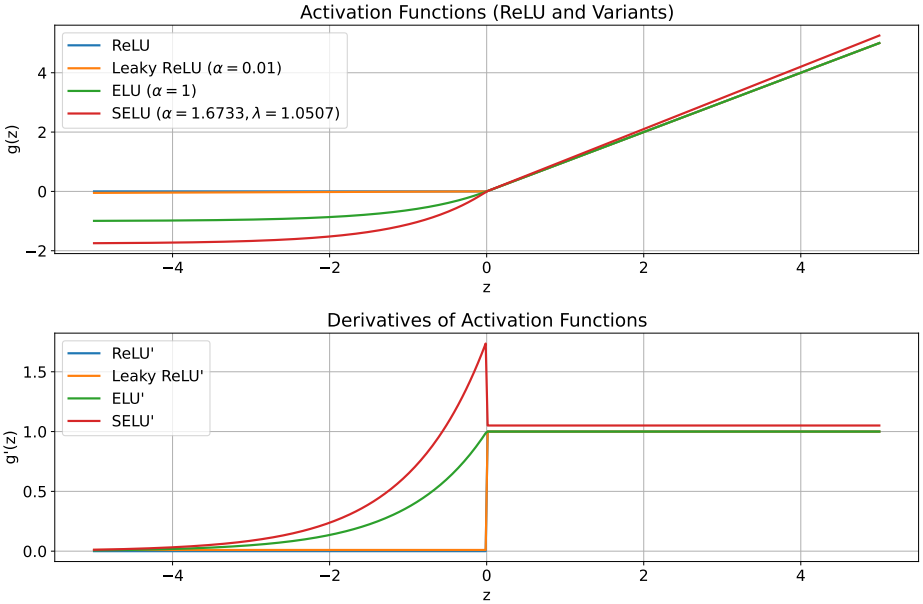


Figure 21: Comparison of ReLU and its variants: Leaky ReLU, PReLU, and ELU. Classic ReLU and Leaky ReLU are almost identical, with the Leaky having a slight slope for negative inputs. PReLU adapts its slope during training, whereas ELU provides a smooth transition for negative inputs. And SELU scales the output to maintain a mean of zero and unit variance.

3.7.3 Weight Initialization

When we initialize a neural network, we want activations and gradients to keep a stable scale across layers. Otherwise:

- If weights are **too small**, signals shrink layer after layer, leading to **vanishing gradients** (vanish gradient problem, page 177).
- If weights are **too large**, signals blow up exponentially, leading to **exploding gradients** (explode gradient problem, page 179).

Hence, initialization must **preserve the variance** of signals *forward* and *backward* through the network.

Forward-pass variance analysis

Consider one neuron:

$$z_j^{(l)} = \sum_{i=1}^{n_{l-1}} w_{ji}^{(l)} h_i^{(l-1)} + b_j^{(l)} \quad \text{where} \quad h_i^{(l-1)} = g\left(z_i^{(l-1)}\right)$$

Where $h_i^{(l-1)}$ are the **activations from the previous layer**, $w_{ji}^{(l)}$ are the **weights**, $b_j^{(l)}$ is the **bias**, and $g(\cdot)$ is the **activation function**. Assume:

- **Inputs** $h_i^{(l-1)}$ are i.i.d. (independent and identically distributed, page 108) with:
$$\underbrace{\mathbb{E}\left[h_i^{(l-1)}\right]}_{\text{mean}} = 0 \quad \text{and} \quad \underbrace{\text{Var}\left[h_i^{(l-1)}\right]}_{\text{variance}} = v_h$$
- **Weights** $w_{ji}^{(l)}$ are i.i.d. with:
$$\mathbb{E}\left[w_{ji}^{(l)}\right] = 0 \quad \text{and} \quad \text{Var}\left[w_{ji}^{(l)}\right] = v_w$$
- **Bias** $b_j^{(l)}$ with:
$$\mathbb{E}\left[b_j^{(l)}\right] = 0 \quad \text{and} \quad \text{Var}\left[b_j^{(l)}\right] = v_b$$

Then, the variance of $z_j^{(l)}$ is:

$$\text{Var}\left[z_j^{(l)}\right] = n_{l-1} \cdot v_w \cdot v_h + v_b$$

Where n_{l-1} is the number of neurons in layer $l-1$ (the previous layer). To keep the variance stable across layers, we want:

$$\text{Var}\left[z_j^{(l)}\right] = \text{Var}\left[h_i^{(l-1)}\right] = v_h \quad \Rightarrow \quad n_{l-1} \cdot v_w \cdot v_h + v_b = v_h$$

Assuming v_b is small, we get:

$$n_{l-1} \cdot v_w \cdot \cancel{v_h} + \cancel{v_b} = \cancel{v_h} \quad \Rightarrow \quad v_w = \frac{1}{n_{l-1}} \tag{69}$$

This means we should **initialize weights with variance** $v_w = \frac{1}{n_{l-1}}$ **to preserve forward-pass variance**.

Backward-pass variance analysis

During backpropagation, the gradient with respect to activations $\delta_i^{(l)} = \frac{\partial E}{\partial z_i^{(l)}}$ is computed as:

$$\delta_i^{(l)} = g'\left(z_i^{(l)}\right) \cdot \sum_{j=1}^{n_{l+1}} w_{ji}^{(l+1)} \delta_j^{(l+1)}$$

Where $g'(\cdot)$ is the derivative of the activation function, and $\delta_j^{(l+1)}$ are the gradients from the next layer. Assume:

- **Gradients** $\delta_j^{(l+1)}$ are i.i.d. with:

$$\mathbb{E}\left[\delta_j^{(l+1)}\right] = 0 \quad \text{and} \quad \text{Var}\left[\delta_j^{(l+1)}\right] = v_\delta$$

- **Weights** $w_{ji}^{(l+1)}$ are i.i.d. with:

$$\mathbb{E}\left[w_{ji}^{(l+1)}\right] = 0 \quad \text{and} \quad \text{Var}\left[w_{ji}^{(l+1)}\right] = v_w$$

- **Activation derivatives** $g'\left(z_i^{(l)}\right)$ are i.i.d. with:

$$\mathbb{E}\left[g'\left(z_i^{(l)}\right)\right] = 0 \quad \text{and} \quad \text{Var}\left[g'\left(z_i^{(l)}\right)\right] = v_{g'}$$

Then, the variance of $\delta_i^{(l)}$ is:

$$\text{Var}\left[\delta_i^{(l)}\right] = n_{l+1} \cdot v_w \cdot v_\delta \cdot v_{g'}$$

Where n_{l+1} is the number of neurons in layer $l+1$ (the next layer). To keep the variance stable across layers, we want:

$$\text{Var}\left[\delta_i^{(l)}\right] = \text{Var}\left[\delta_j^{(l+1)}\right] = v_\delta \quad \Rightarrow \quad n_{l+1} \cdot v_w \cdot v_\delta \cdot v_{g'} = v_\delta$$

Assuming v_δ is non-zero, we get:

$$n_{l+1} \cdot v_w \cdot \cancel{v_\delta} \cdot v_{g'} = \cancel{v_\delta} \quad \Rightarrow \quad v_w = \frac{1}{n_{l+1} \cdot v_{g'}} \tag{70}$$

This means we should **initialize weights with variance** $v_w = \dfrac{1}{n_{l+1} \cdot v_{g'}}$ **to preserve backward-pass variance**.

☆ The Xavier (Glorot) and He (Kaiming) Initialization

To satisfy both forward and backward variance preservation, we can combine equations (69) and (70). Exists two popular initialization schemes:

- **Xavier (Glorot) Initialization**. **Glorot & Bengio (2010)**, two researchers from the University of Montreal, proposed a balanced compromise between forward and backward variance preservation. They combined the two equations by averaging the number of neurons in the previous and next layers:

$$\text{Var}\left[w_{ji}^{(l)}\right] = v_w = \frac{2}{n_{l-1} + n_l} \tag{71}$$

  - n_{l-1} is the number of input units **from** the previous layer **to** layer l.
  - n_l is the number of output units **from** layer l **to** the next layer.

  It works well for activation functions like **tanh** or **sigmoid**, where both positive and negative activations are symmetric and can saturate easily. Hence, Xavier initialization keeps activations within a "safe" non-saturated range, reducing the change of vanishing gradients (or exploding gradients).

- **He (Kaiming) Initialization**. **He et al. (2015)**, researchers from Microsoft Research, proposed an initialization scheme specifically designed for **ReLU** and its variants. He proofed that with ReLU activations, only about half the neurons are active at a time (since ReLU outputs zero for negative inputs). Therefore, the effective variance of activations halves:

$$\text{Var}\left[h^{(l)}\right] = \frac{1}{2} \cdot \text{Var}\left[z^{(l)}\right]$$

  Where $h^{(l)}$ are the activations after ReLU, and $z^{(l)}$ are the pre-activation values. To compensate for this reduction, **He et al. (2015)** proposed to double the variance of weights:

$$\text{Var}\left[w_{ji}^{(l)}\right] = v_w = \frac{2}{n_{l-1}} \tag{72}$$

  This ensures that activations after the ReLU maintain a unit variance and gradients remain stable during backpropagation.

In summary, both initialization methods were designed to **control the variance** of: activations during the **forward pass** and gradients during the **backward pass**. They make sure that, *on average*:

$$\text{Var}\left[z^{(l)}\right] \approx \text{Var}\left[z^{(l-1)}\right] \quad \text{and} \quad \text{Var}\left[\delta^{(l)}\right] \approx \text{Var}\left[\delta^{(l+1)}\right]$$

So they explicitly avoid both:

✗ $\text{Var} < 1$ (vanishing gradients)

✓ $\text{Var} = 1$ (stable)

✗ $\text{Var} > 1$ (exploding gradients)

The goal is a **critical regime** (Var = 1) where the signal neither dies nor explodes. But, remember that Xavier and He initialization prevent both vanishing and exploding gradients only at the **start of training**. During training, weights are updated, and the variance can drift away from the ideal value. Therefore, other techniques like **batch normalization** (page 188) are often used in conjunction to maintain stable activations and gradients throughout training.

Initialization	Recommended for	Weight Variance
Xavier (Glorot)	Sigmoid / Tanh	$\frac{2}{n_{in} + n_{out}}$
He (Kaiming)	ReLU / Leaky ReLU	$\frac{2}{n_{in}}$

Table 4: Summary of popular weight initialization schemes. n_{in} is the number of input units to the layer, and n_{out} is the number of output units from the layer.

In practice, these initialization schemes significantly improve training stability and convergence speed, especially in deep networks. Modern deep learning frameworks (like `TensorFlow` and `PyTorch`) implement these initializations by default when creating layers.

3.7.4 Batch Normalization

When training deep networks, **each layer's input distribution keeps changing** as previous layers update their weights. This phenomenon is called **internal covariate shift**.

In machine learning, the **Internal Covariate Shift** (or simply **Covariate Shift**) happens when the **distribution or input data changes between training and testing**. For example, we train a model to detect cats using bright studio photos; then we test it on dark or outdoor photos. The model struggles because $P_{train}(X) \neq P_{test}(X)$, even though the task (detect cats) remains the same. That's the **classic covariate shift** problem: a change in the *input feature distribution* that forces the model to adapt to new data patterns. In a deep neural network, every layer has its own *inputs*, and those inputs come from the **previous layer's outputs**, which are *constantly changing* as training proceeds. So, while the external dataset stays the same, the **internal data** (the activations flowing between layers) keeps shifting its distribution during training. This is what we call **internal covariate shift**, and it is called "*internal*" because it happens *within* the network itself, not just between training and testing datasets.

In simple terms, every time earlier layers update their weights, the *input distribution* to later layers changes. This means that from the perspective of layer l:

- At iteration 1, its input $z^{(l)}$ might have mean $= 0$ and variance $= 1$.
- At iteration 100, the same input might have mean $= 3$ and variance $= 10$.

So layer l is constantly trying to adapt to a **moving target** distribution, its own input is unstable.

Why is internal covariate shift a problem? Neural networks assume (implicitly) that the distribution of each layer's inputs stays within a "reasonable" range. But if these distributions shift too much:

- Neurons may enter the **saturated region** of activation functions (like sigmoid or tanh), leading to **vanishing gradients**.
- Gradient flow becomes unstable, making optimization **slower** and more **difficult**.
- Convergence slows down dramatically, requiring **smaller learning rates** and careful initialization.

In other words, internal covariate shift makes training **harder and slower**, because every layer must re-learn how to operate each time the layers before it change.

Solution: Batch Normalization

Since covariate shift arises from changing input distributions, the solution is to **stabilize these distributions**. This is where **Batch Normalization (BN)** comes in. Introduced by Sergey Ioffe and Christian Szegedy in 2015 [6], BN fixes this problem by **re-centering and re-scaling** each layer's input at every training step, so that:

$$\mathbb{E}\left[z^{(l)}\right] = 0 \quad \text{and} \quad \text{Var}\left[z^{(l)}\right] = 1 \tag{73}$$

Where $z^{(l)}$ is the input to layer l, $\mathbb{E}$ is the expectation (mean), and Var is the variance. That means the input distribution to each layer remains stable, even as earlier layers' weights keep evolving. So layer l can focus on learning *useful transformation*, not on constantly re-adjusting to a changing scale or mean.

Formally, let's denote the pre-activations of a neuron as $z_i^{(l)}$ for the i-th example in a batch of size m. **Batch Normalization performs the following steps during training**:

1. **Compute the *batch mean***:

$$\mu_B = \frac{1}{m} \cdot \sum_{i=1}^{m} z_i^{(l)} \tag{74}$$

   Where μ_B is the mean of the pre-activations over the batch, and m is the batch size. It means that we take the average of the $z_i^{(l)}$ values across all examples in the batch to **find the central tendency of the activations for that layer at this training step** l.

2. **Compute the *batch variance***:

$$\sigma_B^2 = \frac{1}{m} \cdot \sum_{i=1}^{m} \left(z_i^{(l)} - \mu_B\right)^2 \tag{75}$$

   Where σ_B^2 is the variance of the pre-activations over the batch, and m is the batch size. It means that we calculate the average of the squared differences between each activation $z_i^{(l)}$ and the batch mean μ_B (calculated in the previous step). This gives us a **measure of how spread out the activations are around the mean for that layer at this training step** l.

3. ***Normalize* each activation**:

$$\hat{z}_i^{(l)} = \frac{z_i^{(l)} - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} \tag{76}$$

   Where $\hat{z}_i^{(l)}$ is the normalized activation, and ε is a small constant added for numerical stability, preventing division by zero. After this step, the normalized activations $\hat{z}_i^{(l)}$ have mean 0 and variance 1 across the batch. But, since this step **destroys** the original mean and variance information of z, we need to add a way to **restore** it if needed.

4. ***Scale and shift* with learnable parameters**:

$$y_i^{(l)} = \gamma \cdot \hat{z}_i^{(l)} + \beta \tag{77}$$

Where $y_i^{(l)}$ is the final output of the Batch Normalization layer, and γ and β are learnable **parameters that allow the network to restore the original distribution if needed**.

If Batch Normalization forces every layer to have mean 0 and variance 1, how can the two parameters γ and β possibly recover the *original* distribution? Isn't the information lost? Normalization helps stability, but what if a neuron *needs* its activations to be, say: shifted up (mean = 2) or spread wider (variance = 9)? If we stop at pure normalization we'd **limit the representational power** of the layer, because every neuron would have to operate under the same fixed scale 1 and mean 0. That's where the learnable parameters γ and β come in:

- γ (scale) allows the network to **stretch or compress** the normalized activations, effectively controlling the variance.
- β (shift) allows the network to **move** the normalized activations up or down, effectively controlling the mean.

This lets the network **relearn any affine transformation** of the normalized values.

Example 4: Restoring Original Distribution with γ and β

Suppose before normalization a neuron had:

$$\mu_{\text{orig}} = 2, \quad \sigma_{\text{orig}} = 3$$

After normalization, the activations have:

$$\hat{z}_i^{(l)} = \frac{z_i - 2}{3} \quad \Rightarrow \quad \mathbb{E}[\hat{z}] = 0, \quad \text{Var}[\hat{z}] = 1$$

Then, if the network learns:

$$\gamma = 3, \quad \beta = 2$$

The final output becomes:

$$y_i^{(l)} = 3 \cdot \hat{z}_i^{(l)} + 2 = 3 \cdot \frac{z_i - 2}{3} + 2 = z_i$$

Thus, the original distribution is perfectly restored!

So the full Batch Normalization transformation can be summarized as:

$$y_i^{(l)} = \text{BN}\left(z_i^{(l)}\right) = \gamma \cdot \hat{z}_i^{(l)} + \beta = \gamma \cdot \frac{z_i^{(l)} - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} + \beta \tag{78}$$

When to apply Batch Normalization?

During training, Batch Normalization is applied **inside the forward pass of the network**, right after the linear transformation (affine operation) and before the activation function. Formally, for layer l:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

Batch Normalization acts as:

$$\text{BN: } \hat{z}^{(l)} = \frac{z^{(l)} - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} \quad \text{then} \quad y^{(l)} = \gamma \hat{z}^{(l)} + \beta$$

And then:

$$a^{(l)} = g\left(y^{(l)}\right) \quad \text{where } g \text{ is the activation function (e.g., ReLU, sigmoid)}$$

So the order is typically **Linear** $\rightarrow$ **Batch Norm** $\rightarrow$ **Activation**.

What about during inference (testing)?

During **inference (testing)**, the model process **one example at a time** (or a few), not large batches. Therefore, computing "batch statistics" doesn't make sense anymore because a single sample doesn't have a meaningful mean or variance. Instead, we use **running estimates** (a.k.a. *moving averages*) of mean and variance that were computed during training. Specifically:

1. **Running averages during training**. As we train, we maintain running estimates of the mean and variance for each layer:

$$\mu_{\text{running}} = (1 - \alpha)\mu_{\text{running}} + \alpha \mu_B \tag{79}$$

$$\sigma^2_{\text{running}} = (1 - \alpha)\sigma^2_{\text{running}} + \alpha \sigma_B^2 \tag{80}$$

   Where α is a small constant (e.g., 0.1) that controls the update rate. These running estimates capture the overall distribution of activations over the entire training set.

2. **At inference (testing)**. At inference, Batch Normalization uses these **"frozen" running estimates** instead of batch statistics:

$$y_i^{(l)} = \gamma \cdot \frac{z_i^{(l)} - \mu_{\text{running}}}{\sqrt{\sigma^2_{\text{running}} + \varepsilon}} + \beta$$

   These are constants, no mini-batch dependence, no randomness. This ensures that the model's behavior is consistent and stable during inference.

Benefits of Batch Normalization

- **Stabilizes training**: Keeps activations in non-saturating range, reducing vanishing/exploding effects.
- **Allows higher learning rates**: Because activations are normalized, large updates won't blow up the model.
- **Regularization effect**: Adds small noise (due to batch statistics), improving generalization.
- **Reduces sensitivity to initialization**: Training becomes less dependent on perfect Xavier/He settings.
- **Improves convergence speed**: Layers adapt faster because their inputs are more predictable.

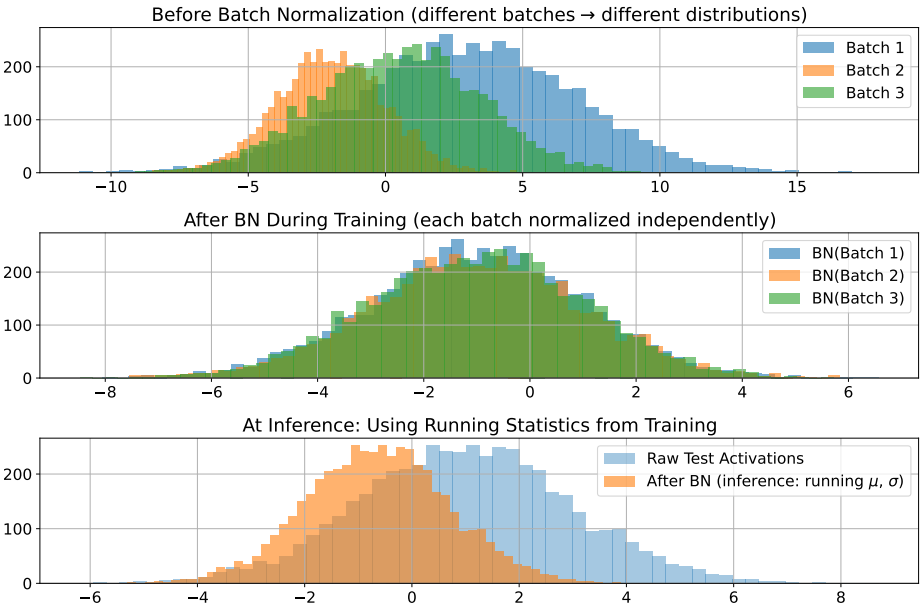


Figure 22: Batch Normalization normalizes layer inputs to stabilize training. Before BN, each training batch has a very different distribution (different mean & variance, top plot). After BN (during training), each batch is independently normalized to roughly the same mean and variance — stable inputs for the next layer (middle plot). At inference (testing), the model uses the running averages of mean and variance collected during training, producing a fixed and consistent normalization for new unseen data (bottom plot).

3.7.5 Mini-Batch Training

When we train a neural network, we want to minimize the **empirical risk**:

$$E(w) = \frac{1}{N} \cdot \sum_{n=1}^{N} L\left(f\left(x_n; w\right), t_n\right)$$

- N is the number of training samples
- L is the loss function
- $f(x_n; w)$ is the model's prediction for input x_n with parameters w (outputs)
- t_n is the true target for input x_n

Theoretically, the gradient for updating weights should be computed over the entire dataset, over **all N samples**:

$$\nabla_w E(w) = \frac{1}{N} \cdot \sum_{n=1}^{N} \nabla_w L\left(f\left(x_n; w\right), t_n\right)$$

But in practice, especially with large datasets, this is computationally expensive and inefficient.

Definition 9: Empirical Risk

The **Empirical Risk** is the average loss computed over the training dataset:

$$E(w) = \frac{1}{N} \cdot \sum_{n=1}^{N} L\left(f(x_n; w), t_n\right)$$

It provides a finite-sample approximation of the **expected (true) risk**:

$$\mathbb{E}_{(x,t)\sim D}\left[L(f(x; w), t)\right]$$

which **cannot be computed directly** because the true data distribution D (from which the training samples are drawn) is **unknown**[a]. Due to this, we approximate D using the dataset $E(w)$ (the empirical risk), and we minimize $E(w)$ to find the best parameters w for our model. Minimizing empirical risk is the core principle of learning from data.

It is called *empirical* because it is based on the **observed data** (our dataset), and *risk* because it quantifies the expected loss (or "risk") of the model's predictions (i.e., how well the model performs on the training data).

[a] It is unknown because we only have access to a finite dataset, and we cannot know the true underlying distribution that generated the data.

Solution: Mini-Batch Gradient Descent

Instead of using all N samples to compute the gradient, we use a smaller subset of the data called a **mini-batch**. In general, the mini-batch technique is one of a set of batching techniques. The main batch techniques are:

- **Full-Batch Gradient Descent (BGD)**: uses the entire dataset to compute the gradient. It is the method that we introduced when we covered gradient descent, as it is the most general (it uses all the data, page 94). However, it allows to get the exact gradient, but it is computationally **expensive** for large datasets and can lead to **slow convergence**.
- **Stochastic Gradient Descent (SGD)**: uses a single sample to compute the gradient at each iteration. We introduced it when we covered stochastic gradient descent (page 119) because we wanted to show how randomness can help in optimization. It is computationally **efficient** and can lead to **faster convergence**, but the gradient estimate is **noisy** and can lead to **unstable updates**.
- **Mini-Batch Gradient Descent (MBGD)**: uses a small subset of the dataset (mini-batch) to compute the gradient at each iteration. It is a compromise between BGD and SGD, **balancing computational efficiency and gradient accuracy**. It is widely used in practice due to its effectiveness in training deep neural networks.

The **Mini-Batch Gradient Descent (MBGD)** algorithm works as follows:

1. Shuffle the training dataset to ensure randomness.
2. **Divide** the dataset into **mini-batches of size** m (where $m \ll N$, much smaller than N).
3. For **each mini-batch** $B = \{x_1, x_2, \ldots, x_m\}$:
   (a) Compute the **gradient of the loss function** with respect to the weights using only the samples in the mini-batch:
   $$\nabla_w E_B(w) = \frac{1}{m} \sum_{i=1}^{m} \nabla_w L\left(f\left(x_i; w\right), t_i\right)$$
   (b) **Update the weights** using the computed gradient:
   $$w \leftarrow w - \eta \nabla_w E_B(w)$$
   where η is the learning rate.
4. Repeat until convergence or for a fixed number of epochs.

```
Input:
    Training dataset 𝒟 = {(x_1, t_1), ..., (x_N, t_N)}
    Model f(x; w)
    Loss function L(ŷ, t)
    Learning rate η
```

```
    Mini-batch size m
    Number of epochs T

Initialize parameters w

for epoch = 1 to T:
    Shuffle the training dataset D

    Split D into mini-batches
        B_1, B_2, ..., B_K
    where each mini-batch has size m
    (except possibly the last one)

    for each mini-batch B_k:
        Compute mini-batch loss:
            E_{B_k}(w) ← 1/|B_k| Σ_{(x_i,t_i)∈B_k} L(f(x_i; w), t_i)

        Compute gradient:
            g_k ← ∇_w E_{B_k}(w)

        Update parameters:
            w ← w − η g_k

Output:
    Trained parameters w
```

Listing 9: Mini-Batch Gradient Descent Algorithm

⚖ The trade-off: Stability vs. Speed

Choosing the right mini-batch size is crucial:

Property	Small ($1 \leq m \leq 32$)	Large ($256 \leq m \leq 8192$)
Computation per step	Very low	Very high
Update frequency	Very high	Very low
Gradient noise	High, helps generalization	Low, risk of sharp minima
Convergence	Fast but noisy	Smooth but slower learning per sample
GPU parallelism	Inefficient	Efficient

So **too small batches** can lead to **noisy updates** and **unstable training**, while **too large batches** can lead to **slow convergence** and **poor generalization**. A common practice is to start with a moderate batch size (e.g., 32, 64, or 128) and adjust based on the model's performance and available computational resources.

Impact on convergence and generalization

Mini-batch training affects both convergence and generalization of neural networks:

- **Convergence**. The stochasticity (small random fluctuations in gradient direction) helps the optimizer: escape shallow or sharp local minima, explore a larger portion of the loss landscape, and converge faster in expectation.
- **Generalization**. Mini-batch noise acts as a **regularizer**, preventing the model from memorizing training data perfectly (and thus overfitting).
  - Too large a batch → almost deterministic gradient → risk of converging to **sharp minima** → poor generalization.
  - Smaller batches → noisier gradients → exploration of **flatter minima** → better generalization.

3.7.6 Learning Rate Scheduling

The **learning rate** η is one of the most important hyperparameters in training neural networks. It controls how big the weight updates are at each step of the optimization process:

$$w_{k+1} = w_k - \eta \nabla_w E(w_k)$$

If η is too **large**, training may **diverge** (overshoot minima). If it's too **small**, training is **slow** and can get stuck in **local minima** or **plateaus**. So we need ways to make the learning rate **adapt intelligently over time**.

⚖ Fixed vs. Adaptive Learning Rates

There are four main strategies for setting the learning rate during training:

- **Fixed Learning Rate** (baseline). The learning rate η remains **constant** throughout training. So **gradient descent (GD)** or **stochastic gradient descent (SGD)** uses the same η at every iteration:

  $$w_{k+1} = w_k - \eta \nabla_w E(w_k)$$

  Works fine for simple convex problems (for example Figure 12, page 98), but for deeper networks, loss landscapes are complex, and a fixed η can lead to **slow convergence** or divergence.

- **Learning Rate Scheduling (decay)**. The learning rate η is **reduced over time** according to a predefined schedule. Typical schedules include:
  - **Step Decay**: reduce η by a factor (e.g., halve it) every s epochs:

    $$\eta_t = \eta_0 \cdot \text{drop}^{\lfloor t \div s \rfloor} \tag{81}$$

    Where η_0 is the initial learning rate, t is the epoch number, and drop is the factor by which to reduce η.
  - **Exponential Decay**: reduce η by a factor **every** epoch:

    $$\eta_t = \eta_0 \cdot e^{-\lambda t} \tag{82}$$

    Where λ is the decay rate and t is the epoch number.
  - **1/t Decay**: reduce η according to the inverse of the epoch number:

    $$\eta_t = \frac{\eta_0}{1 + \lambda t} \tag{83}$$

    Where λ is the decay rate and t is the epoch number. It is a classic stochastic approximation rule.
  - **Cosine Annealing**: vary η according to a cosine function:

    $$\eta_t = \frac{\eta_0}{2} \cdot \left(1 + \cos\left(\frac{t}{T}\pi\right)\right) \tag{84}$$

    Where T is the maximum number of epochs. This allows for periodic "*restarts*" of the learning rate, because the cosine function oscillates (i.e., the interval $[0, \pi]$ can be repeated multiple times during training).

In practice, **learning rate is set to a relatively high value** initially to allow for rapid learning, and then **gradually decreased** to allow for fine-tuning as training progresses.

- **Momentum: adding inertia to updates**. Since plain SGD (Stochastic Gradient Descent) may zig-zag in narrow valleys of the loss surface, **momentum** accelerates learning by accumulating a **velocity vector** in directions of persistent reduction in loss:

$$v_t = \alpha v_{t-1} - \eta \nabla_w E(w_t) \tag{85}$$

Then weights are updated using this velocity:

$$w_{t+1} = w_t + v_t \tag{86}$$

Where v_t is the velocity at time t, $\alpha \in [0, 1)$ is the momentum coefficient (typically around 0.9), and η is the learning rate.

- **Adaptive Learning Rate Methods**. These methods adjust the learning rate for each parameter individually based on historical gradients:
  - **RMSProp (Root Mean Square Propagation)**. Adapts the step using a *running average of squared gradients*:

$$\begin{aligned} s_t &= \rho s_{t-1} + (1-\rho) g_t^2 \\ w_{t+1} &= w_t - \frac{\eta}{\sqrt{s_t + \varepsilon}} g_t \end{aligned} \tag{87}$$

    Where s_t is the running average of squared gradients, ρ is the decay rate (typically around 0.9), g_t is the gradient at time t, and ε is a small constant to prevent division by zero. If gradients are large, then the denominator increases, and the step size decreases, and vice versa. This helps **stabilize updates and allows for larger learning rates**.
  - **Adam (Adaptive Moment Estimation)**: combines momentum (mean of gradients) and RMSProp (variance of gradients) by maintaining both a velocity vector and an exponentially decaying average of squared gradients:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1-\beta_1^t} \\ \hat{v}_t &= \frac{v_t}{1-\beta_2^t} \\ w_{t+1} &= w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \end{aligned} \tag{88}$$

    Where:
    - m_t is the first moment (mean) estimate,

* v_t is the second moment (uncentered variance) estimate,
* β_1 and β_2 are decay rates for the moment estimates (typically $\beta_1 = 0.9$, $\beta_2 = 0.999$),
* $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected estimates,
* ε is a small constant to prevent division by zero, typically around 10^{-8}.

Adam **adapts quickly**, **handles noisy gradients**, and **works well with mini-batches**. However, sometimes **generalizes worse** than SGD with momentum (flatter minima are less explored).

In practice, we can take a hybrid approach:

1. **Start with Adam** for general-purpose training, especially with complex architectures or noisy data.
2. **Switch to SGD with Momentum** for final fine-tuning to potentially achieve better generalization.
3. **Learning rate scheduling** (cosine, step, or warm restarts) is used even *with* Adam; we can adapt the *global* η while Adam adapts per-parameter rates.

4 Recurrent Neural Networks (RNNs)

In the previous sections, we have explored Feed-Forward Neural Networks (page 57), which are well-suited for tasks where inputs and outputs are fixed in size and independent of each other. So far, every model we have discussed processes data in a **static** manner: they took a fixed-size input vector $\mathbf{x}$ and produced a fixed-size output vector $\mathbf{y}$, **without considering any temporal or sequential dependencies** (no history, no sequence, no memory).

However, many real-world applications involve sequential data, where the order of the data points matters, such as time series analysis, natural language processing, and speech recognition. To handle such sequential data, we need a different type of neural network architecture known as **Recurrent Neural Networks (RNNs)**. They are models designed to handle **sequences** of data by maintaining a **hidden state** that captures information about previous time steps, allowing them to exhibit temporal dynamic behavior.

- **Sequence Modeling**: RNNs are specifically designed to model sequences of data, making them suitable for tasks where the order of inputs matters.
- **Memoryless vs Memory-based models**: Unlike FNNs, which are memoryless and process each input independently, RNNs have a form of memory that allows them to retain information from previous inputs in the sequence. We will explore some models that exhibit this property, such as Autoregressive Models, Dynamical Systems, Hidden Markov Models (HMMs), and RNNs themselves.
- **Recurrent Neural Networks (RNNs)**: RNNs are a class of neural networks that have connections that form directed cycles, allowing information to persist over time. This enables them to capture temporal dependencies in sequential data. We will delve into the architecture of RNNs, recurrent connections, and hidden states.
- **Backpropagation Through Time (BPTT)**: Training RNNs involves a specialized version of backpropagation called Backpropagation Through Time (BPTT). We will discuss how BPTT works and how it allows RNNs to learn from sequential data.
- **Vanishing and Exploding Gradients**: RNNs can suffer from vanishing and exploding gradient problems during training, which can hinder their ability to learn long-term dependencies. We will explore these issues and discuss techniques to mitigate them.
- **Long Short-Term Memory (LSTM)**: To address the limitations of standard RNNs, we will introduce Long Short-Term Memory (LSTM) networks, which are a type of RNN designed to capture long-term dependencies more effectively.

In simple terms, if a feed-forward neural network maps a function from input to output, a recurrent neural network maps a sequence of inputs to a sequence of outputs, taking into account the temporal dependencies between the inputs. This makes RNNs particularly powerful for tasks involving sequential data, where the context provided by previous inputs is crucial for making accurate predictions.

4.1 Sequence Modeling

⚖ Static vs Dynamic Datasets

Until now, all the models we have seen (Perceptron, Feed-Forward, etc.) assumed **static datasets**, i.e., each input $\mathbf{x}_n$ is **independent** from the others:

$$\mathbf{x}_n = [x_1, x_2, \dots, x_I] \xrightarrow{\text{produces}} \hat{y}_n = f(\mathbf{x}_n; \theta)$$

This means that the network **does not know any notion of time** or order of the inputs. Each sample is processed as a separate point in the dataset. However, **many real-world problem are sequential**, for example:

- Speech recognition (audio signal is a sequence of sound waves)
- Stock price prediction (time series data)
- Text generation (sequence of words or characters)
- Human motion capture (sequence of body poses over time)

These require **dynamic datasets**, where:

$$\mathbf{X} = [\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T]$$

Represents **a temporal sequence** rather than independent samples. Here, each observation $\mathbf{x}_t$ depends on the past:

$$\mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{x}_{t-2}, \dots)$$

Thus, a model must **remember or integrate information** across time.

🕐 From Fixed Inputs to Temporal Sequences

Now that we have established the difference between *static* and *dynamic* datasets, *how can we adapt our models to process sequences?* In a **feed-forward neural network (FNN)**, the computation is:

$$\hat{y} = f(\mathbf{x}; \mathbf{W})$$

Where:

- $\mathbf{x} \in \mathbb{R}^I$ is a fixed-size input vector where I is the number of features.
- $\mathbf{W}$ are the model parameters (weights and biases).
- $f(\cdot)$ is a non-linear function (the neural network activation functions).
- $\hat{y}$ is the output prediction.

Every sample is processed **independently**, the network "sees" no link between sample $\mathbf{x}_n$ and $\mathbf{x}_{n+1}$. But in sequential data, we have **temporal sequences**:

$$\mathbf{X} = [\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T]$$

Each vector $\mathbf{x}_t$ is not isolated and depends on what came before. A key point to note is that the **current observation x_t only has meaning when considered in the context of the past**. For example, in language modeling, the word "bank" can refer to a financial institution or the side of a river, depending on the preceding words. Therefore, to effectively model sequences, we need architectures that can **capture temporal dependencies** and **retain information** from previous time steps. To handle sequences, we define a **temporal model**:

$$\hat{y}_t = f\left(x_t, x_{t-1}, \ldots, x_0\right) \tag{89}$$

Where:

- $\hat{y}_t$ is the output prediction at time t.
- $f(\cdot)$ is a function that considers ("remembers") the current input x_t and all previous inputs $x_{t-1}, \ldots, x_0$.

In vector form, this corresponds to an **ordered sequence of inputs and outputs**:

$$\mathbf{x} = [x_0, x_1, \ldots, x_T] \quad \mathbf{Y} = [y_0, y_1, \ldots, y_T]$$

Why fixed Networks fail. The first attempt to fix this problem could be to use a **fixed feed-forward network** that reuse knowledge from one input to the next. However, this approach has several limitations:

- There is no explicit mechanism to store **context** or **memory** of previous inputs.
- Parameters are shared across time, but **activations do not carry over** from one time step to the next.
- Most importantly, the model is explicitly **memoryless** and cannot capture temporal correlations beyond the current input.

If we forced a static Neural Network to handle a sequence, we'd need to **concatenate all time steps** into a single giant input vector:

$$\mathbf{x} = [x_0, x_1, \ldots, x_T]$$

Which explodes in size and loses flexibility (variable-length sequences can't be handled).

Idea behind Sequence Modeling. We introduce the concept of a **Temporal Dimension** and **State Propagation**. At each time step t:

1. The model takes x_t as input.
2. It combines it with **internal state** h_{t-1} (the memory from the previous time step).
3. It produces both:
   - Output: y_t
   - New hidden state:

$$h_t = f_h\left(x_t, h_{t-1}\right) \tag{90}$$

Where h_t captures information from all previous inputs up to time t, and y_t is the prediction based on the current input and the accumulated context.

This recursive definition is the core idea of **Recurrent Neural Networks (RNNs)**: a single model unrolled through time, where the same parameters are used at each time step, but the hidden state allows information to flow across time. It is recursive because the output at time t depends on the output at time $t-1$ through the hidden state, then $t-2$, and so on.

Handling Time Dimension: Temporal Data Representation

The theory behind sequence modeling requires us to rethink how we represent data. Instead of a single-fixed input space, we now have a **spatio-temporal space** that includes both **Feature Dimension** I and **Time Dimension** T. So instead of a single flat vector $\mathbf{x}$, we have a **sequence**:

$$\mathbf{X} = [\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_T] \in \mathbb{R}^{I \times (T+1)}$$

This means that at every discrete time t, we observe:

$$\mathbf{x}_t = [x_{t,1},\ x_{t,2},\ \ldots,\ x_{t,I}]$$

Where $x_{t,i}$ is the i-th feature at time t, and $\mathbf{x}_t$ is a vector of features measured at that time step t. Each vector is **not independent** but part of a sequence where the order matters, and this is the essence of **sequential modeling**. We can express this dependency as:

$$P\left(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_{t-2}, \ldots, \mathbf{x}_0\right) \tag{91}$$

This conditional probability captures the idea that the current observation $\mathbf{x}_t$ depends on all previous observations in the sequence.

Types of models

There are **different modeling philosophies** for handling memory in sequence data. We can group them into two big categories:

- Memoryless Models. These models **ignore the internal notion of time**, and treat previous inputs as just *extra inputs*. Two models we will see in the next sections fall into this category:
  - **Autoregressive (AR) Models** (page 205): Predict the current output based on a fixed number of previous inputs.
  - **Feed-Forward Time-Delay Neural Network (TDNN)** (page 207): Use a sliding window of past inputs as additional features.
- Models with Memory. These models **explicitly maintain an internal state** that captures information from previous time steps. The main models we will see in the next sections are:
  - **Linear Dynamical Systems (LDS)** (page 212): Use linear transformations to update the hidden state over time.

- **Hidden Markov Models (HMMs)** (page 219): Use probabilistic transitions between hidden states to model sequences.
- **Recurrent Neural Networks (RNNs)** (page 232): Use non-linear functions to update the hidden state, allowing for complex temporal dependencies.

4.2 Memoryless Models

4.2.1 Autoregressive (AR) Models

The **Autoregressive (AR) Model** is one of the oldest approach to handle temporal sequences, especially used in **signal processing**, **finance**, and **time-series forecasting**. Its principle is simple: the current value of a sequence depends on a finite number of its **past values**. In other words, the model **uses the past to predict the future**, but *without* any hidden memory structure.

Definition 1: Autoregressive (AR) Model

For a scalar time-series x_t, an **Autoregressive (AR) Model**, or simply $AR(p)$ model of order p, is defined as:

$$\hat{x}_t = a_1 x_{t-1} + a_2 x_{t-2} + \ldots + a_p x_{t-p} + \varepsilon_t \tag{92}$$

Where:

- $a_1, a_2, \ldots, a_p$ are the model parameters (weights) that determine the influence of past values.
- p is the number of **delay taps** (how back in time we look).
- ε_t is a white noise error term, accounting for randomness or unexplained variations.

If we arrange data as vectors, we can express the AR model in vector form:

$$\mathbf{x}_t = [x_t, x_{t-1}, x_{t-2}, \ldots, x_{t-p+1}]^T$$

Thus, the AR model can be compactly written as:

$$\hat{x}_t = \mathbf{a}^T \mathbf{x}_{t-1} + \varepsilon_t \tag{93}$$

Where $\mathbf{a} = [a_1, a_2, \ldots, a_p]^T$. Or equivalently, for multivariate data:

$$\hat{\mathbf{x}}_t = \mathbf{A}_1 \mathbf{x}_{t-1} + \mathbf{A}_2 \mathbf{x}_{t-2} + \ldots + \mathbf{A}_p \mathbf{x}_{t-p} + \boldsymbol{\varepsilon}_t$$

Example 1: Autoregressive (AR) Model

Suppose we want to predict the next day's temperature:

$$T_{\text{today}} = 0.7 \cdot T_{\text{yesterday}} + 0.2 \cdot T_{2 \text{ days ago}} + 0.1 \cdot T_{3 \text{ days ago}}$$

This means that the most recent day (yesterday) has the highest influence on today's temperature, while the influence decreases for days further back in time. The model linearly mixes these past temperatures to make a prediction.

Properties and Limitations

- **Linear limitation**: The AR model assumes a linear relationship between past and current values, which **may not capture complex patterns in data** (e.g., if signal rising, slow down).
- **Finite memory (fixed window)**: It only considers a fixed number of past values (determined by p), **ignoring longer-term dependencies**.
- **Stationary assumption**: AR models typically assume that the underlying time series is stationary, meaning its statistical properties do not change over time. This can be a limitation when dealing with real-world data that may exhibit trends or seasonal patterns.
- **Deterministic, no hidden representation**: AR models do not maintain a hidden state or memory of past inputs beyond the fixed window, which can limit their ability to model complex temporal dependencies.

That's why we move toward **Feed-Forward extensions** and eventually **Recurrent Neural Networks**, which can introduce non-linearities and maintain hidden states for better temporal modeling.

4.2.2 Feed-Forward Extensions: TDNNs

Autoregressive models are **linear**:

$$\hat{x}_t = a_1 x_{t-1} + a_2 x_{t-2} + \ldots + a_p x_{t-p}$$

They work only if the relationship between past and future is *linear*. But real-world signals (speech, finance, human motion, etc.) are **nonlinear** and complex. The idea is to replace the simple linear combination with a **Feed-Forward Neural Network (FNN)** that can learn nonlinear dependencies.

We still use the **sliding window** of the last p time steps as input:

$$\mathbf{x}_t = [x_{t-1}, x_{t-2}, \ldots, x_{t-p}]^T$$

But instead of computing a weighted sum, we feed it into a neural network:

$$\hat{y}_t = f(\mathbf{x}_t; \mathbf{W}, \mathbf{b})$$

Where f is the neural network function parameterized by weights $\mathbf{W}$ and biases $\mathbf{b}$, and it can include **one or more hidden layers** with nonlinear activation functions (ReLU, tanh, etc.).

Definition 2: Feed-Forward Time-Delay Neural Network

A **Feed-Forward Time-Delay Neural Network (TDNN)** is a neural network architecture designed for **sequence modeling**, where the **input** consists of a **fixed-size window of past time steps**, and the **network learns to predict future values based on this context**. TDNNs can capture nonlinear relationships in temporal data by utilizing multiple layers and nonlinear activation functions.
Mathematically, a TDNN can be represented as:

$$\begin{aligned} h^{(1)} &= \sigma\left(\mathbf{W}^{(1)} \underbrace{[x_{t-1}, x_{t-2}, \ldots, x_{t-p}]}_{\mathbf{x}_t} + \mathbf{b}^{(1)}\right) \\ h^{(2)} &= \sigma\left(\mathbf{W}^{(2)} h^{(1)} + \mathbf{b}^{(2)}\right) \\ &\vdots \\ \hat{y}_t &= \mathbf{W}^{(L)} h^{(L-1)} + \mathbf{b}^{(L)} \end{aligned} \tag{94}$$

Where:

- σ is a **nonlinear activation function** (e.g., ReLU, tanh).
- $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the weights and biases for layer l.
- L is the total number of layers in the network.
- $h_t^{(l)}$ represents the hidden layer activations at layer l and time t.

Each time step produces its own prediction based on the **previous** p **samples**.

Example 2: TDNN for Time Series Prediction

Suppose we want to predict whether the **temperature** will rise or fall tomorrow. The relation may depend on nonlinear combinations, for example: "if temperature has been rising for 2 days **and** humidity is decreasing, then likely it will rise again". Such behavior cannot be captured by linear AR coefficients, but a neural network can **learn** this rule through nonlinear layers.

Properties and Limitations

- ✔ **Nonlinear**: TDNNs can model complex, nonlinear relationships in temporal data. This allows them to capture patterns that linear models cannot.
- ✘ **Still memoryless**: TDNNs do not have an internal state that persists across time steps. **Each prediction is made solely based on the fixed-size input window**, without retaining information from previous predictions. It is the same limitation as AR models.
- ✘ **Deterministic**: TDNNs produce a single output for a given input window, without modeling uncertainty or variability in predictions. In other words, no stochastic transition or hidden states. This can be a limitation in scenarios where uncertainty is important (e.g., weather forecasting).
- ✘ **Static structure**: The architecture of a TDNN is fixed once defined, meaning it cannot dynamically adjust its structure based on the input sequence length or complexity. Network must be retrained or resized for different sequence lengths.
- ✔ **Parallelizable**: Each window can be processed independently, allowing for efficient training and inference on modern hardware (GPUs).

So TDNNs are a step up from linear AR models, adding **nonlinearity** and **learned feature extraction**, but they **still lack the ability to maintain an internal state or model uncertainty over time**. For tasks requiring memory of past states or probabilistic predictions, more advanced architectures like Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTM) networks are needed.

Autoregressive vs TDNN

Both AR models and TDNNs are **memoryless** in the sense that they **do not maintain an internal state** across time steps. Therefore, **we manually provide past values as input**. Both models use past values, but in **different ways**:

- **AR Models** predict the next value using **past outputs**:

$$y_t = f\left(y_{t-1}, y_{t-2}, \ldots, y_{t-p}\right)$$

- **TDNNs** predict the next value using **past inputs**:

$$y_t = f\left(x_{t-1}, x_{t-2}, \ldots, x_{t-p}\right)$$

Autoregressive and time-delay neural networks **differ in the type of inputs they use** (past outputs vs past inputs), not in whether they are linear or non linear; both can be linear or nonlinear depending on the function f used.

Neither Autoregressive nor TDNN is better or worse; they are simply **different approaches to modeling temporal data**. AR models use past predictions as input, which can lead to error accumulation if the predictions are inaccurate. TDNNs use past observed values, which tend to be more stable, though they may require more complex architectures to capture long-term dependencies. Which approach is better depends on the specific problem, the characteristics of the data, and the available computational resources.

4.3 Models with Memory

4.3.1 Hidden State Dynamics and Outputs

So far, our models only saw **a sliding window** of past data. But what if the dependency extends over *hundreds* of time steps? Keeping all past inputs becomes impossible. Instead of feeding all the history explicitly, we can **summarize** it in a **hidden state vector** $\mathbf{h}_t$. That state:

- ✔ Evolves over time;
- ✔ Captures everything important about the past;
- ✔ Is sufficient to predict the present or future.

$$\text{Past Inputs } (x_0, x_1, \ldots, x_{t-1}) \xrightarrow{\text{compressed into}} \mathbf{h}_{t-1}$$

At each time step, the model receives a new input x_t and updates its hidden state $\mathbf{h}_t$ based on the previous state $\mathbf{h}_{t-1}$ and the new input x_t:

$$\mathbf{h}_t = F\left(\mathbf{h}_{t-1}, \mathbf{x}_t\right)$$

Finally, the model produces an output $\mathbf{y}_t$ based on the current hidden state $\mathbf{h}_t$:

$$\mathbf{y}_t = G\left(\mathbf{h}_t\right)$$

Since we are in a **dynamical system** setting, F and G are special functions:

- F is the **state transition function**, which defines how the hidden state evolves over time;
- G is the **output function**, which defines how the outputs are generated from the hidden states.
- $\mathbf{h}_t$ is the **hidden state** at time step t;
- $\mathbf{y}_t$ is the **output** at time step t.
- $\mathbf{x}_t$ is the **input** at time step t.

The functions F and G can be **deterministic or stochastic**, **linear or non-linear**.

> **Example 3: Linear Dynamical System (LDS)**
>
> If both state transition function F and output function G are linear, we have a **Linear Dynamical System (LDS)**:
>
> $$\begin{cases} \mathbf{h}_t = \mathbf{A}\mathbf{h}_{t-1} + \mathbf{B}\mathbf{x}_t + \omega_t \\ \mathbf{y}_t = \mathbf{C}\mathbf{h}_t + \nu_t \end{cases}$$
>
> Where:
>
> - $\mathbf{A}, \mathbf{B}, \mathbf{C}$ are system matrices defined by the model;
> - ω_t is the process noise (usually Gaussian);

- ν_t is the observation noise (usually Gaussian).

This system is widely used in control theory and signal processing:

- Evolves linearly in state space;
- Has memory through the matrix $\mathbf{A}$, which defines how past states influence the current state.
- And is **stochastic**, so uncertainty is tracked explicitly.

If the hidden state $\mathbf{h}_t$ can't be observed directly, we can estimate it using the **Kalman filter**. If this example is unfamiliar, don't worry; we will cover it in more detail later. It serves here to illustrate the concept of models with memory using hidden states.

Where is the Memory?

The memory in these models is encapsulated in the **hidden state vector** $\mathbf{h}_t$. This vector serves as a **summary** of all past inputs and states, allowing the model to retain information over time without needing to store the entire history explicitly. So it is a sort of *model's memory* of past events, which influences its current and future behavior. This state can be represented conditionally as (91, page 203):

$$P(\mathbf{h}_t \mid \mathbf{h}_{t-1}, \mathbf{h}_{t-2}, \ldots, \mathbf{h}_0)$$

But since $\mathbf{h}_{t-1} \sim P(\mathbf{h}_{t-1} \mid \mathbf{h}_{t-2}, \ldots, \mathbf{h}_0)$, we can simplify this to:

$$P(\mathbf{h}_t \mid \mathbf{h}_{t-1}) \approx P(\mathbf{h}_t \mid \mathbf{h}_{t-1}, \mathbf{h}_{t-2}, \ldots, \mathbf{h}_0)$$

This property is known as the **Markov Property** and is fundamental in the design of models with memory, because it allows us to focus on the most recent state $\mathbf{h}_{t-1}$ to predict the next state $\mathbf{h}_t$, rather than needing to consider the entire history of states. It lightens the computational load and simplifies the modeling process because we only need to keep track of the current state, not the entire sequence of past states.

Definition 3: Markov Property

A process satisfies the **Markov Property** if "the *future* depends **only** on the *present*, and **not** directly on the *past* before it". Formally:

$$P(h_t \mid h_{t-1}, h_{t-2}, \ldots, h_0) = P(h_t \mid h_{t-1}) \tag{95}$$

So, once we know the **current state** h_{t-1}, the **past states** $h_{t-2}, \ldots, h_0$ provide no additional information about the **future state** h_t. All relevant information from the past is **summarized** inside the current state h_{t-1}.

4.3.2 Linear Dynamical Systems and the Kalman Filter

A **Linear Dynamical System (LDS)** is a *state-space model* that represents how a system evolves over time using **linear equations**. It assumes that there's a hidden internal state $\mathbf{h}_t$ that evolves dynamically and generates the observed outputs $\mathbf{y}_t$. It's the mathematical backbone behind: signal tracking, control systems, robotics, and time-series forecasting (before RNNs took over). Formally, an LDS is defined by two main equations:

$$\text{State Equation:} \quad \mathbf{h}_t = \mathbf{A}\mathbf{h}_{t-1} + \mathbf{B}\mathbf{x}_t + \mathbf{w}_t \tag{96}$$

$$\text{Observation Equation:} \quad \mathbf{y}_t = \mathbf{C}\mathbf{h}_t + \mathbf{v}_t \tag{97}$$

- $\mathbf{h}_t$ is the **hidden** (latent) **state** at time t (a continuous-valued vector, linearly evolving over time).
- $\mathbf{x}_t$ is the **input** (control) at time t.
- $\mathbf{y}_t$ is the **observed output** at time t.
- $\mathbf{A}$ is the **state transition matrix** that defines how the hidden state evolves.
- $\mathbf{B}$ is the **control input matrix** that defines how inputs affect the hidden state.
- $\mathbf{C}$ is the **observation matrix** that maps the hidden state to the observed output.
- $\mathbf{w}_t$ and $\mathbf{v}_t$ are **process** and **observation noise**, typically assumed to be Gaussian with zero mean.

All matrices ($\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$) are constant over time, making the system linear and time-invariant. Furthermore, the **noise** terms $\mathbf{w}_t$ and $\mathbf{v}_t$ are usually modeled as **Gaussian** distributions.

⚠ State Estimation Problem

The *state estimation problem* poses a particular challenge in linear dynamical systems. Let's start with the basic situation. We have a **system that evolves in time**, like:

$$h_t = Ah_{t-1} + Bx_t + w_t$$

And we can **observe** something related to it:

$$y_t = Ch_t + v_t$$

But here's the catch: we **don't get to see the hidden state h_t directly**. Instead, we only observe y_t, a noisy version of h_t. This is because the hidden state h_t is not directly measurable due to factors such as sensor limitations, noise, or the system's inherent nature. In a tracking problem, for example, h_t might represent the true position and velocity of an object, while y_t is the noisy measurement from a radar or camera. Clearly, we don't have direct access to h_t; we only have the noisy observations y_t made at each time step. The question we want to answer is: *How can we estimate the hidden state h_t given*

the observations $y_1, y_2, \ldots, y_t$ *up to time* t*?* This is the **State Estimation Problem**.

Mathematically, the equation looks solvable, so *why* can't we just compute h_t directly? The **fundamental issue** is that we **don't know the initial state** h_0. Without knowing h_0, we can't accurately compute h_t for any $t > 0$. This uncertainty about the initial state propagates through time, making it impossible to determine the exact value of h_t just from the equations.

Why not just set $h_0 = 0$ or 1 and move on? After all, it's the start and there's no past. While it's true that we can choose an initial value for h_0, this choice is essentially a **guess**. The problem is that this **guess may be far from the true initial state**, leading to significant errors in estimating h_t as time progresses. The uncertainty in h_0 means that our estimates of h_t will always carry some level of uncertainty, which is why we need methods like the Kalman filter to help us estimate the hidden states more accurately over time. For example, in Figure 23, we see how a bad initial guess can lead to poor estimates if the initial uncertainty is low.

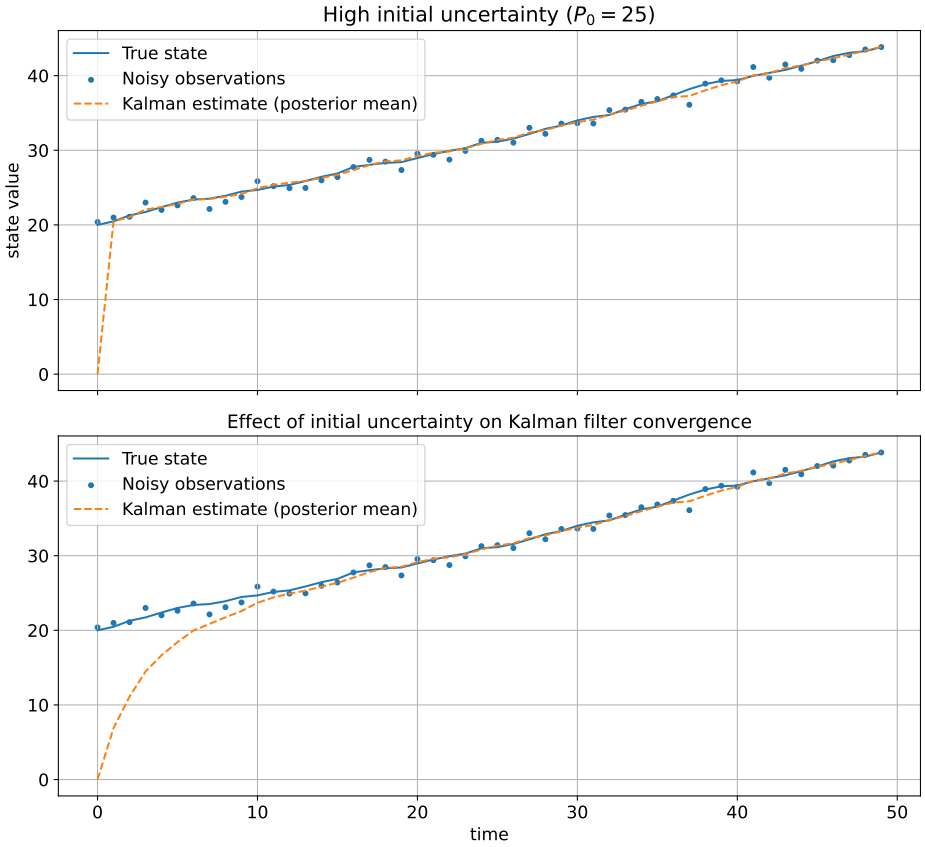


Figure 23: A comparison of Kalman filter performance with different initial uncertainty levels. The top plot shows the filter's ability to recover the true state when starting with high uncertainty about the initial state ($P_0 = 25$). The bottom plot illustrates how low initial uncertainty ($P_0 = 0.25$) can lead to poor estimates when the initial guess is incorrect, causing the filter to trust the wrong initial state too much.

Solution: The Kalman Filter

To tackle the state estimation problem in linear dynamical systems, we use the **Kalman Filter**. The Kalman filter is an **algorithm** that provides an **efficient way to estimate the hidden state $\mathbf{h}_t$ over time, even when we don't know the initial state**. The idea is simple. If we don't have the initial state, then we give our **initial guess a certain degree of uncertainty**, which can be high if we are very unsure about it. Then, as we receive new observations $\mathbf{y}_t$ (at every time step t), we **update our estimate** of the hidden state using a two-step process:

1. **Prediction Step**. Predict the new state and uncertainty based on the previous estimate:

$$\begin{cases} \hat{h}_{(t\,|\,t-1)} = A\,\hat{h}_{(t-1\,|\,t-1)} + B\,x_t \\ P_{(t\,|\,t-1)} = A\,P_{(t-1\,|\,t-1)}\,A^T + Q \end{cases} \tag{98}$$

   - $\hat{h}_{(t\,|\,t-1)}$ is the **predicted state estimate** at time t given observations up to time $t-1$.
   - $P_{(t\,|\,t-1)}$ is the **predicted estimate covariance** (uncertainty) at time t.
   - Q is the process **noise covariance matrix**.

   At time $t=0$, we set:

$$\begin{cases} \hat{h}_{(t\,|\,t-1)} = A\,\hat{h}_{(t-1\,|\,t-1)} + B\,x_t \\ P_{(t\,|\,t-1)} = A\,P_{(t-1\,|\,t-1)}\,A^T + Q \end{cases} \xrightarrow{t=0} \begin{cases} \hat{h}_{(0\,|\,-1)} = \mu_0 \\ P_{(0\,|\,-1)} = P_0 \end{cases}$$

   Where μ_0 is our **initial guess for the state**, and P_0 is the **initial uncertainty** (covariance) **associated with that guess**.

2. **Update Step**. When the new observation $\mathbf{y}_t$ arrives, we update our state estimate and uncertainty:

$$\begin{cases} K_t = P_{(t\,|\,t-1)}C^T\left(C\,P_{(t\,|\,t-1)}\,C^T + R\right)^{-1} \\ \hat{h}_{(t\,|\,t)} = \hat{h}_{(t\,|\,t-1)} + K_t\left(y_t - C\,\hat{h}_{(t\,|\,t-1)}\right) \\ P_{(t\,|\,t)} = (I - K_t\,C)\,P_{(t\,|\,t-1)} \end{cases} \tag{99}$$

   - K_t is the **Kalman gain**, which determines how much we trust the new observation versus our prediction.
   - R is the **observation noise covariance matrix**.
   - $\hat{h}_{(t\,|\,t)}$ is the **updated state estimate** at time t *after* incorporating the observation.
   - $P_{(t\,|\,t)}$ is the **updated estimate covariance** (uncertainty) at time t.

   At time $t=0$, we set:

$$\begin{cases} K_0 = P_{(0\,|\,-1)}C^T\left(C\,P_{(0\,|\,-1)}\,C^T + R\right)^{-1} \\ \hat{h}_{(0\,|\,0)} = \hat{h}_{(0\,|\,-1)} + K_0\left(y_0 - C\,\hat{h}_{(0\,|\,-1)}\right) \\ P_{(0\,|\,0)} = (I - K_0\,C)\,P_{(0\,|\,-1)} \end{cases}$$

⚠ Why can't we just set a big uncertainty and let the Kalman filter fix everything fast?

Setting a large initial uncertainty (P_0) allows the Kalman filter to be **more responsive to new measurements initially**, as it indicates that we are very unsure about our initial state estimate. However, this approach has its **drawbacks**:

- **Overfitting to Noise**: A very high initial uncertainty can lead the filter to **overfit to the noise in the early measurements**, resulting in erratic estimates that do not accurately reflect the true state.
- **Slower Convergence**: While a high uncertainty allows for rapid adjustments, it can also cause the filter to **take longer to converge** to a stable estimate, especially if the measurements are noisy. Due to the high uncertainty, the **filter may oscillate significantly** before settling down.
- **Instability**: In some cases, an excessively high initial uncertainty can lead to **numerical instability in the filter's calculations**, particularly in the computation of the Kalman gain.

Therefore, while a large initial uncertainty can be beneficial in certain scenarios, it is crucial to balance it with the characteristics of the system and the expected noise levels to ensure optimal performance of the Kalman filter. Let's see how different settings of process noise (Q) and observation noise (R) affect the Kalman filter's performance:

- **High Process Noise Q and Low Observation Noise R**: The filter tends to trust the measurements more, leading to rapid adjustments in the state estimate. This can be beneficial when the system is highly dynamic, but it may also result in **overfitting to noisy measurements**.

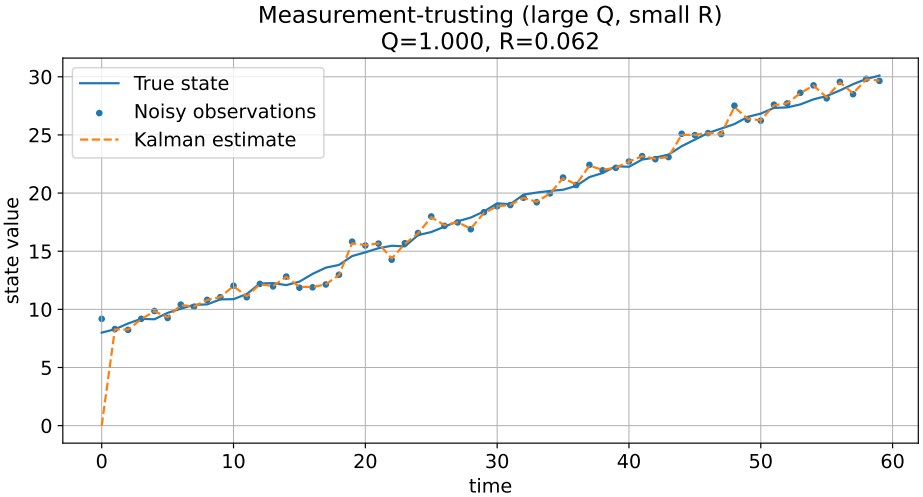


Figure 24: Kalman filter performance with high process noise (Q) and low observation noise (R). The filter quickly adapts to measurements but may overfit to noise.

✅ **Balanced Process and Observation Noise (Q and R)**: The filter strikes a balance between trusting the model predictions and the measurements. This setting is often ideal for many applications, as it allows the filter to adapt to changes while maintaining stability.

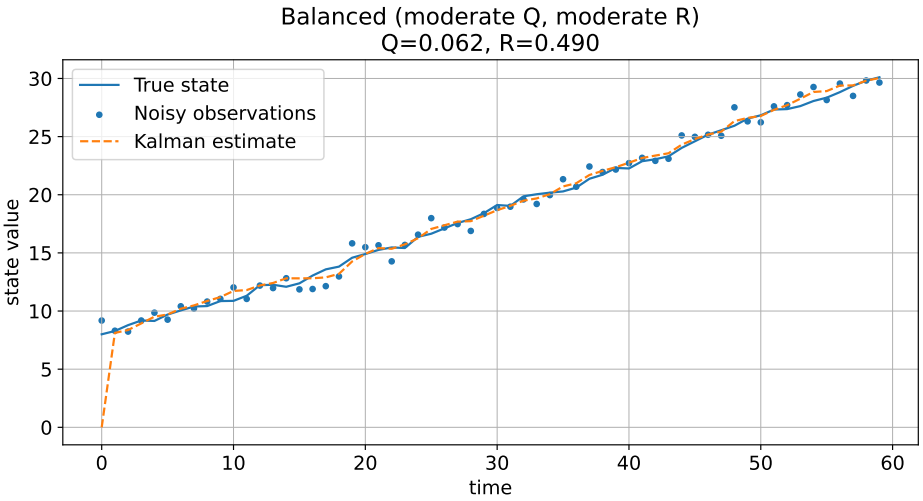


Figure 25: Kalman filter performance with balanced process and observation noise (Q and R). The filter effectively balances predictions and measurements.

⚖️ **Low Process Noise Q and High Observation Noise R**: The filter relies more on the model predictions, leading to smoother estimates. This is useful when the system is relatively stable, but it may result in slower adaptation to actual changes in the state.

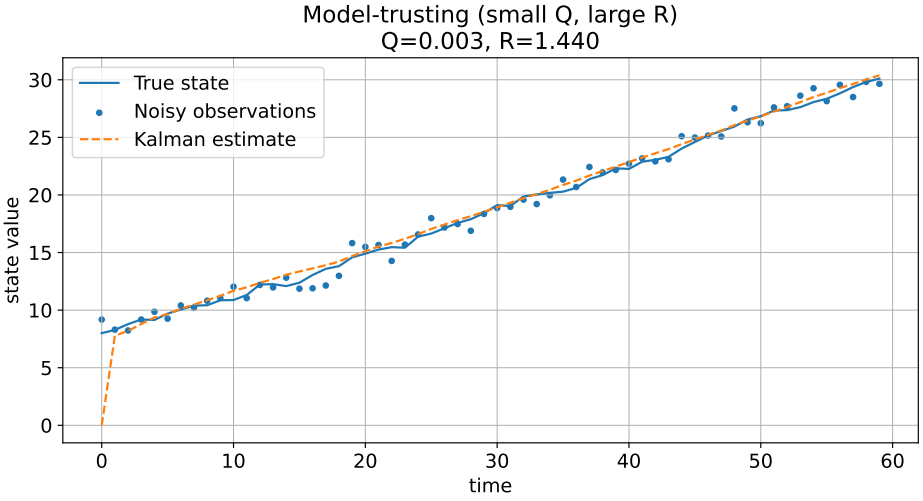


Figure 26: Kalman filter performance with low process noise (Q) and high observation noise (R). The filter produces smoother estimates but may adapt slowly to changes.

- **Kalman Gain K_t Behavior**: The Kalman gain adjusts dynamically based on the noise characteristics. A higher gain indicates more trust in the measurements, while a lower gain indicates more trust in the model predictions.

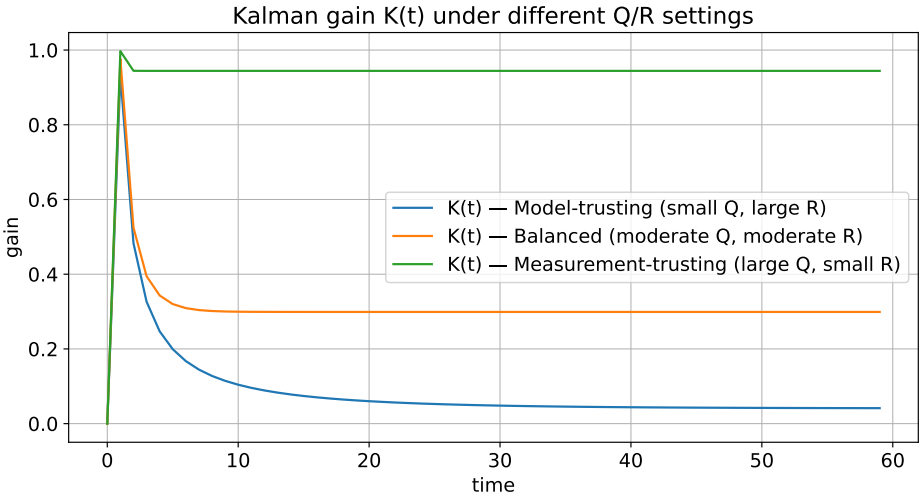


Figure 27: Kalman gain (K_t) behavior under different noise covariance settings. The gain reflects the filter's trust in measurements versus predictions.

4.3.3 Hidden Markov Models (HMMs)

In a **Linear Dynamical System (LDS)** like the Kalman filter, the hidden state h_t was **continuous** (e.g., a vector of real numbers that evolves over time). But in many real problems, the system evolves through **discrete modes or categories**, not continuous quantities. For example:

- A speaker pronouncing *phonemes* (basic sound units) in speech recognition.
- A user switching *behaviors* (e.g., browsing, purchasing) in activity recognition.
- Weather being *sunny*, *rainy*, or *cloudy* in weather modeling (not continuous temperature values).

So instead of continuous state vectors $h_t \in \mathbb{R}^n$, we have **discrete hidden states** $h_t \in \{1, 2, \ldots, K\}$ where K is the number of possible hidden states. This leads us to **Hidden Markov Models (HMMs)**, which are probabilistic models for sequences with discrete hidden states.

Definition 4: Markov Chain

A **Markov Chain** is a **mathematical model** that *implements* the Markov property (page 211). A Markov chain consists of:

- A **finite or countable set of states**.
- A **transition matrix**:

$$A = [a_{ij}], \quad a_{ij} = P(s_t = j \mid s_{t-1} = i)$$

- An **initial state distribution**:

$$\pi_i = [\pi_i], \quad \pi_i = P(s_0 = i)$$

A Markov chain **uses the Markov property** to describe how the state changes over time.

Example 4: Weather Model as a Markov Chain

Consider a simple weather model with three states: *Sunny*, *Rainy*, and *Cloudy*. The transition matrix A could be:

$$A = \begin{bmatrix} 0.8 & 0.1 & 0.1 \\ 0.4 & 0.5 & 0.1 \\ 0.3 & 0.3 & 0.4 \end{bmatrix}$$

This means that if today is Sunny, there is an 80% chance that tomorrow will also be Sunny, a 10% chance of Rainy, and a 10% chance of Cloudy.

The initial state distribution π could be:

$$\pi = \begin{bmatrix} 0.5 \\ 0.3 \\ 0.2 \end{bmatrix}$$

Indicating a 50% chance of starting in Sunny, 30% in Rainy, and 20% in Cloudy. This Markov chain can be used to model and predict weather patterns over time.

Definition 5: Hidden Markov Model (HMM)

A **Hidden Markov Model (HMM)** is a **probabilistic dynamical system** where:

- The system has a sequence of **hidden states**:

  $$s_1, s_2, \ldots, s_T, \quad s_t \in \{1, 2, \ldots, K\}$$

  Which evolve over time according to a **Markov chain**.
- At each time step, the system produces an **observation** y_t whose distribution depends **only on the current hidden state**.

An HMM is fully defined by the triplet:

$$\lambda = (\pi, A, B) \tag{100}$$

Where:

- **Initial state distribution** π:

  $$\pi_i = P(s_1 = i) \tag{101}$$

- **State transition probabilities** A:

  $$A = [a_{ij}] \qquad a_{ij} = P(s_t = j \mid s_{t-1} = i) \tag{102}$$

- **Emission probabilities** B:

  $$B = [b_j(k)] \qquad b_j(k) = P(y_t = k \mid s_t = j) \tag{103}$$

So the Hidden Markov Model defines a *probabilistic process* (stochastic process) with **two parallel processes** happening simultaneously:

1. **Hidden state sequence**. This is a sequence of **hidden states**:

   $$S = (s_1, s_2, \ldots, s_T) \quad s_t \in \{1, 2, \ldots, K\}$$

   These states represent what is *really happening* in the system, but we **cannot observe them directly** (state estimation problem, page 212). Instead, they are **latent variables** that we need to infer from the obser-

vations. This layer evolves according to the **Markov Chain**:

$$P(s_t \mid s_{t-1}) = a_{s_{t-1},s_t}$$

- **Hidden** because we do not observe these states directly.
- **Markov** because the next state depends only on the current state (Markov property).
- **Chain** because it is a sequence of states evolving over time.

⚠ Markov Assumption on the Hidden States: First-Order Markov Property

The Markov Chain assumes the **first-order Markov property** for the hidden states:

✅ Markov Assumption $\Rightarrow P(s_t \mid s_1, s_2, \ldots, s_{t-1}) = P(s_t \mid s_{t-1})$

This means that the **next hidden state** depends **only on the current one**, not on the entire history of past states. This assumption simplifies the model and makes computations tractable, because the entire history before s_{t-1} is irrelevant once current state s_{t-1} is known. Without this assumption, the model would become significantly more complex and harder to work with (**exponentially many parameters**) because we would need to consider all previous states to predict the next one:

⚠ Without Markov Assumption $\Rightarrow P(s_t \mid s_1, s_2, \ldots, s_{t-1})$

2. **Observation sequence**. This is a sequence of **observed data**:

$$Y = (y_1, y_2, \ldots, y_t) \quad y_T \in \mathcal{O}$$

These are the **actual measurements**, the data we *can* see. Each observation is generated **from the current hidden state only**:

$$P(y_t \mid s_t) = b_{s_t}(y_t)$$

This is called the **emission probability**, or **emission model**, because the hidden state *emits* an observation according to this distribution.

- **Observation** because these are the data we actually see.
- **Generated** because they are produced by the hidden states.
- **Not Markov** because observations do not have the Markov property; they depend on the hidden states.

⚠ Emission Independence Assumption

This is the second crucial assumption of Hidden Markov Models. It states that the **observation at time t depends only on the current hidden state s_t**:

✅ Emission Independence $\Rightarrow P(y_t \mid s_t, s_{t-1}, \ldots, y_{t-1}) = P(y_t \mid s_t)$

This means that once we know the hidden state s_t, the observation y_t is conditionally independent of all previous observations and states. In simple terms, the **observation** at time t (what we really see) depends **only on the hidden state** at time t (what is really happening), and **not on previous states or previous observations**. For example, if the hidden state is "Rainy", the probability of seeing "Umbrella" at time t does not depend on yesterday's weather, yesterday's observation, or any earlier time steps (last week, last month, etc.). The hidden state already contains all needed information. Without this assumption, the model would become significantly more complex and harder to work with, as we would need to consider dependencies on all previous states and observations:

Without Emission Independence $\Rightarrow P(y_t \mid s_t, s_{t-1}, \ldots, y_{t-1})$

Instead, the emission independence assumption allows us to **simplify the model and focus on the relationship between the current hidden state and the current observation**:

$$P(y_t \mid s_t) = b_{s_t}(y_t)$$

Why do we need both assumptions for an HMM to work?

Because without these assumptions, the probability of a sequence would involve **all states and all observations interacting** with each other directly. That would make modeling impossible, inference impossible and learning impossible (because the number of parameters would explode combinatorially). Because **hiddent transitions** and **observations** are two independent kinds of dependencies, we can **factor the joint probability** of the entire sequence into manageable parts. Given the two assumptions, the full joint probability of the hidden states and observations can be expressed as:

$$P(S, Y) = P(s_1) \cdot \prod_{t=2}^{T} P(s_t \mid s_{t-1}) \cdot \prod_{t=1}^{T} P(y_t \mid s_t) \tag{104}$$

- The **first term** $P(s_1)$ represents the probability of starting in the initial hidden state.
- The **first product** multiplies all the state transition probabilities together, capturing how the hidden states evolve over time according to the Markov chain.
- The **second product** multiplies all the observation emission probabilities together, capturing how each observation is generated from the corresponding hidden state.

Obviously, this factorization would be **impossible** without the two assumptions (Emission Independence and Markov Property), because we would have to consider all possible dependencies between states and observations across time steps.

Example 5: HMM Analogy – Weather and Activities

Imagine we are trying to infer the weather (hidden states) based on someone's activities (observations). The hidden states could be:

- Sunny
- Rainy
- Cloudy

The observations could be:

- Going for a walk
- Carrying an umbrella
- Staying indoors

The HMM would model how the weather changes over time (Markov chain) and how each type of weather influences the likelihood of different activities (emission probabilities). By observing the activities, we can infer the most likely sequence of weather conditions. For example, if we see someone carrying an umbrella, we might infer that it is likely Rainy. On the other hand, if we see them going for a walk, it might indicate Sunny weather.

The model is **fully specified** by the triplet $\lambda = (\pi, A, B)$:

- **Initial State Distribution** π (where we start). It tells us the **probability that the process starts in each hidden state**:

$$\pi_i = P(s_1 = i)$$

For example, in a weather model, π could represent the probabilities of starting the day as Sunny, Rainy, or Cloudy:

$$\pi = [0.6, 0.3, 0.1]$$

Where there is a 60% chance of starting Sunny, 30% Rainy, and 10% Cloudy. If we don't know the initial state, we can assume a uniform distribution:

$$\pi_i = \frac{1}{K} \quad \forall i$$

But in real tasks, π matters a lot (especially for short sequences).

- **State Transition Matrix** A (how the hidden states evolve over time). It is a $N \times N$ matrix where each entry a_{ij} represents the **probability of transitioning from hidden state** i **to hidden state** j:

$$A = [a_{ij}] \qquad a_{ij} = P(s_t = j \mid s_{t-1} = i)$$

This is the **Markov Chain** component of the HMM, defining how likely it is to move between different hidden states at each time step.

For example, in a weather model, A could represent the probabilities of transitioning between Sunny, Rainy, and Cloudy states:

$$A = \begin{bmatrix} 0.7 & 0.2 & 0.1 \\ 0.4 & 0.5 & 0.1 \\ 0.3 & 0.3 & 0.4 \end{bmatrix}$$

Where $a_{12} = 0.2$ means there is a 20% chance of transitioning from Sunny to Rainy.

- **Emission Probability Matrix** B (given a hidden state, how likely are the observations). It describes the **probability of each observation given a hidden state**. It is a sort of "lookup table" (or mapping) that tells us how likely each observation is for each hidden state:

  $$B = [b_j(k)] \qquad b_j(k) = P\left(y_t = k \mid s_t = j\right)$$

  It encodes the **emission model**: hidden states **emit** observations according to probability distributions.

  For example, in a weather model, B could represent the probabilities of different activities given the weather states:

  $$B = \begin{bmatrix} 0.8 & 0.1 & 0.1 \\ 0.2 & 0.7 & 0.1 \\ 0.5 & 0.2 & 0.3 \end{bmatrix}$$

  Where the rows correspond to hidden states (Sunny, Rainy, Cloudy) and the columns correspond to observations (Walk, Umbrella, Indoors). For instance, $b_1(1) = 0.8$ means that if the weather is Sunny, there is an 80% chance of going for a walk.

So the HMM triplet $\lambda = (\pi, A, B)$ completely defines the model, allowing us to define where the hidden state **starts** (π), how it **evolves** over time (A), and how it **produces observations** (B).

⚠ The Three Fundamental Problems of HMMs

Hidden Markov Models are powerful, but have one fundamental issue: **we don't see the hidden states, but everything depends on them**. When we have an HMM, we **don't directly observe the states**, but we only see:

$$Y = (y_1, y_2, \ldots, y_T)$$

These are the **observations** generated by the hidden states. But what we really want to know is the **hidden state sequence**:

$$S = (s_1, s_2, \ldots, s_T)$$

Because the hidden states represent the underlying process we are interested in (e.g., the actual weather conditions, the speaker's phonemes, etc.). So we have three fundamental problems to solve in order to use HMMs effectively:

1. **Evaluation (Likelihood) Problem**. "*How likely is this observed sequence under our model?*". Given an HMM $\lambda = (\pi, A, B)$ and an observation sequence $Y = (y_1, y_2, \ldots, y_T)$, compute the probability of the observation sequence:

$$P(Y \mid \lambda)$$

Why is this hard? Because:

$$P(Y \mid \lambda) = \sum_S P(S, Y \mid \lambda)$$

We need to sum over **all possible hidden state sequences** S, and there are **exponentially many** of them:

$$K^T \quad \text{(if there are } K \text{ hidden states and } T \text{ time steps)}$$

Calculating this directly is computationally infeasible for large T (e.g., $T = 100$ and $K = 10$ would require summing over 10^{100} sequences).

Solution: Forward Algorithm. The **Forward Algorithm** is not covered here, but it efficiently computes this probability using dynamic programming. A general idea is to recursively compute the probabilities of being in each hidden state at each time step, given the observations up to that point.

2. **Decoding (Inference) Problem**. "*What is the most likely hidden state sequence?*". Given an HMM $\lambda = (\pi, A, B)$ and an observation sequence $Y = (y_1, y_2, \ldots, y_T)$, find the most likely sequence of hidden states $S = (s_1, s_2, \ldots, s_T)$ that could have generated the observations:

$$\hat{S} = \arg\max_S P(S \mid Y, \lambda)$$

Why is this hard? Similar to the evaluation problem, we would need to consider all possible hidden state sequences to find the one that maximizes the probability. This is also computationally infeasible due to the exponential number of sequences.

Solution: Viterbi Algorithm. The **Viterbi Algorithm** is explained in detail later.

3. **Learning (Training) Problem**. "*What parameters* $\lambda = (\pi, A, B)$ *best explain our observed data?*".

Given an observation sequence $Y = (y_1, y_2, \ldots, y_T)$, estimate the HMM parameters $\lambda = (\pi, A, B)$ that best explain the observed data. This involves finding the model parameters that maximize the likelihood of the observations:

$$\lambda^* = \arg\max_\lambda P(Y \mid \lambda)$$

Why is this hard? Because we do not observe the hidden states, we cannot directly count how many times each transition or emission occurs. We need to infer the hidden states while also estimating the parameters, which creates a circular dependency. This is a classic case of **unsupervised learning** where we have to learn from data without direct labels (the hidden states are like labels that we don't have).

Solution: Baum-Welch Algorithm. The **Baum-Welch Algorithm** (a special case of the Expectation-Maximization, EM, algorithm) is used to iteratively estimate the HMM parameters by maximizing the likelihood of the observed data. It is not covered here, but it involves computing expected counts of transitions and emissions based on the current model, then updating the parameters accordingly.

All 3 problems come from the same root: hidden variables make everything combinatorially complex.

The Viterbi Algorithm for the Decoding Problem

The Viterbi algorithm was introduced to address the decoding problem of HMMs. Here, we only explain this algorithm in detail because it is used to demonstrate the limitations of the HMM model and the need for more powerful models, such as RNNs.

First of all, we need to understand why the decoding problem is hard. In an Hidden Markov Model, the **hidden states** $s_1, s_2, \ldots, s_T$ are not directly observable. Instead, we only see the **observations** $y_1, y_2, \ldots, y_T$ that are generated from these hidden states. But often, what we really want is **the hidden states** themselves, because they represent the underlying process we are interested in (e.g., the actual weather conditions, the speaker's phonemes, etc.). So the **Decoding Problem** is about **finding the most likely sequence of hidden states given the observed data**:

$$\hat{S} = \arg\max_S P\left(S \mid Y, \lambda\right)$$

Where:

- $S = (s_1, s_2, \ldots, s_T)$ is a candidate sequence of hidden states.
- $Y = (y_1, y_2, \ldots, y_T)$ is the observed sequence.
- $\lambda = (\pi, A, B)$ are the HMM parameters.

Because there are **exponentially many** possible hidden state sequences (if there are K hidden states and T time steps, there are K^T possible sequences), we need an efficient algorithm to find the most likely one without enumerating all possibilities. The **Viterbi Algorithm** is the **efficient dynamic programming algorithm** that solves this problem in **polynomial time** ($O(T \cdot K^2)$) instead of exponential time ($O(K^T)$).

Intuition Behind the Viterbi Algorithm. Viterbi uses the fact that HMM probabilities factor nicely:

$$P\left(S, Y\right) = P\left(s_1\right) \cdot \prod_{t=2}^{T} P\left(s_t \mid s_{t-1}\right) \cdot \prod_{t=1}^{T} P\left(y_t \mid s_t\right)$$

This means that we do not explore all paths, but rather we only keep the **best partial path** to each state at each time step. At time t, for each possible state, keep the probability of the best path (highest probability) that ends in that state. So, at each time step, we need two main components:

- **The "best score" matrix** δ of size $K \times T$, where $\delta_j(t)$ represents the probability of the most likely path that ends in state j at time t:

$$\delta_j(t) = \max_{s_1, s_2, \ldots, s_{t-1}} P\left(s_1, s_2, \ldots, s_{t-1}, s_t = j, y_1, y_2, \ldots, y_t \mid \lambda\right) \tag{105}$$

- **The "argmax pointer" matrix** ψ of size $K \times T$, where $\psi_j(t)$ stores the index of the state at time $t-1$ that led to the best path to state j at time t:

$$\psi_j(t) = \arg\max_i \left[\delta_i(t-1) \cdot a_{ij}\right] \tag{106}$$

  Later, we will use this matrix to backtrack and reconstruct the most likely sequence of hidden states.

The **Viterbi algorithm** proceeds in four main steps:

1. **Initialization** $t = 1$. For each state j (where a hidden state can be):

$$\delta_j(1) = \pi_j \cdot b_j(y_1)$$

   Where π_j is the initial probability of starting in state j, and $b_j(y_1)$ is the emission probability of observing y_1 from state j. Also, initialize the pointer:

$$\psi_j(1) = 0$$

   That is, there is no previous state at time 1.

2. **Recursion for** $t = 2 \ldots T$. We compute matrices δ and ψ for each state j:

$$\delta_j(t) = \max_i \left[\delta_{t-1}(i) \cdot a_{ij}\right] \cdot b_j(y_t)$$

   This finds the best previous state i that leads to state j at time t, multiplies by the transition probability a_{ij} and the emission probability $b_j(y_t)$. We also update the pointer:

$$\psi_j(t) = \arg\max_i \left[\delta_{t-1}(i) \cdot a_{ij}\right]$$

   This stores the index of the best previous state.

3. **Termination**. After filling in the matrices, we find the probability of the best full path:

$$P^* = \max_j \delta_T(j)$$

   And the final state of the best path:

$$s_T^* = \arg\max_j \delta_T(j)$$

4. **Backtracking**. Finally, we reconstruct the most likely sequence of hidden states by backtracking through the pointer matrix ψ:

$$s_t^* = \psi_{s_{t+1}^*}(t+1) \quad \text{for } t = T-1, T-2, \ldots, 1$$

   At time $t+1$, we look at the best state s_{t+1}^* and use ψ to find which state at time t led to it. We repeat this process until we reach the beginning of the sequence, reconstructing the entire most likely hidden state sequence $S^* = (s_1^*, s_2^*, \ldots, s_T^*)$.

```
Input:
    Observation sequence Y = (y_1, y_2, ..., y_T)
    HMM parameters λ = (π, A, B)
    Number of hidden states K

Output:
    Most likely hidden state sequence Ŝ = (s_1^*, s_2^*, ..., s_T^*)

// Initialization
// For each state j, compute the initial Viterbi score
// (i.e., the probability of starting in state j and emitting
// y_1) and set the pointer to 0
for j = 1 to K:
    δ_j(1) ← π_j · b_j(y_1)
    ψ_j(1) ← 0

// Recursion
// For each time step from 2 to T,
// compute the Viterbi scores and pointers for each state j
// by considering all possible previous states i
// and taking the maximum over them
for t = 2 to T:
    for j = 1 to K:
        δ_j(t) ← max_{i∈{1,...,K}} [δ_i(t − 1) · a_ij] · b_j(y_t)
        ψ_j(t) ← arg max_{i∈{1,...,K}} [δ_i(t − 1) · a_ij]

// Termination
// Find the best score and the final state of the best path
P^* ← max_{j∈{1,...,K}} δ_j(T)
s_T^* ← arg max_{j∈{1,...,K}} δ_j(T)

// Backtracking
// Reconstruct the most likely hidden state sequence
// by backtracking through the pointers
for t = T − 1 down to 1:
    s_t^* ← ψ_{s_{t+1}^*}(t + 1)

Return Ŝ = (s_1^*, s_2^*, ..., s_T^*)
```

Listing 10: Viterbi Algorithm for HMM Decoding

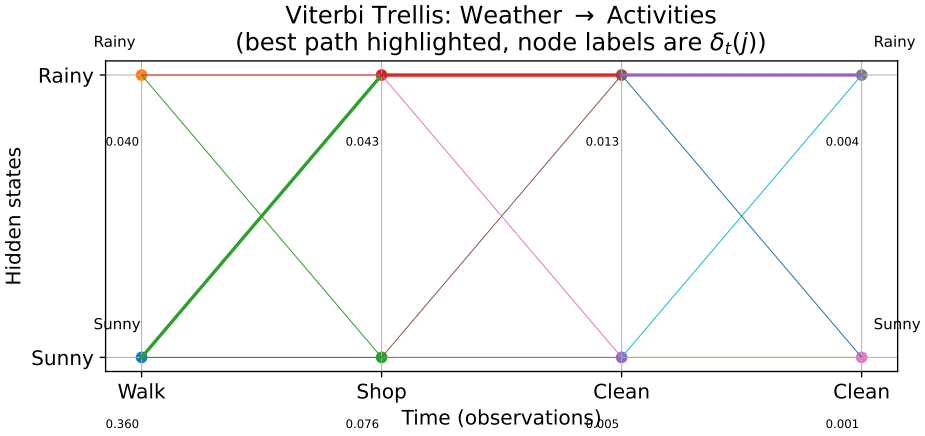


Figure 28: **Viterbi trellis diagram for a simple Weather to Activities HMM.** Each column is a time step (Walk, Shop, Clean, Clean), each row a hidden state (Sunny, Rainy). The node values ($\delta_t(j)$) are the Viterbi scores: the probability of the *best* path that ends in state j at time t. Thin edges show all possible transitions; the thick path highlights the most probable hidden sequence found by Viterbi: Sunny → Rainy → Rainy → Rainy.

4.3.4 Comparison to Deterministic Recurrent Systems

In this section, we will attempt to answer the following question: *How do classical dynamical models (e.g., Kalman filters and Hidden Markov Models) compare to deterministic recurrent systems (e.g., Recurrent Neural Networks)?* The answer will prepare us for the introduction of **neural-based recurrent architectures** in the next section.

First of all, we highlight the difference between **probabilistic** and **deterministic** models.

- **Probabilistic Dynamical Systems** (e.g., HMMs, Kalman filters) model the evolution of a system over time by **incorporating uncertainty and randomness**. They **use probability distributions** to represent the state of the system and the observations, allowing them to handle noise and uncertainty in the data effectively.

  For example, in HMMs, the system transitions between hidden states based on probabilistic rules. Then, observations are generated from these states according to specific probability distributions.

- **Deterministic Recurrent Systems** (e.g., RNNs) model the evolution of a system over time by **using fixed rules or functions** to determine the next state based on the current state and input. They do not inherently account for uncertainty or randomness in their predictions. Instead, they rely on learned weights and activation functions to capture temporal dependencies in sequential data.

Let's make a more detailed comparison between these two types of models:

- **Probabilistic Dynamical Systems** are well-suited for tasks where uncertainty and noise are prevalent, such as speech recognition, natural language processing, and time series forecasting. They can effectively model the underlying stochastic processes that generate the observed data.

  ✔ **Key Properties**

  - State transitions are **probabilistic**, allowing the model to capture uncertainty in the system's evolution.
  - Observations are **probabilistic**, enabling the model to handle noisy data effectively.
  - They explicitly represent **uncertainty** in both state transitions and observations.
  - Inference is based on **probability distributions**, allowing for robust predictions in the presence of noise.
  - Decisions are based on **likelihoods** and **argmax** (e.g., Viterbi algorithm for HMMs) to find the most probable sequence of states.

  In practice, they are often used in scenarios where the data is noisy or incomplete, and where modeling uncertainty is crucial for accurate predictions.

- **Deterministic Recurrent Systems** excel in tasks that require learning complex temporal patterns and dependencies from large datasets, such as language modeling, machine translation, and video analysis. They can capture intricate relationships in sequential data through their learned representations. They are the ancestor of modern RNN architectures like LSTMs and GRUs. A deterministic recurrent system can be described by the following equations:

  $$h_t = f\left(h_{t-1},\, x_t\right)$$

  Where f is a **deterministic function** (no randomness).

  ✔ **Key Properties**

  - **No noise** terms.
  - **No probability** distributions.
  - **Hidden state** evolves **deterministically**.
  - **Same input sequence**, then same hidden state trajectory.
  - **Learning** (if any) is achieved via optimization of a loss function (e.g., MSE, cross-entropy) **using gradient-based methods**, not via probabilistic inference.

  This makes them computationally efficient and easier to implement, especially for large-scale problems, but less expressive for handling uncertainty; less suited for tasks where observations are noisy; harder to reason about probabilistically. In practice, they are often used in scenarios where large amounts of sequential data are available, and the focus is on learning complex temporal patterns rather than modeling uncertainty.

The main reason to transition from probabilistic dynamical systems to deterministic recurrent systems is that the latter can **learn complex temporal patterns and dependencies from large datasets**. This makes them more suitable for tasks such as language modeling, machine translation, and video analysis. RNNs can capture intricate relationships in sequential data through their learned representations, which is often more challenging for probabilistic dynamical systems that rely on predefined probabilistic rules and distributions. Additionally, RNNs can be trained end-to-end using gradient-based optimization techniques, allowing them to adapt to the specific characteristics of their training data. This flexibility and adaptability make RNNs a powerful choice for many modern deep learning applications.

4.4 Definition

4.4.1 What is a RNN?

Feed-Forward Networks (FFNs) cannot handle sequences because:

- They take only **fixed-size** input vectors.
- They have **no internal state**, so the **forget everything** once the forward pass is done.

To handle temporal data, we need **a model that keeps track of past information while processing the present**. This requires **memory**. RNNs introduce memory via **recurrent connections**, which allow information to persist across time steps.

Definition 6: Recurrent Neural Network (RNN)

A **Recurrent Neural Network (RNN)** is a neural architecture designed to process **sequences** by maintaining a **memory** of past inputs through **recurrent connections**. Formally:

$$h_t = f\left(W_x \cdot x_t + W_h \cdot h_{t-1} + b\right) \tag{107}$$

Where:

- x_t is the input at time step t.
- h_t is the hidden state (memory) at time step t (**memory** of the RNN).
- W_x and W_h are weight matrices for input-to-hidden and hidden-to-hidden connections, respectively.
- b is a bias vector.
- f is a non-linear activation function (e.g., tanh, ReLU).

The RNN produces an output y_t at each time step, which can be computed as:

$$y_t = g\left(W_y \cdot h_t\right) \tag{108}$$

Where:

- W_y is the weight matrix for hidden-to-output connections.
- g is an activation function for the output layer (e.g., softmax for classification).

What is a recurrent connection?

In a normal neural network layer, the output is computed as:

$$h = \sigma\left(Wx + b\right)$$

Where h is the output, x is the input, W is the weight matrix, b is the bias, and σ is an activation function. In a layer with **recurrent connections**, the output at time step t also depends on the output from the previous time step $t-1$:

$$h_t = \sigma\left(W_x \cdot x_t + W_h \cdot h_{t-1} + b\right)$$

So, we have a new term $W_h \cdot h_{t-1}$ that incorporates information from the previous time step, allowing the network to maintain a form of memory over time. It is called **recurrent connection** because the output at one time step is fed back into the network as input for the next time step, creating a loop in the network architecture (a link from the **previous hidden state** back into the computation of the **current** hidden state). This single recursive connection is what gives the RNN its "memory".

What is the hidden state?

The **hidden state** h_t in an RNN is a **continuous vector** that serves as the network's **memory** at time step t. It is updated at **every time step** based on the current input x_t and the previous hidden state h_{t-1}:

$$h_t = f\left(h_{t-1}, x_t\right) = \sigma\left(W_x \cdot x_t + W_h \cdot h_{t-1} + b\right)$$

This recovers the structure of **dynamical systems** (e.g., Kalman filters, HMMs) we saw before, but now the update function is **learned**, not fixed. It is more flexible and powerful, allowing the RNN to **capture complex temporal dependencies in the data** (nonlinear relationships, long-term dependencies, etc.).

Why this gives the RNN memory

Because every hidden state h_t contains:

- Information from h_{t-1} (the previous hidden state).
- Which in turn contains information from h_{t-2}.
- And so on, back to h_0.

So the RNN hidden state implicitly **stores a compressed summary of *all previous inputs***:

$$h_t = f\left(h_{t-1}, x_t\right) = f\left(f\left(h_{t-2}, x_{t-1}\right), x_t\right) = f\left(f\left(f\left(h_{t-3}, x_{t-2}\right), x_{t-1}\right), x_t\right) = \ldots$$

This recursive structure is the "memory mechanism" of RNNs, allowing them to **retain information over time** and make predictions based on the entire sequence history.

Hidden State as Distributed Memory

As we have seen, at each time step t, the RNN updates:

$$h_t = f\left(W_x \cdot x_t + W_h \cdot h_{t-1} + b\right)$$

- h_{t-1} contains **what the network remembers about the past**.
- x_t injects **new information from the present**.
- W_h decides **how strongly past memory is kept**.
- W_x decides **how strongly new input is incorporated**.

So the RNN is **carrying forward a summary of all previous inputs**. This summary is the hidden state vector h_t.

What does "distributed memory" mean? The memory is not stored in one neuron. Instead, memory is stored across many components of the hidden vector. Each component of h_t:

- ✗ Does not represent a human-interpretable concept.
- ✗ Does not store one specific piece of information.
- ✗ Is not binary (not like Hidden Markov Model discrete states).

Instead, **memory is encoded across *all dimensions* of the hidden state vector**. This is the same idea as "distributed representations" in deep learning: a concept is not stored in a single neuron, but rather it's stored in the *pattern* of activations and memory is encoded in many dimensions simultaneously.

The RNN hidden state is thus a distributed memory because its information is encoded across all its dimensions, and **each neuron extracts different aspects** of this memory through different learned weight projections.

> **Example 6: Hidden State Analogy**
>
> Consider each neuron in the hidden state vector h_t as a **musician** in an orchestra.
>
> Each musician (**neuron**) has different instruments (**weights**), different skills (**non-linearities**), different interpretations (**activations**), and different roles in the ensemble (**dimensions of the hidden state**).
>
> So although they all read the same sheet music (the **input sequence**), each musician contributes a unique sound (**feature**) to the overall performance (**hidden state**):
>
> - The violinist (neuron 1) might focus on melody (*temporal patterns*).
> - The percussionist (neuron 2) might emphasize rhythm (*short-term dependencies*).
> - The bassist (neuron 3) might provide harmonic support (*long-term*

context).

- And so on for all musicians (neurons) in the orchestra (hidden state).

So, same input (sheet music), different roles (neurons), different extracted information (features from memory).

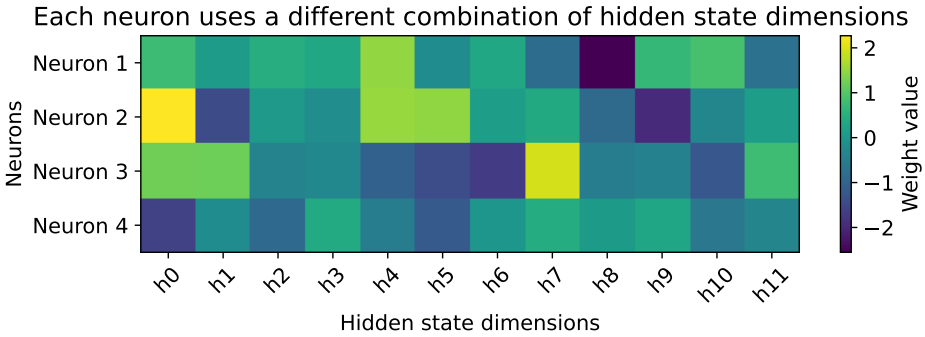


Figure 29: Heatmap with 4 different neurons (rows), each extracting different information from the same hidden state vector (columns). The colors indicate the weight magnitudes, showing how each neuron focuses on different parts of the hidden state to extract unique features. Greater weight magnitudes (lighter colors) indicate stronger influence from those hidden state dimensions. This illustrates the concept of **distributed memory** in RNNs, where each neuron captures different aspects of the overall memory encoded in the hidden state.

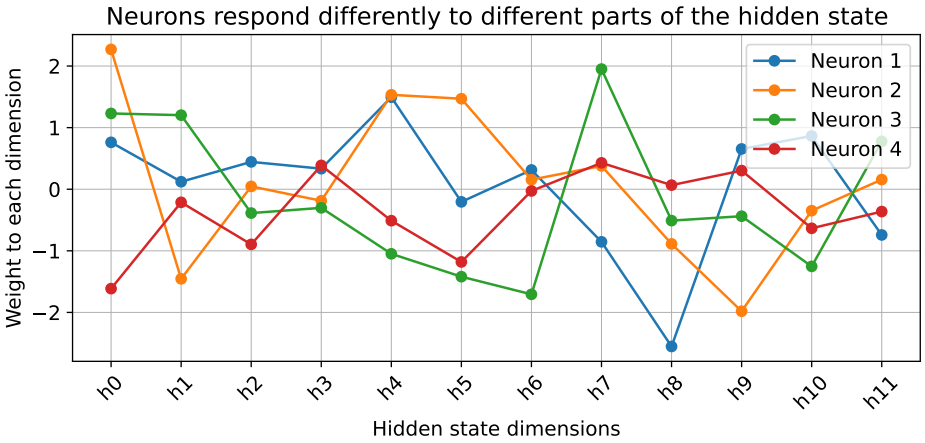


Figure 30: Line plot of 4 different neurons (lines), each extracting different information from the same hidden state vector (x-axis). The y-values represent the weight magnitudes, showing how each neuron focuses on different parts of the hidden state to extract unique features. So even though **all neurons receive the same hidden state vector** h, they *do different things with it*, they extract different aspects of the stored memory.

Why are there two matrices W_x and W_h in the RNN equation?

We will explore them in the next sections, but we can already clarify their roles here (conceptually). In the RNN equation:

$$h_t = f\left(W_x \cdot x_t + W_h \cdot h_{t-1} + b\right)$$

We have two different weight matrices:

- W_x **Input-to-Hidden Weight Matrix**. This matrix transforms the **current input vector** x_t into the hidden space. In other words, it answers the question: *"how does the current input influence the new memory state?"*. If x_t (the input) has dimension d_{in} and the hidden state has dimension d_h, then W_x (the input-to-hidden weight matrix) has shape (d_h, d_{in}). For example, if x_t is a word embedding of size 100 and the RNN hidden state size is 64, then W_x will be a matrix of shape $(64, 100) = 6400$ parameters.
- W_h **Hidden-to-Hidden Weight Matrix**. This matrix transforms the **previous hidden state** h_{t-1} into the new hidden space:

  $$W_h \cdot h_{t-1}$$

  This is the **core of memory** in an RNN, as it tells how past information influences the present, it recycles the previous hidden state into the new one and it is applied **at every time step**. If hidden size is d_h, then W_h has shape (d_h, d_h). It is **square** because it transforms hidden states into hidden states. For example, if the RNN hidden state size is 64, then W_h will be a matrix of shape $(64, 64) = 4096$ parameters.

In the next section, we will see how these matrices operate when the RNN is unrolled over time (i.e., when we visualize the RNN processing a sequence step by step). This will help clarify their distinct roles in shaping the RNN's memory dynamics.

4.4.2 Nonlinear Update Equations with Weights

As we have anticipated in (107) on page 232 of the definition, RNNs use **learned weight matrices** to transform the input and hidden states before applying a nonlinear activation function. The general form of the update equation is:

$$h_t = f\left(W_x \cdot x_t + W_h \cdot h_{t-1} + b\right)$$

But what does the equation really mean? There are two "sources" of information for the new hidden state h_t:

1. **Current input** $\rightarrow$ via W_x:

   $$W_x \cdot x_t$$

   This says "***what does the new observation*** x_t ***mean?***". It captures the information from the current input at time t. It is similar to weights in a normal feedforward neural network, transforming the input into a feature representation.

2. **Previous hidden state** $\rightarrow$ via W_h:

   $$W_h \cdot h_{t-1}$$

   This is the **memory contribution** and carries information from all past time steps (via h_{t-1}). It answers the question "***what do we remember from the past?***".

3. **Add bias and apply nonlinearity**:

   $$f(\cdot) = \tanh,\ \text{ReLU},\ \text{sigmoid},\ \ldots$$

   This introduces **nonlinearity**, making the system much more expressive than a linear dynamical system. The bias b allows shifting the activation function, enabling the model to learn more complex patterns.

An alternative notation that is often used in packages such as `PyTorch` and `TensorFlow` is:

$$h_t = \sigma\left(W_x \cdot x_t + W_h \cdot h_{t-1} + b\right)$$

Where σ is a generic nonlinear activation function (e.g., tanh, ReLU, sigmoid, etc.).

Why this equation is *nonlinear* and why it matters?

If the activations were removed:

$$h_t = W_x \cdot x_t + W_h \cdot h_{t-1}$$

We get a **Linear Dynamical System** (LDS), that is equivalent to Kalman filtering, and cannot learn complex temporal patterns. Also, memory decays or explodes geometrically over time, making it hard to capture long-term dependencies. Instead, adding the nonlinearity:

$$h_t = \tanh(\cdot)$$

Turns the system into a **universal nonlinear function approximator** for sequences.

What do the weight matrices W_x and W_h represent?

As we have anticipated on page 236, the RNN update equation contains two distinct weight matrices.

Every time step uses the same matrices W_x and W_h to process the input and hidden state. This parameter sharing allows the RNN to generalize across time steps, learning temporal patterns that are invariant to position in the sequence.

- W_x**: Input $\xrightarrow{\text{to}}$ Hidden weights**. "***How should the network interpret the current input x_t?***". The role of this matrix is to **map** the **incoming signal** x_t into the **hidden state space**.
  - It **extracts** features from the *current* time step.
  - It **determines** what part of the current input is important.
  - It **transforms** x_t into a form that can be combined with memory.

  If the input is a word embedding, sensor reading, pixel row, etc., W_x learns how to process it. For example, if x_t is a word embedding like "pizza", $W_x \cdot x_t$ might activate "food topic" neurons in the hidden state.

  Mathematically, W_x has shape:

  $$W_x \in \mathbb{R}^{H \times D}$$

  Where D is the input dimension and H is the hidden state dimension.

- W_h**: Hidden $\xrightarrow{\text{to}}$ Hidden weights**. "***How should the network update its memory based on the past?***". This matrix controls how the **previous hidden state** affects the **next one**.
  - It tells the RNN **what to remember**, **what to forget**, and **how**.
  - It controls **how much** of **past information flows into the present**.
  - It defines **how memory evolves over time**.

  This is the **core** of recurrence: **the hidden state feeds back into itself through W_h**, allowing the network to build a temporal context. For example, if earlier in the sequence we saw "not", and now we see "bad", $W_h \cdot h_{t-1}$ helps interpret the meaning based on the negation remembered earlier.

  Mathematically, W_h has shape:

  $$W_h \in \mathbb{R}^{H \times H}$$

  Since it maps the hidden state back into itself. The H terms correspond to the number of hidden units.

Why two separate weight matrices?

Because the RNN must combine **two types of information** at each time step:

- **New information** (from x_t).
- **Past information** (from h_{t-1}).

They play different roles:

- W_x **processes the current input**, extracting relevant features: "*how to read the current input*".
- W_h **manages the memory**, determining how past states influence the present: "*how to transform the past memory*".

Together they form:

$$h_t = \text{memory update}\left(\underbrace{\text{input processing}\,(x_t)}_{\text{via } W_x}, \underbrace{\text{memory transformation}\,(h_{t-1})}_{\text{via } W_h}\right)$$

Where do W_x and W_h come from?

As we will see in the upcoming section on **Backpropagation Through Time (BPTT)**, these matrices are **initialized randomly** and then **learned from data** through gradient descent. During training, the RNN adjusts W_x and W_h to minimize the loss function, effectively learning how to interpret inputs and manage memory for the specific task at hand (e.g., language modeling, time series prediction, etc.). Over time, they converge to values that allow the RNN to capture complex temporal dependencies in the data.

4.4.3 Universal Computation Capability (Hava Siegelmann)

Recurrent Neural Networks (RNNs) with rational weights and sigmoidal activation functions are capable of simulating Turing machines, thus possessing universal computation capability. The idea comes from the work of Hava Siegelmann and Eduardo Sontag in the 1995s, who demonstrated that RNNs can perform computations equivalent to those of Turing machines, given sufficient time and resources. In other words, **a simple Recurrent Neural Network (RNN), with nonlinear activation, can simulate any computable function** (i.e., it is as powerful as a Turing machine).

What does "Universal Computation" mean?

The term "*universal computation*" refers to the **ability of a computational system to perform any computation that can be described algorithmically**. In the context of RNNs, it means that these networks can simulate any Turing machine, which is a theoretical model of computation that can solve any problem that is computable. This implies that RNNs can, in theory, learn and execute any algorithm or function that a Turing machine can, given enough time and resources.

Why is this important? Implications of Universal Computation Capability of RNNs

The universal computation capability of RNNs has several important implications:

- **Expressive Power**: RNNs can represent a wide range of functions and algorithms, making them versatile for various tasks, including language modeling, sequence prediction, and time series analysis.
- **Learning Complex Patterns**: RNNs can learn complex temporal patterns and dependencies in sequential data, which is crucial for tasks like speech recognition and natural language processing.
- **Theoretical Foundation**: The ability of RNNs to simulate Turing machines provides a strong theoretical foundation for their use in machine learning and artificial intelligence.
- **Limitations and Challenges**: While RNNs are theoretically capable of universal computation, practical limitations such as training difficulties, vanishing/exploding gradients, and computational resource constraints can hinder their performance in real-world applications.

Overall, the universal computation capability of RNNs highlights their potential as powerful computational models, capable of tackling a wide range of complex problems in various domains.

Intuition: Why RNNs are Turing Complete?

A machine is considered Turing complete if it can simulate a Turing machine, which means it can perform any computation that a Turing machine can, given enough time and resources. So, why are RNNs Turing complete? Consider:

- The hidden state h_t.
- The recurrence $h_t = f\left(W_x \cdot x_t + W_h \cdot h_{t-1}\right)$

The hidden state acts like a **memory tape** (continuous, high-dimensional) that evolves according to a **rule** feeding into itself indefinitely. This is exactly what a Turing machine does: it has a memory tape, a transition rule, and uses past state and current input to compute the next state. Thus, RNN can emulate read/write operations on a tape, transitions, state machines, counters, stacks, etc. With appropriate weights and activation functions, RNNs can implement the necessary operations to simulate any Turing machine, thereby achieving Turing completeness.

Caveat: Practical Limitations

While RNNs are theoretically Turing complete, practical limitations exist:

- ✗ RNNs cannot run infinite time steps in practice.
- ✗ They do not have infinite precision in weights and activations.
- ✗ They suffer from vanishing and exploding gradient problems during training.
- ✗ They cannot store long-term memory reliably.
- ✗ Training is extremely challenging for complex tasks.

These **limitations have led to the development of more advanced RNN architectures**, such as Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs), which are designed to mitigate some of these challenges and improve the ability of RNNs to learn long-term dependencies in sequential data.

4.5 Backpropagation Through Time (BPTT)

Backpropagation Through Time (BPTT) is the algorithm used to train Recurrent Neural Networks (RNNs). It is simply **backpropagation** (page 100) **applied to the unrolled RNN**. However, an RNN is **not a standard feed-forward network** because it has **cycles** due to its recurrent connections:

$$h_{t-1} \rightarrow [\text{RNN cell}] \rightarrow h_t \rightarrow [\text{RNN cell}] \rightarrow h_{t+1} \rightarrow \dots$$

These cycles create **dependencies over time**, making the RNN **deep in time** rather than deep in space. So, the RNN cannot run backpropagation directly on its cyclic graph. The solution is to **unroll the RNN over time**, converting it into a **Directed Acyclic Graph (DAG)**. Once unrolled, we can perform backpropagation as usual, computing gradients for each time step and accumulating them to update the shared weights.

Once unrolled, we simply:

1. Run forward pass through all timesteps:

$$\text{compute } h_1, h_2, \dots, h_T$$

2. Compute the loss at each timestep and sum them:

$$E = \sum_{t=1}^{T} E_t$$

3. Run backpropagation through the unrolled network, computing gradients at each timestep and accumulating them:

$$\frac{\partial E}{\partial W_x} = \sum_{t=1}^{T} \frac{\partial E_t}{\partial W_x}, \quad \frac{\partial E}{\partial W_h} = \sum_{t=1}^{T} \frac{\partial E_t}{\partial W_h}, \quad \frac{\partial E}{\partial b} = \sum_{t=1}^{T} \frac{\partial E_t}{\partial b}$$

4. Update the shared weights using the accumulated gradients.

That is Backpropagation Through Time in a nutshell. It is just plain backpropagation but applied **through time**, not space. But, **how do we unroll an RNN?** Let's explore that next.

Deepening: Why backpropagation cannot be applied directly to RNNs?

In a Recurrent Neural Network, the main issue is **not** that a parameter is used multiple times, but that the computational graph contains **cycles**.

In a standard feed-forward neural network, the computation flows in one direction:

$$\text{input } x \rightarrow \text{layer } W \rightarrow \text{hidden } h \rightarrow \text{loss } E$$

This defines a **Directed Acyclic Graph (DAG)**, which is required for backpropagation to work.

In contrast, an RNN introduces a **recurrent connection**:

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

This creates a **cycle**:

$$h_{t-1} \rightarrow h_t \rightarrow h_{t+1} \rightarrow \dots$$

Why is this a problem? Because backpropagation requires a DAG, but the RNN defines a **cyclic (recursive) computation**. Therefore, we cannot directly apply backpropagation.

Let's see from a mathematical perspective. In an RNN:

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

The term h_{t-1} is a function of h_{t-2}, which is a function of h_{t-3}, and so on. This is a **nested function composition**:

$$h_t = f\left(W_x x_t + W_h f\left(W_x x_{t-1} + W_h f\left(W_x x_{t-2} + \dots\right)\right)\right)$$

Now, if we try to compute the gradient $\frac{\partial E}{\partial W_h}$, because we want to update W_h, we would have to apply the chain rule through this nested composition. This means we would have to compute:

$$\frac{\partial E}{\partial W_h} = \frac{\partial E}{\partial h_t} \cdot \frac{\partial h_t}{\partial W_h}$$

But $\frac{\partial h_t}{\partial W_h}$ depends on:

$$\frac{\partial h_t}{\partial W_h} = \frac{\partial f}{\partial W_h} + \frac{\partial f}{\partial h_{t-1}} \cdot \frac{\partial h_{t-1}}{\partial W_h}$$

And $\frac{\partial h_{t-1}}{\partial W_h}$ depends on:

$$\frac{\partial h_{t-1}}{\partial W_h} = \frac{\partial f}{\partial W_h} + \frac{\partial f}{\partial h_{t-2}} \cdot \frac{\partial h_{t-2}}{\partial W_h}$$

And so on, leading to a recursive definition of the gradient, where each term depends on the previous one. To compute it explicitly, we must expand this recursion over a finite number of time steps.

Solution: BPTT (unrolling). We convert the cyclic graph into a **finite, acyclic graph** by unrolling the RNN over T time steps:

$$x_1 \to h_1 \to h_2 \to \ldots \to h_T$$

Now the model becomes equivalent to a deep feed-forward network, and backpropagation can be applied.

What changes after unrolling? Although unrolling enables backpropagation, it reveals that the same weights are used at every time step. Therefore, each parameter influences the loss through **many paths across time**:

$$\frac{\partial E}{\partial W_h} = \sum_{t=1}^{T} \frac{\partial E}{\partial h_t} \cdot \frac{\partial h_t}{\partial W_h}$$

This leads to **long chains of dependencies**, which in turn cause the well-known **vanishing and exploding gradient problems**. But we will discuss that separately.

But why not use all time steps? In principle, the correct gradient requires considering **all previous time steps**. However, in practice this is often inefficient:

- The computational cost grows linearly with the sequence length T, requiring both **more memory** (to store all intermediate states) and **more time** for backpropagation.
- More importantly, contributions from distant time steps tend to become **negligible** due to the repeated multiplication of Jacobians, leading to the **vanishing/exploding gradient problem**.

We will discuss these issues in more detail in the next section, but the key point is that while BPTT is the correct algorithm for training RNNs, it is often impractical to apply it over long sequences due to computational and optimization challenges.

For these reasons, a common practical solution is **Truncated Backpropagation Through Time (TBPTT)**, where gradients are propagated only for a limited number of steps (i.e., a BPTT but truncated to a fixed window).

In this course, however, we focus on the **full BPTT formulation**, as it clearly exposes the limitations of RNNs and motivates more advanced architectures such as **LSTMs and GRUs**.

4.5.1 RNN unrolling over U time steps

Imagine a normal Feed-Forward Neural Network (FNN):

$$x \to [\text{Layer 1}] \to [\text{Layer 2}] \to \ldots \to [\text{Layer N}] \to y$$

Each layer transforms the signal and passes it forward to the next layer. This is **deep in space** because we have multiple layers stacked vertically (one on top of the other).

Now, instead of stacking layers in space (vertically), we stack the *same* layer **in time** (horizontally):

$$\underbrace{\text{RNN cell}}_{t=1} \to \underbrace{\text{RNN cell}}_{t=2} \to \ldots \to \underbrace{\text{RNN cell}}_{t=U}$$

That's all unrolling is. **Each cell is the same network**. It just receives **different inputs** (x_1, x_2, …, x_U) at each time step and the **previous hidden state** instead of the previous layer's output. So, an RNN is basically **a small neural network copied over and over along the time axis**, all copies share the same weights and each copy processes one timestep.

What is an RNN cell? A **cell** is the *tiny neural network* that computes:

$$h_t = f\left(W_x \cdot x_t + W_h \cdot h_{t-1} + b\right)$$

This little network contains:

- The weight matrix W_x that processes the **current input** x_t;
- The weight matrix W_h that processes the **previous hidden state** h_{t-1};
- The bias vector b;
- The activation function f (e.g., tanh or ReLU).

This is the **entire RNN**. A cell is a function:

$$(x_t,\, h_{t-1}) \mapsto h_t$$

That's it. See Figure 31, page 246 for a visual representation of the RNN cell.

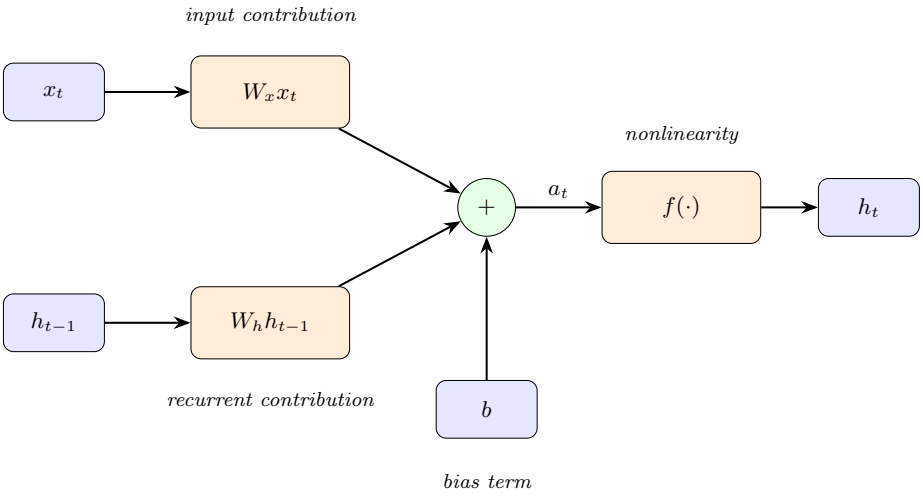
What does it mean that "all cells are the same"? It means:

- They use the **same weight matrices**, the same W_x, W_h, and b, for every time step.
- They are the **same function** applied repeatedly over time. The RNN does not create new parameters at each time step; it **reuses the same ones**.

In code, an RNN is just a **for loop** that applies the same function (the RNN cell) to each input in the sequence, passing along the hidden state:

```
h = h0  # initial hidden state
for t in range(U):
    # same RNN_cell function at each time step
    h = RNN_cell(x[t], h)
```

There is only **one copy** of W_x and W_h in memory, shared across all time steps. But the loop executes **multiple times**, so it *looks like* the function is repeated many times. In previous diagrams, we drew **multiple copies of the cell** to illustrate the unrolling over time, but in reality, it's **just the same cell being applied repeatedly**.



$$a_t = W_x x_t + W_h h_{t-1} + b \qquad h_t = f(a_t)$$

Figure 31: **Zoom-in of a single RNN cell**. The cell combines the current input x_t and the previous hidden state h_{t-1} through two linear transformations, adds the bias b, and applies a nonlinearity $f(\cdot)$ to produce the new hidden state h_t.

What does "unroll" mean?

It means we create **U copies** of the RNN cell, one per time step:

$$\begin{aligned}
x_{t-3} &\to \underbrace{\text{RNN cell}}_{t-3} \to h_{t-3} \\
x_{t-2} &\to \underbrace{\text{RNN cell}}_{t-2} \to h_{t-2} \\
x_{t-1} &\to \underbrace{\text{RNN cell}}_{t-1} \to h_{t-1} \\
x_t &\to \underbrace{\text{RNN cell}}_{t} \to h_t
\end{aligned}$$

Each cell processes its own input and previous hidden state, producing its own output hidden state. But **all cells share the same weights**. Unrolling is just a way to visualize how the RNN processes sequences over time. This means:

- In forward propagation → every timestep uses the same weights to compute its hidden state.
- In backpropagation → gradients from all time steps accumulate to update the *same* weights.
- In update rules → we average (or sum) gradients from all time steps to adjust the shared weights.

Why unroll an RNN? Unrolling helps us understand how the RNN processes sequences step-by-step. But, more importantly, unrolling is essential for **training** the RNN using *Backpropagation Through Time (BPTT)*. BPTT requires a DAG (Directed Acyclic Graph) structure to compute gradients, explicit connections between operations, and a clear path for gradients to flow backward through time. The RNN loop must be turned into a **long chain** before we can apply backpropagation. See Figure 32, page 249.

Mathematical view of unrolling

For U time steps, the loss function E depends on the outputs at each time step:

$$E = \sum_{t=1}^{U} E_t$$

Where E_t is the **loss at time step** t. Each output h_t depends on the weights W_x, W_h, and b shared across all time steps. During backpropagation, we compute the gradients of the loss with respect to the weights:

$$\frac{\partial E}{\partial W_x} = \sum_{t=1}^{U} \frac{\partial E_t}{\partial W_x}, \quad \frac{\partial E}{\partial W_h} = \sum_{t=1}^{U} \frac{\partial E_t}{\partial W_h}, \quad \frac{\partial E}{\partial b} = \sum_{t=1}^{U} \frac{\partial E_t}{\partial b}$$

This means that the **total gradient for each weight matrix is the sum of the gradients from each time step**. This accumulation is crucial for updating the shared weights during training. Each term depends not just on timestep t but via the recurrence on all previous timesteps:

$$h_{t-1}, h_{t-2}, \ldots, h_0$$

This is why we need to unroll the RNN: to explicitly see how each time step's output depends on all previous time steps, allowing us to compute gradients correctly through time. Also, this is what makes RNN training **deep in time** (many time steps) rather than deep in space (many layers).

In summary, when training:

1. **Choose U**, the number of past steps to backpropagate (full sequence or truncated).

2. **Create U replicas** of the RNN cell in computation graph.
3. **Forward pass**, compute:

$$h_{t-U},\, h_{t-U+1},\, \ldots,\, h_t$$

   With shared weights.

4. **Backward pass**, compute gradients from t down to $t-U$:

$$\frac{\partial E}{\partial W_x},\quad \frac{\partial E}{\partial W_h},\quad \frac{\partial E}{\partial b}$$

5. **Combine all gradients** for shared weights W_x, W_h, and b.
6. **Update** the single shared weight matrices using the combined gradients.

RNN Unrolling over U Time Steps (with BPTT)

Require: Sequence $\{x_1, \ldots, x_U\}$, initial hidden state h_0
Require: Parameters W_x, W_h, b, learning rate η
$h_0 \leftarrow$ initial hidden state
$E \leftarrow 0$
▷ Forward pass (unrolling)
for $t = 1$ to U **do**
    $h_t \leftarrow f(W_x x_t + W_h h_{t-1} + b)$
    Compute loss E_t
    $E \leftarrow E + E_t$
end for
▷ Initialize gradients
$\frac{\partial E}{\partial W_x} \leftarrow 0$
$\frac{\partial E}{\partial W_h} \leftarrow 0$
$\frac{\partial E}{\partial b} \leftarrow 0$
▷ Backward pass (BPTT)
for $t = U$ down to 1 **do**
    Compute $\frac{\partial E_t}{\partial h_t}$
    Propagate gradient through time to h_{t-1}
    Accumulate:
        $\frac{\partial E}{\partial W_x} \leftarrow \frac{\partial E}{\partial W_x} + \frac{\partial E_t}{\partial W_x}$
        $\frac{\partial E}{\partial W_h} \leftarrow \frac{\partial E}{\partial W_h} + \frac{\partial E_t}{\partial W_h}$
        $\frac{\partial E}{\partial b} \leftarrow \frac{\partial E}{\partial b} + \frac{\partial E_t}{\partial b}$
end for
▷ Parameter update
$W_x \leftarrow W_x - \eta \cdot \frac{\partial E}{\partial W_x}$
$W_h \leftarrow W_h - \eta \cdot \frac{\partial E}{\partial W_h}$
$b \leftarrow b - \eta \cdot \frac{\partial E}{\partial b}$

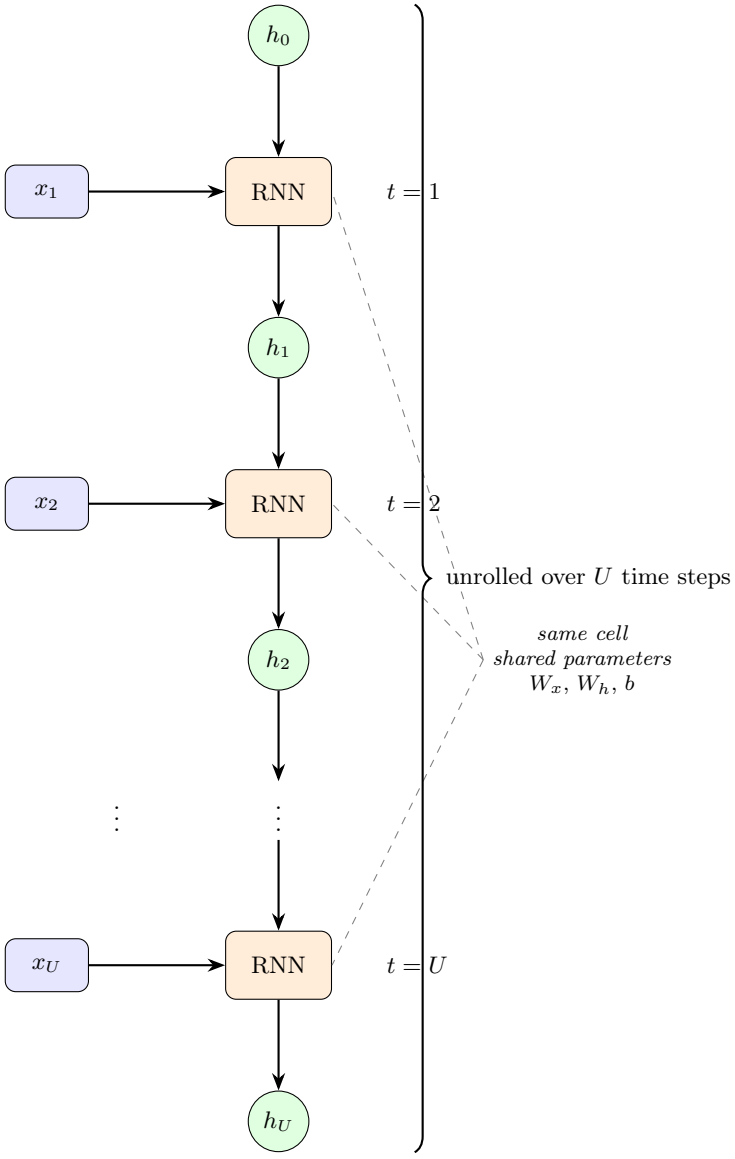


Figure 32: Unrolling an RNN over U time steps. Each RNN cell processes its own input and previous hidden state, producing its own output hidden state. All cells share the same parameters, illustrating how the same function is applied repeatedly across time.

4.5.2 Shared weights across time

In the previous section, we saw how to unroll an RNN over U time steps, creating U copies of the RNN cell. Now, let's discuss a crucial aspect of RNNs: **shared weights across time**. Everything in BPTT, vanishing gradients, and even LSTMs depends on this idea, because it allows RNNs to generalize across different time steps in a sequence.

What "shared across time" means

When we unroll an RNN through time:

$$\begin{aligned} x_1 &\rightarrow [\text{cell}]_1 \rightarrow h_1 \\ x_2 &\rightarrow [\text{cell}]_2 \rightarrow h_2 \\ x_3 &\rightarrow [\text{cell}]_3 \rightarrow h_3 \\ &\vdots \quad\quad \vdots \\ x_U &\rightarrow [\text{cell}]_U \rightarrow h_U \end{aligned}$$

We *draw* multiple RNN cells. But all of them must use the **same parameters**:

- Same W_x (input-to-hidden weights);
- Same W_h (hidden-to-hidden weights);
- Same b (biases);

In the diagram they appear as:

$$\begin{aligned} W_x^{(1)}, W_h^{(1)} &\rightsquigarrow [\text{cell}]_1 \\ W_x^{(2)}, W_h^{(2)} &\rightsquigarrow [\text{cell}]_2 \\ W_x^{(3)}, W_h^{(3)} &\rightsquigarrow [\text{cell}]_3 \\ \vdots \quad\quad &\quad\quad \vdots \\ W_x^{(U)}, W_h^{(U)} &\rightsquigarrow [\text{cell}]_U \end{aligned}$$

But these are **not** different matrices! They are **copies** of the same matrices:

$$W_x^{(1)} = W_x^{(2)} = W_x^{(3)} = \cdots = W_x^{(U)} = W_x$$

$$W_h^{(1)} = W_h^{(2)} = W_h^{(3)} = \cdots = W_h^{(U)} = W_h$$

Why must weights be shared?

The RNN is a *single* function repeated over time. This means that the recurrence:

$$h_t = f\left(W_x x_t + W_h h_{t-1} + b\right)$$

It is the **same rule** applied at each time step t. If we allowed different weights at different time steps, the RNN would not be able to generalize across time,

and it would require a separate set of parameters for each time step, leading to an explosion in the number of parameters. Mathematically, this would mean:

$$\text{Weights not shared} \implies W_x^{(t)}, W_h^{(t)}, b^{(t)} \text{ for each } t = 1, 2, \ldots, U$$

Where the number of parameters grows linearly with the number of time steps U. This would make training infeasible for long sequences and prevent the model from learning temporal patterns that are consistent across time.

✔ Instead, by **sharing weights**, we ensure that the **RNN allows the network to learn consistent temporal dynamics**, because the **same operation is applied again and again over time**, just like the same function is applied to different inputs in a feedforward neural network. This weight sharing is fundamental to the success of RNNs in modeling sequential data.

What does sharing really mean during training?

When we unroll for U time steps, each replica accumulates gradients for the **same parameters**. So the gradient update is:

$$\frac{\partial E}{\partial W_h} = \sum_{t=1}^{U} \frac{\partial E_t}{\partial W_h^{(t)}}$$

But all these $W_h^{(t)}$ are the **same matrix** W_h. So we:

1. Accumulate all gradients across time;
2. Average (or sum) them;
3. Apply the update *once* to the single shared matrix W_h.

In other words, during backpropagation through time, we **compute gradients at each time step**, but since the weights are shared, we **combine these gradients to update the single set of parameters**. This ensures that learning is consistent across all time steps and allows the RNN to effectively capture temporal dependencies in the data.

> **Example 7: Sharing weights analogy**
>
> Imagine we have a single recipe (the RNN cell) that we use to bake multiple cakes (time steps). Each time we bake a cake, we follow the same recipe (shared weights). If we find that the cakes are too sweet, we adjust the recipe once (update shared weights) based on feedback from all the cakes we've baked, rather than changing the recipe for each individual cake. This way, all future cakes benefit from the improved recipe.

⚠ Why is this critical for vanishing/exploding gradients?

As we have seen, the vanishing (page 177) and exploding (page 179) gradient problems arise during backpropagation through time due to the repeated multiplication of very small or very large gradients. But, *why is RNN weight sharing critical here?*

The key point is that **the same weights are used at each time step**, so during backpropagation, the gradients are repeatedly multiplied by the same weight matrices W_h:

$$\begin{aligned} h_t &= f\left(W_h \cdot h_{t-1}\right) \\ h_{t-1} &= f\left(W_h \cdot h_{t-2}\right) \\ h_{t-2} &= f\left(W_h \cdot h_{t-3}\right) \\ \vdots & \quad \vdots \\ h_1 &= f\left(W_h \cdot h_0\right) \end{aligned}$$

And during backpropagation, the gradients uses the **Jacobian** (derivative) of the hidden state with respect to the previous hidden state:

$$\frac{\partial h_t}{\partial h_{t-1}} = W_h^T \cdot f'\left(\cdot\right)$$

So going backwards from T to 0:

$$\frac{\partial h_T}{\partial h_0} = \prod_{t=1}^{T} \frac{\partial h_t}{\partial h_{t-1}} = \prod_{t=1}^{T} \left(W_h^T \cdot f'\left(\cdot\right)\right)$$

Because the same W_h is used at each time step, the gradients are repeatedly multiplied by the same matrix for T time steps. ⚠ **And here lies the problem**: since W_h is constant, if its eigenvalues are less than 1, the gradients will shrink exponentially (vanishing gradients); if they are greater than 1, the gradients will grow exponentially (exploding gradients). So, this entire phenomenon of vanishing/exploding gradients is **directly tied to the fact that weights are shared across time** in RNNs.

> 💬 **Remark: Why eigenvalues matter**
>
> The behavior of repeated matrix multiplications is governed by the eigenvalues of the matrix. If the eigenvalues of W_h are:
>
> - **Less than 1**: The gradients shrink exponentially, leading to vanishing gradients.
> - **Greater than 1**: The gradients grow exponentially, leading to exploding gradients.
> - **Equal to 1**: The gradients remain stable, avoiding both vanishing and exploding issues.
>
> But the real question is: ***Why eigenvalues matter when multiplying matrices repeatedly?*** Because **eigenvalues tell us how the matrix**

stretches or shrinks vectors when repeatedly applied.
What is an eigenvalue? For a square matrix W, a vector v is an eigenvector if:

$$Wv = \lambda v$$

Meaning that v does not change direction when multiplied by W, only its magnitude is scaled by λ (the eigenvalue).

- $\lambda > 1$: vector grows (stretches)
- $\lambda < 1$: vector shrinks (compresses)
- $\lambda = 1$: vector remains the same length
- $\lambda < 0$: vector flips direction

Thus **eigenvalues describe the scaling effect of a matrix**.

4.5.3 Training Algorithm Steps

The training algorithm for Backpropagation Through Time (BPTT) can be summarized in the following steps:

1. **Forward Pass (Unroll for U Time Steps)**. Unroll the RNN across time:

$$\begin{aligned} x_1 \rightarrow & \quad [\text{cell}] \quad \rightarrow h_1 \\ x_2 \rightarrow & \quad [\text{cell}] \quad \rightarrow h_2 \\ & \vdots \qquad\quad \vdots \\ x_U \rightarrow & \quad [\text{cell}] \quad \rightarrow h \end{aligned}$$

   Compute at each time step t:

   (a) The new **hidden state** h_t using the current input x_t and the previous hidden state h_{t-1}:

$$h_t = f\left(W_x \cdot x_t + W_h \cdot h_{t-1}\right)$$

   (b) The **output** y_t based on the hidden state h_t:

$$y_t = g\left(U \cdot h_t\right)$$

   Where g is the output activation function, and U is the output weight matrix that maps hidden states to outputs.

   (c) The **loss** E_t at each time step using the predicted output y_t and the true target $\hat{y}_t$:

$$E_t = \text{loss}\left(y_t, \hat{y}_t\right)$$

   The total loss over the sequence is computed as:

$$E = \sum_{t=1}^{U} E_t$$

2. **Backward Pass (Backprop Through Time)**. Backpropagate errors through the **unrolled network**, from $t = U$ down to $t = 1$, that mathematically is expressed as follows:

$$\frac{\partial E}{\partial \theta} = \sum_{t=1}^{U} \frac{\partial E_t}{\partial \theta}$$

   Where θ represents the shared weights: W_x, W_h, and U. At each timestep t compute:

   (a) The **gradient of the loss with respect to the hidden state** h_t:

$$\frac{\partial E}{\partial h_t} = \frac{\partial E_t}{\partial y_t} \cdot \frac{\partial y_t}{\partial h_t} + \frac{\partial E}{\partial h_{t+1}} \cdot \frac{\partial h_{t+1}}{\partial h_t}$$

   Depends on both E_t and the gradient from the next time step h_{t+1}. In other words, errors are propagated backward through time.

(b) The **gradients with respect to the weights**:

$$\frac{\partial E}{\partial W_x}, \quad \frac{\partial E}{\partial W_h}, \quad \frac{\partial E}{\partial U}$$

3. **Accumulate/Average Gradients Across Time**. Because weights are **shared across time**, each timestep contributes to their gradient. Accumulate these gradients:

$$\frac{\partial E}{\partial W_x} = \sum_{t=1}^{U} \frac{\partial E_t}{\partial W_x}, \quad \frac{\partial E}{\partial W_h} = \sum_{t=1}^{U} \frac{\partial E_t}{\partial W_h}, \quad \frac{\partial E}{\partial U} = \sum_{t=1}^{U} \frac{\partial E_t}{\partial U}$$

4. **Update Shared Weights**. Use the accumulated gradients to update the weights using an optimization algorithm (e.g., Stochastic Gradient Descent, Adam):

$$\begin{aligned} W_h &\leftarrow W_h - \eta \frac{\partial E}{\partial W_h} \\ W_x &\leftarrow W_x - \eta \frac{\partial E}{\partial W_x} \\ U &\leftarrow U - \eta \frac{\partial E}{\partial U} \end{aligned}$$

Where η is the learning rate (page 197). These updated weights will be used for the next training iteration.

5. **Repeat for Multiple Epochs**. Repeat the above steps for multiple epochs over the training dataset until convergence or until a stopping criterion is met (e.g., validation loss stops improving).

The BPTT training algorithm **unrolls the RNN** in time, performs a **forward and backward pass** across all unrolled steps, **accumulates gradients** for shared weights, and **updates them using gradient descent**.

4.5.4 Vanishing and Exploding Gradients Limitation

The vanishing and exploding gradient problem is the **core difficulty** of training "vanilla" RNNs, and the reason LSTMs and GRUs were invented. We saw some anticipation on page 252, but here, we will delve deeper into the mathematical details of this phenomenon.

Why do gradients vanish or explode in RNNs?

Because during Backpropagation Through Time (BPTT) algorithm, gradients must be passed all the way back through **T steps** (the length of the sequence). This means the gradient involves repeatedly multiplying by the same matrix:

$$\frac{\partial h_t}{\partial h_{t-1}} = W_h^T \cdot f'(\cdot)$$

Where $f'(\cdot)$ is the derivative of the activation function (like tanh or σ). So across T time steps, the gradient becomes:

$$\frac{\partial h_T}{\partial h_0} \propto \left(W_h^T\right)^T$$

Where the $\propto$ symbol indicates proportionality, ignoring the activation derivatives for simplicity. In simple terms, this means we are multiplying the weight matrix W_h **T times**. How we explained earlier, **repeated multiplication by the same matrix amplifies or shrinks signals exponentially**. That's the heart of the vanishing/exploding gradient problem in RNNs.

Most common case: Vanish Gradient

If the eigenvalues of W_h are **less than 1**:

$$|\lambda_i| < 1 \quad \forall i$$

Then as we multiply W_h repeatedly, the gradients **shrink exponentially**:

$$\left(W_h^T\right)^T \to 0 \quad \text{as } T \to \infty$$

This kills the gradient exponentially fast, making it nearly impossible for the model to learn long-term dependencies:

- Early time steps receive near-zero gradients;
- Long-term dependencies are impossible to learn;
- The RNN "forgets" anything older than a few time steps (e.g., 5-10);
- Training focuses only on very recent inputs.

This is why vanilla RNNs cannot learn long sequences effectively.

⚠ Less common case: Exploding Gradient

Conversely, if the eigenvalues of W_h are **greater than 1**:

$$|\lambda_i| > 1 \quad \text{for some } i$$

Then the gradients **grow exponentially**:

$$\left(W_h^T\right)^T \to \infty \quad \text{as } T \to \infty$$

This leads to unstable training, because the gradients:

- Blow up to very large values;
- Produce insane updates to the weights;
- Destabilize training;
- Result in NaN values and divergence.

In practice, **exploding gradients cause the loss curve to suddenly shoot to infinity**, making training impossible without special techniques.

✅ Why exploding gradients are easier to fix

A simple technique called **Gradient Clipping** can effectively mitigate exploding gradients. **Gradient Clipping** is a method where we **limit the maximum value of the gradients** during backpropagation. If the gradient norm exceeds a certain threshold, we scale it down to that threshold. This prevents the gradients from becoming too large and destabilizing training. However, **vanishing gradients are much harder to fix**, which is why architectures like LSTMs and GRUs were developed to address this fundamental limitation of vanilla RNNs.

√× Mathematical Intuition

To understand this mathematically, consider a simple vanilla RNN with hidden state update:

$$h_t = f\left(W_h h_{t-1} + W_x x_t + b\right)$$

We care about **how much the loss at time** t:

$$E_t = \text{loss}\left(y_t, \hat{y}_t\right)$$

Depends on a **hidden state at an earlier time** $t-k$, because this dependence (the gradient) determines whether the model can assign credit to events k steps in the past ($k < t$). If that gradient vanishes or explodes, the network will struggle to learn long-range dependencies. In other words, the ability of the RNN to learn from past information hinges on the behavior of this gradient: if it becomes too small (vanishes), the model forgets; if it becomes too large (explodes), training becomes unstable.

Backpropagation Through Time (BPTT) algorithm tells us that:

$$\frac{\partial E_t}{\partial h_k} = \frac{\partial E_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial h_{t-1}} \cdot \frac{\partial h_{t-1}}{\partial h_{t-2}} \cdots \frac{\partial h_{k+1}}{\partial h_k}$$

This expression shows that to compute the gradient of the loss at time t with respect to the hidden state at time k, we need to multiply the gradients at each intermediate time step from k to t. The **gradient starts at time t because that's where the loss is computed**, and we **backpropagate through each time step until we reach time k because that's the point in the past we are interested in** (e.g., to assign credit for events that happened k steps ago).

This can be compactly written as:

$$\frac{\partial E_t}{\partial h_k} = \frac{\partial E_t}{\partial h_t} \cdot \underbrace{\prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}}}_{\text{problem}}$$

So the gradient is a **product of Jacobians** from step k to step t. That product is where vanishing/exploding gradients arise. Since this term causes the gradient to either shrink or grow exponentially, we need to analyze it closely.

Analyzing the Jacobian Terms. Now, we take a closer look at each **Jacobian term** $\frac{\partial h_i}{\partial h_{i-1}}$, because they determine how the gradient behaves as we multiply them together. We can get each Jacobian term from:

$$a_i = W_h h_{i-1} + W_x x_i + b \quad \Rightarrow \quad h_i = \phi(a_i)$$

Where a_i is a **vector** (e.g., 100-dimensional if hidden size is 100), and h_i is obtained by applying the activation function ϕ (like tanh) element-wise to a_i:

$$h_i = \phi\left(a_i\right) \implies \begin{aligned} h_i^{(1)} &= \phi\left(a_i^{(1)}\right) \\ h_i^{(2)} &= \phi\left(a_i^{(2)}\right) \\ &\vdots \\ h_i^{(n)} &= \phi\left(a_i^{(n)}\right) \end{aligned}$$

So **each component of the vector is activated independently**. Using the **chain rule** of calculus (page 100) to understand how h_i changes when we slightly change h_{i-1}:

$$\frac{\partial h_i}{\partial h_{i-1}} = \frac{\partial h_i}{\partial a_i} \cdot \frac{\partial a_i}{\partial h_{i-1}} = \frac{\partial \phi\left(a_i\right)}{\partial a_i} \cdot \frac{\partial a_i}{\partial h_{i-1}}$$

Where:

- $\frac{\partial a_i}{\partial h_{i-1}} = W_h$ is the weight matrix.

- $\dfrac{\partial h_i}{\partial a_i} = \dfrac{\partial \phi(a_i)}{\partial a_i}$ is a **diagonal matrix** with activation derivatives on the diagonal:

$$\frac{\partial h_i}{\partial a_i} = \begin{pmatrix} \phi'(a_i[1]) & 0 & 0 & \dots \\ 0 & \phi'(a_i[2]) & 0 & \dots \\ 0 & 0 & \phi'(a_i[3]) & \dots \\ \vdots & & & \ddots \end{pmatrix} = D_i = \operatorname{diag}(\phi'(a_i))$$

  It is naturally a diagonal matrix because ϕ is applied **element-wise**:

$$h_i = \phi(a_i) = \begin{pmatrix} \phi(a_i[1]) \\ \phi(a_i[2]) \\ \phi(a_i[3]) \\ \vdots \\ \phi(a_i[n]) \end{pmatrix}$$

Putting it together:

$$\frac{\partial h_i}{\partial h_{i-1}} = D_i \cdot W_h$$

Analyzing the Product of Jacobians. Now, we will rewrite the product of Jacobians from time k to time t:

$$\frac{\partial E_t}{\partial h_k} = \frac{\partial E_t}{\partial h_t} \cdot \underbrace{\prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}}}_{\text{problem}}$$

Expanding the product:

$$\prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}} = \prod_{i=k+1}^{t} (D_i \cdot W_h) = D_t \cdot W_h \cdot D_{t-1} \cdot W_h \cdots D_{k+1} \cdot W_h$$

This product alternates between the diagonal matrices D_i (activation derivatives) and the weight matrix W_h. To understand whether this product vanishes or explodes, we need to analyze the **norms** of these matrices. **Why norms?** **Because the norm gives us a measure of the "size" of a matrix**, and repeated multiplication of matrices with norms less than 1 leads to vanishing gradients, while norms greater than 1 lead to exploding gradients (as discussed earlier).

So, we do **not** care about the *exact* gradient matrix. We care about something much simpler: ***does the gradient get bigger or smaller as it flows backwards in time?*** This means we need a **single number** that measures the "magnitude" of a vector or matrix. That number is the **norm**, which is a tool that tells us: if the gradient explodes (norm $\to \infty$), vanishes (norm $\to 0$), or stays the same (norm $\approx$ constant).

The exact gradient is impossible to compute symbolically because of the **non-linear** activations and **complex** interactions. But we don't need the exact value, we need to understand *its trend*. So, instead of computing it exactly, we compute an **upper bound** on its size using norms:

$$\left\|\frac{\partial E_t}{\partial h_k}\right\| = \left\|\frac{\partial E_t}{\partial h_t} \cdot \prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}}\right\|$$

We can use sub-multiplicative property of norms to separate this:

$$\text{sub-multiplicative property} \implies \|AB\| \leq \|A\| \cdot \|B\|$$

$$\left\|\frac{\partial E_t}{\partial h_k}\right\| \leq \left\|\frac{\partial E_t}{\partial h_t}\right\| \cdot \left\|\prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}}\right\|$$

And now, we focus on the second term:

$$\left\|\prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}}\right\|$$

This term determines whether the gradient vanishes or explodes as we back-propagate through time. Using sub-multiplicative property again:

$$\left\|\prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}}\right\| \leq \prod_{i=k+1}^{t} \left\|\frac{\partial h_i}{\partial h_{i-1}}\right\|$$

Recalling that:

$$\frac{\partial h_i}{\partial h_{i-1}} = D_i \cdot W_h$$

We substitute:

$$\left\|\prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}}\right\| \leq \prod_{i=k+1}^{t} \|D_i \cdot W_h\|$$

Using sub-multiplicative property one last time:

$$\|D_i \cdot W_h\| \leq \|D_i\| \cdot \|W_h\|$$

So we have:

$$\left\|\prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}}\right\| \leq \prod_{i=k+1}^{t} \|D_i\| \cdot \|W_h\|$$

Since $\|W_h\|$ is constant across time steps, we can factor it out:

$$\left\|\prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}}\right\| \leq \|W_h\|^{t-k} \cdot \prod_{i=k+1}^{t} \|D_i\|$$

Now, we analyze $\|D_i\|$. Since D_i is a diagonal matrix with activation derivatives on the diagonal, its norm is determined by the maximum absolute value of those derivatives:

$$\|D_i\| = \max_j \left|\phi'\left(a_i[j]\right)\right|$$

For common activation functions like tanh and σ (sigmoid):

- For tanh, the derivative $\phi'(z) = 1 - \tanh^2(z)$ has a maximum value of 1 (at $z = 0$, page 68).
- For σ (sigmoid), the derivative $\phi'(z) = \sigma(z) \cdot (1 - \sigma(z))$ has a maximum value of 0.25 (at $z = 0$, page 64).

We define γ_v as the **upper bound on the size of the recurrent weight matrix** W_h's contribution to the gradient:

$$\gamma_v = \|W_h\|$$

And we define γ'_h as the **upper bound on the size of the activation function derivatives**:

$$\gamma'_h = \max_j |\phi'(a_i[j])| \quad \equiv \quad \gamma'_h = \sup_z |\phi'(z)|$$

Where sup denotes the supremum (least upper bound) over all possible inputs z. For tanh and σ, we have:

$$\gamma'_h = \begin{cases} 1 & \text{for tanh} \\ 0.25 & \text{for } \sigma \end{cases}$$

Thus, we can bound $\|D_i\|$ by γ'_h:

$$\|D_i\| \leq \gamma'_h$$

Using this bound, we have:

$$\prod_{i=k+1}^{t} \|D_i\| \leq \gamma_h'^{t-k}$$

Hence, **for every timestep** i (using sub-multiplicative property):

$$\|D_i \cdot W_h\| \leq \|D_i\| \cdot \|W_h\| \leq \gamma'_h \cdot \gamma_v$$

And defining:

$$\gamma = \gamma'_h \cdot \gamma_v$$

We get:

$$\|D_i \cdot W_h\| \leq \gamma$$

Putting it all together:

$$\left\| \prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}} \right\| \leq \gamma^{t-k}$$

Therefore:

$$\left\| \frac{\partial E_t}{\partial h_k} \right\| \leq \left\| \frac{\partial E_t}{\partial h_t} \right\| \cdot \gamma^{t-k}$$

This inequality shows that the norm of the gradient $\dfrac{\partial E_t}{\partial h_k}$ depends exponentially on the term γ^{t-k}.

- If $\gamma < 1$, then $\gamma^{t-k} \to 0$ as $t - k \to \infty$, leading to **vanishing gradients**.
- If $\gamma > 1$, then $\gamma^{t-k} \to \infty$ as $t - k \to \infty$, leading to **exploding gradients**.
- If $\gamma = 1$, the gradient norm remains constant over time steps.

4.5.5 Dealing with Gradient Problems

We have seen and demonstrated how RNNs are prone to the vanishing and exploding gradient problems. To mitigate these issues, several techniques can be employed. Here, we introduce **three strategies** commonly used to address these gradient problems in RNNs:

- **(Simplest) Using ReLu activation functions**. ReLu (Rectified Linear Unit) activation functions (page 180) help mitigate the vanishing gradient problem due to their linear, non-saturating nature for positive inputs. Their derivative is:

$$\phi'(z) = 1 \quad \text{for } z > 0$$

  This means the activation derivative does **not shrink** the gradient, unlike sigmoid (max 0.25) or tanh (max 1) functions, which can lead to vanishing gradients over time steps. So with ReLu, the per-step Jacobian looks like:

$$\frac{\partial h_t}{\partial h_{t-1}} = D_t W_h \quad \text{where } D_t = \text{diag}(\phi'(z_t)) \text{ with } \phi'(z_t) \in \{0, 1\}$$

  This greatly reduces the risk of vanishing gradients, as the derivative is either 0 or 1, preventing exponential decay of gradients over time steps.

  - ✔ Suffer **less** from vanishing gradients.
  - ✘ Still can experience **exploding gradients**, as the weight matrix W_h can still lead to large values.
  - ✘ Still **struggle with long-term dependencies**, as ReLU does not inherently solve this issue.
  - ✔ **Train better than sigmoid/tanh** RNNs on many tasks.

  ReLU alone **cannot eliminate vanishing gradient** entirely, because the recurrent matrix W_h is still repeatedly multiplied. The LSTM architecture goes further with **gate-controlled linear gradients**.

- **Gradient Clipping**: limiting gradient magnitude (exploding vs. vanishing). When gradients exceed a certain threshold during backpropagation, they are **scaled down** to prevent instability. This is typically done by:

$$g \leftarrow \frac{g}{\|g\|} \cdot \tau \quad \text{if } \|g\| > \tau$$

  Where g is the gradient vector and τ is the threshold.

  - ✔ **Prevents exploding gradients**, stabilizing training.
  - ✘ **Does not solve vanishing gradients**, so long-term dependencies may still be difficult to learn.

- **Designing modules to retain long-term dependencies**. The core idea is to create an architecture that:
  - **Preserves** gradients over long ranges of time steps.
  - **Does not** repeatedly shrink them.
  - Behaves like an **information highway**.

– Propagates gradients without decay.

This is the entire reason LSTMs were invented. The LSTM (Long Short-Term Memory) architecture introduces **gates** that control the flow of information and gradients, allowing the network to maintain long-term dependencies effectively:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

And crucially:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t$$

Where f_t is the **forget gate**, which can be learned to be close to 1, allowing gradients to pass through many time steps without vanishing.

- ✔ **Effectively mitigates vanishing gradients**, enabling learning of long-term dependencies.
- ✔ **Also helps with exploding gradients**, as the gating mechanisms can regulate information flow.
- ✔ **Widely used** in practice for sequence modeling tasks.
- ✖ More complex and computationally intensive than standard RNNs.

In summary, while ReLU activation functions and gradient clipping can help mitigate gradient problems to some extent, the LSTM architecture provides a more robust solution for handling long-term dependencies in RNNs by effectively addressing both vanishing and exploding gradients through its gating mechanisms.

4.6 Long Short-Term Memory Network (LSTM)

Definition 7: Long Short-Term Memory Network (LSTM)

A **Long Short-Term Memory Network (LSTM)** is a special type of Recurrent Neural Network (RNN) architecture designed to **learn long-term dependencies** in sequential data by using an internal memory cell and a system of gates that control how information is stored, forgotten, and exposed.

It introduces a **memory cell state** C_t, a **hidden state** h_t, and three multiplicative **gates** (input, forget, output). These gates allow the network to:

- **Retain information for long periods**.
- **Protect memory from vanishing gradients**.
- **Selectively update and reset its internal memory** based on the input sequence.

The key feature of an LSTM is a **linear, self-recurrent connection** in the cell state (the Constant Error Carousel, CEC), which preserves information without being destroyed by nonlinear activations, allowing gradients to flow across many timesteps.

4.6.1 Architecture

Problem. Before LSTMs were introduced, (Hochreiter & Schmidhuber, 1997), training Recurrent Neural Networks (RNNs) was extremely difficult. The core issue was **not** model capacity (RNNs are Turing complete, Siegelmann & Sontag, 1995), but rather the **inability to learn long-term dependencies due to vanishing and exploding gradients** (page 262).

Solution. LSTMs were designed specifically to address this issue. The key innovation of LSTMs is the introduction of **memory cells** that can maintain information over long periods of time. Each memory cell contains three main components: the **input gate**, the **forget gate**, and the **output gate**. These gates regulate the flow of information into, out of, and within the memory cell.

How it works. Without going into the gate details, which will be covered in the next sections, the **solution to overcoming vanishing gradients lies in the cell state**, called **memory cell** C_t. A **Memory Cell** is a special **kind of RNN unit** that **maintains its state over time**, allowing it to store information for long periods. The cell state is updated at each time step t based on the input data and the previous cell state, but crucially, it can also **retain information without being overwritten**. This is achieved through the use of **gates** that control the flow of information into and out of the cell state. By carefully regulating these gates, LSTMs can **preserve gradients during backpropagation**, allowing them to learn long-term dependencies effectively.

Mathematically, the memory cell C_t is a linear self-loop (i.e., it can maintain its value over time)

$$C_t = C_{t-1} + (\text{some new information})$$

Or more generally:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{109}$$

- f_t is the **forget gate**, which determines how much of the previous cell state C_{t-1} to retain.
- i_t is the **input gate**, which controls how much new information $\tilde{C}_t$ to add to the cell state.
- $\tilde{C}_t$ is the **candidate cell state**, which is computed based on the current input and the previous hidden state.
- $\odot$ denotes **element-wise multiplication**.

The crucial part is the connection from C_{t-1} to C_t **can preserve information without squashing it through a non-linear activation function**, thanks to **Constant Error Carousel (CEC)**.

Insight. The **Constant Error Carousel (CEC)** within the LSTM memory cell is the key mechanism that allows LSTMs to overcome the vanishing gradient problem. Created by Sepp Hochreiter in 1997, the CEC is designed to maintain a constant error signal as it propagates through time, enabling the network to learn long-term dependencies effectively. It achieves this by allowing the cell state to **carry information across many time steps without being altered by non-linear activation functions**, which are typically responsible for diminishing gradients in standard RNNs. It does so through a **linear self-loop** in the cell state update equation, which allows the gradient to be preserved during backpropagation. This means that the error signal can flow back through many time steps without vanishing, enabling the LSTM to learn from long-term dependencies in the data.

Architecture Comparison: RNN vs LSTM

A classic RNN cell handles input x_t and previous hidden state h_{t-1} to produce the new hidden state h_t:

$$h_t = \phi\left(W_x \cdot x_t + W_h \cdot h_{t-1} + b\right) \tag{110}$$

where ϕ is a non-linear activation function (e.g., tanh or ReLU). Here, only the hidden state h_t is maintained over time, and the non-linear activation can lead to vanishing gradients. Every update compresses information through ϕ and there is **no mechanism to protect memory**. As a result, both information and gradients **degrade** over time, and the network suffers from **vanishing gradients**.

The LSTM cell takes the same inputs as an RNN cell x_t and h_{t-1}, but **crucially** also maintains:

- A **cell state** C_t that carries long-term information.

- Three **gates** (input, forget, output) that regulate information flow.

Thus, the **LSTM outputs** both:

- The **cell state** C_t, which can carry information across many time steps without being squashed by non-linearities:

$$C_{t-1} \xrightarrow{\text{linear, gated}} C_t$$

  It indicates what information to **remember** or **forget** over time. It is a long-term memory and acts like a **protected memory highway** (Constant Error Carousel) for gradients.

- The **hidden state** h_t, which is similar to the classical RNN, but modulated by the output gate:

$$h_{t-1} \xrightarrow{\text{gated + nonlinearity}} h_t$$

  It indicates what information to **output** at the current time step. It is a short-term representation.

The gates in the LSTM cell **control the flow of information** into and out of both the cell state and hidden state. By regulating what to remember, forget, and output, the LSTM can effectively manage long-term dependencies without suffering from vanishing gradients. The **cell state** C_t **provides a direct path for gradients to flow back through time**, preserving information over long sequences, while the hidden state h_t allows for dynamic representation of the current input. This architecture enables LSTMs to learn complex temporal patterns that standard RNNs struggle with.

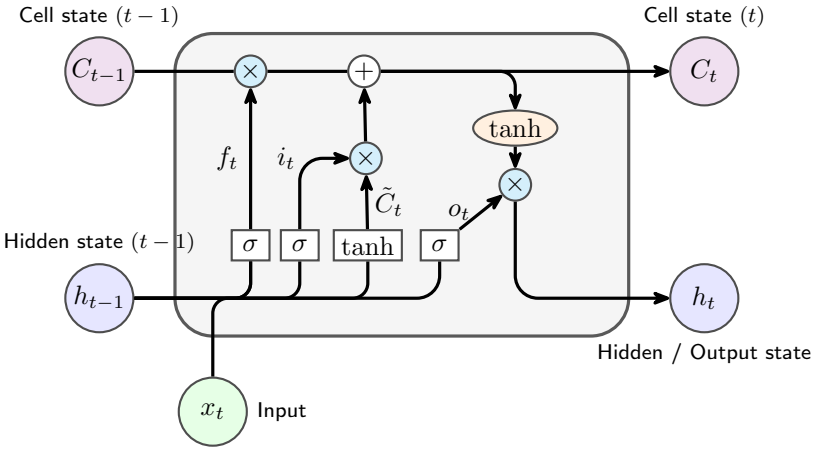


Figure 33: **LSTM cell**. It is the computation performed at a single time step t for one LSTM cell. At time t, the cell receives (1) the current input x_t, (2) the previous hidden state h_{t-1}, and (3) the previous cell state C_{t-1}. It computes the new cell state C_t and the new hidden state h_t.

The LSTM cell implements:

$$(x_t, h_{t-1}, C_{t-1}) \longmapsto (h_t, C_t)$$

The Figure 33, page 266, has three logical stages:

1. **Decide what to keep from the old memory**. The previous cell state C_{t-1} enters from the left along the top path. A gate called the **forget gate** produces f_t, and this is multiplied element-wise with C_{t-1}:

   $$f_t \odot C_{t-1}$$

   This determines how much of the previous cell state to retain. If a component of f_t is close to 1, the corresponding component of C_{t-1} is kept; if it is close to 0, it is forgotten.

2. **Decide what new information to write**. From the current input x_t and previous hidden state h_{t-1}, the cell computes:

   - The **input gate** i_t, which determines how much new information to add.
   - The **candidate cell state** $\tilde{C}_t$, which is a new potential memory content based on the current input and previous hidden state.

   Then they are combined as:

   $$i_t \odot \tilde{C}_t$$

   This determines how much new information should be written into the memory cell C_t. Where $\tilde{C}_t$ proposes new content, and i_t decides how much of it to actually add.

3. **Update memory and expose output**. The two contributions are added:

   $$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

   This gives the new cell state C_t. Then, the cell computes the hidden state as:

   $$h_t = o_t \odot \tanh(C_t)$$

   Where o_t is the **output gate** that decides how much of the current memory should be exposed as output. The hidden state h_t is what gets passed to the next time step and can also be used for making predictions at the current time step.

So the LSTM first updates memory, then decides how much of that memory should be revealed as output. In the next sections, we will go into the details of how each gate is computed and how the parameters are learned.

4.6.2 Gates

Unlike classical RNN that only updates one hidden state h_t, an LSTM maintains **two internal states**: the **cell state** (long-term memory) C_t and the **hidden state** (short-term output) h_t. The cell state C_t flows horizontally through time with minimal interference. This is the *memory highway* or **constant error carousel (CEC)**. But the LSTM must decide:

- What new information to write.
- What old information to forget.
- What part of memory to expose.

To do this, it uses four components called **gates**. Each gate produces a vector of values between 0 and 1 (sigmoid), controlling how information flows through the network. The interaction of these gates creates an extremely flexible state machine inside the LSTM cell. The three gates and one candidate memory are defined as follows:

$$h_t = o_t \odot \tanh(C_t) \quad \text{where} \quad C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

1. **Forget gate** f_t: Decides what information to discard from the cell state. "***Which information stored in the cell state is no longer relevant?***"

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right) \tag{111}$$

   - W_f: Weight matrix for the forget gate.
   - b_f: Bias vector for the forget gate.
   - σ: Sigmoid activation function.
   - $[h_{t-1}, x_t]$: Concatenation of previous hidden state and current input.

   This gate decides **how much of the previous cell state C_{t-1} should be kept**. It performs **selective deletion** of information from the long-term memory.

   ✔ $f_t \approx 1 \rightarrow$ Keep the information.

   ✖ $f_t \approx 0 \rightarrow$ Forget the information.

   ⚖ $f_t \approx 0.5 \rightarrow$ Partially forget the information.

   It is **needed** because **old context** sometimes becomes **irrelevant**. For example, in language modeling, once we move past a topic, we may want to forget its details: "*The capital of France is Paris.* ***By the way****, the capital of Germany is Berlin*". Here, the information about France can be forgotten when discussing Germany. It is needed to **prevent memory saturation** and **allow adaptation to new contexts**.

   ✔ **Vanishing Gradient.** The forget gate is deeply connected to the LSTM's trick for fighting the vanishing gradient problem. We define the

gradient flow through the cell state as:

$$\begin{aligned}\frac{\partial C_t}{\partial C_{t-1}} &= \frac{\partial}{\partial C_{t-1}}\left(f_t \odot C_{t-1} + i_t \odot \tilde{C}_t\right) \\ &= f_t \odot \frac{\partial C_{t-1}}{\partial C_{t-1}} + \frac{\partial (i_t \odot \tilde{C}_t)}{\partial C_{t-1}} \\ &= f_t + 0 \\ &= f_t\end{aligned}$$

This means that the gradient of the cell state at time t with respect to the previous cell state at time $t-1$ is exactly the forget gate f_t. If f_t is **close to 1**, the **gradient flows through unchanged**, preventing vanishing gradients. If f_t is **close to 0**, the **gradient is blocked**, allowing the network to forget irrelevant information. Thus, the forget gate **actively controls the gradient flows**, enabling the LSTM to maintain long-term dependencies while still being able to adapt and forget when necessary.

2. **Input gate** i_t: Decides what new information to add to the cell state. "***How much of the candidate memory*** $\tilde{C}_t$ ***should we store in the cell state?***"

$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] + b_i\right) \tag{112}$$

Controls **how much the candidate memory will influence the cell state**. It protects the memory from random updates.

- ✗ If $i_t = 0 \rightarrow$ No new information is added, ignoring $\tilde{C}_t$.
- ✓ If $i_t = 1 \rightarrow$ All new information is added, fully adopting $\tilde{C}_t$.

This gate allows the model to be selective and **only incorporate relevant new information** into the cell state. Without it, the cell would overwrite its memory at every timestep, which would be catastrophic for long-term dependencies. The input gate is therefore a **filter** that protects memory from being updated too easily.

3. **Candidate cell state** $\tilde{C}_t$: Creates new candidate values that could be added to the cell state. It is extremely important because it determines **what new information is *proposed* to enter the long-term memory**.

$$\tilde{C}_t = \tanh\left(W_C \cdot [h_{t-1}, x_t] + b_C\right) \tag{113}$$

This component generates **potential new information** to be added to the cell state, scaled by the input gate i_t. It is a *vector of new information* generated from the current input x_t and the previous short-term state h_{t-1}.

- It uses **tanh** to generate values between -1 and 1, **allowing both positive and negative updates** to the cell state. It also compresses information to a stable range and avoids uncontrolled growth.
- It is **modulated** later **by the input gate** i_t to determine how much of this candidate information should actually be written to the cell state.

We can think of $\tilde{C}_t$ as the **proposed update** to the cell state (long-term memory), which is then filtered by the input gate to ensure only relevant information is added. It works in tandem with the input gate:

- The candidate generates **what** to write.
- The input gate decides **how much** of that to actually write.

4. **Output gate** o_t: Decides what part of the cell state to output as the hidden state. "***Which part of this memory should we show as our output hidden state*** h_t***?***"

$$o_t = \sigma\left(W_o \cdot [h_{t-1}, x_t] + b_o\right) \tag{114}$$

This gate controls **what portion of the internal memory becomes visible in the hidden state** h_t.

- ✗ If $o_t = 0 \rightarrow$ No information from the cell state is exposed.
- ✓ If $o_t = 1 \rightarrow$ All information from the cell state is exposed.

This is important because not all information in the cell state is relevant for the current output. The output gate allows the LSTM to **selectively reveal** information based on the current context.

The **cell state update** C_t combines the effects of the forget and input gates:

$$C_t = \underbrace{f_t \odot C_{t-1}}_{\text{Forget old info}} + \underbrace{i_t \odot \tilde{C}_t}_{\text{Add new info}}$$

Here, the previous cell state C_{t-1} is scaled by the forget gate f_t, determining what to retain, while the candidate cell state $\tilde{C}_t$ is scaled by the input gate i_t, determining what new information to add. Finally, the **hidden state** h_t is computed by applying the output gate o_t to the activated cell state:

$$h_t = o_t \odot \tanh(C_t)$$

It is the **actual output of the LSTM cell** at time t, influenced by both the current cell state and the output gate.

- $\tanh(C_t)$ transforms the cell state into an output-friendly representation. This operator ensures bounded values (between -1 and 1), stability, good gradient behavior, and non-linearity.
- The output gate o_t filters that representation before exposing it. Allows the LSTM to **protect memory** while revealing only what is relevant

4.6.3 Lightweight Alternative: Gated Recurrent Unit (GRU)

Gated Recurrent Unit (GRU) were proposed by Cho et al. (2014) as a **simplified, faster, and lighter** recurrent architecture that retains much of the power of LSTMs but with fewer parameters and a simpler design.

Why GRU?

Long Short-Term Memory (LSTM) networks are powerful but can be computationally intensive due to their complex architecture involving multiple gates and memory cells:

- ✗ **Computationally heavy**, because of multiple matrix multiplications per time step.
- ✗ **Contain 3 gates + memory cell**, leading to more parameters to train.
- ✗ **Have many parameters (4 matrix multiplications per time step)**, which can lead to overfitting on smaller datasets.

GRUs address these issues by simplifying the architecture while still effectively capturing long-term dependencies in sequential data. Its goal is to:

- ✓ **Reduce computational complexity** by using fewer gates and parameters.
- ✓ **Maintain performance** comparable to LSTMs on many tasks.
- ✓ **Facilitate faster training** and inference times.
- ✓ **Simplify implementation** and tuning due to fewer hyperparameters.

They do this by merging some gates and eliminating the explicit memory cell found in LSTMs.

GRU Architecture

GRUs **do not have a separate cell state** C_t like LSTMs. Instead, they only maintain a **hidden state** h_t, which merges: short-term memory and long-term memory. GRUs use **two gates** to control the flow of information:

1. **Update Gate** z_t. Controls **how much of the past is kept** and **how much of the new information is added**.

$$\begin{bmatrix} i_t \\ f_t \end{bmatrix} \xrightarrow{\text{GRU}} \quad z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \tag{115}$$

   It **replaces both the input and forget gates** of the LSTM.

   - $z_t = 1$: use **only** new information, forget everything from the past (like forget gate closed, 0, and input gate open, 1).
   - $z_t = 0$: keep **only** past information, ignore new input (like forget gate open, 1, and input gate closed, 0).
   - $0 < z_t < 1$: balance between past and new information.

2. **Reset Gate** r_t. Controls **how much of the previous hidden state to consider when creating new candidate** information. This is similar to the input gate in LSTMs, but it directly influences the candidate hidden state. Similarly, it is computed as:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \tag{116}$$

   - r_t close to 0: ignore past hidden state, focus on new input.
   - r_t close to 1: consider past hidden state fully, influence of h_{t-1} is strong.

The number of gates is reduced from 3 in LSTMs to 2 in GRUs, **simplifying dramatically the architecture**. Also, GRUs do not have a separate memory cell; instead, they **maintain a single hidden state that combines both short-term and long-term memory** (h_t). This further reduces the number of parameters and computations required.

The **final hidden state** h_t in a GRU is computed as:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \tag{117}$$

It interpolates between the previous hidden state, h_{t-1}, and the candidate hidden state, $\tilde{h}_t$, weighted by the update gate, z_t. Thus, the **update gate** decides how much past information to retain and how much new information to incorporate. Notably, it **is the only gate that directly influences the final hidden state**. Instead, the **reset gate** r_t **operates when computing the candidate hidden state** $\tilde{h}_t$:

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h) \tag{118}$$

Feature	**LSTM**	**GRU**
Gates	3 (input, forget, output)	2 (update, reset)
Memory Cell	Yes (C_t)	No (only hidden state h_t)
Hidden State	Separate from cell state	Combines short-term and long-term memory
Complexity	Higher	Lower
Parameters	More (4 matrix multiplications per time step)	Fewer (3 matrix multiplications per time step)
Training Speed	Slower	Faster
Performance	Strong on long sequences	Usually comparable to an LSTM, sometimes better on smaller datasets

Table 5: Comparison between LSTM and GRU architectures.

Final thoughts: Is GRU always better than LSTM?

In short, **no**, GRU is not the best RNN. In some scenarios, LSTMs may outperform GRUs. They **do not have the same expressive power** of LSTMs.

- **LSTMs Separate Long-Term and Short-Term Memory**. LSTM has two dedicated components (**cell state** C_t and **hidden state** h_t) to manage long-term and short-term dependencies separately. Instead, GRU merges them into a single vector. The separation in LSTMs allows to:
  - Preserve long-term memory in C_t **without exposing it**.
  - Use h_t for short-term, noisy, rapidly-changing representations.
  - Control what gets exposed to the next layer (via the output gate).

  This separation is **very powerful**, especially for tasks requiring deep context management, such as language modeling or machine translation.

- **LSTMs Have More Control (More Degrees of Freedom)**. LSTM has **three gates** (forget f_t, input i_t, output o_t), while GRU has **two** (update z_t, reset r_t). This gives LSTMs: more expressiveness, finer control of memory flow, and the ability to decouple "writing to memory" from "exposing memory". In some tasks, this extra structure provides: better accuracy, more stable long-term learning, and better handling of irregular or sparse signals. In summary, **with LSTM we can do a more nuanced manipulation of information**.

- **LSTMs Work Better on Very Long Sequences**. Empirically, on tasks requiring **very long-term dependencies** (hundreds or thousands of time steps), **LSTMs typically outperform GRUs**. Some examples:
  - Long text generation.
  - Speech sequences longer than several seconds.
  - Language models (pre-Transformer era).
  - Medical time series with sparse temporal relations.
  - Irregular biomechanical signals.

  Because the cell state in an LSTM is specifically designed for long-term memory, LSTMs **tend to retain information better**.

  GRUs can forget too aggressively because the update gate merges the input and forget gates. This coupling makes GRUs less flexible in controlling what to remember for long periods.

- **GRUs Can Underperform When Selective Visibility Matters**. LSTMs have an **output gate**:

$$h_t = o_t \odot \tanh(C_t)$$

  GRUs **do not**. This means **GRUs cannot selectively hide memory and it is critical in tasks where certain features need to stay internal and not be exposed to the next layer**.

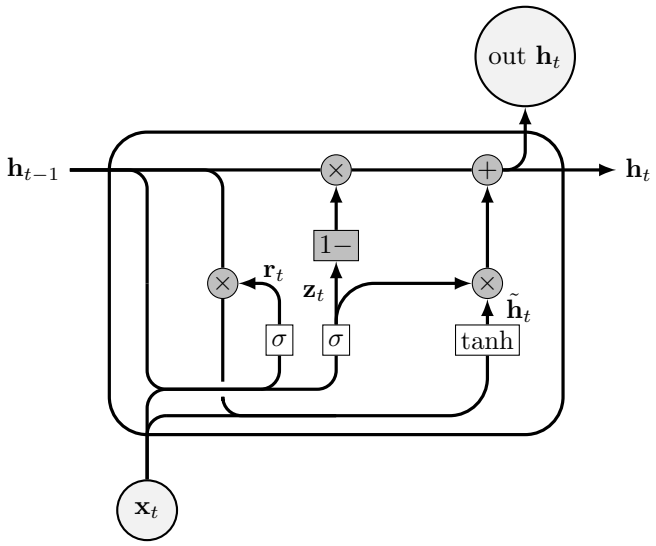


Figure 34: **Gated Recurrent Unit (GRU) cell.** [10] At time step t, the GRU receives the current input x_t and the previous hidden state h_{t-1}, and computes the new hidden state h_t, which serves both as output and as recurrent input to the next time step.

The **reset gate** $r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$ controls how much of the previous hidden state contributes to the candidate activation. When r_t is close to 0, past information is ignored; when it is close to 1, past information is fully considered.

The **candidate hidden state** is computed as

$$\tilde{h}_t = \tanh\big(W_h[r_t \odot h_{t-1}, x_t] + b_h\big),$$

where the reset gate filters the past before combining it with the current input.

The **update gate** $z_t = \sigma(W_z[h_{t-1}, x_t]+b_z)$ determines how much of the previous hidden state is retained and how much of the candidate is incorporated. The $1-$ term is a smart way to control at the same time the contribution of the past and the new candidate: $z = 1$ drops the past $(1 - z = 0)$ and fully incorporates the new candidate $(z = 1)$, while $z = 0$ retains the past $(1 - z = 1)$ and ignores the new candidate $(z = 0)$. The final hidden state is computed as

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t,$$

which can be interpreted as a **learned interpolation** between past and new information.

Unlike LSTMs, GRUs **do not maintain a separate cell state** C_t. Instead, the hidden state h_t directly combines both short-term and long-term information, resulting in a **simpler and more computationally efficient architecture**.

4.6.4 Networks

So far, we have described the internal structure of **one LSTM cell**. But real sequence tasks require using this cell **repeatedly, one timestep at a time**. Thus, an **LSTM network** is simply **a chain of identical LSTM cells, each processing one element of the sequence, sharing all parameters**. With "identical", we mean that all cells have the same weights, same biases, same gates, and same equations. Only the **inputs** x_t and **internal states** (h_t, C_t) differ from cell to cell, depending on the timestep t.

The LSTM **unrolled across timesteps**:

- $x_1, x_2, \ldots, x_T$ are the inputs at each timestep;
- At each timestep t, the LSTM cell receives **current input** x_t and **previous states** (h_{t-1}, C_{t-1});
- It computes **new hidden state** h_t, **new cell state** C_t using the same equations, weights, and biases at each timestep, and produces **output** y_t (if relevant).

The key idea is **all cells share parameters, but each has its own internal states as time progresses**. This allows the network to **learn temporal patterns** in the data, as information can flow through the cell states across many timesteps, while the shared parameters ensure consistency in how information is processed at each step.

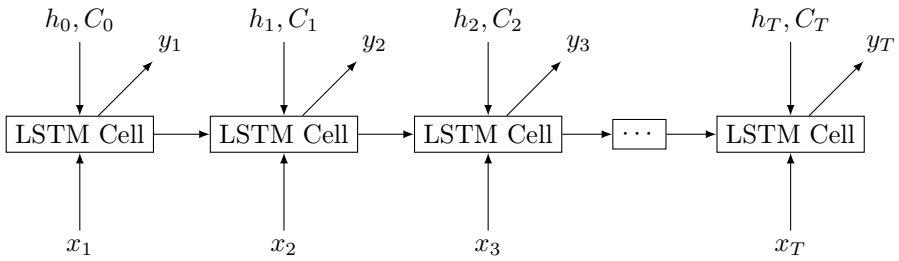


Figure 35: **LSTM network** (many-to-many architecture) **unrolled across timesteps**. Each LSTM cell processes one input at a time, sharing parameters but maintaining its own internal states. Outputs can be produced at each timestep if required.

What does the LSTM actually learn over time?

As it processes a sequence, the LSTM network **learns to update its cell state** C_t and **hidden state** h_t in a way that captures relevant information from the inputs over time. The LSTM network:

- ✔ **Accumulates information in the cell state**. This allows it to remember important features of the input sequence over long periods.
- ✔ **Selective memory updates**. Updates memory only when needed. The gates control when to forget old information and when to add new information, enabling the network to focus on relevant parts of the sequence.

- ✔ **Outputs predictions for each step**. The hidden state h_t can be used to make predictions at each timestep, allowing the network to generate outputs based on the learned temporal patterns.
- ✔ **Adapts context dynamically**. The LSTM can adjust its memory based on the context of the sequence, making it effective for tasks like language modeling, time series prediction, and more.

These capabilities allow LSTM networks to understand temporal relationships (e.g., sequences in text, speech, or time series data), dependencies across distant timesteps (e.g., long-term dependencies in sentences), evolving contexts (e.g., changing topics in a conversation), and patterns different lengths (e.g., variable-length sequences in natural language).

How LSTMs are used in real models

LSTMs can be arranged in different **architectural patterns** depending on the task. Each pattern corresponds to how inputs and outputs are handled across time. These patterns apply not only to LSTMs, but also to GRUs, Transformers, and any sequential model. However, LSTMs are particularly well-suited for these patterns due to their ability to manage long-term dependencies. The main patterns are:

1. **Many-to-Many (Sequence-to-Sequence Output). Output produced at *each* timestep**. This is the architecture shown on Figure 35; each LSTM cell processes a timestep and emits an output:

$$(x_1 \to h_1 \to y_1)\,, \quad (x_2 \to h_2 \to y_2)\,, \quad \ldots, \quad (x_T \to h_T \to y_T)$$

   #### When is this used?

   - **Part-of-Speech tagging**: Assigning a part of speech to each word in a sentence.
   - **Named Entity Recognition (NER)**: Identifying entities (like names, locations) in text.
   - **Frame-by-frame video classification**: Classifying each frame in a video sequence.
   - **Time-series forecasting**: Predicting future values at each time step.
   - **Speech recognition**: Transcribing spoken words into text in real-time.

   In this model, the input sequence and output sequence are of the same length, with each input element corresponding to an output element. For example, if the input is "John likes cats", the output could be "Noun Verb Noun" for part-of-speech tagging.

2. **Many-to-One (Sequence Classification). Use the *final hidden state* h_T to classify the entire sequence**. The architecture looks like this:

$$h_T = \text{LSTM}\,(x_1, x_2, \ldots, x_T)$$

$$y = \text{softmax}\left(Wh_T + b\right)$$

When is this used?

- **Sentiment analysis**: Classifying the sentiment of a movie review (positive/negative).
- **Sequence classification**: Classifying DNA sequences or protein sequences.
- **Spam detection**: Classifying emails or messages as spam or not spam.
- **Audio classification**: Classifying audio clips into categories (e.g., music genre).

Here, the entire input sequence is summarized into a single vector (the final hidden state), which is then used for classification. For example, given a movie review, the model predicts whether the sentiment is positive or negative based on the entire text: "The movie was fantastic!" $\rightarrow$ "Positive". So, the **model uses *all* the timesteps** but **only the *final* state matter for classification**.

3. **One-to-Many (Sequence Generation / Decoder)**. **Start from a single input and generate a sequence**. For example:

$$h_0 = f(x) \quad \text{(embedding of a single input)}$$

$$y_1, y_2, \ldots, y_T = \text{LSTM}\left(h_0\right)$$

This is typically autoregressive:

- At step 1, input h_0 produces output y_1;
- At step 2, input is y_1 (or its embedding), producing y_2;
- This continues until the full sequence is generated.

When is this used?

- **Text generation**: Generating sentences or paragraphs from a prompt.
- **Image captioning**: Generating a descriptive caption for an image.
- **Music generation**: Composing music sequences from a seed note or chord.
- **Auto-regressive generation**: Generating sequences in tasks like language modeling.

Here, the **model starts with a single input** (like a prompt or seed, a vector) and **generates a sequence of outputs step by step**. For example, given the prompt "Once upon a time", the model might generate a full story.

4. **Encoder-Decoder (Seq2Seq) Architecture**. **Use one LSTM (encoder) to process the input sequence into a context vector, then another LSTM (decoder) to generate the output sequence from that vector**. This pattern is still used in modern Transformers. It is a **two-part LSTM system**:

(a) **Encoder**: Processes the input sequence $(x_1, x_2, \ldots, x_T)$ and produces a context vector (final hidden state) h_T:

$$h_T = \text{LSTM}_{\text{enc}}(x_1, x_2, \ldots, x_T)$$

(b) **Decoder**: Takes the context vector h_T and generates the output sequence $(y_1, y_2, \ldots, y_K)$:

$$y_1, y_2, \ldots, y_K = \text{LSTM}_{\text{dec}}(h_T)$$

When is this used?

- **Machine translation**: Translating sentences from one language to another (e.g., English to French).
- **Dialogue systems**: Generating responses in chatbots.
- **Summarization**: Creating concise summaries of longer texts.
- **Speech-to-text**: Converting spoken language into written text.
- **Question answering**: Generating answers based on a given context or passage.
- **Autoencoders for sequences**: Learning compressed representations of sequences.

Here, the **input and output sequences can have different lengths**. For example, translating "I love programming" (3 words) to "J'adore la programmation" (4 words). The encoder captures the meaning of the input sequence, and the decoder generates the corresponding output sequence.

5. **Many-to-Many with Different Lengths (e.g., CTC, translation)**. **Input and output sequences have different lengths, with outputs produced at each timestep**. This is common in tasks like speech recognition or translation where the number of input timesteps does not match the number of output timesteps. The architecture looks like this:

$$y_1, y_2, \ldots, y_K = \text{LSTM}(x_1, x_2, \ldots, x_T)$$

When is this used?

- **Speech recognition**: Converting audio signals into text, where the number of audio frames may differ from the number of words.
- **Machine translation**: Translating sentences where the source and target languages have different word counts.
- **Handwriting recognition**: Recognizing handwritten text from images, where the number of strokes may differ from the number of characters.

Here, the model processes the entire input sequence and produces an output sequence that may be of different length. Techniques like Connectionist Temporal Classification (CTC) are often used to align the input and output sequences during training.

LSTMs are flexible enough to be used in almost every sequential modeling configuration. Their structure allows handling variable-length sequences, producing or consuming sequences, compressing long contexts, and dynamically generating outputs based on learned temporal patterns.

4.6.5 Multi-layer LSTM

Up until now, we have analyzed **single-layer LSTMs**, in which one LSTM cell processes the input sequence at each timestep. Figure 35 illustrates a many-to-many architecture with outputs at each timestep. This is an example of a single-layer LSTM network in which all LSTM cells share the same parameters across timesteps, but only one LSTM cell processes the input at each timestep.

However, modern deep learning models often use **multiple stacked recurrent layers**, forming a **multi-layer LSTM** architecture (or **deep LSTM**).

Why go deeper?

Just like in feedforward networks and CNNs, **depth increases representational capacity**.

✗ In a *single-layer LSTM*, the cell sees only the raw sequence input at each timestep. It must learn both low-level and high-level temporal patterns from this input. This makes **learning complex patterns more difficult**.

✓ In a *multi-layer LSTM*, lower layers can learn **low-level temporal features** from the raw input, while higher layers can focus on **high-level temporal abstractions**. So, each layer builds on top of the previous one:

- **Layer 1** learns low-level temporal features from the raw input sequence (e.g., edges in video frames, or phonemes in audio).
- **Layer 2** learns mid-level temporal patterns from Layer 1's features (e.g., shapes in video frames, or syllables in audio).
- **Layer 3** learns high-level temporal abstractions from Layer 2's patterns (e.g., objects in video frames, or words in audio).

Stacking layers **hierarchically organizes information**, just like depth does in CNNs.

How Multi-layer LSTMs work

In a deep LSTM, we have **multiple LSTM networks stacked one above the other**. For each timestep t:

- **Layer 1** receives the raw input $\mathbf{x}^{(t)}$ and processes it through its LSTM cell, producing hidden state $\mathbf{h}_1^{(t)}$ and cell state $\mathbf{c}_1^{(t)}$.
- **Layer 2** receives Layer 1's hidden state $\mathbf{h}_1^{(t)}$ as input, and processes it through its LSTM cell, producing hidden state $\mathbf{h}_2^{(t)}$ and cell state $\mathbf{c}_2^{(t)}$.
- **Layer 3** receives Layer 2's hidden state $\mathbf{h}_2^{(t)}$ as input, and processes it through its LSTM cell, producing hidden state $\mathbf{h}_3^{(t)}$ and cell state $\mathbf{c}_3^{(t)}$.
- This continues for however many layers the network has.

Each layer maintains its own recurrent connections over time, passing its hidden and cell states to the next timestep. The topmost layer produces the final output at each timestep. This architecture allows each layer to learn different levels of temporal features from the input sequence (**different abstraction levels**).

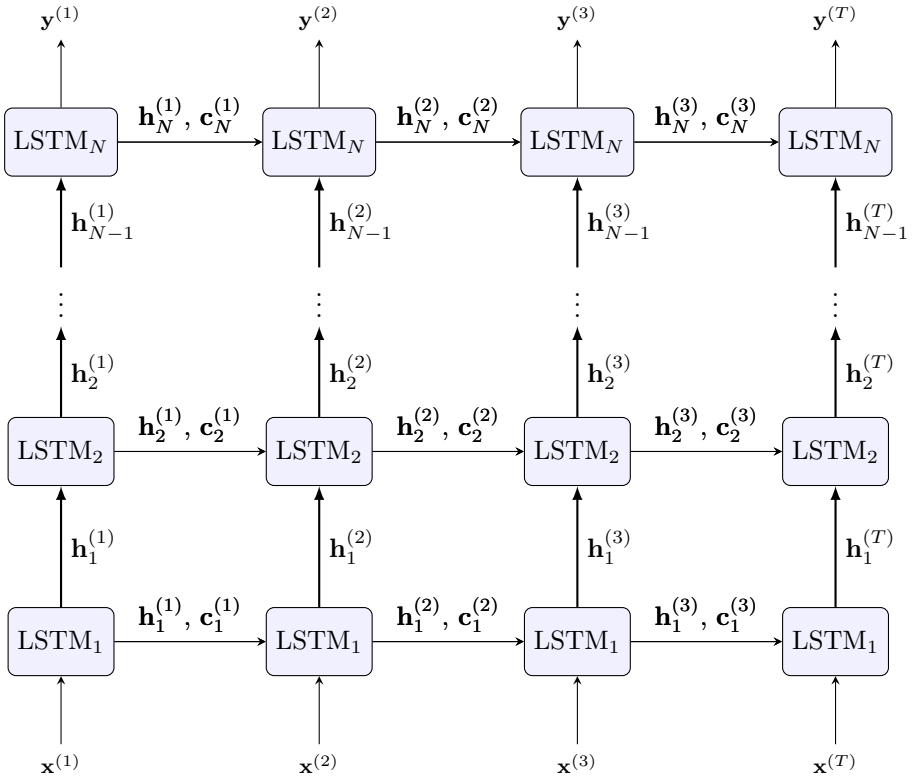


Figure 36: **Multi-layer LSTM network architecture unrolled over time**. Each layer processes the hidden state from the layer below at the same timestep, while maintaining its own recurrent connections over time. The bottom layer receives the raw input sequence, and the top layer produces the output sequence. Each layer has its own LSTM cell with separate parameters.

Formal Equations

We can derive the equations for a multi-layer LSTM by extending the single-layer LSTM equations. For a multi-layer LSTM with N layers, at each timestep t, the computations for layer ℓ ($1 \leq \ell \leq N$) are as follows:

- **Cell update** for layer ℓ:

$$C_t^{(\ell)} = f_t^{(\ell)} \odot C_{t-1}^{(\ell)} + i_t^{(\ell)} \odot \tilde{C}_t^{(\ell)}$$

- **Hidden state update** for layer ℓ:

$$h_t^{(\ell)} = o_t^{(\ell)} \odot \tanh(C_t^{(\ell)})$$

- **Input to layer** ℓ:
  - For layer 1 ($\ell = 1$):

$$h_t^{(0)} = x_t \quad \text{(raw input at timestep } t\text{)}$$

  - For layers $\ell = 2, \ldots, N$:

$$h_t^{(\ell-1)} \quad \text{(hidden state from previous layer at timestep } t\text{)}$$

- **Output from layer** N (topmost layer):

$$y_t = h_t^{(N)} \quad \text{(output at timestep } t\text{)}$$

Weights **are not shared** between layers but are shared across timesteps within each layer. This allows each layer to learn different temporal features at varying levels of abstraction.

⚠ Challenges of Deep LSTMs

While multi-layer LSTMs offer increased representational capacity, they also introduce challenges:

- ✗ **Vanishing/Exploding Gradients across layers**: Although LSTMs mitigate vanishing gradients over time, stacking many layers can still lead to gradient issues during backpropagation through layers.
- ✗ **Heavy Computational Load**: More layers mean more parameters and computations, leading to longer training times and higher resource consumption.
- ✗ **Overfitting Risk**: Deeper models are more prone to overfitting, especially with limited data.
- ✗ **Training Instability**: Deeper LSTMs can be harder to train effectively, requiring careful tuning of hyperparameters and initialization.

To address these challenges, techniques such as **residual connections**, **layer normalization**, and **dropout** are often employed in deep LSTM architectures.

4.6.6 Bidirectional LSTM (BiLSTM) Networks

A standard (unidirectional) LSTM processes a sequence **from left to right**, meaning that at each time step t, the LSTM has access only to the information from the previous time steps $1, 2, \ldots, t-1$:

$$x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow \ldots \rightarrow x_{T-1} \rightarrow x_T$$

This means the hidden state at time t is based only on: the past inputs $(x_1, \ldots, x_{t-1})$ and the current input (x_t). However, **in many tasks we *also* need to consider future context**, i.e., information from the subsequent time steps $t+1, t+2, \ldots, T$. This is where Bidirectional LSTM (BiLSTM) networks come into play.

Definition 8: Bidirectional LSTM (BiLSTM)

A **Bidirectional LSTM (BiLSTM)** is an **extension** of the traditional LSTM that can **capture information from both past and future contexts by processing the input sequence in both directions**. It consists of two LSTM layers:

- A *forward* LSTM that processes the input sequence from left to right (from x_1 to x_T).
- A *backward* LSTM that processes the input sequence from right to left (from x_T to x_1).

At each time step t, the outputs from both LSTMs are concatenated (or combined in some other way) to form the final output for that time step. This allows the network to have access to both past and future context when making predictions.

Example 8: BiLSTM Application

For example, in natural language processing tasks: "*I arrived at the bank to **deposit** money*". The word "*bank*" is ambiguous, because we need the *future* ("*to deposit money*") to understand that it refers to a financial institution rather than the side of a river. A BiLSTM can effectively capture this context by considering both preceding and succeeding words in the sentence.

Core Idea

The core idea behind BiLSTMs is to **leverage two separate LSTM networks to process the input sequence in both directions**, allowing the model to capture a more comprehensive understanding of the data by considering both past and future contexts simultaneously. It consists of two LSTM layers:

- **Forward LSTM**: Processes the input sequence from left to right, cap-

turing information from past time steps.

$$\overrightarrow{h}_t = \text{LSTM}_{\text{forward}}(x_t, \overrightarrow{h}_{t-1})$$

Here, $\overrightarrow{h}_t$ represents the hidden state at time step t from the forward LSTM, and $\overrightarrow{h}_{t-1}$ is the hidden state from the previous time step. This step learns representations from **past context**.

- **Backward LSTM**: Processes the input sequence from right to left, capturing information from future time steps.

$$\overleftarrow{h}_t = \text{LSTM}_{\text{backward}}(x_t, \overleftarrow{h}_{t+1})$$

Here, $\overleftarrow{h}_t$ represents the hidden state at time step t from the backward LSTM, and $\overleftarrow{h}_{t+1}$ is the hidden state from the next time step. This step learns representations from **future context**.

- **Combining Outputs**: At each time step t, the outputs from both the forward and backward LSTMs are combined (e.g., concatenated) to form the final output representation.

$$h_t = [\overrightarrow{h}_t; \overleftarrow{h}_t]$$

Here, h_t is the combined hidden state at time step t, which incorporates information from both past and future contexts.

⚠ When BiLSTM cannot be used

Bidirectional models **cannot be used when future inputs are unknown at the time of prediction** (inference time):

- **Real-time applications**: In scenarios where predictions must be made in real-time (e.g., live speech recognition, online translation), future data points are not available, making it impossible to utilize BiLSTMs effectively.
- **Online filtering**: In applications like online filtering or streaming data analysis, where data arrives sequentially and predictions must be made on-the-fly, BiLSTMs cannot be employed since they require access to future inputs.
- **Streaming sensor systems**: In systems that process data from sensors in real-time (e.g., IoT devices, autonomous vehicles), future sensor readings are not accessible at the moment of prediction, rendering BiLSTMs unsuitable.
- **Autoregressive generation**: In tasks such as text generation or time series forecasting, where each output depends on previously generated outputs, future inputs are inherently unknown during the generation process, preventing the use of BiLSTMs.

In these cases, **unidirectional LSTMs** or **GRUs** are typically employed, as they can make predictions based solely on past and current inputs without requiring future context.

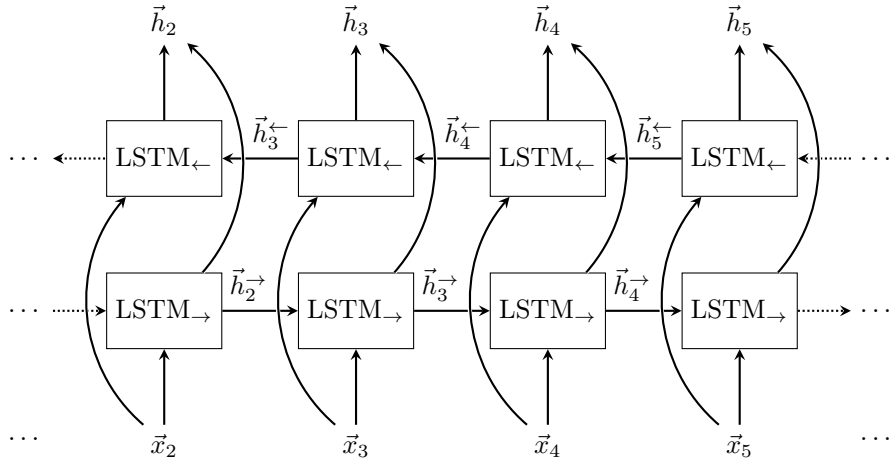


Figure 37: **Bidirectional LSTM (BiLSTM) architecture**. [15] The input sequence $\{x_t\}$ is processed by two independent LSTM chains operating in opposite directions.

The **forward LSTM** (bottom row) processes the sequence from left to right, producing hidden states $\overrightarrow{h}_t$ that summarize information from the *past* (i.e., from x_1 up to x_t). Each forward hidden state is computed recursively based on the current input and the previous hidden state, $\overrightarrow{h}_t = \text{LSTM}_{\text{forward}}(x_t, \overrightarrow{h}_{t-1})$.

In parallel, the **backward LSTM** (top row) processes the same sequence in reverse order, from right to left, producing hidden states $\overleftarrow{h}_t$ that capture information from the *future* (i.e., from x_T down to x_t). Each backward hidden state depends on the current input and the next hidden state in the sequence, $\overleftarrow{h}_t = \text{LSTM}_{\text{backward}}(x_t, \overleftarrow{h}_{t+1})$.

At each time step t, the input x_t is fed simultaneously to both directions, and the corresponding forward and backward hidden states are combined (e.g., via concatenation) to form the final representation $h_t = [\overrightarrow{h}_t; \overleftarrow{h}_t]$. This combined representation integrates both past and future context, allowing the model to make predictions based on the entire sequence.

Dashed connections indicate continuation of the sequence beyond the displayed time steps.

4.6.7 Practical Tips: Initialization & Conditioning

This section provides practical tips for initializing and conditioning Long Short-Term Memory (LSTM) networks to enhance their performance and stability during training. We will go through two key recommendations:

1. **Bidirectional conditioning (when and why to use BiLSTM)**
2. **Learnable initial states (why initializing h_0 and C_0 as parameters is better than zeros)**

Both points relate directly to improving performance and stability when training LSTM networks.

Bidirectional Networks: Condition on the Full Sequence

> **Practical Tip #1:** When the entire input sequence is available, a Bidirectional LSTM is usually superior because it conditions each timestep on both past and future information.

This may sound simple on the surface, but it actually encodes one of the most important design principles in sequence modeling: **context matters**.

In a **standard LSTM**, the hidden state at time t depends only on all previous inputs $\{x_1, x_2, \ldots, x_t\}$ and the current input x_t. But it knows *nothing* about future input x_{t+1}, future context or upcoming events. This creates an **information asymmetry** where our representation of x_t (i.e., h_t, the hidden state at time t) is only half-aware.

Why this is a problem: Suppose we want to classify a small window around time t. Patterns like anomalies, spikes, movement patterns often depend on: what *just happened* before and what *will happen* after. A unidirectional model only sees the previous points. Instead, a **Bidirectional LSTM (BiLSTM)** model sees *both sides* of the window. This often gives *significantly* better performance.

When to use BiLSTM: If our application allows us to see the entire sequence before making predictions (e.g., text classification, speech recognition, video analysis), then BiLSTM is usually the better choice. It leverages full context to create richer representations. **When not to use BiLSTM:** However, if we need to make real-time predictions (e.g., online forecasting, live translation), then we must use unidirectional LSTMs since future data is not available.

Learnable Initial States (Initialization as Parameters)

> **Practical Tip #2:** Instead of initializing the initial hidden state h_0 and cell state C_0 with fixed constants (like zeros), treat them as trainable parameters and learn them during training.

Every LSTM sequence starts with initial values:

- Initial **hidden state** h_0
- Initial **cell state** C_0

These two values define the **starting memory** of the LSTM **before it reads the first input** x_1. In most implementations, these are simply initialized to zeros:

$$h_0 = \mathbf{0}, \quad C_0 = \mathbf{0}$$

This is the *default* choice, but not the best.

Why zero initialization is not ideal: Zero initialization has several drawbacks:

- **Not realistic**. Real sequences rarely "start from zero memory". There is almost always some baseline state.
- **Every sequence starts from the same memory**. Even if our dataset contains very different types of sequences, they all begin with identical h_0 and C_0. This limits the model's ability to adapt to different contexts right from the start.
- **Slow convergence**. The model has to learn to build up useful memory from scratch for every sequence, which can slow down training.
- **Weak signal at early timesteps**. The initial outputs may be weak or uninformative since the model starts with no prior knowledge.
- **Doesn't encode dataset priors**. If certain patterns are common at the start of sequences, zero initialization fails to capture this prior knowledge.

The better idea: make h_0 and C_0 learnable parameters. Instead of fixing h_0 and C_0 to zeros, we can treat them as **trainable parameters**:

$$h_0 = \theta_h, \quad C_0 = \theta_C$$

Where θ_h and θ_C are **trainable vectors initialized randomly** (e.g., using Xavier or He initialization, page 186). They are optimized like any other weight via backpropagation. This means that the model learns the best starting hidden state and the best starting cell state for the entire training dataset. These states become part of the model parameters.

Benefits of learnable initial states:

- **Faster convergence**. The model starts with a better prior memory, leading to quicker learning since it doesn't have to build memory from scratch.
- **Better performance, especially with short sequences**. The model can adapt its initial memory to the dataset, improving accuracy. If sequences are short or medium length, initial state quality affects the entire trajectory more significantly.
- **Better early-time gradients**. The model produces more informative outputs from the start, enhancing gradient flow. A strong trained initial state provides clearer gradients and more stable backpropagation.
- **More expressive temporal priors**. The model can learn common starting patterns in the data. For example, if sequences often start with a certain trend or baseline, the learned initial states can encode this knowledge.
- **More robust to noise and variability**. The model can adjust its initial memory to handle different sequence types better, especially when early inputs are noisy or unreliable.

What about overfitting? Surprisingly, almost **no risk of overfitting** arises from learning initial states. Because **only 2 vectors** are added to the model (h_0 and C_0), and they are **extremely small compared to network weights**. Also, they capture global dataset-level priors, not sample-level details. Thus, they help generalization rather than harm it.

4.7 Sequential Data Problems

Up to now we mostly considered "classic" deep-learning problems where we have a **fixed-size input** and a **fixed-size output**, for example an image classification problem where the input is an image of fixed size (e.g., 224×224 pixels) and the output is a class label (e.g., *cat*, *dog*, etc.). However, many real-world problems involve **sequential data**, where the input and/or output can vary in length. When dealing with sequential data, we often encounter the following types of problems:

One-to-One Problems. It is the **classic** machine learning setup where we have **one input**, and we want to predict **one output**. There is **no sequence** involved on either side.

It is the standard supervised learning scenario: $x \rightarrow y$. Where x is a fixed-size input (e.g., an image, a feature vector, an audio clip, etc.) and y is a fixed-size output (e.g., a class label, a regression value, etc.). In this setting, the input has **no temporal dimension**, the output has **no temporal dimension**, and the mapping from input to output is **static** (i.e., it does not change over time).

What architecture is used in One-to-One problems? Any model that takes fixed-size input and produces fixed-size output can be used, such as **Feed-Forward Neural Networks (FFNs)**, **Convolutional Neural Networks (CNNs)**, etc.

Architectures that are not used? **Recurrent Neural Networks (RNNs)** and their variants (e.g., LSTMs, GRUs) are generally not used in One-to-One problems, as they are designed to handle sequential data (i.e., data with temporal dependencies).

One-to-Many Problems. In this type of problem, the model receives **one fixed-size input** x and produces a **sequence** as output: $y_1, y_2, \ldots, y_T$. The length of the output sequence may be fixed or variable (stopped by an end-of-sequence token). However, the crucial point is that **the input is static; the output is a sequence generated step-by-step**. This is the first case in which RNNs/LSTMs/GRUs become necessary. Some examples of One-to-Many problems include:

- **Image Captioning**: Given an image, generate a descriptive caption.
- **Music Generation**: Given a seed note or chord, generate a sequence of musical notes.
- **Text Generation**: Given a prompt, generate a sequence of words or sentences.

Why RNN/LSTM is needed here? Because the output must be generated *sequentially*:

- The first output depends on the image.
- The next output depends on:
  - The image representation.
  - The previous output word.

* The internal state of the RNN/LSTM/GRU.

Only RNNs/LSTMs (or later Transformer) can model this dynamic, autoregressive structure. A feedforward network cannot generate a *sequence* of arbitrary length based on a single input.

Example 9: Image Captioning

Image Captioning is a classic example of a One-to-Many problem. In this task, the model takes an image as input and generates a descriptive caption as output. The input is a fixed-size image (e.g., 224×224 pixels), while the output is a sequence of words forming a sentence (e.g., "A dog playing with a ball in the park"). The length of the caption can vary depending on the content of the image.
More specifically, an image is processed once (e.g., by a CNN) to obtain a feature vector. Then an RNN/LSTM/GRU **generates words one at a time**.

1. A CNN encodes the image into a feature vector.
2. The feature vector is fed to an LSTM as the initial input/hidden state.
3. At each timestep, the LSTM outputs **one word**.
4. The sequence continues until an End-Of-Sentence (EOS) token is generated.

Formally, if x is the input image and y_t is the word generated at timestep t, the model learns to predict:

$$P\left(y_1, y_2, \ldots, y_T \mid x\right) = \prod_{t=1}^{T} P\left(y_t \mid y_1, y_2, \ldots, y_{t-1}, x\right)$$

Where T is the length of the generated caption, which can vary for different images.

Many-to-One Problems. In this type of problem, the model receives a **sequence** as input: $x_1, x_2, \ldots, x_T$, and produces a **single fixed-size output** y. The length of the input sequence may be fixed or variable. However, the crucial point is that **the input is a sequence; the output is static**. This means the model must "summarize" the entire sequence into **one decision or prediction**.

How is this solved using RNNs/LSTMs/GRUs? The RNN/LSTM/GRU reads the input sequence one timestep at a time:

$$h_t = \text{LSTM}\left(x_t, h_{t-1}\right)$$

At the **final timestep** T, the hidden state h_T contains a representation of the whole sequence:

$$h_T \approx \text{summary of } \left(x_1, x_2, \ldots, x_T\right)$$

Then the model passes it to a classifier (e.g., a fully connected layer followed by softmax) to produce the final output y:

$$\hat{y} = g(h_T) \quad \text{linear} \rightarrow \text{softmax or sigmoid}$$

This output is the predicted label or value for the entire input sequence.

When does many-to-one occur in practice? Besides sentiment analysis, common examples include:

- **Sequence classification**: classify a time-series (e.g., accelerometer data) into categories (e.g., walking, running, sitting); classify an audio clip into genres (e.g., rock, jazz, classical); identify speaker emotions from speech (e.g., happy, sad, angry); human activity recognition from wearable sensor data (e.g., walking, running, sitting); fault detection in sensor streams (e.g., normal vs. faulty operation).
- **Video-level classification**: classify an entire video into categories (e.g., action recognition, scene classification).
- **Anomaly detection**: detect anomalies in sequences (e.g., network traffic, sensor readings).

Many-to-Many Problems. In this type of problem, the model receives a **sequence** as input: $x_1, x_2, \ldots, x_T$; and produces a **sequence** as output: $y_1, y_2, \ldots, y_{T'}$. The lengths of the input and output sequences may be fixed or variable. This is the most general case, where both the input and output are sequences. Some examples of Many-to-Many problems include:

- **Machine Translation**: Given a sentence in one language, translate it into another language.
- **Video Captioning**: Given a video, generate a descriptive caption.
- **Speech Recognition**: Given an audio clip, transcribe it into text.

Why isn't a simple RNN enough? If we tried to output one word for each input word (i.e., $T' = T$), we would have a **Many-to-Many with aligned sequences** problem. However, in many real-world applications, the output sequence length T' may differ from the input sequence length T (e.g., translating a short sentence into a longer one). Therefore, **the model must first understand the entire input sequence, then generate a new sequence**. This motivates the **encoder-decoder architecture**.

How does the Encoder-Decoder architecture work?

The architecture works in two main stages:

1. **Encoder**
   - Reads the entire input sequence $x_1, x_2, \ldots, x_T$.
   - Updates its hidden state at each timestep:

$$h_t = \text{LSTM}(x_t, h_{t-1})$$

- At the end, produces a **fixed-dimensional representation** (context vector) of the entire input sequence:

$$h_T \approx \text{summary of } (x_1, x_2, \ldots, x_T)$$

2. **Decoder**
   - Takes the context vector from the encoder as input.
   - Generates the output sequence **one element at a time**:

$$y_1, y_2, \ldots, y_{T'}$$

   - At each timestep, the decoder predicts the next output element based on:
     * The context vector (encoder output).
     * The previously generated outputs.
     * Its own internal state.

Many-to-Many Problems (with aligned sequences). In this variant of the Many-to-Many problem, the input and output sequences are of the **same length** ($T' = T$), and there is a direct alignment between each input element and its corresponding output element. This means that for each input x_t, there is a corresponding output y_t.

In this settings:

- **Input** is a sequence: $x_1, x_2, \ldots, x_T$.
- **Output** is also a sequence: $y_1, y_2, \ldots, y_T$.
- There is a **direct correspondence** between each input element and its output element: $x_t \to y_t$ for all $t = 1, 2, \ldots, T$.

So same number of timesteps, same temporal grid, and one output **for each input element**. Some examples of Many-to-Many problems with aligned sequences include:

- **Part-of-Speech Tagging**: Given a sentence, assign a part-of-speech tag to each word.
- **Named Entity Recognition**: Given a sentence, identify and classify named entities (e.g., persons, organizations, locations).
- **Video Frame Labeling**: Given a video, label each frame with an action or object.

How is it modeled? A Recurrent Neural Network (RNN)/LSTM/GRU processes the input sequence one timestep at a time:

$$h_t = \text{LSTM}\,(x_t, h_{t-1})$$

Then, at **every timestep** t, the model produces an output y_t based on the current hidden state h_t:

$$\hat{y}_t = g\,(h_t) \quad \text{linear } \to \text{ softmax or sigmoid}$$

So the model generates outputs:

$$\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_T$$

Often called **Sequence Labeling** or **Sequence Tagging**, since each input element is tagged with a corresponding output label.

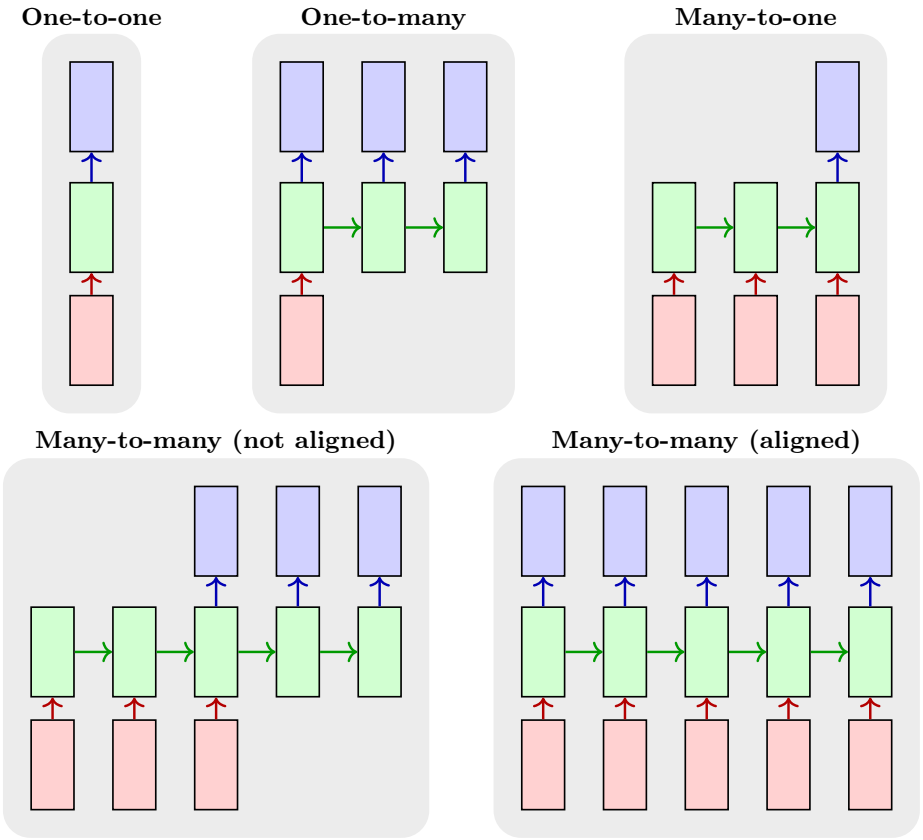


Figure 38: Different types of Sequential Data Problems. Each panel illustrates a different case, showing the structure of the input and output sequences, as well as the flow of information through the model. The red arrows indicate the input sequence, the green arrows indicate the recurrent connections within the model, and the blue arrows indicate the output sequence.

Case	Input	Output	Alignment
One-to-One	Fixed-size	Fixed-size	N/A
One-to-Many	Fixed-size	Sequence	No
Many-to-One	Sequence	Fixed-size	No
Many-to-Many	Sequence	Sequence	No
Many-to-Many (aligned)	Sequence	Sequence	Yes

Table 6: Summary of Sequential Data Problems. "Fixed-size" refers to inputs or outputs that do not vary in length, while "Sequence" indicates variable-length data. The "Alignment" column specifies whether there is a direct correspondence between elements of the input and output sequences.

4.8 Sequence-to-Sequence Learning

After introducing the main categories of sequential problems, we now focus on a particularly important family: **Sequence-to-Sequence Learning** (often abbreviated as **Seq2Seq**). In this setting, the goal is to learn a mapping from an **input sequence** to an **output sequence**. Unlike standard classification, where the output is a single label, here the model must produce a structured sequence of predictions, one element at a time.

This is a natural framework for many real-world applications where both the input and the output carry sequential structure. Some examples are:

- **Image Captioning**: the input is a single image, and the output is a variable-length sentence describing it.
- **Sentiment Analysis**: the input is a variable-length sentence or tweet, and the output is a single label such as positive or negative.
- **Machine Translation**: the input is a sentence in one language, and the output is a sentence in another language.

Although these examples do not all have the same input/output shape, they share the same core principle: the model must learn to generate outputs **conditioned on the input** and, when the output is sequential (i.e., a sentence or a sequence of labels), it must do so **autoregressively**, i.e. one token at a time (predicting the next token based on the previous ones).

Main Idea. In sequence-to-sequence learning, we do not want to predict just one target value, but an entire target sequence:

$$y_1, y_2, \ldots, y_n$$

Given an input sequence:

$$x_1, x_2, \ldots, x_m$$

Where, in general, the two lengths *can* be different:

$$m \neq n \quad \text{but not necessarily}$$

This is what makes Seq2Seq models more general than aligned many-to-many models, where the output is forced to follow the same temporal structure as the input.

Although several types of sequential problems exist, sequence-to-sequence learning is studied in greater detail because it is the most general and expressive setting. It captures the core difficulties of sequence modeling: handling variable-length structured inputs, generating variable-length outputs autoregressively, and learning a conditional distribution $P(y \mid x)$. Simpler sequential tasks, such as many-to-one classification or aligned sequence labeling, can often be interpreted as restricted special cases of this broader framework.

4.8.1 Probabilistic Formulation

Suppose we are given an input sequence:

$$x = (x_1, x_2, \ldots, x_m)$$

And we want to produce an output sequence:

$$y = (y_1, y_2, \ldots, y_n)$$

The goal of sequence-to-sequence learning is to create a model that can predict an output sequence based on an input sequence, even when the two sequences have different lengths (i.e., $m \neq n$).

Naïve Approach: predict the whole sequence at once

A naïve approach would be to treat the entire output sequence as a single target and try to predict it in one shot. However, this is not practical because the output is a **structured object** (a sequence), not a single label. The number of possible sequences grows exponentially with their length, and many different sequences may be valid outputs for the same input (e.g., multiple correct translations of a sentence).

For this reason, instead of predicting a single output directly, we aim to model how *likely* a candidate output sequence is given the input.

Formally, the model learns a **conditional probability distribution**:

$$P(y \mid x) \tag{119}$$

That is:

$$P(y_1, y_2, \ldots, y_n \mid x_1, x_2, \ldots, x_m)$$

Intuitively, this measures how plausible a given output sequence is for a specific input sequence. For example, if x is an English sentence and y is a French sentence, then $P(y \mid x)$ reflects how likely y is as a translation of x.

Thus, Seq2Seq learning is not a deterministic mapping from input to output, but a **probabilistic modeling problem**.

Prediction rule. Once the model has learned this distribution, the predicted output is the sequence with the highest probability:

$$y^* = \arg\max_y P(y \mid x) \tag{120}$$

That is, among all possible output sequences, we select the most likely one given the input.

What is $P(y \mid x)$ really?

Forget neural networks for a second. The expression $P(y \mid x)$ means: "*in the **real world**, how likely is output y given input x?*". For example, if x is "*I am tired*", the possible outputs y could be "*Je suis fatigué*" (which means "I am tired" in French) or "*Je mange une pomme*" (which means "I eat an apple" in French). The first output is much more likely than the second one, so $P(y \mid x)$ should assign a higher probability to the first translation.

This simple example shows that the quantity $P(y \mid x)$ represents a **true underlying distribution** between inputs and outputs in the real world. We can call this distribution $P_{\text{true}}(y \mid x)$, and it is determined by the data-generating process (e.g., natural language).

In practice, it is impossible to know $P_{\text{true}}(y \mid x)$ exactly because we only have access to a finite dataset of examples and can only observe a limited number of input-output pairs.[6] So we only observe a finite set of input-output pairs:

$$\left(x^{(1)}, y^{(1)}\right), \left(x^{(2)}, y^{(2)}\right), \ldots, \left(x^{(N)}, y^{(N)}\right) \sim P_{\text{true}}(x, y)$$

Therefore, we must learn an approximation of $P_{\text{true}}(y \mid x)$ based on the observed data.

To this end, we introduce a parametric model:

$$P(y \mid x; \theta)$$

Where θ denotes the trainable parameters of the model (e.g., the weights of a neural network). This defines a **family of possible distributions**, and different values of θ correspond to different models.

The goal of training is to find parameters θ such that:

$$P(y \mid x; \theta) \approx P_{\text{true}}(y \mid x)$$

That is, the model assigns high probability to correct output sequences and low probability to incorrect ones.

[6] The reason why we observe only a limited number of input-output pairs is that the true distribution $P_{\text{true}}(y \mid x)$ is defined over a very large (often effectively infinite) space of possible inputs and outputs. In practice, it is impossible to observe all possible combinations. Instead, we only have access to a finite dataset of samples drawn from this distribution.

More formally, we observe pairs:

$$\left(x^{(1)}, y^{(1)}\right), \left(x^{(2)}, y^{(2)}\right), \ldots, \left(x^{(N)}, y^{(N)}\right)$$

Which can be seen as samples from the joint distribution $P_{\text{true}}(x, y)$.

For example, in natural language, the number of possible sentences is extremely large, and we can only observe a very small subset of them in our dataset.

4.8.2 Conditional Language Models

In the previous subsection, we defined the goal of sequence-to-sequence learning as modeling the conditional probability:

$$P(y_1, y_2, \ldots, y_n \mid x_1, x_2, \ldots, x_m)$$

However, directly **modeling this probability is intractable**. The output sequence is a structured object whose number of possible configurations grows exponentially with its length. For a vocabulary (i.e., the set of all possible tokens) of size $|\mathcal{V}|$, there are $|\mathcal{V}|^n$ possible sequences of length n. Therefore, assigning a probability to each possible sequence as a whole is **computationally infeasible**.

To make this problem manageable, we use the **chain rule of probability** to decompose the joint probability of the sequence into a product of conditional probabilities:

$$P(y_1, y_2, \ldots, y_n \mid x) = \prod_{t=1}^{n} P(y_t \mid y_1, y_2, \ldots, y_{t-1}, x) \tag{121}$$

This formulation is known as a **Conditional Language Model**, since it extends the idea of a standard language model by conditioning on an external input x.

Definition 9: Language Model

A **Language Model (LM)** is a probabilistic model that assigns a probability to a sequence of tokens:

$$y = (y_1, y_2, \ldots, y_n)$$

By modeling the joint distribution:

$$P(y_1, y_2, \ldots, y_n)$$

Using the chain rule of probability, this distribution is factorized as:

$$P(y_1, y_2, \ldots, y_n) = \prod_{t=1}^{n} P(y_t \mid y_1, y_2, \ldots, y_{t-1}) \tag{122}$$

Thus, a language model predicts each token y_t based on the sequence of previous tokens:

$$y_1 \to y_2 \to \cdots \to y_n$$

In simple terms, a language model learns to predict the next word in a sequence given the words that came before it. For example, given the input sequence "*The cat is on the*", a language model might predict that the next word is likely to be "*mat*" or "*roof*" based on the context provided by the previous words. It is important to note that a language

model does not have access to any external input; it only relies on the sequence of tokens it has seen so far. Furthermore, a **language model does not understand the meaning of the words**; it simply learns statistical patterns in the data to make predictions. For instance, it might learn that the word "*cat*" is often followed by "*is*" and then by "*on*" in many sentences, without any understanding of what a cat is or what being on something means.

What is a token?

A **Token** is a basic unit of text that a language model processes. It can be a word, a subword, or even a character, depending on the tokenization scheme used. More generally, a token represents the basic unit generated by the model at each timestep. For example, in the sentence "*The cat is on the mat*", the tokens could be:

- Words: "The", "cat", "is", "on", "the", "mat".
- Subwords: "Th", "e", "cat", "is", "on", "the", "mat".
- Characters: "T", "h", "e", " ", "c", "a", "t", " ", "i", "s", " ", "o", "n", " ", "t", "h", "e", " ", "m", "a", "t".

The choice of tokenization can affect the performance of the language model, as it determines the granularity of the input and output sequences. For example, using subword tokenization can help the model handle rare words and morphological variations more effectively than word-level tokenization.

Definition 10: Conditional Language Model

A **language model** assigns a probability to a sequence of tokens:

$$P(y_1, y_2, \ldots, y_n) = \prod_{t=1}^{n} P(y_t \mid y_1, \ldots, y_{t-1})$$

That is, each token is predicted based on the previous ones.

A **Conditional Language Model** extends this formulation by conditioning on an additional input x:

$$P(y_1, y_2, \ldots, y_n \mid x) = \prod_{t=1}^{n} P(y_t \mid y_1, \ldots, y_{t-1}, x) \tag{123}$$

Thus, each token is generated based on both:

- The previously generated tokens;
- The external input x.

In other words, a conditional language model **learns to generate a sequence of tokens that is not only coherent with itself but also relevant to the given input** x. For example, if x is an English sentence

and y is its French translation, the model learns to generate a French sentence that is both grammatically correct and semantically related to the English input. This makes conditional language models particularly powerful for tasks like machine translation, where the output must be closely tied to the input.

Key idea. The output sequence is generated **one element at a time**. At each timestep t, the model predicts the next token y_t based on:

- The input sequence x;
- The previously generated outputs $y_1, \ldots, y_{t-1}$.

This transforms sequence generation into a step-by-step process, where the model maintains a memory of the past through its hidden state and uses it to inform future predictions.

Why is this important? This factorization makes the problem tractable: instead of predicting an entire sequence at once, the model learns to predict one token at a time, while maintaining a memory of the past through its hidden state.

Remark. For simplicity, the dependence on the model parameters θ is often omitted in the notation. However, all probabilities should be understood as being parameterized, i.e., $P(y_t \mid y_{<t}, x) = P(y_t \mid y_{<t}, x; \theta)$.

4.8.3 Encoder-Decoder Framework

In the previous subsection, we expressed the conditional probability of an output sequence as:

$$P(y_1, y_2, \ldots, y_n \mid x) = \prod_{t=1}^{n} P(y_t \mid y_1, \ldots, y_{t-1}, x)$$

This formulation requires a model that can:

1. Process the input sequence x;
2. Keep track of the previously generated outputs $y_1, \ldots, y_{t-1}$;
3. Predict the next token y_t at each timestep.

To achieve this, we use the **Encoder-Decoder Framework**, which decomposes the problem into two components (*divide and conquer*):

1. An **encoder**, which processes the input sequence x and produces a fixed-dimensional representation (the context vector).
2. A **decoder**, which generates the output sequence y one token at a time, conditioned on the context vector and the previously generated tokens.

Encoder

The **Encoder** is a Recurrent Neural Network (e.g., RNN, LSTM, or GRU) that **reads the input sequence one element at a time**:

$$h_t^{\text{enc}} = \text{RNN}_{\text{enc}}\left(x_t, h_{t-1}^{\text{enc}}\right) \tag{124}$$

At the final timestep m, the **hidden state contains a representation of the entire input sequence**:

$$c = h_m^{\text{enc}} \tag{125}$$

This vector c is called the **Context Vector**, and it **summarizes the input sequence**.

Decoder

The **Decoder** is another Recurrent Neural Network that **generates the output sequence one token at a time**. It is initialized using the *context vector* c:

$$s_0 = c$$

At each timestep t, the decoder updates its hidden state as:

$$s_t = \text{RNN}_{\text{dec}}\left(y_{t-1}, s_{t-1}\right) \tag{126}$$

And produces a probability distribution over the next token:

$$P(y_t \mid y_{<t}, x)$$

Where $y_{<t} = (y_1, \ldots, y_{t-1})$ are the previously generated tokens.

Where is x in the decoder? The input sequence x is encoded into the context vector c, which is used to initialize the decoder's hidden state. Thus, the decoder has access to the information from x through its initial state and can use it to generate the output sequence.

Key idea. The encoder transforms the input sequence into a fixed-dimensional representation, and the decoder uses this representation to generate the output sequence step by step.

Why is this necessary? This architecture allows us to handle:

- Variable-length input sequences;
- Variable-length output sequences;
- Different input and output lengths ($m \neq n$).

Why not just use a single RNN? A single RNN would struggle to capture the complex dependencies between the input and output sequences, especially when they have different lengths. The encoder-decoder framework provides a structured way to model these dependencies by separating the processing of the input and output sequences.

Limitation of the basic encoder-decoder

The basic encoder-decoder architecture can **struggle with long input sequences**, as the **context vector c may not be able to capture all the necessary information**. This is reasonable, since c is a fixed-dimensional vector that must summarize the entire input sequence, which can be very long and complex. This problem is known as the **Bottleneck Problem**. To address this issue, more advanced architectures such as **attention mechanisms** have been developed, which allow the decoder to access different parts of the input sequence directly, rather than relying solely on a fixed context vector.

Definition 11: Bottleneck Problem

The **Bottleneck Problem** arises because the **encoder compresses the entire input sequence into a single fixed-size context vector**, which may not be sufficient to retain all relevant information, especially for long sequences.

4.8.4 Training Sequence-to-Sequence Models

After defining the encoder-decoder architecture, we now have a parametric model that can compute the conditional probability:

$$P\left(y \mid x; \theta\right)$$

For any input-output pair (x, y), we can evaluate how likely the model thinks y is given x. The remaining question is *how to choose the parameters θ so that the model produces correct output sequences.* This leads to a **learning problem**: given a dataset of input-output pairs, we must find parameters θ so that the model produces correct output sequences.

Formally, given a training dataset of input-output pairs:

$$\left(x^{(1)}, y^{(1)}\right), \left(x^{(2)}, y^{(2)}\right), \ldots, \left(x^{(N)}, y^{(N)}\right)$$

The goal is to learn the model parameters θ such that the conditional probability:

$$P\left(y \mid x; \theta\right)$$

Assigns high probability to the correct output sequences.

Maximum Likelihood Training

Training is typically performed by maximizing the likelihood of the observed data:

$$\theta^* = \arg\max_{\theta} \prod_{i=1}^{N} P\left(y^{(i)} \mid x^{(i)}; \theta\right)$$

Equivalently, we maximize the log-likelihood:

$$\theta^* = \arg\max_{\theta} \sum_{i=1}^{N} \log P\left(y^{(i)} \mid x^{(i)}; \theta\right)$$

Using the factorization introduced earlier (Equation 123, page 297), the log-probability of a sequence can be written as:

$$\log P\left(y \mid x; \theta\right) = \sum_{t=1}^{n} \log P\left(y_t \mid y_{<t}, x; \theta\right)$$

Thus, **training reduces to maximizing the probability of each token in the sequence at every timestep**.

Categorical Cross-Entropy (Cross-Entropy Loss)

In practice, maximizing the log-likelihood is **implemented as minimizing a loss function**. So at each timestep t, the model predicts a probability distribution over the vocabulary, and the loss penalizes the model if it assigns low probability to the correct token y_t. The most common loss function for this is the **Categorical Cross-Entropy**.

Definition 12: Categorical Cross-Entropy

The **Categorical Cross-Entropy (CCE)** is a **loss function** (page 90) used in multi-class classification problems, where the **model predicts a probability distribution over a set of mutually exclusive classes** (i.e., exactly one class is correct).

Given:

- A true distribution $y = (y_1, \ldots, y_K)$, typically a one-hot vector (i.e., $y_i = 1$ for the correct class and $y_j = 0$ for all $j \neq i$);
- A predicted distribution $\hat{y} = (\hat{y}_1, \ldots, \hat{y}_K)$, produced by a softmax;

The categorical cross-entropy is defined as:

$$\mathcal{L} = -\sum_{i=1}^{K} y_i \log \hat{y}_i \tag{127}$$

That is, it measures the dissimilarity between the true distribution y and the predicted distribution $\hat{y}$.

Since y is **one-hot**, this simplifies to:

$$\mathcal{L} = -\log \hat{y}_{\text{true}}$$

Where $\hat{y}_{\text{true}}$ is the predicted probability assigned to the correct class.

Intuition. The loss measures how much probability the model assigns to the correct class:

- If the model assigns **high probability** to the correct class, the **loss** is **small**;
- If the model assigns **low probability**, the **loss** becomes **large**.

Thus, minimizing the categorical cross-entropy encourages the model to assign high probability to the correct class and low probability to the others.

Why Categorical Cross-Entropy? Maximizing the likelihood of the data is equivalent to minimizing the negative log-likelihood. The cross-entropy loss implements exactly this objective: it penalizes the model when it assigns low probability to the correct token. Therefore, minimizing cross-entropy corresponds to making the model assign high probability to the observed data.

Formally, at each timestep t, the model predicts:

$$P\left(y_t \mid y_{<t}, x; \theta\right)$$

A **probability distribution over the vocabulary**. Then we compare it with

the true token y_t. So the loss at timestep t is:

$$\mathcal{L}_t = -\log P\left(y_t \mid y_{<t}, x; \theta\right)$$

The **total loss for the sequence is the sum of the losses at each timestep**:

$$\mathcal{L} = \sum_{t=1}^{n} \mathcal{L}_t = -\sum_{t=1}^{n} \log P\left(y_t \mid y_{<t}, x; \theta\right)$$

So each token contributes one classification loss, and the model is trained to minimize the total loss across the entire sequence.

What do we feed as $y_{<t}$ during training?

In Seq2Seq models, the decoder generates tokens **one by one**:

$$y_1 \to y_2 \to \cdots \to y_n$$

At each timestep t, the model needs to know the previous tokens $y_{<t}$ to maximize $P\left(y_t \mid y_{<t}, x; \theta\right)$. So the question is: **what do we feed as $y_{<t}$ during training?**

✗ **Option 1: Use the model's own predictions**. We could feed the model's own predictions at each timestep:

$$\hat{y}_{t-1} \xrightarrow{\text{feed}} \text{decoder}_t \xrightarrow{\text{predict}} \hat{y}_t$$

However, at early stages of training, the model's predictions are likely to be very poor, which can lead to a **feedback loop of errors** and make training unstable. If the first few predictions are wrong, the model will be conditioned on incorrect tokens, which can cause it to generate even worse predictions in subsequent timesteps.

✓ **Option 2: Use the true previous tokens (Teacher Forcing)**. Instead of feeding the model's own predictions, during training we can feed the **true previous tokens** from the training data:

$$y_{t-1}^{\text{true}} \xrightarrow{\text{feed}} \text{decoder}_t \xrightarrow{\text{predict}} \hat{y}_t$$

This technique is called **Teacher Forcing**.

> **Definition 13: Teacher Forcing**
>
> **Teacher Forcing** is a **training technique** for sequence models in which, at each timestep, the **decoder receives the true previous token as input** instead of the token predicted by the model. Formally, during training:
>
> $$y_{t-1}^{\text{true}} \to \text{decoder} \to \hat{y}_t$$
>
> Instead of:
>
> $$\hat{y}_{t-1} \to \text{decoder} \to \hat{y}_t$$

It simplifies the learning problem by providing the model with the **correct context at each timestep**, allowing it to learn the correct mappings from input to output **without being affected by its own mistakes during training**.

Training vs Inference Mismatch

During training, the decoder is conditioned on the **true previous tokens** (*teacher forcing*), while at inference time it must rely on its **own predictions**:

$$y_{t-1}^{\text{true}} \to \hat{y}_t \quad \text{(training)}, \qquad \hat{y}_{t-1} \to \hat{y}_t \quad \text{(inference)}$$

This mismatch can lead to **error accumulation**: mistakes made at early timesteps propagate and negatively affect subsequent predictions. This phenomenon is closely related to **Exposure Bias**.

Definition 14: Exposure Bias

Exposure Bias is a **phenomenon** in sequence models where there is a **discrepancy between training and inference conditions**.

During training, the model is conditioned on the **true previous tokens** (using teacher forcing):

$$y_{t-1}^{\text{true}} \to \text{decoder} \to \hat{y}_t$$

Whereas during inference, it must rely on its **own predictions**:

$$\hat{y}_{t-1} \to \text{decoder} \to \hat{y}_t$$

As a result, the model is never exposed during training to its own prediction errors. This can lead to **error accumulation**, where early mistakes propagate and degrade the quality of the generated sequence.

Intuition. The model learns to generate sequences assuming a perfect history, but at inference time it must operate on a potentially noisy history, leading to a mismatch in behavior.

4.8.5 Special Tokens & Dataset Batch Preparation

In sequence-to-sequence models, the output is generated token by token. **To properly control the generation process** and **handle variable-length sequences**, it is necessary to introduce special tokens.

Why are special tokens necessary?

1. **How does generation start?** The decoder needs a starting point to begin generating the output sequence. The `SOS` token serves this purpose.
2. **How does generation end?** The model needs a way to indicate when it has finished generating the sequence. The `EOS` token signals the end of the sequence.
3. **How to handle variable-length sequences?** In a batch of sequences, they may have different lengths. The `PAD` token is used to pad shorter sequences to the same length as the longest sequence in the batch, allowing for efficient batch processing.
4. **How to handle out-of-vocabulary words?** The `UNK` token represents words that are not in the model's vocabulary, allowing the model to handle unknown inputs gracefully.

Definition 15: Special Tokens

Special Tokens are **artificial symbols added to sequences** to **indicate structural information** such as the beginning or end of a sequence, or to handle padding and unknown words.

The most commonly used special tokens are:

- **Start-of-Sequence (`SOS`)**: indicates the start of the output sequence. It is used to initialize the decoder:

$$y_0 = \texttt{<SOS>}$$

- **End-of-Sequence (`EOS`)**: indicates the end of the sequence. Generation stops when this token is produced.
- **Padding (`PAD`)**: used to make sequences in a *batch* (see below) have the same length.

  Why do we need batching? In deep learning, models are typically trained using **mini-batches** rather than processing one sequence at a time. This allows efficient parallel computation on modern hardware (e.g., GPUs) and leads to more stable optimization.

  However, sequences in a dataset may have **different lengths**, which makes batching non-trivial. Neural networks require inputs to have the same shape, so sequences must be aligned to a common length. This is where the `PAD` token becomes essential: it is used to extend shorter sequences so that all sequences in a batch have the same length.

❓ **Unknown (`UNK`)**: represents tokens that are not in the vocabulary.

Special tokens allow the model to: start generating a sequence; decide when to stop; handle variable-length sequences in a fixed-size computational framework.

How to prepare batches of sequences?

Neural networks usually operate on **tensors of fixed dimensions**[7], because they are optimized for parallel computation (i.e., on GPUs). However, **sequences** in a dataset can have **varying lengths**, which poses a **challenge for batching**. To address this, sequences must be preprocessed before being fed into the model.

The common approach to prepare batches of sequences is as follows:

1. **Tokenization**: Convert raw text into a sequence of tokens (e.g., words, subwords, or characters).
2. **Adding Special Tokens**: Prepend the `SOS` token to the beginning of each target sequence and append the `EOS` token to the end. This allows the model to know where sequences start and end during training and inference.
3. **Padding**: Determine the length of the longest sequence in the batch and **pad all shorter sequences** with the `PAD` token until they match this length. This ensures that all sequences in the batch have the same length, allowing them to be processed together as a single tensor.

   For example, if the longest sequence has a length of 10 tokens, all sequences in the batch will be padded to have a length of 10. If the longest sequence has a length of 10 tokens, but the tensor requires a fixed length of 12, then all sequences will be padded to have a length of 12, with the last two tokens being `PAD`.
4. **Creating Masks**: Generate masks to **indicate which tokens are actual data and which are padding**. This is important for the model to ignore the padded tokens during training and inference.

By following these steps, we can efficiently prepare batches of sequences for training sequence-to-sequence models, ensuring that the model can learn effectively from the data while handling the variability in sequence lengths.

[7] A tensor is a generalization of scalars, vectors, and matrices to higher dimensions. It is a multi-dimensional array that can represent data in various forms, such as images, sequences, or more complex structures. In deep learning, tensors are the primary data structure used for input and output of models.

4.8.6 Decoding: Greedy Search and Beam Search

Up to this point, we have defined a probabilistic model that assigns a score to any possible output sequence:

$$P(y \mid x)$$

And we have learned how to train this model so that correct sequences receive high probability.

But how do we generate the output? At inference time, the goal is to find the most probable sequence:

$$y^* = \arg\max_y P(y \mid x)$$

However, as we have seen, this optimization problem is intractable, since the number of possible output sequences grows exponentially with their length. Therefore, it is not possible to enumerate all sequences and select the best one.

This motivates the need for **decoding strategies**, which approximate the optimal solution by constructing the output sequence step by step using the probabilities produced by the decoder. The two most common decoding strategies are:

- **Greedy Search**: Generates the output sequence by selecting, at each timestep, the token with the highest probability:

  $$y_t = \arg\max_y P\left(y \mid y_{<t}, x\right)$$

  This **process is repeated sequentially** until the `EOS` token is generated.

  Key idea. At each step, the locally most probable token is selected, without considering future consequences.

  Limitation. Greedy search may **lead to suboptimal sequences**, since a locally optimal choice at one timestep may result in a **globally poor sequence**.

- **Beam Search**: Improves upon greedy search by keeping track of the top k most probable partial sequences (called **beams**) at each timestep. Precisely, at each step:

  - Each current sequence is expanded with all possible next tokens;
  - The resulting candidates are scored using their cumulative probability;
  - Only the top k sequences are retained.

  This process continues until the sequences are completed (i.e., the `EOS` token is generated).

  Key idea. Beam search explores multiple hypotheses simultaneously, balancing exploration and exploitation (i.e., it considers multiple possible continuations of the sequence rather than committing to a single choice at each step).

⚖ **Trade-off.** Beam search is more computationally expensive than greedy search, but it often produces better results by avoiding the pitfalls of local optima.

- With $k = 1$, beam search reduces to greedy search;
- As k increases, the quality of the generated sequence typically improves, but at the cost of increased computational resources.

Example 10: Beam Search vs. Greedy Search

Consider a sequence generation problem where the model must generate a sentence token by token. Suppose the model produces the following probabilities:

- **Step 1 (first word)**
  - "I" $P = 0.6$
  - "You" $P = 0.4$
- **Step 2 (second word).** If the first word is "I":
  - "am" $P = 0.5$ $\Rightarrow$ total: $0.6 \cdot 0.5 = 0.30$
  - "like" $P = 0.5$ $\Rightarrow$ total: $0.6 \cdot 0.5 = 0.30$

  If the first word is "You":
  - "are" $P = 0.9$ $\Rightarrow$ total: $0.4 \cdot 0.9 = 0.36$

Greedy Search. At the first step, greedy selects the most probable token:

$$y_1 = \text{"I"}$$

Then continues from this choice, producing a sequence with total probability 0.30:

"I am" or "I like"

Beam Search (with beam width $k = 2$). At the first step, it keeps both candidates:

"I", "You"

At the second step, it evaluates all expansions and selects the best sequence:

"You are" with probability 0.36

Since:

$$0.36(\text{"You are"}) > 0.30(\text{"I am" or "I like"})$$

Greedy search makes locally optimal choices and may miss the best sequence, while beam search explores multiple hypotheses and can find a better global solution, at the cost of increased computational complexity.

5 Unsupervised Learning: Word Embedding

5.1 Introduction

5.1.1 Recall Machine Learning Paradigms

Before introducing **word embeddings**, it is useful to briefly recall the three main machine learning paradigms (page 11). The distinction is important because word embeddings are one of the most successful examples of **unsupervised learning**.

Machine learning algorithms learn from experience, usually represented as a dataset:

$$D = \{x_1, x_2, \ldots, x_N\}$$

Where each x_i is an observed sample. Depending on the information available during learning, we can distinguish three major paradigms:

- **Supervised Learning**. In supervised learning, **each training example is associated with a desired output** (or target value):

  $$D = \{(x_1, t_1), (x_2, t_2), \ldots, (x_N, t_N)\}$$

  Where x_i is the *input*, and t_i is the *correct output*. The objective (i.e., the goal) is to learn a function:

  $$f(x) \approx t$$

  That can correctly predict the output for previously unseen inputs.

  Some **examples** of supervised learning tasks are: image classification, spam detection, house price prediction, and speech recognition. In all these cases, the training data consists of input-output pairs, and the model learns to map inputs to outputs.

  Most of the neural networks studied so far in the course belong to this category (Perceptron page 41, FeedForward Neural Networks page 57, and we will see in future sections also CNNs for Image Classification). During training, the network compares its prediction with the ground truth and updates its weights to reduce the prediction error.

- **Unsupervised Learning**. In unsupervised learning, **only the inputs are available**:

  $$D = \{x_1, x_2, \ldots, x_N\}$$

  No target labels are provided. The objective is therefore **not** to predict a known output, but rather to **discover useful structures**, **patterns**, or **representations** hidden in the data.

  So, instead of learning $x \to t$ (input to output), the algorithm tries to understand how the data itself is organized. Typical tasks include clus-

tering[8], dimensionality reduction[9], feature extraction[10], density estimation[11], and representation learning[12]. **Word embeddings belong to this category** because they learn meaningful vector representations of words directly from large text corpora without requiring manually annotated labels. The model only observes the co-occurrence patterns of words and discovers semantic relationships automatically.

- **Reinforcement Learning**. Reinforcement learning differs from both previous paradigms because **learning occurs through interaction with an environment**. At each step, an agent:

  1. Observes the current situation (state) of the environment.
  2. Chooses an action to perform based on its current policy.
  3. Receives feedback in the form of a reward (or penalty) from the environment.
  4. Updates its strategy (policy) to maximize the cumulative reward over time.

  The dataset can be represented as a sequence of actions and rewards:

  $$(a_1, r_1), (a_2, r_2), \ldots, (a_N, r_N)$$

  The **goal** is not to predict labels but to learn a policy that maximizes the cumulative reward over time.

  Typical applications include robotics, autonomous driving, game playing (e.g., AlphaGo), and resource allocation and control. Unlike supervised learning, the correct action is not explicitly provided. The agent must discover good behaviors through trial and error.

[8] Clustering (page 15) is the task of discovering groups in the data without labels. For example, given a dataset *cat*, *dog*, *wolf*, *lion*, *tiger*, and *car*, *truck*, *bus*; a clustering algorithm might group *cat*, *dog*, *wolf*, *lion*, and *tiger* together, and *car*, *truck*, and *bus* together, without any prior knowledge of the labels. Some typical clustering algorithms include K-means, hierarchical clustering, and DBSCAN, already discussed in the context of unsupervised learning in the Applied Statistics course.

[9] **Dimensionality Reduction** is the task of reducing the number of variables while preserving important information. For example, Principal Component Analysis (PCA) is a common technique that transforms the data into a new coordinate system, where the greatest variance by any projection of the data comes to lie on the first coordinate (called the first principal component), the second greatest variance on the second coordinate, and so on. This can help in visualizing high-dimensional data or in reducing noise. Already discussed in the context of unsupervised learning in the Applied Statistics course.

[10] **Feature Extraction** is the process of learning useful features automatically. For example, in image processing, instead of manually designing features like edges or textures, a neural network can learn to extract features that are most relevant for a given task.

[11] **Density Estimation** is the process of learning the probability distribution that generated the data. For example, given a dataset of normal credit card transactions, a density estimation model could learn the distribution of legitimate transactions and then identify outliers that might indicate fraud.

[12] **Representation Learning** is the task of learning a better representation of the data automatically. We will see this in more detail in future sections.

Why is this important for Word Embeddings? The remainder of this chapter focuses on **unsupervised learning**. The central idea is that large amounts of text contain hidden regularities that can be exploited to build meaningful numerical representations of words.

Instead of manually defining semantic relationships such as:

- *king* is related to *queen*;
- *Paris* is related to *France*;
- *dog* is related to *animal*;

An unsupervised algorithm can discover these relationships automatically by analyzing how words appear together in natural language. This capability leads to **word embeddings**, where each word is represented by a dense vector that captures semantic and syntactic information, forming the foundation of many modern Natural Language Processing systems.

5.1.2 Neural Autoencoders

Before studying **word embeddings**, it is useful to understand the idea of an **autoencoder**, because it represents one of the most important examples of unsupervised representation learning. In fact, the central **goal of both autoencoders and word embeddings is the same**: learn a compact and meaningful representation of data without requiring labels.

Autoencoder Architecture

A **Neural Autoencoder** is a **neural network trained to reproduce its input at the output**:

$$g(x) \approx x$$

In other words, the network is asked to learn the identity function. At first sight this might seem pointless: ***if the desired output is exactly the input, why not simply copy it?*** The answer is that we deliberately constrain the network so that copying becomes impossible. Let's see how.

The typical architecture (see Figure 39, page 313) is composed of **three parts**:

$$x \xrightarrow{\text{Encoder}} h \xrightarrow{\text{Decoder}} g$$

Where:

- $x \in \mathbb{R}^I$ is the **input** vector;
- $h \in \mathbb{R}^J$ is the **hidden representation** (or **latent code**);
- $g \in \mathbb{R}^I$ is the **reconstructed** output vector.

Usually, the input and output dimensions are the same ($\mathbb{R}^I$), while the hidden representation is much smaller ($\mathbb{R}^J$): $J \ll I$. It means that the hidden layer contains far fewer neurons than the input layer. This creates a **bottleneck**, forcing the network to compress the information contained in the input.

The **encoder** transforms the original data into a compact **latent representation**:

$$h = \text{enc}(x) \tag{128}$$

This latent vector should contain only the most relevant information needed to describe the input. The **decoder** then reconstructs the **original sample** from this compressed representation:

$$g = \text{dec}(h) \tag{129}$$

The complete network can therefore be written as:

$$g = \text{dec}(\underbrace{\text{enc}(x)}_{h}) \tag{130}$$

The **encoder** can be viewed as a **compression** mechanism, while the **decoder** acts as a **decompression** mechanism. Together they learn how to represent the input efficiently.

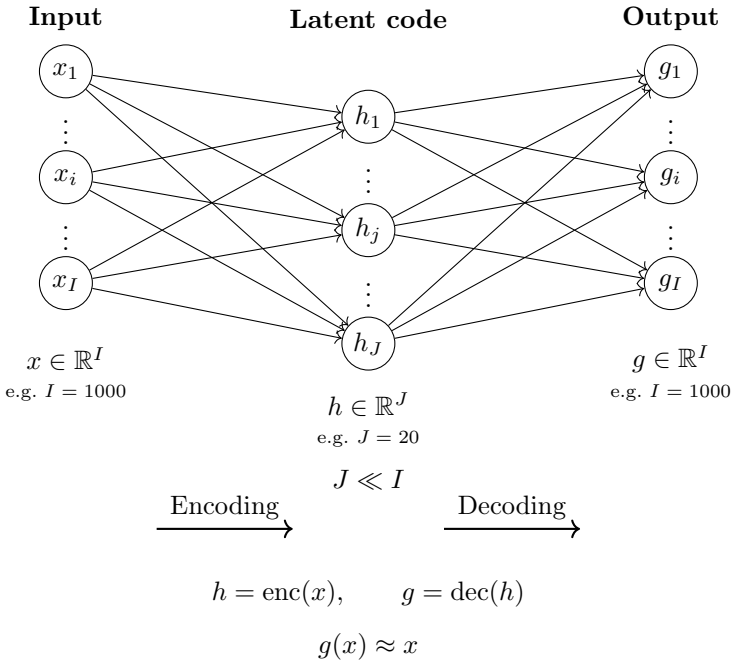


Figure 39: Architecture of a neural autoencoder. The encoder compresses the input vector $x \in \mathbb{R}^I$ into a low-dimensional latent representation $h \in \mathbb{R}^J$ (where $J \ll I$), while the decoder reconstructs the original input, producing $g(x) \approx x$.

Reconstruction Error

Since the objective is to reconstruct the input, the network is trained by minimizing the difference between the original input and its reconstruction. This difference is quantified by a **loss function**, which measures the **Reconstruction Error**. The training process adjusts the parameters of both the encoder and decoder to minimize this error across the training dataset.

A common loss function is the **squared reconstruction error**:

$$E_{\text{rec}} = \sum_i (g_i(x) - x_i)^2 \tag{131}$$

This quantity measures how much information is lost during compression and reconstruction. If the reconstruction error is **small**, the latent representation h has **successfully captured the essential characteristics of the input**.

The network therefore learns a representation that **preserves as much useful information as possible while using fewer dimensions**.

Are there other ways to force compression?

Reducing the size of the hidden layer is not the only way to force compression. Another common strategy is to impose a **sparsity constraint**.

The idea is that only a small number of hidden neurons should be active for any given input: $h_j \approx 0$ for most hidden units j. But, ***how can we enforce this sparsity?*** First of all, with *sparsity* we refer to the **latent representation** h. Suppose the encoder produces a vector h:

$$h = (h_1, h_2, \ldots, h_6)$$

Without any sparsity constraint, an input might produce a dense representation like:

$$h = (0.8, 0.6, 0.9, 0.7, 0.5, 0.4)$$

Almost all hidden units are active (i.e. have non-zero activations), then the representation is **dense**. On the other hand, if we enforce sparsity, we might get a representation like:

$$h = (0.0, 0.9, 0.0, 0.0, 0.8, 0.0)$$

In this case, only two hidden units are active, while the others are close to zero. This is a **sparse** representation, which forces the network to learn more meaningful and specialized features instead of spreading information everywhere.

However, the question remains: ***how can we encourage the network to produce sparse representations?*** One approach is to add a penalty term to the loss function that encourages most hidden activations to be close to zero:

$$E_{\text{total}} = E_{\text{rec}} + \lambda E_{\text{sparse}} \tag{132}$$

Where E_{rec} is the reconstruction error, while the second term E_{sparse} **discourages excessive activation** of hidden neurons. The hyperparameter λ **controls the trade-off between accurate reconstruction and sparsity**:

- If $\lambda = 0$, the network focuses solely on minimizing reconstruction error, which may lead to dense representations.
- If λ is **very small**, the network mostly cares about reconstruction, but sparsity has a small influence.
- If λ is **large**, the network is heavily punished for activating neurons. The representation becomes very sparse.
- If λ is **too large**, the network becomes obsessed with sparsity. It may decide to activate no neurons at all, resulting in a trivial solution where h is always zero, which is not useful for reconstruction. This is **extremely sparse**, but now the **decoder cannot reconstruct anything useful**. The reconstruction error explodes.

Therefore, increasing λ encourages sparsity, but it affects negatively the reconstruction quality; decreasing λ allows for better reconstruction, but it may lead to dense representations and thus less meaningful features.

Why not just make J smaller? Since the goal is to learn a compact representation, one may wonder ***why not simply reduce the number of hidden units*** J. Instead of adding a sparsity penalty, we could just set J to a small value. The answer is that the two approaches enforce compression in different ways:

- Reducing J creates a **hard bottleneck**. The latent representation has fewer dimensions than the input, forcing the network to compress information into a smaller space. Compression is imposed directly by the architecture.
- Adding a sparsity penalty creates a **soft bottleneck**. The network may still have many hidden units, but for any given input only a small subset is encouraged to activate. Compression is therefore imposed by the learning objective rather than by the architecture.

In practice, sparse representations often lead to neurons that specialize in detecting particular patterns or features, making the learned representation more interpretable. Moreover, sparsity can be useful even when the latent space is not smaller than the input space ($J \geq I$), preventing the network from trivially learning the identity mapping.

Definition 1: Autoencoder

A **Neural Sparse Autoencoder** is a **neural network** trained to **reconstruct its input while also encouraging sparsity in the hidden representation**. The loss function is defined as:

$$E = \underbrace{\|g_i(x_i \mid w) - x_i\|^2}_{\text{Reconstruction Error}} + \lambda \underbrace{\sum_j \left| h_j \left(\sum_i w_{ji}^{(1)} x_i \right) \right|}_{\text{Sparsity Term}} \tag{133}$$

Where:

- $\|g_i(x_i \mid w) - x_i\|^2$ is the squared reconstruction error. It is simply the **error between the original input** x_i and the **reconstructed output** $g_i(x_i \mid w)$.

> **Deepening: Notation for Network Output**
>
> The notation $g_i(x_i \mid w)$ indicates the i-th output neuron produced by the network g_i with parameters w when the input is x_i. It is a notation common in older machine learning papers where the neural network is viewed as a function parametrized by weights w. In modern deep learning, we often write $g(x)$ to denote the output of the network for input x, and the parameters are implicitly understood to be part of the function.

If reconstruction is perfect $g_i(x_i \mid w) = x_i$, then the reconstruction error is zero:

$$\|g_i(x_i \mid w) - x_i\|^2 = 0$$

And the network has successfully learned to reproduce the input.

- $\lambda \sum_j \left| h_j \left(\sum_i w_{ji}^{(1)} x_i \right) \right|$ is the sparsity penalty.
  - $w_{ji}^{(1)}$ is the **weight** connecting the i-th input neuron to the j-th hidden neuron in the first layer $^{(1)}$.
  - $\sum_i w_{ji}^{(1)} x_i$ is the **pre-activation** of the j-th hidden neuron, which is the weighted sum of the inputs. Usually it is called z_j. In other words, it computes the input of hidden neuron j before applying the activation function:

$$z_j = \sum_i w_{ji}^{(1)} x_i$$

  - $h_j(z_j)$ is the **activation** of the j-th hidden neuron after applying a non-linear activation function to the pre-activation z_j. For example, if we use ReLU, then $h_j(z_j) = \max(0, z_j)$.
  - $|h_j(z_j)|$ is the absolute value of the hidden activation, it measures **how active the j-th hidden neuron is**. If it is close to zero, the neuron is inactive; if it is large, the neuron is active.
  - $\sum_j |h_j(z_j)|$ sums the absolute values of the **hidden activations across all hidden neurons**. It measures the total activity of the hidden layer. It answers the question: ***how many hidden neurons are active for this input, and how strongly?***.
  - λ is a hyperparameter that controls the strength of the sparsity penalty (see page 314).

Connection with Word Embeddings

In general, an autoencoder learns a **compressed representation** of the input data $x \to h$, where h is a compact representation containing the essential information about x.

Word embeddings follow **exactly** the same philosophy. Instead of compressing images, signals, or generic feature vectors, they **learn a compact vector representation for words**. The embedding vector plays a role very similar to the latent representation h of an autoencoder.

5.1.3 Applications of Autoencoders

Although at first glance an autoencoder (page 312) may seem useless (after all, it is trained to reproduce its input) forcing the network to pass information through a compressed or constrained latent representation allows it to **learn useful structures hidden in the data**. Because of this property, autoencoders have many practical applications.

- **Data Compression**. One of the most natural applications of autoencoders is **data compression**. The encoder transforms the original input vector $x \in \mathbb{R}^I$ into a lower-dimensional latent representation $h \in \mathbb{R}^J$, with $J < I$, which contains only the most relevant information needed to reconstruct the input. The decoder can later use this compressed representation to approximately recover the original data.

  Unlike traditional compression algorithms, the encoder and decoder are learned directly from data and can therefore capture complex nonlinear relationships.

- **Dimensionality Reduction**. Autoencoders can also be viewed as a form of **nonlinear dimensionality reduction**. Classical techniques such as Principal Component Analysis (PCA) project data onto a lower-dimensional linear subspace. Autoencoders generalize this idea by learning nonlinear mappings, allowing them to discover more complex structures in the data. For this reason, they are often described as a kind of nonlinear PCA.

  The latent representation learned by the encoder can then be used for visualization, clustering, or as input to other machine learning algorithms.

- **Feature Extraction**. The hidden representation learned by an **autoencoder often contains more meaningful information than the raw input**. Instead of using the original data directly, we can use the latent vector h as a new set of features. These features typically capture higher-level patterns and correlations present in the data.

  Historically, autoencoders were frequently used as a pretraining mechanism: a network was first trained in an unsupervised way to learn useful features and then fine-tuned on a supervised task.

- **Denoising**. Autoencoders can be trained to **remove noise from data**. Instead of reconstructing a clean input from itself, the network receives a corrupted version $\tilde{x}$ and is trained to reconstruct the original clean sample x. This forces the encoder to learn robust features that capture the underlying structure of the data rather than the noise itself. Such **denoising autoencoders** are widely used in image restoration, audio enhancement, and signal processing.

- **Anomaly Detection**. An autoencoder **trained only on normal examples becomes highly specialized at reconstructing them**.

  When an unusual or anomalous sample is presented, the network is typically unable to reconstruct it accurately because it does not match the

patterns seen during training. Consequently, anomalous inputs tend to produce a much larger reconstruction error:

$$\text{Reconstruction Error} = \|x - \hat{x}\|^2$$

Where x is the original input and $\hat{x}$ is the reconstructed output. This property makes autoencoders useful for fraud detection, industrial monitoring, cybersecurity, and fault diagnosis.

- **Generative Models**. More advanced variants of autoencoders can be used to **generate new data**. The most famous example is the Variational Autoencoder (VAE), which learns a structured latent space from which new samples can be generated by drawing random latent vectors and decoding them. VAEs represent an important family of generative models and are the foundation of several modern generative techniques.

- **Word Embeddings** (our main focus in this section). Autoencoders have also **inspired methods for learning word embeddings**, which are **dense vector representations of words**.

  The main idea is conceptually similar: instead of representing a word as a very large sparse one-hot vector (i.e., a vector with a single 1 and the rest 0s, a word gets its own unique position in the embedding space), a neural network learns a compact latent representation in a much lower-dimensional continuous space. Similar words tend to be mapped to nearby points in this space.

  For example, words such as *king* and *queen*, or *Paris* and *Rome*, often exhibit meaningful geometric relationships in the embedding space. Word embeddings will be studied in detail in the next chapter, where we will examine techniques such as Word2Vec and GloVe.

5.2 Motivation for Word Embeddings

5.2.1 The Limits of Traditional Word Representations

Before introducing word embeddings, we must understand why traditional representations of text are inadequate. The central **problem is that computers do not naturally understand words**: every word must be converted into a numerical representation before it can be processed by a machine learning algorithm. Early Natural Language Processing (NLP) systems achieved this using dictionary identifiers, one-hot encodings, and bag-of-words representations. While simple and intuitive, these approaches suffer from severe limitations. Let's explore these limitations in detail.

⚠ Dictionary IDs

The simplest way to represent words is to assign each word a unique identifier in a dictionary. For example:

Word	ID
`cat`	537
`dog`	143
`house`	892
`computer`	1245

A sentence such as "*the cat sleeps*" might therefore be represented as the sequence of IDs:

$$[17, 537, 981]$$

This approach is easy to implement, but it introduces a fundamental problem: **the numerical values have no semantic meaning**. For example:

$$\text{cat} = 537 \quad \text{and} \quad \text{dog} = 143$$

Does **not** imply that cats and dogs are more or less similar than:

$$\text{cat} = 537 \quad \text{and} \quad \text{house} = 892$$

The identifiers are merely arbitrary labels. The machine cannot infer any relationship between words from these numbers alone.

⚠ One-Hot Encoding

To remove the artificial ordering introduced by dictionary IDs, NLP systems often use **One-Hot Encoding**.

Suppose the vocabulary contains V words. Each word is represented by a vector of length V:

$$w_i = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

Where w_i is the one-hot vector for the i-th word in the vocabulary, and the 1 is at the position corresponding to that word. For example, if we have a vocabulary of 5 words: `cat`, `dog`, `house`, and `computer`, their one-hot encodings might be:

Word	One-Hot Encoding
`cat`	$[1 \quad 0 \quad 0 \quad 0 \quad 0]^T$
`dog`	$[0 \quad 1 \quad 0 \quad 0 \quad 0]^T$
`house`	$[0 \quad 0 \quad 1 \quad 0 \quad 0]^T$
`computer`	$[0 \quad 0 \quad 0 \quad 1 \quad 0]^T$

This representation avoids assigning artificial numerical meaning to dictionary IDs. However, it **creates another problem**:

$$\text{cat}^T \cdot \text{dog} = 0 \quad \text{and} \quad \text{cat}^T \cdot \text{house} = 0$$

From the machine's perspective, **every pair of different words is equally unrelated**. The representation contains information about identity, but no information about meaning.

⚠ Bag-of-Words (BoW)

When working with complete documents, *one-hot vectors* are often **aggregated** into a **Bag-of-Words (BoW) representation**.

Instead of storing word order, we simply count how many times each word appears. Suppose the vocabulary is:

$$\{\texttt{cat}, \texttt{dog}, \texttt{house}, \texttt{computer}\}$$

The document "*cat cat dog*" would be represented as:

$$\begin{bmatrix} 2 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

This representation captures word frequency but **completely ignores grammar and word order**. The sentences "*dog bites man*" and "*man bites dog*" would produce identical Bag-of-Words vectors even though their meanings are very different. In general, a document is transformed into a large vector containing words occurrences, but **the relationships between words are lost**.

⚠ The Curse of Dimensionality

The most serious **issue** appears when the **vocabulary** becomes **large**. Modern corpora (i.e., large text collections) can easily contain:

$$|V| > 10^5 \quad \text{or even} \quad |V| > 10^6$$

Distinct words. Consequently:

- One-hot vectors require hundreds of thousands or **millions of dimensions** (one for each word in the vocabulary).
- **Most entries are zero** because each word is represented by a single 1 in a sea of 0s.
- **Documents** become extremely **sparse vectors** because each document contains only a small fraction of the total vocabulary.
- **Similar words** remain completely **disconnected** (e.g., `cat` and `dog` have no relationship in the vector space).

For example:

$$\text{cat} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad \text{and} \quad \text{dog} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Are almost entirely zeros despite representing semantically related concepts. This phenomenon is known as the **Curse of Dimensionality**: as dimensionality grows, data becomes increasingly sparse, distances become less informative, and learning becomes more difficult.

Why We Need Something Better

Traditional representations suffer from four major limitations:

1. **No semantic information** (dictionary IDs, one-hot encoding): cat and dog appear as unrelated as cat and computer.
2. **Extreme sparsity** (one-hot encoding, bag-of-words): most vector components are zero.
3. **High dimensionality** (bag-of-words): vector size grows with vocabulary size.
4. **Poor generalization** (curse of dimensionality): similar words cannot share information.

These limitations motivate the search for a new representation capable of: being dense instead of sparse; using far fewer dimension; capturing semantic relationships; allowing similar words to be represented similarly. This is precisely the **goal of word embeddings**, which we will introduce in the next section. If we asked a human "*are 'cat' and 'dog' more similar than 'cat' and 'computer'?*", the answer would be immediate. Traditional representations cannot express this knowledge. The entire purpose of word embeddings is to transform words into vectors such that:

$$\text{similar meaning} \implies \text{similar vectors}$$

This simple idea will completely change how machines process language.

5.2.2 Word Similarity Ignorance

The previous section showed that traditional representations such as one-hot encoding are sparse and high-dimensional. However, an even more serious problem exists: **one-hot vectors completely ignore Semantic Similarity between words**.

Definition 2: Semantic Similarity

Semantic Similarity is the degree to which two words, phrases, or concepts have similar meanings. More formally, two linguistic entities are semantically similar if they can be used in similar contexts and convey related meanings.

In the context of Word Embeddings, words with similar meanings should have similar vectors, then:

$$\text{semantic similarity} \Rightarrow \text{geometric proximity}$$

The whole goal of word embeddings is to transform an abstract concept such as **meaning** into something measurable using geometry.

Similarity vs Relatedness. There is an important nuance. Consider *car* and *road*. They are strongly related, but they do not mean the same thing. Similarly, *doctor* and *hospital* are related, but not similar. In contrast, *car* and *automobile* are both related and similar. Therefore, many embedding models actually capture a mixture of:

- **Semantic Similarity** is the degree to which two words have similar meanings and can be used interchangeably in many contexts. For example, *car* and *automobile* are semantically similar because they refer to the same concept.
- **Semantic Relatedness** is the degree to which two words are associated or connected, regardless of whether they have similar meanings. For example, *car* and *road* are semantically related because they frequently occur together and belong to the same conceptual domain, but they are not semantically similar because they refer to different concepts.

For example, consider the following twosentences:

- *Obama speaks to the media in Illinois.*
- *The President addresses the press in Chicago.*

To a human reader, these sentences convey almost the same meaning. The words are different, but the concepts they express are strongly related:

Sentence 1	Sentence 2
Obama	President
speaks	addresses
media	press
Illinois	Chicago

A human immediately recognizes these semantic relationships. A machine using one-hot representations does not because the one-hot vectors for these words are orthogonal to each other. In other words, the one-hot representation does not capture any notion of similarity between words. Let's see this in action.

⚠ One-Hot Encoding Cannot Express Similarity

Recall that in a vocabulary of size V, each word is represented by a vector containing a single 1 and $V-1$ zeros. For example:

$$\begin{aligned} \text{speaks} &= [0, 0, 1, 0, \ldots, 0] \\ \text{addresses} &= [0, 0, 0, 0, \ldots, 1, 0] \end{aligned}$$

Similarly:

$$\begin{aligned} \text{obama} &= [0, 0, 0, 0, \ldots, 1, 0, 0] \\ \text{president} &= [0, 0, 0, 1, \ldots, 0, 0, 0] \end{aligned}$$

And:

$$\begin{aligned} \text{illinois} &= [1, 0, 0, 0, \ldots, 0] \\ \text{chicago} &= [0, 1, 0, 0, \ldots, 0] \end{aligned}$$

These vectors have no overlapping active components (i.e., no 1s in the same position), so they are orthogonal.

Mathematically, **any two different one-hot vectors are orthogonal**. For example:

$$\begin{aligned} \text{speaks}^T \cdot \text{addresses} &= 0 \\ \text{obama}^T \cdot \text{president} &= 0 \\ \text{illinois}^T \cdot \text{chicago} &= 0 \end{aligned}$$

We can also use the symbol $\perp$ to denote orthogonality:

$$\begin{aligned} \text{speaks} &\perp \text{addresses} \\ \text{obama} &\perp \text{president} \\ \text{illinois} &\perp \text{chicago} \end{aligned}$$

Geometrically, the angle between the vectors is 90°, which means they are completely unrelated.

Why is this a problem? Suppose a language model has learned many examples containing the word "*president*" and later encounters "*Obama*". A human

immediately understands that these words are related, but the **model cannot**. From the model's perspective "*Obama*" is just another compleltely unrelated symbol. And the same happens for "*speaks*" and "*addresses*", or "*Illinois*" and "*Chicago*". Since the vectors are orthogonal, the model has no mechanism for transferring knowledge from one word to another.

Failure to Generalize

This inability to capture similarity directly **affects a model's capacity to generalize**. Imagine that during training we only observe: "*Obama speaks to the media in Illinois*". If we later encounter: "*The President addresses the press in Chicago*" the model should ideally recognize that the second sentence is semantically similar to the first.

With one-hot representations, however, every key word is represented by a completely different orthogonal vector. Consequently, the model sees the second sentence as almost entirely new. The knowledge learned from the first sentence cannot be reused effectively.

What we actually need: Word Embeddings

To generalize successfully, **similar words** should have **similar representations**. Ideally, we would like:

$$\text{similar meaning} \Rightarrow \text{similar vectors}$$

Word pairs share no similarity in their one-hot representations, but we need word similarity to be reflected in the representations.

The fundamental **goal** of **word embeddings** is therefore to replace orthogonal one-hot vectors with dense continuous vectors such that:

$$\begin{aligned} \mathbf{v}_{\text{Obama}} &\approx \mathbf{v}_{\text{President}} \\ \mathbf{v}_{\text{speaks}} &\approx \mathbf{v}_{\text{addresses}} \\ \mathbf{v}_{\text{Illinois}} &\approx \mathbf{v}_{\text{Chicago}} \end{aligned}$$

Instead of **representing words** as isolated symbols, we want to represent them **as points in a continuous semantic space where distance reflects meaning**. This idea leads directly to the concept of word embeddings, which we introduce next.

5.2.3 What is a Word Embedding?

The previous section highlighted the fundamental limitation of one-hot representations: semantically related words such as *Obama* and *President* appear completely unrelated because their vectors are orthogonal. The natural question is therefore: ***how can we represent words so that similar words have similar representations?*** The answer is **word embeddings**.

Definition 3: Word Embedding

A **Word Embedding** is any technique that **maps a word** (or phrase) from its **original symbolic representation** into a **lower-dimensional numerical vector space**.

Formally, given a vocabulary:

$$V = \{w_1, w_2, \ldots, w_{|V|}\}$$

We define a mapping:

$$C : V \to \mathbb{R}^m$$

Which **associates each word with a dense vector** of m real numbers.

For example:

$$\text{Obama} \mapsto [0.12, -0.25, 0.81, \ldots, 0.03]$$

Instead of representing a word using a huge sparse vector of length $|V|$, we represent it using a much smaller dense vector.

From One-Hot Vectors to Embeddings

Recall the one-hot representation (page 319):

$$\text{Obama} = [0, \ldots, 0, 1, 0, \ldots, 0]$$

Where the vector length equals the vocabulary size. If the vocabulary contains $|V| = 1,000,000$ words, then the representation contains one million components, almost all equal to zero. A word embedding transforms this representation into something like: $\text{Obama} = [0.12, -0.25, 0.81, \ldots]$, with perhaps only $100 \leq m \leq 500$ dimensions. Thus, word embeddings achieve:

- **Compression**: reducing the dimensionality of the representation from $|V|$ to m (where $m \lll |V|$).
- **Densification**: transforming a sparse vector (mostly zeros) into a dense vector (mostly non-zero values).
- **Smoothing**: similar words have similar representations, so the embedding space is smooth and continuous, rather than discrete and sparse.

All of which help fight the curse of dimensionality (page 320).

Not only Compression, but also Semantic Proximity

The key idea of word embeddings is not merely dimensionality reduction (compression). The real objective is that words with similar meanings should have similar vectors. In a good embedding space:

$$\mathbf{v}_{\text{Obama}} \approx \mathbf{v}_{\text{President}} \qquad \mathbf{v}_{\text{speaks}} \approx \mathbf{v}_{\text{addresses}} \qquad \mathbf{v}_{\text{Illinois}} \approx \mathbf{v}_{\text{Chicago}}$$

Unlike one-hot vectors, which are orthogonal, embeddings allow distances and similarities to carry semantic information.

Geometric Interpretation

A useful way to think about word embeddings is geometrically. Each **word becomes a point** in an m-dimensional space. Words with **similar meanings** tend to appear **near one another**. Words with **unrelated meanings** tend to be **farther apart**. In the Figure 40, we can see how words cluster together based on their semantic categories (e.g., food, cities, animals, emotions). The Figure 40 also shows one of the most remarkable properties of embedding spaces: **related concepts naturally organize into clusters**. Words belonging to the same conceptual category tend to occupy nearby regions of the embedding space. The model is never explicitly told these categories. They emerge automatically from the statistical patterns observed in language.

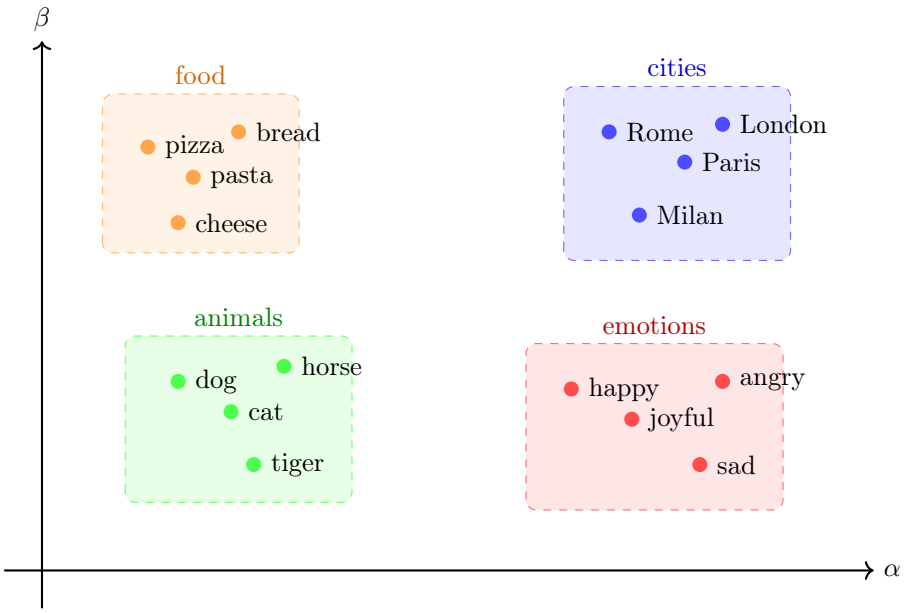


Figure 40: Geometric interpretation of word embeddings as clusters in a continuous vector space. Words with similar meanings are mapped to nearby points in the embedding space. α and β represent abstract dimensions that capture semantic features. This is a simplified 2D visualization; real embeddings typically have hundreds of dimensions, which are impossible to visualize directly.

Where does Semantic Similarity come from?

The ability of word embeddings to capture Semantic Similarity (page 322) originates from the **Distributional Hypothesis** in linguistics, which states that **words that occur in similar contexts tend to have similar meanings**. The natural question is therefore: ***how can a learning algorithm discover that two words have similar meanings without access to a dictionary?***

A famous quote by linguist John Rupert Firth summarizes this idea: *"You shall know a word by the company it keeps."* In practice, this means that if two words frequently appear in similar contexts (e.g., *Obama* and *President*), they are likely to have similar meanings, and thus their embeddings should be close together in the vector space. Or more formally, *semantic similarity* can be inferred from contextual similarity.

For example, let's start with humans. Suppose we invent a new word with no prior meaning:

*The **blorg** barked loudly at the mailman.*

We don't know what a *blorg* is. But from the surrounding words (*barked*, *loudly*, *mailman*), we can infer that a *blorg* is likely some kind of animal, probably a dog. Then, we learned something about *blorg* without ever seeing a dictionary definition.

After reading many sentences:

- *The blorg barked.*
- *The blorg chased the cat.*
- *The blorg ran in the park.*
- *The blorg wagged its tail.*

We become increasingly convinced that blorg $\approx$ dog, even though nobody ever explicitly told us that a *blorg* is a dog. This is exactly what word embedding algorithms do. They never see a dictionary definition of a word. Instead, they see millions of sentences and learn to infer the meaning of words from their contexts. The more similar the contexts, the more similar the meanings, and thus the closer the embeddings.

In other words:

$$\textbf{contextual similarity} \implies \textbf{semantic similarity}$$

This principle is known as the *Distributional Hypothesis* and forms the foundation of most modern word embedding techniques.

Connection with Autoencoders

At this point, the connection with the previous section (page 312) should become apparent. Recall that an autoencoder learns a compact latent representation:

$$x \in \mathbb{R}^n \quad \xrightarrow{\text{Encoder}} \quad h \in \mathbb{R}^m \quad \xrightarrow{\text{Decoder}} \quad \hat{x} \in \mathbb{R}^n$$

The hidden vector h acts as a latent representation that captures the most relevant information about the input while using fewer dimensions. Word embeddings follow a very similar philosophy:

$$\begin{aligned} \text{word} \in V \quad &\xrightarrow{\text{Embedding}} \quad \mathbf{v} \in \mathbb{R}^m \\ C : V \to \mathbb{R}^m \quad &\text{with} \quad m \ll |V| \end{aligned}$$

Instead of representing a word using a huge sparse one-hot vector, we **learn a compact dense representation v** that **captures meaningful semantic information**. In this sense, a word embedding can be viewed as a learned latent representation of a word.

It is important to note that word embedding models are generally not trained as autoencoders. The similarity lies in the objective of learning a compact latent representation rather than in the specific training procedure. While an autoencoder learns a representation by reconstructing its input, word embedding methods typically learn representations by exploiting contextual information in large text corpora.

5.2.4 Distributed Representations

In the previous section we introduced the concept of a word embedding as a mapping from words to vectors in a continuous space. We can now make this idea more precise.

Consider a vocabulary of V words:

$$V = \{w_1, w_2, \dots, w_{|V|}\}$$

In a one-hot representation (page 319), each word is associated with a unique position:

$$\text{Obama} = [0, \dots, 0, 1, 0, \dots, 0]$$

Only one component contains information. For this reason, **one-hot vectors** are often called **Local Representations**: each word is identified by a single location in the vector.

In a **Distributed Representation**, instead, a word is represented by a vector of real numbers:

$$\text{Obama} = [0.12, -0.25, 0.81, \dots, 0.03]$$

Now **information is spread** across all dimensions. Each component contributes a small amount of information about the word. This is why the representation is called **distributed**. With local representations, semantic similarity is not captured, while with distributed representations, semantically similar words are mapped to nearby points in the vector space.

What does each position in the vector mean? In short, we usually **do not know** what each dimension means individually. And this is the fundamental difference between one-hot vectors and embeddings.

- In a **one-hot vector** is easy to interpret because we know that the i-th position corresponds to the i-th word in the vocabulary. But it is not useful for learning.
- In a **distributed representation** is hard to interpret because the dimensions are learned automatically by the neural network. The coordinates of the vector are not assigned by us, but are discovered automatically by the model during training. A dimension may partially encode gender, profession, nationality, formality, syntactic role, all mixed together.

Fighting the Curse of Dimensionality

Distributed Representation is not only a different way of represeting words; it also addresses one of the major limitations of traditional Natural Language Processing (NLP) representations: the **curse of dimensionality**. This goal is achieved through three mechanisms:

1. **Compression**. A typical vocabulary may contain $|V| > 10^6$ distinct words. A one-hot representation therefore requires more than one million dimensions. A word embedding instead maps each word to a vector of dimension $100 < m < 500$. Thus $\mathbb{R}^{|V|} \to \mathbb{R}^m$ with $m \ll |V|$ mapping is a compression of the original representation.

2. **Smoothing**. One-hot encodings are inherently discrete (i.e., they are either 0 or 1). There is **no notion of partial similarity**. Instead, embeddings transform words into points of a continuous space:

$$C(w) \in \mathbb{R}^m$$

This process is called **Smoothing** because **nearby points can represent similar concepts**.

3. **Densification**. One-hot vectors are extremely sparse, and $\frac{1}{|V|}$ of the components are non-zero. For a vocabulary of one million words $\frac{1}{10^6} = 0.0001\%$, almost the entire vector contains no information. Instead, embeddings **transform** these **sparse vectors into dense feature vectors** where most components contribute information.

5.3 Language Modeling

Before studying **how embeddings are learned**, we need to understand the problem that motivated their development. Historically, word embeddings did not emerge as an isolated idea. They were discovered while researchers were trying to solve a much broader problem: ***Language Modeling***. That is: *given a sequence of words, can we predict how likely that sequence is, and what word should come next?*

5.3.1 The Language Modeling Problem

First of all, a **Language Model (LM)** is a **probabilistic model that assigns probabilities to sequences of words**. Given a sentence:

$$s = (w_1, w_2, \ldots, w_k)$$

We want to estimate the probability of that sentence:

$$P(s) = P\left(w_1, w_2, \ldots, w_k\right)$$

Where w_i is the i-th word in the sentence. This quantity measures how likely the sentence is according to the language.

Why do we need Language Models?

Many Natural Language Processing (NLP) applications depend on language modeling: Chatbots, Machine Translation, Speech Recognition, Information Retrieval, Document Classification, Text Generation, Large Language Models (LLMs). The performance of many real-world systems depends on how language is represented and modeled.

For example, consider two sentences:

1. *The cat is sleeping on the sofa.*
2. *Sofa sleeping cat the on is.*

A human immediately recognizes that:

$$P(\text{The cat is sleeping on the sofa}) \gg P(\text{Sofa sleeping cat the on is})$$

The first sentence is a valid English sentence, while the second one is not. A language model **attempts to learn exactly this distinction**. But now the question is: ***how can we build a model that learns to assign high probabilities to valid sentences and low probabilities to invalid ones?***

Probability of Sentences

At first glance, estimating the probability of a sentence:

$$P(s_k) = P\left(w_1, w_2, \ldots, w_k\right)$$

Seems impossible. Even a short sentence can contain many words, and the number of possible sentences is astronomical. Fortunately, probability theory gives us a useful tool: the **Chain Rule of Probability**. To compute the probability of several events happening together, we can decompose it into a product of conditional probabilities. For example, for two events A and B:

$$P(A, B) = P(A)\, P(B \mid A)$$

For three events A, B, and C:

$$P(A, B, C) = P(A)\, P(B \mid A)\, P(C \mid A, B)$$

In general, for n events $A_1, A_2, \ldots, A_n$:

$$\begin{aligned} P(A_1, A_2, \ldots, A_n) &= P(A_1)\, P(A_2 \mid A_1) \cdots P(A_n \mid A_1, A_2, \ldots, A_{n-1}) \\ &= \prod_{i=1}^{n} P(A_i \mid A_1, A_2, \ldots, A_{i-1}) \end{aligned} \tag{134}$$

It is called the ***Chain Rule of Probability*** because it allows us to break down a complex joint probability into simpler conditional probabilities. Applying the chain rule to language modeling, we can express the probability of a sentence as:

$$P(w_1, w_2, \ldots, w_k) = P(s_k) = P(w_1)\, P(w_2 \mid w_1) \cdots P(w_k \mid w_1, w_2, \ldots, w_{k-1})$$

This means that to compute the probability of a sentence, we can **compute the probability of each word given the previous words**. For example, the sentence "*the cat sleeps*" can be decomposed as:

$$P(\text{the cat sleeps}) = P(\text{the})\, P(\text{cat} \mid \text{the})\, P(\text{sleep} \mid \text{the cat})$$

With the Chain Rule, have we actually solved anything?

With the Chain Rule, we have reduced the problem of estimating the probability of a sentence to estimating the probability of each word given the previous words. However, *have we actually solved the problem?* After all, we **still need to estimate probabilities**. The answer is that the Chain Rule does **not solve the language modeling problem**. It only reformulates it into a more structured form.

Instead of asking "*what is the probability of every possible sentence?*" we ask "*what is the probability of a word appearing after a given context?*". For example:

$$P(\text{sleep} \mid \text{the cat})$$

Can be interpreted as: *given the context "the cat", how likely is the next word "sleeps"?* This transforms language modeling into a collection of **next-word prediction problems**.

The chain rule reveals that **if we can estimate probabilities** of the form:

$$P(w_i \mid w_1, w_2, \ldots, w_{i-1})$$

Then we can automatically compute the probability of any sentence. Consequently, researchers shifted their attention from sentence probabilities to next-word prediction. For example, given the context:

The cat is

A language model might estimate the following probabilities for the next word:

$$P(\text{sleeping} \mid \text{The cat is}) = 0.70$$
$$P(\text{eating} \mid \text{The cat is}) = 0.20$$
$$P(\text{flying} \mid \text{The cat is}) = 0.001$$

The most likely continuation is therefore: *sleeping*. Notice that the objective is not really to generate text. The **objective is to estimate conditional probabilities**. Text generation is simply a consequence: once the model knows the probabilities of possible next words, it can choose one and continue the sentence.

But again, we have not yet solved the language modeling problem. We have only transformed it into a more structured task: estimating probabilities of the form:

$$P(w_i \mid w_1, w_2, \ldots, w_{i-1})$$

The natural question is now: ***how can we estimate these probabilities in practice?***

Connection with LLMs

The problem introduced here (i.e., estimating the probability of a word given its context) is essentially the same problem solved by modern Large Language Models (LLMs). When ChatGPT generates text, it is repeatedly estimating:

$$P(w_i \mid w_1, w_2, \ldots, w_{i-1})$$

And choosing the next token according to that distribution. The difference is that:

- Classical language models use simple statistical methods.
- Neural Language Models (our focus) use neural networks to estimate probabilities.
- Modern LLMs use transformers with billions of parameters.

But the underlying objective remains the same: **predict the next word (or token) given the previous context**.

Why is this difficult?

At this point we might think: ***great, we just need to estimate conditional probabilities*** (i.e., the probability of a word given its context). Unfortunately, there is a **problem**. To compute:

$$P(w_i \mid w_1, w_2, \ldots, w_{i-1})$$

We may need to consider the entire history of the sentence. In general, for a sentence of length k, the context can become arbitrarily large, because the information needed to predict the next word may depend on words that appeared much earlier in the sentence. This challenge led researchers to develop N-gram Language Models, which approximate the problem by considering only a limited number of previous words.

5.3.2 N-gram Language Models

In the previous section we saw that a language model should estimate:

$$P(w_i \mid w_1, \ldots, w_{i-1})$$

That is, the **probability of the next word given all previous words**. At first this seems reasonable. However, in practice this is impossible because the context can become arbitrarily long. Consider the sentence:

"*The student who attended the deep learning course and studied for several weeks finally passed the ...*"

To predict the next word, should we consider the previous 3 words? 10 words? 100 words? A classical solution to this problem is the **N-gram Language Model**, which makes a simplifying assumption: **only a small number of previous words matter**.

The Main Idea

Instead of using the entire history:

$$(w_1, w_2, \ldots, w_{i-1})$$

We only consider the last $N-1$ words. For example: "*the cat is sleeping on the sofa*". To predict *sofa*:

- Bigram ($N = 2$): $P(\text{sofa} \mid \text{the})$
- Trigram ($N = 3$): $P(\text{sofa} \mid \text{on the})$
- 4-gram ($N = 4$): $P(\text{sofa} \mid \text{sleeping on the})$

The sequence of n consecutive words is called an N-gram.

The **set of previous words used for prediction** is called the **Context Window**. For example, for a trigram model:

$$P(w_i \mid w_{i-2}, w_{i-1})$$

The context window contains two words: w_{i-2} and w_{i-1}. Instead, for a 5-gram model:

$$P(w_i \mid w_{i-4}, w_{i-3}, w_{i-2}, w_{i-1})$$

The context window contains four words: w_{i-4}, w_{i-3}, w_{i-2}, and w_{i-1}. The larger the context window, the more information we capture, the more difficult learning becomes, and the more data we need to train the model.

The Markov Assumption

The N-gram models are based on the **Markov Assumption**. So, instead of using the entire history:

$$P(w_i \mid w_1, \ldots, w_{i-1})$$

We approximate:

$$P(w_i \mid w_1, \ldots, w_{i-1}) \approx P(w_i \mid w_{i-N+1}, \ldots, w_{i-1}) \tag{135}$$

If we **substitute the approximation into the exact chain rule** (page 332), we obtain:

$$\hat{P}(s_k) = \prod_i P(w_i \mid w_{i-N+1}, \ldots, w_{i-1}) \tag{136}$$

The hat symbol $\hat{P}$ indicates that this is not the true probability anymore, but an approximation. We are essentially saying that the **future depends only on the recent past**.

Why is it called a Markov assumption? Because it is analogous to a **Markov process**. In a first-order Markov chain:

$$P(X_t \mid X_1, \ldots, X_{t-1}) = P(X_t \mid X_{t-1})$$

The future depends only on the current state. Similarly, in an N-gram model:

$$P(w_i \mid \text{all previous words}) \approx P(w_i \mid \text{last} n-1 \text{words})$$

The distant past is ignored, and the future depends only on the recent past.

Example 1: Trigram Model

Suppose the sentence:

$$s = (\text{The, cat, is, sleeping})$$

The exact chain rule is:

$$P(s) = P(\text{The}) \cdot P(\text{cat} \mid \text{The}) \cdot P(\text{is} \mid \text{The cat}) \cdot P(\text{sleeping} \mid \text{The cat is})$$

With a trigram model ($N = 3$), we approximate:

$$P(\text{sleeping} \mid \text{The cat is}) \approx P(\text{sleeping} \mid \text{cat is})$$

We only keep the last two words in the context window. Therefore, the trigram approximation of the sentence probability is:

$$\hat{P}(s) = P(\text{The}) \cdot P(\text{cat} \mid \text{The}) \cdot P(\text{is} \mid \text{The cat}) \cdot P(\text{sleeping} \mid \text{cat is})$$

The sentence probability is still a product of conditional probabilities, but the condition now contains only a limited context.

The problem of Language Modelling has not yet been solved!

After the Markov assumption we have reduced the problem to:

$$P(w_i \mid w_{i-N+1}, \ldots, w_{i-1})$$

Now we know **what** probability we want to estimate. The next question becomes: ***how do we assign a numerical value to it?***. In general, the actual problem of language modeling is, ***how do we estimate***:

$$P(w_i \mid w_1, \ldots, w_{i-1})$$

From data? This is where models enter and each model has a different way of estimating these probabilities. The N-gram model estimates these probabilities by counting how many times each word appears after a given context in the training data:

$$\begin{aligned} P(w_i \mid w_{i-N+1}, \ldots, w_{i-1}) &= P(w_i \mid \text{context}) \\ &= \frac{\#(\text{context}, w_i)}{\#(\text{context})} \\ &= \frac{\#(w_{i-N+1}, \ldots, w_{i-1}, w_i)}{\#(w_{i-N+1}, \ldots, w_{i-1})} \end{aligned} \tag{137}$$

Where:

- $\#(w_{i-N+1}, \ldots, w_{i-1}, w_i)$ counts how many times the **N-gram** (context + word) **appears in the training data**.
- $\#(w_{i-N+1}, \ldots, w_{i-1})$ counts how many times the **context appears in the training data**.

This is just a conditional probability estimated through frequencies:

$$P(A \mid B) = \frac{\#(A \cap B)}{\#(B)}$$

Where B is the context and A is the event "the next word is w_i".

Example 2: Estimating Trigram Probabilities

Suppose our corpus (i.e., training data) contains the following sentences:

- "*the cat sleeps*" (appears 3 times)
- "*the cat eats*" (appears 1 time)

We want to estimate $P(\text{sleeps} \mid \text{the cat})$. We count how many times the trigram "*the cat sleeps*" appears in the training data:

$$\#(\text{the cat sleeps}) = 3$$

We also count how many times the bigram "*the cat*" appears in the training data:

$$\#(\text{the cat}) = 3 + 1 = 4$$

Therefore, we estimate:

$$P(\text{sleeps} \mid \text{the cat}) = \frac{\#(\text{the cat sleeps})}{\#(\text{the cat})} = \frac{3}{4} = 0.75$$

Similarly, we can estimate:

$$P(\text{eats} \mid \text{the cat}) = \frac{\#(\text{the cat eats})}{\#(\text{the cat})} = \frac{1}{4} = 0.25$$

Thus, the trigram model estimates that after the context "*the cat*", the word "*sleeps*" is more likely to appear than the word "*eats*".

The Zero-Probability Problem

The **Zero-Probability Problem** is probably the biggest weakness of classical N-gram models. The issue is very simple: ***what happens if a word sequence never appears in the training corpus?*** Since N-gram probabilities are estimated by counts:

$$P(w_i \mid \text{context}) = \frac{\#(\text{context}, w_i)}{\#(\text{context})}$$

If the sequence "*context,* w_i" never appears in the training data, then:

$$\#(\text{context}, w_i) = 0$$

And therefore:

$$P(w_i \mid \text{context}) = 0$$

For example, suppose the training data contains the following sentences:

- "*the cat sleeps*" (appears 3 times)
- "*the cat eats*" (appears 1 time)

We want to estimate $P(\text{dances} \mid \text{the cat})$. We count how many times the trigram "*the cat dances*" appears in the training data:

$$\#(\text{the cat dances}) = 0$$

Because it never appears. We also count how many times the bigram "*the cat*" appears in the training data:

$$\#(\text{the cat}) = 3 + 1 = 4$$

Therefore, we estimate:

$$P(\text{dances} \mid \text{the cat}) = \frac{\#(\text{the cat dances})}{\#(\text{the cat})} = \frac{0}{4} = 0$$

Why is this a problem? Because "*never seen*" does not mean "*impossible*". A cat **can** *dance* in a sentence (i.e., a sentence like "*the cat dances*" is perfectly valid), but humans understand that it is **unusual, but not impossible**. However, the N-gram model assigns a probability of zero to this event, which is clearly incorrect, because it thinks that "*the cat dances*" is impossible.

The core problem with N-gram models is that they **only memorize observed contexts** and **cannot generalize**. For example, if the training data contains the following sentences:

- "*the dog sleeps*"
- "*the cat sleeps*"
- "*the rabbit sleeps*"

A human immediately learns that "*animals often sleep*" and would assign a reasonable probability to the sentence "*the hamster sleeps*" even if it was never seen before. In contrast, an N-gram model cannot do this because if it never saw the trigram "*the hamster sleeps*" in the training data, it may assign probabilities of zero to all words that follow the context "*the hamster*". And this is the **worst-case scenario**! It arises from the fact that N-gram models use the chain rule of probability:

$$P\left(\text{sentence}\right) = \prod_i P\left(w_i \,|\, \text{context}_i\right)$$

And since this is a product, if **one factor is zero**, the entire sentence probability becomes zero. Therefore, if the model assigns a probability of zero to any word in the sentence, it will assign a probability of zero to the entire sentence. This is clearly undesirable.

The Classical Solution: Smoothing. To solve this problem, researchers introduced **smoothing techniques**. The idea is really simple: ***reserve a small amount of probability mass for unseen events***. For example, if the model assigns a probability of 0.75 to "*the cat sleeps*" and 0.25 to "*the cat eats*", we can reserve a small amount of probability mass (e.g., 0.01) for unseen events like "*the cat dances*". This way, the model will assign a small but non-zero probability to unseen events, allowing it to **generalize better**. One of the most popular smoothing techniques is Katz smoothing (1987).[13]

[13] **Katz Smoothing** is a smoothing technique that redistributes a small amount of probability mass from frequent N-grams to unseen N-grams, preventing zero probabilities. It was introduced by Slava Katz in his 1987 paper "Estimation of Probabilities from Sparse Data for the Language Model Component of a Speech Recognizer". [7]

5.3.3 The Curse of Dimensionality in N-gram Models

At first sight, N-gram language models seem like an elegant solution to the language modeling problem. Yes, the zero-probability problem is a problem, but it can be solved with smoothing techniques. However, N-gram models suffer from a more fundamental problem: the **curse of dimensionality**.

We started with the impossible task of estimating:

$$P(w_i \mid w_1, \ldots, w_{i-1})$$

And simplified it to:

$$P(w_i \mid w_{i-N+1}, \ldots, w_{i-1})$$

By considering only the last $N-1$ words as context. Unfortunately, this simplification introduces a new and fundamental problem: the number of possible contexts grows exponentially with the context size.

Vocabulary Explosion

Suppose we have a vocabulary containing:

$$|V| = 100,000$$

Unique words. This is actually a realistic number for a moderately large corpus (i.e., a large collection of text). Now, consider a **10-gram model**, which looks at the previous 9 words as context:

$$P(w_i \mid w_{i-9}, w_{i-8}, w_{i-7}, w_{i-6}, w_{i-5}, w_{i-4}, w_{i-3}, w_{i-2}, w_{i-1})$$

Each position can contain any of the $100,000$ words. Therefore, the number of possible contexts is:

$$|V|^{N-1} = 100,000^9$$

And if we inlcude the target word, we have:

$$|V|^{N} = 100,000^{10}$$

Thus, the model lives in a 10D hypercube where each dimension has $100,000$ slots. Explicitly, the number of parameters we need to estimate is:

$$|V|^{N} = 100,000^{10} = \left(10^5\right)^{10} = 10^{50}$$

This number is astronomically larger than the number of books ever written, the number of webpages ever created, or even the number of sentences we could realistically collect. That's the **curse of dimensionality**: the number of parameters grows exponentially with the context size, making it impossible to estimate them all from a finite corpus.

⚠ Consequences of the Curse of Dimensionality

- **Sparse Statistics**. Since the number of parameters is so large (10^{50} possible combinations), our corpus only covers a tiny fraction of them. This means that most of the parameters will be zero, leading to sparse statistics.

  For example "*the cat sleeps on the sofa*" may appear. But "*the cat sleeps under the sofa*" may never appear. And "*the cat sleeps beside the sofa*" may never appear. So when we estimate:

  $$P\,(\text{beside}\,|\,\text{the cat sleeps})$$

  We will get zero, simply because we have never seen that exact combination of words in our corpus.

  This is what "*sparse statistics*" means. It refers to the fact that only a tiny fraction of all possible N-grams are observed in the training corpus. As a consequence, many valid N-grams have count zero, leading to the **zero-probability problem** (page 337), where unseen events are assigned probability zero even though they may occur in reality.

- **Probability Mass Vanishes**. This is not a new problem. It's just another way of saying the same thing. Let's imagine we distribute 1 euro among 10^{50} people. Everybody gets essentially nothing. Similarly:

  $$\sum P\,(\text{all contexts}) = 1$$

  But we must distribute this probability over an enormous number of possible N-grams. Most probabilities become 0 or nearly zero, which is called **probability mass vanishing**.

- **Huge Data Requirements**. We may think: "*If we just collect more data, we can estimate all these parameters.*" But if the space contains 10^{50} possible combinations, how much text would we need? An absurd amount. Even billions of sentences are nowhere near enough. Therefore, N-gram models require unrealistic amounts of data.

- **What Happens in Practice?** Researchers realized that N-gram models are not practical for large N. They typically use small values of N (like 2 or 3) to keep the number of parameters manageable. However, this leads to a loss of context and poorer performance. To mitigate this, they developed smoothing techniques to assign non-zero probabilities to unseen N-grams, but this is just a band-aid solution that doesn't address the fundamental issue of dimensionality.

? Why are Neural Language Models motivated by this?

At this point we face a dilemma:

- Large n, which captures more context but suffers from the curse of dimensionality.

- Small n, which is manageable but loses context and performs worse.

Researchers therefore began searching for a new idea. Instead of memorizing counts for every possible N-gram, could we **learn a continuous representation of words** that allows the model to generalize across similar contexts? This question led directly to the work of **Bengio et al. (2003)** and the **first Neural Network Language Model**, which introduced the concept of learned word embeddings.

5.4 Neural Language Models

5.4.1 Bengio et al. (2003)

At the beginning of the 2000s, language modeling faced a major obstacle. On one hand, traditional N-gram models were simple and effective. On the other hand, they suffered from the curse of dimensionality (page 339): large vocabularies, exponentially many contexts, sparse statistics, inability to generalize between similar words.

Researchers needed a fundamentally different approach. Instead of counting occurrences of word sequences, ***could a neural network learn a compact representation of words and use it to predict the next word?*** This question led to the influential work: **A Neural Probabilistic Language Model** by Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin (2003). [1] The paper is now considered one of the foundational works that eventually led to modern word embeddings and large language models.

Historical Context

To understand why Bengio's work was revolutionary, recall the **main limitation of N-gram** models. Consider the contexts:

- *Obama speaks to the media.*
- *President addresses the press.*

An N-gram model **treats them as** completely **different** sequences. The model cannot exploit the fact that:

- *Obama* and *President* are semantically similar.
- *speaks* and *addresses* are semantically similar.
- *media* and *press* are semantically similar.

Because every word is represented as an independent symbol. Consequently, knowledge learned from one context cannot be reused in another semantically similar context. In short, N-gram models **cannot generalize to unseen contexts**.

The Core Observation. Bengio realized something fundamental: **similar contexts should produce similar predictions**. If two contexts have similar meanings, the model should be able to share statistical strength between them. For this to happen, words must no longer be represented as isolated symbols. Instead, they must be represented by **continuous vectors**.

> **Remark: Difference Between Isolated Symbols and Continuous Vectors**
>
> Suppose we have the vocabulary:
>
> *Obama*, *President*, *speaks*, *addresses*, *media*, *press*

An N-gram model doesn't really understand these words. It only counts occurrences. For example, the model stores something conceptually like:

- ("*Obama*", "*speaks*") → 100 occurrences
- ("*President*", "*addresses*') → 80 occurrences

These are completely independent entries.

Now imagine we learn: "*Obama speaks to the media*" 10'000 times. Does this help us estimate the probability of "*President addresses the press*"? No, because for the N-gram model, "*Obama*" and "*President*", "*speaks*" and "*addresses*", "*media*" and "*press*" are just different symbols. No information is shared.

What is an isolated symbol? An isolated symbol is a word represented as a discrete atomic entity. For example, if the vocabulary is:

Obama, *President*, *speaks*, *addresses*, *media*, *press*

The model may internally associate an identifier to each word:

- *Obama* → 1
- *President* → 2
- *speaks* → 3
- *addresses* → 4
- *media* → 5
- *press* → 6

However, these identifiers are merely labels and carry no semantic meaning. The model has no notion that *Obama* and *President* are related, or that *speaks* and *addresses* are similar. From the model's perspective, they are simply different symbols. Therefore, the model only knows that *Obama* $\neq$ *President* (or equivalently $1 \neq 2$) and nothing more. Since **semantic relationships are not represented**, knowledge learned from one context cannot be transferred to similar contexts, limiting the model's ability to generalize. This is the **fundamental limitation of N-gram models**.

Bengio's Insight: Continuous Vectors. Instead of representing words as IDs, Bengio proposed representing each word as a **continuous vector** in a high-dimensional space. For example:

- *Obama* → $[0.2, 0.8, -0.1, \dots]$
- *President* → $[0.25, 0.75, -0.15, \dots]$

Now the vectors for *Obama* and *President* can be close to each other in this vector space, reflecting their semantic similarity. Similarly, *speaks* and *addresses* can have similar vectors, as can *media* and *press*.
They are called continuous vectors because their components are real numbers and every coordinate can vary continuously. Not just 0 or 1, but any real value (infinitely many possible values).

⚖ **The Fundamental Difference.** N-gram models treat words as unrelated dictionary entries (isolated symbols), while Belgio's neural language model represents words as continuous vectors, so they are nearby points in a geometric space.

The Main Idea

The key innovation of Bengio et al. (2003) can be summarized in one sentence: learn a distributed representation of words while simultaneously learning a language model. This was a radical departure from traditional N-gram approaches. Instead of:

$$word \rightarrow count\ occurrences \rightarrow estimate\ probabilities$$

The Neural Language Model performs:

$$word \rightarrow learn\ embedding\ vector \rightarrow feed\ vectors\ to\ neural\ network \rightarrow predict\ next\ word$$

Two Problems Solved at Once. The model is trained to predict the next word:

$$P\left(w_t \mid w_{t-N+1}, \ldots, w_{t-1}\right)$$

But during training it also learns an embedding vector for every word (i.e., a continuous vector representation, dense vector). Therefore, the model simultaneously solves two problems:

1. Primary Objective: **learn a language model**. The model is **trained to predict the next word** given the previous words.
2. Secondary Effect: **learn useful word embeddings**. The model **learns a continuous vector representation for each word**, capturing semantic relationships between words.

Interestingly, Bengio originally viewed the improved language modeling accuracy as the main contribution. The word vectors themselves were considered a by-product and left as future work. This is one of the great ironies of machine learning history: *the "side effect" became one of the most important ideas in modern NLP (Natural Language Processing).*

Distributed Representations. Bengio's model learns **distributed representations** of words. Instead of representing a word using a one-hot vector:

$$Obama = [0, \ldots, 1, \ldots, 0]$$

The model learns a dense vector (page 314):

$$Obama = [0.2, 0.8, -0.1, \dots]$$

Similarly, *President* can be represented as:

$$President = [0.25, 0.75, -0.15, \dots]$$

Because these vectors become similar, the network can naturally generalize between related contexts. For example, if the model has learned that "*Obama speaks to the media*" is a valid sequence, it can also predict that "*President addresses the press*" is likely, even if it has never seen that exact sequence before.

Why this was revolutionary?

The breakthrough was not merely using a neural network to predict the next word, because they already existed. The revolutionary idea was: replace symbolic words with learned continuous representations. This transforms language modeling from a counting problem into a representation learning problem. Instead of memorizing every possible N-gram:

- *Obama speaks*
- *President addresses*
- *Prime Minister says*

Independently, the model learns common patterns through shared embeddings.

Connection with Autoencoders. Notice how the idea resembles what we saw with autoencoders (page 312). In autoencoders:

$$Input \rightarrow Compressed\ latent\ representation \rightarrow Reconstruction\ of\ input$$

Similarly, in Neural Language Models:

$$Input \rightarrow Embedding\ vector \rightarrow Prediction\ of\ next\ word$$

In both cases:

- A compact latent representation is learned;
- The representation is not manually designed;
- Useful features emerge automatically from data.

5.4.2 Architecture of the Neural Language Model

After understanding the motivation behind Bengio et al. (2003), we can now study the architecture itself. The goal of the model is simple: **given the previous $n-1$ words, predict the next word**. For a training example:

$$(w_{t-N+1}, \ldots, w_{t-1}, w_t)$$

The model receives the context:

$$(w_{t-N+1}, \ldots, w_{t-1})$$

And tries to predict the next word w_t. The complete architecture consists of four main components (see Figure 41):

1. **Input Layer**. The input layer contains the **context words**. The dimension of the input layer is:

   $$|\text{Input Layer}| = (n-1) \cdot |V|$$

   Where $n-1$ is the **number of context words** and $|V|$ is the **vocabulary size**.

   Here, nothing is stored. The input layer contains only identifiers, which are the indices of the context words in the vocabulary. The input layer is a one-hot representation of the context words, where each word is represented as a vector of length $|V|$ with all zeros except for a single one at the index corresponding to the word in the vocabulary. As we saw in One-Hot Encoding, this representation is very sparse and high-dimensional (i.e., it has many zero entries). Therefore, the **network does not use these vectors as meaningful features**. Instead, they are simply fed to the next layer, the **embedding layer**.

2. **Embedding (Projection) Layer**. Where the magic happens. The model contains a **trainable matrix**:

   $$C \in \mathbb{R}^{|V| \times m}$$

   Where $|V|$ is the vocabulary size and m is the **embedding dimension**, which is a hyperparameter that we can choose (e.g., $m = 100$) and is typically much smaller than $|V|$, $m \ll |V|$. This matrix is called the **Embedding Matrix** and each row corresponds to one word.

   Lookup Operation. Suppose the word *Obama* corresponds to row 537 in the embedding matrix C. Its one-hot vector will have a one at index 537 and zeros everywhere else:

   $$\text{One-Hot}(\textit{Obama}) = [0^{(0)}, \ldots, 1^{(537)}, \ldots, 0^{(|V|-1)}]$$

   $$C = \begin{bmatrix} \vdots \\ C_{537} \\ \vdots \end{bmatrix} \xrightarrow{\text{lookup}} C(\textit{Obama}) = C_{537} \in \mathbb{R}^m$$

And the same applies to all other words in the context. The embedding layer performs a **lookup** in the embedding matrix C for each context word, retrieving their corresponding embedding vectors. Due to the lookup operation, this layer is called **projection**, since it projects the high-dimensional one-hot vectors into a lower-dimensional embedding space $\mathbb{R}^{|V|} \to \mathbb{R}^m$.

Concatenation. After retrieving the embeddings, the model concatenates them. Suppose we have a context of $n-1=3$ words, the concatenation operation is:

$$C(w_{t-3}) = \begin{bmatrix} 0.1 \\ 0.5 \end{bmatrix} \quad C(w_{t-2}) = \begin{bmatrix} 0.3 \\ 0.7 \end{bmatrix} \quad C(w_{t-1}) = \begin{bmatrix} 0.2 \\ 0.4 \end{bmatrix}$$

$$x = \begin{bmatrix} C(w_{t-3}) \\ C(w_{t-2}) \\ C(w_{t-1}) \end{bmatrix} = \begin{bmatrix} 0.1 \\ 0.5 \\ 0.3 \\ 0.7 \\ 0.2 \\ 0.4 \end{bmatrix} \in \mathbb{R}^{(n-1)\cdot m}$$

The **concatenation** is important because it preserves the order of the context words, which is crucial for language modeling. The resulting vector x is then fed to the next layer, the **hidden layer**. Its dimension is:

$$|\text{Embedding Layer}| = (n-1) \cdot m$$

Where $n-1$ is the number of context words and m is the embedding dimension for each word.

3. **Hidden Layer**. The purpose of the hidden layer is the classical one: **learns interactions between context words**. The hidden layer is a **fully connected layer** with a **nonlinear activation function**, typically tanh (page 67). The hidden layer computes:

$$h = \tanh(d + Hx)$$

Where $H \in \mathbb{R}^{h \times (n-1)\cdot m}$ is the **projection-to-hidden weight matrix**, $d \in \mathbb{R}^h$ is the **bias vector**, and h is the number of **hidden units**, which is a hyperparameter that we can choose ($500 < h < 1000$). The hidden layer captures complex relationships between the context words and transforms the concatenated embeddings into a higher-level representation that is more suitable for predicting the next word. Its dimension is:

$$|\text{Hidden Layer}| = h$$

4. **Output Layer**. The **final layer predicts the next word**. Its output size equals the vocabulary size $|V|$. For example, if the vocabulary size is $10,000$, then the network produces a vector of $10,000$ scores, one for each word in the vocabulary.

These scores are converted into probabilities using the **softmax function** (page 81):

$$\hat{P}\left(w_i = w_t \mid w_{t-N+1}, \ldots, w_{t-1}\right) = \frac{\exp\left(y_i\right)}{\sum_{j=1}^{|V|} \exp\left(y_j\right)}$$

Why Softmax? Because softmax guarantees:

$$0 \leq P\left(w_i\right) \leq 1 \quad \text{and} \quad \sum_i P\left(w_i\right) = 1$$

The output therefore becomes a proper probability distribution.

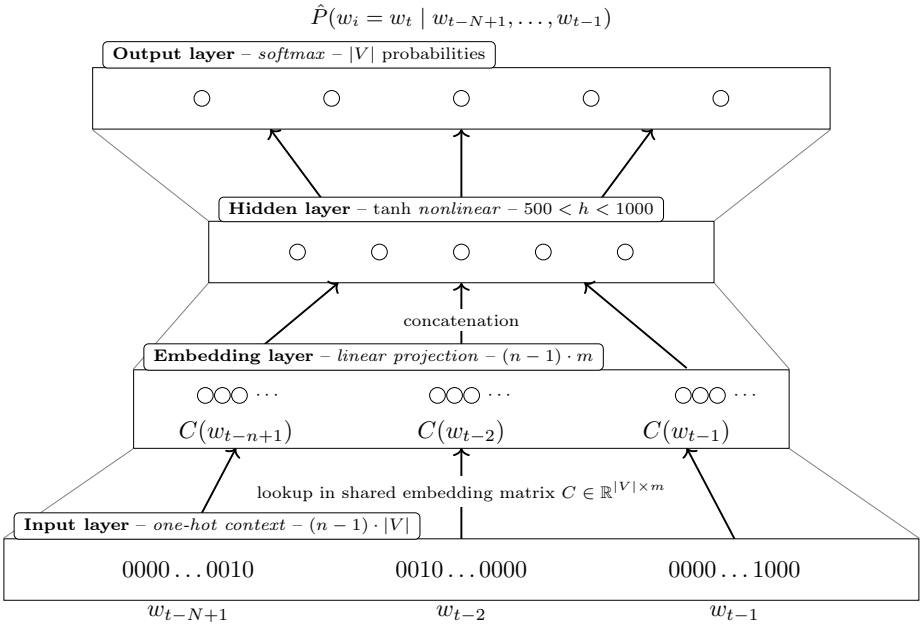


Figure 41: Neural probabilistic language model architecture inspired by Bengio et al. (2003).

How do we measure whether the prediction is good?

Suppose the model predicts:

Word	Predicted Probability
sleeps	0.8

This is very good. Instead, suppose the model predicts:

Word	Predicted Probability
sleeps	0.01

This is terrible. We therefore need a loss function that:

- Is **small** when the correct word has **high probability**;
- Is **large** when the correct word has **low probability**.

The Bengio's paper uses the **negative log-likelihood loss**:

$$E = -\log \hat{P}\left(w_t \,|\, w_{t-N+1}, \ldots, w_{t-1}\right) \tag{138}$$

This loss function has the desired properties:

- **Very good prediction**:

  $$P\left(w_t \,|\, \text{context}\right) = 0.9$$

  Then the loss is:

  $$E = -\log\left(0.9\right) \approx 0.105$$

  Small error.

- **Mediocre prediction**:

  $$P\left(w_t \,|\, \text{context}\right) = 0.5$$

  Then the loss is:

  $$E = -\log\left(0.5\right) \approx 0.693$$

  Larger error.

- **Terrible prediction**:

  $$P\left(w_t \,|\, \text{context}\right) = 0.01$$

  Then the loss is:

  $$E = -\log\left(0.01\right) \approx 4.605$$

  Huge error.

In other words, Bengio's idea was to punish the network when it assigns a low probability to the correct next word.

Example 3: Two-Word Context

The model receives two previous words (w_{t-2}, w_{t-1}) and must predict the next word w_t. So the model estimates: $P\left(w_t \,|\, w_{t-2}, w_{t-1}\right)$.

1. **Input Words as One-Hot Vectors**. Suppose the context is: *the cat* $\rightarrow$ *sleeps*. The input words are:

   $$w_{t-2} = \textit{the} \quad \text{and} \quad w_{t-1} = \textit{cat}$$

   Each word is represented as a one-hot vector of size $|V|$. These one-hot vectors are not used because they are semantically meaningful. They are used only to select rows from the embedding matrix.

2. **Lookup in the Embedding Matrix**. The model contains an

embedding matrix $C \in \mathbb{R}^{|V| \times m}$, where each row of C is the embedding vector of one word. So the two input words are mapped to their corresponding embedding vectors:

$$C(w_{t-2}) = C(the) = \begin{bmatrix} 0.10 & -0.30 & 0.25 \end{bmatrix}^T$$
$$C(w_{t-1}) = C(cat) = \begin{bmatrix} 0.72 & 0.18 & -0.45 \end{bmatrix}^T$$

3. **Concatenate the Embeddings**. After lookup, the two embeddings are concatenated:

$$x = [C(w_{t-2}) \oplus C(w_{t-1})]$$

With the example embeddings, the concatenation is:

$$x = \begin{bmatrix} 0.10 \\ -0.30 \\ 0.25 \\ 0.72 \\ 0.18 \\ -0.45 \end{bmatrix}$$

If each word vector has dimension $m = 3$, the concatenated vector has dimension $2m = 6$. In general, with $n - 1$ context words, the concatenated vector has dimension $(n - 1) \cdot m$.

4. **Hidden Layer**. The concatenated vector is passed to the hidden layer:

$$h = \tanh(d + Hx)$$

Where x is the concatenated context embedding, H is the "projection to hidden" weight matrix, d is the hidden bias, and tanh is the activation function. The hidden layer combines the information from the two context words.

5. **Output Scores**. The hidden representation is then transformed into a vector of scores $y \in \mathbb{R}^{|V|}$. Each component y_i is the score assigned to one possible next word.

6. **Softmax Probabilities**. The scores are converted into a probability distribution using softmax:

$$\hat{P}(w_i = w_t \mid w_{t-2}, w_{t-1}) = \frac{\exp(y_i)}{\sum_{j=1}^{|V|} \exp(y_j)}$$

The output is a probability for every word in the vocabulary, indicating how likely it is to be the next word given the context.

7. **Training Signal**. If the correct target word is *sleeps*, the training objective is:

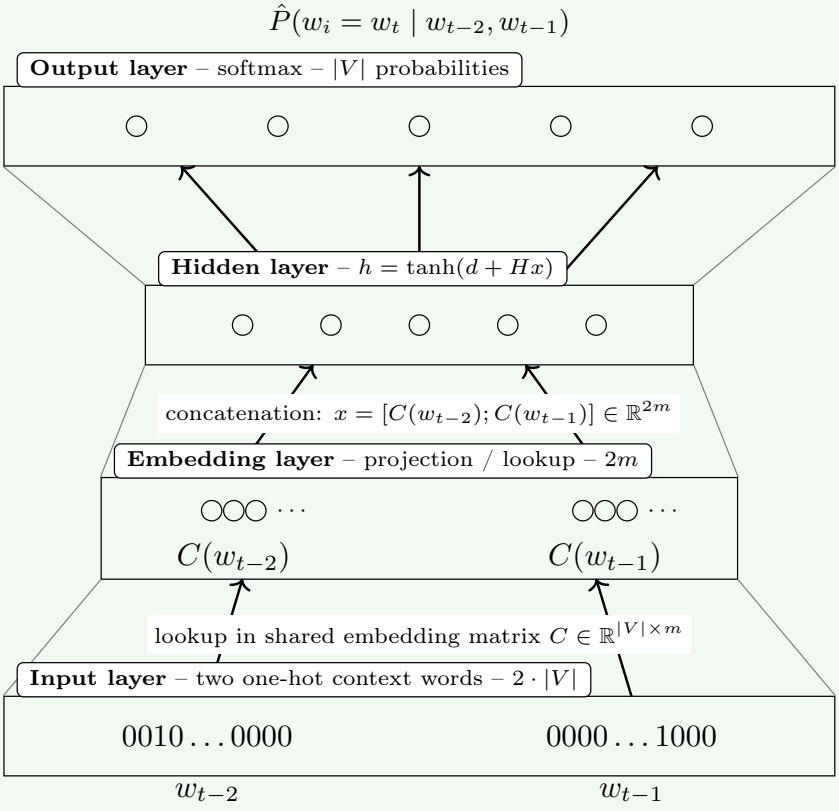
$$E = -\log \hat{P}(sleeps \mid \text{the}, \ cat)$$

During backpropagation, the model updates the embedding vectors for *the* and *cat*, as well as the hidden and output layer parameters, to increase the probability of *sleeps* in this context. This is the crucial point: while learning to predict the next word, the model also learns useful word embeddings.

The entire computation can be summarized as:

$$\begin{aligned} w_{t-2}, w_{t-1} &\xrightarrow{\text{lookup}} C\left(w_{t-2}\right), C\left(w_{t-1}\right) \xrightarrow{\text{concat}} \\ x &\xrightarrow{\text{hidden layer}} h \xrightarrow{\text{output layer}} \\ y &\xrightarrow{\text{softmax}} \hat{P}\left(w_i = w_t \mid w_{t-2}, w_{t-1}\right) \end{aligned}$$

And visually:



Neural probabilistic language model with a two-word context. The words w_{t-2} and w_{t-1} are mapped to embeddings, concatenated, processed by a hidden layer, and used to predict the next word w_t.

5.4.3 Training the Neural Language Model

Up to now we have studied:

1. *Why Bengio's model was proposed* (page 342);
2. *Its architecture* (page 346);
3. *A complete forward pass of the model* (page 349).

Now we must answer the next question: ***how is the network actually trained?*** The answer is surprisingly simple: train the network so that the correct next word receives the highest probability.

What parameters are being learned? The goal of training is to learn the parameters of the network, which are:

$$\theta = (C, H, U, d, b)$$

Where C is the word embedding matrix (stores word vectors), H is the hidden layer weight matrix (connects projection layer to hidden layer), U is the output layer weight matrix (connects hidden layer to output layer), d is the hidden layer bias vector, and b is the output layer bias vector. These parameters are learned by minimizing a loss function that quantifies how well the model predicts the next word given the context.

Maximum Likelihood Training

Suppose our training sentence is "*the cat sleeps*". Using a two-word context "(*The*, *cat*) $\rightarrow$ *sleeps*", the network outputs:

Word	Probability
sleeps	0.40
eats	0.30
jumps	0.20
computer	0.0001

The correct answer is "*sleeps*". We therefore want $P\,(\mathit{sleeps} \mid \mathit{the}, \mathit{cat})$ to becomes as large as possible.

Formally, for a single training example:

$$(w_{t-N+1}, \ldots, w_{t-1}, w_t)$$

The model predicts:

$$\hat{P}(w_t \mid \underbrace{w_{t-N+1}, \ldots, w_{t-1}}_{\text{context}})$$

This is the probability assigned to the correct next word w_t given the context. The training objective is:

$$\max \hat{P}(w_t \mid w_{t-N+1}, \ldots, w_{t-1})$$

In other words, we want to maximize the probability of the correct next word given the context. This is known as ***maximum likelihood training***.

How do we maximize the probability of the correct next word?

Naturally we would like to solve:

$$\max_{\theta} \hat{P}(w_t \mid w_{t-N+1}, \ldots, w_{t-1}; \theta)$$

Where θ represents all network parameters. So, ***why not maximize the probability of the correct next word directly?*** Two major problems arise:

1. **We have many training examples**. A corpus (i.e., a collection of training sentences) contains millions of examples:

   $$\left(w^{(1)}, w^{(2)}, \ldots, w^{(M)}\right)$$

   The probability of the entire dateset is:

   $$L(\theta) = \prod_{k=1}^{M} P\left(w_t^{(k)} \mid \text{context}^{(k)}\right)$$

   This is called the **likelihood function**. Notice the product over all training examples. If we have a million examples and each probability is around $0.01 - 0.1$, then:

   $$0.1^{1,000,000} \approx 0$$

   Is an absurdly tiny number. Numerically impossible to handle.

   Solution: Take the logarithm. Instead of maximizing:

   $$L(\theta) = \prod_{k} P_k$$

   We maximize:

   $$\log L(\theta) = \log\left(\prod_{k} P_k\right)$$

   Using the logarithm property $\log(ab) = \log a + \log b$:

   $$\log L(\theta) = \sum_{k} \log P_k$$

   This is the classical mathematical trick of log-likelihood maximization. Instead of maximizing the product of probabilities, we maximize the sum of log-probabilities. This is possible because the logarithm is a monotonic function. Thus, if $a > b$, then $\log a > \log b$, thus maximizing $\log L(\theta)$ is equivalent to maximizing $L(\theta)$.

   In short, the logarithm transforms products into sums, avoids numerical underflow caused by multiplying many small probabilities, and produces a loss function that is easier to optimize with gradient-based methods (solution to the second problem, see below).

2. **Deep learning optimizers minimize**. Historically, optimization algorithms are formulated as:

$$\min_{\theta} E(\theta)$$

Not:

$$\max_{\theta} L(\theta)$$

This is a minor problem, but it is solved by simply negating the log-likelihood:

$$\min_{\theta} -\log L(\theta)$$

Therefore, we will minimize the **Negative Log-Likelihood (NLL)**:

$$-\log L(\theta) = -\sum_{k} \log P_k \tag{139}$$

Furthermore, there is another **deep-learning reason** to minimize the negative log-likelihood instead of maximizing the log-likelihood. Imagine we maximize directly P. When P gets close to 1, gradients become very small and learning becomes difficult. Instead, using the negative log-likelihood $-\log(P)$ gives a much stronger signal when the model is wrong. Look:

P	$-\log(P)$
0.9	0.105
0.5	0.693
0.1	2.30
0.01	4.60
0.001	6.90

Notice how a terrible prediction $P = 0.001$ gets punished much more strongly.

In summary, the purpose of Negative Log-Likelihood is to **quantify how good or bad the prediction is**. But, notice something important: NLL assumes we already know a probability. It requires $P(\textit{correct word})$ as input. But *where does this probability come from?* The network does **not output probabilities**. After the hidden layer (see page 348), the network computes:

$$y = b + Uh$$

Where y is a vector of raw scores (logits) for each word in the vocabulary. To convert these scores into probabilities, we apply the **softmax function**. For example, given:

$$y = [7.2, 3.1, -2.0]$$

Softmax produces:

$$P = [0.983, 0.016, 0.001]$$

Now all values are positive, between 0 and 1, and sum to 1.

Softmax Output

To compute probabilities, the output layer uses **Softmax**. The hidden layer produces a **vector of scores** y for each word in the vocabulary:

$$y_1, y_2, \ldots, y_{|V|}$$

These scores are often called *logits*, *activations*, or *raw scores*. At this stage they are not probabilities. To **convert them into probabilities**, we apply the softmax function:

$$\hat{P}\left(w_i = w_t \mid w_{t-N+1}, \ldots, w_{t-1}\right) = \frac{\exp\left(y_{w_i}\right)}{\sum_{i'=1}^{|V|} \exp\left(y_{w_{i'}}\right)} \tag{140}$$

Computational Complexity

The **complete mathematical description** of the entire neural language model is:

$$y = b + U \cdot \tanh\left(d + Hx\right) \tag{141}$$

Where:

1. First the x vector is the concatenation of context embeddings:

$$x = \left[C\left(w_{t-N+1}\right), C\left(w_{t-N+2}\right), \ldots, C\left(w_{t-1}\right)\right]$$

2. Then, the hidden layer computes:

$$h = \tanh\left(d + Hx\right)$$

   Where H is the hidden layer weight matrix, d is the hidden layer bias vector, and tanh is the hyperbolic tangent activation function.

3. Finally, the output layer computes:

$$y = b + Uh$$

   Where U is the output layer weight matrix, b is the output layer bias vector, and y is the vector of raw scores (logits) for each word in the vocabulary.

4. The softmax function is applied to y to obtain the predicted probabilities for each word in the vocabulary:

$$\hat{P}\left(w_i = w_t \mid w_{t-N+1}, \ldots, w_{t-1}\right) = \frac{\exp\left(y_{w_i}\right)}{\sum_{i'=1}^{|V|} \exp\left(y_{w_{i'}}\right)}$$

Where the parameters C, H, U, d, b are updated during training, which is done using **Stochastic Gradient Descent** (SGD, page 119). Precisely, once the loss is computed:

$$E = -\log P(\text{correct word})$$

We perform (1) forward pass, (2) compute loss, (3) backpropagation, (4) update parameters, using SGD.

And here we discover the **hidden weakness of Bengio's model**. The computational complexity of the training is:

$$O(n \times m + n \times m \times h + h \times |V|) \tag{142}$$

Where:

- **Embedding Lookup** $n \times m$. The variable n is the **context size** (number of previous words), and m is the **embedding dimension** (i.e., the size of the word vectors). So, the embedding lookup requires n lookups (# of words in the context), each of size m (the embedding dimension, length of each word vector). This is **usually not a problem**, as n is small (e.g., 2 or 3) and m is moderate (e.g., 100 or 300). Thus, the embedding lookup is relatively fast.

- **Hidden Layer** $n \times m \times h$. After the embedding lookup, we have n context words. Each embedding has size m. After concatenation $x \in \mathbb{R}^{n \times m}$. Now, the hidden layer needs to compute:

  $$h = \tanh(d + Hx)$$

  Where $H \in \mathbb{R}^{h \times (n \times m)}$ is the hidden layer weight matrix. To compute the product Hx, we need $O(h \times (n \times m))$ operations:

  $$(Hx)_j = \sum_{i=1}^{n \times m} H_{ji} x_i$$

  And here is the **problem**: h is usually large (e.g., 500 or 1000). Thus, the **hidden layer computation can be expensive**, especially when n and m are also large. This is a significant bottleneck in training.

  For example, if we have $n = 5$, $m = 100$, and $h = 500$, then the hidden layer computation requires $5 \times 100 \times 500 = 250,000$ multiplications and additions. A quarter million multiply-adds for a single training example is quite expensive, especially when we have millions of training examples. But the story does not end here.

- **Output Layer** $h \times |V|$. This is the **killer term**. The output layer computes:

  $$y = b + Uh$$

  Where $U \in \mathbb{R}^{|V| \times h}$ is the output layer weight matrix. To compute the product Uh, we need $O(|V| \times h)$ operations:

  $$(Uh)_j = \sum_{i=1}^{h} U_{ji} h_i$$

And here is the **real problem**: $|V|$ is usually very large (e.g., $10,000$ or $100,000$). Thus, the **output layer computation is extremely expensive**, especially when h is also large. This is the final bottleneck in training. For example, if we have $h = 500$ and $|V| = 50,000$, then the output layer computation requires $500 \times 50,000 = 25,000,000$ multiplications and additions. Twenty-five million multiply-adds for a single training example is prohibitively expensive, especially when we have millions of training examples.

Softmax Bottleneck. The issue with computational complexity comes from Softmax, because to compute:

$$P(w_i) = \frac{\exp(y_{w_i})}{\sum_{i'=1}^{|V|} \exp(y_{w_{i'}})}$$

We must compute a score for **every word in the vocabulary**. Even though we only care about one correct word. This becomes extremely expensive when the vocabulary is large. This is known as the **Softmax bottleneck**. This is why Bengio's model was revolutionary but not immediately practical. Therefore, the model solved the curse of dimensionality conceptually, but introduced a new challenge: ***how can we learn embeddings without computing a gigantic Softmax over the entire vocabulary?***

5.4.4 Results and Impact

We have talked about the limitations of N-gram models, Bengio's Neural Language Model, and the architecture and training process. The natural question now is: ***did it actually work?*** The answer is **yes**. Bengio's model demonstrated that neural networks could outperform traditional N-gram language models while simultaneously learning meaningful word representations. This was a turning point in Natural Language Processing.

Experimental Results

The Bengio et al. (2003) paper evaluated their Neural Language Model on several benchmark datasets, including the Brown Corpus and the AP News Corpus.

Network Configurations. The paper used relatively small embedding dimensions compared to modern standards.

- **Brown Corpus**

$$h = 100, \quad n = 5, \quad m = 30$$

- **AP News Corpus**

$$h = 60, \quad n = 6, \quad m = 100$$

Where h is the hidden layer size, n is the N-gram context size, and m is the embedding dimension.

Training Cost. In 2003, training a neural language model was computationally expensive. The paper reported that on AP News Corpus, training took 3 weeks on 40 CPUs (running only 5 epochs!). [1] Today a comparable experiment can often be run in hours or minutes. However, this highlights how computationally expensive neural NLP was in the early 2000s, which was a significant barrier to adoption.

Performance Improvements. Bengio proposed a model that worked significantly better, but was computationally expensive. The paper reported that their Neural Language Model achieved 24% lower perplexity on the Brown Corpus and 8% lower perplexity on the AP News Corpus compared to traditional N-gram models.

> **Deepening: What is perplexity?**
>
> Perplexity is a common evaluation metric for language models. Imagine a multiple-choice exam where we have to choose a word from a set of options to complete a sentence. For example, given the sentence "*The cat is ___.*", and the options are "*sleeping*", "*driving*", "*flying*", and "*quantum*". We immediately think "*sleeping*" is the most likely word to complete the sentence, and our uncertainty is low. Instead, if the options were "*sleeping*", "*eating*", "*running*", and "*hunting*" we might be less certain about which word is correct, and our uncertainty is higher.

Perplexity measures this uncertainty.

A language model is constantly asking "*what is the next word?*", and **Perplexity** measures "*how confused is the model?*" when trying to predict the next word, or similarly, "*how many plausible next words am I considering?*". Higher perplexity means the model has a larger set of plausible next words, while lower perplexity means the model is more confident in its prediction. Therefore, a lower perplexity indicates a better language model.

Why Bengio's result was impressive? Suppose an N-gram model had a perplexity of 200 on a dataset. A 24% reduction in perplexity means the Neural Language Model would have a perplexity of 152, thus the model understands language structure better and can predict the next word more confidently.

Generalization Capability. The most important contribution of Bengio's Neural Language Model was not just the performance improvement, but the fact that it could generalize to unseen N-grams. Traditional N-gram models would assign zero probability to any N-gram not seen during training, while the Neural Language Model could still make reasonable predictions based on learned word embeddings and shared parameters. In one word, **generalization** was the key breakthrough.

Furthermore, one of the most fascinating aspects of Bengio's work is that **nobody explicitly told the network to learn semantic relationships between words**. The objective was simply to *predict the next word*. However, the network learned to represent words in a way that captured their meanings and relationships. For example, the model could learn that "*king*" and "*queen*" are related, and that "*king*" is to "*man* as "*queen*" is to "*woman*". This emergent property of learning meaningful word representations was a significant milestone in NLP.

Limitations of the Neural Network Language Model

Despite its success, Bengio's model had significant weaknesses.

- **Computation Cost**. Training was extremely slow. The giant Softmax over the vocabulary was expensive.
- **Scalability**. The model struggled on very large datasets and vocabularies, which limited its applicability in real-world scenarios.
- **Rare Words**. Words appearing only a few times receive very little training.

However, it introduced the idea of learning distributed word representations. It focused on language-model accuracy, but the learned embeddings became the most influential contribution.

5.5 Word2Vec

5.5.1 From Neural Language Models to Word2Vec

At the end of the previous section we identified the main paradox of Bengio's Neural Language Model: it solved the curse of dimensionality, but it was too computationally expensive to scale. This is where **Word2Vec** enters the story. [12] In 2013, a team at Google led by **Tomas Mikolov** asked a simple but revolutionary question: *do we really need a full language model to learn good word embeddings?* Their answer was a **no**, and they focused directly on learning the word vectors, without the need to learn a full language model.

The New Idea. Mikolov noticed something interesting. When people used Bengio's model, what they really cared about was often not the language model itself. They cared about the learned word vectors. The embeddings were the valuable product. The neural language model was merely a mechanism to obtain them. Therefore, instead of training a large language model to obtain embeddings as a side effect, Word2Vec proposes to directly train specifically to learn embeddings, and forget about building a complex Language Model. This shift completely changed the design philosophy. The quote by linguist John Rupert Firth (1957), *"You shall know a word by the company it keeps"*, became the guiding principle of Word2Vec. The idea was to learn word embeddings by predicting a word based on its context, or vice versa, without the need for a full language model.

For example, imagine a corpus containing sentences such as:

- *Pelé has called Neymar an excellent player*
- *At the age of 22 years, Neymar had scored 40 goals*
- *Neymar was called a true phenomenon*

The word *Neymar* appears surrounded by words like "*player*", "*goals*", "*phenomenon*", "*Pelé*", "*football*". Word2Vec learns the representation of *Neymar* from these neighboring words. The idea is that the context defines the meaning of a word.

Efficiency Improvements. The biggest contribution of Word2Vec was not necessarily better **embeddings**. It was simply a **much faster way to learn** them.

- **Bye Bye Hidden Layer.** Bengio's model contained a hidden layer, which was computationally expensive. The idea of Mikolov was to remove the hidden layer entirely, and make the architecture much shallower.
- **Shared Projection Layer**. In Bengio's model, each word had its own projection layer. Each context position had its own contribution to the hidden layer. Word2Vec instead focuses on learning a single embedding space used throughout the model. Therefore, the emphasis shifts from *learning a complex predictor* to **learning high-quality vectors** (the embeddings). The model is now much simpler, and the training is much faster.

- **Large Contexts**. Another important difference is that Word2Vec uses words from the past and future as context, instead of just the previous words. This allows the model to learn from a larger context window, which can lead to better embeddings.

In general, the Bengio's goal was to get the best possible language model, while Mikolov's goal was to get the **best possible word vectors**. These are related but not identical objectives.

In general, the main idea was to achieve better performance by allowing a simpler (shallower) model to be trained on much larger amounts of data, but without the need to learn a full language model. And this is one of the most important lessons in machine learning: a simpler model trained on much more data can outperform a sophisticated model trained on less data. Word2Vec embraced this philosophy completely.

5.5.2 Word2Vec Architectures

The main innovation of **Word2Vec** is not only that it is much simpler than Bengio's Neural Language Model, but also that it can be **trained in two different ways**, each optimizing a slightly different prediction task. Mikolov et al. proposed two architectures:

- **Continuous Bag-of-Words** (CBOW, page 320);
- **Skip-Gram**.

Although they differ in the prediction objective, they **both learn** the same kind of object: a dense vector (embedding) for every word in the vocabulary.

Why Two Architectures? The central idea of Word2Vec is that a word should have a vector that allows us to predict its surrounding words. There are two possible ways to formulate this learning task. Suppose our sentence is "*The cat eats fresh fish every day*" and we focus on the word "*eats*". We can ask two different questions:

- Given the surrounding words "*The cat*" and "*fresh fish every day*", can we predict the missing word? This is the **Continuous Bag-of-Words (CBOW)** model.
- Instead, we can reverse the problem. Given the center word "*eats*", can we predict the surrounding words? This is the **Skip-Gram** model.

Thus the two **architectures solve opposite prediction problems**. CBOW predicts **one word from many**, while Skip-Gram predicts **many words from one**. This small difference leads to different training behaviors.

Architecture	Input	Output
CBOW	Context words	Target word
Skip-Gram	Target word	Context words

Continuous Bag-of-Words (CBOW)

Continuous Bag-of-Words (CBOW) predicts the **center word** given the surrounding context. Suppose the sentence is "*The cat eats fresh fish*" and we choose a context window of size 2. Then, the context "*The cat*" and "*fresh fish*" should predict the center word "*eats*". The model works with **continuous word vectors** (embeddings), not one-hot vectors, and it **ignores the order of the context words** (thus the name "*bag-of-words*"). For example, the context words "*cat eats fish*" and "*fish eats cat*" would produce exactly the same averaged context representation. The words are treated like a **bag**, not a sequence.

Unlike Bengio's NNLM (Neural Network Language Model), there is **no hidden layer** and **no concatenation of embeddings**. Instead, the context embeddings are simply averaged. Let:

$$C \in \mathbb{R}^{|V| \times m}$$

Be the **shared embedding matrix**, where each row corresponds to the embedding of one word in the vocabulary. Given the context words:

$$w_1, w_2, \ldots, w_C$$

Their embedding vectors are obtained through the lookup operation:

$$C(w_1), C(w_2), \ldots, C(w_C)$$

CBOW computes the **average context representation** as:

$$\bar{\mathbf{v}} = \frac{1}{C} \cdot \sum_{i=1}^{C} C(w_i) \tag{143}$$

Where C is the **number of context words**. The vector $\bar{\mathbf{v}}$ summarizes the semantic information contained in the surrounding context and is then used as the input to the output layer, which predicts the most likely center word using a Softmax over the vocabulary.

The main steps of the CBOW architecture are illustrated in Figure 42:

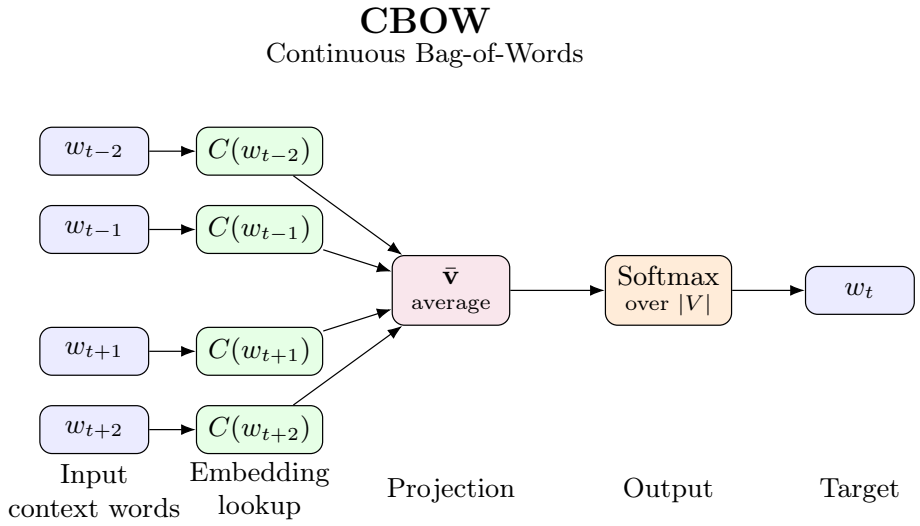
1. Convert each context word into one-hot form.
2. Retrieve its embedding vector from the shared embedding matrix C.
3. Average all embeddings to get the context representation $\bar{\mathbf{v}}$.
4. Use $\bar{\mathbf{v}}$ as input to a Softmax layer that predicts the center word.

This architecture brings **three main advantages**:

- ✔ **Fast training.** Since multiple context words are averaged together, training is computationally efficient.
- ✔ **Stable embeddings.** The averaging operation reduces variance, making optimization easier.
- ✔ **Works well for frequent words.** The averaging operation reduces variance, making optimization easier.

However, some **information is lost** by averaging:

- ✖ **Word order disappears.** The model ignores syntax due to the bag-of-words architecture.
- ✖ **Rare words.** Rare words appear only occasionally as prediction targets. Consequently, CBOW is usually less effective for infrequent vocabulary.



Given the surrounding context words,
CBOW predicts the center word.

$$\bar{\mathbf{v}} = \frac{1}{C}\sum_{i=1}^{C} C(w_i)$$

Figure 42: CBOW architecture in Word2Vec. [12]

Skip-Gram

Skip-Gram reverses the prediction problem. Instead of asking "*which word belongs in the middle?*", it asks "***which surrounding words are likely to appear around this word?***". Here, the model starts from a single word and predicts each context word independently.

Imagine hearing only one word, for example "*football*". Immediately, many related words come to mind: "*ball*", "*goal*", "*team*", "*stadium*", etc. Skip-Gram learns exactly these contextual associations.

Mathematically, given a center word w_t, Skip-Gram **maximizes the probability of observing all neighboring context words**:

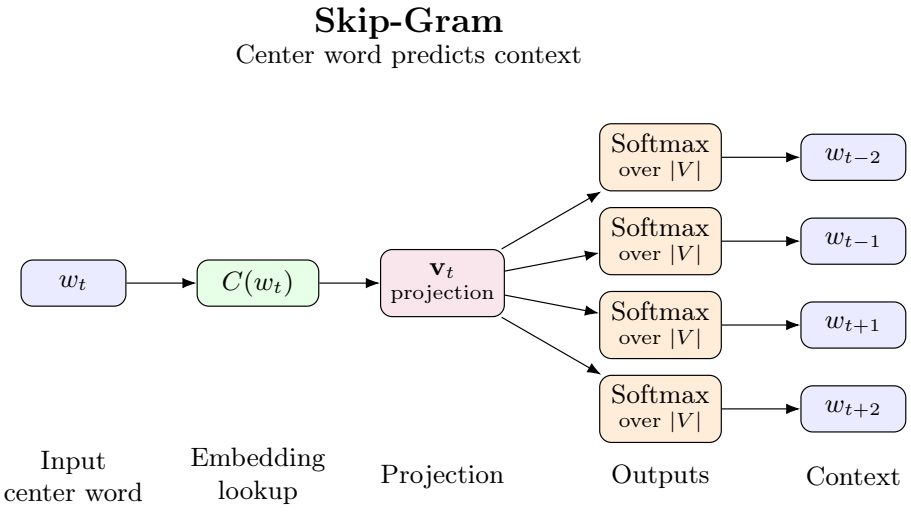
$$P\left(\text{context} \mid w_t\right) = \prod_{j=-k}^{k} P\left(w_{t+j} \mid w_t\right) \quad j \neq 0 \tag{144}$$

Where k is the context window and each surrounding word w_{t+j} is predicted independently from the center word w_t. The Skip-Gram architecture is illustrated in Figure 43.

The **advantages** of Skip-Gram are the downside of CBOW:

- ✔ **Better for rare words.** Every occurrence of a word generates several training examples, so even infrequent words receive stronger learning signals.
- ✔ **Better semantic representations.** Empirically, Skip-Gram often learns higher-quality embeddings, especially for semantic relationships and analogies. Because it focuses on predicting context words, it captures richer semantic associations.

⚠ However, Skip-Gram has one main disadvantage: **computational cost**. Each center word requires predicting multiple context words, making training slower than CBOW.



Given the center word, Skip-Gram predicts the surrounding context words.

$$w_t \longrightarrow \{w_{t-c}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+c}\}$$

Figure 43: Skip-Gram architecture in Word2Vec. [12]

⚖ CBOW vs Skip-Gram, which one is better?

There is no universally superior architecture. In practice, **CBOW** is preferred when training speed is important and the corpus is very large. **Skip-Gram** is generally preferred when the goal is to learn the highest-quality embeddings, especially for rare words and semantic relationships.

Despite these differences, both architectures learn embeddings using the **same underlying principle**: words that appear in similar contexts gradually acquire similar vector representations in the embedding space. This shared objective is what allows Word2Vec to capture meaningful semantic and syntactic relationships between words.

Continuous Bag-of-Words (CBOW)	**Skip-Gram**
Context → Target	Target → Context
Input: surrounding words	Input: center word
Output: center word	Output: surrounding words
Faster training	Slower training
Better for frequent words	Better for rare words
Averages context embeddings	Predicts each neighbor separately
Simpler optimization	Richer learning signal

Table 7: Comparison of CBOW and Skip-Gram architectures in Word2Vec.

5.5.3 Continuous Bag-of-Words (CBOW)

The **Continuous Bag-of-Words (CBOW)** architecture is the first of the two Word2Vec models proposed by Mikolov et al. (2013). [12] Its goal is straightforward: given the surrounding words (the context), predict the missing (central) word. Unlike Bengio's Neural Language Model, CBOW removes the hidden layer and greatly simplifies the network, allowing it to be trained on billions of words while still learning high-quality word embeddings.

Suppose our sentence is:

"*The quick brown fox jumps over the dog.*"

And we choose a context window of size 2. Our training example becomes:

Input: {"the", "brown", "jumps", "over"}
Output: "fox"

Thus, instead of learning:

$$P\left(w_t \mid w_{t-2}, w_{t-1}\right)$$

As Bengio's model did, CBOW learns:

$$P\left(w_t \mid w_{t-c}, \ldots w_{t-1}, w_{t+1}, \ldots, w_{t+c}\right)$$

Where w_t is the central word, and c is the **context radius** $\frac{n}{2}$, where n is the **total number of context words**. In our example, the context words are "the", "brown", "jumps", and "over" (two previous and two future words) and the target word is "fox". Notice one important improvement over Bengio's model: Bengio used **previous words**, while CBOW uses **both previous and future words**, giving more information to predict the target.

Context Aggregation

The key idea of CBOW is how it combines the context words. Suppose our vocabulary is represented by the embedding matrix:

$$C \in \mathbb{R}^{|V| \times m}$$

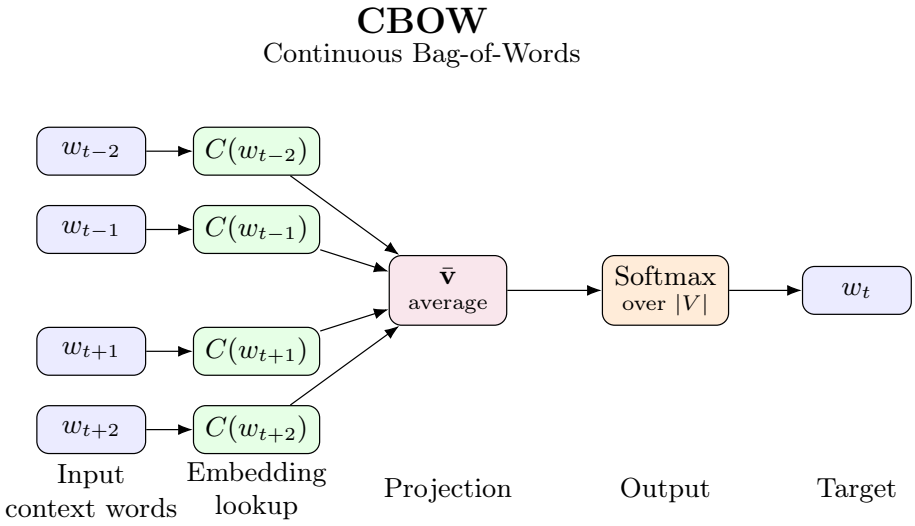
Where $|V|$ is the vocabulary size and m is the embedding dimension. Each context word w_i performs an embedding lookup in C to retrieve its corresponding vector:

$$C\left(w_i\right) \in \mathbb{R}^m$$

Unlike Bengio's model, **these vectors are not concatenated**. Instead, CBOW computes their average:

$$\bar{\mathbf{v}} = \mathbf{h} = \frac{1}{C} \cdot \sum_{i=1}^{C} C\left(w_i\right) \tag{145}$$

Where C is the number of context words. This average vector $\bar{\mathbf{v}}$ is the representation of the **entire context**.



Given the surrounding context words,
CBOW predicts the center word.

$$\bar{\mathbf{v}} = \frac{1}{C}\sum_{i=1}^{C} C(w_i)$$

Figure 44: CBOW architecture in Word2Vec. [12]

Why average? Suppose the context is "*coffee*", "*morning*", "*drink*". Their embeddings might encode different semantic aspects:

- *coffee* → *beverage, caffeine, hot*
- *morning* → *time, day, early*
- *drink* → *consume, liquid, beverage*

Averaging them produces a vector roughly representing "*something people drink in the morning*". This vector is then used to predict the missing word.

Output Prediction. The averaged vector is fed directly into the output layer, which computes the probability of every word in the vocabulary. The predicted word is:

$$\hat{w} = \arg\max_{w} P(w \mid \text{context})$$

During training, the correct target word is known, so the prediction error is computed and the embeddings are updated through backpropagation.

Which Embeddings are updated? One interesting property of CBOW is that only the embeddings of the context words are updated. Suppose the training example is:

Input: {"the", "brown", "jumps", "over"}
Target: "fox"

The model retrieves the embeddings of "the", "brown", "jumps", and "over" from the embedding matrix C. These four embeddings are averaged to produce the prediction. After computing the prediction error, **only these input (context) embeddings are modified** to better predict *fox*. The target word itself is represented in the output weight matrix (often denoted D' or C'), which is updated separately during training.

Computational Complexity

Because CBOW removes the hidden layer, its computational cost is **dramatically lower than Bengio's NNLM**. For each training example, the complexity is approximately:

$$O\left(n \cdot m + m \cdot \log |V|\right) \tag{146}$$

Where:

- n is the number of context words;
- m is the embedding dimension;
- $|V|$ is the vocabulary size.

Compared to Bengio's complexity:

$$O\left(n \cdot m + n \cdot m \cdot h + h \cdot |V|\right)$$

The expensive **hidden-layer computations disappear**, making Word2Vec roughly three orders of magnitude faster to train on large corpora.

5.5.4 CBOW Training and Weight Updates

Suppose we have the sentence:

"*The quick brown fox jumps over the lazy dog*"

With context size $n = 4$, thus $c = \frac{4}{2} = 2$ radius. Our training pair is:

$$(\underbrace{\textit{quick}, \textit{brown}, \textit{jumps}, \textit{over}}_{\text{context}}, \textit{fox})$$

Or, mathematically:

$$(\underbrace{w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2}}_{\text{context}}, w_t)$$

Where w_t is the target word, and $w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2}$ are the context words. The goal of the network is to answer the question: "***given these four words, what is the missing one?***".

The Objective Function. Since the correct answer is known, we can measure how good the prediction is. Thus, the network computes:

$$P\left(w_t \mid \text{context}\right)$$

Namely, the probability that the central word is w_t. For our example, where the context is:

"*quick*, *brown*, *jumps*, *over*"

The ideal output of the network would be:

Word	**Probability**
fox	1.0
wolf	0.0
dog	0.0
cat	0.0
⋮	⋮

Obviously, at the beginning of the training, the probabilities will be:

Word	**Probability**
fox	0.12
wolf	0.18
dog	0.10
cat	0.09
⋮	⋮

So we define the classic loss function as the **negative log-likelihood** of the correct word (like in the Bengio et al. paper, page 349):

$$E = -\log P\left(w_t \mid \text{context}\right) = -\log P\left(w_t \mid w_{t-\frac{n}{2}}, w_{t-1}, w_{t+1}, w_{t+\frac{n}{2}}\right) \tag{147}$$

The reason we use negative log likelihood is the same as in the case of Neural Network Language Models, page 349. The only difference is that now we are **trying to predict the central word given the context**, instead of predicting the next word given the previous ones.

Training Computations

As usual, we perform a forward pass to compute the output probabilities, and then a backward pass to compute the gradients and update the weights.

→ Forward Pass

1. **Input.** The context words arrive as one-hot vectors. For example, taking the context words "*quick*, *brown*, *jumps*, *over*", we have:

$$\mathbf{x}_{\text{quick}} = \begin{bmatrix}1\\0\\0\\\vdots\\0\end{bmatrix} \quad \mathbf{x}_{\text{brown}} = \begin{bmatrix}0\\1\\0\\\vdots\\0\end{bmatrix} \quad \mathbf{x}_{\text{jumps}} = \begin{bmatrix}0\\0\\1\\\vdots\\0\end{bmatrix} \quad \mathbf{x}_{\text{over}} = \begin{bmatrix}0\\0\\0\\\vdots\\1\end{bmatrix}$$

2. **Embedding Lookup.** Each one-hot vector selects one row of the embedding matrix C giving us the corresponding word embedding. For example, if C is the embedding matrix, then:

$$\mathbf{e}_{\text{quick}} = C^T\mathbf{x}_{\text{quick}} \quad \mathbf{e}_{\text{brown}} = C^T\mathbf{x}_{\text{brown}}$$
$$\mathbf{e}_{\text{jumps}} = C^T\mathbf{x}_{\text{jumps}} \quad \mathbf{e}_{\text{over}} = C^T\mathbf{x}_{\text{over}}$$

   This gives us the embeddings for the context words.

3. **Average.** We average the embeddings of the context words to get a single vector representation of the context:

$$\mathbf{e}_{\text{context}} = \frac{1}{n}\sum_{i=1}^{n}\mathbf{e}_{w_i} = \frac{1}{4}\left(\mathbf{e}_{\text{quick}} + \mathbf{e}_{\text{brown}} + \mathbf{e}_{\text{jumps}} + \mathbf{e}_{\text{over}}\right)$$

   This is the "*bag of words*" part, where we ignore the order of the context words.

4. **Output scores.** Now we compute the scores for each word in the vocabulary by multiplying the context embedding with the output weight matrix C'. As can be seen in the architecture on page 364, the **output weight matrix** C' is not explicitly drawn, but it links the average embedding to the output layer. It contains one vector **for every vocabulary word**. The scores are computed as:

$$\mathbf{u} = C'\mathbf{e}_{\text{context}}$$

   Where $\mathbf{u}$ is a vector of scores for each word in the vocabulary. For example:

$$u_{\text{fox}} = \mathbf{c}'_{\text{fox}} \cdot \mathbf{e}_{\text{context}} = 3.7 \quad u_{\text{wolf}} = \mathbf{c}'_{\text{wolf}} \cdot \mathbf{e}_{\text{context}} = 2.9$$
$$u_{\text{dog}} = \mathbf{c}'_{\text{dog}} \cdot \mathbf{e}_{\text{context}} = 1.2 \quad u_{\text{cat}} = \mathbf{c}'_{\text{cat}} \cdot \mathbf{e}_{\text{context}} = 0.4$$

5. **Softmax.** It transforms scores into probabilities. The softmax function is applied to the scores vector **u** to get the predicted probabilities for each word in the vocabulary:

$$P(w_i \mid \text{context}) = \frac{\exp(u_i)}{\sum_{j=1}^{V} \exp(u_j)}$$

Where V is the size of the vocabulary. For example:

$$P(\text{fox} \mid \text{context}) = 0.81 \quad P(\text{wolf} \mid \text{context}) = 0.09$$
$$P(\text{dog} \mid \text{context}) = 0.05 \quad P(\text{cat} \mid \text{context}) = 0.01$$

The network predicts *fox*. If correct, great! If not, an **error** is produced and the **backpropagation algorithm is used to update** the weights of the network.

Which parameters are updated? Only the rows of the **embedding matrix** C that correspond to the context words, **not every word** in the vocabulary. In other words, for each $(context, target)$ pair only the context words are updated.

Similarly, only the row of the **output weight matrix** C' that corresponds to the target word is updated.

Geometric Intuition. Geometrically, the context representation is moved **away from overestimated wrong words** and **toward the correct target word**. This is illustrated in Figure 45. For example, if the context is "*quick*, *brown*, *jumps*, *over*" and the target word is "*fox*", but the network predicts "*wolf*":

- Case 1. Suppose $P(wolf \mid \text{context})$ is too high (overestimated). Then, the gradient (computed during backpropagation), roughly speaking, says "*make this context* ***less similar*** *to wolf*". Therefore, a small portion of $C'(wolf)$ is ***subtracted*** from the context embeddings. Geometrically, the context vectors move **away from the wrong word** "*wolf*".
- Case 2. Suppose $P(fox \mid \text{context})$ is too low (underestimated). Then, the gradient says "*move the context* ***closer*** *to fox*". Therefore, a small portion of $C'(fox)$ is ***added*** to the context embeddings. Geometrically, the context vectors move **toward the correct word** "*fox*".

Therefore, the CBOW training algorithm is simply a standard forward pass followed by backpropagation, with the only distinctive feature being that the learned parameters are the **word embedding matrix** C and the **output weight matrix** C'. The training process gradually organizes the embedding space, so that words that appear in similar contexts end up with similar embeddings.

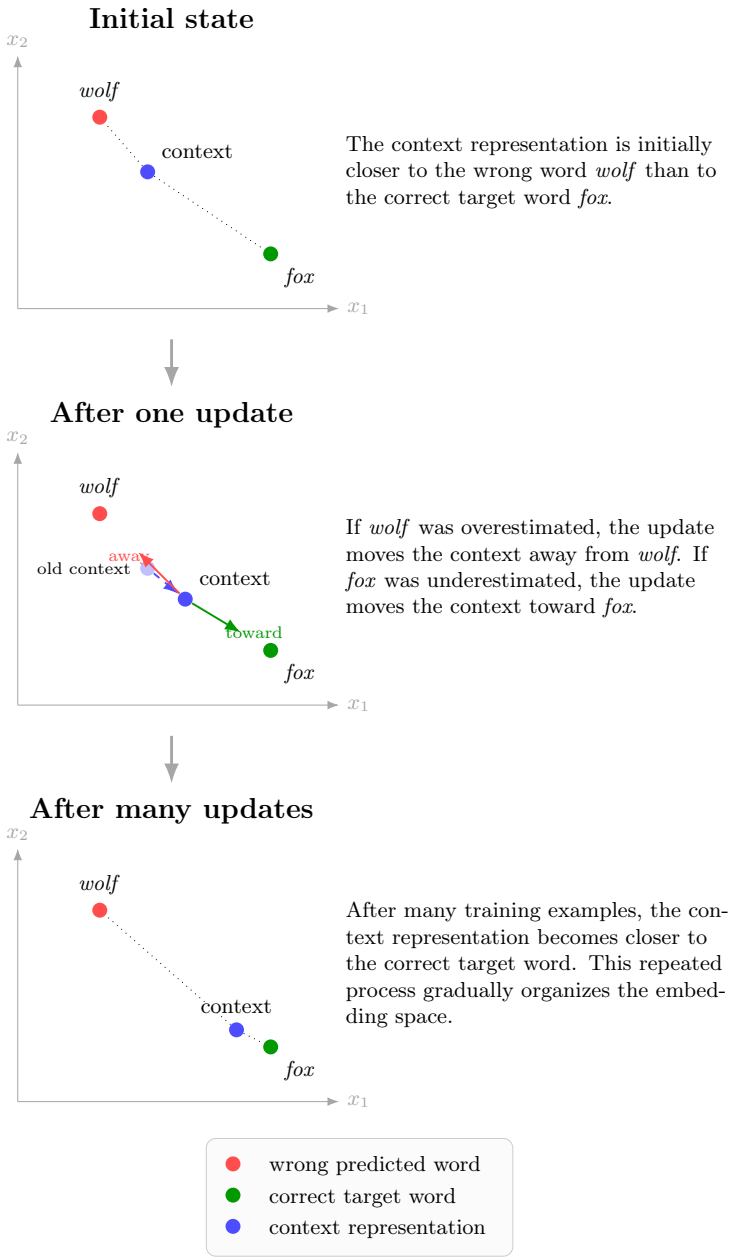


Figure 45: **Geometric intuition** of CBOW training. The context representation is moved away from overestimated wrong words and toward the correct target word.

5.5.5 Word2Vec Facts

The introduction of **Word2Vec** marked a turning point in Natural Language Processing because it demonstrated that **simpler neural architectures, when trained on massive datasets, can outperform much more complex language models**. Rather than focusing on building deeper neural networks, Mikolov et al. concentrated on making training extremely efficient, allowing the model to learn from billions of words.

Training Speed

One of the greatest strengths of Word2Vec is its **training efficiency**. Compared to Bengio's Neural Network Language Model (NNLM), Word2Vec removes the hidden layer and greatly simplifies the computations performed for each training example. As a result, the computational complexity decreases from:

$$\mathcal{O}\left(nm + nmh + h\left|V\right|\right)$$

To approximately:

$$\mathcal{O}\left(nm + m\log\left|V\right|\right)$$

Where n is the context size, m is the embedding dimension, V is the vocabulary size. The expensive hidden-layer computations disappear, leaving only the embedding lookup and the output prediction. This seemingly simple modification reduces the computational cost dramatically.

The improvement is not merely theoretical. According to the original experiments reported by Mikolov, Word2Vec processes approximately $100,000$ to $5,000,000$ words **every second**. This made it feasible, for the first time, to train high-quality embeddings on web-scale corpora containing billions of words.

Scalability. Because of its computational efficiency, Word2Vec scales extremely well. Instead of being limited to relatively small corpora, it can exploit: millions of documents, billions of words, vocabularies containing millions of unique terms. As the amount of available text increases, the learned embeddings generally improve because the model observes many more contexts for each word.

Therefore, rather than increasing the complexity of the neural network, Word2Vec increases the amount of data used for learning. Modern NLP follows exactly this philosophy: **large datasets often compensate for simpler architectures**.

Large Corpora. The scalability of Word2Vec is evident from the experiments reported in the original paper. The model was trained on the **Google News** corpus containing approximately **6 billion words** and a vocabulary of roughly $|V| \approx 10^6$ unique words. Despite this enormous dataset, training required only:

- **CBOW**: about **2 days** on **140 CPU cores**.
- **Skip-gram**: about **2.5 days** on **125 CPU cores**.

For comparison, Bengio's original Neural Network Language Model required: **14 days**, **180 CPU cores**, and an embedding size of only **100 dimensions**. Furthermore, the Word2Vec was trained using a much larger embedding size of 1000 dimensions, which would have been computationally prohibitive for the original NNLM.

Accuracy Improvements. Perhaps the most surprising result was that Word2Vec was **not only faster**, but also **more accurate**.

Model	Accuracy
Best Neural Network Language Model (Bengio)	12.3%
Word2Vec (Skip-Gram)	53.3%

Thus, Word2Vec achieved both **dramatically faster training** and **substantially higher predictive performance**, demonstrating that learning high-quality word embeddings from massive corpora can be more beneficial than using a deeper, more computationally expensive language model.

In general, Word2Vec's impact extends far beyond its original architecture. Its key contributions include:

- Demonstrating that **distributional semantics** ("*a word is known by the company it keeps*") can be learned automatically from raw text;
- Showing that **simple shallow networks** can outperform deeper models when trained on enough data;
- Producing embeddings that capture rich semantic and syntactic relationships;
- Making large-scale unsupervised representation learning practical.

Perhaps most importantly, Word2Vec shifted the community's attention from **predicting the next word** to **learning high-quality vector representations**. These embeddings became the foundation for numerous subsequent NLP models, including GloVe, FastText, ELMo, BERT, and many modern transformer-based language models.

5.6 Semantic Regularities in Embedding Spaces

5.6.1 Semantic Relationships

After training Word2Vec on a sufficiently large corpus, the learned embedding space exhibits an unexpected and remarkable property: semantic relationships between words become encoded as vector directions.

Recall that each word is represented by a dense vector:

$$\mathbf{w} \in \mathbb{R}^m$$

Where nearby vectors correspond to words that tend to appear in similar contexts.

Initially, researchers expected only that semantically similar words would form clusters, for example:

- *dog*, *wolf*, *cat*;
- *France*, *Germany*, *Italy*;
- *eat*, *drink*, *cook*.

However, Word2Vec revealed something much stronger: many semantic relationships correspond to approximately constant vector offsets throughout the embedding space. In other words, not only are similar words close together, but the **difference vector** between two related words is often nearly the same regardless of the specific words involved. This phenomenon is referred to as a **semantic regularity** in the embedding space.

√× Geometric Interpretation

Suppose each word is represented as a point in a high-dimensional vector space. Instead of looking only at the positions of the points, we can also consider the **vector connecting two words**. For two words a and b, this relationship is represented by the vector:

$$\overrightarrow{ab} = \mathbf{w}_b - \mathbf{w}_a$$

If two different pairs of words express the same semantic relationship, their difference vectors tend to be almost parallel and have similar magnitudes. This means that semantic information is encoded not only in the location of individual words but also in the **directions** connecting them.

Country-Capital Relationships

One of the first observations made by Mikolov and colleagues concerns country-capital pairs. For example:

$$\begin{aligned}\mathbf{w}_{Paris} - \mathbf{w}_{France} &\approx \mathbf{w}_{Rome} - \mathbf{w}_{Italy} \\ &\approx \mathbf{w}_{Berlin} - \mathbf{w}_{Germany} \\ &\approx \mathbf{w}_{Madrid} - \mathbf{w}_{Spain} \\ &\approx \mathbf{w}_{Lisbon} - \mathbf{w}_{Portugal}\end{aligned}$$

Although these countries occupy **different regions of the embedding space**, the vectors connecting each country to its capital point in **approximately the same direction**.

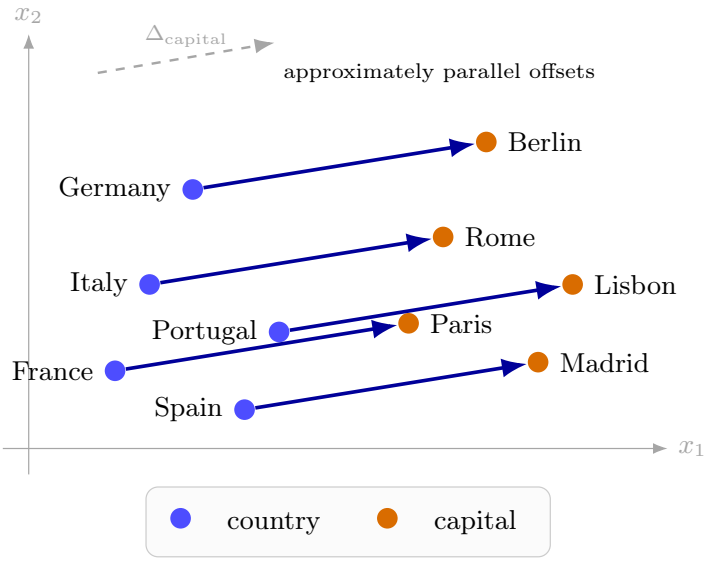


Figure 46: Geometric interpretation of semantic regularities in Word2Vec. Country–capital relationships are represented by approximately parallel vector offsets in the embedding space.

Why does this happen? The network is **never explicitly told** that Paris is the capital of France or that Rome is the capital of Italy. Instead, it repeatedly observes contexts such as:

- *Paris is the capital of France.*
- *Rome is the capital of Italy.*
- *Madrid is the capital of Spain.*

As these contextual patterns appear throughout the corpus, gradient updates gradually arrange the embeddings so that the same **semantic relationship is represented by similar vector offsets**. Consequently, the notion of "*capital of*" emerges automatically from the training data.

King-Queen Relationships. Another famous example concerns **gender relationships**. The vector:

$$\mathbf{w}_{\text{king}} - \mathbf{w}_{\text{queen}}$$

Is approximately parallel to:

$$\mathbf{w}_{\text{prince}} - \mathbf{w}_{\text{princess}}$$

And similarly to:

$$\mathbf{w}_{\text{man}} - \mathbf{w}_{\text{woman}}$$

Has nearly the same direction. Again, Word2Vec has not been taught what "*gender*" means. It only learns from contextual information, yet this semantic concept naturally emerges as a consistent direction in the embedding space.

Why are these relationships important?

These semantic regularities demonstrate that Word2Vec learns **more than word similarity**. Earlier embedding methods primarily captured the idea that *similar words are close together*. Word2Vec goes one step further: relationships between words become linear transformations in the embedding space. This discovery showed that semantic knowledge can be represented geometrically, allowing simple vector operations to reveal complex linguistic relationships. This observation directly motivates the next section, where we will see that these vector relationships make it possible to solve analogies such as:

$$\mathbf{w}_{\text{king}} - \mathbf{w}_{\text{man}} + \mathbf{w}_{\text{woman}} \approx \mathbf{w}_{\text{queen}}$$

Using nothing more than vector arithmetic.

5.6.2 Linear Structure of Embedding Spaces

One of the most remarkable discoveries of Word2Vec is that the embedding space exhibits a **linear structure**. This means that semantic relationships are not only represented by vector directions, but they can also be **combined through simple vector arithmetic**.

Suppose every word is represented by an embedding vector:

$$\mathbf{w} \in \mathbb{R}^m$$

where m is the dimensionality of the embedding space. Since these vectors live in a continuous vector space, we can perform the usual vector operations: addition, subtraction, scalar multiplication. Initially, this might seem like a purely mathematical property. However, after training, these operations acquire a surprising **linguistic meaning**.

√× Vector Arithmetic

In the previous section, we observed that:

$$\mathbf{w}_{\text{Paris}} - \mathbf{w}_{\text{France}} \approx \mathbf{w}_{\text{Rome}} - \mathbf{w}_{\text{Italy}}$$

This tells us that the vector:

$$\mathbf{w}_{\text{Paris}} - \mathbf{w}_{\text{France}}$$

Encodes the semantic relation "*capital of*". Now suppose we want to answer the following question: ***what is the capital of Italy?***. Starting from the known relationship:

$$\mathbf{w}_{\text{Paris}} - \mathbf{w}_{\text{France}} \approx \mathbf{w}_{\text{Rome}} - \mathbf{w}_{\text{Italy}}$$

We simply **apply the same semantic transformation** to the vector of *Italy*:

$$\mathbf{w}_{\text{Paris}} - \mathbf{w}_{\text{France}} + \mathbf{w}_{\text{Italy}} \approx \mathbf{w}_{\text{Rome}}$$

This operation can be interpreted as:

- Remove the concept of "*France*" from the vector of "*Paris*";
- Keep the concept of "*capital*" (the remaining part of the vector);
- Add the concept of "*Italy*" to the vector.

The nearest embedding to the resulting vector is "*Rome*".

In general, we can express this operation as:

$$\mathbf{w}_B - \mathbf{w}_A + \mathbf{w}_C \approx \mathbf{w}_D \tag{148}$$

Where D is the word whose embedding is closest to the resulting vector. Rather than searching for an exact equality, the model searches for the **nearest neighbor** according to a similarity measure (typically cosine similarity).

Why does this work?

It is important to understand that Word2Vec **does not contain explicit symbolic knowledge**. It has never been programmed with rules such as:

- Paris is the capital of France;
- A king is a male monarch;
- Copper has chemical symbol Cu.

Instead, these relationships emerge because words that participate in similar semantic patterns appear in similar contexts throughout the training corpus. Consequently, gradient descent gradually arranges the embedding vectors so that many semantic transformations become approximately linear.

Limitations

Although these analogies are impressive, they are only **approximate**. The equations are **not exact identities**:

$$\mathbf{w}_{\text{Paris}} - \mathbf{w}_{\text{France}} + \mathbf{w}_{\text{Italy}} \neq \mathbf{w}_{\text{Rome}}$$

Instead, the resulting vector is simply **very close** to the embedding of *Rome*. The final answer is obtained by finding the word whose embedding has the highest similarity (typically measured with cosine similarity). Moreover, not every semantic relationship is linear. Some concepts are much more complex and cannot be captured by a single constant vector offset.

5.7 Applications of Word Embeddings

So far, we have focused on **how word embeddings are learned**. We introduced Word2Vec, studied its CBOW and Skip-Gram architectures, analyzed the training process, and observed that the resulting embedding space exhibits remarkable semantic regularities, such as meaningful vector offsets and word analogies. The next natural question is: ***once these embeddings have been learned, how can they be used?***

One of the greatest strengths of Word2Vec is that the learned embeddings are **general-purpose vector representations**. After training, the neural network itself is no longer needed; instead, the embedding matrix can be reused in many downstream Natural Language Processing (NLP) tasks. Each word is simply replaced by its corresponding embedding vector, providing a dense semantic representation that can be exploited by other algorithms.

In this section, we explore some of the earliest and most influential applications of word embeddings, including **Information Retrieval** and **Document Classification/Similarity**. These applications demonstrate how replacing sparse one-hot representations with dense semantic vectors enables systems to compare texts based on their *meaning* rather than relying solely on exact word matching.

5.7.1 Information Retrieval

One of the earliest and most influential applications of word embeddings is **Information Retrieval (IR)**. Information Retrieval is the field concerned with **finding documents that are relevant to a user's query**. For example, when we type "*restaurants in mountain view that are not very good*" into a search engine, we expect to retrieve restaurants with **poor reviews** in Mountain View, even if the documents do not contain the same words.

Traditional IR

Classical search engines typically represent both the query and the documents, using **Bag-of-Words** (BoW, page 320) or TF-IDF[14] vectors.

For example, the query:

"*restaurants in mountain view that are not very good*"

Might be represented simply as a collection of independent words:

[14] **Term Frequency-Inverse Document Frequency (TF-IDF)** is one of the most important classical techniques in **Information Retrieval (IR)**. Before neural embeddings like Word2Vec, it was the standard way to represent documents for search engines and document classification. The main idea is simple: a word is important for a document if it appears frequently in that document, but not frequently in all documents. So TF-IDF assigns each word a weight rather than simply recording whether the word appears. For example, if the word "*the*" appears in every document, it would have a low TF-IDF score, while a more unique word like "*quark*" would have a higher score in documents where it appears. This allows search engines to better capture the relevance of documents to a query, as it emphasizes the unique and informative words rather than common ones.

restaurants, *mountain*, *view*, *not*, *very*, *good*

The problem is that this representation ignores semantics. For instance:

- *restaurant* and *cafe* are treated as unrelated words;
- *bad*, *poor*, and *terrible* are considered completely different terms;
- Documents containing synonyms may never be retrieved, even if they are relevant to the query.

The retrieval system therefore depends heavily on **exact word matching**.

Using Word Embeddings

Word embeddings provide a richer representation. Instead of representing each word as a sparse one-hot vector:

$$w \longrightarrow \mathbf{w} \in \mathbb{R}^m$$

Each word is represented by a dense embedding. Consequently, an entire query can also be represented by combining the embeddings of its words. For the example query:

"*restaurants in mountain view that are not very good*"

The process becomes:

1. **Query** (the user's input): "*restaurants in mountain view that are not very good*".
2. **Phrase decomposition**:

$$\textit{restaurants} \qquad (\textit{mountain view}) \qquad (\textit{not very good})$$

3. **Embedding representation**:

$$\mathbf{w}_{restaurants} + \mathbf{w}_{in} + \mathbf{w}_{(mountain\ view)} + \mathbf{w}_{that} + \mathbf{w}_{are} + \mathbf{w}_{(not\ very\ good)}$$

Where every element is represented by an embedding vector rather than by a one-hot encoding.

How to build the query vector?

Suppose the embeddings are:

$$\mathbf{w}_{restaurants}, \quad \mathbf{w}_{mountain\ view}, \quad \mathbf{w}_{bad}$$

A simple representation of the whole query is obtained by averaging (or summing) the embeddings:

$$\mathbf{q} = \frac{1}{n} \cdot \sum_{i=1}^{n} \mathbf{w}_i \tag{149}$$

Where n is the **number of words** (or phrases) in the query, and $\mathbf{w}_i$ is the **embedding** of the i-th **word** produced by a model like Word2Vec. The resulting vector $\mathbf{q}$ acts as a **semantic representation of the entire query**.

Retrieving relevant documents

Now that we have a valid vector representation of the query, we need to understand how to retrieve relevant documents.

Each document can be represented in exactly the same way:

$$\mathbf{d}_1, \mathbf{d}_2, \ldots, \mathbf{d}_N$$

To determine which document best matches the query, the search engine computes a similarity measure, typically the **Cosine Similarity**:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \cdot \|\mathbf{d}\|}$$

Documents whose vectors are closest to the query vector are returned first (i.e., those with the highest cosine similarity). Unlike Bag-of-Words or TF-IDF, this comparison is based on **semantic similarity** rather than exact word overlap. For example, a document containing the phrase "*poor reviews*" would be considered relevant to the query "*not very good*", even though the words do not match exactly.

Definition 4: Cosine Similarity

When representing queries and documents as vectors, we need a way to measure **how similar two vectors are**. A natural idea would be to compute the Euclidean distance between them. However, this is often not appropriate for text because the **length (magnitude) of a vector is usually less important than its direction**. Instead, Information Retrieval systems commonly use the **Cosine Similarity**, which measures the cosine of the angle between two vectors.

Mathematically, given two vectors:

$$\mathbf{q}, \mathbf{d} \in \mathbb{R}^m$$

Representing a query and a document, respectively, the cosine similarity is defined as:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \cdot \|\mathbf{d}\|} \tag{150}$$

Where:

- $\mathbf{q} \cdot \mathbf{d}$ is the dot product of the two vectors, which measures their alignment.
- $\|\mathbf{q}\|$ is the Euclidean norm (length) of the query vector.
- $\|\mathbf{d}\|$ is the Euclidean norm (length) of the document vector.

Why does it work? Recall from linear algebra that the dot product of two vectors can be expressed as:

$$\mathbf{q} \cdot \mathbf{d} = \|\mathbf{q}\| \cdot \|\mathbf{d}\| \cdot \cos\theta$$

Where θ is the angle between the two vectors. Dividing both sides by the vector norms gives:

$$\frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \cdot \|\mathbf{d}\|} = \cos\theta$$

Therefore, cosine similarity simply measures **how aligned the two vectors are**.

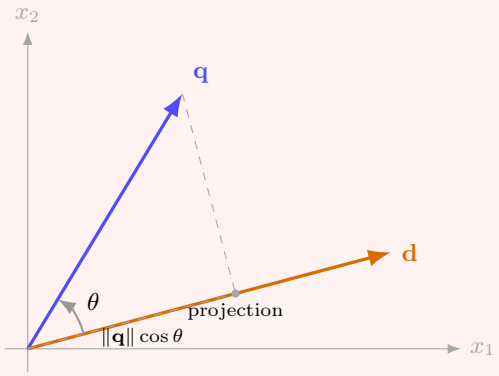
Interpretation. The cosine similarity always lies between:

$$-1 \leq \text{sim}\,(\mathbf{q}, \mathbf{d}) \leq 1$$

Where:

- $\text{sim}\,(\mathbf{q}, \mathbf{d}) = 1$ indicates that the vectors point in exactly the **same direction** (maximum similarity).
- $\text{sim}\,(\mathbf{q}, \mathbf{d}) = 0$ indicates that the vectors are orthogonal, meaning they share **no directional similarity**.
- $\text{sim}\,(\mathbf{q}, \mathbf{d}) = -1$ indicates that the vectors point in **opposite directions** (maximum dissimilarity).

In the context of word embeddings, values close to 1 indicate that the query and the document have very similar semantic representations.



Cosine similarity measures the angle between two vectors:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\|\,\|\mathbf{d}\|} = \cos\theta$$

$\theta \approx 0°$	$\Rightarrow$	high similarity
$\theta \approx 90°$	$\Rightarrow$	no directional similarity
$\theta \approx 180°$	$\Rightarrow$	opposite direction

Why not use Euclidean distance? Suppose we compare two

vectors using Euclidean distance:

$$\mathbf{q} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{d} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

The Euclidean distance between $\mathbf{q}$ and $\mathbf{d}$ is:

$$\|\mathbf{q} - \mathbf{d}\| = \sqrt{(1-2)^2 + (2-4)^2} = \sqrt{1+4} = \sqrt{5}$$

Which is **not zero**, even though the vectors point in exactly the same direction.

Why is this better than traditional IR?

Suppose a user searches for:

"*good restaurant*"

But a document contains:

"*excellent cafe*"

A Bag-of-Words model may assign a very low similarity because none of the words match exactly. Instead, with word embeddings:

- *restaurant* is semantically similar to *cafe*;
- *good* is semantically similar to *excellent*.

Therefore, the document vector remains close to the query vector, allowing the system to retrieve relevant documents even when different words are used. This is one of the **major advantages of dense vector representations**.

Limitations

Although this approach is simple and computationally efficient, it has important limitations. It works reasonably well for **short queries**, but it does **not scale well to long sentences or entire documents**. The reason is that averaging many word embeddings removes important information:

- Word order is lost, so "*not very good*" and "*very good not*" would have the same representation;
- Syntactic structure disappears, so "*not very good*" and "*very good not*" would have the same representation;
- Different meanings may be mixed into a single vector, so "*bank*" in the context of a river and "*bank*" in the context of finance would be averaged together.

A simple average of the embeddings would produce almost the same document vector, even though the meanings are opposite. This illustrates why later architectures, such as **Recurrent Neural Networks (RNNs)**, **Attention Mechanisms**, and **Transformers**, were developed to model the relationships between words rather than treating them as an unordered set.

5.7.2 Document Classification and Similarity

After seeing how word embeddings can improve **Information Retrieval (IR)**, another important application is **Document Classification and Similarity**. The central idea is simple: if words have semantic embeddings, then entire documents can also be represented using those embeddings. This allows us to compare documents based on **their meaning**, rather than only on the exact words they contain.

⚠ The problem with Bag-of-Words (BoW)

Recall the Bag-of-Words representation. Suppose we have three documents:

- **Document** D_0

  "*The President greets the press in Chicago.*"

- **Document** D_1

  "*Obama speaks to the media in Illinois.*"

- **Document** D_2

  "*The band gave a concert in Japan.*"

With Bag-of-Words, every word is treated as an independent symbol. Therefore, the vectors of D_0 and D_1 share very few identical words. Surprisingly, D_2 also shares very few identical words. Consequently, Bag-of-Words considers:

$$\text{sim}\,(D_0, D_1) \approx \text{sim}\,(D_0, D_2)$$

Which is clearly **wrong**. But, *whi is this wrong?* As humans, we immediately understand that "*Obama*" is closely related to "*President*". Similarly, "*speaks*" and "*greets*" are semantically related. Likewise, "*Illinois*" and "*Chicago*" refer to the same geographical area. The meaning is almost identical even though the words are different.

✅ Word Embeddings solve this

Suppose Word2Vec has learned:

- *Obama* $\rightarrow \mathbf{w}_\text{A}$
- *President* $\rightarrow \mathbf{w}_\text{B}$

Because they often appear in similar contexts, their embeddings satisfy: $A \approx B$. Similarly, we have: *speaks* $\approx$ *greets*, *media* $\approx$ *press*, and *Illinois* $\approx$ *Chicago*. Therefore, we can represent each document as a vector by averaging the embeddings of its words:

$$\mathbf{d} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{w}_i \tag{151}$$

Where N is the number of words in the document, and $\mathbf{w}_i$ is the embedding of the i-th word. The result is **one vector representing the entire document**. Notice that this is not CBOW. We're simply **reusing the learned embeddings**.

⚖ **Comparing two documents.** Once every document has a vector representation, we compare them using the cosine similarity:

$$\text{sim}\,(D_i, D_j) = \frac{\mathbf{d}_i \cdot \mathbf{d}_j}{\|\mathbf{d}_i\| \cdot \|\mathbf{d}_j\|}$$

If the cosine similarity is high, the documents discuss similar topics.

❓ **Why is this useful?** This simple representation can already improve many tasks.

- ✔ **Document similarity.** Find documents discussing the same topic.
- ✔ **Document classification.** Instead of classifying sparse Bag-of-Words vectors, a classifier receives dense semantic embeddings. For example, "*Politics*", "*Sports*", "*Technology*", "*Medicine*", etc. Documents about politics tend to occupy a similar region of the embedding space, making them easier to classify.
- ✔ **Clustering.** Documents discussing similar topics naturally group together.

The averaging strategy presented here was one of the **first practical ways** to represent an entire document using Word2Vec. It demonstrated that embeddings learned from a simple language model could be reused as powerful features for downstream NLP tasks such as similarity search and classification. Today, more advanced methods such as **Doc2Vec**, **Sentence-BERT**, and modern transformer-based embedding models learn document or sentence embeddings directly, but they all build on the same core idea introduced by Word2Vec: **represent text in a continuous semantic vector space where meaning corresponds to geometric proximity**.

5.7.3 Sentiment Analysis

After seeing how word embeddings improve **Information Retrieval (IR)** and **Document Similarity**, another important application is **Sentiment Analysis**. The goal of sentiment analysis is to automatically determine the emotional polarity expressed by a word, sentence, or document. For example, "*I love this move*" has a **positive** sentiment, while "*I hate this movie*" has a **negative** sentiment.

Traditional Approach

Before word embeddings, sentiment analysis was usually formulated as a **classification problem**. The workflow looked like this:

1. Sentence
2. Bag-of-Words (BoW)/TF-IDF representation
3. Classifier (e.g., SVM, Logistic Regression, etc.)
4. Positive/Negative/Neutral label

The classifier had to learn which words were associated with positive or negative emotions. This required labeled training data, feature engineering, and training a supervised model.

Word Embeddings offer another possibility

Because Word2Vec organizes words according to their **contexts**, words expressing similar emotions naturally become close in the embedding space. For example, "*happy*", "*joyful*", "*excited*", and "*delighted*" will tend to occupy the same region. Likewise, "*depressed*", "*sad*", "*miserable*", and "*heartbreaking*" will form another cluster. Therefore, instead of training a classifier, we can simply measure **how close two words are in the embedding space** to determine their sentiment similarity.

As always, suppose we choose a reference word "*sad*", we compute its embedding $\mathbf{w}_{sad}$ and compare it with every other word in the vocabulary using the cosine similarity:

$$\cos(\theta) = \frac{\mathbf{w}_i \cdot \mathbf{w}_{sad}}{\|\mathbf{w}_i\| \, \|\mathbf{w}_{sad}\|}$$

Words with the highest cosine similarity are considered the most semantically related.

Advantages. Using word embeddings for sentiment analysis has several advantages:

- **Words with similar emotional meanings** are automatically grouped together;
- **Synonyms** can be **recognized** even if they never appeared in the labeled training set;

- **Semantic relationships** are captured instead of relying on exact word matching;
- **Cosine similarity** can retrieve emotionally similar words without explicitly training a sentiment classifier.

Limitations. Although this approach is elegant, it also has important limitations. Word2Vec learns **word embeddings**, not **sentence meanings**. Therefore, it cannot correctly handle:

- Negation: "*I am not happy*" would still be close to "*happy*" in the embedding space, leading to incorrect sentiment classification;
- Irony or sarcasm: "*Great, another rainy day*" might be interpreted as positive due to the presence of "*Great*", even though the overall sentiment is negative;
- Long-range context: The sentiment of a sentence can depend on the entire context, which word embeddings alone cannot capture.

In these cases, modern contextual models such as **BERT**, **RoBERTa**, or **GPT** are much more effective because they analyze the entire sentence rather than individual word embeddings.

In summary, Word2Vec embeddings can be used for sentiment analysis because words expressing similar emotions appear in similar contexts and therefore become neighbors in the embedding space. By computing cosine similarity between embeddings, we can retrieve emotionally related words without explicitly training a sentiment classifier. This demonstrates once again that Word2Vec learns a **semantic organization of language**, and that these learned embeddings can be reused across many downstream NLP tasks.

5.8 GloVe: Global Vectors for Word Representation

5.8.1 Motivation behind GloVe

After the success of **Word2Vec**, researchers observed that its embeddings exhibited remarkable semantic properties:

- Semantically related words were close together;
- Vector arithmetic captured analogies such as:

$$king - man + woman \approx queen$$

- Relationships such as *country-capital* emerged naturally.

However, **Word2Vec never explicitly models these relationships**. Instead, they appear as a by-product of the prediction task. During training, the model learns to predict surrounding words from a target word (Skip-Gram) or vice versa (CBOW), and the semantic structure of the embedding space emerges implicitly.

The Central Question. The authors of **GloVe (Global Vectors for Word Representation)** [13] wondered whether these semantic regularities could be learned more directly. Instead of learning embeddings by solving a prediction problem, they proposed learning them from the statistical structure of the entire corpus. In other words:

- Word2Vec asks: *Can we predict neighboring words?*
- GloVe asks: ***What do the co-occurrence statistics of all words tell us about their meanings?***

This is the main conceptual difference between the two approaches.

Local versus global information. Recall how Word2Vec learns. Suppose we observe the sentence:

The cat sits on the mat.

When training on the word *cat*, Word2Vec only looks at the words inside a small context window, for example:

The cat sits

Later it processes another sentence:

The cat drinks milk.

Using another local window. Therefore, **each update depends only on a small portion of the corpus**. Although millions of these local updates eventually produce excellent embeddings, the model never explicitly computes how often every pair of words co-occurs in the entire corpus.

GloVe's idea

GloVe starts from a completely different observation. Instead of asking "*which words appear around ice?*", it asks ***how often does ice appear together with every other word in the corpus?***. For example, suppose we count co-occurrences in a large corpus and find:

Context word	**Times observed with *ice***
solid	1900
water	3000
cold	2500
steam	22
fashion	17

These counts immediately reveal something important:

- *ice* is strongly related to *solid*;
- *ice* is moderately related to *water*;
- *ice* is almost unrelated to *fashion*.

Instead of predicting neighboring words one example at a time, **GloVe attempts to directly learn embeddings that explain these global statistics**.

Why is this useful? The intuition is that meaning is reflected by patterns of co-occurrence. For example:

- *ice* frequently appears with *cold*, *snow*, *winter*, and *frozen*;
- *steam* frequently appears with *hot*, *boiling*, *vapor*, and *heat*.

Notice something interesting: both words occur with *water*, but they differ greatly in how often they occur with *solid* or *gas*. Therefore, simply knowing that two words co-occur is not sufficient. The **relative frequencies** of their co-occurrences contain richer semantic information. This observation is the key insight that motivates GloVe.

The GloVe authors argued that semantic relationships should not simply emerge by chance during prediction. Instead, they should be encoded directly from the statistical properties of the corpus. As stated in the original paper:

- **Word2Vec implicitly** learns semantic vector offsets through prediction;
- **GloVe explicitly** models the statistical relationships that give rise to those offsets.

In summary, the motivation behind GloVe is to combine the strengths of:

- **Count-based methods**, which exploit the **global co-occurrence statistics** of the corpus;

- **Prediction-based methods** like Word2Vec, which produce dense, high-quality embeddings.

Rather than learning word vectors only through local prediction tasks, GloVe directly learns embeddings that capture the **global statistical structure of language**. This idea will naturally lead to the next section, where we introduce the global co-occurrence matrix and explain how GloVe uses it to learn word embeddings.

The **motivation** behind **GloVe (Global Vectors for Word Representation)** is to learn word embeddings by exploiting the **global statistical structure** of the corpus. While **Word2Vec** learns embeddings *implicitly* by solving a local prediction task (predicting surrounding words), GloVe learns them *explicitly* by modeling the global co-occurrence statistics of words. Rather than asking *"Which word should be predicted next?"*, GloVe asks *"Which word pairs frequently occur together throughout the entire corpus?"*.

5.8.2 Global Co-occurrence Statistics

In the previous sections, we saw that **Word2Vec learns word embeddings by repeatedly solving a local prediction task**. Each training example consists of a small context window, and the model gradually updates the embeddings through millions of local predictions.

GloVe follows a different strategy. Before learning any embeddings, it first analyzes the entire corpus to collect statistical information about how words co-occur. Instead of asking "*which word should I predict?*", it asks "***how often do words appear together throughout the whole corpus?***" This information is summarized in a **global co-occurrence matrix**, which becomes the **input to the training algorithm**.

What is co-occurrence?

Two words are said to **co-occur** if they **appear within a predefined context window**. For example, consider the sentence:

The cat drinks fresh milk every morning.

Suppose we choose a context window of size 2. When the central word is "*cat*", its context consists of:

The cat drinks fresh

Therefore, the following co-occurrences are recorded:

- (*cat*, *the*)
- (*cat*, *drinks*)
- (*cat*, *fresh*)

Each occurrence increments a counter by one.

Later, another sentence is processed:

The cat eats fish.

Again, the word *cat* appears together with *the*, *eats*, and *fish*, and their counters are incremented. After scanning the entire corpus, every pair of words has an associated co-occurrence count.

Building the global co-occurrence matrix

Suppose our vocabulary contains only five words:

$$V = \{ice, steam, water, solid, gas\}$$

The co-occurrence matrix might look like this:

Target word	***solid***	***gas***	***water***	***fashion***
ice	1900	66	3000	17
steam	22	780	2200	10

These numbers are not probabilities. They are simply **counts collected from the corpus**. For example:

- *ice* appears near *water* about 3000 times;
- *ice* appears near *solid* about 1900 times;
- *ice* almost never appears near *fashion*.

Therefore, the co-occurrence matrix captures the **global statistical information** about how words appear together in the corpus. This information is crucial for GloVe to learn meaningful word embeddings that reflect the relationships between words based on their co-occurrence patterns.

→ From counts to probabilities

⚠ *What if we doubled the size of the corpus?* The counts in the co-occurrence matrix would approximately double as well. This means that raw counts are **not** invariant to the size of the corpus, which can lead to inconsistencies when comparing co-occurrence statistics across different corpora.

✅ To address this issue, GloVe converts the raw counts into **co-occurrence probabilities**. This normalization step ensures that the co-occurrence statistics are comparable across different corpora, regardless of their size.

Let X_{ij} denote the number of times word j appears in the context of word i. The total number of context words observed around word i is:

$$X_i = \sum_k X_{ik} \tag{152}$$

Where X_i is the sum of all co-occurrence counts for word i. The **co-occurrence probability** of observing context word j given the center word i is then:

$$P(j \mid i) = \frac{X_{ij}}{X_i} \tag{153}$$

In other words, ***among all context words that appeared around i, what fraction corresponds to word j?*** For example, if *ice* appears 3000 times with *water*, 1900 times with *solid*, and 17 times with *fashion*, then:

$$P(\textit{water} \mid \textit{ice}) > P(\textit{solid} \mid \textit{ice}) \gg P(\textit{fashion} \mid \textit{ice})$$

These probabilities are much more informative than raw counts because they **normalize for the frequency of the center word**.

With the co-occurrence probabilities computed, GloVe can now build a single sparse co-occurrence matrix. Once this matrix has been built, the original corpus is no longer needed for training. The optimization algorithm works directly on the stored statistics, learning embeddings that explain the observed co-occurrence patterns.

In summary, the core idea of GloVe is to replace millions of local prediction tasks with a **global statistical view of the corpus**. Every pair of words is

associated with a co-occurrence count, from which conditional probabilities are computed. Rather than predicting neighboring words as Word2Vec does, GloVe learns embeddings that capture these **global co-occurrence statistics**, providing the foundation for the training objective introduced in the next section.

5.8.3 GloVe Training Objective

We have already seen that GloVe starts from the global co-occurrence matrix. For every pair of words i and j, we know X_{ij}, the number of times word j appears in the context of word i. From this we compute:

$$P(j \mid i) = \frac{X_{ij}}{X_i}$$

Which expresses the probability of observing word j near word i. The key question is now: ***how do we transform these probabilities into word vectors?***

Suppose we have the words "*ice*", "*steam*", "*solid*", and "*gas*". From the corpus we observe:

Pair	Co-occurrence
ice - *solid*	high
steam - *gas*	high
ice - *gas*	low
steam - *solid*	low

Ideally we would like the vectors to satisfy "*ice*" close to "*solid*", "*steam*" close to "*gas*", without explicitly programming these relationships. Instead, the vectors themselves should explain the observed statistics.

What Should the Vectors Predict? The previous section introduced an important idea. The **meaning** of a word is reflected by how often it appears together with other words. Therefore GloVe asks: ***can we choose vectors whose relationships reproduce the observed co-occurrence information?*** Unlike Word2Vec, which predicts neighboring words through a neural network, GloVe directly learns vectors that fit the global co-occurrence statistics.

Linear Relationship Between Vectors

Imagine two words *ice* and *steam*, and one context word *solid*. Suppose:

$$P(solid \mid ice) = 0.30$$

While:

$$P(solid \mid steam) = 0.01$$

Clearly, *solid* is much more informative for *ice* than for *steam*. So the learned vectors should somehow encode this large difference. The question becomes: ***what mathematical relationship between vectors best captures these probability differences?***

The GloVe authors observed that a very simple model works remarkably well. They assume that the **dot product** between two vectors should approximate the logarithm of their co-occurrence count. Specifically:

$$w_i^T \tilde{w}_j + b_i + \tilde{b}_j \approx \log(X_{ij}) \tag{154}$$

Where:

- w_i is the embedding of the target word (transposed for the dot product),
- $\tilde{w}_j$ is the embedding of the context word,
- $b_i, \tilde{b}_j$ are bias terms for the target and context words, respectively.
- X_{ij} is the observed co-occurrence count of word j in the context of word i.

Why the Dot Product? Remember what a dot product measures. If two vectors point in similar directions:

$$w_i^T \tilde{w}_j$$

Is large. If they are unrelated, it is small. Therefore the dot product naturally measures how strongly two words are related.

Why the Logarithm? This is probably the most interesting part. Suppose we have:

Pair	Count
the - of	1.000.000
cat - animal	100
cat - keyboard	1

Using raw counts is problematic because they vary enormously. A very common word like "*the*" would dominate the optimization. Instead, GloVe compresses the scale:

$$\log(1) = 0 \quad \log(100) \approx 4.6 \quad \log(1'000'000) \approx 13.8$$

The logarithm preserves the ordering while greatly reducing the range of values. As a result, frequent word pairs no longer overwhelm the learning process.

This equation is the heart of GloVe.

Learning the Embeddings

Now we know what we want. For every observed pair:

$$w_i^T \tilde{w}_j + b_i + \tilde{b}_j$$

Should be as close as possible (approximately) to:

$$\log(X_{ij})$$

Of course this equality cannot hold perfectly for every word pair simultaneously. Instead, we **minimize the overall error across the entire corpus**.

The GloVe authors define the following cost function:

$$J = \sum_{i,j} f(X_{ij}) \cdot \left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log(X_{ij}) \right)^2 \tag{155}$$

Called the **weighted least squares objective**, it sums the squared differences between the predicted and actual log co-occurrence counts. The terms are:

- **The Prediction**

$$w_i^T \tilde{w}_j + b_i + \tilde{b}_j$$

This is the model's estimate of the relationship between two words.

- **The Target**

$$\log(X_{ij})$$

This comes directly from the corpus. It is the quantity we want to reproduce.

- **The Squared Error**

$$(\text{prediction} - \text{target})^2$$

If the prediction is accurate, the error is close to zero. Otherwise it becomes large.

- **The Weight Function**

$$f(X_{ij})$$

This is a very clever addition. Not all co-occurrence counts are equally reliable. For example:

Count	Reliability
1	very noisy
3	noisy
500	reliable
10.000	reliable enough

The weighting function assigns **less importance to rare co-occurrences**, which are often due to chance, while giving more importance to frequent, reliable observations. At the same time, it **prevents extremely frequent pairs from dominating the optimization**.

In summary, GloVe learns word vectors by minimizing the difference between the **similarity predicted by the vectors** (their dot product plus biases) and the **similarity observed in the corpus** (the logarithm of the co-occurrence count), giving greater importance to reliable co-occurrence statistics than to noisy ones.

In other words, GloVe learns one vector for every word such that the dot products between word vectors reproduce the co-occurrence statistics observed in the corpus.

5.8.4 Comparison with Word2Vec

At first glance, Word2Vec and GloVe appear to be very different algorithms.

- **Word2Vec** is a **prediction-based model**: it learns embeddings by predicting surrounding words.
- **GloVe** is a **count-based model**: it learns embeddings by fitting global co-occurrence statistics.

Despite these different training procedures, both methods produce remarkably similar embedding spaces and capture many of the same semantic relationships. In fact, the GloVe authors summarize their contribution by saying: "**GloVe makes explicit what Word2Vec does implicitly**". [13] Let's understand what this means.

Different Learning Strategies

The main difference lies in **what the model is trying to optimize during training**.

- **Word2Vec** never computes the complete co-occurrence matrix. Instead, it repeatedly observes small context windows. For example:

  $$\textit{The cat sleeps on the sofa.}$$

  Using Skip-Gram, it creates training pairs such as:

  $$(\textit{cat} \rightarrow \textit{The}) \quad (\textit{cat} \rightarrow \textit{sleeps}) \quad (\textit{cat} \rightarrow \textit{on})$$

  And learns vectors that make these predictions accurate. The embeddings emerge **indirectly** as a consequence of solving this prediction task.

- **GloVe** takes a completely different approach. Before training even begins, it scans the entire corpus and builds the global co-occurrence matrix X_{ij}. Training then consists of finding vectors whose dot products reproduce these observed statistics. Instead of predicting neighboring words one training example at a time, GloVe directly fits the global statistical structure of the corpus.

Local Context vs Global Statistics

This is the easiest way to understand the difference between Word2Vec and GloVe.

- **Word2Vec learns from local context**. At every training step it only sees a small window. Example:

  $$\textit{The cat sleeps on the sofa.}$$

  If the window size is 2, the word "*cat*" only interacts with "*The*" and "*sleeps*". The model has no direct knowledge of how often "*cat*" co-occurs with every other word in the corpus. It gradually discovers these relationships after millions of training examples.

- **GloVe learns from global statistics**. Before optimization starts, it already knows:
  - How often "*cat*" appears near "*animal*".
  - How often it appears near "*pet*".
  - How often it appears near "*keyboard*".

  And every other word. The optimization process simply tries to encode this information into vectors.

⚖ Implicit vs Explicit Learning

In general, GloVe makes explicit what Word2Vec does implicitly.

- **Word2Vec**. The model never explicitly computes co-occurrence probabilities. Nevertheless, because it repeatedly predicts neighboring words, it eventually learns embeddings that reflect these statistics. The statistical information is **hidden inside the learned vectors**.
- **GloVe**. Instead of hoping that the model discovers these statistics, we explicitly provide them through the **co-occurrence matrix** and **optimize the vectors to reproduce them**. The same semantic relationships are learned, but the statistical information is used directly.

◎ Training Objective

Another important difference is the optimization objective.

- **Word2Vec**. The objective is: **predict neighboring words**. Training uses objective such as Skip-Gram, CBOW, Negative Sampling, and Hierarchical Softmax.
- **GloVe**. The objective is: **reproduce the logarithm of the observed co-occurrence counts**. Training minimizes the weighted least-squares loss:

$$J = \sum_{i,j} f(X_{ij}) \left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

  So although both methods learn vectors, they optimize completely different objectives.

? What do they learn? Interestingly, the resulting embedding spaces are extremely similar. Both methods learn that:

- Similar words are close together;
- Semantic analogies correspond to vector offsets;
- Arithmetic operations on vectors make intuitive sense.

For example:

$$king - max + woman \approx queen$$

A property that we already observed for Word2Vec and that also appears in GloVe embeddings.

✅ Advantages and ❌ Disadvantages

Word2Vec	GloVe
Learns by predicting neighboring words	Learns from global co-occurrence counts
Uses only local context during each update	Uses statistics from the entire corpus
Does not explicitly build a co-occurrence matrix	Requires building the co-occurrence matrix first
Very efficient on large corpora	Can better exploit global statistical information
Embeddings emerge implicitly	Embeddings are optimized explicitly from corpus statistics

In summary, there is no universal winner. In practice:

- **Word2Vec** is computationally simpler and scales very well to extremely large corpora.
- **GloVe** often produces embeddings of comparable or slightly better quality because it incorporates global co-occurrence information from the beginning.

The differences in downstream performance are usually quite small, and both methods have become standard techniques for learning static word embeddings. However, the distinction between the two approaches can be summarized as follows:

- **Word2Vec learns by prediction**. Predict surrounding words and let the embeddings emerge naturally.
- **GloVe learns by reconstruction**. Fit the observed co-occurrence statistics directly to obtain the embeddings.

6 Image Classification

6.1 Computer Vision and Digital Images

6.1.1 What is Computer Vision?

Humans continuously perform visual recognition: we recognize faces, identify objects, understand scenes, estimate distances, recognize actions, read text, notice anomalies. All of this happens almost effortlessly and automatically, without conscious thought. The question behind **Computer Vision (CV)** is: ***can we teach a computer to do the same?*** That is the entire purpose of the field.

> **Definition 1: Computer Vision (CV)**
>
> **Computer Vision (CV)** is an interdisciplinary scientific field that deals with how computers can be made to gain high-level understanding from digital images or videos.

Let's make a few important observations about this definition:

- "*Interdisciplinary*". Computer Vision is not only Artificial Intelligence. It **combines ideas from several disciplines**, including Mathematics, Linear Algebra, Statistics, Signal Processing, Image Processing, Machine Learning, Deep Learning, Geometry, Optics, and Robotics.
- "*Digital images or videos*". The input is always some visual data. For example, a JPEG image, a PNG image, a medical scan, a satellite image, frames from a camera, an entire video. However, notice something important. The **computer does not see objects**. It only receives numbers.

  For example, an RGB image is simply: $I \in \mathbb{R}^{R \times C \times 3}$, where R is the number of rows (height), C is the number of columns (width), and 3 is the number of channels (Red, Green, Blue). Later in this section, we'll see exactly how images are represented.

Thus, a computer starts from **low-level data** (numbers):

$$\begin{matrix} 145 & 210 & 34 & \cdots \\ 144 & 208 & 36 & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{matrix}$$

These are just pixel intensities. Humans never think like this, instead we immediately understand "*there is a dog*", or "*the dog is running*", or "*the dog is looking left*". These are **high-level concepts**. So Computer Vision tries to build a function:

$$f : \text{Pixels} \longrightarrow \text{Meaning}$$

Or more generally:

$$f(I) = \text{semantic information}$$

Where I is the input image, and the output is some semantic information about the image. For example, the output could be a label (e.g., "dog"), a bounding

box (e.g., "the dog is in this rectangle"), a segmentation mask (e.g., "these pixels belong to the dog"), or even a caption (e.g., "a dog is running in the park"). Thus, the whole challenge of Computer Vision is that **meaning is not explicitly present in the pixels**, the computer must **learn** it!

Low-level information	High-level understanding
Pixel intensities	Dog
RGB values	Car
Brightness	Person
Colors	Bicycle
Edges	"A person riding a bike"

Why is this difficult?

Suppose we show a computer two images of a cat. To us, they look very similar. But to a computer they may look completely different because different colors, different lighting, different pose, different background, different size, etc. Although **semantically identical**, their pixel values may be almost unrelated. This is why Computer Vision is difficult. The mapping:

$$\text{pixels} \rightarrow \text{meaning}$$

Is highly **non-linear and extremely complex**.

Why has Computer Vision grown so much recently? Computer Vision has grown tremendously in the last decade, and is now a very active field of research. The main reasons are:

- **Huge datasets**. Millions of labeled images became available. Without large datasets, deep learning would not have achieved today's performance.
- **GPU computing**. Training a CNN involves billions of floating-point operations. Modern GPUs made this computationally feasible.
- **Deep Learning**. Perhaps the biggest revolution. Before roughly 2012, most Computer Vision algorithms were **hand-designed**. Researchers manually created features such as: edges, corners, textures, gradients, SIFT, HOG, etc. And then fed them into classifiers. Today, Deep Neural Networks learn these features automatically from data. This dramatically improved performance across almost every vision task.

In summary **Computer Vision** aims to make computers understand the content of images and videos. The input is **low-level numerical data** (pixels), while the output is **high-level semantic information** (objects, scenes, actions). The central challenge is learning the mapping:

$$\text{Pixels} \rightarrow \text{Meaning}$$

Modern Computer Vision has advanced rapidly thanks to **large datasets**, **GPU computing**, and especially **Deep Learning**.

6.1.2 Examples of Visual Recognition Tasks

Now that we know what Computer Vision is, the next question is: ***what kinds of problems does Computer Vision solve?*** Here, we present a few examples of visual recognition tasks that are commonly solved using Computer Vision techniques. These tasks differ mainly in **what information the algorithm is expected to produce**.

- **Object Detection**. Given an image I find **what** objects exist and **where** they are located. Mathematically:

  $$I \rightarrow \{(c_i, b_i)\}_{i=1}^{N}$$

  Where c_i is the class of the i-th object and b_i is the bounding box of the i-th object.

  For example, given an image of a street scene, the algorithm should be able to identify and locate cars, pedestrians, traffic lights, etc. Unlike image classification, there may be multiple objects, classes and instances of the same class in the image (i.e., multiple cars, pedestrians, etc.).

  Why is difficult? The detector must answer simultaneously "*what is this?*" and "*where is it?*". It is therefore solving both **recognition** and **localization**.

- **Pose Estimation**. Instead of recognizing objects, the goal is to understand the **configuration of the human body**. Rather than returning a label "*Person*" or a bounding box, the algorithm predicts the locations of key points on the human body, such as joints (e.g., elbows, knees, shoulders). Mathematically:

  $$I \rightarrow \{(x_i, y_i)\}_{i=1}^{K}$$

  Where K is the number of key points and (x_i, y_i) are the coordinates of the i-th key point. Thus, the algorithm is **not classifying** a person, but rather it is estimating **their geometry**.

- **Quality Inspection**. Factories continuously produce components, some of which contain defects. The goal of quality inspection is to use a vision system to automatically identify normal products, defective products, and defect types instead of asking a human operator to inspect thousands of products.

  Here, the model can perform classification, and even **novel-class detection**. This means that if the defect has never been observed before, the system should recognize "*this does not belong to any known class*". This capability is extremely valuable in manufacturing, where unexpected failure modes can appear.

- **Image Captioning**. Instead of producing a class, the model produces an **entire sentence**. For example, given an image of a dog playing with a ball, the model might generate the caption: "*A dog is playing fetch with a ball in the park.*". Mathematically:

  $$I \rightarrow \textit{Sentence}$$

> **Why is difficult?** Now the model must combine Computer Vision with Natural Language Processing (NLP). It must understand objects, actions, relationships, grammar, word order. This is much harder than classification.

In general, even very advanced Computer Vision systems still make mistakes because visual understanding remains a **challenging problem**.

The evolution of Computer Vision

Years ago, algorithms were designed manually. Researchers created mathematical models describing edges, textures, corners, gradients and other features. The focus was on explicitly engineering features.

Nowadays, Machine Learning, and especially Deep Learning, has become the dominant paradigm. Instead of manually defining useful features, the **model learns them directly from data**.

- **Before**: mathematical/statistical descriptions of images.
- **Now**: machine-learning methods are much more popular.

Thus, why start from Image Classification? If Computer Vision can do so many things, *why do we start with Image Classification?* The answer is that Image Classification is the **simplest visual recognition problem**. The input is only an image, and the output is only a class. Although it is simple, it is still a very useful task. Almost every advanced Computer Vision task extends this basic problem.

6.1.3 Digital Images as Arrays

Until now, we have discussed what Computer Vision does. The next question is: ***how does a computer actually represent an image?*** Humans perceive an image as a continuous visual scene, but computers cannot directly understand colors, shapes, or objects. Before any neural network can process an image, the image must first be represented numerically. This is the purpose of this section.

Why represent images as arrays?

A neural network only knows how to perform mathematical operations. It cannot process concepts like "*red car*" or "*person sitting*". Instead, it receives a large collection of numbers. Therefore, every digital image must be converted into a **multidimensional array**. The network will later learn how to associate patterns in these numbers with semantic concepts.

In general, an image is composed of a finite number of tiny elements called **pixels** (picture elements). Each pixel stores information about the light intensity reaching that location. If the image is grayscale, every pixel stores only **one number**. If the image is colored, every pixel stores **three numbers**.

- **Grayscale images**. Suppose we have a grayscale image. It can be represented as:

  $$I \in \mathbb{R}^{R \times C}$$

  Where R is the number of rows (height) and C is the number of columns (width). Each element $I(r, c)$ contains the brightness of one pixel.

  For example, consider the following 3×3 grayscale image:

  $$I = \begin{bmatrix} 10 & 35 & 80 \\ 50 & 120 & 200 \\ 15 & 90 & 255 \end{bmatrix}$$

  Means that dark pixels correspond to small numbers, while bright pixels correspond to large numbers. So a grayscale image is simply a **2-dimensional matrix**.

- **RGB images**. Most images are not grayscale. Instead, they are represented using the **RGB color model**. RGB stands for *Red*, *Green*, and *Blue*. Every color can be generated by combining different amounts of these three primary colors. Each channel measures **how much of that color is present** at every pixel. When combined, they reconstruct the original color image.

  Mathematically, instead of one matrix, an RGB image consists of **three matrices**: R, G, and B. Each having size $R \times C$. Therefore, the complete image becomes a **3-dimensional array**:

  $$I \in \mathbb{R}^{R \times C \times 3}$$

  The third dimension stores the three color channels.

Pixel values

Images are usually stored using **8 bits per color channel**, so each channel value is an integer in the range **0-255**. Since $2^8 = 256$, each channel has 256 possible values: 0 represents no intensity (black), while 255 represents full intensity (white). Therefore, a pixel in an RGB image can be represented as a triplet of integers:

$$\text{pixel} = (R, G, B) \in [0, 255]^3$$

For example, the pixel $(253, 5, 6)$ means that the red channel is almost at full intensity, while the green and blue channels are very low. This pixel will appear as a bright red color.

If pixel values are integers between 0 and 255, why are we saying the image belongs to $\mathbb{R}$ (real numbers)? When the image is read from disk, the pixel values are indeed integers (`uint8`). However, before entering a neural network, these integers are almost always converted to floating-point numbers (`float32`), often normalized to the range $[0, 1]$ or $[-1, 1]$. This is because later computations (matrix multiplications, convolutions, gradients, etc.) are all performed on **real-valued tensors**.

Images in memory vs. on disk

When loaded into memory, images occupy much more space than they do on disk because image files (e.g., JPEG) are typically compressed. For example, a JPEG image might occupy only a few hundred kilobytes on disk. After decompression, it becomes a full array containing every pixel, which is much larger. This distinction is important because neural networks always operate on **decompressed numerical array**, not on the compressed JPEG representation.

In general, a neural network never receives an image as a "picture". Instead, it receives $I \in \mathbb{R}^{R \times C \times 3}$, which is simply a **3-dimensional tensor of numbers**. Everything the network learns (e.g., edges, textures, objects, faces, animals, etc.) is ultimately learned by discovering patterns within these numerical arrays.

6.1.4 RGB Images and Videos

Up to now we have represented a single image as:

$$I \in \mathbb{R}^{R \times C \times 3}$$

Where R is the number of rows, C is the number of columns, and 3 is the number of channels (Red, Green, Blue). This representation is sufficient for photographs. But what about a **video**? A video is nothing more than a sequence of images.

Imagine recording one second of video. Instead of obtaining one image, we obtain many images taken one after another. Each image is called a **frame**. The number of frames per second is called the **frame rate** and is usually expressed in frames per second (fps). For example, a frame rate of 30 fps means that 30 images are recorded every second. Of course, the higher the frame rate, the smoother the video will be. For example, a video with a frame rate of 1 fps will look like a slideshow, while a video with a frame rate of 60 fps will look very smooth.

Each individual frame is still:

$$I \in \mathbb{R}^{R \times C \times 3}$$

Nothing changes regarding the representation of a single image. However, since a video contains many frames, we simply add a **new dimension**. If there are T frames, the video becomes:

$$V \in \mathbb{R}^{R \times C \times 3 \times T}$$

The first three dimensions $R \times C \times 3$ describe **space and color**, while the last dimension T describes **time**. Roughly speaking, we ask each time: "*what color is this pixel* ***at this instant in time****?*".

Why does data size grow so quickly?

The answer is very simple. In general, the total number of stored values is simply:

$$\text{Total number of values} = R \cdot C \cdot 3 \cdot T$$

For example, if we have a very small video of $R = 144$ rows, $C = 180$ columns, and $T = 30$ frames (3 channels), the total number of values is:

$$144 \cdot 180 \cdot 3 \cdot 30 = 2'332'800$$

About 2.3 million values! This is a very small video, but it is already quite large (on disk, it would take about ≈ 2 MB of space).

Every additional frame stores another complete image. If we double the number of frames, we double the required memory. If we double both image dimensions, memory grows by roughly four times. The increase is extremely fast. Let's

imagine a full HD video with $R = 1080$ rows, $C = 1920$ columns, and $T = 30$ frames. The bytes required only for each frame are:

$$1080 \cdot 1920 \cdot 3 = 6'220'800 \approx 6 \text{ MB}$$

This is about 6.2 million values for each frame. If we have 30 frames, the total number of values is:

$$6'220'800 \cdot 30 = 186'624'000 \approx 187 \text{ MB}$$

This is about 187 million values per second!

With our phone, we can record videos at 60 fps, and we didn't see any explosions in memory. How is this possible? If one second requires roughly 187 MB, why is an MP4 often only a few megabytes? Because videos are **compressed**. Let's consider two consecutive frames of a video, and let's say that the first frame is a picture of a person standing in front of a wall. The second frame is the same person, but they have moved slightly to the right. The two frames are very similar, and the only difference is the position of the person. Instead of storing both frames completely, we can store only the first frame and then store only the difference between the two frames. This is called **inter-frame compression**. Obviously there are many other compression techniques, but the main idea is to store only the information that changes between frames.

Important for Deep Learning. However, there is an important and unfortunate fact. Although videos are compressed on disk, neural networks do not process compressed files. Before training, the **compressed video must be decoded into its full tensor**. This mans that a model actually receives a bomb of data like $(1080, 1920, 3, 30) = 186'624'000$ values. Thus, larger tensor imply more RAM, more GPU memory, slower training, large datasets, higher computational cost. This is one reason why video understanding is significantly more computationally demanding than image classification.

6.2 Local Spatial Transformations and Correlation

6.2.1 Local Neighbourhoods

So far we have represented an image as a tensor:

$$I \in \mathbb{R}^{R \times C \times 3}$$

Where every pixel has RGB values. But, ***how can we transform an image?*** More specifically, ***how can we compute a new image from an existing one?*** The answer introduced in this sections is to use **local spatial transformations**.

⚖ Global vs Local Transformations

Suppose we want to modify an image. There are two possible philosophies:

- **Global Transformations**. Each output pixel depends on **every pixel** in the image. Obviously, this is very expensive to compute. For example, if we want to compute the average color of an image, we need to look at every pixel in the image.
- **Local Transformations**. Instead of looking at every pixel in the image, the output pixel only depends on a **small neighborhood** around it. Everything outside the square is ignored. This is much more efficient and, more importantly, **it matches the structure of natural images**. Because nearby pixels are usually much more related than distant pixels.

❓ What is a Neighbourhood? And why is it called Local?

A **neighbourhood** U is a **small window centered on the pixel we are processing**. The central pixel is denoted as (r, c), where r is the row and c is the column of the pixel. The neighbourhood is usually a square of size $k \times k$, where k is an odd number. The reason for using an odd number is that it allows us to have a single central pixel with an equal number of pixels on each side. For example, a 3×3 neighbourhood has one central pixel and 8 surrounding pixels.

Again, only these pixels are used to compute the output pixel. Everything outside the square is ignored. This is why it is called a **local** transformation.

🛠 Transformation Function T_U. Up to now we have only defined the **local neighbourhood** $U_{r,c}$. Now suppose we want to blur the image, sharpen it, detect edges, remove noise, or perform any other image processing task. All these operations create **another image** (output) G from the original image I. But ***how should the value of one output pixel*** $G(r, c)$ ***be computed from the input image*** I***?*** The answer is that we need to look only at the neighbourhood around (r, c), then apply some rule T_U to compute the output pixel.

In other words, we can define a **transformation** T_U as a **function** that takes as input a neighbourhood $U_{r,c}$ and produces as output a single pixel value $G(r, c)$:

$$G(r, c) = T_U\left(I\right)(r, c)$$

The output pixel at (r, c) is obtained by applying the transformation T_U to the local neighbourhood centered at (r, c) in the original input image I.

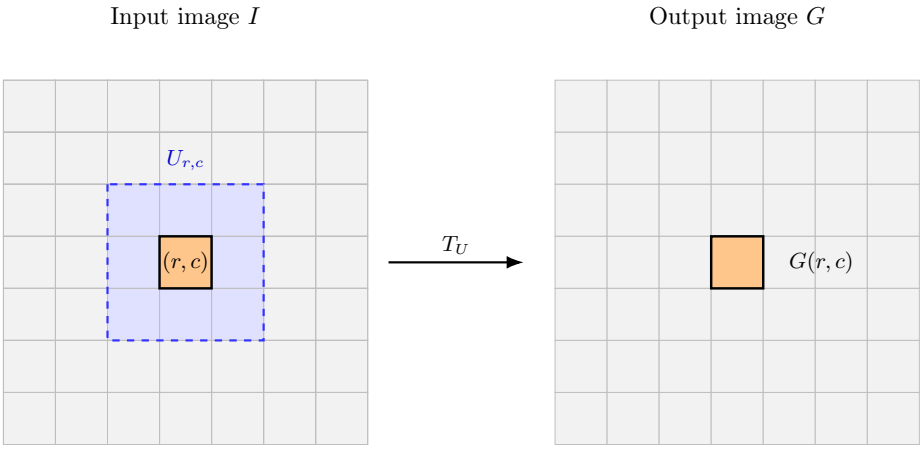


Figure 47: A local transformation T_U that computes the output pixel $G(r, c)$ from the input image I using only the local neighbourhood $U_{r,c}$.

How is a Neighbourhood described? Suppose we are currently computing the output pixel at (r, c). The neighbourhood $U_{r,c}$ is a square of size $k \times k$ centered at (r, c). For example, if $k = 3$, the neighbourhood is:

$$U_{r,c} = \begin{bmatrix} I(r-1, c-1) & I(r-1, c) & I(r-1, c+1) \\ I(r, c-1) & I(r, c) & I(r, c+1) \\ I(r+1, c-1) & I(r+1, c) & I(r+1, c+1) \end{bmatrix}$$

The neighbourhood is **not described by absolute coordinates**, but rather by **offsets from the center**. Thus, we introduce the following notation:

$$I\left(r + u, c + v\right), \quad (u, v) \in U$$

Where r, c are the **absolute position** of the current pixel, and u, v are the **relative displacement** from that pixel. For example, in a 3×3 neighbourhood, the offsets are:

$$U = \{(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 0), (0, 1), (1, -1), (1, 0), (1, 1)\}$$
$$\rightarrow \begin{bmatrix} (-1, -1) & (-1, 0) & (-1, 1) \\ (0, -1) & (0, 0) & (0, 1) \\ (1, -1) & (1, 0) & (1, 1) \end{bmatrix}$$

The central pixel is at offset $(0, 0)$, but (r, c) can be any pixel in the image, such as $(r, c) = (20, 50)$. Thus:

$$I\left(20 + 0, 50 + 0\right) = I(20, 50) \quad u, v = (0, 0) \in U$$

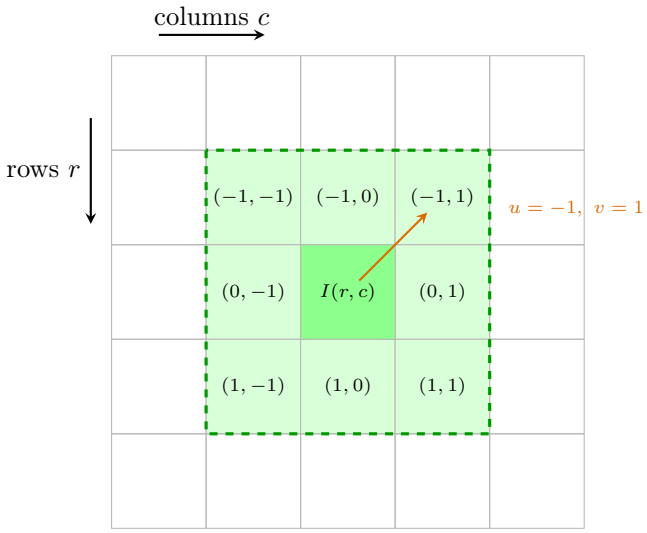


Figure 48: A local neighbourhood $U_{r,c}$ represented as a set of displacement vectors (u, v) around the central pixel (r, c).

How do we transform the entire image?

We have defined how to compute a single output pixel $G(r, c)$ from the input image I using the local neighbourhood $U_{r,c}$. But, ***how do we compute the entire output image*** G***?*** The answer is that we need to **move the neighbourhood** to the next pixel in the image, and repeat the process for every pixel in the image. This is done by iterating over all pixels (r, c) in the input image I, applying the transformation T_U to each local neighbourhood $U_{r,c}$, and storing the result in the corresponding output pixel $G(r, c)$. This movement of the neighbourhood is called the **Sliding Window**.

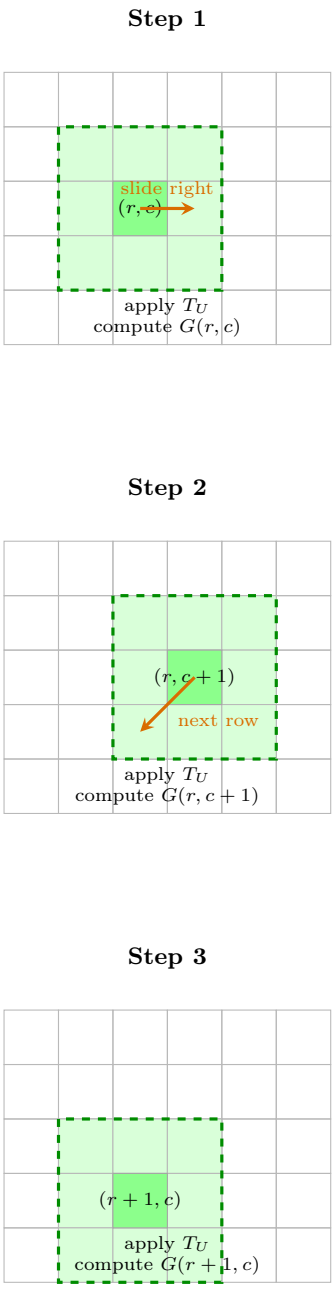


Figure 49: Sliding window mechanism. The same local transformation T_U is applied at different spatial positions to compute the corresponding output pixels.

6.2.2 Spatial Filters

In the previous section we introduced the idea of a local neighbourhood. For every pixel (r, c), we considered a small window $U_{r,c}$ containing only the nearby pixels. The natural next question is: ***what should we do with those pixels?*** The answer is that we can apply a **spatial filter**, which is a function that receives the neighbourhood and computes the output pixel.

The filter T_U receives all the pixels inside the neighbourhood $U_{r,c}$ and returns one new pixel value. Formally, we can write this as:

$$T_U : U_{r,c} \to G(r, c) \tag{156}$$

Where $G(r, c)$ is the output pixel value at position (r, c). The transformation could be the average of the pixels, the median, or any other function that takes a set of pixel values and returns a single value. The important point is **not yet how we compute the value**, but **that the computation uses only the local neighbourhood**.

Space-Invariant Transformations

The spatial filter T_U is said to be **space-invariant** if it does not depend on the position of the pixel (r, c). In other words, the same function is applied everywhere in the image. This property is important because it allows us to apply the same filter to different parts of the image without having to redefine it for each location.

For example, imagine an edge detector. If the filter detects vertical edges, we want it to detect them at any location in the image, not just at a specific row or column. This is what makes the filter space-invariant. In simple terms, with this property, the **input values change** (the pixel values in the neighbourhood), but the **function itself does not change** (the way we compute the output pixel value).

General Pipeline. The complete processing pipeline is now:

1. Input image I.
2. Select a neighbourhood $U_{r,c}$ for each pixel (r, c).
3. Apply a spatial filter T_U to the neighbourhood $U_{r,c}$ to compute the output pixel value $G(r, c)$.
4. Move the sliding window to the next pixel.
5. Repeat the process until all pixels have been processed.
6. Output image G.

Every pixel of the output image is obtained by repeating exactly this procedure.

6.2.3 Correlation as a Local Linear Operation

In the previous section we defined a **spatial filter** as a completely generic function T_U. It could compute the maximum, the average, the median, or any arbitrary function of the neighbouring pixels. Now we introduce an important restriction: ***what if the spatial filter is linear?*** This leads to the operation called **correlation**.

Why Linear?

The first obvious question is: *why do we care about linear operations?* The reason is that linear operations have several **advantages**:

- Mathematically simple (i.e., easy to analyze);
- Computationally efficient (i.e., easy to implement);
- Differentiable (i.e., important for optimization and learning);
- Can approximate many useful image processing operations (e.g., edge detection, blurring, sharpening, etc.).

Instead of applying an arbitrary function, we compute **a weighted sum of the neighbouring pixels**.

The Linear Filter. Suppose the neighbourhood U is a square of size $k \times k$:

$$U = \begin{matrix} a & b & c \\ d & e & f \\ g & h & i \end{matrix}$$

Instead of applying an arbitrary function, we assign a weight to every position:

$$\begin{matrix} w_a & w_b & w_c \\ w_d & w_e & w_f \\ w_g & w_h & w_i \end{matrix} \xrightarrow{\text{for example}} \begin{matrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{matrix}$$

The output of the linear filter is then computed as a weighted sum of the neighbouring pixels:

$$1 \cdot a + 0 \cdot b + (-1) \cdot c + 2 \cdot d + 0 \cdot e + (-2) \cdot f + 1 \cdot g + 0 \cdot h + (-1) \cdot i$$

Every pixel contributes, but **not equally**.

In general, the output of a linear filter is computed as:

$$T(I)(r,c) = \sum_{u,v \in U} w_{u,v} \cdot I(r+u, c+v) \tag{157}$$

Where:

- $I(r+u, c+v)$ is the neighbouring pixel at position $(r+u, c+v)$.
- $w_{u,v}$ is the **weight** associated with that neighbouring pixel. Every neighbour has its own weight.

- $\sum_{u,v \in U}$ For every neighbour, we compute multiply the pixel value by its weight. So the output pixels is a **weighted sum** of the neighbouring pixels.

So the **entire operation is completely determined by the weights**, because the weights determine how much each neighbouring pixel contributes to the output.

How is a linear filter represented? In general, the **collection of weights** is called **Filter** or **Kernel**: $\{w_{u,v} : (u,v) \in U\}$[15] [16]. Each filter is a small matrix of weights, which is applied to the neighbourhood of each pixel in the image. For example, for a 3×3 neighbourhood, the filter is a 3×3 matrix of weights:

$$\mathbf{w} = \begin{matrix} w_{-1,-1} & w_{-1,0} & w_{-1,1} \\ w_{0,-1} & w_{0,0} & w_{0,1} \\ w_{1,-1} & w_{1,0} & w_{1,1} \end{matrix}$$

This matrix is called the **filter kernel** or simply the **kernel**.

Image Processing calls this operation *correlation*

The operation defined in Equation 157 is how any local linear transformation can be written. It is the **general form of a local linear operator**. However, in the context of image processing, this operation is called **Correaltion**. Instead of writing $T(I)$, we write $I \otimes w$ (or sometimes $I \star w$, depending on the notation). Its definition is:

$$(I \otimes w)(r,c) = (I \star w)(r,c) = \sum_{u=-L}^{L} \sum_{v=-L}^{L} w_{u,v} \cdot I(r+u, c+v) \tag{158}$$

Where L is the **radius of the neighbourhood** (how far from the center pixel we consider), which is half the size of the neighbourhood. For example, for a 3×3 neighbourhood, $L = 1$; for a 5×5 neighbourhood, $L = 2$; and so on. Usually, the neighbourhood is square, so L is the same in both dimensions and can be computed as:

$$k = 2L + 1 \Rightarrow L = \frac{k-1}{2} \tag{159}$$

Where k is the size of the neighbourhood, and obviously k must be an odd number. The reason is that we want a **center pixel** in the neighbourhood, which is the pixel we are currently processing. If k was even, there would be no center pixel.

Element-wise multiplication, not matrix multiplication! The operation defined in Equation 158 is not matrix multiplication. It is an **element-wise multiplication** of the filter and the neighbourhood, followed by a sum of all the elements. In other words, we multiply each pixel in the neighbourhood by

[15] The term *kernel* is used in the context of machine learning, while the term *filter* is used in the context of image processing. In this course, we will use the two terms interchangeably.
[16] The notation $w_{u,v}$ is the same as $w(u,v)$, they are equivalent.

its corresponding weight in the filter, and then sum all the weighted pixels to get the output pixel.

Pseudocode

The Equation 157 tells us how to compute **one output pixel** $G(r, c)$. The pseudocode simply repeats this computation for **every pixel of the image**. Suppose:

- Input image I has R rows and C columns;
- Filter w has size $K \times K$ (where K is odd);
- We ignore border handling for now (i.e., we only compute the output for pixels that have a full neighbourhood).

```
# Input: I (image), w (filter)
R, C = I.shape
K = w.shape[0]  # assuming square filter
L = (K - 1) // 2  # radius of the neighbourhood

# Output: G (filtered image)
G = np.zeros((R, C))  # initialize output image

# iterate over rows from L to R - L
for r in range(L, R - L):

    # iterate over columns from L to C - L
    for c in range(L, C - L):

        # initialize accumulator for the weighted sum
        acc = 0

        # compute the weighted sum of the neighbourhood
        for u in range(-L, L + 1):
            for v in range(-L, L + 1):
                acc += w[u + L, v + L] * I[r + u, c + v]

        # store the result in the output image
        G[r, c] = acc
```

Listing 11: Pseudocode for correlation.

For every output pixel $G(r, c)$, the algorithm performs:

1. Extract the neighbourhood of size $K \times K$ around the pixel $I(r, c)$;
2. Multiply each pixel in the neighbourhood by its corresponding weight in the filter w;
3. Sum all the weighted pixels to get the output pixel $G(r, c)$;
4. Store the result in the output image G;

5. Move the kernel to the next pixel and repeat until all pixels have been processed.

If we suppose the **input image** has $R \times C$ pixels, and the **filter** has size $K \times K$, then for **each** of the $R \cdot C$ output pixels, we perform roughly K^2 multiplications and additions. Therefore the total **computational complexity** of the correlation operation is:

$$O\left(R \cdot C \cdot K^2\right) \tag{160}$$

For example, a small image of size 512×512 pixels, and a small filter of size 3×3, would require roughly $512 \cdot 512 \cdot 9 \approx 2.36$ million multiplications and additions. This is why correlation is computationally intensive, and also why GPUs are so effective at accelerating CNNs (we will see what CNNs are in future), since the same local computation is repeated independently at every image location.

6.2.4 Correlation as Template Matching

In the previous section, we have seen that correlation is a local linear operation:

$$(I \otimes w)(r,c) = (I \star w)(r,c) = \sum_{u=-L}^{L} \sum_{v=-L}^{L} w_{u,v} \cdot I(r+u, c+v)$$

But ***what does this number represent?*** In other words, what is the meaning of the correlation between a local neighbourhood of an image and a filter? The answer is simple: it measures **how well the image patch matches the filter (template)**.

The Main Idea. Imagine that the filter is not just a random matrix. Instead, suppose it is an image of the object we are looking for. For example, if we are looking for a particular shape in an image, we can use a filter that is an image of that shape. Then, we slide this template over the image and compute the correlation at each position.

To make the idea easier, considers **binary images**. A binary image contains only two colors, typically black and white.

What happens during correlation? Suppose our template is:

$$\mathbf{w} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Now suppose we place it on top of the image. If the image patch under the template is exactly the same as the template, then the correlation will be high:

$$(I \otimes w)(r,c) = 1\cdot 1 + 1\cdot 1 + 1\cdot 1 + 1\cdot 1 + 0\cdot 0 + 1\cdot 1 + 1\cdot 1 + 1\cdot 1 + 1\cdot 1 = 8$$

Now suppose we move one pixel. The image patch under the template is now different from the template:

$$\begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

The correlation will be lower:

$$(I \otimes w)(r,c) = 1\cdot 1 + 1\cdot 1 + 0\cdot 1 + 1\cdot 1 + 0\cdot 0 + 0\cdot 1 + 1\cdot 1 + 1\cdot 1 + 1\cdot 1 = 6$$

Therefore, the correlation score decreases whenever the image becomes less similar to the template. So the location with **the highest score** is where the template best matches the image.

Correlation Output. Instead of producing an image, we can think of the correlation output as a **similarity map**. Each pixel in the output image represents how similar the corresponding patch in the input image is to the template. The higher the value, the more similar the patch is

to the template. Thus, an example of a correlation output could look like this:

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 1 & 3 & 5 & 4 \\ 2 & 5 & 9 & 6 \\ 1 & 2 & 4 & 3 \end{bmatrix}$$

The largest number in the output is 9, which indicates that the patch at that location is the most similar to the template. Therefore, we can conclude that the **template is located at that position in the input image**.

Why does it work? Remember the correlation equation:

$$(I \otimes w)(r, c) = \sum_{u=-L}^{L} \sum_{v=-L}^{L} w_{u,v} \cdot I(r + u, c + v)$$

Each multiplication contributes positively when bright template pixels overlap bright image pixels. If they don't match, many products become zero (or very small), so the total decreases. Therefore, larger correlation means grater similarity.

Maximum Correlation

When performing template matching, we are often interested in finding the location of the template in the image. To do this, we can look for the **maximum correlation value** in the output similarity map. The coordinates of this maximum value correspond to the location of the best match between the template and the image.

That's sound good, but there is a **problem**: the correlation value depends on the size of the template. Suppose the template is:

$$\mathbf{w} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

And the image contains a large white region:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

If we place the template anywhere inside this large white rectangle, the correlation will be:

$$(I \otimes w)(r, c) = 1 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 = 9$$

This means that the correlation is **maximized at multiple locations**, which makes it difficult to determine the exact position of the template.

But, ***why is this a problem?*** We wanted the highest score to find the exact object location. Instead, we obtain the same score at multiple locations. At this point, the algorithm cannot distinguish the *exact object* from *any larger region* that completely contains the object.

The Need for Normalization

To solve this problem, the idea is to **normalize** the score so that it **reflects similarity**, not just the magnitude of the pixel values. Without normalization, large bright regions naturally produce high correlation values, even if they are not the precise object we want to detect.

So, **normalization is needed when using correlation for template matching**. Later in computer vision, this leads to Normalized Cross-Correlation (NCC), where the score is divided by quantities related to the energy (or norm) of both the template and the image patch. This makes the score independent of overall brightness and patch size.

6.2.5 Correlation on RGB Images

Until now we assumed that an image was a single matrix:

$$I \in \mathbb{R}^{R \times C}$$

Where every pixel contains only one values (its intensity). However, real images are usually **RGB images**, so each pixel contains **three numbers**: Red (R), Green (G), and Blue (B) intensities. Therefore, an RGB image can be represented as a three-dimensional array:

$$I \in \mathbb{R}^{R \times C \times 3}$$

Instead of one image, we actually have three matrices, one for each color channel:

$$I_R \in \mathbb{R}^{R \times C}, \quad I_G \in \mathbb{R}^{R \times C}, \quad I_B \in \mathbb{R}^{R \times C}$$

Each channel stores the intensity of a single color.

Thus, does the correlation formula change for RGB images? Fortunately, **no**! The idea of correlation remains exactly the same. For grayscale images we had:

$$T(I)(r,c) = \sum_{u,v} w(u,v)\, I(r+u, c+v)$$

This computes a weighted sum over the neighborhood. For RGB images, every pixel has **three components**, so the filter must also have **three channels**. Instead of a single matrix $k \times k$, the filter becomes a three-dimensional array $k \times k \times 3$ because every position stores three weights, one for each color channel.

An RGB Filter

A filter is no longer one matrix. It is actually three matrices, one for each color channel:

$$\mathbf{w} = \{w_R, w_G, w_B\}, \quad w_R \in \mathbb{R}^{k \times k}, \quad w_G \in \mathbb{R}^{k \times k}, \quad w_B \in \mathbb{R}^{k \times k}$$

Together they form a three-dimensional array:

$$\mathbf{w} \in \mathbb{R}^{k \times k \times 3}$$

Now, the **correlation is computed channel-wise and summed**. Thus:

1. Correlate the Red channel of the image with the Red channel of the filter.
2. Correlate the Green channel of the image with the Green channel of the filter.
3. Correlate the Blue channel of the image with the Blue channel of the filter.
4. Sum the three results to obtain the final output.

Mathematically, we **extend** the compact **correlation** formula (page 415) introducing an **additional summation over the color channels**:

$$T(I)(r,c) = \sum_{i} \sum_{u,v} w_i(u,v)\, I_i(r+u, c+v, i) \tag{161}$$

Where $i \in \{R, G, B\}$ is the color channel (e.g., $i = 1$ for Red, $i = 2$ for Green, and $i = 3$ for Blue). Compared with the grayscale case, the only new operation is the outer sum over the three channels.

Why is this useful? Imagine we want to detect *red objects*. The filter could learn *red weights* are high, while *green and blue weights* are zero. Then only regions rich in red produce a high response. Similarly, a filter detecting the sky might assign high weights to the blue channel and low weights to the red and green channels. This allows filters to learn **color-sensitive features**, not just shapes or edges.

6.3 Image Classification with Linear Classifiers

6.3.1 The Image Classification Problem

Until now, we've been discussing **how to represent images** and **how to process them using correlation filters**. Now the question becomes: ***given an image, how can a computer automatically recognize what it contains?***

Motivation. Humans solve image recognition effortlessly. We recognize objects almost instantly, regardless of: viewpoint, lighting, scale, background, partial occlusions. For a computer, however, an image is **just a collection of numbers**. The goal of **image classification** is to teach the computer to associate these numbers with a semantic label.

Formally, suppose we have an RGB image:

$$I \in \mathbb{R}^{R \times C \times 3}$$

The classifier must assign it to **one category** among a predefined set Λ of K possible categories:

$$\Lambda = \{\lambda_1, \lambda_2, \dots, \lambda_K\}$$

For example:

$$\Lambda = \{\text{wheel}, \text{car}, \text{castle}, \text{baboon}, \dots\}$$

So the task is simply: image $\rightarrow$ classifier $\rightarrow$ label.

Classification is more than choosing a label

A modern classifier usually does **not** produce only one word. Instead, it computes a **confidence score** for every possible class. For example, a classifier might output:

$$\text{scores} = [0.1, 0.7, 0.2]$$

This means that the classifier is 10% confident that the image is a wheel, 70% confident that it is a car, and 20% confident that it is a castle. The predicted class is simply the one with the **highest score**.

Mathematically, we can define the **classification task** as:

$$\mathcal{K} : I \rightarrow y$$

Where I is the **input image**, y is the **predicted label**, and $\mathcal{K}$ is the **classifier**. Since $y \in \Lambda$, the classifier is simply a function that maps an image into one of the predefined classes. Initially, the classifier takes random numbers $\mathcal{K}_\theta$ as parameters θ. After training, these parameters are adjusted to encode everything the model has learned about recognizing different classes.

Output Space. If there are L possible classes, the classifier actually computes L scores simultaneously. Thus, instead of producing a single label:

$$\mathcal{K} : I \rightarrow \Lambda$$

We can think of the classifier as producing a vector of scores:

$$\mathcal{K} : \mathbb{R}^{R \times C \times 3} \to [0, 1]^L$$

Meaning that the input is an RGB image, and the **output is a vector** of L **confidence values** (often probabilities after a Softmax layer). Compared to a single label, the scores reveal something important: the classifier can express **uncertainty**. For example, it might be 70% sure that an object is a car and 30% sure that it is a truck. This is crucial for real-world applications, in which decisions must be made based on confidence levels.

What is the Image Classification Problem?

The **Image Classification Problem** can be stated as: given an image, automatically determine which semantic category it belongs to. Formally, given $I \in \mathbb{R}^{R \times C \times 3}$, find $y \in \Lambda$ where I is the image, Λ is the finite set of possible classes, and y is the correct class label.

Why is this actually difficult? If images were always identical, classification would be trivial. Unfortunately, the same object can appear in infinitely many ways. For example, the class "*Dog*" may contain images like *a dog running in a park*, *a dog sitting on a couch*, *a dog playing with a ball*, etc. All of these belong to the **same class**. So the classifier must ignore variations that **do not change the object's identity**.

But this is **not the only challenge**. Different classes may look very similar. For example, a *Wolf*, a *Dog*, and a *Fox* or a *Car*, a *Truck*, and a *SUV* may share many visual features. The classifier must therefore learn:

- **Intra-class invariance**: The ability to recognize the same object despite appearance changes (e.g., different angles, lighting, or occlusions).
- **Inter-class discrimination**: The ability to distinguish between visually similar objects (e.g., a dog vs. a wolf).

Why do we need Machine Learning? We could imagine writing rules manually (e.g., "if the image has four wheels, it is a car"). However, this approach is impractical for many reasons:

- The number of possible rules is enormous, and it is impossible to cover all cases.
- The rules would be brittle and fail to generalize to new images.
- The rules would be hard to maintain and update as new classes are added.

The number of rules quickly becomes impossible to manage. Instead, we let a model **learn** these rules from examples.

In general, the complete image classification process introduced in this section can be summarized as follows:

1. **Input:** An RGB image $I \in \mathbb{R}^{R \times C \times 3}$.
2. **Classifier:** A function $\mathcal{K}$ that takes the image as input with parameters θ and produces a vector of scores.
3. **Output:** A vector of scores scores $\in [0, 1]^L$, where each score corresponds to a class in the predefined set Λ.
4. **Decision:** The predicted label y is the class with the highest score, i.e., $y = \arg\max_i \text{scores}[i]$.

But ***how does the classifier compute these scores from millions of pixel values?*** The next subsection answers this by introducing the simplest possible classifier: the *linear classifier*. Rather than searching for objects explicitly, it learns a set of weights that directly transform the image pixels into class scores. This will be the foundation for understanding neural networks and, later, convolutional neural networks.

6.3.2 Feeding Images to Neural Networks

So far, we know that an RGB image is represented as:

$$I \in \mathbb{R}^{R \times C \times 3}$$

Where R is the number of rows, C is the number of columns, and 3 is the number of channels (Red, Green, Blue). This is a **3-dimensional tensor** because it has three dimensions: height, width, and channels. However, a standard fully-connected neural network expects something completely different:

$$\mathbf{x} \in \mathbb{R}^{d}$$

A **1-dimensional vector** of size d. Therefore, we need to convert the 3D tensor into a 1D vector.

Why can't we feed the image directly to the neural network? A dense (fully-connected) neuron computes a weighted sum of all its inputs and a bias term:

$$y = \sum_{i=1}^{d} w_i x_i + b$$

So the neural network expects a **list of numbers**, not a matrix or a 3D tensor. Therefore, before entering the network, the image must become a 1D vector.

How do we convert a 3D tensor into a 1D vector?

The process of converting a 3D tensor into a 1D vector is called **Flattening** (or **Unfolding**, or **Vectorization**). Simply put, instead of keeping pixels arranged spatially, we place them one after another into a single long vector.

In general, the flattening process can be implemented in multiple ways:

- **Row-Wise Flattening**: We take each row of the image and place it one after another into a single long vector.
- **Column-Wise Flattening**: We take each column of the image and place it one after another into a single long vector.

> **Example 1: Column-wise flattening**
>
> Suppose we have a tiny grayscale image:
>
> $$I = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$
>
> Reading **column by column** gives us the following vector:
>
> $$\mathbf{x} = \begin{bmatrix} 1 \\ 3 \\ 2 \\ 4 \end{bmatrix}$$

- **Channel-First Flattening**: We flatten all the values of the first channel, then all the values of the second channel, and so on, until all channels have been concatenated into a single vector. Obviously, this is only relevant for images with multiple channels (e.g., RGB images).
- **Channel-Last Flattening**: For each spatial location (pixel), we place all of its channel values consecutively into the vector before moving to the next pixel. Again, this is only relevant for images with multiple channels (e.g., RGB images).

Also, row/column-wise and channel-first/last describe **different aspects** of the ordering. Row and column-wise flattening describe the **spatial dimensions** (R and C) are traversed, while channel-first and channel-last describe where the **channel dimension** (C) appears in the traversal. Therefore, they can be **combined** (e.g., row-wise channel-first flattening, column-wise channel-last flattening, etc.).

What about RGB images? We described the flattening process for grayscale images, but RGB images have **three channels** (matrices). Conceptually, we can concatenate the three channels into a single long vector. As explained above, there are two common ways to define the order of concatenation: **channel-first** and **channel-last flattening**.

However, regardless of the order of concatenation, the **complete vector** contains the following, mathematically:

$$d = R \times C \times 3$$

Elements. So, an RGB image of size $R \times C$:

$$I \in \mathbb{R}^{R \times C \times 3}$$

Is transformed into a vector of size d:

$$\mathbf{x} \in \mathbb{R}^d \rightarrow \mathbf{x} \in \mathbb{R}^{R \times C \times 3}$$

This transformation **changes only the data layout**, not the data itself.

Example 2: Channel-first/last flattening

For example, if we have a very small RGB image of size 2×2 ($R = 2$, $C = 2$), where each color channel is represented as a 2×2 matrix:

$$\text{Red} = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \quad \text{Green} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad \text{Blue} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Flattening each channel using column-wise flattening and concatenating them in channel-first order gives us the following vector:

$$\mathbf{x} = [\underbrace{10,\ 30,\ 20,\ 40}_{\text{Red channel}},\ \underbrace{1,\ 3,\ 2,\ 4}_{\text{Green channel}}\ ,\ \underbrace{5,\ 7,\ 6,\ 8}_{\text{Blue channel}}\ ]^T$$

Thus, the final vector has $d = 2 \times 2 \times 3 = 12$ elements.

Just to clarify, the order of concatenation can be changed (e.g., **channel-last flattening**), but the total number of elements remains the same. Indeed:

$$\mathbf{x} = [\underbrace{10, 1, 5}_{\text{Pixel (1,1)}}, \underbrace{30, 3, 7}_{\text{Pixel (2,1)}}, \underbrace{20, 2, 6}_{\text{Pixel (1,2)}}, \underbrace{40, 4, 8}_{\text{Pixel (2,2)}}]^T$$

Which is the same vector as before, just with a different ordering of the elements.

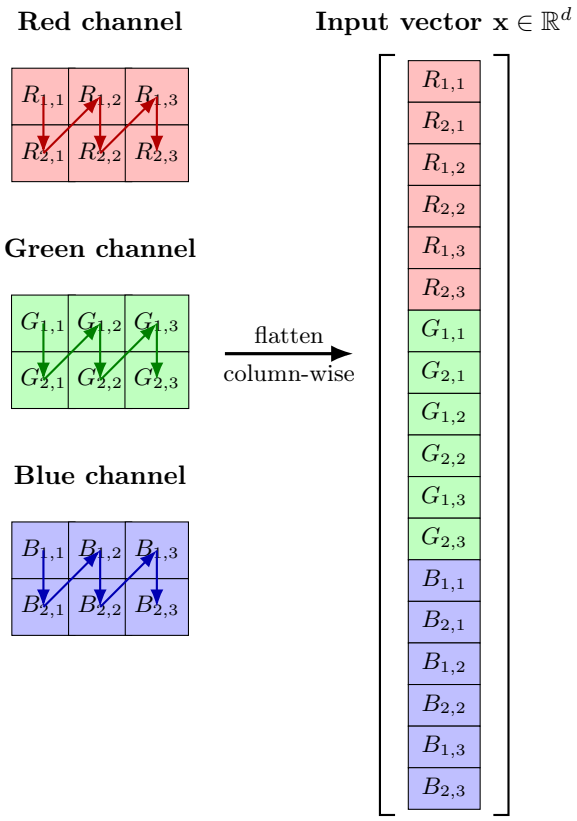


Figure 50: Flattening an RGB image of size $R \times C$ into a vector of size $d = R \times C \times 3$. The flattening is done in a column-wise manner, and the channels are concatenated in a channel-first order.

What information is lost during flattening?

Before flattening an image, the pixels are arranged in a 2D grid, which preserves the spatial relationships between neighboring pixels:

$$\begin{bmatrix} p_{1,1} & p_{1,2} & p_{1,3} \\ p_{2,1} & p_{2,2} & p_{2,3} \end{bmatrix}$$

But after flattening, the pixels are arranged in a 1D vector, which loses the

spatial relationships between neighboring pixels:

$$\begin{bmatrix} p_{1,1} & p_{2,1} & p_{1,2} & p_{2,2} & p_{1,3} & p_{2,3} \end{bmatrix}^T$$

Thus, the network no longer knows that two pixels were neighbors in the original image. Ultimately, a layer of a neural network simply takes a vector of numbers as input; it doesn't know anything about the spatial relationships between pixels. Flattening **preserves every pixel value** but **destroys the explicit spatial organization**.

Why is flattening a problem? Imagine these two binary images of size 2×2:

$$I_1 = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \quad I_2 = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}$$

The white pixels (1s) are in different locations in the two images. Humans immediately recognize them as very similar images because they are both just a single white pixel in a 2×2 black image. However, after flattening, the two images become:

$$\mathbf{x}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad \mathbf{x}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

Now, the two vectors are **orthogonal** (perpendicular) to each other, which means they are **maximally different** in the eyes of the neural network. The network has **no built-in notion of locality or neighboring pixels**.

Despite this limitation, flattening was the simplest way to apply existing neural networks to images, because this approach allowed us to reuse the same architecture that was already used for other types of data (e.g., tabular data). However, this approach:

- ✗ Ignore the two-dimensional structure of images;
- ✗ Every pixel is treated as an independent input feature;
- ✗ The network must learn spatial relationships from scratch.

This inefficiency is precisely why **Convolutional Neural Networks (CNNs)** were later introduced: they process images directly in their 2D (or 3D with channels) form instead of flattening them at the input.

6.3.3 One-layer Neural Network for Images

Let's start by considering a simple one-layer neural network for image classification.

Suppose we have already flattened an image into:

$$\mathbf{x} \in \mathbb{R}^d \quad \text{where } d = R \times C \times 3$$

Our goal is simple: given this vector, predict which object appears in the image.

Network Architecture

Let's start with a simple neural network. The network has only two layers: an **input layer** and an **output layer**. There are **no hidden layers**. If there are d input features and L possible classes, then the architecture is as follows:

- The input layer has d neurons, one for each input feature.
- The output layer has L neurons, one for each class.

For example, using the CIFAR-10 dataset, we have $d = 32 \times 32 \times 3 = 3072$ input features and $L = 10$ output classes.

> **Deepening: CIFAR-10**
>
> **CIFAR-10** is one of the most famous benchmark datasets in computer vision. It is widely used to **train and evaluate image classification algorithms**, especially when introducing CNNs. The name comes from:
>
> - CI: Canadian Institute
> - FAR: for Advanced Research
> - 10: the dataset contains **10 object classes**
>
> The dataset contains $60'000$ **color images** of size 32×32 pixels, 3 color channels (RGB). Therefore, every image has shape $32 \times 32 \times 3$. If we flatten it into a vector, we obtain the following feature vector:
>
> $$\mathbf{x} \in \mathbb{R}^{32 \times 32 \times 3} = \mathbb{R}^{3072}$$
>
> The images belong to one of these ten categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck. Each image has exactly one label.

What does each output neuron compute? Each output neuron computes a **weighted sum of every input pixel**. For class i:

$$s_i = \sum_{j=1}^{d} w_{ij} x_j + b_i \tag{162}$$

Where:

- s_i is the **score** for class i.
- x_j is the j-th input feature (pixel value).
- w_{ij} is the weight connecting input feature j to output neuron i.
- b_i is the bias term for output neuron i.

Weights. Each weight answers the question: "***how important is this pixel for recognizing this class?***". Large positive weight means that the pixel supports the class, while large negative weight means that the pixel contradicts the class; near zero weight means that the pixel is irrelevant for this class.

All the **weights are stored in a weight matrix**:

$$W \in \mathbb{R}^{L \times d} \quad \text{where } W = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1d} \\ w_{21} & w_{22} & \cdots & w_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ w_{L1} & w_{L2} & \cdots & w_{Ld} \end{bmatrix} \tag{163}$$

Where L is the number of classes and d is the number of input features. So, each row corresponds to a class and each column corresponds to an input feature. Thus, the **score** for class i can be computed as:

$$s_i = \sum_{j=1}^{d} w_{ij} x_j + b_i = W[i,:] \cdot \mathbf{x} + b_i \tag{164}$$

Where $W[i,:]$ is the i-th row of the weight matrix W, and $:$ denotes all columns. In other words, it is simply the **dot product** between the input image $\mathbf{x}$ and the weights for class i, plus the bias term b_i.

More compactly, instead of computing every score s_i separately, we perform one single matrix multiplication:

$$\mathbf{s} = W\mathbf{x} + \mathbf{b} \tag{165}$$

Where:

- $\mathbf{s} \in \mathbb{R}^L$ is the vector of scores for all classes.
- $W \in \mathbb{R}^{L \times d}$ is the weight matrix.
- $\mathbf{x} \in \mathbb{R}^d$ is the input image vector.
- $\mathbf{b} \in \mathbb{R}^L$ is the vector of bias terms for all classes.

Example 3: Example of a one-layer neural network for images

Imagine an input image:

$$\mathbf{x} = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} \in \mathbb{R}^3$$

A weight matrix and a bias vector:

$$W = \begin{bmatrix} 1 & 2 & 1 \\ 0 & -1 & 2 \end{bmatrix} \in \mathbb{R}^{2\times 3}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \in \mathbb{R}^2$$

Then, the first output neuron computes the score for class 1:

$$s_1 = 1 \cdot 2 + 2 \cdot 1 + 1 \cdot 3 + 1 = 2 + 2 + 3 + 1 = 8$$

The second output neuron computes the score for class 2:

$$s_2 = 0 \cdot 2 + (-1) \cdot 1 + 2 \cdot 3 + 0 = 0 - 1 + 6 + 0 = 5$$

Therefore, the output vector of scores is:

$$\mathbf{s} = \begin{bmatrix} 8 \\ 5 \end{bmatrix} \in \mathbb{R}^2$$

Since $s_1 > s_2$, the network predicts that the input image belongs to class 1.

Why is there no Activation Function? Normally Dense layers are followed by ReLU or another activation. However, in this case, we are **not interested in the actual scores**, but only in which class has the highest score. Therefore, we can skip the activation function and directly use the raw scores for classification.

Why is Softmax ignored? Usually classification networks finish with a Softmax layer to convert the scores into probabilities. However, we can ignore it because **Softmax never changes the order of the scores**. Therefore, the class with the highest score before Softmax will also have the highest probability after Softmax. Softmax is useful during training (to compute cross-entropy loss) or when calibrated probabilities are needed, but it is **not required to determine the predicted class**.

Why is this called a Linear Classifier? The complete model is simply:

$$K(x) = Wx + b \tag{166}$$

Which is a linear transformation from the input vector to the vector of class scores. There is no hidden layer, no non-linear activation, only one affine transformation. Therefore, a one-layer neural network is mathematically identical to a linear classifier. This is an important insight because it shows that a neural network does not become "*deep*" simply by using Dense layers. Depth only becomes meaningful when hidden layers are separated by nonlinear activation functions.

Do multiple linear layers make the network more expressive?

Surprisingly, the answer is **no**. Imagine inserting a hidden layer. The first layer

computes:

$$\mathbf{h} = W_1\mathbf{x} + \mathbf{b}_1$$

The second computes the scores:

$$\mathbf{s} = W_2\mathbf{h} + \mathbf{b}_2$$

Now substitute the first equation into the second:

$$\mathbf{s} = W_2\left(W_1\mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2 = W_2W_1\mathbf{x} + W_2\mathbf{b}_1 + \mathbf{b}_2$$

But this is still a linear transformation of the input vector $\mathbf{x}$, with new weights $W' = W_2W_1$ and new bias $b' = W_2b_1 + b_2$. Therefore, adding more linear layers does not increase the expressive power of the network. The network remains a linear classifier.

In general, no matter how many linear layers we stack, the whole network can always be rewritten as:

$$\mathbf{s} = W\mathbf{x} + \mathbf{b}$$

Because the matrix multiplication is associative:

$$W_n\left(W_{n-1}\left(\cdots\left(W_2\left(W_1\mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2\right) + \cdots + \mathbf{b}_{n-1}\right) + \mathbf{b}_n\right) = W'\mathbf{x} + \mathbf{b}'$$

And the composition of affine transformations is still an affine transformation. Therefore, a deep network **without nonlinearities** is mathematically equivalent to a shallow linear model.

Furthermore, a larger network could be too large. For the sake of argument, let's assume that adding more linear layers increases the network's expressive power. In that case, we could keep adding more and more linear layers to make the network arbitrarily complex. However, the **number of parameters would grow extremely quickly**, especially when using fully connected layers, such as dense layers for images. For example, consider the CIFAR-10 dataset. If we add a hidden layer with $1'536$ neurons, the number of parameters would be:

$$\text{Parameters} = (3072 \times 1536) + 1536 = 4'720'128$$

The second layer would have:

$$\text{Parameters} = (1536 \times 10) + 10 = 15'370$$

So even this modest network already has about 4.7 million trainable parameters, and almost all of them are in the first dense layer.

A fully connected layer assumes that **every pixel is connected to every hidden neuron**. For images, this is inefficient because:

- Nearby pixels are highly correlated;
- Most pixels are irrelevant for recognizing a particular object;
- Learning millions of independent weights requires large amounts of data and computation.

This observation motivates the next major topic: **Convolutional Neural Networks (CNNs)**, which dramatically reduce the number of parameters by exploiting the spatial structure of images instead of connecting every neuron to every pixel.

6.3.4 Training the Linear Classifier

The Learning Problem. Up to now we have assumed that the weight matrix W and bias vector $\mathbf{b}$ of our linear classifier are given, but now we discuss how they are obtained from data. Suppose we have a training dataset:

$$TR = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_N, y_N)\}$$

Where $\mathbf{x}_i$ is an image (already flattened into a vector) and y_i is its correct class. Our linear classifier is:

$$\mathbf{s} = W\mathbf{x} + \mathbf{b}$$

Where $\mathbf{s}$ is the vector of scores for each class, $\mathbf{x}$ is the input image, and W and $\mathbf{b}$ are the unknown parameters we want to learn. Thus, training means **finding the values** of W and $\mathbf{b}$ that **make the classifier correctly recognize the training images**.

Initially, weights and biases are **randomly initialized**, but during training they are gradually updated until the correct class tends to receive the highest score.

How do we know whether the current weights are good or not?

To measure the quality of a classifier, we need a **loss function** (page 90). The loss measures **our unhappiness** with the scores assigned to the training images. Intuitively, correct predictions should have low loss, while incorrect predictions should have high loss. The goal of training is to **minimize the loss** by adjusting the weights and biases.

Formally, the training goal is to minimize the **total loss over the entire training set**:

$$(W, b) = \arg\min_{W,b} \sum_{(\mathbf{x}_i, y_i) \in TR} \mathcal{L}(W, b; \mathbf{x}_i, y_i) \tag{167}$$

Where:

- $\mathcal{L}(W, b; \mathbf{x}_i, y_i)$ is the **loss** produced by **one image**. For example, if we use the **Sum of Squared Errors (SSE) loss** (page 91), it is:

  $$\mathcal{L}(W, b; \mathbf{x}_i, y_i) = \sum_{j=1}^{C} [t_{ij} - s_{ij}]^2$$

  However, in general the loss functions used for training linear classifiers are different from the SSE loss. Usually, they are binary cross-entropy loss for binary classification, or categorical cross-entropy loss for multi-class classification, or hinge loss for support vector machines. For image classification, the standard choice is **categorical cross-entropy** (page 302).

  Softmax and Categorical Cross-Entropy. Using the **categorical cross-entropy loss**, we first apply the **softmax function** to the scores $\mathbf{s}$ to obtain a probability distribution over the classes, and then compute the cross-entropy loss between this predicted distribution and the true distribution (which is a one-hot vector indicating the correct class).

Therefore, for **prediction**, softmax is unnecessary (page 433), but for **cross-entropy loss**, softmax is required because the loss operates on probabilities.

How is the loss minimized? The loss is minimized using **gradient descent** (page 94, or one of its variants). The gradients of the loss with respect to the weights and biases are computed, and then the weights and biases are updated in the direction that reduces the loss. This process is repeated iteratively over many epochs until convergence.

- $\sum_i \mathcal{L}_i$ is the **sum over the dataset**, because we want the classifier to work well **on all training images**, not just a few. If one image is classified poorly, its large loss increases the total loss.
- $\arg\min_{W,b}$ finds the **values** of W and b that make the total loss **as small as possible**. The min operator returns the minimum loss value, while arg min returns the parameters that achieve that minimum.

Watch Out for Overfitting: use Regularization

We cannot simply minimize the loss on the training set, because this may lead to **overfitting**. To prevent overfitting, we can add a **regularization term** to the loss function, which penalizes large weights and encourages simpler models. Obviously, the regularization cannot be too strong, otherwise the model will underfit.

Therefore, instead of minimizing only the loss:

$$\sum_i \mathcal{L}_i$$

We minimize the **regularized loss**:

$$\sum_i \mathcal{L}_i + \lambda \mathcal{R}(W, b)$$

Where:

- $\mathcal{R}(W, b)$ is the **regularization term**, which is a function of the weights and biases. For example, the L2 regularization (sum of squares of weights) is a common choice (page 164).
- λ is the **regularization strength**, which controls the trade-off between fitting the training data and keeping the model simple. A larger λ means more regularization (simpler model), while a smaller λ means less regularization (more complex model). This is a hyperparameter that needs to be tuned. It tells us **how important** the penalty is (i.e., how much we care about keeping the weights small).

Thus, the final training objective is to minimize the regularized loss:

$$(W, b) = \arg\min_{W,b} \sum_{(\mathbf{x}_i, y_i) \in TR} \mathcal{L}(W, b; \mathbf{x}_i, y_i) + \lambda \mathcal{R}(W, b) \tag{168}$$

6.4 Interpreting Linear Classifiers on Images

Now that we know **how to train** a linear classifier, the next question is: ***what has the classifier actually learned?*** This is the purpose of this section. Instead of discussing optimization, we now interpret the learned parameters W and b. We present two complementary viewpoints:

- **Geometric Interpretation**: each class corresponds to a hyperplane in the input space.
- **Image-based interpretation**: each row of W can be visualized as an image template.

Let's begin with the geometric view.

6.4.1 Geometric Interpretation

Once training has finished, we have learned the parameters:

- Weight matrix W;
- Bias vector b.

The original training images are no longer needed. Instead of storing thousands of labeled examples, we only keep the learned parameters W and b. The classifier has compressed everything it learned about the training set into these parameters.

What does one row of W represent?

The weight matrix W is defined as:

$$W = \begin{bmatrix} \mathbf{w}_1^T \\ \mathbf{w}_2^T \\ \vdots \\ \mathbf{w}_L^T \end{bmatrix} \quad W \in \mathbb{R}^{L \times d}$$

Where L is the number of classes and d is the number of image features (pixels). Each row $\mathbf{w}_i^T$ contains **all the weights associated with class** i:

$$\mathbf{w}_i = W\,[i, :] = \begin{bmatrix} W_{i,1} & W_{i,2} & \cdots & W_{i,d} \end{bmatrix}$$

Each row has exactly one weight for every input pixel.

Inner product between a row of W and an input image $\mathbf{x}$

For the class i, the classifier computes:

$$s_i = \mathbf{w}_i^T \mathbf{x} + b_i$$

Where $\mathbf{w}_i^T \mathbf{x}$ is the inner product between:

- The input image vector $\mathbf{x}$;
- The weight vector $\mathbf{w}_i$ for class i.

This inner product measures **how well two vectors align**. If the two vectors point in the same direction, the inner product is large and positive. If they point in opposite directions, the inner product is large and negative. If they are orthogonal, the inner product is zero.

Intuition. Think of the weight vector as asking: "*does this image look like the pattern I am searching for?*" If the image resembles the pattern, the inner product becomes large. If it is very different, the inner product becomes small or even negative.

Independence. Each row contains all the weights associated with class i. Thus, **each row of W works independently of the other rows**, because there is no link between the rows (each row has its own bias term b_i). The network computes the scores for each class independently:

$$\begin{aligned} s_1 &= \mathbf{w}_1^T \mathbf{x} + b_1 \\ s_2 &= \mathbf{w}_2^T \mathbf{x} + b_2 \\ &\vdots \\ s_L &= \mathbf{w}_L^T \mathbf{x} + b_L \end{aligned}$$

Only after all scores are computed do we compare them and choose the largest. So we can imagine having L **independent binary classifiers**, one for each class, competing against one another.

Remember that this independence is true **only for a single linear layer**. If we introduce a hidden layer, the output neurons all depend on the hidden representation. So, changing one hidden neuron affects **every output class simultaneously**. Consequently, the simple interpretation "each row of W is independent" no longer holds.

Geometric Interpretation

Now that we understand the inner product, we can explain the geometric interpretation of linear classifiers. Instead of thinking of $\mathbf{x}$ as an image, imagine it as **a point in a very high-dimensional space**. For CIFAR-10 images, this space has $32 \times 32 \times 3 = 3072$ dimensions. Therefore every image corresponds to one point in this 3072-dimensional space $\mathbb{R}^{3072}$. This is called the **feature space** or **input space**.

Since understanding 3072 dimensions is impossible, for example, let's imagine a simpler case with only two dimensions $d = 2$ (i.e., each image has only two pixels):

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \xrightarrow{\text{for example}} x = \begin{bmatrix} 3 \\ 5 \end{bmatrix}$$

In this case, each image becomes simple point in a 2D plane (x_1, x_2). Now the score is:

$$f(x_1, x_2) = w_1 x_1 + w_2 x_2 + b$$

This is just a linear function of two variables.

Suppose we are trying to recognize **cats**. The classifier has learned (i.e., trained):

$$w = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \quad b = -6$$

Now to compute the score for a new image, we simply plug in the pixel values x_1 and x_2:

$$f(x_1, x_2) = 2x_1 + 1x_2 - 6$$

For example, suppose a new image is: $x = (1, 1)$. Then the score is:

$$f(1, 1) = 2 \cdot 1 + 1 \cdot 1 - 6 = -3$$

Instead, if the new image is: $x = (4, 2)$, then the score is:

$$f(4, 2) = 2 \cdot 4 + 1 \cdot 2 - 6 = 4$$

And finally, if the new image is: $x = (2, 2)$, then the score is:

$$f(2, 2) = 2 \cdot 2 + 1 \cdot 2 - 6 = 0$$

So every image gets a number.

Where does the classifier change its decision? This happens when the score is exactly zero:

$$2x_1 + 1x_2 - 6 = 0$$

Generally, for any linear classifier, the **decision boundary** is defined by the equation:

$$w^T x + b = 0$$

In two dimensions, this is a **line**:

$$w_1 x_1 + w_2 x_2 + b = 0$$

That line divides the plane into two halves:

- One half where the score is positive (the classifier predicts "cat");
- One half where the score is negative (the classifier predicts "not cat").

So the classifier is really asking: "*which side of the line does this point belong to?*". For binary classification, one hyperplane separates the two classes. For multiclass classification, each class has its own weight vector, so there are many competing hyperplanes. The predicted class is simply the one whose hyperplane produces the **largest score**.

6.4.2 Image-based Interpretation

In the previous section we interpreted each row of W geometrically as a hyperplane. Now we'll interpret exactly the same weights from a completely different perspective.

Our linear classifier computes, for every class i, a score:

$$s_i = \mathbf{w}_i^T \mathbf{x} + b_i$$

Previously, we interpreted:

- $\mathbf{x}$ as a point in a high-dimensional space;
- $\mathbf{w}_i$ as the normal vector of a hyperplane.

Now, ***what happens if we reshape*** $\mathbf{w}_i$ ***back into the original image dimensions?*** The answer is that each row of the weight matrix can be visualized as an **image template** for one class.

Remember how images were stored. Originally an RGB image had dimensions $R \times C \times 3$, where R is the number of rows, C is the number of columns, and 3 is the number of color channels. Before feeding it into the neural network we flattened it into a vector:

$$\mathbf{x} \in \mathbb{R}^d \quad \text{where} \quad d = R \cdot C \cdot 3$$

For example, for CIFAR-10, $R = 32$, $C = 32$, and $d = 32 \cdot 32 \cdot 3 = 3072$:

$$\mathbf{x}_{\text{CIFAR-10}} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{3072} \end{bmatrix}$$

Why does this make sense? First of all, each class has its own weight vector $\mathbf{w}_i$ and notice something interesting:

$$\mathbf{w}_i \in \mathbb{R}^d \rightarrow \mathbb{R}^{R \times C \times 3}$$

For example, for CIFAR-10, $d = 3072$ and we can reshape $\mathbf{w}_i$ back into the original image dimensions:

$$\mathbf{w}_{i,\text{CIFAR-10}} = \begin{bmatrix} w_{i,1} & w_{i,2} & \cdots & w_{i,32} \\ w_{i,33} & w_{i,34} & \cdots & w_{i,64} \\ \vdots & \vdots & \ddots & \vdots \\ w_{i,3072-31} & w_{i,3072-30} & \cdots & w_{i,3072} \end{bmatrix}$$

That means we can reverse the flattening operation and reshape the weight vector back into an image! Thus, the result is no longer just a vector, but a 3D array of pixel values that can be visualized as an image, which we call the **template image** for class i (or **learned template**). This particular image is **not** a clean

image of the class, but rather a template that captures the essence of that class, because the classifier is **not trying to generate realistic images**. It is trying to maximize classification accuracy (i.e., it learns colors, backgrounds, rough object locations, typical shapes, etc. that are useful for classification).

A second reason this makes sense is the **correlation**. Remember that the score for class i is computed as:

$$s_i = \mathbf{w}_i^T \mathbf{x} + b_i$$

The dot product $\mathbf{w}_i^T \mathbf{x}$ can be expanded as:

$$\mathbf{w}_i^T \mathbf{x} = \sum_{k=1}^{d} w_{i,k} x_k \quad \Rightarrow \quad s_i = \sum_{k=1}^{d} w_{i,k} x_k + b_i$$

This looks extremely familiar. In fact, it is exactly the same as the correlation between an image and a filter (template) that we saw on page 419 (or more generally, the linear filter on page 415):

$$T(I)(r,c) = \sum_{u,v \in U} w_{u,v} \cdot I(r+u, c+v)$$

The only difference is that:

- The filter is now **as large as the entire image**, instead of a small neighbourhood;
- Instead of sliding across the image, is is **applied only once** because both have the same dimensions (the sliding window is the entire image).

Thus, instead of saying "class i has weight vector $\mathbf{w}_i$", we can say "class i has learned a template describing what that class should look like".

> **Example 4: Interpreting the learned template for a class**
>
> For example, imagine we are classifying cars. Training gradually modifies $\mathbf{w}_{\text{car}}$ until it resembles the average characteristics of a car (i.e., the learned template for the car class). When a new image arrives, the classifier asks "*how similar is this image to my learned car template?*" If similarity is high, the score becomes high and the classifier is more likely to predict that the image is a car. If similarity is low, the score becomes low and the classifier is less likely to predict that the image is a car.

Why is this interpretation useful? This interpretation explains why linear classifiers work surprisingly well. Instead of memorizing images (which would require a huge amount of data), they learn one template per class (i.e., a single image that captures the essence of that class). Prediction then becomes simply a matter of comparing the new image to the learned template. This is much more intuitive than thinking only in terms of matrix multiplication.

Data/Image Augmentation

Imagine we have two images of an horse, similar but not identical. Since they look similar, the dot product $\mathbf{w}_{\text{horse}}^T\mathbf{x}$ will be similar for both images.

Now suppose the input image is flipped horizontally. Now the pixels no longer algin with the learned template. Even though **we immediately recognize it as a horse**, the linear classifier sees: "*this doesn't match my template very well*". The score becomes much lower. This is exactly the weakness of using a **single template**.

Why is this a problem? The learned template is **not invariant to transformations** (e.g., flipping, rotation, translation, scaling, etc.). This is a problem because the classifier will fail to recognize the same object if it appears in a different orientation or position. In other words, the classifier cannot specialize, it is forced to compromise.

Solution: Data Augmentation To solve this problem, we can use **Data Augmentation**. Suppose we **change the training set**. Originally we had one image of a horse. Now we can add more images of the same horse, but in different orientations and positions (e.g., flipped, rotated, translated, scaled, etc.). So instead of collecting more data, we create additional training examples by transforming existing ones. These transformations **do not change the class label**. A flipped horse is still a horse.

It is called *augmentation* because we are literally **augmenting** (**enlarging**) the training dataset. In image classification, data augmentation is commonly called **Image Augmentation** because we are augmenting images.

What changes inside the image template? The learned template for the horse class will now be modified to capture the essence of a horse in all orientations and positions. After seeing flipped images, it can no longer rely on "the left side of the horse" to recognize it. Instead, it must learn a more general template that captures the essence of a horse in all orientations and positions.

Limitations of this interpretation

This template interpretation is extremely useful because, in a **linear classifier**, each weight corresponds directly to one **input pixel**. Therefore, the weight vector can be reshaped into an image, allowing us to visualize the **learned template** for each class.

However, this interpretation becomes much harder in deeper neural networks. After the first layer, the input is no longer the original image but a set of **feature maps** (i.e., 3D arrays of learned features). Consequently, the weights no longer correspond to **image pixels**, but to combinations of **abstract features**. Since these weights cannot be directly visualized as images, the notion of a **class template** loses its intuitive interpretation. The network is actually learning richer and more powerful representations, but they are much more difficult for

humans to interpret. This is why the template interpretation is mainly useful for **linear classifiers** and, to some extent, **very shallow neural networks**.

Also, this section prepares us for convolutional neural networks (CNNs). Later, CNNs will also perform template matching, but with an important improvement:

- **Linear Classifier:** one large template covering the entire image.
- **CNNs:** many small templates (filters) applied everywhere in the image.

Thus, the image-based interpretation introduced here is the conceptual bridge toward convolutions.

6.4.3 Linear Classifier as Template Matching

In the previous section, we introduced the concept of image templates and their application in image classification. In this section, we will explore how a linear classifier can be interpreted as a template matching system. This interpretation provides a more intuitive understanding of how linear classifiers work and why they are limited in their ability to effectively classify images.

The Main Idea. After training, each class has learned **one template**:

$$T_i = W[i,:]$$

To classify an image I, the classifier compares it with **every learned template**. Mathematically, this can be expressed as:

$$s_i = W[i,:]^T x + b_i$$

After reshaping $W[i,:]$ into an image, this operation becomes the **correlation** between the input image and the template of class i. In other words, a linear classifier can be interpreted as a **template matching system**, because it computes the correlation between the input image and each class template.

In general, the whole prediction can be summarized as:

1. Take an input image I.
2. Compare it with template T_1 to compute score s_1.
3. Compare it with template T_2 to compute score s_2.
4. Repeat for all templates T_L to compute scores s_L.
5. Select the largest score s_i to determine the predicted class.

Every class **independently** computes $W[i,:]^T x + b_i$, and the **classifier** simply **selects the largest score**.

Why is this called template matching? Recall the correlation operation (page 416):

$$(I \otimes w)(r,c) = (I \star w)(r,c) = \sum_{u=-L}^{L} \sum_{v=-L}^{L} w_{u,v} \cdot I(r+u, c+v)$$

Now, in our case, nothing changes mathematically. The only difference is that the template is **learned automatically** during training. Instead of designing edge detectors or circle detectors by hand, gradient descent learns templates for each class (e.g., dog, car, horse, plane) directly from the data. This is why neural networks are often referred to as **learnable template matching systems**. So, in the correlation operation, the template w is now learned from data rather than being manually designed.

Advantages of this interpretation

There are several reasons why this viewpoint is useful:

1. It explains what the classifier computes. Instead of thinking "*a neural network performs matrix multiplication*", we can think "*the classifier compares the input image against learned templates*". This is much easier to visualize.
2. We can visualize the parameters. Because $W[i,:]$ has exactly the same size as the input image, we can reshape it and inspect what the model learned. This gives an intuitive understanding of important colors, important object parts, and dataset biases. Few machine learning models allow such direct visualization.
3. It explains mistakes. Suppose an airplane template contains lots of blue pixels. Then a blue sky automatically increases the airplane score. If another class also contains blue backgrounds, confusion becomes understandable. The templates therefore help explain why certain classification errors occur.

Why doesn't this work for deep networks?

The most important observation is that the template interpretation **does not disappear** in deep neural networks. Instead, it **changes**.

In a linear classifier, the template is correlated with the input image (pixels), and we can visualize it. However, in a deep neural network, after the first layer, the input is no longer pixels; instead, it is a feature map. Each layer learns templates over these **feature maps** rather than raw images.

Why is this a problem? A feature map is simply a collection of learned numerical features. For example, instead of RGB values, one channel may respond to vertical edges, another to curved textures, and another to animal fur. The next layer learns templates over these responses. Consequently, its **weights no longer resemble photographs**; they become high-dimensional feature detectors that humans cannot easily interpret. Therefore, the template matching interpretation still holds, but the templates are correlated with the outputs of previous layers, which are **hardly interpretable** as images.

In summary, the **template matching does not disappear** in deep networks; it simply operates on increasingly abstract representations, making it difficult to visualize or interpret. The only difference is **what it is matching**:

- In a linear classifier, the template matches pixels.
- In a hidden layer, the template matches learned features.
- In a CNN layer, the template matches local feature maps.
- In a deep CNN, the template matches high-level semantic features.

So the mathematical operation never changes; only the representation being compared becomes increasingly abstract.

Why is the linear classifier too simple?

The linear classifier is limited for two main reasons:

1. The model is too simple. Each class has **only one template**, which cannot capture all the variations in pose, viewpoint, scale, color, and deformation that real objects exhibit.

2. The dataset is limited. Even if the model were better, **small datasets produce noisy templates**. For example, if many cars in the dataset are red, the classifier will incorrectly associate red with cars due to the training statistics.

In summary, although linear classifiers are not competitive today, their mathematical operation appears everywhere. For example, the final classification layer of almost every CNN is still $Wx + b$. Thus, understanding template matching helps understand multilayer perceptrons, convolutional neural networks, and transformers, because they all eventually compute linear combinations followed by nonlinearities.

6.5 Why Image Classification is Difficult

6.5.1 Dimensionality

This section introduces the **first fundamental reason** why image classification is hard. Until now we have seen that an image can simply be flattened into a vector and fed into a linear classifier:

$$\mathbf{x} \in \mathbb{R}^d, \quad \mathbf{s} = W\mathbf{x} + b$$

So a natural question arises: ***if this works mathematically, why don't we just use larger neural networks?*** In other words, *why don't we just use a larger linear classifier with more parameters to learn the mapping from images to labels?* The answer is **dimensionality**.

⚠ Images are huge vectors

Recall that every RGB image can be represented as a vector. For the CIFAR-10 dataset, each image is 32×32 pixels with 3 color channels, which means that each image is a vector of size $d = 32 \cdot 32 \cdot 3 = 3072$.

The dimensionality 3072 may not sound enormous. Indeed, for modern machine learning, this is relatively small. However, the real issue is not only the number of dimensions, but also the way they are represented. **Every single pixel is an independent input feature**. This does not mean pixels are physically independent, but the **model treats them as unrelated variables**. So the model assigns a completely separate parameter to every pixel, without assuming any relationship between them.

In other words, treating each image as ordinary vectors wastes the spatial structure of the image. The model does not know that pixels that are close to each other are more likely to be related than pixels that are far apart. This is a fundamental limitation of linear classifiers and fully connected neural networks: they do not exploit the spatial structure of images.

Therefore, the problem is not that 3072 dimensions are inherently too many. Rather, **representing an image as a flat vector ignores its spatial organization**, and this becomes increasingly problematic as image resolution grows.

However, today's normal photographs are much larger than 32×32 pixels. For example, a 1024×1024 pixel image has $d = 1024 \cdot 1024 \cdot 3 = 3,145,728$ dimensions. So real images easily contain millions of input values.

⚠ Why high dimensionality is a problem

Imagine classifying handwritten digits. Suppose each image had only: $10 \times 10 = 100$ pixels. Already, every image is a point in a **100-dimensional space**. Now imagine CIFAR: 3072 dimensions. Each image is one point in a **3072-dimensional space** $\mathbb{R}^{3072}$. That space is unimaginably large. The issue is **not computational geometry**, but rather the **learning**.

Machine learning works by discovering patterns from examples. Suppose we only had 2 input variables (e.g., dog and cat). We could visualize all examples on a plane. Nearby examples often belong to the same class. This is the basis of machine learning: **similar inputs have similar outputs**.

Now imagine 3072 variables. The training data becomes incredibly sparse. Almost every possible input configuration has **never been observed**. This is one aspect of the **curse of dimensionality**: as dimensionality grows, the amount of data needed to adequately cover the input space grows explosively. Even simple grid-based intuition breaks down because the number of possible regions increases exponentially with the number of dimensions. Thus:

high-dimensional space $\rightarrow$ *few observed examples* $\rightarrow$ *difficult to generalize*

Example 5: The curse of dimensionality

For example, suppose we want to "cover" the space with training samples.

- **One dimension**: we have a line segment of length 1. To cover it with training samples spaced 0.1 apart, we need 10 samples.
- **Two dimensions**: we have a square of area 1. To cover it with training samples spaced 0.1 apart, we need 100 samples, because we need 10 samples along each axis, $10 \cdot 10 = 100$.
- **Three dimensions**: we have a cube of volume 1. To cover it with training samples spaced 0.1 apart, we need 1000 samples, because we need 10 samples along each axis, $10 \cdot 10 \cdot 10 = 1000$.
- d **dimensions**: we have a hypercube of volume 1. To cover it with training samples spaced 0.1 apart, we need 10^d samples, because we need 10 samples along each axis, 10^d.

Suppose our dataset contains $50,000$ images. In a 3072-dimensional space, those images are **extremely far apart**. There are huge regions of the space where the model has never seen any example. So when a new image arrives, the model is forced to **guess**.

The parameter explosion problem. The growing dimensionality of images also leads to a **parameter explosion problem**. Since a linear classifier is too simple, we might want to use a deeper neural network with a hidden layer. However, the number of parameters grows rapidly with the number of input dimensions. For example, a single hidden layer with 1536 neurons would require $3072 \cdot 1536 = 4,718,592$ weights, plus 1536 biases, $4,718,592 + 1,536 = 4,720,128$ parameters total. This is a **huge number of parameters to learn from a limited dataset**, leading to **overfitting and poor generalization**.

Imagine having 5 million parameters but only $50,000$ training images. The network could simply memorize the training set.

The real problem is not only the number of parameters! Even if we had infinite GPU power, high dimensionality would still be challenging. This is because images occupy an enormous input space. Changing just one pixel creates a different point. Changing two pixels creates another point. Changing ten pixels creates yet another point. The number of possible images grows astronomically. Most of those images are never observed during training. Therefore, the **classifier must infer the correct label from only a tiny fraction of all possibilities**.

Why CNNs will solve this

The problem is that a fully connected network assumes that **every pixel is equally important and independent**. But images have much richer structure: nearby pixels are correlated, edges repeat, corners repeat, textures repeat, objects appear everywhere. Instead of learning millions of unrelated weights, CNNs reuse the same small filters across the entire image, dramatically reducing the number of parameters while exploiting spatial locality. This is exactly why CNNs become the natural architecture for images. We will see this in the next sections.

6.5.2 Label Ambiguity

The previous challenge was **dimensionality**: images contain too many input variables (pixels) for a model to learn from. The next challenge is a different kind of problem: **even if we had a perfect representation of the image, the correct label may not be unique**.

What is label ambiguity?

Suppose we show the following picture: *A man sitting at a restaurant with a beer and sausages on the table.* What is the correct label? Man, Beer, Dinner, Restaurant, Sausages, Food, Table, or something else? All of these describe the same image, and **none of them is objectively wrong**.

So, **Label Ambiguity**, in image classification, means that a single image can reasonably correspond to more than one possible class, making it impossible to assign a unique "*correct*" label based solely on the visual information.

Why is label ambiguity a problem? In supervised learning we assume that every training image has **one ground-truth label**. For example, with the previous image, the dataset label might be "Restaurant". However, the network may instead predict "Dinner". *Is that prediction incorrect*? From a human point of view, it is not. The image certainly predicts a dinner scene. But from the dataset's point of view, the network is wrong because the annotation only accepts "Restaurant" as the correct label. This creates an intrinsic ambiguity in the training data.

Also, notice that the ambiguity does not come from the neural network itself (the **issue is not the network**). It comes from **the way datasets are labeled**. A single image often contains multiple objects, multiple activities, and multiple concepts. Yet, many datasets force us to choose **exactly one label**.

Why does this make learning harder? During training, the network minimizes the loss assuming there is **one correct answer**. For example, suppose the dataset says "Label = Restaurant", but the network predicts "Dinner". Although the prediction is perfectly reasonable, the loss function still treats it as 100% wrong. Consequently, the **model is penalized for making a semantically valid prediction**.

If the problem is a dataset labeling issue, why not just assign multiple labels to each image? This is a good idea. In fact, many modern datasets do exactly this. Instead of assigning a single class, they use **multi-label classification**, where one image can belong to several classes simultaneously. For example, the image of a man at a restaurant could be labeled as both "Restaurant" and "Dinner". However, throughout this part of the course we are studying the simpler **single-label classification problem**, where every image must belong to exactly one class. Label ambiguity is therefore an **inherent limitation of this formulation**.

6.5.3 Transformations

After dimensionality and label ambiguity, another challenge in image classification is the presence of **transformations** in the images. Many transformations change the image dramatically, while leaving its semantic class unchanged. This is one of the fundamental reasons why image classification is difficult.

The central idea. Humans have no problem recognizing an object under many different appearances. For example, consider a picture of a cat. Now imagine we apply some transformations: translate it a few pixels, rotate it slightly, scale it, crop it, mirror it, change its position inside the image. Every transformed image still represents a cat. The **pixels have changed**, often a lot, but the **semantic meaning** (i.e., the class) **has not changed**. This is exactly what makes computer vision hard: the same object can look very different in different images, and yet we want to classify them as the same class.

Why is this difficult for a linear classifier?

Recall how our simple linear classifier works. After flattening, an image becomes a vector of pixel values:

$$\mathbf{x} \in \mathbb{R}^d$$

The classifier computes:

$$\mathbf{s} = W\mathbf{x} + b$$

Where W is a weight matrix and b is a bias vector. The output $\mathbf{s}$ is a vector of scores, one for each class. The predicted class is the one with the highest score. However, notice that the classifier only sees d numbers (e.g., $d = 3072$ for CIFAR-10 dataset). It has **no notion** of objects, shapes, positions, rotations. It only compares pixel values. Therefore, moving an object even slightly changes many entries of the input vector $\mathbf{x}$, which can drastically change the output scores $\mathbf{s}$.

Desired property: Invariance

Ideally, we would like our classifier to be **invariant** to transformations. This means that if we apply a transformation to an image, the predicted class should remain the same. Formally, if T is a transformation and $\mathbf{x}$ is an image, we want:

$$\text{class}(T(\mathbf{x})) = \text{class}(\mathbf{x})$$

In other words, the **predictions should remain unchanged**, even though the pixels have changed. This property is called **Transformation Invariance**.

Couldn't we reach invariance by simply applying *Data Augmentation*? Data Augmentation (page 442) is very simple, because it artificially creates new training images by applying transformations that do not change the image label. For example, we can take an image of a cat and create new images by rotating it, translating it, scaling it, etc. This way, the classifier sees many different appearances of the same object during training. Therefore, the **model generalizes much better to unseen images** than a model trained on the original dataset.

Ok, so we win! We apply data augmentation, and then we're done, right? Not quite. Data Augmentation does not magically "*give*" invariance. Instead, it **encourages the network to learn invariance** (or, more precisely, **robustness**) by repeatedly seeing transformed versions of the same object during training. However, there are **several limitations**:

- We cannot augment with **every possible transformation**, because there are essentially **infinitely many possible transformations**. It is impossible to collect examples for every possible variation.
- Some transformations are **hard to model**. For example, consider a picture of a cat. If we rotate it by 90 degrees, it is still a cat. But if we rotate it by 180 degrees, it may look like an upside-down cat, which is not a common appearance. Similarly, if we scale it down too much, it may become unrecognizable. Therefore, some transformations can change the semantic meaning of the image.

In general, a classifier **must generalize** from the example it has seen to unseen examples. Data augmentation helps, but it cannot guarantee invariance to all possible transformations.

Why this motivates CNNs? This challenge is one of the strongest motivations for convolutional neural networks. CNNs exploit two key ideas: **local receptive fields** (detect local patterns regardless of where they appear); **shared filters** (the same detector is applied everywhere in the image). As a result, if a feature moves slightly, the same filter can still detect it. A fully connected network does **not** naturally possess this property.

Illumination Changes

One of the most common transformations in images is a **change in illumination**.

An image can be taken under very different lighting conditions. For example, the same object may be photographed: on a sunny day, on a cloudy day, at sunset, at night, indoors, with artificial light, with shadows partially covering the object, etc. Although the **appearance of the pixels changes significantly**, humans immediately recognize that the object is still the same.

For a classifier, however, this is a very difficult problem. A change in illumination can drastically change the pixel values, and therefore the input vector $\mathbf{x}$. As a result, the output scores $\mathbf{s}$ may change significantly, leading to incorrect predictions. This is another example of how transformations can make image classification challenging.

Again, *can data augmentation help?* Of course, as we discussed previously, it can improve robustness by generating training images with random brightness, contrast, color jitter and gamma changes. The network therefore learns that these different appearances correspond to the same class. However, just as with geometric transformations, **data augmentation cannot cover every possible lighting condition**. The model must still generalize to unseen illumination changes.

So CNNs are generally more robust than fully connected networks because they learn **local features** such as edges, corners, textures, which are often less sensitive to global brightness changes than raw pixel values. This does **not** make CNNs perfectly illumination invariant, but it makes learning much easier than relying on raw pixel intensities alone.

Deformations and Viewpoint Changes

Besides geometric transformations and illumination changes, objects may also appear very different because of **deformations** or **changes in te viewpoint** (camera position). Despite these changes, humans immediately recognize that the object is the same, whereas a classifier operating directly on pixels may struggle.

What is a deformation? A **Deformation** changes the **shape of the object itself**. Unlike a rigid transformation (translation or rotation), the object is no longer simply moved or rotated. Instead, different parts of the object change their relative positions. For example a smiling face, a bent arm, a running dog, a cat stretching, a waving flag, a crumpled piece of paper, etc. In all these cases, the object is still the same, but its geometry has changed.

Why is deformation difficult to handle? Similar to the other transformations, a **deformation can drastically change the pixel values of an image**. For example, a cat stretching its body may look very different from a cat sitting still. A classifier that relies on raw pixel values may fail to recognize that both images represent the same class.

What is viewpoint change? A **Viewpoint Change** occurs when the **camera observes the same object from a different position or angle**. The object itself does not change. Only the observer moves. For example, a car can be photographed from the front, the side, the back, or from above. A cat can be photographed from the left, the right, or from below. In all these cases, although these images look very different, humans immediately recognize them as the same object.

What is viewpoint change difficult to handle? Changing the viewpoint can dramatically modify the image. Different parts of the object may become visible, disappear, overlap each other, or appear larger or smaller because of perspective. A classifier that relies on raw pixel values may fail to recognize that these images represent the same class.

In general, humans do **not** memorize one image for every object. Instead, our visual system learns the **concept** of a car (wheels, windows, doors, headlights, etc.) and can recognize it from many different viewpoints. Deep CNNs aim to learn similarly **abstract features**, making them much more robust to viewpoint changes than linear classifiers.

The common goal of all these transformations remains exactly the same: **different images should still map to the same semantic class**.

6.5.4 Inter-Class Variability

This section introduces another fundamental challenge that is **different from all the previous ones**. Until now, we always assumed: "*The object is the same, but the image changes*". Now the situation is reversed: "*The label is the same, but the objects themselves can be very different*".

The fourth challenge in image classification is **Inter-Class Variability**. The idea is very simple: **images belonging to the same class may look dramatically different from one another**. Previously we had translation, rotation, illumination, deformation, viewpoint, where **one object** generated many different images. Now we have **many different objects** that all belong to **the same semantic class**.

What does "inter-class variability" mean? The name can be a little confusing. Here, "*variability*" refers to the **variation inside a semantic class**. For example, consider the class "dog". This class includes, Labradors, Bulldogs, Golder Retrievers, Epagneul Breton, which may have completely different colors, sizes, ears, body portions, and so on. Yet every image must receive exactly the same label: "dog".

Why is inter-class variability a problem?

Recall our linear classifier:

$$\mathbf{s} = W\mathbf{x} + b$$

Earlier interpreted every row of W as a **template** for a class. For example, template for class "dog" would be a prototypical dog image. But now, ***what should the template for class "dog" look like?*** Should it be a Labrador, a Bulldog, or a Golden Retriever? The answer is: **there is no single appearance that represents every possible dog**. A single template therefore struggles to match all the visual variations inside the class. The humans do not memorize every dog individually. Instead, we learn the abstract concept of "dog" and can recognize a dog even if we have never seen that particular dog before. Deep neural networks attempt to learn these higher-level concepts automatically.

Why data augmentation does not solve inter-class variability?

Earlier we said that data augmentation helps because we generate new images of the same object. But can data augmentation help with inter-class variability? The answer is **no**. Data augmentation can only generate new images of the same object, but it cannot generate new objects that belong to the same class. Those are **different objects**, not different transformations of the same object. So the model still needs many real examples from the class to learn all its variations.

Why CNNs help with inter-class variability? CNNs do not learn one giant template for each class (e.g., for a "dog"). Instead, they learn many reusable features, such as "ear", "tail", "eye", "nose", and so on. These features can be combined in many different ways to recognize many different objects that belong to the same class.

6.5.5 Perceptual Similarity vs Pixel Similarity

When we say that two images are "*similar*", we must ask: ***similar in what sense?***; ***Similar according to whom?*** There are actually two completely different notions of similarity:

1. **Pixel Similarity**: two images are pixel-similar if corresponding pixels have similar values.

   A **computer** initially sees an image only as a vector, for example, for CIFAR-10, a vector $x \in \mathbb{R}^{3072}$. The simplest way to compare two images is to compare their pixels:

   $$\text{Pixel Similarity}(x_1, x_2) = \|x_1 - x_2\|_2$$

   Or another pixel-wise distance.

   - If most pixels are identical, the distance is small.
   - If many pixels differ, the distance is large.

   The computer therefore concludes: "*these images are similar because their pixels are similar*".

2. **Perceptual Similarity** (or **Semantic Similarity**): ignores the exact pixel values and focuses on **what the image represents**.

   Humans work very differently. We **don't compare images pixel by pixel**. Instead, we compare objects, shapes and structures, as well as semantics. To us, two different breeds of dog are clearly similar, even though they are made up of completely different pixels. Humans still perceive them as similar because they share the same *concept* of "dog".

Why are these two notions of similarity so different? Let's revisit everything we have said about the difficulty of image classification. Suppose we have a dog image. Now create five new images by applying the following transformations to the original image:

1. **Translation**: Move the dog to the left or right.
2. **Rotation**: Rotate the dog by a few degrees.
3. **Shadow**: Add a shadow to the dog.
4. **Different Viewpoint**: Take a picture of the dog from a different angle.
5. **Different Pose**: Take a picture of the dog in a different pose.

Humans immediately say "*these are all pictures of the same dog*". But mathematically, each transformed image has many different pixels. The Euclidean distance between pixel vectors may become very large.

Now consider another example. Take a picture of a "Wolf" and a picture of a "Dog". The pixel values may actually be closer than a transformed image of the same dog. Yet semantically, the first pair belongs to two different classes, while

te second pair belongs to the same class. This shows that **pixel similarity and semantic similarity are fundamentally different concepts**.

Every challenge we have discussed so far in this section is a manifestation of the same underlying problem: **images that humans perceive as similar may have very different pixel values**. Although our simple linear classifier was effective in representing pixel similarity, it was completely ineffective in representing abstract concepts due to limitations in semantic similarity.

Again, this is why we need CNNs. Instead of comparing single pixels, CNNs compare **features** of images. At the deepest layers, the representation is no longer based on raw RGB values. Instead, it encodes **semantic features**. Consequently, the similarity measured inside a CNN becomes much closer to **perceptual similarity** than to raw pixel similarity. This is why CNNs are dramatically better than linear classifiers.

6.6 Nearest Neighbour Classifiers for Images

After spending the previous section understanding **why image classification is difficult**, now we introduce what is probably the **simplest possible classifier**.

The idea is almost simple: "*If we have already seen a similar image, we will give the new image the same label*". No training, no weights, no neural network. Just compare images and copy the label of the most similar one. Before doing that, however, we immediately face a fundamental question: ***what does "similar" mean for two images?***

6.6.1 Pixel-wise Distances

Suppose we have two images:

$$I_1, I_2 \in \mathbb{R}^{R \times C \times 3}$$

Since we already know how to flatten an image, we can represent them as vectors:

$$x_1, x_2 \in \mathbb{R}^d, \quad d = R \cdot C \cdot 3$$

Now the problem becomes much simpler. Instead of comparing **images**, we compare **vectors**.

Measuring the distance between two vectors

The most natural idea is to **compare every corresponding pixel**. If corresponding pixels differ only slightly, the images should be close. If many pixels differ, the images should be far apart. This is why these distances are called **Pixel-Wise Distances**.

Euclidean ℓ_2 distance. One of the most common pixel-wise distances is the **Euclidean distance**. For two flattened images:

$$d(x_1, x_2) = \|x_1 - x_2\|_2$$

Expanding the norm:

$$d(x_1, x_2) = \sqrt{\sum_{i=1}^{d} \left(x_1^{(i)} - x_2^{(i)}\right)^2}$$

Where:

- $x_1^{(i)}$ is the i-th pixel of the first image.
- $x_2^{(i)}$ is the i-th pixel of the second image.
- $x_1^{(i)} - x_2^{(i)}$ is the difference between the two pixels. Subtract corresponding pixels and if a pixel is identical, its contribution is zero. If the pixels are very different, the contribution is large.

- $\left(x_1^{(i)} - x_2^{(i)}\right)^2$ is the squared difference. Squaring ensures that the contribution is always positive and emphasizes larger differences.
- $\sum_{i=1}^{d} \dots$ is the sum of all squared differences. Sum over all pixels to get a single number that represents the overall difference between the two images.
- $\sqrt{\dots}$ is the square root. This restores the units of the original pixel values and gives the final Euclidean distance.

Geometrically, the Euclidean distance is simply **the straight-line distance between two points in this 3072-dimensional space** (for CIFAR-10).

Manhattan ℓ_1 distance. Another very common distance is the **Manhattan distance** (also called ℓ_1 distance):

$$d(x_1, x_2) = \|x_1 - x_2\|_1 = \sum_{i=1}^{d} \left| x_1^{(i)} - x_2^{(i)} \right|$$

Here, instead of squaring the differences, we simply take the absolute value.

Why is this idea attractive? Imagine we already have 10,000 labeled training images. Given a new image, we can compute the distance to all 10,000 training images and find the closest one. That sounds extremely reasonable. No optimization, gradient descent, or learning. Only distance computations.

But there is a hidden assumption. This whole approach assumes that **if two images have similar pixels, they should represent similar objects**. Mathematically, it assumes small pixel-wise distances imply small semantic distances. At first glance, this seems perfectly reasonable. Unfortunately, everything we have learned in previous sections tells us that this assumption is often false. For example, a dog shifted by just a few pixels can have a surprisingly large Euclidean distance from the original image, while a completely different object with a similar background or color distribution might end up being closer in pixel space.

We conclude the previous section with a simple but important observation: **perceptual similarity is not pixel similarity**. Now, the nearest-neighbour method is built entirely on **pixel similarity**. So before even seeing the complete algorithm, we should already suspect something: **this classifier will fail whenever pixel similarity does not correspond to semantic similarity**. That is exactly the limitation of nearest-neighbour classifiers for images.

6.6.2 k-NN for Images

In the previous section, we answered the question: ***how can we measure similarity between two images?*** Our answer was Euclidean distance (ℓ_2 distance) and Manhattan distance (ℓ_1 distance) between the pixel values of the two images. Now we use these distances to build an actual classifier. This classifier is called **k-Nearest Neighbours (k-NN)**.

The main idea. Suppose we have a huge collection of labeled images. Now a new image arrives, and we want to classify it. Instead of learning a model, k-NN simply asks: ***which training images look most similar to this new image?*** Then it predicts the label from those neighbors.

Why is it called "*Nearest Neighbour*"? Because every image is now a point in $\mathbb{R}^d$, where d is the number of pixels in the image (after flattening). The query image (i.e., the new image we want to classify) is also a point in $\mathbb{R}^d$ and it has no label (because it is new). We compute the distance from the query to every training image, and the closest ones are called its **nearest neighbours**.

The algorithm

Suppose we have a training set:

$$TR = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$$

Where:

- n is the **number** of training **images**.
- $x_i \in \mathbb{R}^d$ is the i-th **training image** (flattened).
- $y_i \in \mathbb{R}$ is the **label** of the i-th training image.

Given a new image (query) x_q, the algorithm works as follows:

1. Compute the distance from the query to every training image. For example:
$$d(x_q, x_1), \quad d(x_q, x_2), \quad \ldots, \quad d(x_q, x_n)$$
Usually we use Euclidean distance (ℓ_2 distance) or Manhattan distance (ℓ_1 distance).

2. Sort the distances. For example, given a query image x_q, we might get:
$$d(x_q, \text{Dog}) = 3.2, \quad d(x_q, \text{Dog}) = 4.1, \quad d(x_q, \text{Cat}) = 5.0,$$
$$d(x_q, \text{Horse}) = 8.3, \quad d(x_q, \text{Car}) = 12.4$$
The smallest distances correspond to the most similar images. In this case, the query image is most similar to a *Dog*, then another *Dog*, then a *Cat*, then a *Horse*, and finally a *Car*.

3. Take the first k images. If $k = 3$, then the first three images are *Dog*, *Dog*, and *Cat*.

4. Vote for the most common label among the k images. In this case, the most common label is *Dog*, so we predict that the query image is a *Dog*.

The parameter k simply means ***how many neighbours do we consider?*** This is the only parameter of the algorithm.

- $k = 1$. This is the **simplest** possible classifier. We look only at the closest image and predict its label. Thus, the last step (voting) is not needed.
- $k = 3$. Instead of trusting one image, we trust the three closest images. We take a vote among them and predict the most common label. So increasing k makes the classifier **less sensitive to noise and mislabeled examples**.
- *What if k becomes too large?* If k is too large, the classifier will start to ignore the local neighborhood and will simply predict the most common classes in the training set. For example, if $k = n$, then the classifier will always predict the most common class in the training set, regardless of the query image. So:
  - Very small k, then there is too noise (i.e., mislabeled examples).
  - Very large k, then the classifier is oversmoothed (i.e., it ignores the local neighborhood).

Choosing k is therefore a trade-off that is typically selected using a validation set.

⚠ There is no training

This is the most important point about k-NN. Unlike neural networks, there is **no optimization problem**. So, no weights W are learned, no biases b are learned, no loss function is minimized, and no gradient descent is performed. The training phase is literally just storing the training images and their labels. This is why k-NN is called a **lazy learning algorithm**. It postpones all the work until a query arrives.

$ The cost of k-NN

The **computational cost** of a k-NN classifier is clear. For **every** new test image we must compute the distance to **every** training image. If each image has d number of pixels (the feature dimension after flattening), and we have n number of training images, then for each query image we must compute the distance to each of the n training images; and each distance computation compares d components (the pixel values). Thus, computing one distance requires $O(d)$ operations, and computing the distance to all n training images requires $n \times O(d) = O(n \cdot d)$ operations. This is a **huge cost** for **every single prediction**.

Another drawback is **memory**. A neural network stores only its learned parameters. In contrast, a k-NN classifier stores **every training image**. For CIFAR-10, that's already tens of thousands of images. For ImageNet, that's millions of images. So **memory grows directly with the training set size**.

The hidden assumption. Everything depends on the distance. The **algorithm assumes that a small pixel distance between two images implies that they are perceptually similar** (page 456). But this is not always true. The algorithm is only as good as the distance measure it uses. We will talk about this in the next section.

6.6.3 Why Pixel Distance is Not Perceptual Distance

This section is, in many ways, the **most important conclusion** of everything we have discussed so far. We have seen:

1. Images are represented as vectors of pixel values.
2. We compare vectors using Euclidean or Manhattan distance.
3. We build a k-NN classifier using those distances to find the nearest neighbors of a query (input) image.
4. **Now we ask**: *is this notion of similarity actually meaningful for images?*

The answer is **no**. Pixel distance is generally a poor measure of perceptual (semantic) similarity.[17]

⚠ The hidden assumption of k-NN

Recall how k-NN classifier works. For a query (input) image x_q, we compute the distance between x_q and all training images x_i:

$$d(x_q, x_i) = \underbrace{\sqrt{\sum_{j=1}^{n} (x_{q,j} - x_{i,j})^2}}_{\text{Euclidean}} \quad \text{or} \quad d(x_q, x_i) = \underbrace{\sum_{j=1}^{n} |x_{q,j} - x_{i,j}|}_{\text{Manhattan}}$$

The k-NN classifier is based on the idea that if two images have a small pixel distance, they are perceptually similar and belong to the same class. However, this **idea contains a hidden assumption**: pixel similarity is equivalent to semantic (or perceptual) similarity. As we saw in the previous section, that is **not true**.

≠ Two completely different notions of similarity

There are actually two different concepts of similarity as we have seen on page 456:

- **Pixel Similarity**. This is what Euclidean or Manhattan distance measures. Two images are considered similar if their corresponding pixels have similar RGB values. Mathematically:

  $$d(x_1, x_2) = \|x_1 - x_2\|_2 \quad \text{or} \quad d(x_1, x_2) = \|x_1 - x_2\|_1$$

  The algorithm never asks "*what object is present?*", instead it only asks "***how similar are the pixel values?***".

- **Perceptual Similarity**. Humans do something completely different. When we compare two images we ask:

[17] Remark: Perceptual (semantic) similarity means that two images are similar in a way that is meaningful to humans. For example, two images of cats are perceptually similar, even if they have different pixel values.

- *Is it the same object?*
- *Does it have the same shape?*
- *Does it represent the same concept?*

We ignore many pixel differences. For example, for us a dog in sunlight, a dog in shadow, a dog rotated or a dog translated is still the same dog. Our visual system focuses on **meaning**, not raw pixel values.

But why don't we like pixel distance as a metric? Because the Euclidean distance, or Manhattan distance, **treats every pixel independently**. Euclidean computes the distance between two images by summing the squared differences of each pixel:

$$\sum_i (x_i - y_i)^2$$

Every **pixel contributes equally** to the distance. Thus, the algorithm has **no idea** what an eye is, or a wheel, or a nose, or a face. It **only sees numbers**.

This problem is closely related to the section on page 447 that discusses the challenges of image classification:

- **Translation**: same object, we move all pixels to the right or left slightly, and the pixel distance becomes large, even though the images are perceptually similar.
- **Rotation**: same object, we rotate the image slightly, and the pixel distance becomes large, even though the images are perceptually similar.
- **Illumination**: same object, we change the lighting, and the pixel distance becomes large.
- **Deformation**: same object, we deform it slightly, and the pixel distance becomes large.
- **Viewpoint**: same object, we change the viewpoint, and the pixel distance becomes large.
- **Inter-class variability**: in this case, even worse, two images of the same class can have a **very large** pixel distances, while two images of different classes (e.g., a dog on grass and a wolf on grass) may accidentally have **smaller** pixel distances because of similar colors or backgrounds.

So, the **real problem** is **not** Euclidean or Manhattan distance, but rather the **pixels are not semantic features**.They are only measurements of light intensity. They are a very low-level representation of the image. In other terms, Euclidean distance is measuring similarity in the **wrong space**. It is measuring similarity in the **pixel space**, not the **semantic space**.

A better representation is needed

The above problem is the main reason to use deep learning for image classification. We will CNNs that instead of computing distances between raw pixels,

they compare different features of the image like edges, corners, textures, object parts, and so on.

Thus, we don't necessarily need a better distance. We first **need a better representation of the image**.

If k-NN is not good for images, why do we even discuss it? For different reasons:

- We tried to build the simplest possible image classifier and saw why that was impossible (because pixel distance is not the same as perceptual distance).
- **They are still used!** Suppose we have a database of 10 millions images. A user uploads a picture of a dog. They don't necessarily want to classify it, they want to **find similar images**. Thus, we can use a CNN to extract features from an image and create an embedding, which is a vector containing 512 (for example) numbers that represent the features the network has learned to be useful for distinguishing classes (e.g., dog face or dog body). Then, we can use k-Nearest Neighbors (k-NN), which calculates the distance between the embedding of the query image and the embeddings of all images in the database and returns the closest ones. This is called **image retrieval**.

Today, the k-NN classifier is not used for image classification, but it is still used for image retrieval combined with deep learning. The main idea is to use a CNN to extract features from the image, and then use k-NN to find the closest images in the feature space. This is a very powerful technique that is used in many applications, such as Google Images, Pinterest, and so on.

6.6.4 CIFAR-10 and t-SNE Interpretation

Until now, we argued theoretically that:

- Images are represented as very high-dimensional vectors;
- k-NN compares images using pixel distances;
- Pixel distances do **not** correspond to semantic similarity, i.e., perceptual distances.

Now, we ask ourselves: ***can we actually see this phenomenon in practice?*** The answer is **yes**, using **t-SNE**.

Recall that each CIFAR-10 image is represented as $x \in \mathbb{R}^{3072}$. Obviously, we cannot visualize points in a 3072-dimensional space. The idea is therefore: project the 3072-dimensional vectors into **2 dimensions**, while trying to preserve their neighbourhood relationships. This is exactly what **t-SNE** does.

What is t-SNE?

t-distributed Stochastic Neighbor Embedding (t-SNE) is a **dimensionality reduction technique** that is particularly well-suited for the visualization of high-dimensional datasets. We don't go into the details of how t-SNE works because we will use it only as a **visualization tool**.

Its goal is simple: given a set of high-dimensional vectors, t-SNE finds a low-dimensional representation of the data that preserves neighbourhoods. For example, if two images are close in the original space, they should also appear close in the 2D visualization. Likewise, if two images are far apart, they should also appear separated.

What can we expect to see?

Imagine Euclidean distance perfectly reflected semantic similarity. Then, all images of the same class would be clustered together in the 2D visualization. But this is exactly what we would hope for if pixel distances were approximately equal to perceptual distances. In other words, if pixel distances were a good proxy for semantic similarity, we would expect to see **well-separated clusters of images of the same class** in the t-SNE visualization.

What actually happens on CIFAR-10? As we will see, **this is not what happens**. Instead, the classes are heavily mixed. The different classes overlap significantly. This means that, in the original pixel space:

- Many dogs are closer to cars than to other dogs;
- Many airplanes are close to birds;
- Many ships are mixed with trucks.

The neighbourhoods are **not semantic**. And we have already argued the reason: the Euclidean distance only measures pixel-wise differences independently of the spatial structure of the image. Thus, two images with similar backgrounds or colours may therefore be close in pixel space even if they depict completely different objects.

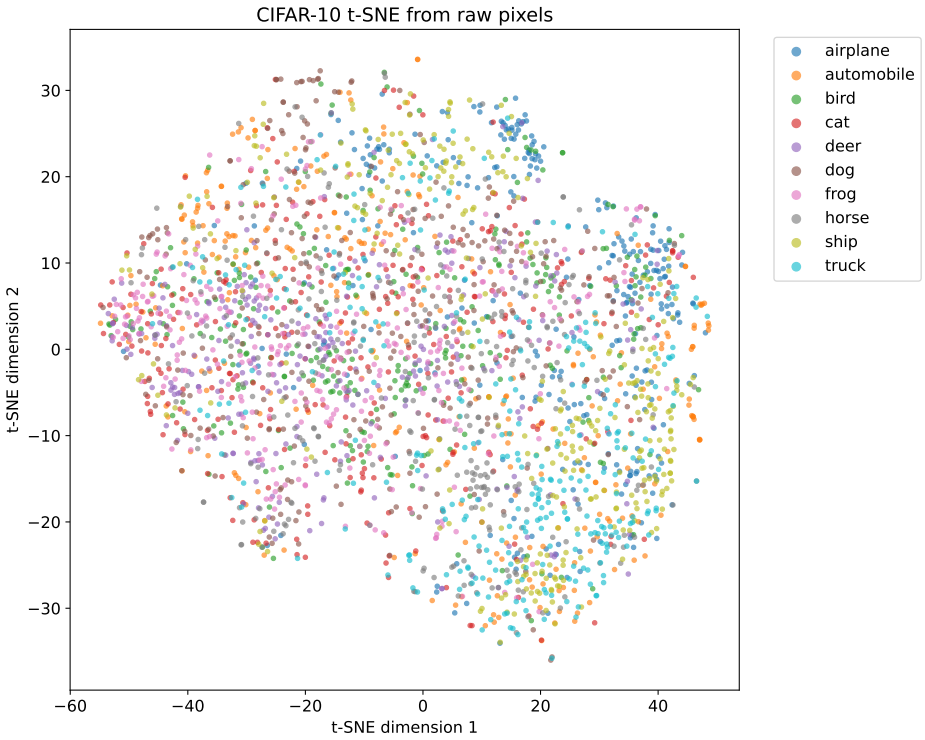


Figure 51: t-SNE visualization of CIFAR-10 images in the original pixel space.

Each image is composed of 32×32 pixels, each with three RGB values. The t-SNE algorithm projects the 3072-dimensional vectors into 2D while trying to preserve neighbourhoods. The points are colored according to their class.

Observation. Every color is mixed with every other color. There are **no clear clusters** and almost every class overlaps with the others. For example airplanes are mixed with trucks, ships are mixed with automobiles, and so on. There are only tiny local groups. This phenomenon occurs because each image is represented by 3072 RGB values. Two images that are semantically identical can have very different pixel values.

Note: we are not applying any k-NN classifiers here. We have found that when images are compared using pixel distances, as the k-NN algorithm does, many images of different classes appear closer to each other than to images of the same class. This is precisely why k-NN performs poorly on CIFAR-10.

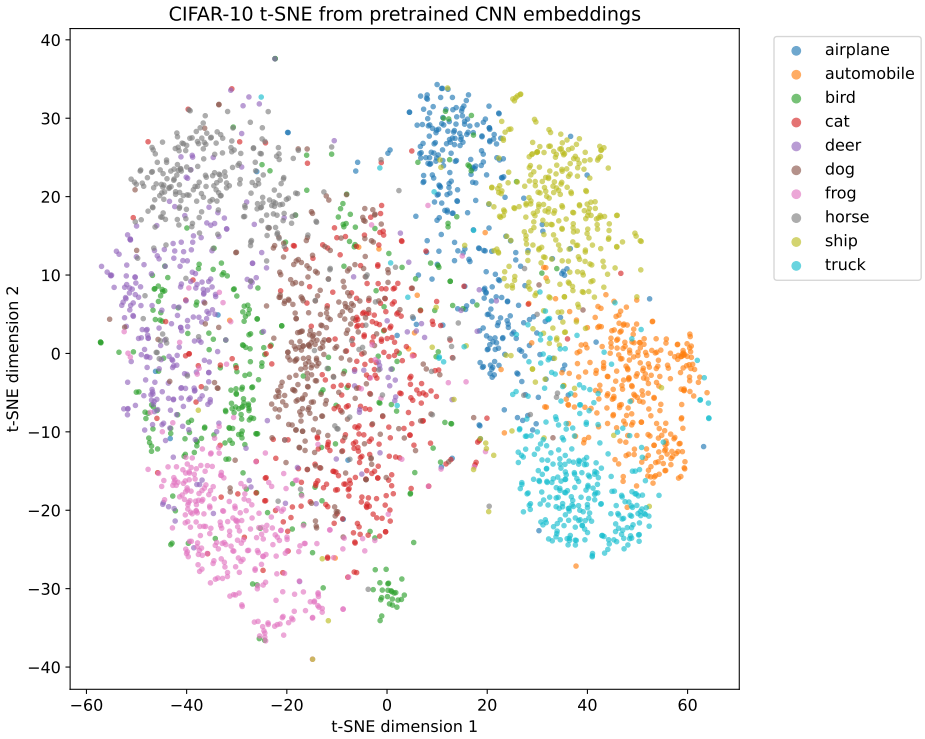


Figure 52: t-SNE visualization of CIFAR-10 images in the CNN embedding space.

Each image is passed through a Convolutional Neural Network (CNN), specifically a pretrained ResNet18. The final classification layer is removed, and the output of the feature-extraction part of the network is used as a new vector representation of the image. Therefore, each image is represented by a 512-dimensional embedding.

The t-SNE algorithm projects these 512-dimensional vectors into two dimensions while trying to preserve local neighbourhood relationships. Each point represents one image and is coloured according to its ground-truth class.

Observation. The CNN produces a representation that is much more semantically meaningful than the original pixel representation. Images belonging to the same class tend to be close to one another, while different classes form more distinguishable regions. For example, airplanes, ships, automobiles, and trucks are much more clearly separated than in the raw-pixel representation.

However, the classes are not perfectly separated. Some semantically related categories, especially animals such as cats, dogs, deer, and horses, still partially overlap because they share visual features such as fur, heads, eyes, legs, and similar body shapes.

Since neighbourhoods in this embedding space are more semantically meaningful, a k-NN classifier applied to these embeddings would be expected to perform

much better than a k-NN classifier applied directly to raw pixels.

This is also the idea behind **image retrieval systems**, where a query image and the images in a database are converted into embeddings and compared in the learned feature space rather than in the original pixel space.

This experiment motivates the use of Convolutional Neural Networks (CNNs) for image classification. Using any pixel-wise distance measure, and in particular the Euclidean distance, is **not sufficient to capture semantic similarity** (see Figure 51). Instead, a **CNN learns an embedding (latent representation)** in which semantically similar images are close together (see Figure 52).

7 Convolutional Neural Networks (CNNs)

In the previous section, we discussed **image classification** using relatively simple models:

- Linear classifiers, which are based on linear decision boundaries
- Multi-layer perceptrons, which are based on fully connected layers

These models can classify images, but they have an important limitation: **they treat an image simply as a long vector of numbers**. For example, a 32×32 RGB image becomes:

$$32 \times 32 \times 3 = 3072$$

Independent input features. The model does **not know** that nearby pixels are related, or that edges form shapes, or that shapes form objects. Every pixel is treated almost independently.

CNNs were invented to exploit one fundamental property of images: **pixels that are close together *usually* belong to the same visual structure**. Instead of learning from isolated pixels, CNNs learn **local visual patterns**, such as: edges, corners, textures, object parts, and complete objects. These patterns are called **features**, and learning them automatically is the central idea of this section.

7.1 From Hand-Crafted to Data-Driven Features

Before deep learning became popular, computers did **not** learn these visual features automatically. Instead, engineers manually designed algorithms that extracted useful information from an image before classification. The complete pipeline for image classification was as follows:

$$\text{Image} \rightarrow \text{Feature Extraction} \rightarrow \text{Classifier}$$

Where:

- The **feature extraction** was designed by humans!
- Only the **classifier** was learned from data.

This chapter begins by explaining why this approach eventually became insufficient, motivating CNNs.

7.1.1 The Feature-Extraction Perspective

This first section asks a simple but fundamental question: ***why can't we classify images directly from pixels?*** The answer, already hinted at in the previous section, is that **raw data usually contains much more information than is actually useful for the task**. Our goal is therefore not simply to classify images. Our goal is first to find a **better representation** of an image.

What is a feature?

Let's start from the beginning. A **Feature** is any **quantity extracted from the original data** that helps distinguish one class from another.

For example, imagine distinguishing between cats and dogs. The raw image contains thousands of pixel values. However, what really matters are properties such as ears, eyes, fur texture, snout shape, body portions. These are much more informative than the extract RGB value of pixel $(125, 87)$.

A feature is therefore **a representation that emphasizes useful information while ignoring irrelevant details**.

Why raw pixels are a poor representation

Suppose we have a grayscale image of size 100×100. The classifier receives a vector of $100 \times 100 = 10,000$ pixel values. Most of these numbers are **not directly meaningful**. For example, consider two photos of the same cat. The pixel values may change because of lighting, shadows, camera position, slight translations, background, image noise, or even the cat's pose.

Although the **object is the same**, thousands of **pixel values are different**. The classifier therefore has a difficult task: it must discover from scratch that all these images represent the same object.

Feature Extraction

Instead of using all raw pixels, we can transform the image into a more meaningful representation. Conceptually:

$$\text{Image} \rightarrow \text{Feature Extraction} \rightarrow \text{Feature Vector}$$

The feature extractor **removes unnecessary information** and **keeps only what is useful** for recognition.

For example, instead of $10,000$ pixel values, we might obtain only 100 meaningful descriptors. Those descriptors may describe edges, corners, textures, shapes, orientations, or other properties of the image, depending on the extraction method. The classifier now receives information that is much easier to separate into classes.

Dimensionality reduction

Another advantage is that feature extraction usually **reduces the number of variables**. Instead of working with $10,000$ pixel values, we may work with only a few hundred informative features.

This has several **advantages**:

- ✔ Less memory is required to store the data.
- ✔ Faster computation, since the classifier has fewer variables to process.
- ✔ Fewer parameters to learn, which reduces the risk of overfitting.
- ✔ Lower risk of overfitting, since the classifier has fewer variables to process.
- ✔ More robust classification, since the classifier is less sensitive to irrelevant details in the input.

Notice that **dimensionality reduction is not the primary goal**. The real goal is to produce a **better representation**. The reduction in dimensionality is often a **side effect** of removing irrelevant information.

The classifier is only as good as its features

This is one of the most fundamental principles in machine learning. The performance of a model is ultimately limited by the quality of its input features. If the input data does not contain informative or discriminative features, even a very powerful model will struggle to learn a good decision boundary. This idea is commonly summarized by the expression: "***Garbage in, garbage out.***".

The same principle applies to image classification. A classifier does **not invent information**. It can only use the information it receives. Imagine two situations:

- **Bad features**. Suppose we want to distinguish apples from oranges, but our feature extractor only measures *average brightness*. Both fruits may have similar brightness. The classifier has almost no useful information. Even a very powerful neural network will struggle to separate the two classes.
- **Good features**. Now suppose the features include color, texture, roundness, diameter. These characteristics clearly separate the two fruits. Even a relatively simple classifier can easily distinguish between apples and oranges.

As we said before, good features are often more important than a sophisticated classifier. In fact, before CNNs became popular, much of computer vision research focused on inventing better feature extractors rather than better classifiers.

Key Takeaways 1

- A raw image is usually **not the best representation** for classification.
- A **feature** is a more informative description of the image.
- Feature extraction removes irrelevant information while preserving useful patterns.
- It often reduces dimensionality, making learning easier.
- The quality of the classifier depends heavily on the quality of the features.
- Before CNNs, these features were designed manually. CNNs will instead learn them automatically from data.

7.1.2 Hand-Crafted Features for Image Classification

The previous section introduced the general principle:

$$\text{raw image} \rightarrow \text{useful representation} \rightarrow \text{classifier}$$

Now, we will discuss how this was traditionally implemented before data-driven feature learning became dominant. The central idea is that the image is first converted into a **small vector of manually designed measurements**, and only then passed to a classifier. This approach is known as **hand-crafted features**.

Hand-Crafted Features

A **Hand-Crafted Feature** is a **measurement chosen and implemented by a human expert**. It is called *hand-crafted* because the learning algorithm does not decide what information to extract. A person designs the extraction procedure based on prior knowledge of the problem. The important point is that these choices come from **human reasoning about the task**.

Example 1: Hand-Crafted Features

For example, suppose we want to build a system that classifies fruit images. The input is an RGB image and the goal is to predict the type of fruit: apple or orange.

- The computer **doesn't know what an apple or an orange looks like**. It only receives pixels, thousands of numbers, and those numbers alone are not easy to classify.
- Now imagine we are an engineer. We ask ourselves: "*what characteristics distinguish apples from oranges?*" We may come up with a few ideas: average color, roundness, texture, diameter, weight, etc. Suppose we implement algorithms that compute these characteristics from the image.

  For every image we obtain a small vector of measurements, for example:

$$\text{image} \rightarrow \begin{pmatrix} \text{average color} \\ \text{roundness} \\ \text{texture} \\ \text{diameter} \\ \text{weight} \end{pmatrix} \in \mathbb{R}^5$$

- We collect many examples of apples and oranges, then we can train a classifier on these vectors of measurements. The **neural network never sees the original image**, it only sees the measurements we designed.

This is the essence of hand-crafted features: we design a small set of measurements that we believe are useful for the classification task, and then we train a classifier on those measurements.

From an image to a feature vector

Let the input image be:

$$I \in \mathbb{R}^{r \times c}$$

It contains $r \cdot c$ pixel values. The feature-extraction algorithm transforms it into:

$$\mathbf{x} \in \mathbb{R}^d$$

Where usually $d \ll r \cdot c$. For example, the vector could be:

$$\mathbf{x} = \begin{pmatrix} \text{area} \\ \text{minimum height} \\ \text{mean height} \\ \text{maximum height} \\ \text{perimeter} \\ \text{aspect ratio} \end{pmatrix}$$

The image-classification problem is therefore split into two transformations:

$$I \xrightarrow{\text{feature extraction}} \mathbf{x} \xrightarrow{\text{classifier}} t$$

Where $t \in \Lambda$ is one class from the label set Λ. The classifier never sees the original image. It only sees the extracted vector $\mathbf{x}$.

Example 2: Continue Apple vs Orange

Suppose we extract these features from the fruit images:

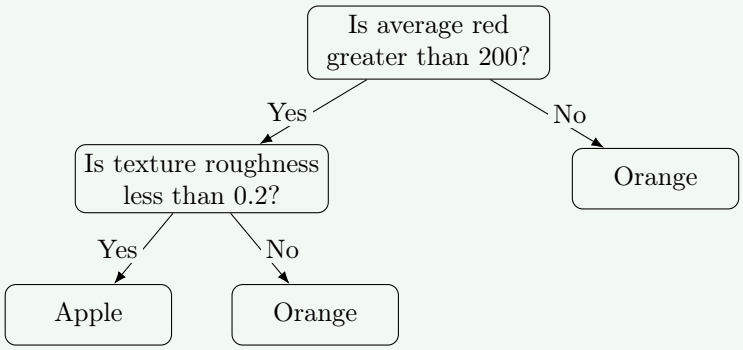
- Average red: 220
- Roundness: 0.95
- Texture roughness: 0.1

Now we have a small vector of measurements:

$$\mathbf{x} = \begin{pmatrix} 220 \\ 0.95 \\ 0.1 \end{pmatrix} \in \mathbb{R}^3$$

Instead of the original image, the classifier receives this vector $\mathbf{x}$ and predicts whether it is an apple or an orange.

As classifiers, we can use a simple decision tree where we ask a sequence of questions about the features. For example:



Again, the classifier never sees the original image. It only sees the extracted features and makes a decision based on them.

The classifier can also be learned from data

The decision rules could be manually written, but the more general approach is:

$$\text{hand-crafted feature extraction} \;+\; \text{data-driven classification}$$

The **engineer defines the features** (i.e., the hand-crafted features), but the **classifier learns how to combine them from labelled examples**.

Suppose the feature extractor produces a vector:

$$\mathbf{x} = \begin{pmatrix} \text{area} \\ \text{min} \\ \text{mean} \\ \text{max} \\ \text{perimeter} \\ \text{aspect ratio} \end{pmatrix} \in \mathbb{R}^d$$

A training set then consists of pairs of feature vectors and labels:

$$\mathcal{D} = \{(\mathbf{x}_1, t_1), (\mathbf{x}_2, t_2), \ldots, (\mathbf{x}_n, t_n)\}$$

Where:

- $\mathbf{x}_n$ is the manually extracted feature vector for the n-th image.
- t_n is the correct class label for the n-th image.

The classifier uses these examples to learn a mapping from feature vectors to class labels:

$$f_\theta : \mathbb{R}^d \to \Lambda$$

The parameters θ may belong to a decision tree, a linear classifier, a neural network, or another supervised classifier. So even in the traditional pipeline, part of the system is already data-driven. **What is not data-driven is the choice of representation (the features), which is still hand-crafted**.

Using a neural network as the classifier

At some point, researchers started asking: ***why not use a neural network after the hand-crafted feature extraction?*** This is perfectly reasonable, and the pipeline would look like this:

$$I \xrightarrow{\text{hand-crafted feature extraction}} \mathbf{x} \rightarrow \text{MLP} \rightarrow \text{class probabilities}$$

Where the MLP (Multi-Layer Perceptron) is a neural network that takes the hand-crafted feature vector $\mathbf{x} \in \mathbb{R}^d$ as input and outputs class probabilities. The neural network then contains:

- An **input layer** of dimension d (the number of hand-crafted features).
- One or more **hidden layers** with a certain number of neurons.
- An **output layer with one output for each class**.

After training, the outputs may represent probabilities such as:

$$\text{MLP}(\mathbf{x}) = \begin{pmatrix} P(\text{apple} \mid \mathbf{x}) \\ P(\text{orange} \mid \mathbf{x}) \end{pmatrix}$$

The predicted class is typically the one with the highest probability:

$$\hat{l} = \arg\max_{l \in \Lambda} P\left(l \mid \mathbf{x}\right)$$

Notice that this is still **not a CNN**. The neural network only learns the classifier $\mathbf{x} \rightarrow t$, while the feature extraction image $\rightarrow \mathbf{x}$ is still hand-crafted. It does not learn how to transform the original image into $\mathbf{x}$.

Which parts of the pipeline are learned?

To summarize, the traditional pipeline consists of two parts:

1. **Hand-crafted feature extraction**:

   $$I \xrightarrow{\text{hand-crafted feature extraction}} \mathbf{x}$$

   This part is designed by a human expert (**hand-crafted**). It is not learned from data. The engineer decides which features to extract based on prior knowledge of the problem.

   For example, a human decides which measurements to compute from the image, how to segment the object, how to calculate area and perimeter, how to measure height, which features to keep, and which to discard. This is **all done manually**.

2. **Classification**:

   $$\mathbf{x} \xrightarrow{\text{classifier}} t$$

   This part is **data-driven**. The classifier learns from labeled examples how to map the feature vector $\mathbf{x}$ to the correct class label t. For a tree-based classifier, it learns the decision rules. For a linear classifier, it learns how to separate the classes in the feature space. For a neural network, it learns the weights and biases that define the mapping from $\mathbf{x}$ to t.

In general, we can separate the pipeline into two stages:

$$\underbrace{\text{image} \to \mathbf{x}}_{\textbf{hand-crafted}} \qquad \underbrace{\mathbf{x} \to t}_{\textbf{data-driven}}$$

And the complete system contains **two kinds of intelligence**:

- The **human expert** contributes knowledge by deciding what to measure.
- The **learning algorithm** contributes adaptability by learning how those measurements correspond to the classes.

So the traditional pipeline can be written as:

$$\text{expert knowledge} \; + \; \text{supervised learning} \; \to \; \text{image classifier}$$

This works well when the useful properties are easy to describe, as in the apple vs orange example. But the upcoming problem is that, for **more complex images**, it becomes extremely difficult to decide manually which features should represent an object. That limitation motivates the **transition from hand-crafted to data-driven** feature extraction.

Why use an intermediate feature vector?

Assume the original image has size $r \times c$. A direct classifier would receive $r \cdot c$ pixel values as input. Instead, the manually designed extractor produces only d features, where $d \ll r \cdot c$.

This can make the task much easier because the classifier works in a compact space containing measurements already believed to be relevant for the task.

For example, separating apples from oranges may be easier in a 3D space of features (average color, roundness, texture) than in the original high-dimensional pixel space.

Conceptually, the feature extractor tries to **transform a difficult problem into an easier** one:

$$\text{complex pixel-space boundary} \to \text{simple feature-space boundary}$$

That is why a simple decision tree may work after good feature extraction, while it may fail in the original pixel space.

Key Takeaways 2

- A hand-crafted feature is a measurement manually designed by an expert.
- The image $I \in \mathbb{R}^{r \times c}$ is converted into a compact vector:

$$\mathbf{x} \in \mathbb{R}^d, \quad d \ll r \cdot c$$

- A decision tree can classify objects using simple thresholds on the features.
- An MLP can also operate on the extracted vector and output class probabilities.
- In the traditional approach:

$$\underbrace{I \to \mathbf{x}}_{\text{hand-crafted}} \qquad \underbrace{\mathbf{x} \to t}_{\text{data-driven}}$$

- The classifier learns from data, but the representation is still chosen by humans. This is a limitation for complex images, motivating the transition to data-driven feature extraction.
- CNNs will change precisely the first part: they will learn the feature extractor as well as the classifier, making the entire pipeline data-driven.

7.1.3 Advantages and Limitations of Hand-Crafted Features

The previous section showed the traditional pipeline:

$$\text{image} \rightarrow \underbrace{\text{hand-crafted features extractor}}_{\text{designed by humans}} \rightarrow \underbrace{\text{classifier}}_{\text{learned from data}} \rightarrow \text{class}$$

Now, we will evaluate this approach. While hand-crafted features can be very effective in the right setting, we will not conclude that they are useless. However, they become difficult to design and maintain as the visual task becomes more complex.

Advantages of hand-crafted features

The advantages of hand-crafted features are:

1. **Exploiting prior and expert knowledge**. The first advantage is the possibility of using a **priori knowledge**. An expert may already know which physical or geometric properties matter for the task.

   In the apple vs orange example, useful quantities may include the *color*, *shape*, and *texture* of the fruit. These are not arbitrary measurements, but rather they are chosen because the expert understands the objects being classified.

   This priori knowledge can make the **classification problem much easier**. Instead of asking the classifier to discover everything from raw pixels, the engineer directly provides variables that are already meaningful for the task.

2. **Interpretability**. Hand-crafted features are usually **easy to understand**.

   Suppose a classifier produces the wrong prediction because it relies heavily on *average color* to distinguish between apples and oranges. The engineer can inspect the feature and ask whether it is reasonable to use this property for the task. This is much easier than inspecting thousands of internal neural-network activations to understand why the model made a mistake.

3. **Features can be manually adjusted**. Because the features are explicit, the engineer can **modify them directly**.

   For example, if the current feature vector is:

   $$\mathbf{x} = [\text{average color},\ \text{average shape},\ \text{average texture}]^T$$

   But the classifier confuses two types of oranges, the engineer can add a new feature to the vector:

   $$\mathbf{x} = [\text{average color},\ \text{average shape},\ \text{average texture},\ \text{average size}]^T$$

Or replace the raw *average color* with a more sophisticated *color histogram* feature. The **system** can therefore be **improved through targeted human intervention**, rather than requiring a complete retraining of the model. This gives the designer considerable control over the representation of the data and the classification process.

4. **Limited training data may be sufficient**. A good hand-crafted feature **extractor already embeds knowledge about the problem**. Therefore, the **classifier** does **not need to learn everything from the training set**.

   Consider the difference between learning from $r \cdot c$ raw pixels values and learning from a compact feature vector of size d:

   $$\mathbf{x} \in \mathbb{R}^d \quad \text{with} \quad d \ll r \cdot c$$

   If the features are well chosen, the classes may already be almost separated in feature space. A simple classifier can then learn the final decision boundary from **relatively few labelled examples**.

   This is particularly useful when collecting data is expensive, labels require expert annotation, or only a small number of examples exists.

   ⚠ This does **not** mean that hand-crafted systems never overfit. It means that carefully chosen prior knowledge can reduce how much must be learned from data.

5. **Giving more relevance to selected properties**. The engineer can explicitly decide that **some features are more important than others**.

   For example, if the engineer knows that *color* is more important than *shape* for distinguishing between apples and oranges, they can assign a higher weight to the color feature in the classifier. This **introduces a deliberate human bias into the system**. That bias can be useful when the expert knowledge is correct.

✖ Limitations of hand-crafted features

The disadvantages explain why this approach becomes problematic for complex image-recognition tasks. The limitations of hand-crafted features are:

1. **High design and programming cost**. A **hand-crafted feature** is not just an idea. It must be **implemented as an algorithm**.

   For example, computing the *average color* of an image is straightforward, but computing a *color histogram* or a *shape descriptor* may require more sophisticated algorithms. The engineer must also ensure that the feature extractor is efficient and robust to variations in the input data.

Therefore, the full pipeline may contain many manually programmed stages, each of which must be carefully designed, implemented, and tested:

$$\begin{aligned}\text{image} &\to \text{preprocessing}\\ &\to \text{segmentation}\\ &\to \text{measurement}\\ &\to \text{normalization}\\ &\to \text{classification}\end{aligned}$$

Each stage can require **significant engineering effort**, and every stage can **fail**.

2. **Risk of overfitting during feature design**. Overfitting is not limited to trained models. The **human designer can also overfit the development dataset**.

   Suppose the engineer repeatedly inspects the same training images and keeps adding rules that fix specific mistakes: a new feature is added to distinguish between two oranges, a new threshold is set to separate apples from oranges, etc. Eventually, the feature extractor may work very well on the images used during development, but poorly on new data.

   This is still overfitting, even if the feature extractor itself was not trained through gradient descent. In general, the problem is:

   features tailored too closely to known examples $\implies$ poor generalization

3. **Poor portability and generality**. A feature **extractor designed for one task often cannot be reused directly for another**.

   The engineer may have to start from scratch when the task changes. For example, a feature extractor designed to distinguish between apples and oranges may not work well for distinguishing between cats and dogs. A change in the acquisition system (e.g., camera, lighting, or viewpoint) may also break the features. So the system is often tightly coupled to the task, the sensor, the environment, and the object geometry. This makes it difficult to adapt the system to new tasks or conditions (**poor portability and generality**).

4. **Natural images are too complex**. For **structured industrial objects**, we can often describe **useful properties explicitly**. But for **natural images**, the question becomes **much harder**: *which features should we manually extract to distinguish a car from a motorbike?* We could try to define features such as *number of wheels*, *object width*, *object height*, *presence of windows*, *body shape*. But every proposed feature creates new problems. For example:

   - *Number of wheels.* A car may show only two visible wheels from the side. A motorbike may show one wheel because the other is occluded.

- *Object width.* A car from the camera may occupy fewer pixels than a nearby motorbike.
- *Shape.* Objects appear differently under changes in viewpoint, lighting, occlusion, and so on.
- *Color.* Cars and motorbikes can have almost any color.

Therefore, a simple list of measurements is unlikely to capture all the visual variability of natural images.

Why humans find the task easy. An interesting question is: *why do humans find it easy to distinguish between a car and a motorbike, while it is difficult to design a feature extractor that can do the same?* This happens because human visual recognition relies on a very rich hierarchy of learned patterns, rather than a small set of hand-crafted features.

We do not consciously classify a vehicle by checking a small fixed list such as: wheel count > 2, width > 1.5m, height < 2m, etc. Instead, we recognize many interacting patterns, such as the *shape of the body*, *the presence of a windshield*, *the shape of the headlights*, *the presence of a license plate*, and so on. These **patterns are difficult to translate into a small set of manually written rules**. That is the central failure of hand-crafted vision for natural images: the relevant representation exists, but humans often cannot describe it precisely enough to program it.

The deeper problem: the feature extractor is fixed

In the traditional pipeline we have:

$$I \xrightarrow{h} \mathbf{x} \xrightarrow{f_\theta} t$$

The **hand-crafted feature extractor** h is manually designed and then **fixed**. Only the classifier f_θ is learned from data. If h discards useful information due to a poor design, the classifier cannot recover it.

For example, suppose the feature extractor keeps only:

$$\mathbf{x} = [\text{average color},\ \text{average shape},\ \text{average texture}]^T$$

If wheel structure is necessary to distinguish cars from motorbikes, but the extractor never represents it, then even a powerful classifier cannot use it.

Formally, the information discarded by the feature extractor h is **unavailable** to every later stage. So the classifier's **performance is limited by the human-designed representation**.

When hand-crafted features are still appropriate

Hand-crafted features are not always a bad choice. They remain attractive when:

- ✔ The domain is narrow and well understood.
- ✔ The relevant measurements are physically meaningful.
- ✔ Interpretability is important.
- ✔ Little training data is available.
- ✔ The environment is controlled.
- ✔ The input structure does not vary much.

By contrast, **data-driven feature learning becomes more attractive** when:

- The images are natural and highly variable.
- It is unclear which measurements are relevant.
- Large labelled datasets are available.
- Portability and generality are important.

This comparison can be summarized as follows:

$$\text{Hand-crafted features} \rightarrow \begin{array}{l} \checkmark \text{ More human knowledge} \\ \checkmark \text{ More control} \\ \checkmark \text{ Less representation learning} \end{array}$$

This provides:

$$\text{interpretability} \; + \; \text{data efficiency}$$

But often at the **cost** of:

$$\text{Hand-crafted features} \rightarrow \begin{array}{l} \times \text{ Engineering effort} \\ \times \text{ Poor scalability} \\ \times \text{ Limited generality} \end{array}$$

The central trade-off is therefore:

$$\text{Human control} \quad \text{vs} \quad \text{Automatic adaptability}$$

Why the next solution is needed. The limitation is not necessarily the classifier. We may already use a powerful neural network after the features. The **limitation** is that the neural network receives a **representation chosen in advance by a human**:

$$\text{image} \rightarrow \underbrace{\text{fixed human-designed features}}_{\text{bottleneck}} \rightarrow \text{neural network}$$

The natural next question is therefore: ***instead of manually deciding which features to extract, can the model learn the feature extractor from data?***

7.1.4 Data-Driven Feature Learning

In the traditional approach, a human explicitly decides which measurements should represent an image:

$$\text{image} \rightarrow \underbrace{\text{hand-crafted feature extractor}}_{\text{designed by humans}} \rightarrow \underbrace{\text{classifier}}_{\text{learned from data}} \rightarrow \text{class}$$

For natural images, however, these manually chosen **measurements are often insufficient**. The alternative is to define a **parameterized feature extractor**:

$$\mathbf{x} = h_\phi(I)$$

Where:

- I is the **input** image.
- h_ϕ is the **feature-extraction function**, parameterized by ϕ.
- ϕ represents its **trainable parameters**.
- $\mathbf{x}$ is the learned **feature vector**.

The **classifier** is another parameterized function f_θ:

$$\hat{\mathbf{y}} = f_\theta(\mathbf{x})$$

Where θ contains the **classifier parameters**. The complete model is therefore:

$$\hat{\mathbf{y}} = f_\theta(h_\phi(I))$$

Now both ϕ (i.e., the parameters of the *feature extractor*) and θ (i.e., the parameters of the *classifier*) are learned from data.

What does "*learning the features*" mean?

It does **not** mean that the model **produces a list of human-readable measurements** such as: *wheel count*, *average height*, *perimeter*, *number of windows*, etc. Instead, the model learns numerical transformations that produce internal activations useful for classification.

The learned feature vector may look like:

$$\mathbf{x} = [0.81,\ -0.24,\ 1.37,\ 0.05,\ \ldots]^T$$

Individual components may not have an obvious semantic label. **What matters** is that, taken together, they make the classes easier to distinguish.

For example, images of cars may map to one region of the learned feature space, while motorbikes map to another:

$$h_\phi(I_{\text{car}}) \approx \text{ car region}, \quad h_\phi(I_{\text{motorbike}}) \approx \text{ motorbike region}$$

The **representation** is therefore **optimized for the task** rather than chosen according to what humans find immediately interpretable.

The role of the training objective

Suppose the training set is:

$$\mathcal{D} = \{(I_n,\ t_n)\}_{n=1}^{N}$$

Where I_n is the n-th **image** and t_n is its corresponding **class label**. The model produces:[18]

$$\hat{\mathbf{y}}_n = f_\theta\left(h_\phi\left(I_n\right)\right)$$

A loss function measures the classification error:[19]

$$\mathcal{L}\left(f_\theta\left(h_\phi\left(I_n\right)\right),\ t_n\right)$$

Remember that the loss function is a measure of how well the model's predictions match the true labels. The **goal of training is to minimize this loss**:[20]

$$\min_{\phi,\theta} \sum_{n=1}^{N} \mathcal{L}\left(f_\theta\left(h_\phi\left(I_n\right)\right),\ t_n\right)$$

The crucial point is that the optimization updates both: the **feature extractor parameters** ϕ and the **classifier parameters** θ. So the classification error influences not only the classifier, but also the feature extractor.

Deepening: How does the classification error influence the feature extractor?

Let's **imagine the feature extractor is fixed**. So we have this pipeline:

$$\text{Image} \rightarrow \underbrace{\text{Feature Extractor}}_{\text{fixed}} \rightarrow \text{Classifier} \rightarrow \text{Prediction}$$

Suppose the feature extractor outputs only two numbers: the *average brightness* and the *average color*. So every image is represented as $x = (\textit{brightness}, \textit{color})$.

Now suppose we want to distinguish between *cars* and *motorbikes*. Immediately we have a problem: cars and motorbikes can easily have the same average brightness and color. Let's say:

Image	Brightness	Color
Car	0.63	blue
Motorbike	0.64	blue

These two feature vectors are almost identical.

[18] The equation means that the feature extractor h_ϕ first transforms the input image I_n into a feature vector, which is then passed to the classifier f_θ to produce the predicted class probabilities $\hat{\mathbf{y}}_n$.

[19] The loss function $\mathcal{L}$ quantifies the difference between the predicted class probabilities $\hat{\mathbf{y}}_n$ and the true class label t_n.

[20] Across all training examples (from $n = 1$ to N), we want to find the parameters ϕ and θ that minimize the total classification error.

Can the classifier fix this? Obviously not. The classifier only sees the feature vector (*brightness*, *color*), for example (0.63, blue). It never sees *wheels*, *windows*, or *doors*, because those were discarded by the feature extractor.
So no matter how powerful the classifier becomes, it **cannot invent information that no longer exists**.

Therefore the error must say... Come back to the classification error. Suppose the network predicts a *car* as a *motorbike*. The loss function will produce a large error. If **only the classifier changes**, it is trying to solve a bad input representation to a good output, which may even be **impossible**.

Instead, the loss says "*maybe the representation itself is bad*". So it **modifies the classifier but also the feature extractor**. Thus, the error signal is **backpropagated** through the classifier to the feature extractor because if the loss increases, the only way to reduce it is to ask "*which quantities influenced this prediction?*" The prediction depended on classifier weights and feature vector values. Then we ask "*where did the feature vector come from?*" It came from the feature extractor. The error naturally propagates backwards because every quantity depends on the previous one.

Finally, the classification error influences the feature extractor because a high loss indicates that the current representation is not sufficiently useful for the classifier. Therefore, during training, both the feature extractor and the classifier are updated so that the feature extractor produces a better representation of the input and the classifier makes better use of that representation.

How the feature extractor receives a learning signal

The classifier f_θ sits after the feature extractor h_ϕ:

$$I \xrightarrow{h_\phi} \mathbf{x} \xrightarrow{f_\theta} \hat{\mathbf{y}}$$

Suppose the **classifier makes a wrong prediction**. Then, the loss gradient is propagated backward through the classifier and then into the feature extractor:

$$\text{loss} \rightarrow \text{classifier parameters} \rightarrow \text{feature-extractor parameters}$$

Through **backpropagation**, the **feature extractor is adjusted** so that future representations are more useful for the final task. Conceptually:

$$\text{bad classification} \implies \text{change classifier} + \text{change extracted features}$$

This is fundamentally **different from the hand-crafted pipeline**, where the feature extractor remains fixed during training.

This joint optimization (i.e., learning both the feature extractor and the classifier) is called **End-to-End Learning**. The word *end-to-end* means that the system is trained from raw input (the image) all the way to the final output (the class label). For image classification, the pipeline is:

$$\text{pixels} \rightarrow \text{learned features} \rightarrow \text{class probabilities}$$

Which is optimized as one connected model. There is no separate stage where a human first produces a fixed vector and then trains a classifier (as in the hand-crafted approach). Instead, the **entire transformation is learned for the task at hand**.

Don't humans make any decisions? Data-driven feature learning does **not mean that humans make no design choices**! Humans still choose the architecture, the number and type of layers, the loss function, the training procedure, optimization hyperparameters, the dataset and labels, and so on. However, **what changes is that humans no longer explicitly define each visual feature**. Instead of programming the feature extractor, we define a trainable architecture and let the data determine its parameters.

Thanks to end-to-end learning, *task-dependent features* are possible. A learned feature extractor is optimized for a particular objective. Suppose two networks receive the same images but are trained for different tasks:

- Network A: classify vehicle type (car, motorbike, truck, etc.)
- Network B: predict vehicle color (red, blue, green, etc.)

They may learn different representations. For vehicle type, shape and parts may be especially useful. For color prediction, chromatic information becomes more important. So learned features are usually **task-dependent**:

$$\text{training objective} \implies \text{representation features}$$

This is one reason end-to-end learning is so powerful: the **representation is adapted directly to the desired output**.

Is a data-driven approach perfect?

First of all, the feature extract can be interpreted as a transformation between spaces:

$$h_\phi : \mathcal{I} \rightarrow \mathbb{R}^d$$

Where $\mathcal{I}$ is the space of all possible images and $\mathbb{R}^d$ is the d-dimensional feature space. Raw image space is often **difficult for classification** because images of the same class may look very different at the pixel level (as discussed larger in previous sections). The learned transformation h_ϕ aims to reorganize them so that:

$$\text{same-class images} \rightarrow \text{similar representations}$$

And:

$$\text{different-class images} \rightarrow \text{separable representations}$$

The classifier then operates on this new feature space. A **major goal of the feature extractor** is therefore **make the classification problem easier**.

⚠ Now, can we define data-driven feature learning as a perfect solution? Unfortunately, **no**. Data-driven features **solve many limitations** of hand-crafted systems, but they **also change the requirements**. Hand-crafted features could exploit expert knowledge and work with limited data. Instead, a learned feature extractor usually **needs enough examples to discover useful regularities**. So the transition is approximately:

$$\text{more manual engineering} \xrightarrow{\text{data-driven}} \text{more learning from data}$$

Deep learning reduces the need to manually specify visual rules, but often requires:

- ❗ More training data.
- ❗ More computation.
- ❗ More trainable parameters.
- ❗ More careful optimization.

However, the **benefits of learning features from data** often outweigh these costs, especially for complex tasks like image classification.

Now, if our goal is to learn the feature extractor from images (i.e., to learn the transformation h_ϕ), the next question is: ***what type of trainable operation should we use to process images?*** A standard dense network could receive flattened pixels, but it would ignore the local spatial structure of images and require many parameters. Instead, **CNNs provide a more appropriate feature-extraction architecture** using local filters, shared parameters, multiple feature maps, hierarchical processing. In the next sections, we will explore how CNNs are designed to learn features from images in a data-driven way.

Key Takeaways 3

- Data-driven feature learning makes the feature extractor trainable.
- The image is transformed through a learned feature extractor into a feature vector:
$$\mathbf{x} = h_\phi(I)$$
- The classifier uses this feature vector to produce class probabilities:
$$\hat{\mathbf{y}} = f_\theta(\mathbf{x})$$
- Training optimizes both sets of parameters ϕ and θ to minimize the classification loss:
$$\min_{\phi,\theta} \sum_{n=1}^{N} \mathcal{L}\left(f_\theta\left(h_\phi\left(I_n\right)\right),\ t_n\right)$$

- Classification errors are backpropagated through the classifier into the feature extractor.
- End-to-end learning means training the complete mapping:

$$\text{raw image} \rightarrow \text{prediction}$$

- Learned features need not be human-readable; they need to be useful for the task.
- CNNs are the architecture used to learn features from images in a data-driven way.

7.2 From Image Filtering to CNNs

Our goal is to understand **how to train a feature extractor using data**. In this section, we will introduce the basic operation that convolutional neural networks (CNNs) use to process images: **local filtering**.

7.2.1 Correlation as a Local Linear Operation

In this section, we will revisit the concept introduced on page 415. First of all, we define the **correlation** operation as a **local linear operation** that computes a weighted sum of the neighbouring pixels:

$$T[I](r,c) = \sum_{u,v \in U} w_{u,v} \cdot I(r+u, c+v)$$

This formula computes one output value at spatial location (r,c) by taking a weighted sum of the neighbouring pixels in the neighbourhood U. To understand it clearly, we need to distinguish three different coordinate systems:

- (r,c): the **position** where we are computing one **output**.
- (u,v): an **offset inside the local neighbourhood** U.
- $(r+u, c+v)$: the corresponding **position in the input image** I.

Let the input grayscale image be:

$$I \in \mathbb{R}^{H \times W}$$

The filtering operation produces another image:

$$T[I] \in \mathbb{R}^{H' \times W'}$$

For each valid output position (r,c), correlation examines a small region of the input image around that position and converts that entire region into one scalar output value (see Figure 53, page 492).

The set U specifies which nearby pixels participate in the computation, and for this reason it is called the **local neighbourhood**. For a 3×3 filter centred on the current pixel, one possible definition is:

$$U = \{-1, 0, 1\} \times \{-1, 0, 1\}$$

Explicitly, this means that the local neighbourhood U contains the following offsets:

$$U = \{(-1,-1), (-1,0), (-1,1), (0,-1), (0,0), (0,1), (1,-1), (1,0), (1,1)\}$$

These are not absolute image coordinates. They are **relative offsets** from the current pixel (r,c), which is the centre of the neighbourhood. Therefore, when computing $T[I](r,c)$, the operation uses the pixels:

$$I(r+u, c+v), \quad \forall (u,v) \in U$$

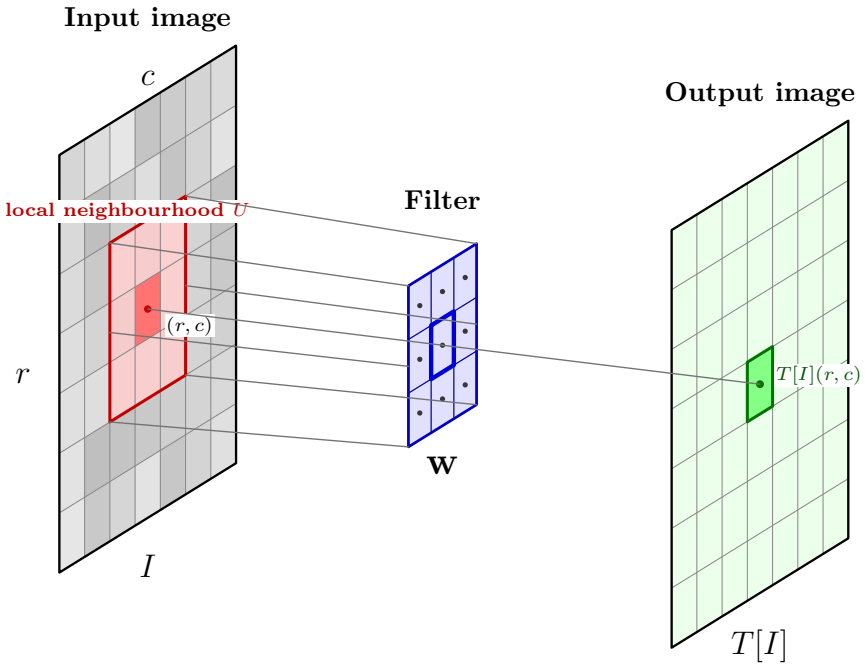


Figure 53: Correlation as a local linear operation. The filter $\mathbf{w}$ is applied to the neighbourhood U centred at (r, c), producing the corresponding output value $T[I](r, c)$.

Why is correlation local? The operation is called **local** because one output value depends only on a small neighbourhood, not on the complete image. This locality is well suited to images because nearby pixels are often strongly related:

- An edge is formed by neighbouring pixels;
- A corner is a local arrangement of edges;
- Texture is characterized by repeated local intensity variations.

This directly recalls the principle of the previous section: meaningful visual structures can often be detected by examining small spatial neighbourhoods.

What is applied to the neighbourhood? Each offset $(u, v) \in U$ has an associated weight $w(u, v)$. The complete collection of weights is called the **filter**:

$$\mathbf{w} = \{w(u, v) \,:\, (u, v) \in U\}$$

For a classic 3×3 neighbourhood, the filter can be written as a matrix:

$$\mathbf{w} = \begin{bmatrix} w(-1,-1) & w(-1,0) & w(-1,1) \\ w(0,-1) & w(0,0) & w(0,1) \\ w(1,-1) & w(1,0) & w(1,1) \end{bmatrix}$$

These weights determine **how much each nearby pixel contributes to the output value**. For example:

- A large positive weight increases the output when the corresponding pixel is bright;
- A negative weight decreases the output when that pixel is bright;
- A zero weight ignores that position.

In summary, the filter **w** entirely defines the operation. Once U and the weights are fixed, the behavior of the filter is fixed.

Example 3Correlation as a Local Linear Operation

Consider the input patch (i.e., the local neighbourhood U):

$$\begin{pmatrix} 2 & 4 & 1 \\ 3 & 5 & 7 \\ 0 & 6 & 8 \end{pmatrix}$$

And the filter:

$$\mathbf{w} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

At the corresponding output location, we multiply values element by element and sum:

$$T[I](r,c) = 0\cdot 2 + 1\cdot 4 + 0\cdot 1 + 1\cdot 3 + (-4)\cdot 5 + 1\cdot 7 + 0\cdot 0 + 1\cdot 6 + 0\cdot 8 = 0$$

That single number becomes one pixel of the output image $T[I]$ at the same spatial location (r,c). The same operation is then repeated at other spatial locations to produce the complete output image.

How to apply the filter to the entire image? The filter slides across the input image. In this way, correlation applies the same filter at every valid position.

Mathematically, at one location (r,c), the output value is computed as:

$$T[I](r,c) = \sum_{u,v\in U} w_{u,v} \cdot I(r+u, c+v)$$

At the next horizontal position $(r, c+1)$, the output value is computed as:

$$T[I](r,c+1) = \sum_{u,v\in U} w_{u,v} \cdot I(r+u, c+1+v)$$

The filter weights do not change, only the input patch changes. So the procedure is:

1. Place the filter over a local image region;
2. Multiply corresponding entries;

3. Sum the product;
4. Store the result in the output;
5. Move the filter to the next location.

This repeated use of the same weights across the image will later become one of the defining properties of convolutional layers.

Why correlation is called a linear operation? Correlation is linear with respect to the input image I. Formally, let I_1 and I_2 be two images, and let α and β be two scalars $\alpha, \beta \in \mathbb{R}$. Then:

$$T\left[\alpha I_1 + \beta I_2\right](r,c) = \sum_{u,v \in U} w_{u,v} \cdot \left[\alpha I_1\left(r+u, c+v\right) + \beta I_2\left(r+u, c+v\right)\right]$$

Distributing the sum gives:

$$T\left[\alpha I_1 + \beta I_2\right](r,c) = \alpha T\left[I_1\right](r,c) + \beta T\left[I_2\right](r,c)$$

Therefore, for all valid output positions (r, c), we have:

$$T\left[\alpha I_1 + \beta I_2\right] = \alpha T\left[I_1\right] + \beta T\left[I_2\right] \tag{169}$$

Thus, correlation is a linear operation because it satisfies the property of linearity.[21] At every location, the output is simply a linear combination of input values, with the filter weights as coefficients.

Properties of Correlation

We have the transformation:

$$T : I \mapsto T[I]$$

The operation T, called **correlation**, has **two independent properties**:

1. **Locality**: ***which input pixels influence one output pixel?*** For correlation:

   $$T[I](r,c)$$

   Depends only on the pixels in the local neighbourhood U around (r, c):

   $$\{I(r+u, c+v) : (u,v) \in U\}$$

   So locality is about **the set of input that are used** to compute one output value. If U is 3×3, then only those 9 pixels matter. Everything else in the image is ignored. So locality is a property of **how the operator accesses the input**.

[21] In general, a linear operation is a function T that satisfies the following two properties:

1. **Additivity:** $T\left[I_1 + I_2\right] = T\left[I_1\right] + T\left[I_2\right]$;
2. **Homogeneity:** $T\left[\alpha I\right] = \alpha T\left[I\right]$

2. **Linearity**: ***once those pixels have been selected, how are they combined?*** Correlation combines them by:

$$\sum_{u,v \in U} w_{u,v} \cdot I(r+u, c+v)$$

Which is simply a weighted sum. Weighted sums satisfy the property of linearity:

$$T[\alpha I_1 + \beta I_2] = \alpha T[I_1] + \beta T[I_2]$$

Therefore the operator is linear. So linearity is a property of **how the selected pixels are mathematically combined** to produce the output value.

√× Matrix Representation

We do not introduce any flattening techniques or algorithm changes here. We simply demonstrate that converting the local image patch and the filter itself into vectors by flattening them allows us to **express correlation as a simple dot product**. While this notation is not useful for practical implementations now, it will be perfect when we introduce CNNs.

Suppose the local patch U is 3×3. We change the notation and we write $P_{r,c}$ for the local patch of the input image I centred at (r, c):

$$P_{r,c} = \begin{pmatrix} p_1 & p_2 & p_3 \\ p_4 & p_5 & p_6 \\ p_7 & p_8 & p_9 \end{pmatrix}$$

And the filter is:

$$W = \begin{pmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{pmatrix}$$

The correlation result at that position is:

$$w_1 p_1 + w_2 p_2 + \cdots + w_9 p_9$$

Then, we can rewrite the same computation as a dot product of two vectors:

$$\mathbf{p}_{r,c} = \begin{pmatrix} p_1 \\ p_2 \\ \vdots \\ p_9 \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_9 \end{pmatrix}$$

$$T[I](r,c) = \mathbf{w}^T \mathbf{p}_{r,c}$$

Nothing changes mathematically. Thanks to flattening, we have simply expressed the same operation in a different way. This will be useful when we introduce CNNs because it allows us to express the correlation operation as simple matrix multiplication.

Relationship with a Dense Neuron

There is an interesting relationship between correlation and a dense neuron.

What does a dense neuron actually do? Forget images for a moment, and suppose a neuron receives three numbers:

$$x_1, x_2, x_3 \in \mathbb{R}$$

It has three weights:

$$w_1, w_2, w_3 \in \mathbb{R}$$

The neuron computes a weighted sum of its inputs:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

That's all a dense neuron does before the activation function. It computes a weighted sum.

What does correlation do? Now look at correlation. Suppose the image patch is:

$$\begin{pmatrix} 1 & 4 \\ 2 & 3 \end{pmatrix}$$

And the filter is:

$$\begin{pmatrix} 2 & 1 \\ 0 & -1 \end{pmatrix}$$

Correlation computes a weighted sum of the input patch:

$$2 \cdot 1 + 1 \cdot 4 + 0 \cdot 2 + (-1) \cdot 3 = 3$$

Again, correlation is just a weighted sum of its inputs.

Using the same notation as before, we can see an interesting pattern:

- Dense neuron:
$$w_1x_1 + w_2x_2 + w_3x_3 + \dots$$
- Correlation:
$$w_1p_1 + w_2p_2 + w_3p_3 + \dots$$

The only thing that changes is **what the input represent**.

- For a dense neuron, the inputs are generic features x_i;
- For correlation, the inputs are pixels from a local image patch p_i.

But the computation itself is identical: **weighted sum**. Once we introduce CNNs, we will see that this same computation becomes the core operation of Convolutional Neural Networks (CNNs).

Key Takeaways 4

- Correlation computes one output value from a local image neighbourhood.
- The neighbourhood U contains relative spatial offsets.
- The filter is the set of weights:

$$\mathbf{w} = \{w(u, v) : (u, v) \in U\}$$

- One output value is a weighted sum of nearby pixels:

$$T[I](r, c) = \sum_{u,v \in U} w(u, v) \cdot I(r + u, c + v)$$

- The operation is **local** because it uses only a small patch.
- It is **linear** because it computes a linear combination of the input values.
- The same filter is applied across spatial positions, producing a complete output image.
- Different filters produce different image transformations or responses to different visual structures.

7.2.2 Examples of Linear Image Filters

Now we make the abstract formula from the previous section concrete. Recall:

$$T[I](r,c) = \sum_{(u,v)\in U} w(u,v)I(r+u,c+v)$$

The operation is always the same:

$$\text{local patch} \rightarrow \text{weighted sum} \rightarrow \text{one output pixel}$$

What changes is only the filter $\mathbf{w}$. So different filter weights produce different image transformations.

Identity filter

The **Identity Filter** is the simplest possible filter. It has a single non-zero weight in the center of the filter, and all other weights are zero. The effect of this filter is to leave the image unchanged.

$$\mathbf{w}_{\text{id}} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \tag{170}$$

At spatial position (r,c), the output pixel is simply the input pixel at that position:

$$T[I](r,c) = 0 \cdot I(r-1,c-1) + \cdots + 1 \cdot I(r,c) + \cdots + 0 \cdot I(r+1,c+1) = I(r,c)$$

So the output image is identical to the input image: $T[I] = I$.

Averaging filter

The **Averaging Filter** is a simple filter that computes the **average of the pixel values in a local neighborhood**. It has equal weights for all pixels in the filter, and the sum of the weights is 1. The effect of this filter is to **smooth the image**, reducing noise and detail.

$$\mathbf{w}_{\text{avg}} = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \tag{171}$$

At every position, the output pixel is:

$$T[I](r,c) = \frac{1}{9}\sum_{u=-1}^{1}\sum_{v=-1}^{1} I(r+u,c+v)$$

This is simply the average of the 9 pixels in the 3×3 neighborhood centered at (r,c). For example, suppose the local patch of the image is:

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & 100 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

The central pixel is much brighter than its neighbors. The filtered value becomes:

$$T[I](r,c) = \frac{1}{9}(10+10+10+10+100+10+10+10+10) = \frac{180}{9} = 20$$

So the central value changes from 100 to 20, which is much closer to the values of its neighbors.

For example, consider the following Lena image. The left image is the original, and the right image is the result of applying the 3×3 averaging filter to every pixel in the original image. The filtered image appears slightly softer because each pixel is replaced by an average of itself and its neighbours. Fine variations are reduced, while slowly varying regions remain similar.



(a) Original image (b) After a 3×3 averaging filter

Figure 54: Effect of a 3×3 averaging filter. Each output pixel is the average of its 3×3 neighbourhood, producing a smoother image and reducing fine details.

What happens at an edge? Suppose we look at the pixel exactly on the white side. Its neighbourhood is:

$$\begin{bmatrix} 0 & 0 & 255 \\ 0 & 255 & 255 \\ 0 & 255 & 255 \end{bmatrix}$$

Notice that some neighbours are black, while others are white. If we apply the averaging filter, the filter computes the average of these 9 values:

$$T[I](r,c) = \frac{1}{9}(0+0+255+0+255+255+0+255+255) = \frac{1530}{9} = 170$$

So the central pixel value changes from 255 to 170, which is a gray value. We do not get a sudden jump from black to white, but rather a **smooth transition**.

What happens to noise? Imagine a completely gray wall. Every pixel should be 100. But one sensor measurement is wrong and the center is much

brighter than it should be:

$$\begin{bmatrix} 100 & 100 & 100 \\ 100 & 180 & 100 \\ 100 & 100 & 100 \end{bmatrix}$$

Applying the averaging filter, the central pixel value becomes:

$$T[I](r,c) = \frac{1}{9}(100{+}100{+}100{+}100{+}180{+}100{+}100{+}100{+}100) = \frac{980}{9} = 108.9 \approx 109$$

Instead of being 180, the central pixel is now 109, which is much closer to the expected value of 100. The averaging filter has **reduced the effect of noise**.

In summary, every averaging filter performs the same operation: average nearby pixels. Therefore:

- If neighbouring differences are caused by **noise**, averaging is beneficial;
- If neighbouring differences correspond to **real image details** (edges, textures, thin structures), averaging destroys part of that information.

What happens if we use a larger averaging filter? A larger averaging filter will include more pixels in the average, which will **increase the smoothing effect**. For example, a 5×5 averaging filter will average over 25 pixels instead of 9:

$$\mathbf{w}_{\text{avg},5} = \frac{1}{25}\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Now each output value is the average of 25 nearby pixels:

$$T[I](r,c) = \frac{1}{25}\sum_{u=-2}^{2}\sum_{v=-2}^{2} I(r+u, c+v)$$

In general, for an $m \times m$ averaging filter, the normalized coefficients are:

$$w(u,v) = \frac{1}{m^2}$$

Thus, the output pixel is:

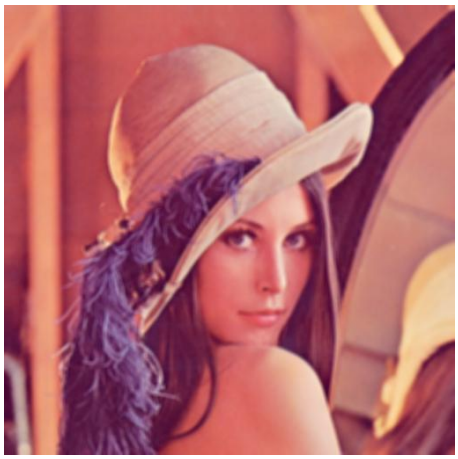
$$\mathbf{w}_{\text{avg},m} = \frac{1}{m^2}\begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & 1 & \cdots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & 1 \end{bmatrix}$$

So increasing the kernel size m **increases the smoothing effect** without systematically changing constant-region brightness (i.e., the average of a constant region remains the same). However, it also increases the **risk of losing important image details** because the filter averages over a larger area, which can blur edges and textures.

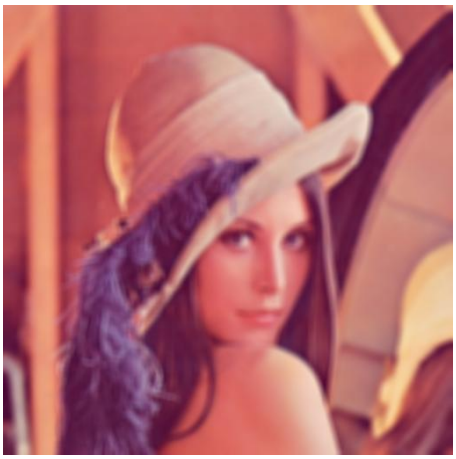


(a) Original image

(b) After a 3×3 averaging filter



(c) After a 5×5 averaging filter



(d) After a 9×9 averaging filter

Figure 55: Effect of increasing the averaging filter size. As the filter size increases, the image becomes progressively smoother and more blurred, losing fine details and textures.

Motion Blur filter

The **Motion Blur Filter** is a filter that simulates the effect of motion blur in an image. It has non-zero weights along a diagonal line, and all other weights are zero. The effect of this filter is to **blur the image in a specific direction**, simulating the effect of camera motion during exposure.

$$\mathbf{w}_{\text{motion}} = \frac{1}{L} \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{bmatrix} \tag{172}$$

Where:

- L is the **length of the motion blur** (number of pixels to average over, called also blur length);
- The **main diagonal** determines the **direction** of the blur (e.g., top-left to bottom-right, or top-right to bottom-left);
- The factor $\frac{1}{L}$ normalizes the kernel so that its coefficients sum to 1, preserving the average brightness.

The diagonal filter averages the pixel values along the diagonal direction, thus the details are spread along that diagonal, structures become blurred preferentially in that direction, and the effect is directional rather than isotropic like the averaging filter.

Mathematically, we can express the motion blur as:

$$I_{\text{blurred}} = I_{\text{sharp}} * \mathbf{w}_{\text{motion}}$$

Where I_{sharp} is the original image, I_{blurred} is the resulting blurred image, and $*$ denotes correlation (or convolution) with the motion blur kernel $\mathbf{w}_{\text{motion}}$.



(a) Original Image

(b) After a 9×9 motion blur filter at 45 degrees (main diagonal)

(c) After a 9×9 motion blur filter at 90 degrees (vertical)

(d) After a 9×9 motion blur filter at 135 degrees (anti-diagonal)

Figure 56: Effect of a 9×9 motion blur filter. The image appears blurred along the diagonal direction, simulating the effect of camera motion during exposure.



(a) Original Image

(b) After a 35×35 motion blur filter at 45 degrees (main diagonal)

(c) After a 35×35 motion blur filter at 90 degrees (vertical)

(d) After a 35×35 motion blur filter at 135 degrees (anti-diagonal)

Figure 57: Effect of a 35×35 motion blur filter. Compared to the 9×9 filter, the image appears more blurred along the diagonal direction, simulating a longer camera motion during exposure.

7.3 The Typical CNN Architecture

The previous section introduced the basic computational idea: apply local filters to an image. Now we will show how these operations are organized into a complete CNN.

The central change is that the network no longer processes a single filtered image. Instead, it **repeatedly transforms the input** into a **sequence of multi-channel volumes** until it eventually obtains a vector suitable for a traditional classifier.

First of all, the **input** of a CNN is an entire image. This distinguishes CNNs from the hand-crafted pipeline discussed earlier:

$$\text{Image} \rightarrow \text{Manually Extracted Vector of Features} \rightarrow \text{Classifier}$$

Now, the CNN takes the image as input and automatically learns the features:

$$\text{Image} \rightarrow \text{CNN}$$

The **network** itself is **responsible for transforming the image into useful features**.

For a color image, the input is naturally represented as a **3D vector**:

$$\mathbf{x} \in \mathbb{R}^{H \times W \times C}$$

Where H is the image height, W is the image width, and C is the number of channels (3 for RGB images, 1 for grayscale images).

Usually, the input image of a CNN is called **input volume**. An ordinary image is often drawn as a two-dimensional rectangle. However, a color image contains three values at every spatial position:

$$x(r, c, 1), \quad x(r, c, 2), \quad x(r, c, 3)$$

These correspond to the red, green, and blue channels of the image. Therefore, the image has three dimensions:

$$\text{Height} \times \text{Width} \times \text{Channels}$$

It can be visualized as a stack of C two-dimensional maps:

$$\mathbf{x} = \begin{bmatrix} \text{channel } 1 \\ \text{channel } 2 \\ \vdots \\ \text{channel } C \end{bmatrix}$$

This stack of channels is called a **volume**. The depth here means number of channels, not physical depth in the sense of 3D space.

Example 4: Input Volume

Suppose the image is only 2×2 pixels. Its three channels might be:

$$R = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}, \quad G = \begin{bmatrix} 50 & 60 \\ 70 & 80 \end{bmatrix}, \quad B = \begin{bmatrix} 90 & 100 \\ 110 & 120 \end{bmatrix}$$

Each channel is still a 2×2 matrix. Stacking them produces one three-dimensional array:

$$I \in \mathbb{R}^{2\times 2\times 3}$$

Defined by:

$$I(:,:,1) = R, \quad I(:,:,2) = G, \quad I(:,:,3) = B$$

Conceptually, the input volume can be visualized as a stack of three 2×2 matrices, one for each channel:

$$I = \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

The brackets do **not** mean ordinary vertical matrix concatenation. They mean that the matrices occupy different slices along a new dimension. Thus, at spatial position $(1, 2)$, the corresponding values are:

$$R(1,2) = 20, \quad G(1,2) = 60, \quad B(1,2) = 100$$

Therefore:

$$I(1,2,:) = \begin{bmatrix} 20 \\ 60 \\ 100 \end{bmatrix}$$

So every spatial position contains an RGB triplet:

$$I(r,c,:) = \begin{bmatrix} R(r,c) \\ G(r,c) \\ B(r,c) \end{bmatrix}$$

The structure is therefore:

Spatial Position	**Values along depth**
$(1,1)$	[10, 50, 90]
$(1,2)$	[20, 60, 100]
$(2,1)$	[30, 70, 110]
$(2,2)$	[40, 80, 120]

The rows and columns remain aligned across all three channels.

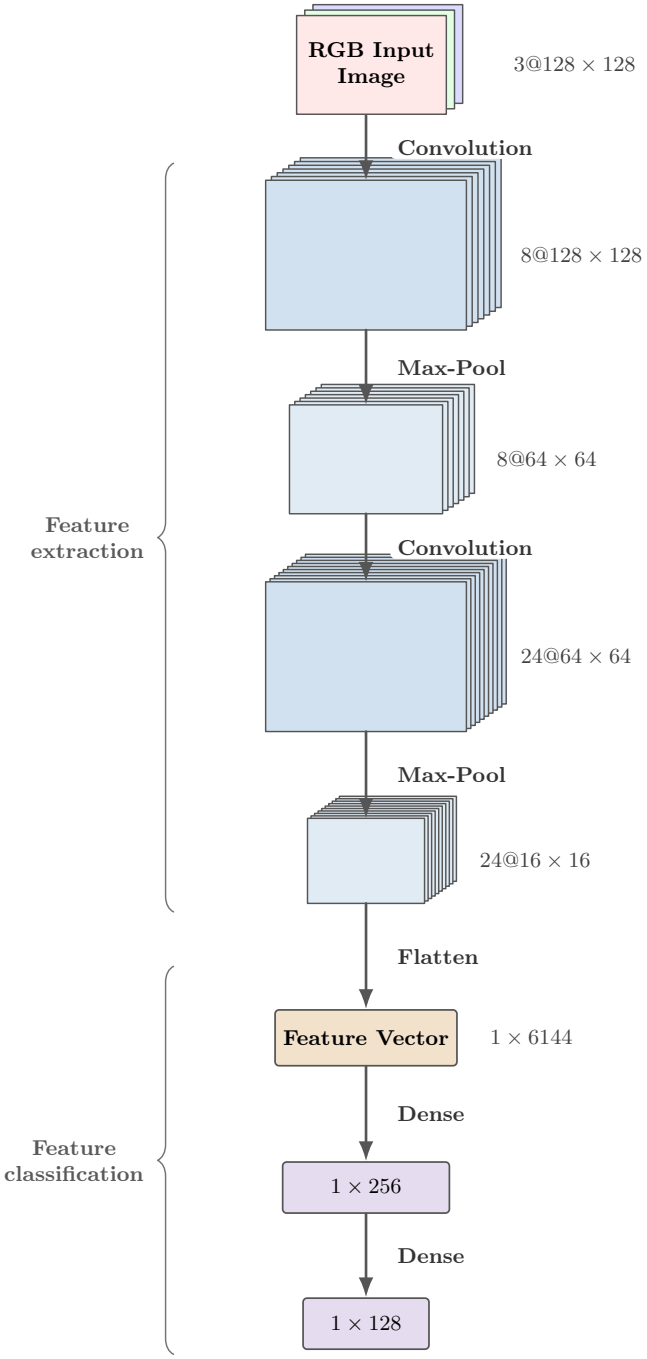


Figure 58: A typical CNN architecture. The input is a color image (3 channels) of size 128×128. The network applies two convolutional layers, each followed by a max-pooling layer. The output of the second max-pooling layer is flattened into a feature vector, which is then processed by two dense layers to produce the final classification output.

The notation used in the architecture diagram. In Figure 58, page 507, we represent a typical CNN architecture. The intermediate representations use the notation:

$$\text{Number of Channels@Height} \times \text{Width} \quad \text{or} \quad C@H \times W$$

For example, the input image is represented as 3@128×128, indicating 3 channels (RGB) with a height and width of 128 pixels each.

Understanding A Typical CNN Architecture. The figure illustrates a typical CNN architecture. We do not pretend to explain all the details of CNNs here, but we will highlight the main components.

- The numbers on the right of each layer are simply an example of the dimensions of the output volume at that layer.
- The **input** is a color image of size 128×128 pixels, represented as a volume with 3 channels (RGB).
- The first layer is a **convolutional layer**. The image is convolved against **many filters**.

  Suppose a convolutional layer contains N_F filters:

  $$\mathbf{w}^1, \mathbf{w}^2, \ldots, \mathbf{w}^{N_F}$$

  Each filter produces one two-dimensional output map:

  $$a(:;:;1), \quad a(:;:;2), \quad \ldots, \quad a(:;:;N_F)$$

  These maps are stacked together to form the output volume of the convolutional layer:

  $$a \in \mathbb{R}^{H' \times W' \times N_F}$$

  Therefore, the **number of output channels** of a convolutional layer is **equal** to the **number of filters** in that layer. This is the reason the architecture drawing shows several parallel gray sheets after every convolutional layer. They are not several copies of the same image; they are different responses produced by different learned filters.

  In the Figure 58, the first convolutional layer applies 8 learned filters to the input volume. The output volume has 8 channels, and the height and width remain 128 × 128 due to padding. Thus, the output of this layer is represented as 8@128 × 128.

  From an image to feature maps. At the beginning, the channels have an obvious meaning (RGB). However, after the first convolution, the channels are no longer RGB channels. They become **learned feature maps**:

  $$a(:;:;1), \quad a(:;:;2), \quad \ldots, \quad a(:;:;8)$$

  Each map represents the spatial response of one learned filter. Conceptually, one map may respond strongly to one type of local structure, while another responds to a different structure.

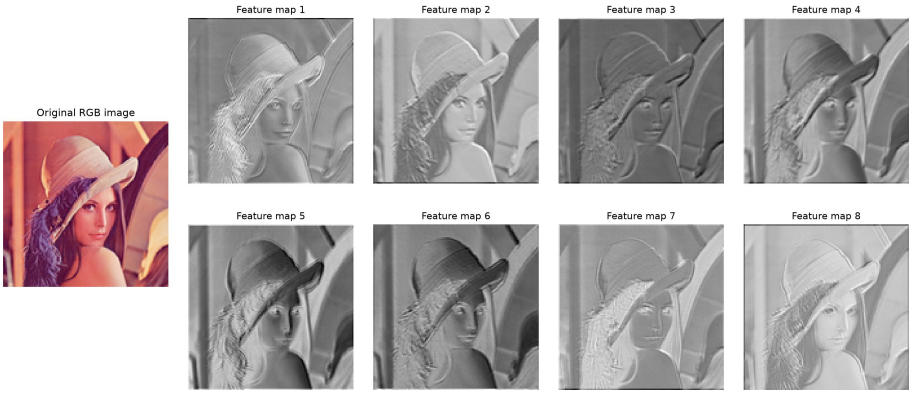


Figure 59: Output of the first convolutional layer. Each of the 8 learned filters produces one feature map containing both positive and negative responses to local image patterns.

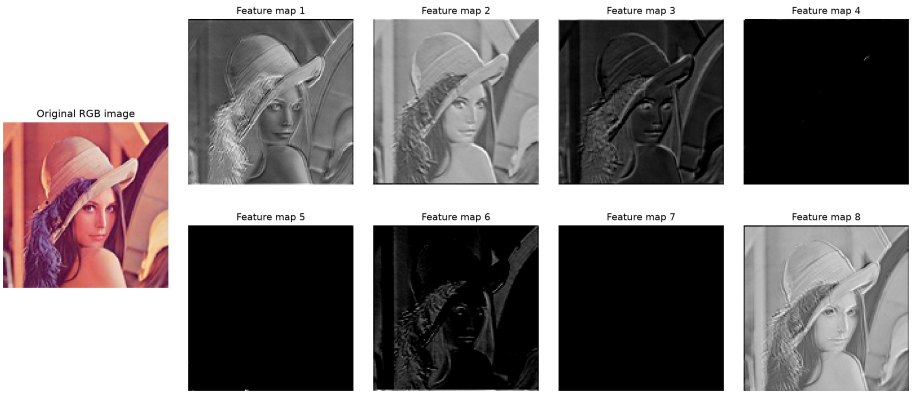


Figure 60: Output of the first convolutional layer after applying the ReLU activation function. Negative responses are set to zero, producing sparser feature maps that are passed to the subsequent layers of the CNN. The reason we apply ReLU will be explained in the following sections.

- The second layer is a **max-pooling layer**. We do not talk about the details of pooling yet, but we can observe that the output volume has the same number of channels as the input (8), but the height and width are halved (64×64). Thus, pooling reduces the spatial resolution (height and width). The output of this layer is represented as $8@64 \times 64$.
- The third layer is another **convolutional layer**. It applies 24 learned filters to the input volume. The number of filters has no theoretical reason to be 24; it is simply an example. The output volume has the same height and width as the input (64×64) because we use padding. Thus, the output of this layer is represented as $24@64 \times 64$.
- The fourth layer is another **max-pooling layer**. The output volume has the same number of channels as the input (24), but the height and width are halved again (16×16). Thus, the output of this layer is represented as $24@16 \times 16$.
- The fifth layer is a **flattening layer**. It takes the output of the previous layer, which is a volume of size $24@16 \times 16$, and flattens it into a one-dimensional feature vector of size 1×6144:

  $$24 \times 16 \times 16 = 6144$$

  This is the point where the CNN transitions from spatial processing to ordinary vector processing.
- The sixth layer is a **dense layer**. It takes the feature vector of size 1×6144 and produces a new feature vector of size 1×256. This layer is **fully connected**, meaning that each of the 256 output features is computed as a weighted sum of all 6144 input features, followed by a non-linear activation function. We will discuss dense layers in detail later.

Note that these numbers are just an example, but they are **hyperparameters** that can be tuned. The network designer can choose the number of filters, the size of the filters, the number of pooling layers, and the size of the dense layers. The **goal** is to **find a good architecture** that balances model capacity and computational efficiency.

Why does depth increase while width decreases? Notice the pattern:

$$3@128 \times 128 \rightarrow 8@128 \times 128 \rightarrow 8@64 \times 64 \rightarrow 24@64 \times 64 \rightarrow 24@16 \times 16$$

As we go deeper into the network, the number of channels (depth) increases, while the spatial dimensions (height and width) decrease. Two trends are at play here:

- ↓ **Spatial dimensions decrease**: The image becomes smaller in height and width due to pooling layers, which reduce the **spatial resolution** (i.e., how many pixels are used to represent the image). So the network is compressing **where** the information is located in the image.
- ↑ **Channel depth increases**: The number of channels increases because the network learns more and more different kind of features. Each channel

becomes a learned feature detector that responds to specific patterns in the input image. In general, each channel answers a different learned question about the input image.

A typical architecture therefore follows the pattern:

$$H_1 \times W_1 \times C_1 \to H_2 \times W_2 \times C_2 \to H_3 \times W_3 \times C_3 \to \dots$$

Where usually:

$$H_{l+1} \leq H_l, \quad W_{l+1} \leq W_l$$

The height and width decrease, while the number of channels increases:

$$C_{l+1} \geq C_l$$

In other words, smaller spatial maps, but more feature channels.

Key Takeaways 5

- A CNN receives the complete image directly.
- An input image is represented as a volume:
$$H \times W \times C$$
- H and W are spatial dimensions; C is the number of channels (depth).
- A convolutional layer applies many learned filters to the input volume.
- Each filter produces one output channel (feature map).
$$\text{output depth} = \text{number of filters}$$
- Intermediate channels represent learned features, not necessarily RGB information.
- A common CNN pattern is: spatial dimensions decrease while channel depth increases.
- Pooling typically reduces height and width without changing channel count.
- Convolution can change channel depth by changing the number of filters.
- The final volume is converted into a vector, usually through flattening:
$$H \times W \times C \to 1 \times (H \cdot W \cdot C)$$
- The resulting vector is fed to a traditional neural network classifier (dense layers).
- The complete image transformation is: (1) image to (2) sequence of feature volumes to (3) feature vector to (4) classification.

7.4 Convolutional Layers

7.4.1 Convolution over Multi-Channel Inputs

Up to now, we have always talked about filtering an image with a kernel. Now we explain what happens when the input is no longer a single image, but a volume (e.g. an RGB image or the activations coming from a previous CNN layer).

From a 2D image to a 3D volume

Previously we considered grayscale images:

$$I \in \mathbb{R}^{H \times W}$$

Where each spatial location contains a single value (the pixel intensity). However, CNNs almost never process single-channel images, but rather multi-channel images, such as RGB images:

$$I \in \mathbb{R}^{H \times W \times C}$$

Where C is the number of channels (e.g. 3 for RGB images). Furthermore, the output of a convolutional layer is also a volume:

$$O \in \mathbb{R}^{H' \times W' \times K}$$

Where K is the number of filters (or kernels) applied to the input volume, for example, $K = 64$ in the first convolutional layer of a CNN. The input is therefore a **volume**, not simply an image.

Suppose the input is a volume of size $32 \times 32 \times 3$ (a 32×32 RGB image) and we want to apply a 3×3 filter to it. *Should we use one filter for Red, one for Green, and one for Blue?* **No**, and this is exactly what distinguishes CNN convolutions from traditional RGB image filtering.

A CNN filter spans the whole depth of the input volume

A convolutional filter is **not** a simple 2D kernel 3×3. It is a 3D kernel $3 \times 3 \times 3$ for an RGB image. The filter is therefore a **small box**, not a flat square, because it spans the entire depth of the input volume.

Imagine looking at one pixel. Instead of looking at a single value (the pixel intensity, e.g., pixel = 120), we have an object with three values (the RGB values, e.g., pixel: $R = 120$, $G = 35$, $B = 90$). To compute one output value, the filter must decide how much importance to assign to Red, Green, and Blue simultaneously. If it ignored two channels, much information would be lost.

What happens at one spatial position? Suppose we are centred at (r, c). The filter extracts:

- 3×3 patch from the Red channel: $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$

- 3×3 patch from the Green channel: $\begin{pmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{pmatrix}$
- 3×3 patch from the Blue channel: $\begin{pmatrix} 5 & 5 & 5 \\ 5 & 5 & 5 \\ 5 & 5 & 5 \end{pmatrix}$

Each channel has its own kernel weights:

$$\text{Red} \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}, \quad \text{Green} \begin{pmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \quad \text{Blue} \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{pmatrix}$$

Thus, the computation is:

$$\text{output} = \text{Red contribution} + \text{Green contribution} + \text{Blue contribution}$$

Or more formally:

$$a\,(r, c, 1) = \sum_{u,v \in U} \sum_{k} w_1(u, v, k) \cdot x(r + u, c + v, k) + b_1 \tag{173}$$

Only **after** summing all channels do we obtain a single activation. Let's interpret this equation:

- $x(r+u, c+v, k)$ represents the **input** at position $(r+u, c+v)$ in channel k;
- $w_1(u, v, k)$ represents the **weight** of the filter at position (u, v) in channel k. The filter has **different weights for different channels**. The 1 subscript indicates that this is the first filter (we can have multiple filters);
- $\sum_{u,v \in U}$ **sums over** the spatial extent of the **filter** (e.g., 3×3);
- $\sum_{k}$ **sums over** the **channels** of the input volume (e.g., 3 for RGB);
- b_1 is the **bias** term for the first filter.

Note, each filter owns **one single bias** (see Figure 61, page 514). That same bias is added to **every spatial position** where the filter is applied. For example, if the output map has 128×128 values, all 128×128 neurons share the same bias. This is another consequence of **parameter sharing** in CNNs.

Why only one bias? Because we are **detecting the same feature everywhere**. If the feature is *vertical edge*, it should have the same baseline response regardless of where it appears. Changing the bias from one position to another would contradict the idea that the filter detects the **same pattern** across the image.

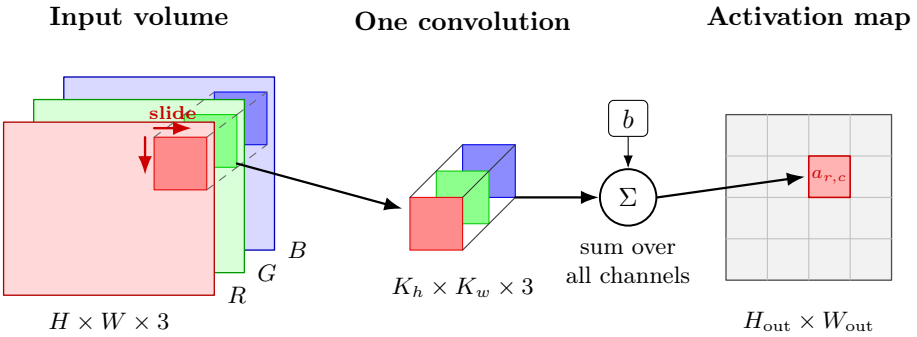


Figure 61: Local convolution over a multi-channel input volume. A single filter of size $K_h \times K_w \times 3$ is applied to one spatial location of the input volume. The contributions from the 3 input channels are summed, a single bias term b is added, and the resulting scalar activation $a(r, c)$ is stored at the corresponding position of the output activation map. Sliding the filter across the input repeats this computation to produce the **complete activation map**.

One filter produces one activation map. As shown in Figure 61, consider an input volume of size $64 \times 64 \times 3$ and a single convolutional filter of size $3 \times 3 \times 3$. At each spatial location, the filter processes a 3×3 neighbourhood from **all three input channels**. The responses obtained from the individual channels are summed together, a single bias is added, and the result is **one scalar** activation. Repeating this operation over the entire image produces a single output feature map of size $62 \times 62 \times 1$. Therefore, **one convolutional filter always produces exactly one output channel**.

Key Takeaways 6

In summary, a convolutional filter is a 3D kernel that spans the entire depth of the input volume. At every spatial location, it combines information from all input channels into one scalar activation. Sliding the filter over the input produces one output feature map. The steps are:

1. A convolutional filter spans the entire depth of the input volume.
2. At each spatial location, it combines information from all input channels.
3. The channel responses are summed and one bias is added to produce a single activation value.
4. Sliding one filter across the input generates one output feature map.

7.4.2 Multiple Filters and Output Feature Maps

In the previous section, we learned that for one filter, there is one output feature map. Now suppose the input is still a tiny image of size $64 \times 64 \times 3$, but instead of using **one filter**, we use **eight filters**. *What happens?*

Why do we use multiple filters?

Imagine trying to describe a face. If we ask only "*is there a vertical edge?*", we obtain only one type of information. But a face contains *eyes*, *nose*, *mouth*, *hair*, *contours*, *textures*, and so on. One detector cannot capture everything.

Therefore, CNNs learn **many different filters simultaneously**. Precisely, different filters specialize in different visual patterns.

Every filter learns a different pattern. Suppose we have four filters:

- Filter 1 might respond to *vertical edges*.
- Filter 2 might respond to *horizontal edges*.
- Filter 3 might respond to *corners*.
- Filter 4 might respond to *textures*.

Notice that **all filters receive exactly the same input volume**. The difference lies only in their weights.

All filters operate in parallel. Every filter independently performs exactly the computation described in the previous section (page 512, precisely, the flow described on page 514).

For example, suppose the input is a $64 \times 64 \times 3$ image, and every filter produces a $62 \times 62 \times 1$ output. Instead of keeping N maps separate, we **stack** them. If $N = 4$, the depth of the output volume is therefore **the number of filters**. In this case, the output volume is $62 \times 62 \times 4$.

Why does the depth increase? On page page 510, we observed an interesting pattern in CNNs: as we move deeper into the network, the number of channels (depth) generally increases, while the spatial dimensions (width and height) decrease. We can now **understand one of the reasons** why this happens. The depth increases because a convolutional layer typically **applies multiple filters**, and **each filter produces exactly one output feature map**. Therefore, the more filters we use, the deeper the output volume becomes. This also explains why deeper CNNs are able to represent increasingly rich visual information: by learning many different filters, each layer can specialize in detecting different patterns, from simple edges and textures to progressively more complex structures.

Key Takeaways 7

This section can be summarized by three rules:

1. **One filter** always produces **one output** feature map.
2. **Different filters** learn **different visual patterns** from the same input volume.
3. The output **depth** of a convolutional layer is exactly the **number of filters**.

7.4.3 General Formula for a Convolutional Layer

In this section we formalize everything we have seen so far about convolutional layers, and we provide a general equation. The previous sections established two facts about convolutional layers:

- **One filter spans all input channels**.
- **Multiple filters produce multiple output feature maps**.

The formula simply combines these two ideas.

Mathematically, a **convolutional layer** can be described with the following equation:

$$a\left(r,c,l\right)=\sum_{\substack{u,v\in U\\k=1,\ldots,C}} w_l\left(u,v,k\right)\cdot x\left(r+u,c+v,k\right)+b_l \tag{174}$$

Valid for $l=1,\ldots,N_F$.

Why do we need a new formula? Earlier, we had essentially a formula for a single filter, which was:

$$a\left(r,c\right)=\sum_{u,v\in U} w\left(u,v\right)\cdot x\left(r+u,c+v\right)+b$$

Which described the output of a single filter applied to a single input channel. Then, on page 512 we extended this formula to **multiple input channels** (page 513):

$$a\left(r,c\right)=\sum_{u,v\in U}\sum_{k} w\left(u,v,k\right)\cdot x\left(r+u,c+v,k\right)+b$$

Now, we are extending it to **multiple output feature maps**. The only missing ingredient was the index l to indicate **which filter we are using**, and thus **which output feature map we are computing**.

Understanding every symbol. Let's examine the formula term by term:

The output activation $a(r,c,l)$. This is the value computed by the convolutional layer. It has **three indices**:

- r is the row index of the output feature map.
- c is the column index of the output feature map.
- l is the index that identifies the filter and, equivalently, the output channel (e.g., $l=1$ is the first filter associated with the first feature map, $l=2$ is the second filter associated with the second feature map, and so on).

The input $x(r+u,c+v,k)$. This denotes the input volume. Notice the three indices:

- $(r+u)$ is the row index of the input volume.

- $(c+v)$ is the column index of the input volume.
- $(r+u, c+v)$ selects one pixel inside the local neighbourhood.
- k selects Red, Green, Blue, or, more generally, one of the C input feature maps.

The filter weights $w_l(u,v,k)$. These are the **trainable parameters**. Notice that the filter depends on u, v and k, but also on l. This means that each filter has its own set of weights, and each filter is applied to all input channels.

Why does the filter have index l? Suppose we have 32 filters in a convolutional layer. Instead of writing each filter as $w_1, w_2, \ldots, w_{32}$, we can write them as w_l with $l = 1, \ldots, 32$. This is a more compact notation.

The double summation. The computation is:

$$\sum_{u,v \in U} \sum_{k=1}^{C}$$

These two sums correspond to two different dimensions of the convolution operation:

- Sum over u, v is the usual convolution. It scans the local neighbourhood of the input volume, and it computes a weighted sum of the pixels in that neighbourhood.
- Sum over k is the contribution from all channels. For an RGB image $k = 1, 2, 3$ corresponds to Red, Green, Blue, but for a hidden layer $k = 1, \ldots, C$ corresponds to the C input feature maps.

Bias. Finally, we add a bias term b_l **for each filter**. The same bias is reused everywhere in the corresponding output feature map.

The equation can be read almost like **pseudocode**:

```
for each filter l = 1, ..., N_F:
    for each output row r:
        for each output column c:

            a(r, c, l) = b_l

            for each input channel k = 1, ..., C:
                for each spatial offset (u, v) in U:

                    a(r, c, l) += w_l(u, v, k) * x(r + u, c + v, k)
```

Number of Trainable Parameters of One Convolutional Layer

We can compute the exact number of all the weights of all filters in a convolutional layer, plus all their biases, with a simple formula. Let's build the formula step by step. First, suppose the input has C channels and each filter has a spatial size:

$$h_r \times h_c$$

Where h_r is the height of the filter and h_c is the width of the filter. Because one filter **spans all C input channels**, its actual shape is:

$$h_r \times h_c \times C$$

This means that the **number of weights in one filter** is:

$$h_r \cdot h_c \cdot C$$

Then, that filter has a bias term, so the **number of trainable parameters in one filter** is:

$$h_r \cdot h_c \cdot C + b, \quad b = 1$$

Now generalize. Suppose the layer contains N_F different filters. Each filter has its own weights and its own bias, so:

$$N_F \cdot (h_r \cdot h_c \cdot C + 1)$$

Expanding the parentheses, we get the final formula for the **number of trainable parameters in a convolutional layer**:

$$\text{Number of trainable parameters} = N_F \cdot (h_r \cdot h_c \cdot C) + N_F \tag{175}$$

$$\#\text{ params} = \#\text{ filters} \times (\text{weights per filter} + \text{bias per filter})$$

Notice that the bias term is simply N_F, because there is **one bias per filter**: $b \cdot N_F = 1 \cdot N_F = N_F$.

Furthermore, the spatial size of the input does not appear in the parameter-count formula. Because the **same filters are reused at every spatial location**. We do not learn a new $3 \times 3 \times 3$ filter for every pixel in the input image. Instead, we learn a single $3 \times 3 \times 3$ filter, and we apply it to all pixels in the input image.

> **Key Takeaways 8**
>
> The general convolution formula extends the single-filter computation by introducing the filter index l: for every output location (r, c) and every filter l, the convolution sums the contributions from all spatial positions in the kernel and all input channels, then adds one bias b_l to produce the activation $a(r, c, l)$. The **number of trainable parameters** in a convolutional layer is given by $N_F \cdot (h_r \cdot h_c \cdot C + 1)$, where N_F is the number of filters, h_r and h_c are the height and width of the filter, and C is the number of input channels.

7.4.4 Convolutional-Layer Hyperparameters

In this section, we will see **how to configure a convolutional layer** answering the following question: ***what are the design choices when creating a Conv2D layer?*** Let's start from an example:

```
Conv2D(
    filters = n_f,
    kernel_size = (h_r, h_c),
    activation = 'relu',
    strides = (1,1),
    padding = 'same'
)
```

Which of these values are chosen by us, and which are determined automatically?

What is a hyperparameter?

A **parameter** is **learned** during training. For example, the filter weights of a convolutional layer, or the biases. Thus, a **hyperparameter** is a parameter that is **set before training**, and it is not learned. For example, the number of filters, the kernel size, the stride, and the padding are all hyperparameters of a convolutional layer and the optimizer never changes them during training.

Thus, the Convolutional-Layer Hyperparameters are the following:

- **Number of filters**. This is:

  `filters = n_f`

  For example, if we set `filters = 32`, then the convolutional layer will learn 32 different filters, and thus it will produce 32 different output feature maps. This is probably the **most important hyperparameter** of a convolutional layer, as it determines the **capacity of the layer** to learn different features.

- **Kernel size**. This is:

  `kernel_size = (h_r, h_c)`

  This determines the **spatial size** of every filter. Notice that the kernel size specifies only height and width, but **not** the depth of the filter. *Why?* Because the **depth of the filter is automatically equal to the input depth**, it is not a free design choice.

  For example, if the input volume is a color image of size $32 \times 32 \times 3$, and we set `kernel_size = (5,5)`, then the convolutional layer will learn filters of size $5 \times 5 \times 3$. The depth of the filter is automatically equal to the input depth, which is 3 in this case.

  Similarly, if the previous layer is a convolutional layer that outputs a volume of size $16 \times 16 \times 64$, and we set `kernel_size = (3,3)` in the next

convolutional layer, then the next convolutional layer will learn filters of size $3 \times 3 \times 64$.

- **Stride**. This is:

  ```
  strides = (1, 1)
  ```

  Stride controls **how far the filter moves after each convolution**. For example, if we set `strides = (1, 1)`, then the filter moves one pixel at a time. If we set `strides = (2, 2)`, then the filter moves two pixels at a time. We will see later that the stride affects the size of the output feature map, and it is a hyperparameter that we can choose.

- **Padding**. This is:

  ```
  padding = 'same'
  ```

  Padding determines **how the image borders are handled**. If we set `padding = 'same'`, then the output feature map will have the same spatial size as the input. Again, the detailed explanation of padding will be given later, but for now it is enough to know that it is a hyperparameter that we can choose.

Notice that layers with the same hyperparameters can have different numbers of parameters depending on where they are located in the network.

At first this sounds strange. Suppose we build this CNN architecture:

1. Input image: $64 \times 64 \times 3$
2. Conv2D layer: 32 filters, kernel size 3×3, stride 1
3. Activation: $62 \times 62 \times 32$
4. MaxPool: $31 \times 31 \times 32$
5. Conv2D layer: 32 filters, kernel size 3×3, stride 1

The first convolutional layer has 32 filters of size $3 \times 3 \times 3$, thus it has:

$$\text{Number of parameters Conv2D layer } 1 = 32 \cdot (3 \cdot 3 \cdot 3) + 32 = 896$$

Now we apply max pooling, which reduces the spatial size of the feature maps to $31 \times 31 \times 32$. Then, we apply the second convolutional layer, which has 32 filters of size $3 \times 3 \times 32$, thus it has:

$$\text{Number of parameters Conv2D layer } 2 = 32 \cdot (3 \cdot 3 \cdot 32) + 32 = 9248$$

Much larger than the first convolutional layer, even though they have the same hyperparameters. The reason is that the input depth C is always **the depth of the input activation volume**, not the original image.

⚖ Parameters vs. Activations

Parameters are the **memory** of the network. They are **learned during training** and remain **fixed during inference**. For a convolutional layer, the parameters are only:

- The filter weights;
- One bias per filter.

Instead, activations are **not learned**. They are simply the outputs produced when an input image passes through the network. For each image that we feed to the network, the activations are different. For a convolutional layer, the activations are the output feature maps produced by convolving the input with the filters: $a(r, c, l)$. In general:

- **Parameters** describe the model.
- **Activations** describe the current input after it has been processed by the model.

Key Takeaways 9

A convolutional layer is defined by a few design choices (hyperparameters), while its trainable parameters are determined by those choices together with the depth of its input volume.

- A convolutional layer is configured by a small set of hyperparameters: number of filters, kernel size, stride, and padding.
- The filter depth is not chosen manually; it automatically matches the input depth.
- The number of filters determines the output depth.
- Trainable parameters (weights and biases) are different from activations (output values).
- Two convolutional layers can have identical hyperparameters but different numbers of trainable parameters because the input depth C depends on the previous layer.

7.5 CNN Arithmetic and Parameter Counting

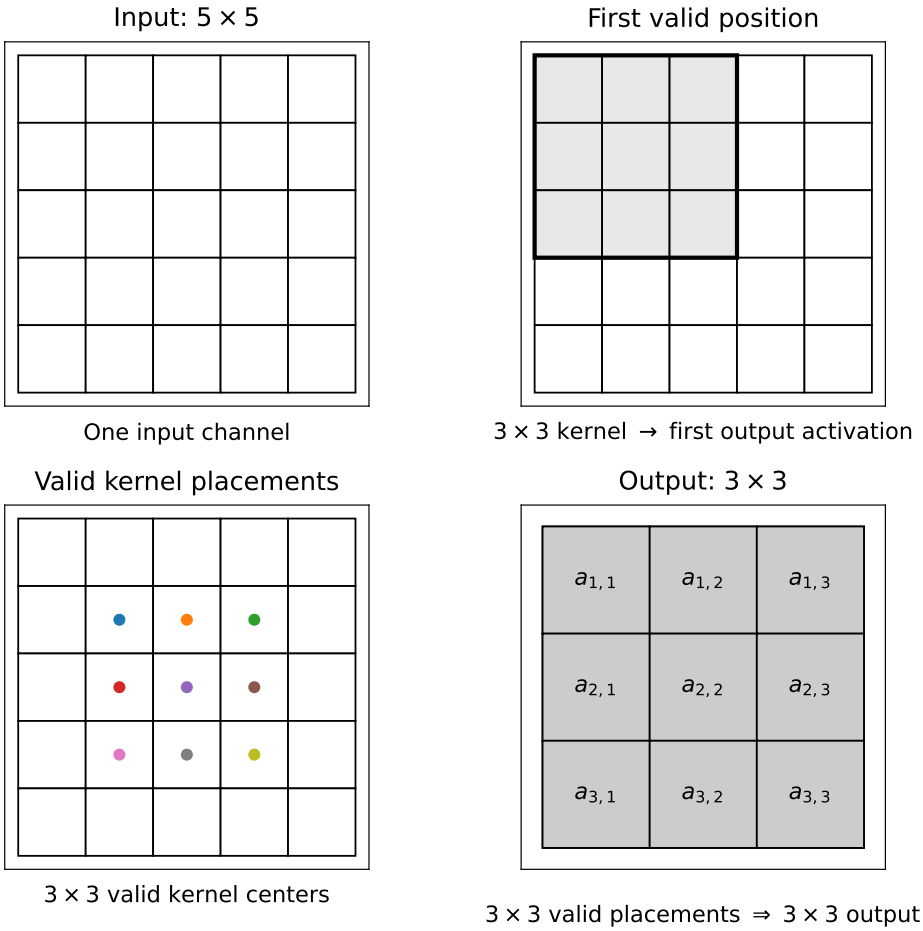
7.5.1 One Input Channel and One Filter

Until now, we focused on **how a convolutional layer works**. Now, the focus changes to: **given a convolutional layer, *how do we compute the dimensions of its output and how many parameters does it contain?*** Let's start with the simplest case: a convolutional layer with **one input channel and one filter**. Suppose we have a convolutional layer with one input channel:

$$\text{Input: } 32 \times 32 \times 1 \quad \text{and} \quad \text{Filter: } 3 \times 3 \times 1$$

Nothing changes compared with the previous section: **one filter still produces one output map**. The only simplification is that there is now only one input channel, so there is no summation over multiple channels k.

Now, if the input is $N \times N$, what is the size of the output? Until now we mostly discussed *what* convolution computes. Now we begin asking: ***how large is the result?*** Suppose the input is 5×5 and the kernel filter is 3×3. The filter cannot be centered on every input pixel, because near the borders, part of the filter would fall outside the image:



Considering the previous figure. If the input is 5×5, and the kernel is 3×3, with a stride (i.e., step size) of 1 and no padding (page 540), the filter can occupy only 3×3 different positions. Therefore, the output will be $3 \times 3 \times 1$.

In this section, we address the question, *why and when does the spatial size shrink?*. The answer is that the **filter must remain entirely within the image**.

However, this reasoning raises an issue. Without padding, the output is smaller than the input because the filter must remain entirely within the image. *How can we fix this?* First, *what happens when we add more filters and input channels? How many parameters do we get? Is there an equation to compute the output size?* We will answer these questions in the next sections.

7.5.2 Multiple Filters

Suppose the input is a single-channel image:

$$H \times W \times 1$$

Now use two different filters:

$$w_1 \in \mathbb{R}^{k_h \times k_w \times 1}, \quad w_2 \in \mathbb{R}^{k_h \times k_w \times 1}$$

Each filter is applied independently to the same input map, producing two different output feature maps:

$$\text{input} \rightarrow \begin{cases} w_1 \rightarrow \text{output feature map 1} \\ w_2 \rightarrow \text{output feature map 2} \end{cases}$$

Since one filter produces one output feature map, then the **number of output channels is equal to the number of filters**:

$$\text{number of output channels} = \text{number of filters}$$

Example 5: Multiple filters and output feature maps

Suppose:

$$\text{input} = 32 \times 32 \times 1 \quad \text{and} \quad 2 \text{ filters of size } 5 \times 5$$

Assuming stride 1 and no padding, each filter produces an output feature map of size:

$$32 - 5 + 1 = 28 \Rightarrow 28 \times 28 \times 1$$

Stacking the two output feature maps together, we get an output volume of size:

$$28 \times 28 \times 2$$

The spatial dimensions are determined by the convolution geometry, while the **depth 2 comes from the two filters**.

Why are the two output maps different? Because the filters have different learned weights. For example, conceptually:

- Filter 1 may respond strongly to vertical structures;
- Filter 2 may respond strongly to horizontal structures.

So even though both filters receive the same input, they usually produce different activation maps. This is why increasing the number of filters increases the number of patterns the layer can represent.

Key Takeaways 10

Each convolutional filter independently produces one feature map, so the number of filters determines the depth of the output volume. In symbols: $C_{\text{out}} = N_F$.

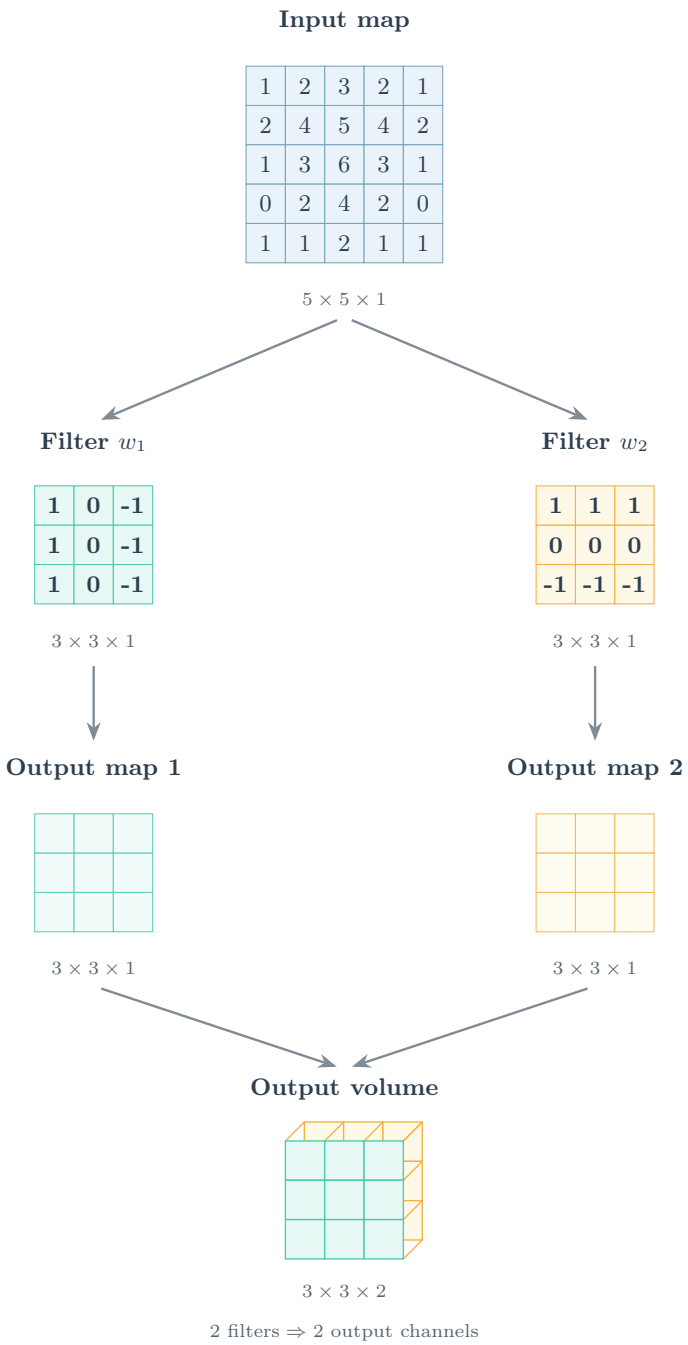


Figure 62: Multiple filters operating on the same input feature map. Each filter is independently applied to the entire input map and produces its own output feature map. The output feature maps are then stacked along the depth dimension to form the output volume. Consequently, the number of output channels is equal to the number of convolutional filters.

7.5.3 Multiple Input Channels

Now suppose the input is $32 \times 32 \times 3$. The three channels could represent RGB or, in deeper layers, three different feature maps. The arithmetic is exactly the same in both cases.

Does each channel have its own filter?

This is the most common misconception and the answer is **no**. Instead, **one filter spans all input channels**. For a 5×5 kernel and three input channels, one filter has shape $5 \times 5 \times 3$. So the filter is really:

- 5×5 for the first channel,
- 5×5 for the second channel, and
- 5×5 for the third channel.

These three slices belong to **one single filter**.

If each filter spans all input channels, what happens during convolution? Suppose the filter is placed at one spatial position:

- For the red channel we compute: s_R
- For the green channel we compute: s_G
- For the blue channel we compute: s_B

The convolution does **not** produce three separate outputs. Instead, the three responses are added together to produce a single output value:

$$a(r, c) = s_R + s_G + s_B + b$$

Therefore, one spatial position still produces **one scalar output value**, not three.

Even though channels are processed separately, their contributions are summed. The input depth affects **how much computation** is performed, but **not** the number of output feature maps. They are determined **only** by the number of filters.

Example 6: Multiple Input Channels

Suppose the input is $32 \times 32 \times 3$ and one filter has shape $5 \times 5 \times 3$. Assuming stride 1 and no padding, the spatial dimensions become:

$$32 - 5 + 1 = 28 \Rightarrow 28 \times 28 \times 1$$

Notice something important: although the filter contains 3 different 5×5 slices, the output is **still** a single 28×28 feature map. The **number of output channels is determined by the number of filters**, not the number of input channels.

Connection with the previous section: Multiple Filters

In the previous section, we saw that the depth of the output volume is determined by the number of filters:

$$\text{Output depth} = \text{Number of filters}$$

Now we see something equally important:

$$\text{Input depth} = \text{Filter depth}$$

Therefore:

- Input depth → Determines filter depth (how many slices each filter has)
- Number of filters → Determines output depth (how many output feature maps are produced)

Why does the number of parameters increase? Suppose previously we had a single filter of shape $5 \times 5 \times 1$ (one input channel). One filter contained $5 \cdot 5 \cdot 1 = 25$ weights. Now, the input has three channels, so the filter has shape $5 \times 5 \times 3$. This results in $5 \cdot 5 \cdot 3 = 75$ weights, which is three times more than before. However, the output size stays the same: $28 \times 28 \times 1$. But the filter is three times larger because it must learn how to combine information from all three input channels.

Relationship with the general convolution formula. This arithmetic example is exactly the concrete interpretation of the general convolution formula:

$$a(r, c, l) = \sum_{k=1}^{C} \sum_{u,v} w_l(u, v, k) \cdot x(r + u, c + v, k) + b_l$$

Previously, this looked like an abstract equation. Now tis meaning becomes clear:

- The inner sum computes the convolution within one channel;
- The outer sum over k adds the responses from all input channels;
- The result is **one activation** $a(r, c, l)$ for the l-th output channel.

Key Takeaways 11

- Every convolutional filter spans all input channels.
- The convolution is computed independently on each channel, and the channel-wise responses are summed.
- One filter still produces exactly one output feature map, regardless of how many input channels exist.
- The input depth determines the filter depth, while the number of filters determines the output depth.

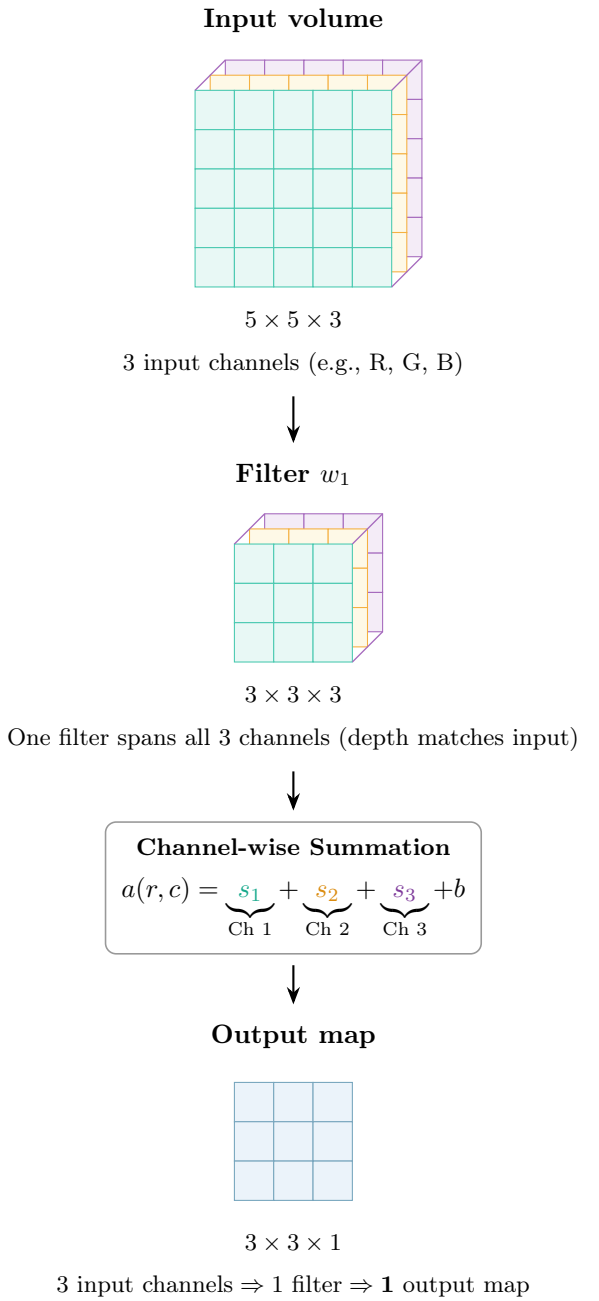


Figure 63: Convolution over multiple input channels. A convolutional filter spans the complete depth of the input volume, so its depth matches the number of input channels. At each spatial location, the filter computes one response for each input channel; these channel-wise responses are summed together with a single bias to produce one activation. Sliding the filter across the input produces one output feature map. Thus, the input depth determines the filter depth, whereas one filter still produces exactly one output channel.

7.5.4 Counting Convolutional Parameters

Given the input depth, the kernel size, and the number of filters, ***how many trainable numbers does this convolutional layer contain?***

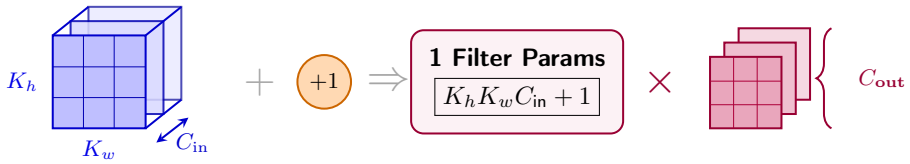
For a convolutional layer with:

- C_{in} input channels (depth),
- C_{out} output channels (number of filters),
- K_h kernel height,
- K_w kernel width,
- $K_h \times K_w$ kernel size,

The **number of trainable parameters** is:

$$\text{Number of trainable parameters} = C_{\text{out}} \left(K_h K_w C_{\text{in}} + 1\right) \tag{176}$$

Where the $+1$ is the **bias** associated with each filter. This equation is exactly the same as the one derived on page 519.



1 Filter Weights	1 Bias	Single Filter Total	Number of Filters
$K_h \times K_w \times C_{\text{in}}$	(per filter)	Weights + Bias	(C_{out} total filters)

Figure 64: Derivation of the number of trainable parameters in a convolutional layer. Each filter spans the complete input depth and therefore contains $K_h K_w C_{\text{in}}$ weights, plus one bias. Since the layer contains C_{out} independent filters, the total number of trainable parameters is $C_{\text{out}} \left(K_h K_w C_{\text{in}} + 1\right)$.

Build the formula

The easiest way to understand it is to forget the full formula at first. Let's suppose we have **one filter**. Because a **filter** spans the entire input depth, its **full shape** is:

$$\text{Filter shape} = K_h \times K_w \times C_{\text{in}}$$

Therefore, one filter has $K_h K_w C_{\text{in}}$ weights. Then it also owns one bias, so the **total number of trainable parameters for one filter** is:

$$\text{Trainable parameters for one filter} = K_h K_w C_{\text{in}} + 1 \tag{177}$$

Now if the layer has C_{out} filters, we simply multiply by the number of filters:

$$\text{Trainable parameters for the layer} = C_{\text{out}} \left(K_h K_w C_{\text{in}} + 1\right)$$

That is the entire derivation.

Example 7: RGB Input

Suppose $C_{\text{in}} = 3$ (RGB input), with 32 filters of size 3×3. One filter has $3 \cdot 3 \cdot 3 = 27$ weights, plus one bias, for a total of 28 trainable parameters. Now multiply by the number of filters: $28 \cdot 32 = 896$ trainable parameters in total. So the formula is:

$$\text{Trainable parameters} = 32\,(3 \cdot 3 \cdot 3 + 1) = 32\,(27 + 1) = 32 \cdot 28 = 896$$

Example 8: Deeper Convolutional Layer

Suppose the input is not RGB anymore, but the output of a previous convolutional layer which produces 64 feature maps. Therefore, $C_{\text{in}} = 64$, and we still have 32 filters of size 3×3. Then one filter has $3 \cdot 3 \cdot 64 = 576$ weights, plus one bias, for a total of 577 trainable parameters. Now multiply by the number of filters: $577 \cdot 32 = 18464$ trainable parameters in total. So the formula is:

$$\text{Trainable params} = 32\,(3 \cdot 3 \cdot 64 + 1) = 32\,(576 + 1) = 32 \cdot 577 = 18464$$

So even though the kernel size and number of filters are the same as before, the **layer** has many **more parameters** because the **input depth is larger**.
This is exactly the point emphasized earlier when we noted that layers with the same apparent hyperparameters may have different parameter counts depending on where they occur in the network.

What does not appear in the formula? The input spatial dimensions H_{in} and W_{in} do **not** appear in the parameter-count formula. This is because the same filter is reused at every spatial position. Whether the input is $32 \times 32 \times 3$ or $1024 \times 1024 \times 3$, a layer with 32 filters of size 3×3 will always have 896 trainable parameters. The input spatial dimensions only affect the output spatial dimensions, not the number of trainable parameters. This is the consequence of the **weight sharing** property of convolutional layers.

Key Takeaways 12

- Each filter contains $K_h K_w C_{\text{in}}$ weights because it spans the complete input depth.
- Each filter has one bias.
- There are C_{out} filters, so the total parameters count is:

$$C_{\text{out}}\,(K_h K_w C_{\text{in}} + 1)$$

- Input height and width do not affect the number of trainable parameters because the same filter is reused spatially.

7.5.5 CNN-Arithmetic Recap

This last section is essentially a **recap** rather than a new theoretical concept. After progressively introducing one input map, multiple filters, multiple input maps, and parameter counting, we can now summarize the **arithmetic of convolutional layers**.

The main goal is to make sure that we do **not confuse 5 different quantities**:

spatial size, input channels, filters, filter size, parameters

They are related, but they mean different things.

- **Image / activation size**. A CNN receives a volume:

  $$H \times W \times C_{\text{in}}$$

  Where:

  - H is the height of the input volume,
  - W is the width of the input volume,
  - C_{in} is the number of input channels.

  For the first layer, this could be an RGB image:

  $$H \times W \times 3, \quad \text{e.g., } 32 \times 32 \times 3$$

  For a deeper layer, it could instead be something like:

  $$H \times W \times 64, \quad \text{e.g., } 16 \times 16 \times 64$$

  At that point those 64 channels are **feature maps produced by a previous convolution**, not RGB channels.

- **Number of input channels**. C_{in} tells us the **depth of the incoming volume**. This quantity is important because every filter must span that entire depth. The filter depth is always equal to the input depth, so if the input has C_{in} channels, then every filter will have depth C_{in}.

  $$\begin{aligned} C_{\text{in}} = 3 \quad &\Rightarrow \quad 3 \times 3 \times \boxed{3} \\ C_{\text{in}} = 32 \quad &\Rightarrow \quad 3 \times 3 \times \boxed{32} \end{aligned}$$

  This is also why two convolutional layers with otherwise similar settings can have very different parameter counts.

- **Filter size**. When we say 3×3 filters, we normally specify only its **spatial dimensions**. Its complete shape is actually:

  $$K_h \times K_w \times C_{\text{in}}$$

  Because its depth is **automatically determined by the input**. So `kernel_size = (3, 3)` does **not** mean that the filter is literally two-dimensional. For an input with 64 channels, it represents a $3 \times 3 \times 64$ filter.

- **Number of filters**. Now comes the other depth: C_{out}. This is the **number of filters in the convolutional layer**. Each filter process **all** C_{in} input channels, combines their contributions, and produces **one output feature map**.

  $$C_{\text{out}} = \text{number of filters}$$

  For example, if we have 32 filters, we will produce 32 output feature maps. This gives us a very important relationship:

  $$C_{\text{in}} \rightarrow \text{filter depth}, \quad \text{whereas} \quad C_{\text{out}} \rightarrow \text{output depth}$$

- **Number of parameters**. Finally, parameters are the actual **learned numbers** contained in the filters and biases. As we already derived earlier on page 530:

  $$\#\text{ parameters} = C_{\text{out}} \cdot (K_h \cdot K_w \cdot C_{\text{in}} + 1)$$

  The $+1$ accounts for one bias per filter. Importantly, the H and W of the input volume do **not** appear in this equation because the number of parameters is **independent of the input size**. The same filter weights are reused at every spatial position.

In general, the input depth determines filter depth; number of filters determines output depth; kernel geometry determines how the spatial dimensions change; filter weights and biases determine the number of trainable parameters.

$$\begin{aligned}
\text{Input} &: H \times W \times C_{\text{in}} \\
\text{One filter} &: K_h \times K_w \times C_{\text{in}} \\
\text{Number of filters} &: C_{\text{out}} \\
\text{Output} &: H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}} \\
\text{Number of parameters} &: C_{\text{out}} \cdot (K_h \cdot K_w \cdot C_{\text{in}} + 1)
\end{aligned}$$

7.6 Important Details of Convolution

7.6.1 Correlation vs. Mathematical Convolution

This section is mainly a **terminology clarification**. Up to now, we have been calling the sliding filter operation a **convolution**, but mathematically the operation written in the earlier sections is actually a **correlation**. The two operations are **almost identical**. The only difference is whether the filter is **used *as it is*** or ***flipped*** before being applied.

For **correlation**, we write the operation as (page 423):

$$T[I](r,c) = \sum_{u,v \in U} w(u,v) \cdot I(r+u, c+v)$$

Here, the filter weights are used directly: $w(u,v)$.

For mathematical **convolution**, instead, we flip the filter before applying it:

$$T[I](r,c) = \sum_{u,v \in U} w(-u,-v) \cdot I(r+u, c+v)$$

Here, the filter weights are flipped: $w(-u,-v)$. Or equivalently, we can flip the image instead of the filter:

$$T[I](r,c) = \sum_{u,v \in U} w(u,v) \cdot I(r-u, c-v)$$

So the image indexing can be written in exactly the same form as correlation, provided that we replace $w(u,v) \to w(-u,-v)$. Thus, the convolution correspond to using the **flipped filter**.

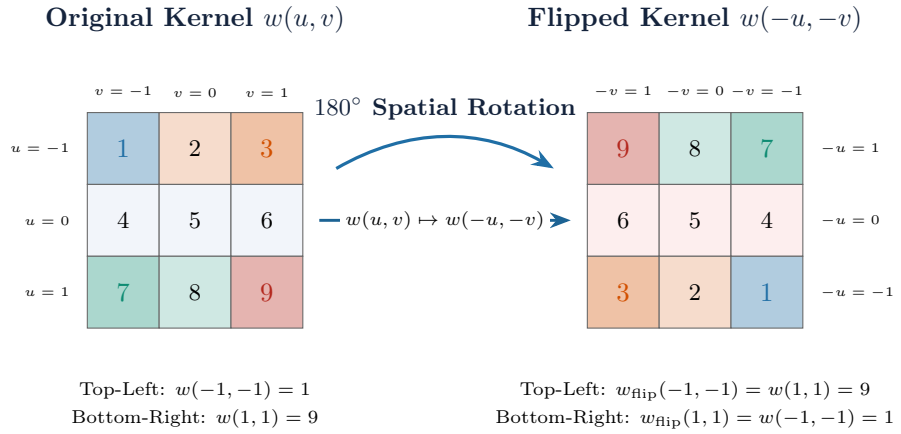


Figure 65: Kernel flipping in mathematical convolution. Replacing $w(u,v)$ with $w(-u,-v)$ reverses the kernel along both spatial dimensions, which is equivalent to a 180° rotation. Thus, while correlation applies $w(u,v)$ directly, mathematical convolution applies its spatially flipped version $w(-u,-v)$.

Why this distinction?

This distinction is important in different contexts:

- **Classical Signal/Image Processing, the distinction matters**. Suppose we manually choose a fixed kernel:

$$w = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

  Correlation uses exactly this matrix. True convolution first flips it:

$$w_{\text{flip}} = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

  Those can give different responses. For example, opposite signs for an oriented edge. So if the kernel is **given beforehand**, we absolutely need to know which operation we are performing.

  Mathematical convolution specifically includes the flip because that definition gives convolution its useful algebraic structure in signal processing, particularly through the Fourier theorem.

- **In a CNN, however, nobody gives us** w. Suppose true convolution would need to learn:

$$w = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

  If our library performs correlation instead, training can simply learn the flipped version:

$$w_{\text{flip}} = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

  Then $\text{corr}(x, w_{\text{flip}})$ **produces the same result** that $\text{conv}(x, w)$ would have produced. So from the optimizer's perspective, learn w and flip it during convolution, and learn the already-flipped w_{flip} and use correlation, are **equivalent** parameterizations.

  That is what Goodfellow et al. [3] means when saying that an algorithm using kernel flipping simply learns a kernel that is flipped relative to the one learned by an algorithm without flipping.

> **Deepening: What is Goodfellow's message?**
>
> Goodfellow means that two CNNs can learn the same operation even if one implements true convolution and the other implements correlation; their stored kernels will simply be flipped versions of each other.

Suppose the useful operation requires, under **true convolution**, the learned kernel:

$$W = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

True convolution flips it before applying it:

$$W' = \begin{bmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{bmatrix}$$

So the input actually gets multiplied against the pattern W'.

Now consider another CNN implementation that performs **correlation**. It doesn't flip anything, so during training, it can simply learn the flipped version of the kernel:

$$W' = \begin{bmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{bmatrix}$$

Then it applies W' directly. Therefore:

$$\underbrace{\text{Conv}(X, W)}_{\text{flip } W\text{, then apply}} = \underbrace{\text{Corr}(X, W')}_{\text{apply } W' \text{ directly}}$$

In other words:

$$W_{\text{convolutional implementation}} = \text{flip}\left(W_{\text{correlation implementation}}\right)$$

The **numerical values stored as the kernel are different**, but the operation performed on the input can be identical. This is possible precisely because **the kernel is not predetermined**. Gradient descent is free to learn whichever orientation is needed.

True convolution		**CNN correlation**
learn W		learn $\text{flip}(W)$
↓		↓
flip W	$\equiv$	no flip
↓		↓
apply		apply

So Goodfellow is **not saying that training explicitly takes a learned kernel and flips it**. Rather, if we trained two otherwise equivalent systems, one using mathematical convolution and one using correlation, the kernels they learn would correspond to each other through a flip. That's why the choice does not reduce what the CNN can learn.

Key Takeaways 13

- Correlation applies the filter directly, whereas mathematical convolution flips the filter before applying it.
- CNN libraries typically implement correlation but conventionally call it convolution.
- Since CNN filters are learned from data, this distinction is usually irrelevant in practice, provided that the same convention is used consistently during learning and inference.
- Correlation: $I(r+u, c+v)$.
- Convolution: $I(r-u, c-v)$.

7.6.2 CNN Convolution vs. RGB Image Filtering

In this section, we clarify the distinction that is easy to miss because "*convolution on an RGB image*" can mean two different operations depending on whether we are doing classical image filtering or using a CNN.

The main distinction is simple:

Traditional RGB filtering keeps channels separate

Whereas:

CNN convolution mixes the input channels together to produce a single output channel

Traditional Filtering of an RGB Image

Recall that an RGB image consists of three channels:

$$I \in \mathbb{R}^{H \times W \times 3}$$

Which we can think of as R, G, and B channels. In the case of traditional filtering, convolution $\circledast$ is performed **separately on each color channel**. For example:

$$R' = R \circledast w_R, \quad G' = G \circledast w_G, \quad B' = B \circledast w_B$$

The crucial point is that we **do not sum** these three channels together. Instead, we keep them separate and produce a new RGB image:

$$I' = \begin{bmatrix} R' \\ G' \\ B' \end{bmatrix} \in \mathbb{R}^{H' \times W' \times 3}$$

This makes sense for operations such as applying a blur to a color image (page 498): we may want to blur the red, green, and blue planes while still obtaining an RGB image.

A CNN does something different

Now consider **one CNN filter** applied to the same RGB image (page 509). We already know that a CNN filter spans the complete input depth. For an RGB input and a 3×3 spatial kernel, one filter has shape $3 \times 3 \times 3$. We can conceptually split that filter into three slices: w_R, w_G, and w_B. At a particular spatial position (r, c), the CNN first computes a contribution from each channel: s_R, s_G, and s_B. But here comes the important difference:

$$a(r, c) = s_R + s_G + s_B + b$$

The channel-wise contributions are **summed**. So:

$$R, G, B \xrightarrow{\text{one CNN filter}} \text{one feature map}$$

Not three feature maps. Formally:

$$a(r,c) = \sum_{k=1}^{3} \sum_{u,v \in U} w(u,v,k) \cdot I(r+u, c+v, k) + b$$

The value $a(r,c)$ depends simultaneously on values from R, G, and B channels. Consequently, after the convolution we should **not** think of the resulting map as a red, green, or blue image. Instead, it is a learned **feature map**.

Key Takeaways 14

Traditional RGB image filtering may convolve the R, G, and B channels separately, preserving the three-channel structure. In a standard CNN convolution, instead, each filter spans all C_{in} input channels: the channel-wise responses are summed at every spatial location, so one filter produces one 2D feature map. Thus, CNN convolution mixes information across channels, whether those channels are RGB components or feature maps from a previous layer.

7.6.3 Padding

This section answers a problem that has been implicit on page 523: ***what happens when the filter reaches the boundary of the image?*** Or in other words: *how to define convolution output close to image boundaries?* We will see the **zero padding** technique, which is the most common solution to this problem, while mentioning **no padding** and **symmetric padding** as alternative solutions.

The boundary problem

Suppose we have a 5×5 input image and a 3×3 filter. If we require the filter to remain completely inside the input, the first valid position of the filter is when its center is at $(2, 2)$, which corresponds to the pixel $x_{2,2}$ in the input image.

But we cannot center the filter on the top-left pixel, for example, because part of the 3×3 filter would fall **outside the image**. So without doing anything special at boundary, the filter has fewer valid positions than there are pixels. This is called the **boundary problem** in convolution, because the filter cannot be centered on boundary pixels (see Figure 66).

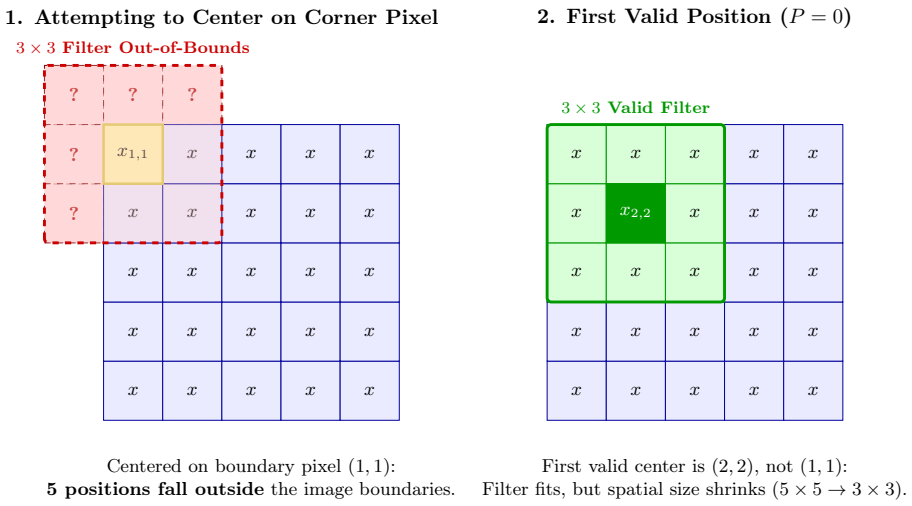


Figure 66: On the left, we attempt to center a 3×3 filter on the top-left corner pixel of a 5×5 input image. The filter extends outside the image boundaries, which is the **boundary problem**. On the right, we show the first valid position of the filter, centered on pixel $(2, 2)$, which is fully contained within the input image. This results in a smaller output size (3×3) compared to the input size (5×5).

No padding / Valid convolution

The simplest possibility is therefore to do nothing. The filter is evaluated **only where it fits completely inside the input**. This is commonly called **valid**

convolution: $P = 0$ (no padding). Assuming a stride of 1, the output size is calculated as:

$$N_{\text{out}} = N_{\text{in}} - K + 1$$

Where N_{in} is the size of the input image (height or width), K is the kernel size of the filter (height or width), and N_{out} is the size of the output image (height or width). In our example, this means that a 5×5 input image convolved with a 3×3 filter produces a:

$$N_{\text{out}} = 5 - 3 + 1 = 3, \quad 3 \times 3 \text{ output image.}$$

With no padding, the **spatial dimensions of the output shrink after each convolutional layer**. This isn't necessarily bad. Sometimes shrinking the representation is exactly what we want.

The idea of padding

Instead, we can artificially extend the input around its borders. For example, start with:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Add one layer of values around it:

$$\begin{bmatrix} ? & ? & ? & ? & ? \\ ? & 1 & 2 & 3 & ? \\ ? & 4 & 5 & 6 & ? \\ ? & 7 & 8 & 9 & ? \\ ? & ? & ? & ? & ? \end{bmatrix}$$

Now a 3×3 filter can be position so that its center corresponds even to an original **border pixel**, because the missing neighborhood outside the image has been supplied artificially. That's padding: it extends the input at its spatial boundaries. Importantly, padding is spatial. We pad height and width, but we are **not adding channels**.

Zero Padding

The most common choice is to **fill** those additional **positions with zeros**. Thus:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \xrightarrow{P=1} \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 & 0 \\ 0 & 4 & 5 & 6 & 0 \\ 0 & 7 & 8 & 9 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

The original image is surrounded by zero-valued pixels so that convolution can also be evaluated at the boundaries.

What does $P = 1$ mean? It means one row/column of padding on each side of the input. So an $N \times N$ input becomes temporarily:

$$\underbrace{(N + 2P)}_{\text{rows}} \times \underbrace{(N + 2P)}_{\text{columns}} \xrightarrow{P=1} (N + 2) \times (N + 2)$$

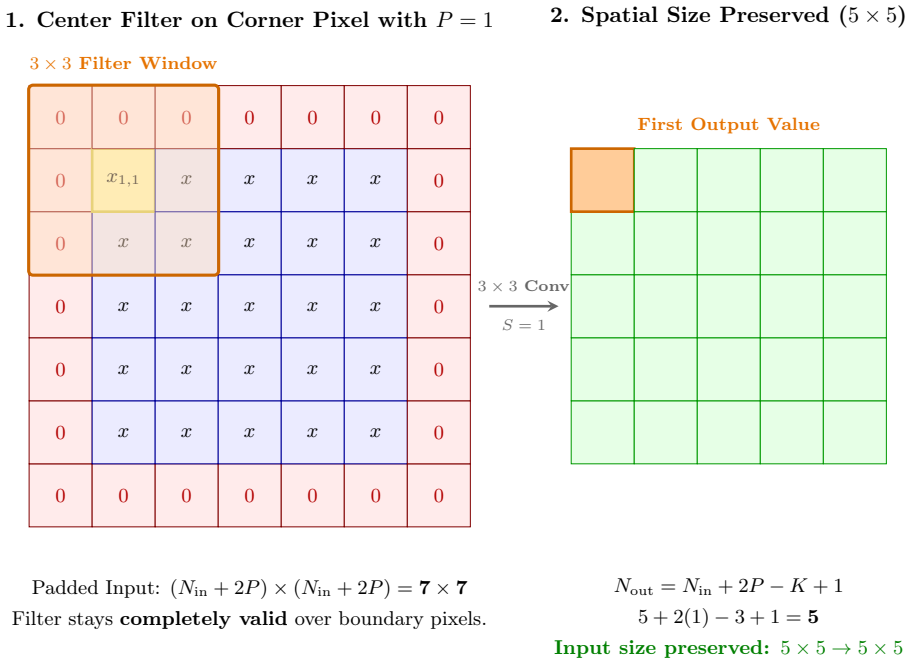


Figure 67: Zero padding ($P = 1$) extends spatial boundaries with zeros, allowing the 3×3 filter to align directly with edge pixels and preserving the original spatial resolution.

How much padding do we need to preserve the size? In general, if we assume:

- Square input of size $N \times N$,
- Square filter of size $K \times K$,
- Stride $S = 1$,
- Equal padding on all sides,

We want to find the amount of padding P that preserves $N_{\text{out}} = N_{\text{in}}$. Using the formula for **output size of a convolution operation** (with stride $S = 1$):

$$N_{\text{out}} = N_{\text{in}} + 2P - K + 1 \tag{178}$$

Where:

- N_{in} is the size of the input image (height or width),
- K is the kernel size of the filter (height or width),
- P is the amount of padding on **each side**,
- N_{out} is the size of the output image (height or width).

Setting $N_{\text{out}} = N_{\text{in}}$ gives:

$$\begin{aligned} N_{\text{in}} &= N_{\text{in}} + 2P - K + 1 \\ 0 &= 2P - K + 1 \\ 2P &= K - 1 \\ P &= \frac{K-1}{2} \end{aligned}$$

For odd-sized filters (K odd), this results in an integer padding value. For even-sized filters, the padding would be fractional, which is not practical, so typically odd-sized filters are used in practice.

✣ Symmetric padding

Instead of inventing zeros outside the image, **Symmetric Padding** extends the boundary using a mirrored version of the existing image values. Conceptually, if the beginning of a row is:

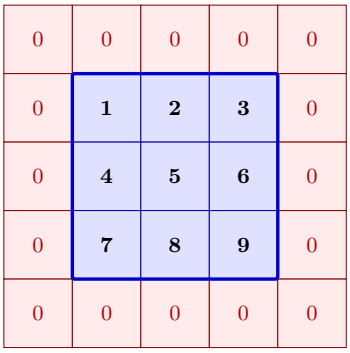
$$[a, b, c, d, \dots]$$

A symmetric extension uses values reflected from the boundary rather than:

$$[0, a, b, c, d, \dots]$$

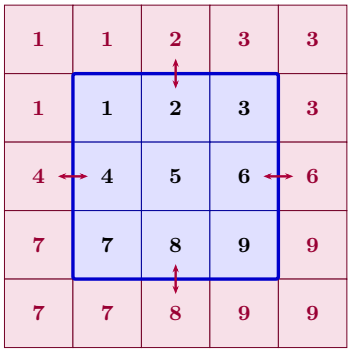
The important difference is that symmetric padding uses **real image values** rather than zeros, which can help preserve edge information in the convolution output.

1. Zero Padding ($P = 1$)



Outside values are fixed to **zeros**:
[**0**, **1**, **2**, **3**, **0**]

2. Symmetric Padding ($P = 1$)



Outside values are **mirrored** from boundary:
[**1**, **1**, **2**, **3**, **3**]

Figure 68: Comparison of boundary extension strategies. **Left:** Zero padding fills outer positions with 0. **Right:** Symmetric padding mirrors existing image boundary values outward across the original image border.

⚠ **Padding does NOT add trainable parameters.** Because **padding changes where the filters can be applied**, not the filters themselves. Therefore, P affects output spatial dimensions but not parameter count.

7.6.4 Stride

In short, **Stride** controls how far the filter moves between two consecutive convolution positions. With stride 1, the filter moves one pixel at a time. With stride 2, it "*jumps*" every second row and every second column. Larger stride values reduce the spatial extent of the output, and that stride > 1 is conceptually equivalent to convolution followed by downsampling, but without computing values that would then be discarded (*optimization*).

Suppose we have a one-dimensional input first. With stride $S = 1$, the kernel moves as: position 0, position 1, position 2, position 3, and so on. With stride $S = 2$, the kernel moves as: position 0, position 2, position 4, and so on. So we compute **fewer output activations** with larger stride values. That is why stride performs a kind of spatial downsampling (i.e., reduces the spatial dimensions of the output).

Why stride is downsampling? Because it skips some input locations, and therefore the output is smaller. Imagine first doing the full stride-1 convolution:

$$[a_0, a_1, a_2, a_3, a_4]$$

Then keep only every second value:

$$[a_0, a_2, a_4]$$

That gives the same sampling pattern as a convolution performed directly with stride 2. So conceptually:

$$\text{stride-2 convolution} \approx \text{stride-1 convolution} + \text{downsampling by 2}$$

Notice that doing the two operations separately would be **less efficient** because many intermediate activations would be computed and then discarded. Instead, stride > 1 allows us to skip those computations entirely.

Example 9: Stride and Downsampling

Suppose:

$$N_{\text{in}} = 7, \quad K = 3, \quad P = 0$$

With stride 1, the valid kernel starting position are:

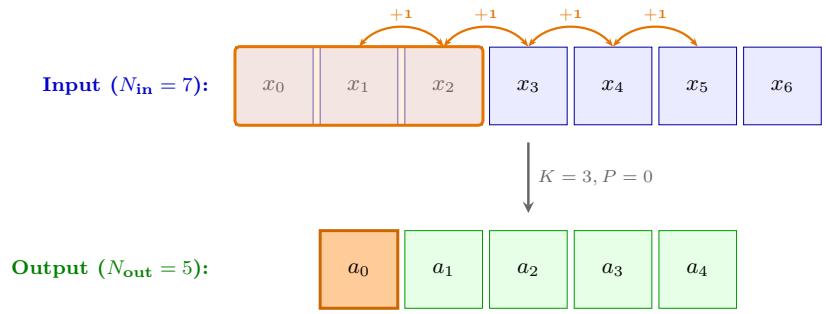
$$0, 1, 2, 3, 4$$

So the output size is $N_{\text{out}} = 5$. With stride 2, instead, we only use:

$$0, 2, 4$$

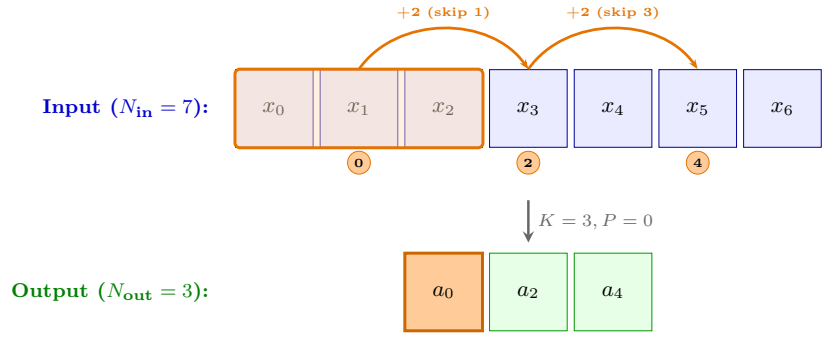
Therefore $N_{\text{out}} = 3$. Same input, same kernel, fewer positions because we skip locations.

1. Stride $S = 1$ (Move 1 position at a time)



$N_{\text{out}} = N_{\text{in}} + 2P - K + 1 = 7 + 0 - 3 + 1 = \mathbf{5}$ valid activations

2. Stride $S = 2$ (Jump 2 positions at a time / Downsampling)



$N_{\text{out}} = \left\lfloor \frac{N_{\text{in}}+2P-K}{S} \right\rfloor + 1 = \left\lfloor \frac{7-3}{2} \right\rfloor + 1 = \mathbf{3}$ (**Downsampled**)

Figure 69: Visualizing the effect of stride on spatial resolution ($N_{\text{in}} = 7, K = 3, P = 0$). **Top:** Stride $S = 1$ evaluates every valid position $(0, 1, 2, 3, 4)$, producing 5 activations. **Bottom:** Stride $S = 2$ skips intermediate starting locations (1 and 3), evaluating only positions $0, 2, 4$ and downsampling the output to 3 activations.

7.6.5 General Output-Size Formula

Now we can extend the formula introduced conceptually in the padding section (page 542). With padding but stride 1, the output size is:

$$N_{\text{out}} = N_{\text{in}} + 2P - K + 1$$

Because the number of valid kernel starting positions was $N_{\text{in}}+2P-K+1$. Now we introduce stride S. We are no longer visiting **every** valid starting position, but only every S-th position. So the available travel distance is still:

$$N_{\text{in}} + 2P - K$$

If we move by steps of size S, the number of jumps that fit in that distance is:

$$N_{\text{out}} = \left\lfloor \frac{N_{\text{in}} + 2P - K}{S} \right\rfloor$$

Then we add 1 because the initial position counts too:

$$N_{\text{out}} = \left\lfloor \frac{N_{\text{in}} + 2P - K}{S} \right\rfloor + 1 \tag{179}$$

The floor function is necessary because the last jump may not fit exactly (e.g., if $N_{\text{in}} = 8, K = 3, P = 0, S = 2$, the jump to perform are $(8-3)/2 = 2.5$, but we cannot place a kernel at half a stride, so we take the floor and add 1 for the initial position, giving 3 valid positions: $0, 2, 4$).

🔗 Relationship with the previous formulas

If stride $S = 1$, the general formula becomes:

$$N_{\text{out}} = \left\lfloor \frac{N_{\text{in}} + 2P - K}{1} \right\rfloor + 1 = N_{\text{in}} + 2P - K + 1$$

Which is exactly the formula we derived in the padding section (page 542). If also there is no padding ($P = 0$), we get simply:

$$N_{\text{out}} = \left\lfloor \frac{N_{\text{in}} - K}{S} \right\rfloor + 1 = \left\lfloor \frac{N_{\text{in}} - K}{1} \right\rfloor + 1 = N_{\text{in}} - K + 1$$

⚠ Finally, just like padding, stride affects **where the filter is evaluated**, not the filter weights themselves. Stride does not change the number of parameters.

7.7 Activation Functions

7.7.1 Why CNNs Need Nonlinearities

This section reconnects CNNs with something we already saw earlier for MLPs: stacking linear operations alone does not make the model nonlinear.

A convolutional layer is a linear/affine operation. If we flatten the input appropriately, it can be written in the familiar form:

$$\mathbf{a} = W\mathbf{x} + \mathbf{b}$$

The same is true for a dense layer (i.e., a fully connected layer). So if we stack only convolutional and dense layers, without inserting nonlinear activations, the whole network can still collapse into one affine transformation.

For example, with two layers:

$$\mathbf{h} = W_1\mathbf{x} + \mathbf{b}_1$$

And then the second layer:

$$\mathbf{y} = W_2\mathbf{h} + \mathbf{b}_2$$

Substitute the first into the second:

$$\mathbf{y} = W_2\left(W_1\mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2 = W_2W_1\mathbf{x} + W_2\mathbf{b}_1 + \mathbf{b}_2$$

Define:

$$W' = W_2W_1, \quad \mathbf{b}' = W_2\mathbf{b}_1 + \mathbf{b}_2$$

Then we have:

$$\mathbf{y} = W'\mathbf{x} + \mathbf{b}' \tag{180}$$

Which is still just one affine transformation. This is also the same argument used earlier in the image-classification section for MLPs (Multi-Layer Perceptron): multiple linear layers without nonlinearities are equivalent to a single linear classifier.

The same reasoning applies to CNNs because convolution itself is linear with respect to the input. So a stack such as:

$$\text{Conv}_1 \to \text{Conv}_2 \to \text{Conv}_3 \to \dots$$

With no nonlinearities between them is **still equivalent**, in expressive power, to one larger linear transformation. That would defeat one of the main purposes of using a deep network: depth would not allow the model to represent genuinely nonlinear relationships.

✅ **Activation functions solve this problem** by inserting a nonlinear transformation between linear layers:

$$\text{Conv}_1 \to g(\cdot) \to \text{Conv}_2 \to g(\cdot) \to \text{Conv}_3 \to g(\cdot) \to \dots$$

For example:

$$\mathbf{h} = g\left(W_1\mathbf{x} + \mathbf{b}_1\right)$$

Now, because $g(\cdot)$ is nonlinear, we cannot generally combine the two layers into a single matrix multiplication:

$$\mathbf{y} = W_2 g\left(W_1 \mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2$$

Cannot be simplified to a single linear transformation. That is the fundamental reason activation functions increase the expressive power of the CNN.

However, it is important to note a fundamental detail: activation functions operate **element by element** on the activation volume. Suppose a convolution produces:

$$A \in \mathbb{R}^{H \times W \times C}$$

An activation function $g(\cdot)$ produces:

$$A'(r, c, k) = g\left(A(r, c, k)\right)$$

So if:

$$A = \begin{bmatrix} -2 & 3 \\ 1 & -4 \end{bmatrix}$$

And we layer use an activation function such as ReLU:

$$g(x) = \max(0, x) \quad \Rightarrow \quad A' = \begin{bmatrix} 0 & 3 \\ 1 & 0 \end{bmatrix}$$

Each number is transformed independently. This also explains why activation layers normally **do not change tensor dimensions**. They change the **values**, not the shape.

> **Key Takeaways 15**
>
> Convolutional layers are linear operations. Therefore, stacking several convolutional layers without activation functions would still yield an overall linear transformation and would not increase the expressive power of the network. Nonlinear activation functions are inserted between convolutional layers to allow the CNN to model nonlinear relationships. They operate independently on each activation value and therefore change the values of a feature volume without changing its shape.

7.7.2 ReLU and Leaky ReLU

This section is about **which nonlinear activation to use after convolution**. We will focus on ReLU first, then introduce Leaky ReLU as a small modification that avoids one of ReLU's main failure modes.

The standard ReLU activation function is (page 180):

$$\text{ReLU}(x) = \max(0, x) \quad \equiv \quad \text{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$

So ReLU simply keeps positive activations unchanged and clips negative ones to zero. Thus, we can use ReLU as a thresholding operation on the feature maps.

For example:

$$[-2, 3, -1, 5] \xrightarrow{\text{ReLU}} [0, 3, 0, 5]$$

As we said, ReLU is applied **independently to every activation value**. If a convolutional layer produces:

$$A \in \mathbb{R}^{H \times W \times C}$$

Then ReLU will produce:

$$A'(r, c, k) = \text{ReLU}(A(r, c, k))$$

It does not mix channels, it does not slide like a convolution, and it does not aggregate neighboring values. It **just transforms each scalar separately**.

Because of this, the tensor shape does not change after ReLU:

$$A' \in \mathbb{R}^{H \times W \times C}$$

Only the values are changed.

⚠ **Dying Neuron Problem.** The main drawback of ReLU is that it can produce **dead neurons**. We already explained this issue on page 180. In short, if a unit (i.e., a neuron) enters a regime where its pre-activation is always negative (i.e., the convolutional filter is producing negative values for that unit), then ReLU always outputs zero:

$$x < 0 \implies \text{ReLU}(x) = 0$$

And on the negative side, the gradient is also zero:

$$x < 0 \implies \frac{d}{dx}\text{ReLU}(x) = 0$$

So no useful gradient flows backward through that unit, and the **neuron may remain permanently inactive**. In simple terms, dying ReLU units can become inactive for essentially all inputs and stop learning.

Leaky ReLU. Leaky ReLU addresses this by replacing the flat zero region on the negative side with a small nonzero slope:

$$\text{LeakyReLU}(x) = \begin{cases} x & x \geq 0 \\ \alpha x & x < 0 \end{cases}$$

Where α is usually a small positive constant, e.g., $\alpha = 0.01$. A tiny negative slope means the activation is not completely dead on the negative side, which helps preserve gradient flow.

Key Takeaways 16

ReLU applies $x \mapsto \max(0, x)$ independently to every activation, preserving the feature-map shape while introducing nonlinearity. Its main drawback is the dying-neuron problem, because negative activations have zero slope. Leaky ReLU mitigates this by using a small nonzero negative slope, typically $\alpha = 0.01$, so negative activations and gradients are not completely suppressed.

7.7.3 Hyperbolic Tangent and Other Activations

This section is mostly a **comparison** rather than a new mechanism. Here we will introduce **tanh** and **sigmoid** as other valid activation functions, but we will also make clear that, in CNNs, ReLU is the usual practical choice.

The **hyperbolic tangent** function is defined as (page 67):

$$\tanh(x) = \frac{2}{1 + e^{-2x}} - 1$$

And its output range is $(-1, 1)$. So unlike ReLU, which can produce any nonnegative value, tanh **squashes** every input into a bounded interval. For example:

$$x \to +\infty \quad \Rightarrow \quad \tanh(x) \to 1$$

While:

$$x \to -\infty \quad \Rightarrow \quad \tanh(x) \to -1$$

Also unlike ReLU, tanh is **differentiable** everywhere, continuous, and bounded.

Sigmoid behaves similarly (page 64):

$$S(x) = \frac{1}{1 + e^{-2x}}$$

But its range is: $(0, 1)$.

However, in general sigmoid and tanh are more typical of MLP architectures than CNNs. Let's compare the four activation functions we have discussed so far:

Activation	Output range	Negative outputs?	Bounded?
ReLU	$[0, +\infty)$	No	No
Leaky ReLU	$(-\infty, +\infty)$	Yes	No
tanh	$(-1, 1)$	Yes	Yes
sigmoid	$(0, 1)$	No	Yes

The practical reason ReLU became dominant is related to training. Tanh and sigmoid **saturate** for large positive or negative inputs: once they are near their asymptotes, their derivatives become very small. That can make gradients shrink during backpropagation and contribute to the vanishing-gradient problem (page 177).

ReLU instead has a much simpler behavior on the positive side:

$$\text{ReLU}(x) = x \quad \text{for } x > 0,$$

With derivative 1 there.

Tanh and sigmoid are valid nonlinear activations, but ReLU became the standard choice for CNN hidden layers because it is simpler and generally easier to optimize in deep networks.

Finally, just like ReLU, tanh and sigmoid are still applied **element-wise**. So if the input activation volume is $H \times W \times C$, the output remains $H \times W \times C$. **Only the values change**.

- **Sigmoid**: works, but saturates and can cause vanishing gradients.
- **Tanh**: generally better than sigmoid around zero because it is zero-centered, but it still saturates.
- **ReLU**: usually preferred because it is simple, fast, and does not saturate for positive inputs.
- **Leaky ReLU**: fixes one important ReLU weakness by allowing a small gradient for $x < 0$, reducing the **dying ReLU** problem.

7.8 Pooling Layers

We now move to another important component of the classical CNN architecture: **pooling**.

Until now, convolutional layers have extracted features and activation functions have introduced nonlinearity. Pooling has a different purpose: it reduces the spatial size of the feature maps. They operate independently on every depth slice and spatially resizing it, commonly using the **maximum** operation.

7.8.1 Max Pooling

Suppose we have one feature map produced by a convolution followed by ReLU:

$$A = \begin{bmatrix} 1 & 3 & 2 & 0 \\ 4 & 6 & 1 & 2 \\ 2 & 1 & 5 & 3 \\ 0 & 2 & 4 & 7 \end{bmatrix}$$

With a 2×2 *max-pooling window*, we divide the feature map into local regions and keep only the **largest activation** from each region. For example, the max pooling operation on the top-left 2×2 region of A is:

$$\text{MaxPool}\left(\begin{bmatrix} 1 & 3 \\ 4 & 6 \end{bmatrix}\right) = 6$$

Doing this for all four regions gives us the following output:

$$\text{MaxPool}(A) = \begin{bmatrix} 6 & 2 \\ 2 & 7 \end{bmatrix}$$

So we have **compressed** the 4×4 feature map into a 2×2 feature map, while keeping the most important information. A 2×2 support discards 75% of the spatial samples when used for this downsampling operation.

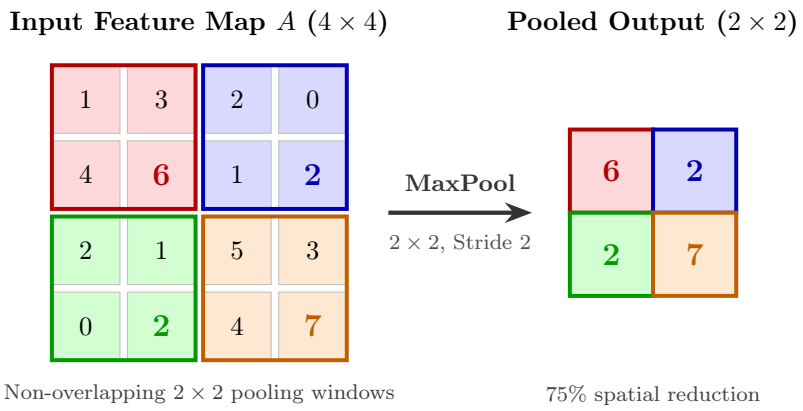


Figure 70: Illustration of a 2×2 max pooling operation on a 4×4 feature map.

Why take the maximum?

In a feature map, the numbers represent the **activation** of a feature detector at a particular spatial location. A convolutional filter may have learned to respond strongly to some pattern. After ReLU, a large activation roughly tells us: "***the feature detected by this filter is strongly present here***".

Max pooling summarizes a small neighborhood by retaining its strongest response:

$$y(r, c) = \max_{(u,v)\in U} x(r + u, c + v) \tag{181}$$

So instead of preserving the exact activation values, we only keep the strongest response. We **lose spatial detail**, but **preserve the strongest evidence** that the feature occurred somewhere inside that region.

This connects to the broader CNN idea: as representations become deeper, we are often increasingly interested in **whether a feature exists**, rather than its exact pixel-level position.

Pooling does NOT mix channels

Pooling behaves very differently from convolution. Suppose the input volume is $32\times32\times64$. There are **64 feature maps**. Max pooling is applied independently to each feature map:

$$\begin{aligned} A_1 &= \text{MaxPool}(A_1) \\ A_2 &= \text{MaxPool}(A_2) \\ &\vdots \\ A_{64} &= \text{MaxPool}(A_{64}) \end{aligned}$$

It **acts separately on each layer of the depth dimension**. Therefore, for a typical 2×2 pooling operation:

$$32 \times 32 \times 64 \xrightarrow{\text{MaxPool}} 16 \times 16 \times 64$$

The spatial dimensions H and W decrease, but the number of channels remains unchanged.

	Convolution	**Max Pooling**
Mixes input channels?	✔ Yes	✖ No
Can change depth?	✔ Yes	✖ No
Operates spatially?	✔ Yes	✔ Yes

Table 8: Comparison between convolution and max pooling.

There are no trainable parameters in pooling layers

Another crucial distinction is that max pooling does **not learn anything**. For convolution, we had parameters such as:

$$w_1, w_2, \ldots, w_n, b$$

Max pooling simply performs the predefined operation of taking the maximum value in a local region: $\max(\cdot)$. There are no filter weights and no biases to learn. Therefore:

$$\#\text{ trainable parameters in max pooling} = 0$$

The pooling-window size and stride are **hyperparameters**, not trainable parameters.

What pooling is doing conceptually

Max Pooling is a form of **downsampling**. It reduces the spatial size of the feature maps. So it completes the three main functions of a CNN:

$$\underbrace{\text{Convolution}}_{\text{detect features}} \rightarrow \underbrace{\text{ReLU}}_{\text{nonlinearity}} \rightarrow \underbrace{\text{Max Pooling}}_{\text{spatially summarize/compress}}$$

For example:

$$32 \times 32 \times 3 \xrightarrow[\text{32 filters}]{\text{Conv}} 32 \times 32 \times 32 \xrightarrow{\text{ReLU}} 32 \times 32 \times 32 \xrightarrow{\text{MaxPool}} 16 \times 16 \times 32$$

Where:

- **Convolution** determines the new depth through its number of filters.
- **ReLU** changes values but not dimensions.
- **Max pooling** reduces spatial dimensions but does not change depth.

> **Key Takeaways 17**
>
> Max pooling reduces the spatial resolution of CNN feature maps by replacing each local region with its maximum activation. It operates independently on every channel, so it reduces height and width without mixing or changing the number of feature maps. Max pooling contains no trainable parameters: its window size and stride are predefined hyperparameters.

7.8.2 Pooling Size and Stride

In the previous section, we have seen that pooling layers are used to reduce the spatial dimensions of the input feature maps. But, ***how does the pooling window move across the feature map?*** The movement of the pooling window is determined by two parameters: the **pooling size** and the **stride**. Typically, the stride is assumed equal to the pooling size.

Pooling size

The **pooling size** specifies the spatial neighborhood over which we compute the maximum.

For example, with $K = 2 \times 2$, each output activation summarizes a 2×2 region of the input feature map. And as we have seen, this is done **independently for every channel** of the input feature map.

Stride

The **stride** S (page 544) tells us how far the pooling window moves between two consecutive operations. The most common configuration is to set the stride equal to the pooling size, i.e., $S = K$.

For example:

$$K = 2 \times 2, \quad S = 2 \times 2$$

The first 2×2 region is processed, then the window moves 2 **positions**, so the regions do not overlap. If we have one feature map of size 8×8, and apply a 2×2 max pooling with stride 2, along the width, 8 positions are divided into groups of 2:

$$\boxed{1\ 2} \quad \boxed{3\ 4} \quad \boxed{5\ 6} \quad \boxed{7\ 8}$$

From each pair/group, pooling produces one output position. Therefore, the output width is 4. The same happens along the height, so the output height is also 4. The output feature map has size 4×4. Consequently, the **total number of spatial activations** becomes:

$$\text{Reduction Width} \times \text{Reduction Height} = \frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$$

So a 2×2 max pooling with stride 2 **reduces the spatial size to 25%**, or equivalently discards 75% of the spatial samples. However, the number of channels remains the same.

What if stride is 1?

Suppose we have a 2×2 max pooling with stride 1:

$$K = 2 \times 2, \quad S = 1$$

Now the pooling windows **overlap** because the window moves only one position at a time. So we're computing a **local maximum around essentially every**

spatial position of the input feature map (see Figure 71). So, stride 1 can be used when pooling is intended primarily as a **local nonlinear maximum operation**, rather than primarily for spatial reduction.

Example 10: Pooling Size and Stride

Suppose a convolutional filter produced this 4×4 feature map:

$$X = \begin{bmatrix} 1 & 2 & 1 & 0 \\ 3 & \mathbf{9} & 2 & 1 \\ 1 & 2 & 4 & 2 \\ 0 & 1 & 2 & 3 \end{bmatrix}$$

The bold value 9 means that the convolutional filter detected its feature very strongly at that position. Now, we apply a 2×2 max pooling:

- Stride $S = 2$ (non-overlapping), we move the window **two positions at a time**:

$$\begin{bmatrix} 1 & 2 \\ 3 & \mathbf{9} \end{bmatrix} \to 9, \quad \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix} \to 2,$$
$$\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \to 2, \quad \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix} \to 4$$

  Therefore:

$$\begin{bmatrix} 1 & 2 & 1 & 0 \\ 3 & 9 & 2 & 1 \\ 1 & 2 & 4 & 2 \\ 0 & 1 & 2 & 3 \end{bmatrix} \xrightarrow[\text{stride 2}]{\text{2×2 max pool}} \begin{bmatrix} 9 & 2 \\ 2 & 4 \end{bmatrix}$$

  We went from 4×4 to 2×2, discarding 75% of the spatial samples. So here pooling is clearly being used for **downsampling**.

- Stride $S = 1$ (overlapping), we move the window **one position at a time**. First window:

$$\begin{bmatrix} 1 & 2 \\ 3 & \mathbf{9} \end{bmatrix} \to 9$$

  Move one position to the right:

$$\begin{bmatrix} 2 & 1 \\ \mathbf{9} & 2 \end{bmatrix} \to 9$$

  Move one position right again:

$$\begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix} \to 2$$

  Then we repeat the same process for the next rows, moving one position at a time. The final output is:

$$\begin{bmatrix} 1 & 2 & 1 & 0 \\ 3 & 9 & 2 & 1 \\ 1 & 2 & 4 & 2 \\ 0 & 1 & 2 & 3 \end{bmatrix} \xrightarrow[\text{stride 1}]{\text{2×2 max pool}} \begin{bmatrix} 9 & 9 & 2 \\ 9 & 9 & 4 \\ 2 & 4 & 4 \end{bmatrix}$$

Notice something interesting: the maximum value 9 is repeated four times in the output. Because the original 9 belongs to **four different overlapping** 2×2 **windows**, and it is the maximum in all of them.

So why would I ever want this? Imagine that 9 means that the convolutional filter detected its feature very strongly at that position. After stride-1 max pooling, we're saying that the feature is still present in the **local neighborhood** of that position. So stride-1 max pooling is a way to **preserve local maxima** and **suppress weaker activations within each neighborhood**, rather than aggressively downsampling the feature map.

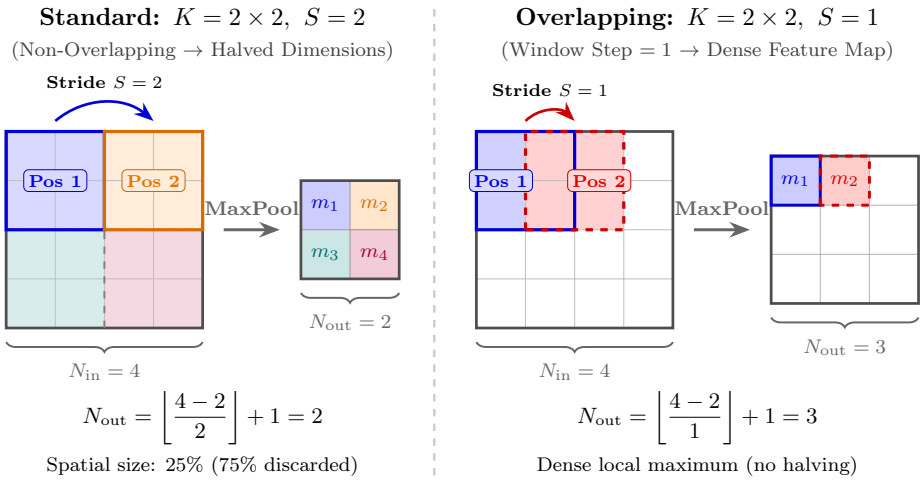


Figure 71: Side-by-side comparison of standard non-overlapping pooling ($S = 2$) and overlapping pooling ($S = 1$).

7.9 Dense Layers and the Complete CNN

Up to this point, everything we have discussed belongs to the part of the CNN that processes **volumes**:

$$\text{Convolution} \rightarrow \text{Activation} \rightarrow \text{Pooling} \rightarrow \dots$$

Eventually, however, we want to perform **classification**. The convolutional part extracts features, and a traditional fully connected neural network uses those features for classification. However, the fully connected part of the network is **not convolutional** and does not process feature volumes. Instead, it processes **vectors**. Therefore, we need to transform the final feature volume into a vector before feeding it into the fully connected layers. This operation is called **flattening**.

7.9.1 Flattening and Dense Layers

Suppose that after several convolution and pooling operations we obtain a volume:

$$X \in \mathbb{R}^{H \times W \times C}$$

For example, a final convolution volume of $16 \times 16 \times 24$. This is still a 3-dimensional volume: for each of the 24 feature maps, we have a 16×16 spatial grid. But a traditional dense layer expects a **vector** as its input. Therefore, we need **flattening** (page 427).

Flattening simply rearranges all the activations of the volume into one long vector:

$$H \times W \times C \xrightarrow{\text{Flatten}} HWC$$

So we could conceptually imagine:

$$\begin{bmatrix} \text{feature map 1} \\ \text{feature map 2} \\ \vdots \\ \text{feature map C} \end{bmatrix} \rightarrow \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{HWC} \end{bmatrix}$$

Nothing is learned during flattening. It simply changes the organization of the activations so that they can be fed to a dense layer.

What does "*the spatial dimension is lost*" mean?

Before flattening, an activation has an explicit position: $x(r, c, k)$. We know that r and c are the row and column of the activation in the feature map, and k is the index of the feature map. **After flattening**, everything is represented as:

$$\mathbf{x} = [x_1, x_2, \dots, x_{HWC}]^T$$

The values themselves have **not disappeared**. What disappears is the explicit **2D spatial structure** of the representation.

Then come the Dense Layers

Once we have the vector, we can use the same dense layers we already know from Feed-Forward Neural Networks:

$$\mathbf{a} = W\mathbf{x} + \mathbf{b}$$

Followed, where appropriate, by activation functions.

The key **difference** from convolution is connectivity. In a **dense layer**, every output neuron is connected to **every input activation**. That's precisely why it is called *dense* or *fully connected*. For example:

$$6144 \to 512 \to 128 \to 10$$

Where they could represent:

$$\underbrace{6144}_{\text{flattened CNN features}} \quad \to \quad \underbrace{512 \to 128}_{\text{dense hidden layers}} \quad \to \quad \underbrace{10}_{\text{class scores}}$$

The final fully connected layer has as many outputs as there are classes and provides a score for each class.

Why not flatten the original image immediately?

This brings us back to the entire motivation for CNNs. If we did:

$$\text{Image} \to \text{Flatten} \to \text{Multi-Layer Perceptron}$$

We would essentially return to a traditional fully connected neural network operating directly on pixels. Instead, we want to use the convolutional part of the network to extract features from the image first. The dense layers then use those features for classification (see the architecture on page 507).

> **Key Takeaways 18**
>
> **Flattening is the bridge between feature extraction and classification**. The convolutional part produces a feature volume $H \times W \times C$; flattening rearranges it into an HWC-dimensional vector, losing its explicit spatial organization but not its activation values. This vector can then be processed by traditional dense/fully connected layers, where every output neuron is connected to every input activation.

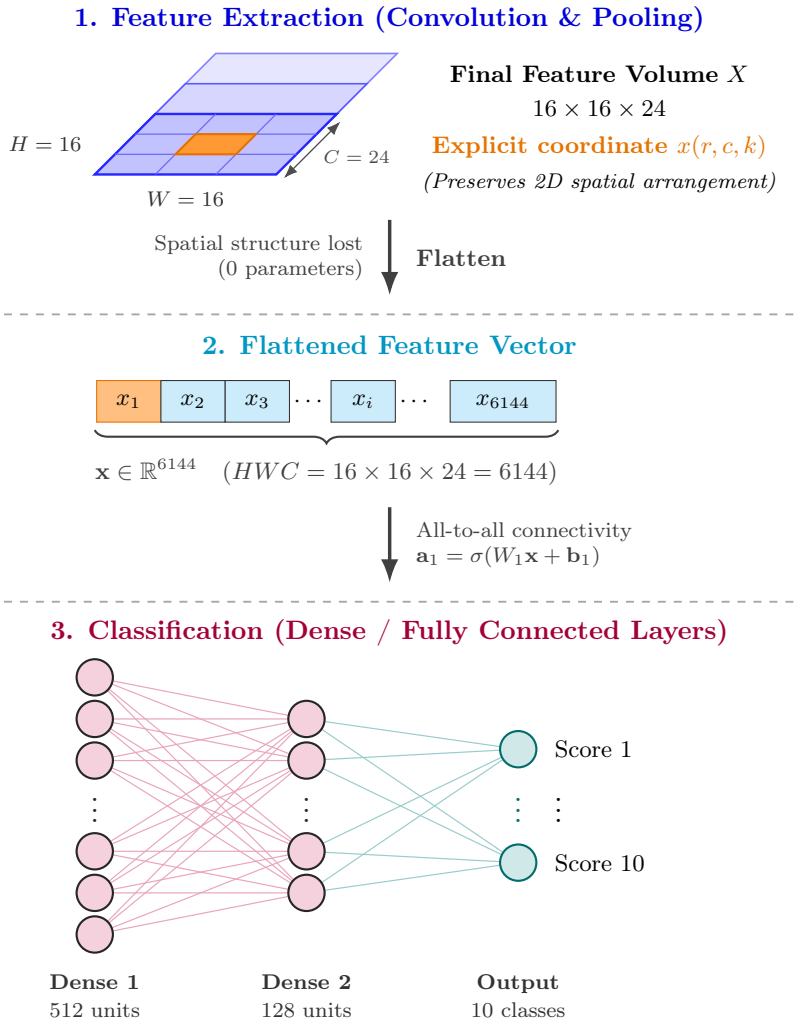


Figure 72: Progression from 3D convolutional feature extraction ($16 \times 16 \times 24$) to dense classification. Flattening converts the spatial volume into a 1D feature vector of 6144 elements, which is sequentially processed by dense layers ($512 \to 128 \to 10$) using all-to-all connectivity.

7.9.2 Following Shapes Through a CNN

This section puts together everything we have studied so far. We take the CNN architecture introduced on page 507 and now explain what happens to the data **layer by layer**.

First of all, the central idea is:

$$\text{image} \to \text{sequence of feature volumes} \to \text{feature vector} \to \text{class scores}$$

Let's follow the architecture of a CNN, layer by layer, and see how the data changes shape.

1. **Input image**. We start from an image:
$$X \in \mathbb{R}^{H \times W \times C}$$
For an RGB image: $C = 3$. At this point, we **still have the original spatial structure** of the image.

2. **First convolution**. Suppose that the first convolutional layer has 8 **filters**. Remember the rule:
$$C_{\text{out}} = \text{number of filters}$$
Therefore:
$$H \times W \times C \xrightarrow{\text{Conv. 8 filters}} H' \times W' \times 8$$
The precise H' and W' depend on kernel size, padding and stride. Here we can assume **zero padding** (page 542), so the convolution preserves the spatial dimensions of the input. Therefore, we have:
$$H \times W \times C \xrightarrow{\text{Conv. 8 filters}} H \times W \times 8$$
Each of those 8 channels is the activation map produced by one learned filter.

3. **Activation**. We then apply the nonlinear activation function, such as ReLU (page 547):
$$a \mapsto \max(0, a)$$
As we established on page 547, the activation function operates **element by element** on the feature volume. Therefore, it does not change the shape of the data: it changes the **values** of the feature volume, but not its shape. Therefore, we still have:
$$H \times W \times 8 \xrightarrow{\text{ReLU}} H \times W \times 8$$

4. **Subsampling / Max Pooling**. Then we perform pooling (page 553). It **halves the spatial dimensions** of the feature volume, assuming the usual 2×2 pooling with stride 2. Therefore, we have:
$$H \times W \times 8 \xrightarrow{\text{Max Pooling}} \frac{H}{2} \times \frac{W}{2} \times 8$$
Notice that the number of channels stays 8, because pooling operates independently on each feature map. So:
$$\text{Pooling changes } H, W \text{ but not } C$$

5. **Another convolutional block**. Now we can repeat the **same pattern**:

$$\text{Conv} \rightarrow \text{ReLU} \ / \ \text{Leaky ReLU} \rightarrow \text{Max Pooling}$$

The second convolution could have, for example, 24 filters. So its output depth becomes $C = 24$.

$$32 \times 32 \times 8 \xrightarrow{\text{Conv. 24 filters}} 32 \times 32 \times 24$$

Followed by ReLU:

$$32 \times 32 \times 24 \xrightarrow{\text{ReLU}} 32 \times 32 \times 24$$

And then max pooling:

$$32 \times 32 \times 24 \xrightarrow{\text{Max Pooling}} 16 \times 16 \times 24$$

This pattern follows the same tendency explained on page 510:

$$\text{deeper CNN} \quad \Rightarrow \quad \begin{cases} H, W & \text{generally decrease} \\ C & \text{generally increases} \end{cases}$$

6. **Flattening: volume → vector**. The convolutional part is still producing something like:

$$16 \times 16 \times 24$$

Dense layers need a vector as input, so we need to **flatten** the feature volume into a one-dimensional vector (page 559):

$$16 \times 16 \times 24 \xrightarrow{\text{Flatten}} 16 \cdot 16 \cdot 24 = 6144$$

Thus:

$$16 \times 16 \times 24 \xrightarrow{\text{Flatten}} \mathbf{x} \in \mathbb{R}^{6144}$$

Flattening changes only the organization of the activations and has no trainable parameters.

7. **Dense classification**. Now we have left the convolutional feature volumes behind and we are in the dense classification part of the CNN. The vector is processed by traditional fully connected layers:

$$6144 \rightarrow N_1 \rightarrow N_2 \rightarrow \cdots \rightarrow N_{\text{classes}}$$

The final layer has **one output per class** and provides a score for each possible class.

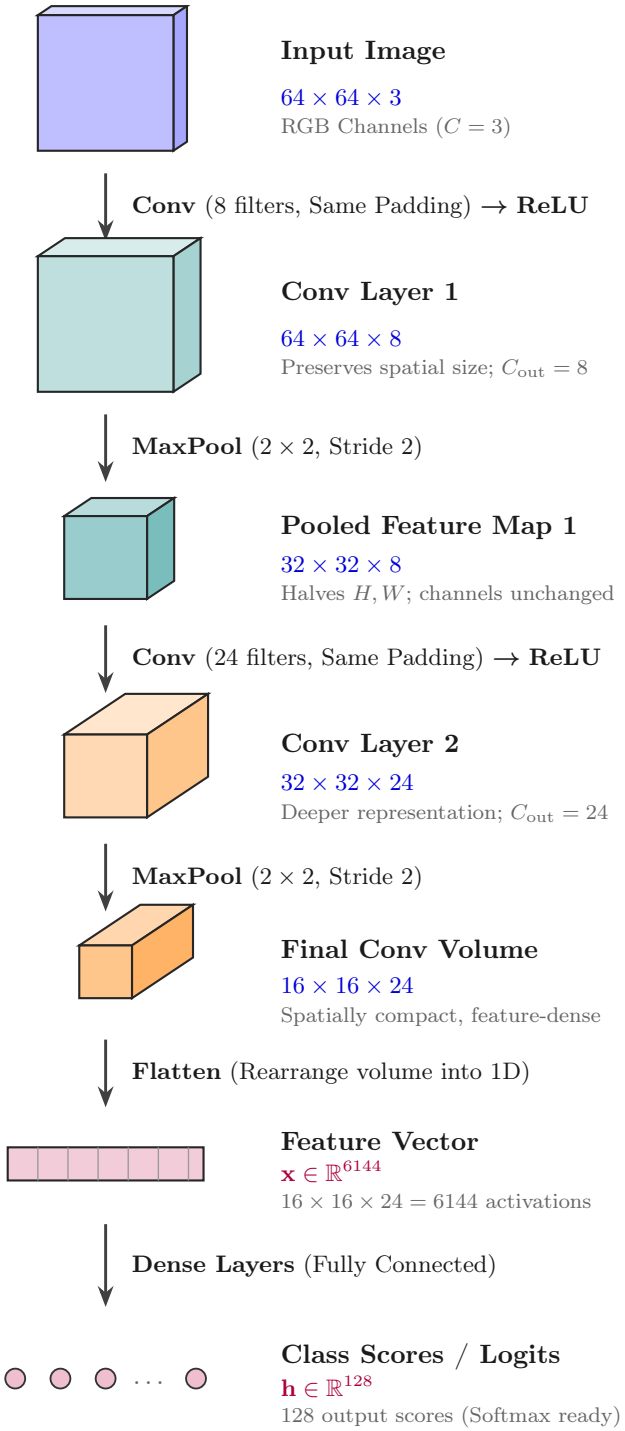


Figure 73: **Step-by-step shape transformation across a Convolutional Neural Network.** The spatial dimensions (H, W) shrink via Max Pooling while channel depth (C) increases through learned convolutions, culminating in a 1D flattened feature vector for dense classification.

7.9.3 Feature Extraction Network and Classifier

Now that we have a good understanding of convolutional and pooling layers, we can interpret the entire CNN as two systems that work together rather than as a long sequence of layers:

$$\text{CNN} = \text{Feature Extraction Network (FEN)} + \text{Feature Classifier}$$

The **Feature Extraction Network (FEN)** is the **convolutional block**: convolution, activation, and pooling layers progressively transform raw pixels into a learned representation. The **classifier** is the fully connected part that takes that learned representation and maps it to class scores or probabilities.

The Feature Extraction Network

The FEN is the part before the dense classifier:

$$\text{Image} \to \text{Conv} \to \text{Activation} \to \text{Pooling} \to \text{Conv} \to \text{Activation} \to \cdots$$

Its **goal** is not directly to say: "*this image is a cat*". Instead, it transforms the raw image into a **useful set of learned features**. So conceptually:

$$I \longrightarrow \text{FEN}(I)$$

Where I is the input image and $\text{FEN}(I)$ is the learned representation of the image. We will call this learned representation the **latent representation** or **data-driven feature vector** of the image, because it is a vector of features that the network has learned to extract from the image.

What is the Latent Representation?

Suppose the convolutional part ends with:

$$16 \times 16 \times 24$$

After flattening, the **latent representation is a vector** of size:

$$16 \cdot 16 \cdot 24 = 6144$$

That resulting feature vector can be thought of as:

$$\mathbf{z} = \text{FEN}(I)$$

Where $\mathbf{z}$ is a vector of size 6144 that contains the network's learned representation of the input image I. The important idea is that $\mathbf{z}$ is **not raw pixels anymore**. It contains features learned by the CNN to be useful for the classification task.

What does "*latent*" mean? In machine learning, latent means hidden/internal: something that is not directly observed in the original data. We observed data is the image:

$$I = \text{raw pixels}$$

But inside the CNN, the network transforms those pixels into another representation:

$$I \xrightarrow{\text{FEN}} \mathbf{z} = \text{FEN}(I)$$

The vector $\mathbf{z}$ is **not provided to the network by us**. It is an internal representation that emerges from the learned transformations. That's why we call it *latent*. In other words, the latent representation is the learned internal representation of the input image.

Furthermore, all intermediate activations are also latent representations. But in this case, we are particularly interested in the representation produced by the **Feature Extraction Network (FEN)** and passed to the classifier.

Then the classifier uses those features: Feature Classifier

Once we have the latent vector $\mathbf{z}$, the dense part acts like a conventional Multi-Layer Perceptron (MLP) classifier:

$$\mathbf{z} \to \text{Dense} \to \text{Dense} \to \text{Softmax} \to \text{Class Probabilities}$$

So FEN asks "*what useful features are present?*", while the classifier asks "*given these features, which class is it?*".

Why is this better than the old hand-crafted pipeline? At the beginning, the pipeline for image classification was:

$$\text{Image} \longrightarrow \underbrace{\text{Hand-crafted Extractor}}_{\text{Fixed / Expert Designed}} \longrightarrow \underbrace{\text{Classifier}}_{\text{Data-Driven}}$$

Now it becomes:

$$\text{Image} \longrightarrow \underbrace{\text{learned feature extractor}}_{\text{data driven}} \longrightarrow \underbrace{\text{learned classifier}}_{\text{data driven}}$$

The key difference is that the feature extractor is now **learned from data**, rather than being hand-crafted by experts. It also allows for **end-to-end training**, where the feature extractor and classifier are trained together to minimize the classification loss.

What does end-to-end training mean? Suppose the final prediction is wrong. The loss is computed at the classifier output:

$$L(\hat{y}, y)$$

Backpropagation does **not stop at the dense classifier**. Instead, the gradient is propagated back through the entire network, including the FEN. This means that the FEN is updated to produce better features for the classifier, leading to a more effective overall model. So the classifier effectively tells the feature extractor: "*the features you are producing are not good enough for this classification task; change them*". That is what makes the features **task-dependent and data-driven**.

Formally, if:

$$\hat{y} = C\left(\text{FEN}(I)\right)$$

Where $\hat{y}$ is the predicted class probabilities, C is the classifier, and $\text{FEN}(I)$ is the latent representation of the input image I. The training adjusts the parameters of **both** the FEN and the classifier C to minimize the loss $L(\hat{y}, y)$, where y is the true label.

Why does this decomposition matter? This is not just a conceptual trick. Later, it becomes extremely useful for **transfer learning**, where we can take a pretrained FEN and use it as a fixed feature extractor for a new classification task, only training a new classifier on top of it. This allows us to leverage learned features from large datasets and apply them to smaller datasets.

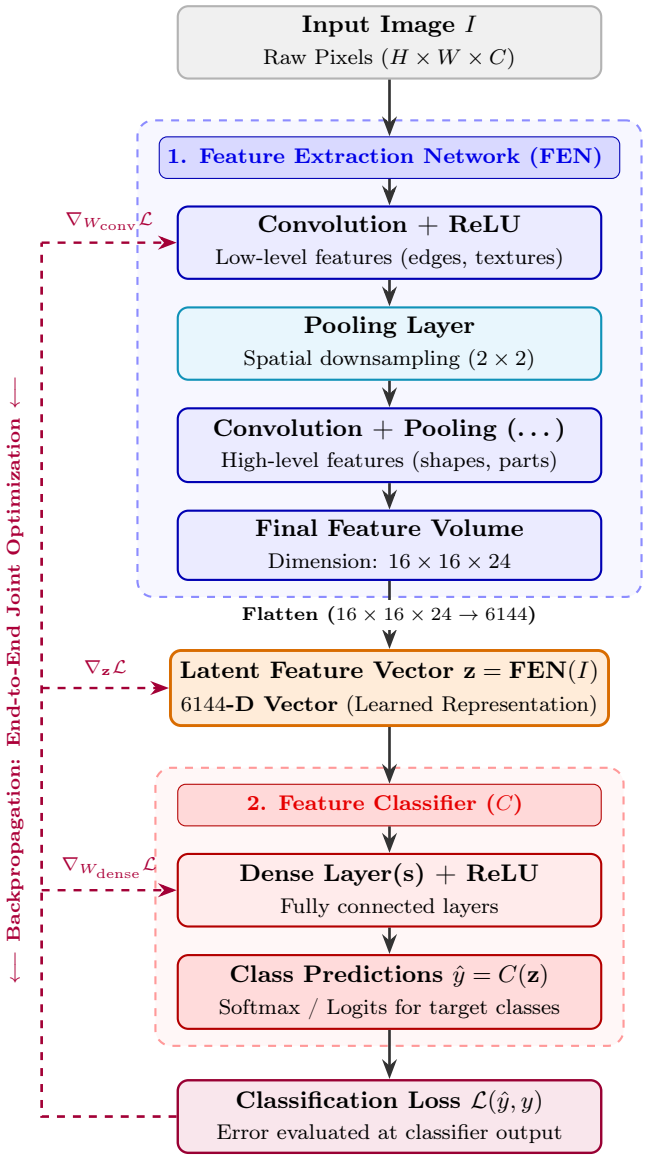


Figure 74: Decomposition of a CNN into a Feature Extraction Network (FEN) and a Classifier, linked by the learned latent vector $\mathbf{z}$. End-to-end backpropagation jointly optimizes all parameters directly from data. The ∇ symbols indicate the gradients computed during backpropagation for each component:

- $\nabla_{W_{\text{conv}}}\mathcal{L}$ represents the **gradient of the loss** with respect to the convolutional weights, which is used to update the FEN during training.
- $\nabla_{\mathbf{z}}\mathcal{L}$ term indicates **how the latent representation $\mathbf{z}$ is adjusted** to improve classification performance.
- $\nabla_{W_{\text{dense}}}\mathcal{L}$ shows the **gradient for the dense classifier** weights.

7.10 CNNs in Action

Up to now, we have talked about filters and feature maps abstractly. Here, we actually visualize the intermediate activations produced by a convolutional network and show that different channels contain different kinds of image information.

Suppose a convolutional layer outputs a volume of size $H \times W \times C$. That means it contains C separate feature maps: $A_1, A_2, \ldots, A_C$. Each channel A_k is a two-dimensional matrix of activations:

$$A_k \in \mathbb{R}^{H \times W}, \quad k = 1, 2, \ldots, C$$

So we can visualize it as a **gray-scale image**. A bright location represents a high activation, meaning that the feature associated with that channel responds strongly at that spatial position. Dark regions correspond to weak or zero activations.

For example, consider the activations produced by a pretrained Convolutional Neural Network called VGG16 [14] for the Lena image:

$$\begin{aligned}
&\texttt{block1_conv2} : 224 \times 224 \times 64 \\
&\texttt{block2_conv2} : 112 \times 112 \times 128 \\
&\texttt{block3_conv3} : 56 \times 56 \times 256 \\
&\texttt{block4_conv3} : 28 \times 28 \times 512 \\
&\texttt{block5_conv3} : 14 \times 14 \times 512
\end{aligned}$$

For example, `block1_conv2` does **not** produce one single feature map (image) of size 224×224, but rather a **volume** containing **64 separate feature maps**, each of size 224×224:

$$224 \times 224 \times 64 \quad \Rightarrow \quad \underbrace{64}_{\text{channels}} \text{ feature maps of size } 224 \times 224$$

Early layers: low-level features. Look at the activations produced by the first convolutional layer of VGG16 for the Lena image in Figure 75. We can still very clearly recognize Lena in almost every channel. But notice that **different channels emphasize different properties**. For example, some channels highlight the edges of Lena's face, while others highlight the background. So, every channel is the response of a different learned feature detector to the same input. Conceptually:

$$\text{Lena} \xrightarrow{\text{different learned filters}} \begin{cases} A_1 & \text{responds to one kind of pattern} \\ A_2 & \text{responds to another kind of pattern} \\ \vdots & \\ A_C & \text{responds to yet another kind of pattern} \end{cases}$$

This is why the images look related but different.



Figure 75: Activations produced by the first convolutional layer of VGG16 for the Lena image.

Middle layers: combinations of features. Then look at the activations produced by the second convolutional layer of VGG16 for the Lena image in Figure 76, and especially the third convolutional layer in Figure 77. Something interesting happens: Lena becomes **progressively harder to recognize**.

In Figure 76, many channels still look like edges and textures, but in Figure 77, the channels look more like abstract patterns. This is intentional because deeper layers are no longer asking only questions such as "*is there an edge here?*"; they can respond to **combinations of simpler features** detected in previous layers.

Notice also that a filter in block 3 is **not operating directly on Lena's RGB pixels anymore**. It receives the feature maps generated by the preceding network layers as input.

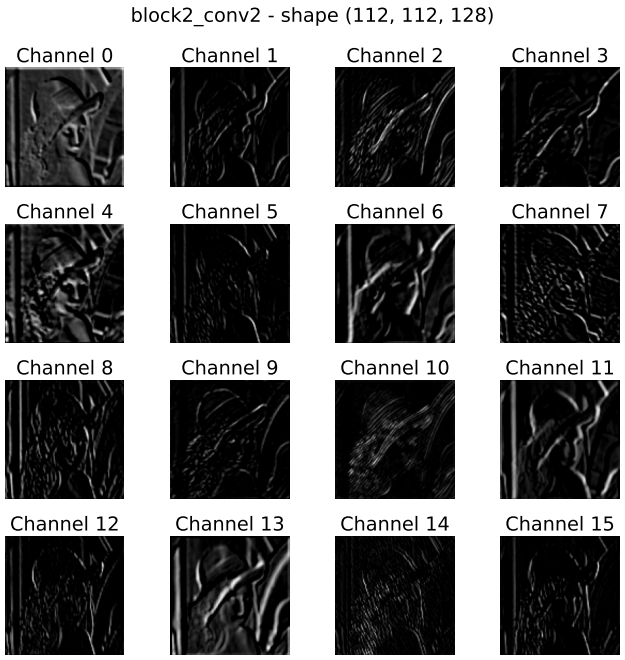


Figure 76: Activations produced by the second convolutional layer of VGG16 for the Lena image.

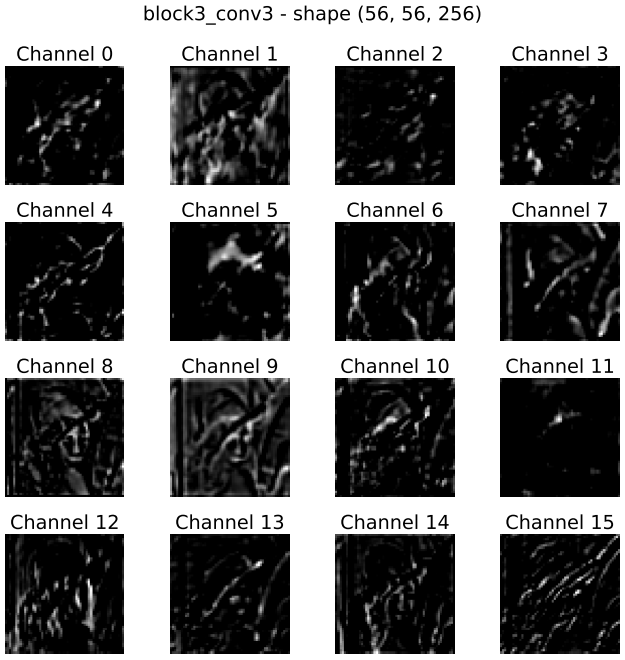


Figure 77: Activations produced by the third convolutional layer of VGG16 for the Lena image.

Deep layers: sparse, abstract activations. Finally, look at the activations produced by the fourth convolutional layer of VGG16 for the Lena image in Figure 78 and the fifth convolutional layer in Figure 79.

Many channels are almost completely black, with only a few strongly activated regions. That does **not** mean that the CNN has progressively "*lost*" the image in a bad way. Rather, its representation has changed from something like "*what are the pixel values?*" toward something like "*which learned high-level patterns are present, and approximately where?*".

For example, suppose some hypothetical deep feature A_{137} has learned to respond strongly to a particular complex visual pattern. Then only locations containing something similar to that learned pattern produce large activations:

$$A_{137}(r,c) = \begin{cases} \text{large} & \text{feature strongly present around location } (r,c) \\ \text{small / 0} & \text{otherwise} \end{cases}$$

This explains those little bright blobs in the Figure 79.

One caution about interpretation. We should **not look at one of those blobs and claim**: "*Ah, that blob is the nose!*" or "*That blob is the eye!*". Because the visualization alone doesn't establish exactly what semantic concept a particular channel represents. What we *can* safely say is that deeper channels respond selectively to increasingly complex patterns learned during training.

Key Takeaways 19: Hierarchical feature learning

Each channel of a convolutional volume is a separate feature map and can be visualized as a grayscale image. Bright locations indicate where that learned feature produces a strong activation.

Early CNN layers preserve fine spatial information and typically respond to simple visual patterns such as edges and textures. Deeper layers combine features produced by previous layers, producing increasingly complex and abstract representations. Consequently, the original image becomes progressively less recognizable when individual deep feature maps are visualized.

Through the network, spatial resolution generally decreases while the number of feature channels increases:

$$H, W \downarrow \quad \text{and} \quad C \uparrow$$

Thus, CNNs progressively transform raw pixels into a collection of learned, increasingly abstract visual features.

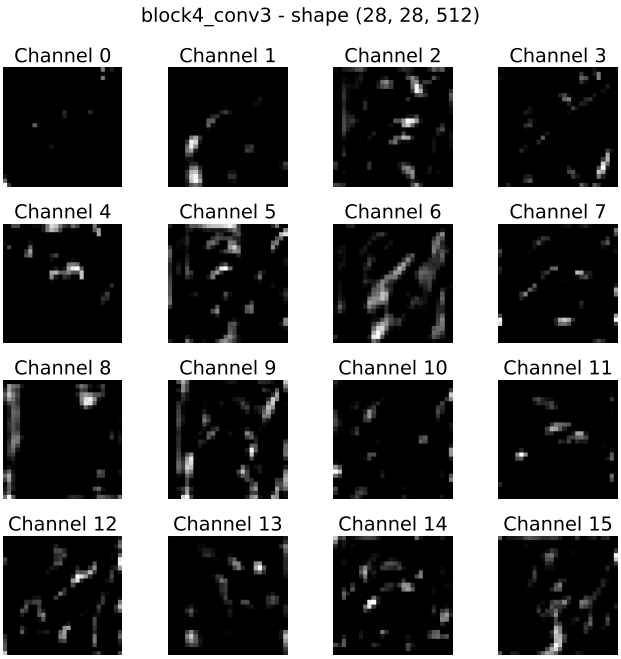


Figure 78: Activations produced by the fourth convolutional layer of VGG16 for the Lena image.

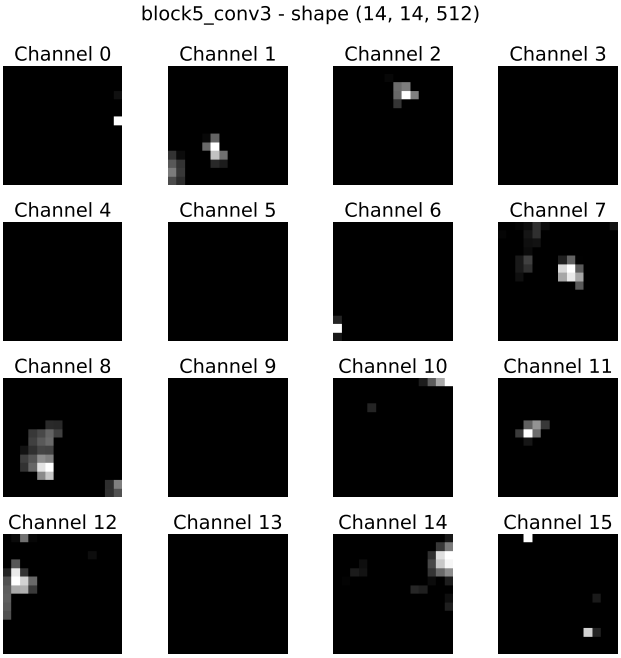


Figure 79: Activations produced by the fifth convolutional layer of VGG16 for the Lena image.

7.11 LeNet-5: The First Famous CNN

7.11.1 Historical Motivation

Let's move on the first concerete CNN architecture: LeNet-5, which was proposed by *Yann LeCun* in 1998 [9]. It is one of the first successful convolutional neural networks. It was designed for **handwritten-digit recognition**: given a small grayscale image, the network predicts one of the ten digit classes.

⚠ The problem: handwritten-digit recognition

LeNet-5 was introduced by *Yann LeCun* and collaborators for **handwritten digit recognition**. The input consists of very small **grayscale images**, and the goal is straightforward:

$$\text{image of a digit} \rightarrow \text{digit class (0, 1, 2, 3, 4, 5, 6, 7, 8, or 9)}$$

For example, an image containing a handwritten digit '3' should be classified as the digit class '3'. The recognition of single handwritten digits can then be used to recognize complete handwritten numbers as a **sequence of digits** (e.g., a handwritten number '123' can be recognized as the sequence of digits '1', '2', and '3').

So, conceptually, this is exactly the image-classification problem we have been discussing in the previous sections:

$$I \in \mathbb{R}^{H \times W \times 1} \rightarrow \text{CNN} \rightarrow 10 \text{ classes (0, 1, 2, 3, 4, 5, 6, 7, 8, or 9)}$$

The 1 in the input shape indicates that the input images are **gray-scale** (single channel).

❓ Why not simply use an Multi-Layer Perceptron (MLP)? We already know that this is possible, but it is **inefficient**. We could flatten the image immediately: $H \times W \rightarrow HW$, and feed every pixel directly into an MLP. Technically, **this works**, there is nothing mathematically preventing us from doing it. But we should **not** treat every pixel as an independent input feature because images are **highly spatially correlated**.

✅ Why is this more efficient? Imagine connecting the $32 \times 32 = 1024$ pixels directly to just 84 dense neurons: $1024 \times 84 = 86'016$ weights already. And every neuron learns completely separate weights for every pixel. For now, it is sufficient to know that the first convolutional layer of the LeNet-5 architecture contains six 5×5 filters. Since the image is gray-scale $C_{\text{in}} = 1$ and thus:

$$\#\text{ params} = 6\,(5 \cdot 5 \cdot 1 + 1) = 156$$

Those 156 parameters are enough to apply six learned feature detectors **everywhere in the image**. Thus:

- MLP: different weights for different pixel connections.
- CNN: small local filters reused across spatial positions.

7.11.2 LeNet-5 Architecture

The main idea of the architecture is (see also Figure 80, page 578):

$$\begin{aligned}\text{Image} &\to \text{Conv} \to \text{Non-linearity} \to \text{AvgPool} \\ &\to \text{Conv} \to \text{Non-linearity} \to \text{AvgPool} \\ &\to \text{Flatten} \to \text{Multi-Layer Perceptron (MLP)} \to \text{Softmax}\end{aligned}$$

LeNet-5 receives a **small grayscale image** of a handwritten digit as input. Since it is grayscale, the input initially has only **one channel**. The first part of the network is the **feature extractor**:

$$\text{Image} \to \boxed{\text{Conv} \to \text{Activation} \to \text{Pooling}} \to \boxed{\text{Conv} \to \text{Activation} \to \text{Pooling}}$$

The second part is the **classifier**:

$$\text{Feature Volume} \to \text{Flatten} \to \boxed{\text{Dense layers}} \to \boxed{\text{Softmax}}$$

So the idea introduced on page 559 becomes concrete in the LeNet-5 architecture:

$$\text{Image} \to \underbrace{\text{Learned Feature Extraction}}_{\text{CNN}} \to \underbrace{\text{Classification}}_{\text{MLP}} \to \text{Digit}$$

Where the feature extraction is done by a **convolutional neural network** (CNN) and the classification is done by a **multi-layer perceptron** (MLP).

Convolutional layers: learn *what* to look for

The convolutional layers contain **learnable filters**.

The *first* convolution operates directly on the grayscale pixels. Its filters can learn relatively simple local structures such as edges, strokes and curves.

After several transformations, the *second* convolution works on the **feature maps produced by the previous layer**, rather than directly on the original image.

Therefore, we can think:

$$\text{Pixels} \to \text{Simple Local Features} \to \text{More Complex Combinations of Features}$$

This connects directly with the idea from page 569: deeper activations represent progressively more abstract information (e.g., page 573).

Nonlinearities: make the network actually nonlinear

After convolution, LeNet applies an activation function. As we know, this is crucial because convolution itself is a linear operation. Without nonlinearities, the network would still collapse mathematically into one linear transformation, no matter how many layers we stack. The original LeNet family is associated with **tanh-like nonlinearities**. So, convolution extracts features, and activation introduces nonlinearity to the network.

Average pooling: progressively reduce spatial resolution

This is an interesting historical difference from many modern CNNs, where **max pooling** is more common. LeNet-5 uses **average pooling**, which conceptually computes a local average of the feature map:

$$\begin{bmatrix} x_1 & x_2 \\ x_3 & x_4 \end{bmatrix} \to \frac{x_1 + x_2 + x_3 + x_4}{4}$$

As max pooling, average pooling reduces the spatial dimensions while keeping the number of channels constant. In other words, it does **spatial downsampling**, which is a form of **dimensionality reduction**.

The characteristic CNN shape evolution

This architecture gives us a concrete example of the **typical CNN shape evolution** discussed on page 510. Indeed, the spatial dimensions of the feature maps decrease while the number of channels increases:

$$\begin{aligned} 1@32 \times 32 &\to \text{Conv}(5 \times 5,\ 6) \to 6@28 \times 28 \to \text{AvgPool}(2 \times 2) \to 6@14 \times 14 \\ &\to \text{Conv}(5 \times 5,\ 16) \to 16@10 \times 10 \to \text{AvgPool}(2 \times 2) \to \boxed{16@5 \times 5} \end{aligned}$$

The last volume contains **16 feature maps of size** 5×5, which is a total of $16 \cdot 5 \cdot 5 = 400$ activations.

Flatten: transition from CNN to MLP

Now comes the *boundary* between **feature extraction** and **classification**. The volume:

$$5 \times 5 \times 16$$

Is flattened into:

$$\mathbf{z} \in \mathbb{R}^{400}$$

This 400-dimensional vector is a **learned representation of the input image**. Then LeNet stops behaving spatially like a CNN and starts behaving like a conventional MLP: $400 \to 120 \to 84 \to 10$.

Dense layers: combine the extracted features

The convolutional part answers something roughly like: "*which useful visual patterns are present?*". Now, the dense classifier combines all of those learned features to answer: "***which digit is most compatible with this combination of features?***". For example, the CNN might internally represent evidence for curves, vertical strokes, loops, etc. The dense layers learn how combinations of those features correspond to 0, 1, . . . , 9.

Softmax: turn the final scores into class probabilities

There are ten possible digits, so the final dense layer has **10 outputs**:

$$z_0, z_1, \ldots, z_9$$

Softmax converts these scores into probabilities:

$$P\left(y = i \mid x\right) = \frac{e^{z_i}}{\sum_{j=0}^{9} e^{z_j}}$$

Therefore, the output of the network is a **probability distribution over the ten classes**, which sums to 1:

$$\sum_{i=0}^{9} P\left(y = i \mid x\right) = 1$$

For example, if the network outputs:

$$\mathbf{p} = [0.01, 0.02, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, \mathbf{0.90}, 0.01]$$

We can interpret this as: "*the network is* 90% *confident that the input image is a* 8.".

Key Takeaways 20

LeNet-5 combines the main building blocks of a CNN into a complete image classifier:

Image → Conv → Non-linearity → AvgPool
→ Conv → Non-linearity → AvgPool
→ Flatten → Multi-Layer Perceptron (MLP) → Softmax

The convolutional part learns spatial features, while the dense part uses the resulting representation to classify the input digit.

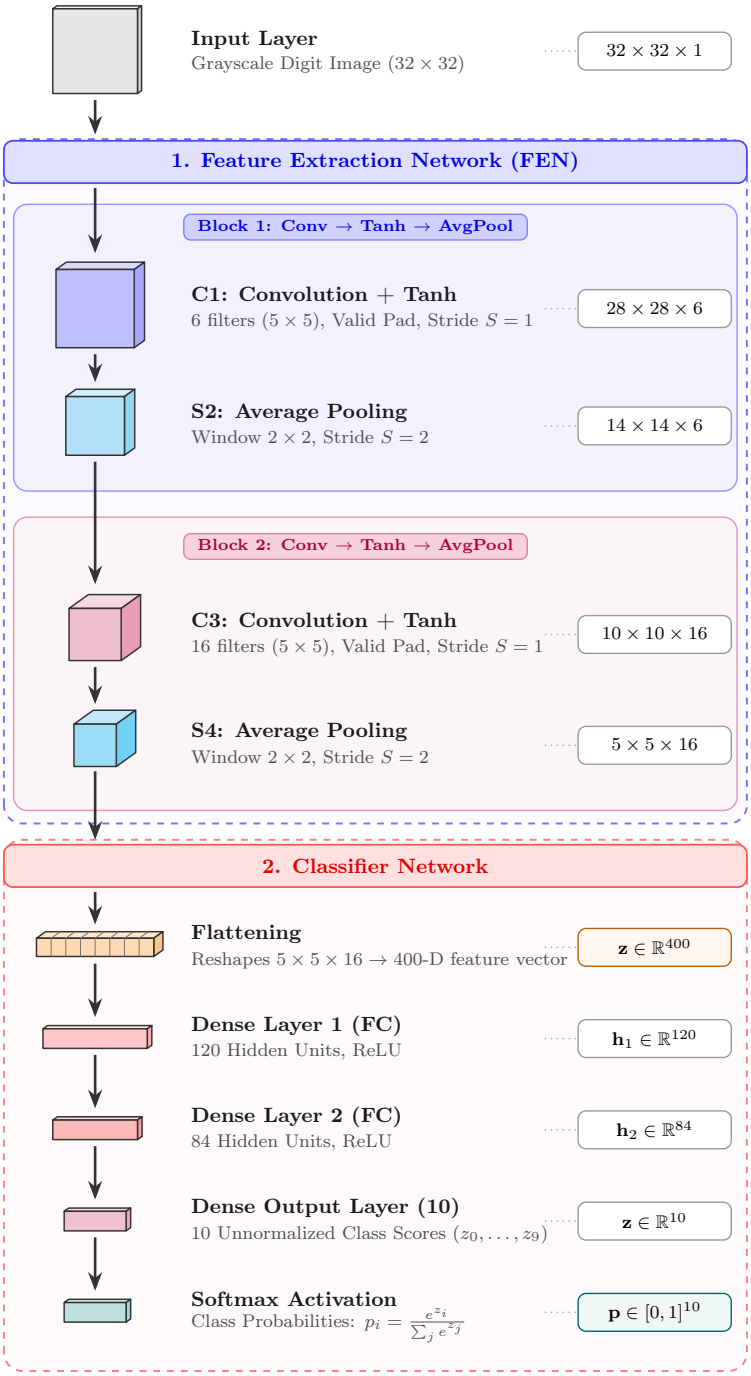


Figure 80: **LeNet-5 architecture.** The network consists of a convolutional feature extractor followed by a fully connected classifier. Two blocks of 5×5 convolution, nonlinear activation, and 2×2 average pooling progressively transform the 32×32 grayscale input into a $5 \times 5 \times 16$ feature volume. This representation is flattened into a 400-dimensional vector and processed by dense layers, with a final softmax producing probabilities over the 10 digit classes.

7.11.3 LeNet-5 Implementation

In this section, we will take the architecture we have just described and asks a very practical question: ***given each layer, what is its output shape and how many trainable parameters does it have?*** This is the main implementation on Keras[22] of the LeNet-5 architecture:

```
from keras.models import Sequential
from keras.layers import (
    Dense, Flatten, Conv2D, AveragePooling2D
)

num_classes = 10
input_shape = (32, 32, 1)

model = Sequential()

model.add(Conv2D(
    filters=6,
    kernel_size=(5, 5),
    activation='tanh',
    input_shape=input_shape,
    padding='valid'
))

model.add(AveragePooling2D(pool_size=(2, 2)))

model.add(Conv2D(
    filters=16,
    kernel_size=(5, 5),
    activation='tanh',
    padding='valid'
))

model.add(AveragePooling2D(pool_size=(2, 2)))

model.add(Flatten())

model.add(Dense(120, activation='relu'))
model.add(Dense(84, activation='relu'))
model.add(Dense(num_classes, activation='softmax'))
```

Let's analyze **each number**.

1. **Input**. We start from: $32 \times 32 \times 1$. Where 1 is the number of channels (**grayscale** image). Thus:

   $$H = 32, \quad W = 32, \quad C_{\text{in}} = 1$$

   And the code is a tuple:

   ```
   input_shape = (32, 32, 1)
   ```

[22] **Keras** is a high-level deep learning API for building and training neural networks, providing simple abstractions for defining layers, models, and training procedures.

There are obviously **no trainable parameters** in the input layer.

2. **First convolution:** $32 \times 32 \times 1 \to 28 \times 28 \times 6$. The first layer is declared as:

```
model.add(Conv2D(
    filters=6,
    kernel_size=(5, 5),
    activation='tanh',
    input_shape=input_shape,
    padding='valid'
))
```

So we have:

- Input channels $C_{\text{in}} = 1$;
- Six filters (`filters = 6`);
- Each filter is spatially 5×5 (`kernel_size = (5, 5)`);
- The activation function is `tanh`;
- The stride is $S = 1$ (default);
- The padding is `valid` (page 540), so no padding is applied $P = 0$.

The output spatial dimensions is computed using the Equation 179 (page 546):

$$N_{\text{out}} = \left\lfloor \frac{N_{\text{in}} + 2P - K}{S} \right\rfloor + 1 = \frac{32 + 0 - 5}{1} + 1 = 28$$

Each of the **6 filters produces one feature map**, so the output shape is:

$$32 \times 32 \times 1 \xrightarrow{\texttt{Conv2D}} 28 \times 28 \times 6$$

Instead, how many trainable parameters does this layer have? Using the Equation 176 (page 530), we have:

$$\begin{aligned} \#\text{ trainable parameters} &= C_{\text{out}} \left(K_h K_w C_{\text{in}} + 1\right) \\ &= 6 \left(5 \cdot 5 \cdot 1 + 1\right) \\ &= 6 \left(25 + 1\right) \\ &= 6 \cdot 26 \\ &= 156 \end{aligned}$$

3. **First average pooling:** $28 \times 28 \times 6 \to 14 \times 14 \times 6$. The first pooling layer is declared as:

```
model.add(AveragePooling2D(pool_size=(2, 2)))
```

With a 2×2 window and the corresponding downsampling:

$$28 \times 28 \times 6 \xrightarrow{\texttt{AveragePooling2D}} 14 \times 14 \times 6$$

Pooling operates **independently on each channel** (page 556), so it does **not change the depth** (number of channels). Thus, the output has the same number of channels as the input, $C_{\text{out}} = C_{\text{in}} = 6$. The number of trainable parameters is **zero** because pooling layers do not have any learnable parameters.

4. **Second convolution: the important one**. Now we have $14 \times 14 \times 6$. The second convolution is:

```
model.add(Conv2D(
    filters=16,
    kernel_size=(5, 5),
    activation='tanh',
    padding='valid'
))
```

Differently from the first convolution, we do not specify the `input_shape` because Keras can infer it from the previous layer. Also, here a CNN filter spans the complete depth of its input. The input depth is now $C_{\text{in}} = 6$, so each of the 16 filters has a shape of $5 \times 5 \times 6$.

The output spatial dimensions is:

$$N_{\text{out}} = \left\lfloor \frac{N_{\text{in}} + 2P - K}{S} \right\rfloor + 1 = \frac{14 + 0 - 5}{1} + 1 = 10$$

And because there are 16 filters, the output shape is:

$$14 \times 14 \times 6 \xrightarrow{\texttt{Conv2D}} 10 \times 10 \times 16$$

Every one of those 16 filters sees **all 6 input channels**.

The number of trainable parameters is:

$$\begin{aligned} \#\text{ trainable parameters} &= C_{\text{out}} \left(K_h K_w C_{\text{in}} + 1\right) \\ &= 16 \left(5 \cdot 5 \cdot 6 + 1\right) \\ &= 16 \left(150 + 1\right) \\ &= 16 \cdot 151 \\ &= 2416 \end{aligned}$$

5. **Second average pooling**. The second pooling layer is declared as:

```
model.add(AveragePooling2D(pool_size=(2, 2)))
```

With a 2×2 window and the corresponding downsampling:

$$10 \times 10 \times 16 \xrightarrow{\texttt{AveragePooling2D}} 5 \times 5 \times 16$$

Again, the number of channels remains the same, $C_{\text{out}} = C_{\text{in}} = 16$, and the number of trainable parameters is **zero**. At this point, the convolutional feature extractor has finished.

6. **Flattening:** ***where does 400 come from?*** We now have the volume $5 \times 5 \times 16$. Flatten simply rearranges all its activations into one vector:

$$5 \cdot 5 \cdot 16 = 400, \quad \mathbf{z} \in \mathbb{R}^{400}$$

Obviously, the number of trainable parameters is **zero** because flattening does not have any learnable parameters. This is exactly the transition from the CNN feature extractor to the MLP classifier.

7. **First dense layer:** $400 \to 120$. The first dense layer is declared as:

```
model.add(Dense(120, activation='relu'))
```

Now we are back to ordinary Neural-Network arithmetic. Every one of the 120 neurons receives all 400 inputs. So the **weights** are:

$$400 \times 120 = 48'000$$

And there are 120 **biases**, one for each neuron:

$$48'000 + 120 = 48'120$$

Thus:

$$\mathbb{R}^{400} \xrightarrow{\text{Dense}} \mathbb{R}^{120}$$

Notice that number of weights is **much larger** than the number of weights in the convolutional layers (156 and $2'416$). The first dense layer alone contains vastly more parameters than both convolutional layers combined.

8. **Second dense layer:** $120 \to 84$. The second dense layer is declared as:

```
model.add(Dense(84, activation='relu'))
```

Again, every one of the 84 neurons receives all 120 inputs. So the **weights** are:

$$120 \times 84 = 10'080$$

And there are 84 **biases**, one for each neuron:

$$10'080 + 84 = 10'164$$

Thus:

$$\mathbb{R}^{120} \xrightarrow{\text{Dense}} \mathbb{R}^{84}$$

9. **Output layer:** $84 \to 10$. The output layer is declared as:

```
model.add(Dense(num_classes, activation='softmax'))
```

There are 10 output neurons because there are 10 digit classes. Here, the number of parameters is:

$$84 \times 10 + 10 = 850$$

Thus:

$$\mathbb{R}^{84} \xrightarrow{\texttt{Dense}} \mathbb{R}^{10}$$

And the softmax then converts these ten outputs into class probabilities. It adds **no trainable parameters**.

Thus, the total number of trainable parameters in the LeNet-5 architecture is:

$$156 + 2'416 + 48'120 + 10'164 + 850 = 61'706$$

Layer	Output	Parameters
`Input`	$32 \times 32 \times 1$	0
`Conv2D`	$28 \times 28 \times 6$	156
`AveragePooling2D`	$14 \times 14 \times 6$	0
`Conv2D`	$10 \times 10 \times 16$	$2'416$
`AveragePooling2D`	$5 \times 5 \times 16$	0
`Flatten`	400	0
`Dense`	120	$48'120$
`Dense`	84	$10'164$
`Dense`	10	850

CNN is parameter efficient. Look at where the parameters are:

$$\underbrace{156 + 2'416}_{\text{convolutions}} = 2'572 \lll \underbrace{48'120 + 10'164 + 850}_{\text{dense classifier}} = 59'134$$

So roughly 96% of LeNet's parameters are in the dense part. This is a very important observation: **the convolutional part of the network is very parameter efficient.** The convolutional layers are able to extract features from the input images with a relatively small number of parameters, thanks to local connectivity and weight sharing, while the dense layers, which are responsible for classification, contain the majority of the parameters. This parameter efficiency is one of the main advantages of convolutional layers for image processing.

7.11.4 LeNet-5 Performance

LeNet-5 demonstrates that a CNN can successfully learn both feature extraction and classification directly from image data. Its effectiveness on handwritten-digit recognition validates the key CNN idea: instead of manually designing image features, convolutional filters can be **learned end-to-end from data**.

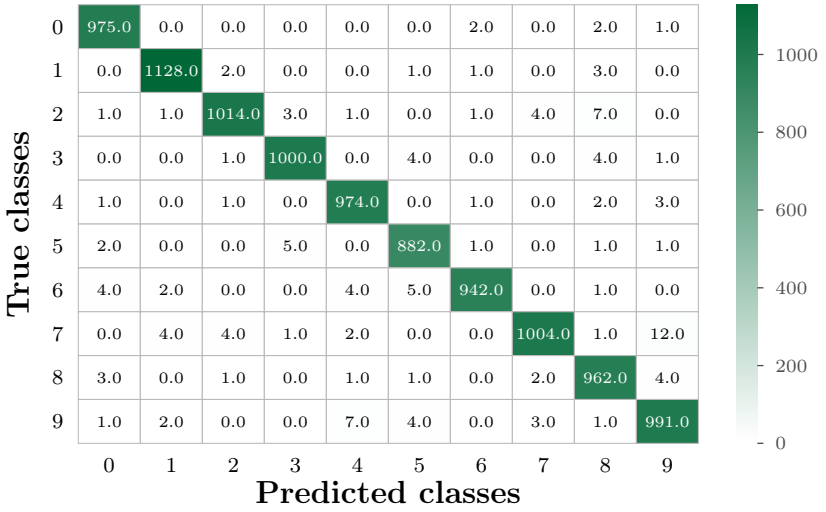


Figure 81: **LeNet-5 classification performance.** Confusion matrix on the 10′000 test images, where rows indicate true classes and columns predicted classes. The strongly dominant diagonal shows that most digits are correctly classified, with 9′872 correct predictions and only 128 errors, corresponding to an accuracy of 98.72%. The few off-diagonal entries indicate occasional confusion between visually similar handwritten digits.

The Figure 81 provides experimental evidence for the entire CNN idea developed throughout the chapter:

$$\text{image} \rightarrow \text{learned local features} \rightarrow \text{higher-level representation} \rightarrow \text{classification}$$

LeNet does not need manually designed digit features. The convolutional feature extractor learns useful representations directly from the images, and the resulting network achieves **98.72% test accuracy** on the dataset represented by this confusion matrix.

7.11.5 CNN vs Direct MLP Parameter Count

After looking at the LeNet-5 architecture, the next question is: ***if most of LeNet's parameters are in its dense layers anyway, why not simply flatten the original image and feed it directly to an MLP?***

First of all, LeNet has 61′706 parameters in total. Its convolutional layers contain only:

$$156 + 2416 = 2572$$

Parameters, while the dense part contains:

$$48'120 + 10'164 + 850 = 59'134$$

So about $59'134 \div 61'706 \approx 95.8\%$ of the parameters are in the MLP classifier part of the network. But that does **not** mean that the convolutional part is useless. Quite the opposite: it efficiently transforms the raw image into a much smaller, meaningful representation before classification.

What if we remove the CNN?

The grayscale input contains $32 \times 32 = 1024$ pixels. Instead of using the CNN to reduce the input size:

$$32 \times 32 \to \text{CNN} \to 400 \to 120 \to 84 \to 10$$

Suppose we flatten the image immediately and feed all 1024 pixels directly to an MLP:

$$1024 \to 84 \to 10$$

Why 84? Because we are using the last FC (fully connected) part of LeNet-5 as a simple comparison. The first dense layer requires:

$$1024 \times 84 + 84 = 86'100 \text{ parameters}$$

The output layer requires:

$$84 \times 10 + 10 = 850 \text{ parameters}$$

Therefore:

$$86'100 + 850 = 86'950 \text{ parameters in total}$$

So **the MLP has** $86'950 - 61'706 = 25'244$ **more parameters than LeNet-5, which is about** 41% **more.** This is a significant increase in the number of parameters.

But the more important issue is how they scale. The difference becomes much clearer when the input changes. The original LeNet input is grayscale $32 \times 32 \times 1$. Now suppose that the same image becomes **RGB** $32 \times 32 \times 3$. This triples the amount of input data $1024 \to 3072$ pixels. And this is where CNN and MLP behave very differently.

What happens to the CNN with RGB? Originally, the first convolution has 6 filters. Since the image is grayscale, each filter has $5 \times 5 \times 1$ weights. Its parameters are:

$$6 \times (5 \times 5 \times 1 + 1) = 156$$

With RGB, each filter must span all three channels $5 \times 5 \times 3$ weights. Therefore, the first convolution now has:

$$6 \times (5 \times 5 \times 3 + 1) = 456 \text{ parameters}$$

But crucially, **only this first convolution changes**. The remainder of the architecture still receives 6 feature maps from `Conv1`, so its parameter count remains unchanged. So adding two extra color channels only adds:

$$456 - 156 = 300 \text{ parameters}$$

And what happens to the MLP with RGB? For the MLP, the input vector changes from 1024 to $32 \times 32 \times 3 = 3072$ pixels. Every one of the 84 neurons is connected to **every input value**. Therefore, the weight matrix changes from 1024×84 to 3072×84. The number of weights in that first dense layer is literally **tripled**.

Why is convolution so much more efficient?

Because the convolution has two key properties:

- **Local connectivity**. A dense neuron sees the **entire image**:

  $$\text{every input} \rightarrow \text{every neuron}$$

  A convolutional neuron sees only a **small patch** of the image. In our case, $5 \times 5 \times C_{\text{in}}$, where C_{in} is the number of input channels.

  So rather than learning separate connections from every pixel, the CNN exploits the fact that image information is strongly **spatially correlated**.

- **Weight sharing**. Even more importantly, the same filter is reused at **every spatial position**. Suppose a 5×5 filter learns to detect some useful local patterns. The CNN does **not** learn a different 5×5 detector for every spatial position. Instead, it learns **one filter** and slides it over the whole image. Therefore, same weights are reused across spatial positions. This is the reason why the convolution does not require a separate parameter for every spatial connection.

Scaling Difference. For a **dense layer** receiving an image, the number of parameters scales with the **number of input pixels** $H \times W$, the **number of input channels** C_{in} and the **number of output neurons** in the layer C_{out}:

$$\#W_{\text{MLP}} = H \times W \times C_{\text{in}} \times N_{\text{out}}$$

Therefore, if the image becomes larger, the height H and width W increase, and the number of weights in the dense layer increases too:

$$H \uparrow, \quad W \uparrow \quad \Rightarrow \quad \#W_{\text{MLP}} \uparrow$$

Instead, for a **convolutional layer**, the number of weights depends only on the **filter size** $K_{\text{h}} \times K_{\text{w}}$, the **number of input channels** C_{in} and the **number of output channels** (number of filters) C_{out}:

$$\#W_{\text{Conv}} = K_{\text{h}} \times K_{\text{w}} \times C_{\text{in}} \times C_{\text{out}}$$

So if the image becomes larger, the height H and width W increase, but the number of weights in the convolutional layer **does not change**. In other words, the convolutional parameter count does not depend directly on the spatial dimensions of the image.

	Direct MLP	**Convolution**
Connectivity	Global	Local
Weight Sharing	No	Yes
Exploits image spatial structure	Not explicitly	Yes
Parameters depend on H, W	**Yes**	**No, for a given conv layer**
Larger image	Many more parameters	Same filters reused
More input channels	First dense weights scale	First conv filters deepen

Table 9: Comparison between Direct MLP and Convolution for image inputs.

Key Takeaways 21

Convolutional layers are much more parameter-efficient for images because they exploit local connectivity and weight sharing. A direct MLP requires separate connections from every input pixel to every neuron, so its parameter count grows directly with image height, width, and number of channels. A convolution instead learns small filters and reuses them over the whole image; therefore, for a fixed convolutional layer, its parameter count depends on the kernel size and channel depths, **not on the spatial image dimensions**.

7.12 Latent Representations Learned by CNNs

7.12.1 The CNN Latent Representation

As we have seen on page 565, a CNN does not only produce a class prediction; it also learns a numerical representation of the image. We also called this representation the **latent representation** or **data-driven feature vector** of the image, and placed it in the middle of the network, between the Feature Extraction Network (FEN, which is the convolutional part of the network) and the final MLP classifier (which is the dense part of the network).

Where is the Latent Representation?

Conceptually, the latent representation is located at the end of the convolutional part of the network, right before the first dense layer of the MLP classifier. In practice, it is located at the output of the last convolutional layer, after flattening:

$$\text{Image} \rightarrow \underbrace{\text{FEN}}_{\text{Conv+Pooling}} \rightarrow \boxed{\text{Latent Representation}} \rightarrow \underbrace{\text{MLP classifier}}_{\text{task-specific}} \rightarrow \text{class}$$

In other words, the latent representation is the **feature vector produced by the CNN before the final classifier**. Mathematically, for a generic image I, we can write:

$$\mathbf{z} = \text{FEN}(I)$$

Where $\mathbf{z}$ is the latent representation of the original image I, and $\text{FEN}(I)$ is the convolutional feature extraction network applied to the image I.

≠ The FEN and the classifier have different roles

We reinforce the idea that we have already seen on page 565:

$$\underbrace{\text{Image} \rightarrow \text{Convolutional layers}}_{\text{Feature Extraction Network (FEN)}} \rightarrow \mathbf{x} \rightarrow \underbrace{\text{MLP}}_{\text{Classifier}}$$

The FEN is the part that **extracts high-level features from pixels**, while the MLP performs feature classification.

- FEN: "*What visual information is present?*"
- Classifier: "*Which class does this feature vector correspond to?*"

So it means the feature vector itself can be useful **even without the original classifier**.

Thus, instead of representing an image by its raw pixels:

$$I = [x_1, x_2, \ldots, x_n]$$

We can represent it by its latent representation:

$$\mathbf{z} = \text{FEN}(I)$$

Where $\mathbf{z}$ summarizes the high-level information learned by the CNN from the original image I.

Why is this representation useful?

A natural question about the latent representation is: *why is it useful to have a feature vector that summarizes the image?* The answer is that images with similar semantic content tend to have similar learned representations! Mathematically, if two images I_1 and I_2 are similar, then their latent representations $\mathbf{z}_1$ and $\mathbf{z}_2$ will also be similar:

$$d(I_1, I_2) = \|\text{FEN}(I_1) - \text{FEN}(I_2)\|_2 = \|\mathbf{z}_1 - \mathbf{z}_2\|_2 \tag{182}$$

Latent-space distance measures **how similar the features learned by the CNN are**. This is in contrast to pixel distance, which measures the similarity of raw intensity values. In other words, the distance in the latent representation of a CNN are much more meaningful than distances in the data itself.

Key Takeaways 22

A CNN learns not only a classifier, but also a feature representation of the input image. The Feature Extraction Network transforms raw pixels into a **latent, data-driven feature vector**:

$$\mathbf{z} = \text{FEN}(I)$$

This vector summarizes high-level visual information and can be used independently of the final classifier. Distances between latent vectors are often more meaningful than distances between raw images because the representation has been learned to capture task-relevant visual structure.

7.12.2 Visualizing CNN Features with t-SNE

We already said that a CNN transforms an image I into a latent feature vector:

$$\mathbf{z} = \text{FEN}(I)$$

Not the question is: ***does this learned feature space organize images in a more meaningful way than raw pixel space?*** To answer this question, we can repeat the t-SNE experiment we did on page 466, but this time using **CNN latent vectors instead of raw pixels**.

We use a pretrained architecture called AlexNet, which was trained on the ImageNet dataset, and consider its **4096-dimensional latent representation** right before the classifier. Thus, for an image I, we compute its latent representation:

$$\text{FEN}(I) \in \mathbb{R}^{4096}$$

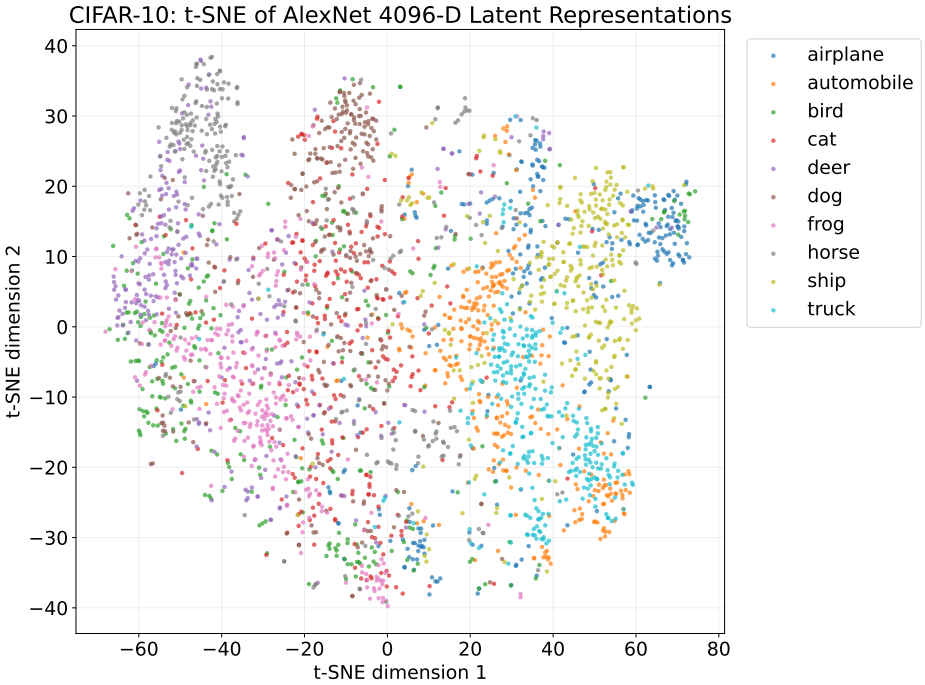


Figure 82: **t-SNE visualization of AlexNet latent representations on CIFAR-10.** Each CIFAR-10 image is mapped by a pretrained AlexNet to its 4096-dimensional latent representation immediately before the final classifier, and t-SNE projects these feature vectors into two dimensions. Colors indicate the ground-truth classes. Despite some overlap, semantically related images tend to occupy similar regions of the latent space, showing that distances between learned CNN features are more meaningful than distances between raw pixels.

What do we expect from a good latent space? Suppose two images are semantically similar, for example two different images of the same type of object. Even if their pixel values differ considerably because of position, illumination, background, etc., the CNN may produce similar high-level representations for both images:

$$\text{FEN}(I_1) \approx \text{FEN}(I_2)$$

Therefore their latent-space distance becomes small:

$$\|\text{FEN}(I_1) - \text{FEN}(I_2)\| \approx 0$$

That means semantically related images tend to appear close together in the t-SNE visualization. Indeed, on Figure 82, we see that images of the same class tend to cluster together, and semantically related classes (e.g., cats and dogs) are also close to each other. This is in stark contrast to the t-SNE visualization of raw pixels on Figure 51 (page 467), where images of different classes are heavily mixed.

Key Takeaways 23

CNNs transform raw images into learned latent vectors in which semantic similarity is better represented. AlexNet, for example, produces a 4096-dimensional feature vector before the classifier, and image similarity can be measured as

$$d(I_1, I_2) = \|\text{FEN}(I_1) - \text{FEN}(I_2)\|_2$$

When these vectors are visualized with t-SNE, semantically similar images tend to lie close together. The key improvement is therefore **not a different distance metric, but a better representation space**.

7.13 Image Retrieval and Classification in Latent Space

7.13.1 Content-Based Image Retrieval

In this section, we will turn the latent representation into something operational. Instead of using the CNN only to classify an image, use its latent vector to search for similar images. The workflow is very simple:

$$\text{Query Image} \rightarrow \text{FEN} \rightarrow \text{Query Feature Vector}$$

And then compare that vector with the latent vectors of the images stored in the training/database set.

Step 1: compute the query representation

Given a query image I_q, pass it through the **Feature Extraction Network**:

$$\mathbf{z}_q = \text{FEN}(I_q)$$

It's similar to feeding a test or query image and computing its **latent representation**, i.e. a data-driven feature vector.

Step 2: compute the stored representations

For every database image I_i, compute its latent representation:

$$\mathbf{z}_i = \text{FEN}(I_i)$$

So now we are no longer comparing the images themselves, but their learned representations:

$$I_i \rightarrow \mathbf{z}_i \quad \forall i$$

In practice, these database features can be computed in advance and stored in a database, so that the retrieval process is fast.

Step 3: measure distances in latent space

For the query vector $\mathbf{z}_q$, compute a distance to every stored vector:

$$d_i = d(\mathbf{z}_q, \mathbf{z}_i) = \|\mathbf{z}_q - \mathbf{z}_i\|_2 \quad \forall i$$

We could use any distance metric, but the most common is the Euclidean distance. The important part is **not** the exact metric. The key is that the comparison happens in the CNN feature space, not raw pixel space.

Step 4: retrieve the nearest neighbours

Sort the images according to increasing distance:

$$d_{(1)} \leq d_{(2)} \leq \ldots \leq d_{(N)}$$

And return the nearest ones. For example, with the **3-nearest neighbourhood** of a query image, we would return the three images with the smallest distances:

$$\{I_{(1)}, I_{(2)}, I_{(3)}\} \quad \text{where} \quad d_{(1)} \leq d_{(2)} \leq d_{(3)}$$

And then retrieve the original training images corresponding to those closest latent representations.

So the complete pipeline is:

$$I_q \rightarrow \text{FEN} \rightarrow \mathbf{z}_q \rightarrow \text{nearest-neighbour search} \rightarrow \text{similar images}$$

Why is it called *content-based* image retrieval? Because the search is driven by the **visual content of the image itself**, represented by the CNN feature vector. The query does not need to be a textual label such as "*dog*" or "*car*".

Key Takeaways 24

A CNN's latent representation can be used as an image descriptor for retrieval. Given a query image:

$$\mathbf{z}_q = \text{FEN}(I_q)$$

Compute its distance from the latent vectors of the database images and retrieve the nearest ones. Since CNN latent space captures high-level visual structure, the closest vectors tend to correspond to visually or semantically similar images.

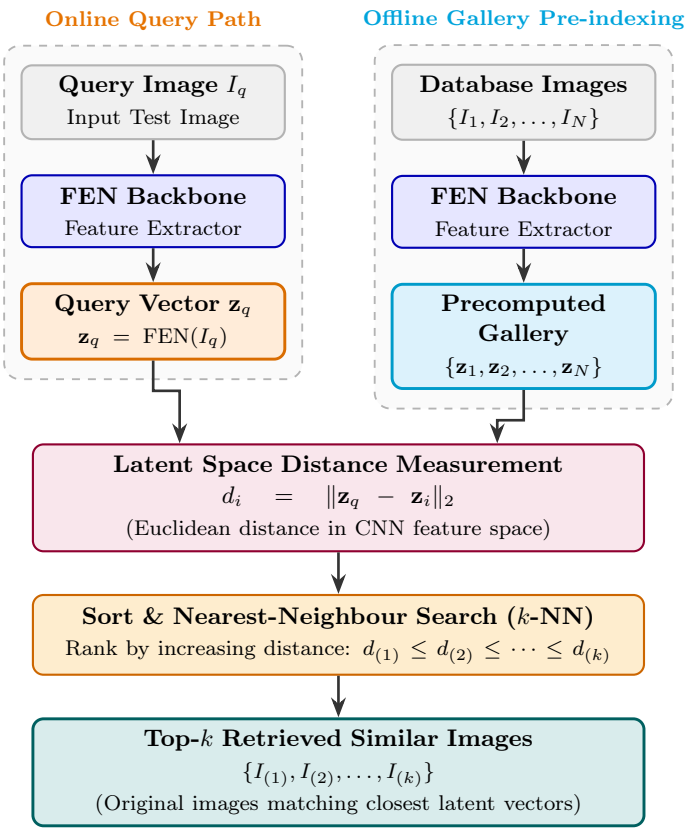


Figure 83: **Content-based image retrieval in CNN latent space.** Database images are encoded offline by the Feature Extraction Network (FEN) and their latent representations are stored in a feature gallery. At query time, the same FEN maps the query image I_q to its latent vector $\mathbf{z}_q$. Distances between $\mathbf{z}_q$ and the stored feature vectors are computed and ranked, and the k nearest vectors identify the most similar database images.

7.13.2 One-Nearest-Neighbour Classification

In this section, we will make a very small change to the image-retrieval pipeline we have built in the previous section. Instead of only retrieving the closest image, use its label as the prediction for the query image. We will implement a **1-NN classification in the latent space** by extracting the query feature vector, extracting all training-set feature vectors, computing distances, sorting them, and taking the label associated with the nearest training image.

```
# feed the test image to the FEN
image_features = fen.predict(test_image)

# feed the entire training set to the FEN
features = fen.predict(
    X_train_val,
    batch_size=512,
    verbose=0
)

# compute L1-like distances
distances = np.mean(
    np.abs(features - image_features),
    axis=-1
)

# sort by increasing distance
sortedDistances = distances.argsort()

# reorder images and labels
ordered_images = X_train_val[sortedDistances]
ordered_labels = y_train_val[sortedDistances]

# closest sample is ordered_images[0]
```

Let's see step by step how this works:

1. **Extract the query feature vector**:

   ```
   # feed the test image to the FEN
   image_features = fen.predict(test_image)
   ```

   Given a query image I_q:

   $$\mathbf{z}_q = \mathrm{FEN}(I_q)$$

   The original CNN classifier is not needed here. We only use the **Feature Extraction Network (FEN)**.

2. **Extract the training representations**:

   ```
   # feed the entire training set to the FEN
   features = fen.predict(
       X_train_val,
       batch_size=512,
       verbose=0
   ```

```
)
```

For each training image I_i:

$$\mathbf{z}_i = \text{FEN}(I_i)$$

So training set becomes a labelled feature database:

$$\mathcal{D} = \{(\mathbf{z}_i, y_i)\}_{i=1}^{N}$$

In the code snippet above, we use a batch size of 512 to speed up the feature extraction process. The batch indicates how many images are processed in parallel by the FEN.

3. **Compute the distance to every training feature**:

```
# compute L1-like distances
distances = np.mean(
    np.abs(features - image_features),
    axis=-1
)
```

We compute the distance between the query feature vector and every training feature vector. In this case, we use a **L1-like distance**:

$$d(\mathbf{z}_i, \mathbf{z}_q) = \frac{1}{D} \sum_{j=1}^{D} |z_{i,j} - z_{q,j}|$$

Where D is the dimension of the latent space. The result is a vector of distances:

$$\mathbf{d} = [d(\mathbf{z}_q, \mathbf{z}_1), d(\mathbf{z}_q, \mathbf{z}_2), ..., d(\mathbf{z}_q, \mathbf{z}_N)]$$

Then all distances are sorted from smallest to largest:

```
# sort by increasing distance
sortedDistances = distances.argsort()
```

4. **Reorder the training images and labels**:

```
# reorder images and labels
ordered_images = X_train_val[sortedDistances]
ordered_labels = y_train_val[sortedDistances]
```

The training images and labels are reordered according to the sorted distances. The closest training image is now the first element of the ordered list:

$$I_{closest} = \texttt{ordered_images}[0]$$

$$y_{closest} = \texttt{ordered_labels}[0]$$

5. **Copy the label of the closest sample**. Once the nearest representation has been found $\mathbf{z}_{i^*}$, where:

$$i^* = \arg\min_i d\left(\mathbf{z}_q, \mathbf{z}_i\right)$$

The predicted label is simply the label of the closest training image:

$$\hat{y}_q = y_{i^*}$$

So the classifier is simply a **1-NN classifier in the latent space**:

$$\hat{y}_q = y_{\arg\min_i d(\mathbf{z}_q, \mathbf{z}_i)}$$

In contrast to the image-retrieval pipeline, we do not need to display the retrieved images. We only need to return the label of the closest training image:

$$\text{query} \rightarrow \text{nearest latent vector} \rightarrow \text{take its label}$$

In summary, the CNN is valuable not only as a classifier, but also as a learned feature extractor whose latent space can support retrieval and even a simple nearest-neighbour classifier.

> **Key Takeaways 25**
>
> **1-NN in latent space replaces the CNN's original classifier with a nearest-neighbour rule.** The FEN is retained to produce meaningful feature vectors; the query is classified with the label of the training image whose latent representation is closest.

8 CNN Anatomy, Transfer Learning, and Data Scarcity

This section deepens the understanding of CNNs by interpreting convolution through sparse connectivity, weight sharing, and receptive fields. It then covers CNN backpropagation and the practical ingredients that enabled deep CNN training, before addressing data scarcity through data augmentation and transfer learning.

8.1 CNNs as Structured Neural Networks

8.1.1 Convolution as a Linear Operation

We already know **how** a convolutional layer works. The goal here is to look at the same operation from a different perspective: a convolutional layer is fundamentally a linear layer, just like a fully connected layer. The important **difference** between them is not the mathematical type of operation, but the **structure imposed on their weights**.

Convolution is a linear operation. Consider an input activation volume $\mathbf{x}$. For one filter l, the activation at spatial position (r, c) is (page 517):

$$a(r, c, l) = \sum_{\substack{u,v \in U \\ k=1,\ldots,C}} w_l(u, v, k)\, x(r + u, c + v, k) + b_l$$

This is exactly the convolutional operation introduced in the previous section: the filter looks at a **local spatial region**, spans **all input channels**, computes a weighted sum, and adds a bias.

For fixed weights, every output activation is therefore just a **linear combination of input values plus a bias**. This is the same fundamental operation performed by a dense neuron:

$$a_j = \sum_i W_{ji} x_i + b_j$$

So both can ultimately be written as:

$$\mathbf{a} = W\mathbf{x} + \mathbf{b}$$

Both convolutional and dense layers can be described as the same linear operator.

But an image is not a vector. At first this might look strange. A convolution works on something like a 3D tensor:

$$X \in \mathbb{R}^{H \times W \times C}$$

While the matrix expression:

$$\mathbf{a} = W\mathbf{x} + \mathbf{b}$$

Expects a vector input $\mathbf{x}$. The trick is simply to **flatten the input volume conceptually**:

$$X \in \mathbb{R}^{H \times W \times C} \longrightarrow \mathbf{x} \in \mathbb{R}^{HWC}$$

We can do the same with the output activation volume:

$$A \in \mathbb{R}^{H' \times W' \times C_{\text{out}}} \longrightarrow \mathbf{a} \in \mathbb{R}^{H'W'C_{\text{out}}}$$

Then there must exist some matrix W such that:

$$\mathbf{a} = W\mathbf{x} + \mathbf{b}$$

Importantly, we are not saying that CNN implementations actually need to flatten the image and construct this enormous matrix. This is a mathematical interpretation that lets us compare convolution with a dense layer.

Example 1: Convolution as a Linear Layer

Suppose for simplicity that we have a 1D input:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix}$$

And a convolutional filter of size 3 with weights:

$$\mathbf{w} = \begin{bmatrix} w_1 & w_2 & w_3 \end{bmatrix}$$

With stride 1 and no padding, the output of the convolutional layer is:

$$\begin{aligned} a_1 &= w_1x_1 + w_2x_2 + w_3x_3 + b \\ a_2 &= w_1x_2 + w_2x_3 + w_3x_4 + b \\ a_3 &= w_1x_3 + w_2x_4 + w_3x_5 + b \end{aligned}$$

We can rewrite all three equations as a single matrix multiplication:

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} w_1 & w_2 & w_3 & 0 & 0 \\ 0 & w_1 & w_2 & w_3 & 0 \\ 0 & 0 & w_1 & w_2 & w_3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} + \begin{bmatrix} b \\ b \\ b \end{bmatrix}$$

And now it visibly has the familiar form of a linear layer:

$$\mathbf{a} = W\mathbf{x} + \mathbf{b}$$

This simple matrix already hints at what the next section will discuss: W contains many **zeros**, and the same weights w_1, w_2, w_3 appear repeatedly.

⚖ **So what is the difference between a CNN and an MLP?** If both dense and convolutional layers compute $\mathbf{a} = W\mathbf{x} + \mathbf{b}$, then what makes them different? The answer is that the difference is in the structure of the weight matrix W.

- For a **dense layer**, W is dense: every output neuron can have an independent connection to every input neuron.
- For a **convolutional layer**, W has a very particular constrained structure because convolution imposes **local connectivity** and **reuse of the same filter at different locations**. In other words, the weight matrix is **sparse** and **structured** (as shown in the example above).

Previously we described convolution operationally:

$$\text{filter} \rightarrow \text{slide over input} \rightarrow \text{activation map}$$

Now we have a second, equivalent perspective:

$$\text{convolution} \equiv \text{a specially structured linear layer}$$

This is useful because CNNs are not a completely different mathematical species from ordinary neural networks. They are neural networks in which we impose a carefully chosen structure on the connections. That structure is especially appropriate for images.

Key Takeaways 1

Essentially, a convolutional layer is a linear layer with a **structured weight matrix**:

$$\text{Dense layer: } \mathbf{a} = W\mathbf{x} + \mathbf{b}$$
$$\text{Convolutional layer: } \mathbf{a} = W\mathbf{x} + \mathbf{b}$$

So the difference is not in the type of operation, but in the **structure of the weights**:

CNN vs. MLP $\quad\Rightarrow\quad$ same linear formulation, different structure of W

8.1.2 Sparse Connectivity and Weight Sharing

From the previous section, both dense and convolutional layers can be written as

$$\mathbf{a} = W\mathbf{x} + \mathbf{b}$$

So the question is: ***what is different about W in a convolutional layer compared to a dense layer?*** The answer depends on two properties of convolutional layers: **sparse connectivity** and **weight sharing**. These are what make convolution fundamentally more parameter-efficient than a fully connected layer.

⏳ Dense connectivity in a fully connected layer

In a dense layer, every output neuron has its own weight connecting it to **every input neuron**. If:

$$\mathbf{x} \in \mathbb{R}^{N_{\text{in}}}, \quad \mathbf{a} \in \mathbb{R}^{N_{\text{out}}}$$

Where N_{in} is the number of input neurons and N_{out} is the number of output neurons, then the weight matrix W has dimensions:

$$W \in \mathbb{R}^{N_{\text{out}} \times N_{\text{in}}}$$

Schematically, the matrix W looks like this:

$$W = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1N_{\text{in}}} \\ w_{21} & w_{22} & \cdots & w_{2N_{\text{in}}} \\ \vdots & \vdots & \ddots & \vdots \\ w_{N_{\text{out}}1} & w_{N_{\text{out}}2} & \cdots & w_{N_{\text{out}}N_{\text{in}}} \end{bmatrix}$$

There are generally **no forced zeros** and **no requirement** that **two entries be equal**. Therefore the layer has $N_{\text{in}} \cdot N_{\text{out}}$ weights, plus N_{out} biases, for a total of $N_{\text{in}} \cdot N_{\text{out}} + N_{\text{out}}$ parameters.

✅ Sparse connectivity

A convolutional neuron is different because it does **not depend on the entire input**. It only receives information from a **local spatial neighbourhood**.

For example, with a 3×3 convolution, one output activation depends only on the corresponding 3×3 region of the input, while all pixels outside that region have no influence on that particular activation.

In the flattened matrix representation, this means that most entries of W are zero:

$$W = \begin{bmatrix} w_1 & w_2 & w_3 & 0 & 0 \\ 0 & w_1 & w_2 & w_3 & 0 \\ 0 & 0 & w_1 & w_2 & w_3 \end{bmatrix}$$

Each row corresponds to one output neuron. Look at the first row:

$$a_1 = w_1 x_1 + w_2 x_2 + w_3 x_3$$

The weights multiplying x_4, x_5 are implicitly zero. Then:

$$a_2 = w_1 x_2 + w_2 x_3 + w_3 x_4$$

So x_1 and x_5 have zero connection to a_2. This is why the matrix representation of convolution is **sparse**. In other words, sparse connectivity means that each output neuron sees only a local part of the input, and therefore most entries of the equivalent matrix are zero.

Sparse does not mean the learned filters contain many zeros. When we say "*convolutional layers have sparse weights*", we are referring to the **equivalent large matrix** W. The actual convolutional filter:

$$w = \begin{bmatrix} w_1 & w_2 & w_3 \end{bmatrix}$$

Does not need to contain any zeros. Rather, when that small filter is represented as a full input-to-output matrix, all the connections that do not belong to the local receptive field become zero. Thus, sparsity comes from local connectivity, not necessarily from zero filter coefficients.

Weight sharing

There is a second restriction. Suppose the filter contains:

$$w_1, w_2, w_3$$

When we move the filter to another spatial position, we **do not learn another set of weights**. We reuse exactly the same parameters:

$$\begin{aligned} a_1 &= w_1 x_1 + w_2 x_2 + w_3 x_3 \\ a_2 &= w_1 x_2 + w_2 x_3 + w_3 x_4 \\ a_3 &= w_1 x_3 + w_2 x_4 + w_3 x_5 \end{aligned}$$

Indeed, the same weights w_1, w_2, w_3 are used for all output neurons. In matrix form:

$$W = \begin{bmatrix} \boxed{w_1} & \boxed{w_2} & \boxed{w_3} & 0 & 0 \\ 0 & \boxed{w_1} & \boxed{w_2} & \boxed{w_3} & 0 \\ 0 & 0 & \boxed{w_1} & \boxed{w_2} & \boxed{w_3} \end{bmatrix}$$

Those repeated entries are not independent parameters. There are still only 3 learnable weights. This is **weight sharing**: the same filter parameters are reused at every spatial location.

Bias sharing

The same idea applies to the bias. A **dense layer** generally has **one bias per output neuron**:

$$b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \end{bmatrix}$$

In a **convolutional layer**, the bias belongs to the **filter**, not to each spatial output neuron. For filter l, the activation at every spatial location is:

$$a(r, c, l) = \sum_{u,v,k} w_l(u, v, k)x(r + u, c + v, k) + b_l$$

The bias b_l is **used for every spatial location** (r, c) in feature map l. In the equivalent flattened representation, the bias vector therefore looks conceptually like:

$$b = \Big[\underbrace{b_1, b_1, b_1}_{\text{all spatial locations of feature map 1}} \quad \underbrace{b_2, b_2, b_2}_{\text{all spatial locations of feature map 2}} \quad \dots\Big]^T$$

For this reason, we call it **block-wise constant** (i.e., constant within each feature map).

Now, we can refine the previously stated idea:

$$\text{CNN} = \text{structured fully connected layer}$$

Where the structure is:

$$W = \text{sparse} + \text{shared}$$

And:

$$\mathbf{b} = \text{shared within each output channel}$$

So if we imagine the enormous W corresponding to convolution:

- Most entries are **0** because connections are local;
- The nonzero entries repeatedly contain the **same filter weights**;
- The corresponding biases repeatedly contain the **same filter bias**.

Key Takeaways 2

The core distinction is:

Fully connected layer

- Every output sees every input;
- Each connection has its own weight;
- Each output neuron has its own bias;
- W is dense.

Whereas:

Convolutional layer

- **Sparse Connectivity**: Each output sees only a local region;
- **Weight Sharing**: The same filter is reused at all spatial positions;

- **Bias Sharing**: One bias is reused over an entire feature map;
- Therefore the number of learnable parameters is dramatically smaller.

In summary, a convolutional layer is equivalent to a fully connected linear layer whose weight matrix is sparse because of local connectivity and contains repeated entries because the same filter is shared across spatial locations. The bias is also shared across all neurons of the same output feature map.

8.1.3 Translation Invariance as an Inductive Bias

From the previous section, a convolutional layer has two structural properties: **sparse connectivity** and **weight sharing**. The second one, *weight sharing*, has an important consequence: the same feature detector is applied at every spatial location of the input.

Reusing the same detector everywhere

Suppose a filter w has learned to detect a vertical edge. At one position (r_1, c_1), the convolution computes:

$$a(r_1, c_1) = \sum_{u,v,k} w\,(u, v, k) \cdot x\,(r_1 + u, c_1 + v, k) + b$$

At another position (r_2, c_2), the convolution computes:

$$a(r_2, c_2) = \sum_{u,v,k} w\,(u, v, k) \cdot x\,(r_2 + u, c_2 + v, k) + b$$

The important point is that the parameters w and b are the same in both cases. So if the filter recognizes a useful pattern in the upper-left part of an image, it can recognize the same pattern in the lower-right part as well. Thus, there is no need to learn a separate set of parameters for every spatial location.

The underlying assumption of translation invariance

The **Translation Invariance** means that if a feature is useful in one part of the image, it is likely to be useful in other parts as well. That is the **inductive bias** imposed by the convolution operation.

An **Inductive Bias** is simply an assumption built into the model architecture about the kind of functions that are likely to be useful. In this case, the assumption is that the usefulness of a visual feature should **not depend strongly** on its absolute position in the image.

For images, this is usually a very sensible prior. For example, a vertical edge is still a vertical edge whether it appears in the upper-left corner of an image or in the lower-right corner. Likewise, a texture or local corner pattern can be meaningful anywhere in the image.

Why does weight sharing create this property? Imagine instead a fully connected layer. A neuron looking at pixels in the top-left region would use parameters completely unrelated to a neuron looking at the same pattern in the bottom-right region. Therefore the model could need to learn separately: *pattern at position A*, *pattern at position B*, *pattern at position C*, etc. A convolution forces them all to use the same detector:

$$\text{same filter} \rightarrow \text{same feature definition everywhere}$$

Hence the spatial position where the pattern appears do not change **what the filter is looking for**.

Natural images have a strong spatial regularity, then the same kinds of local structures can appear almost anywhere in the image. For example edges, corners, and textures can appear in many different locations. It would therefore be wasteful to learn independent copies of the same detector for every possible image location. CNNs exploit this prior directly through their architecture.

Why is this called an inductive bias? A CNN is not merely learning everything from scratch. Its architecture already tells the network: "*if a pattern is useful here, try using the same pattern detector elsewhere too*". So the model search space is deliberately restricted to functions that are likely to be useful for images. Instead of allowing arbitrary weights $W_{\text{arbitrary}}$, we impose a highly structured W:

$$W_{\text{CNN}} = \text{sparse} + \text{shared}$$

That restriction makes the model less flexible than an arbitrary dense mapping, but in a **useful direction** for images. This is precisely what an inductive bias does: restrict the hypothesis space using assumptions about the data.

Key Takeaways 3

The chain is:

$$\text{weight sharing} \Rightarrow \text{same filter at every spatial location}$$

Which introduces the prior that **a useful visual feature should remain useful when translated**. This is a very appropriate inductive bias for images and simultaneously gives a major parameter-efficiency advantage.

In other words, CNNs share the same filter weights across spatial locations. This imposes the inductive bias that a feature useful at one image position is also useful elsewhere. As a consequence, the network does not need to learn separate detectors for every position, dramatically reducing the number of parameters.

8.1.4 Convolution-as-Fully Connected Summary

In general, we have seen that any convolutional layer can be represented by a fully connected layer performing the same computation. After flattening the input and the output, convolution can still be written as:

$$\mathbf{a} = W\mathbf{x} + \mathbf{b}$$

The difference is entirely in the **structure** of W and $\mathbf{b}$.

⚠ **The equivalent weight matrix is very large.** Suppose the flattened input has N_{in} values and the flattened output has N_{out} activations. Then the equivalent weight matrix W has shape:

$$W \in \mathbb{R}^{N_{\text{out}} \times N_{\text{in}}}$$

Exactly as for a dense layer. So, one **row** corresponds to one output neuron, and one **column** corresponds to one input neuron. The matrix may therefore be enormous, but unlike the matrix of an ordinary dense layer, **most of its entries are not independent parameters**:

1. **The matrix is sparse**: Because convolution is a **local operation**, each output activation depends only on a small region of the input. So most input-output connections do not exist. In the equivalent weight matrix,[23] most entries of W are zero. For example, in a simple 1D convolution:

$$W = \begin{bmatrix} w_1 & w_2 & w_3 & 0 & 0 & 0 \\ 0 & w_1 & w_2 & w_3 & 0 & 0 \\ 0 & 0 & w_1 & w_2 & w_3 & 0 \\ 0 & 0 & 0 & w_1 & w_2 & w_3 \end{bmatrix}$$

   Each row contains nonzero entries only where the corresponding receptive field lies. The matrix is mostly zero except in the blocks where local connectivity takes place. This is the **sparse connectivity** discussed on page 601.

2. **The nonzero entries are shared**. The nonzero entries are also **not arbitrary**. The same convolutional filter is reused at every spatial position, therefore the same values appear repeatedly:

$$W = \begin{bmatrix} \boxed{w_1} & \boxed{w_2} & \boxed{w_3} & 0 & 0 \\ 0 & \boxed{w_1} & \boxed{w_2} & \boxed{w_3} & 0 \\ 0 & 0 & \boxed{w_1} & \boxed{w_2} & \boxed{w_3} \end{bmatrix}$$

   The blocks are essentially **shifted copies of the same filter parameters**. This is the parameter sharing, or **weight sharing**, discussed on page 601, precisely on page 602.

3. **The bias vector is block-wise constant**. A dense layer normally has one independent bias for every output neuron. A convolution layer instead

[23] With equivalent weight matrix, we mean the weight matrix that would be used if we were to implement the convolution as a fully connected layer.

has one bias per filter b_l. That same bias is added at every spatial location in output channel l:

$$a(r, c, l) = \cdots + b_l$$

After flattening the output, the bias vector therefore looks conceptually like:

$$\mathbf{b} = \begin{bmatrix} b_1 & b_1 & \cdots & b_1 & b_2 & b_2 & \cdots & b_2 & \cdots \end{bmatrix}^T$$

Hence $\mathbf{b}$ is **block-wise constant**, because the bias belongs to the **filter**, not to each spatial output neuron individually.

In summary, for a *fully connected layer*, the output is computed as:

$$\mathbf{a} = W\mathbf{x} + \mathbf{b}$$

With W that is **dense** and **independently parameterized**. For a *convolutional layer*:

$$\mathbf{a} = W\mathbf{x} + \mathbf{b}$$

But W is **sparse** and **shared** (repeated entries), and $\mathbf{b}$ is **shared within each output channel** (block-wise constant). So the convolution operation can be understood as a **constrained fully connected layer**.

8.2 The Receptive Field

The previous section established that convolutional layers are sparsely connected: each output neuron depends only on a local portion of the previous layer.

The idea of **receptive field** tells us exactly **which portion of the original input can influence a given neuron**. It is one of the fundamental concepts for understanding deep CNNs.

8.2.1 Definition of Receptive Field

In a fully connected layer, every output neuron depends on the entire input. If:

$$\mathbf{a} = W\mathbf{x} + \mathbf{b}$$

With W dense, then an output neuron a_j can potentially depend on every component of the input $\mathbf{x}$.

A convolutional layer is **different**. Because of **sparse connectivity**, one output activation only depends on a specific local region of the input. This local region is called the **receptive field** of that output neuron. In other words, the **Receptive Field** of a neuron is the **region of the input that can affect its value** (i.e, its activation).

Example 2: Receptive Field of a 1D Convolution

Consider a 1D convolution with a kernel of size 3:

$$a_1 = w_1x_1 + w_2x_2 + w_3x_3$$
$$a_2 = w_1x_2 + w_2x_3 + w_3x_4$$
$$a_3 = w_1x_3 + w_2x_4 + w_3x_5$$

What can influence a_2? Only x_2, x_3, x_4. Therefore the receptive field of a_2 is:

$$\text{Receptive Field of } a_2 = \{x_2, x_3, x_4\}$$

Its size is 3. The other input values cannot influence a_2 directly, because they are outside the receptive field. The same reasoning applies to a_1 and a_3.

Receptive field in a 2D CNN

Now consider the usual image case. Suppose an input image is processed with a 3×3 convolutional filter. One activation depends on the 3×3 patch underneath the filter. Therefore, the relative to the input of that layer:

$$\text{Receptive Field of a 2D convolutional neuron} = 3 \times 3$$

For that activation, only the 3×3 patch of the input can influence its value. The other pixels are outside the receptive field and have no effect on that particular activation.

In contrast, in a Multi-Layer Perceptron (MLP), every output neuron depends on the entire input:

- For an **MLP**, each output neuron is connected to the entire previous layer, so its receptive field is the entire input to that layer.
- For a **CNN**, each output neuron is connected only to a local neighbourhood of the previous layer, so its receptive field is a small patch of the input.

Thus, with a fully connected mapping, an output unit is influenced by all input units; with convolution, only the inputs inside its local receptive field matter.

⚖ **Direct vs. Indirect dependencies.** The dependencies described above are **direct dependencies**: the output neuron is directly connected to the input neurons in its receptive field. In a multi-layer CNN, an output neuron can also depend on input neurons that are **indirectly connected** through intermediate layers. Any intermediate activation $a^{(l)}(r, c, k)$ has a receptive field that is the region of the network input whose values can propagate through the preceding layers and affect that activation. Thus, every spatial neuron in every feature map has its own receptive field.

For example, at the **first convolutional layer**, the receptive field of an output neuron is equal to the kernel size. If the kernel size is 3×3, then the receptive field is 3×3. At the **second convolutional layer**, the receptive field of an output neuron can be larger than the kernel size, because it can depend on a larger patch of the network input through the first layer. We will discuss this in more detail in the next section.

❓ Why receptive field matters

The receptive field tells us **how much context a neuron can use**. Imagine trying to detect a *small edge*. A tiny receptive filed may be sufficient. But for something more complex, such as a *face*, the network eventually needs activations that depend on progressively larger portions of the image. So **deep CNNs combine two apparently contradictory ideas**:

- Direct connections remain local, i.e., each neuron only depends on a small patch of the previous layer.
- Deep neurons indirectly depend on increasingly large regions, i.e., the receptive field grows with depth, allowing the network to capture global context.

The deep-learning reference book [3] makes exactly this observation: *despite sparse direct connections, deeper units can indirectly interact with much or even all of the input image.*

Key Takeaways 4

The definition to remember is:

Receptive Field is the region of the input that can influence a particular activation.

Where:

- In an **MLP**: the RF (Receptive Field) is **effectively global**, because every output neuron is connected to all inputs.
- In a **CNN**: the RF **starts local** (equal to the kernel size) because of sparse connectivity, but as layers as stacked, **deeper neurons** acquire progressively **larger receptive fields**, even though every individual convolution remains local.

In summary, the receptive field of a CNN neuron is the portion of the network input that can affect that neuron's activation. Unlike fully connected networks, convolutional neurons have local receptive fields because of sparse connectivity. As layers are stacked, deeper neurons indirectly depend on increasingly larger regions of the original input.

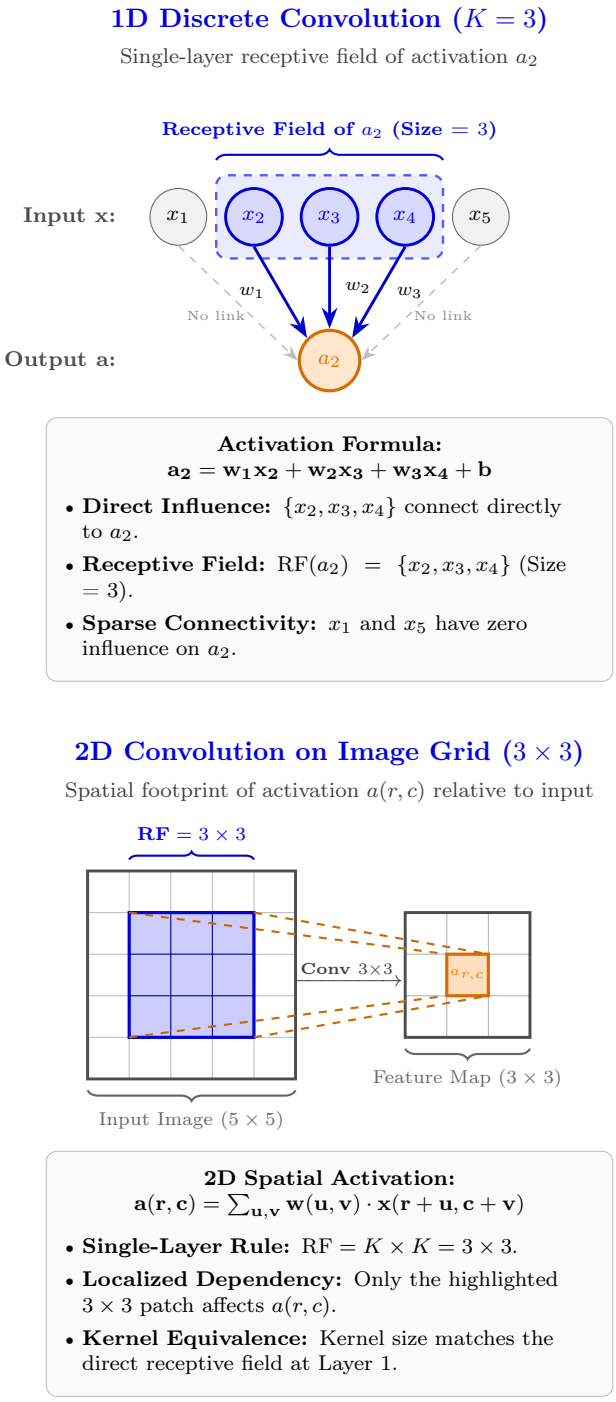


Figure 84: **Direct receptive field of a convolutional activation.** A convolutional neuron depends only on a local region of its input due to sparse connectivity. In the 1D example, the activation a_2 is directly influenced only by $\{x_2, x_3, x_4\}$; analogously, in the 2D case, an activation $a(r, c)$ produced by a 3×3 convolution depends directly only on the corresponding 3×3 input patch.

8.2.2 Receptive Field Growth with Depth

In the previous section, we anticipated that stacking convolutional layers increases the Receptive Field (RF) of deeper neurons even when every layer uses only small kernels. This increase is also observed when convolutional layers are placed one after the other without pooling.

Why deeper neurons see larger regions

Consider two consecutive 3×3 convolutional layers. A neuron in the first convolution depends on a 3×3 region of the input. Now consider a neuron in the second convolution. It depends on a 3×3 region of the output of the **first feature map**. But each of those first-layer activations already depends on its own 3×3 patch of the original input. So the second-layer neuron indirectly depends on a larger region of the original image. For stride 1, the second-layer neuron depends on a 5×5 region of the input (see Figure 85 for a visual explanation). This is the **Receptive Field (RF) growth with depth**.

Why does it become 5×5? Take the 1D case first, because it is easier to visualize. Imagine the 1D input:

$$x_1, x_2, x_3, x_4, x_5$$

And a convolutional kernel of size 3. The first layer contains, among others, the following neurons:

$$\begin{aligned} h_1 &= f(x_1, x_2, x_3) \\ h_2 &= f(x_2, x_3, x_4) \\ h_3 &= f(x_3, x_4, x_5) \end{aligned}$$

Where f is the activation function, in this case, the convolution operation. Each h_i individually sees **3 input positions**. Therefore:

$$\mathrm{RF}(h_1) = \mathrm{RF}(h_2) = \mathrm{RF}(h_3) = 3$$

Now we **add the second convolutional layer**. Suppose the second convolution also has kernel size 3. One neuron y in layer 2 looks at three neurons in layer 1:

$$y = f(h_1, h_2, h_3)$$

So **directly**, y has only three inputs: h_1, h_2, h_3. But Receptive Field is measured **with respect to the original input**, not the immediately preceding layer. So expand what h_1, h_2, h_3 themselves depend on:

$$\begin{aligned} h_1 &\leftarrow \{x_1, x_2, x_3\} \\ h_2 &\leftarrow \{x_2, x_3, x_4\} \\ h_3 &\leftarrow \{x_3, x_4, x_5\} \end{aligned}$$

Now take their **union**:

$$\mathrm{RF}(y) = \{x_1, x_2, x_3\} \cup \{x_2, x_3, x_4\} \cup \{x_3, x_4, x_5\} = \{x_1, x_2, x_3, x_4, x_5\} = 5$$

Even though the second convolution itself still has a kernel of size 3, the Receptive Field of its neurons has grown to 5 because they depend on a larger region of the original input.

Visually, on Figure 85, we can see this effect.

- The red $h_1^{(1)}$ neuron depends on the first three input values x_1, x_2, x_3.
- The blue $h_2^{(1)}$ neuron depends on the middle three input values x_2, x_3, x_4.
- The orange $h_3^{(1)}$ neuron depends on the last three input values x_3, x_4, x_5.

The top neuron $y^{(2)}$ in the second layer depends on **all three** of these first-layer neurons, and therefore indirectly depends on all five input values x_1, x_2, x_3, x_4, x_5. For example, changing x_1 (the leftmost input) could change $h_1^{(1)}$, which could change $y^{(2)}$.

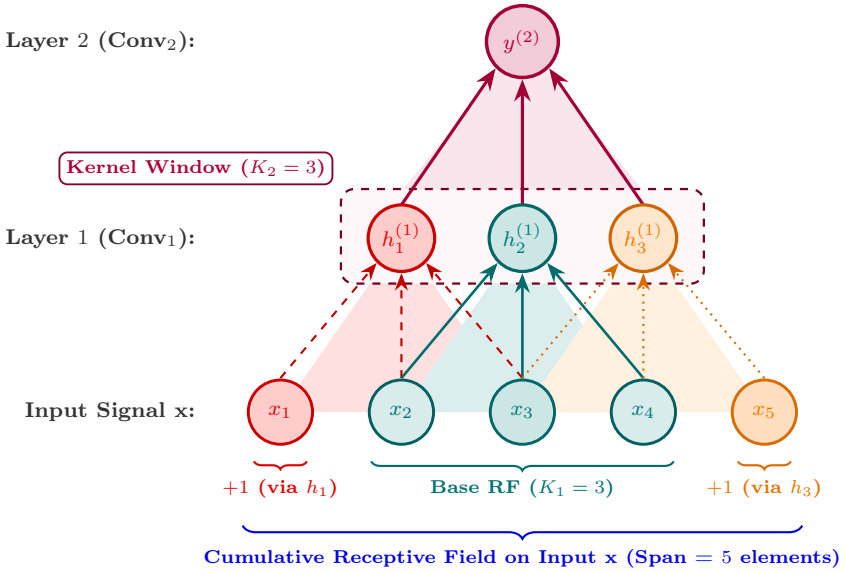


Figure 85: **Receptive Field (RF) growth through stacked convolutions.** A second-layer neuron $y^{(2)}$ directly processes three adjacent Layer-1 activations using a kernel of size $K_2 = 3$. Each of these activations already depends on three neighboring input values. Because their receptive fields overlap, their union spans $x_1, \ldots, x_5$, so the receptive field of $y^{(2)}$ with respect to the original input has size 5, even though its direct kernel size remains 3.

The Receptive Field (RF) Formula. The receptive field (RF) can be automatically calculated for each layer using a simple rule. Assuming a **kernel size** of K and a **stride** of $S = 1$, the **general recursive rule** is:

$$R_l = R_{l-1} + (K_l - 1) \tag{183}$$

Where R_l is the receptive field after layer l, and K_l is the kernel size of that layer.

This formula makes the difference between the **kernel size** and the **receptive field** more explicit. A neuron may belong to a layer whose filter size is still only 3×3, but its receptive field relative to the original input may already be much larger, e.g. 13×13. So the kernel size tells us the local operation of one layer, whereas the receptive field tells us how much of the original input can influence a deeper neuron. They are equal only for the first convolutional layer.

Example 3: Receptive Field Growth with Depth

Start from the input: $R_0 = 1$. If **every** convolution is 3×3, then $K_l = 3$, so every new layer adds:

$$K - 1 = 3 - 1 = 2$$

Therefore:

$$\begin{aligned} R_0 &= 1 \\ R_1 &= 1 + 2 = 3 \\ R_2 &= 3 + 2 = 5 \\ R_3 &= 5 + 2 = 7 \\ R_4 &= 7 + 2 = 9 \end{aligned}$$

After L such layers, we've added 2 exactly L times:

$$R_L = 1 + 2L = 2L + 1$$

So, we can simplify:

$$R_l = R_{l-1} + (K_l - 1) \quad \xrightarrow[\text{all } K_l=3]{} \quad R_L = 2L + 1 \tag{184}$$

However, the recursive formula:

$$R_l = R_{l-1} + (K_l - 1)$$

Is more useful because the **kernels can differ**. For example:

$$K_1 = 3, \quad K_2 = 5, \quad K_3 = 3$$

Gives:

$$1 \xrightarrow{3} 3 \xrightarrow{5} 7 \xrightarrow{3} 9$$

The shortcut:

$$R_L = 2L + 1$$

Only works when all L convolutions are 3×3, stride 1.

Key Takeaways 5

For stacked stride-1 convolutions, the receptive field is calculated recursively as:

$$R_l = R_{l-1} + (K_l - 1)$$

And for repeated 3×3 convolutions, a common shortcut is:

$$R_L = 2L + 1$$

Therefore:

$$6 \text{ stacked } 3 \times 3 \text{ convolutions} \implies 13 \times 13 \text{ receptive field}$$

The architecture principle is that:

$$\text{small local convolutions} + \text{depth} \implies \text{large contextual receptive field}$$

And, importantly, max pooling is not necessary for Receptive Field growth.

8.2.3 Effect of Pooling and Stride

In the previous section we considered the simplest case of a convolutional network:

$$\text{Conv } 3 \times 3, \quad \text{stride } = 1$$

There, every additional convolution enlarged the Receptive Field (RF) by exactly 2 pixels per dimension:

$$1 \to 3 \to 5 \to 7 \to \dots$$

However, in practice, convolutional networks often use pooling layers and/or convolutions with stride greater than 1. And we will see that these operations can significantly increase the growth of the RF. In particular, **max pooling** and **stride** greater than 1 increase the spacing between neighbouring activations relative to the original image. As a consequence, subsequent kernels cover much larger regions of the original input.

Why does pooling change Receptive Field (RF) growth?

Consider a standard 2×2 max pooling layer (page 553) with stride 2. It does two things simultaneously:

1. Each output depends on a 2×2 region of the previous layer;
2. The spatial resolution of the output is halved.

The second point is particularly important: after pooling, two adjacent activations no longer correspond to adjacent locations in the original input. Instead, they are now separated by a distance of 2 pixels in the original input.

Therefore, when we later apply a 3×3 convolution, moving one position inside that 3×3 kernel corresponds to moving **two pixels in the original input**, rather than one. This means that the RF formula **must be modified** to account for the pooling or stride.

How to modify the RF formula for pooling and stride?

To extend the RF Equation 183 (page 614) to the **general case** of pooling and stride, we need to keep track of two quantities:

- R_l: the Receptive Field size at layer l;
- J_l: the **jump** (sometimes called **effective stride**), that is the distance (in original-input pixels) between two adjacent activations at layer l.

Initially, the first layer has a receptive field of size 1, because each input pixel corresponds to itself, and a jump of size 1, because adjacent pixels are separated by a distance of 1:

$$R_0 = 1, \quad J_0 = 1$$

Then, for a layer with kernel size K_l and stride S_l, the receptive field is updated as follows:

$$R_l = R_{l-1} + (K_l - 1) \cdot J_{l-1} \tag{185}$$

And the jump is updated as follows:

$$J_l = J_{l-1} \cdot S_l \tag{186}$$

Example 4: Convolution without downsampling (stride = 1)

For a 3×3 convolution with stride 1:

$$K = 3, \quad S = 1$$

Initially, we have:

$$R_0 = 1, \quad J_0 = 1$$

After the first convolution, we have:

$$R_1 = 1 + (3-1) \cdot 1 = 3, \quad J_1 = 1 \cdot 1 = 1$$

Because there was **no downsampling**, neighbouring activations are still one input pixel apart. After the second convolution, we have:

$$R_2 = 3 + (3-1) \cdot 1 = 5, \quad J_2 = 1 \cdot 1 = 1$$

Again, neighbouring activations are still one input pixel apart. This recovers the previous formula for the case of convolution without downsampling (page 614).

Example 5: Convolution with downsampling (stride = 2)

Suppose instead we have:

$$\text{Conv } 3 \times 3 \to \text{MaxPool } 2 \times 2, \quad S = 2$$

After the first convolution, we have:

$$R_1 = 1 + (3-1) \cdot 1 = 3, \quad J_1 = 1 \cdot 1 = 1$$

As before, neighbouring activations are still one input pixel apart. After the max pooling layer, we have:

$$R_2 = 3 + (2-1) \cdot 1 = 4, \quad J_2 = 1 \cdot 2 = 2$$

Now neighbouring activations are **two input pixels apart**, because of the downsampling. The Receptive Field (RF) itself increased from 3 to 4, but the jump increased from 1 to 2. Now adjacent neurons correspond to positions separated by 2 pixels in the original input.

What happens to the next convolution? If add another 3×3 convolution, its Receptive Field (RF) growth is no longer merely:

✗ $R_3 = 4 + (3-1) \cdot 1 = 6$

Because the jump is now 2, each step of its kernel represents a movement of 2 pixels in the original input. Therefore, the Receptive Field (RF) growth is:

$$R_3 = 4 + (3 - 1) \cdot 2 = 8$$

So after just one convolution after the max pooling layer, the Receptive Field (RF) has already grown to 8, which is **larger than the** 5 **we would have obtained without pooling**.

From the two examples above, we can see that pooling increase the jump J, which in turn increases the growth of the Receptive Field (RF) in subsequent layers. In other words, pooling allows deeper neurons to depend on larger regions of the original input.

And what about Stride?

The **same reasoning** applies to convolutions with stride greater than 1. In general, **max pooling**, **convolutions**, and **stride** operations all increase the jump J when they have a value greater than 1. For example, a convolution with a kernel size of $K = 3$ and a stride of $S = 2$, simultaneously:

- Performs feature extraction;
- Downsamples the feature map;
- Increases the jump J by a factor of 2: $J_l = J_{l-1} \cdot 2$.

Therefore subsequent convolutions again experience faster Receptive Field (RF) growth. So in general:

$$\text{downsampling} \Rightarrow \text{larger effective stride} \Rightarrow \text{faster Receptive Field (RF) growth}$$

Why is downsampling useful?

Downsampling gives a CNN an **efficient way** to progressively move from **local details** toward **larger context**.

- **Early layers** may see *small edges* and *textures*,
- While **deeper layers** can combine information over *larger shapes* and *structures*.

Pooling/downsampling allows that Receptive Field (RF) to grow much faster than stacking stride-1 convolutions alone.

However, there is a **trade-off**:

$$\text{larger Receptive Field (RF)} \iff \text{lower spatial resolution}$$

After stride 2, the feature map has fewer spatial positions (i.e., lower resolution), and therefore the network has less information about the exact location of features. This will lead directly into the next section, where we will discuss what happens to CNN representations as we move deeper into the network.

Key Takeaways 6

For the **simple no-downsampling case**:

$$R_l = R_{l-1} + (K_l - 1)$$

Works because the jump J is always 1. But for the **general case** of pooling and stride, we need to keep track of the jump J:

$$R_l = R_{l-1} + (K_l - 1) \cdot J_{l-1}, \quad J_l = J_{l-1} \cdot S_l$$

From a conceptual point of view, the chain of reasoning is:

1. Stride > 1 or pooling;
2. Spatial downsampling;
3. Larger spacing between neighbouring activations in input coordinates;
4. Subsequent kernels cover larger portions of the original input;
5. Receptive Field (RF) grows faster.

So convolution **alone** can increase the receptive field, as we saw previously, but pooling or other stride > 1 operations make the growth substantially faster.

In summary, pooling and stride greater than one increase the effective spacing between neighbouring activations relative to the original input. Consequently, subsequent convolutional kernels cover increasingly larger input regions, causing the receptive field to grow faster.

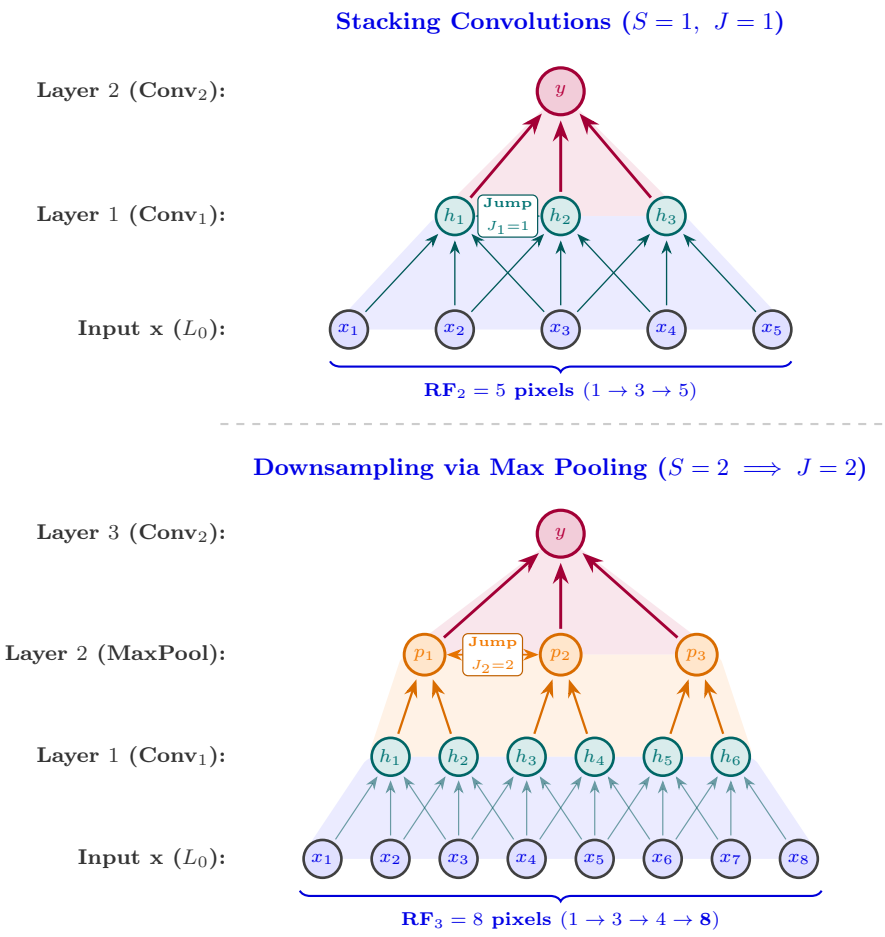


Figure 86: **Effect of downsampling on receptive-field growth.** Without downsampling (top), adjacent activations remain one input position apart ($J = 1$), so stacked convolutions progressively enlarge the receptive field from 1 to 3 to 5. With stride-2 max pooling (bottom), the effective jump becomes $J = 2$; consequently, the following convolution spans more distant input positions and the receptive field grows faster, from 1 to 3 to 4 to 8.

8.3 How CNN Representations Change with Depth

The previous section explained **why the Receptive Field (RF) grows** as we move deeper into a CNN. However, we left unexplored the trade-off between the **size of the RF** and the **resolution of the representation**. There is an architectural trend in CNNs:

$$\text{deeper layers} \implies \text{lower spatial resolution} + \text{more feature maps}$$

And, correspondingly, the network moves from representing **local low-level** details toward **higher-level patterns**.

In general, a CNN activation volume can be thought of as:

$$H \times W \times C$$

Where:

- H, W describe **where** the information is located;
- C describes **which kinds of features** are represented at that location.

As we go deeper, CNN architectures commonly evolve approximately as follows:

$$H \downarrow \quad W \downarrow \quad C \uparrow$$

So **spatial dimensions** (i.e., the resolution of the representation) **shrink**, while **channel depth** (i.e., the number of feature maps) **grows**. We already discussed this phenomenon on page 510.

↓ Spatial resolution decreases. Suppose an early activation volume is:

$$64 \times 64 \times 32$$

After downsampling, we may obtain:

$$32 \times 32 \times 64$$

Then:

$$16 \times 16 \times 128$$

And later:

$$8 \times 8 \times 256$$

The exact numbers depend on the architecture, but the general trend is:

$$64 \times 64 \to 32 \times 32 \to 16 \times 16 \to 8 \times 8$$

In spatial resolution. This happens because operations such as pooling or stride greater than 1 in convolutional layers discard spatial samples. As discussed on page 617, this also increases the effective stride and allows the **Receptive Field to grow faster**.

What does lower spatial resolution mean?

Consider an **early feature map** of size 64×64. There are many spatial positions, so the representation can preserve relatively precise information about **where** a pattern occurred.

After several downsampling operations, imagine a map of only 8×8. Now each activation corresponds to a much larger region of the original image. Therefore the representation becomes **less precise** spatially. Conceptually, in an early layer, for example, we might have a feature map that represents the presence of edges. While deeper layers increasingly represent something closer to **object parts** or even **whole objects**. The deeper we go, the more we can say about the **kind of structure** present in a larger region, but we lose information about the **exact location** of that structure.

And this is the trade-off we mentioned earlier! In the previous section, we discussed how the downsampling gives us **larger Receptive Field** but simultaneously **lower spatial resolution**. So yes, the network gradually sacrifices precise localization in exchange for access to more contextual information. The same activation can now **integrate evidence from a much larger input region**, but there are fewer distinct spatial locations left in the feature map.

So, the **trade-off** is clear:

$$\text{more context} \iff \text{less precise position}$$

Thus **aggressive downsampling** is a **double-edged sword**: it allows the network to **see more of the input** and **integrate information from a larger region**, but it also **loses spatial precision**. This is why, in some applications, it can be problematic to downsample too aggressively, especially if the task requires precise localization (e.g., object detection or segmentation[24]).

The number of feature maps increases. We analyzed the decrease in spatial resolution, but *what about the increase in the number of feature maps?* CNN architectures usually increase C. For example:

$$64 \times 64 \times 32$$

May become:

$$32 \times 32 \times 64$$

And eventually:

$$8 \times 8 \times 256$$

Because different channels correspond to different learned feature detectors.

- At **shallow layers** we may only need **filters** for relatively **simple local structures**: *edges*, *orientations*, *simple textures*, etc.

[24] Segmentation is a task where the goal is to assign a class label to each pixel in the image, which requires precise localization of objects within the image. For example, in medical imaging, accurately segmenting tumors or organs is crucial for diagnosis and treatment planning.

- At **deeper layers**, combinations of these **features** can form many **different higher-level patterns**.

In other words, there are more high-level patterns than low-level details, so the **number of maps increases** as we go deeper. This is why the **channel depth** C tends to grow with depth.

Also, we can think of the architectural evolution as:

$$\text{spatial richness} \rightarrow \text{feature richness}$$

Because the network gradually shifts from representing **local low-level details** toward **higher-level patterns**. In other words, the network moves from representing **"where"** (*where something is present*) to representing **"what"** (*what is present*). This does **not** mean the later representation necessarily contains "*more information*" in an information-theoretic sense. It means its **capacity is organized differently**.

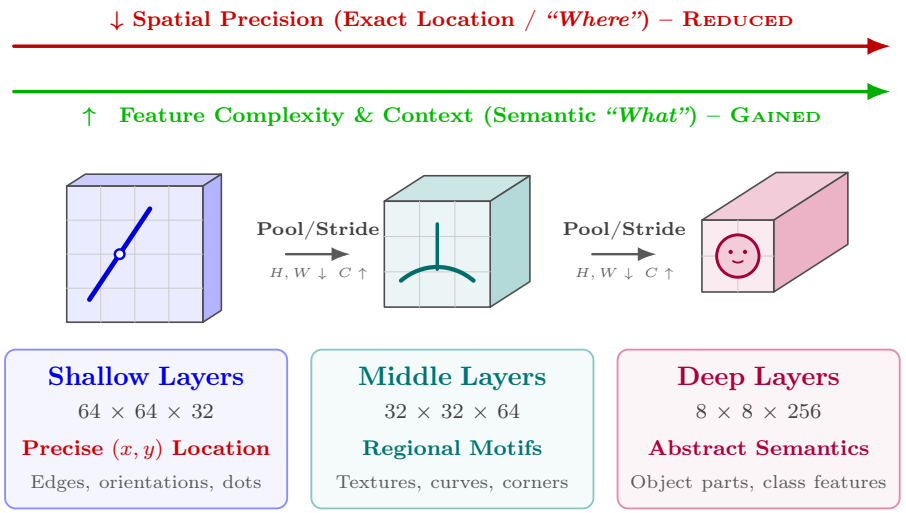


Figure 87: **Evolution of CNN representations with depth.** As the network becomes deeper, spatial resolution $H \times W$ decreases while the number of feature maps C typically increases. Larger receptive fields and progressive downsampling shift the representation from spatially precise, low-level features toward increasingly contextual and higher-level patterns, reducing the importance of exact feature location.

Key Takeaways 7

As a CNN becomes deeper, its spatial resolution typically decreases while the number of feature maps increases. Deeper units have larger receptive fields and therefore represent higher-level patterns over larger portions of the input, while retaining less precise information about their exact spatial location.

8.4 Training Convolutional Neural Networks

The Convolutional Neural Networks (CNNs) do **not require a fundamentally new training algorithm**. Since a CNN can be interpreted as an Multi-Layer Perceptron (MLP) with sparse and shared connections, it can still be trained by minimizing a loss with gradient-based optimization and computing gradients through backpropagation.

8.4.1 Backpropagation Through CNNs

Remark: Refreshing the concept of training a Neural Network

What does it mean to "*train*" a Neural Network? Imagine we have an image of a cat:

$$x = \text{image}$$

And the correct answer is:

$$y = \text{cat}$$

The CNN contains parameters, so lots of numbers that we can change:

$$\theta = \{w_1, w_2, \ldots, b_1, b_2, \ldots\}$$

Initially, these parameters are essentially guesses. We give the image to the network:

$$x \xrightarrow{\text{CNN}} \hat{y}$$

Now, suppose the network says:

$$P(\text{cat}) = 0.10, \qquad P(\text{dog}) = 0.90$$

That's a **bad prediction**. We therefore calculate a **loss**:

$$\mathcal{L}(\hat{y}, y)$$

Think of the loss simply as: "*How bad was my prediction?*". Small loss is good prediction; large loss is bad prediction. Training means **changing the parameters so that this number becomes smaller**.

What does this scary expression mean? We had:

$$\theta^* = \arg\min_{\theta} \mathcal{L}(\theta)$$

Where:

- θ are all **trainable parameters** of the network. For a CNN, this includes things such as convolutional filter weights, convolutional biases, weights and biases of FC (Fully Connected) layers, etc.
- $\mathcal{L}(\theta)$ is the **loss** obtained when the network uses parameters θ.

- $\min_\theta$ means "we want to make the loss as small as possible by changing the parameters θ".
- $\arg\min_\theta$ is slightly different from min. For example, if $f(x) = (x-3)^2$, then $\min_x f(x) = 0$ but $\arg\min_x f(x) = 3$. In other words, arg min asks "*which θ produces the smallest loss?*".

So the expression means: find the parameters θ^* that make the network's loss as small as possible.

But how do we know how to change the parameters? This is where the **gradient** comes in. Let's forget CNNs completely for a moment. Suppose our entire "network" contains one parameter w:

$$\hat{y} = wx$$

Suppose:

$$x = 2, \qquad y = 6$$

Where x is the input and y is the correct answer. Use squared error:

$$\mathcal{L} = (\hat{y} - y)^2$$

If initially $w = 1$, then:

$$\hat{y} = 1 \cdot 2 = 2.$$

So:

$$\mathcal{L} = (2-6)^2 = 16$$

We want to change w so that the loss decreases. The derivative:

$$\frac{\partial \mathcal{L}}{\partial w}$$

Is just the information we use to decide **how to change the weight** w.

Suppose our simple network with only one parameter $w = 2$ produces a loss of $\mathcal{L} = 10$. We know the model is making some error, so we'd like to change w to reduce that 10. But *should we increase w or decrease it?* That's what the derivative tells us. Answers approximately: "***if we slightly increase w, what happens to the loss?***". Three cases:

1. **Derivative is positive**. Suppose for instance:

$$\frac{\partial \mathcal{L}}{\partial w} = 5$$

   Then, if we increase w slightly, the loss will increase. So increasing w makes things worse. Therefore we want to move w in the **opposite direction** of the derivative, i.e., we should **decrease** w.

2. **Derivative is negative**. Suppose instead:

$$\frac{\partial \mathcal{L}}{\partial w} = -5$$

   Then, if we increase w slightly, the loss will decrease. So increasing w makes things better. Therefore we want to move w in the **same direction** of the derivative, i.e., we should **increase** w.

3. **Derivative is approximately zero**. Suppose finally:

$$\frac{\partial \mathcal{L}}{\partial w} \approx 0$$

Then changing w slightly doesn't change the loss much. We may be **near a minimum**, although in a large neural network zero gradient can also occur at other kinds of **stationary points**.

Thus we use the derivative, thanks to the **sign**, tells us the **direction** in which to change w to make the loss smaller. Also, the **magnitude** (i.e., $\left|\frac{\partial \mathcal{L}}{\partial w}\right|$) tells us how sensitive the loss is to that weight. That's the central idea behind the entire training procedure.

Gradient descent. As we anticipated, suppose we calculate the derivative and find:

$$\frac{\partial \mathcal{L}}{\partial w} = -16$$

The negative sign tells us: "*increasing w would decrease the loss*". Thus, to make the loss smaller, we should increase w. So we move w in the opposite direction of the derivative:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial \mathcal{L}}{\partial w}$$

Where η is the **learning rate**. Notice the minus sign $-\eta \nabla L$, because the derivative points toward increasing loss, and we **want to go the opposite way**.

Example 6: Positive derivative

Suppose $w = 2$, and:

$$\frac{\partial \mathcal{L}}{\partial w} = 5$$

Choosing $\eta = 0.1$, we update the weight:

$$w_{\text{new}} = 2 - 0.1(5) = 2 - 0.5 = 1.5$$

So because the derivative was positive, we decreased w to reduce the loss.

The **learning rate** is clearly fundamental. Suppose the derivative is:

$$\frac{\partial \mathcal{L}}{\partial w} = 100$$

We don't necessarily want to do:

$$w_{\text{new}} = w_{\text{old}} - 100$$

That could be an enormous change, and we **might overshoot the minimum**. Instead, if we choose a small learning rate:

$$\eta = 0.001$$

Then we update the weight:

$$w_{\text{new}} = w_{\text{old}} - 0.001(100) = w_{\text{old}} - 0.1$$

So the learning rate controls **how strongly we respond to the gradient**.

So gradient descent is basically: calculate which direction makes the loss increase the fastest, and then move in the opposite direction. The derivative tells us how to do that.

If we have many weights, how do we know how to change each one? Our real network doesn't have one w. It might have millions of parameters:

$$\theta = \{w_1, w_2, w_3, \ldots, w_N\}$$

We therefore need:

$$\frac{\partial \mathcal{L}}{\partial w_1}, \quad \frac{\partial \mathcal{L}}{\partial w_2}, \quad \ldots, \quad \frac{\partial \mathcal{L}}{\partial w_N}$$

Collect all of them:

$$\nabla_\theta \mathcal{L} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial w_1} \\ \frac{\partial \mathcal{L}}{\partial w_2} \\ \vdots \end{bmatrix}$$

That's the **gradient**. So:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$$

Is just a compact way of saying: "update every trainable parameter a little bit in the direction that reduces the loss".

How do we calculate the gradient? At first glance, the answer seems naively simple: "*the gradient is just the derivative of the loss with respect to the parameters*". And that is true, but we cannot calculate it directly.

Imagine a tiny network:

$$x \xrightarrow{w_1} h \xrightarrow{w_2} \hat{y} \to \mathcal{L}$$

With:

$$h = w_1 x, \qquad \hat{y} = w_2 h \qquad \mathcal{L} = (\hat{y} - y)^2$$

We want to train w_1, so we need:

$$\frac{\partial \mathcal{L}}{\partial w_1}$$

We could simply substitute everything and calculate the derivative directly:

$$\mathcal{L} = (w_2 w_1 x - y)^2$$

And differentiate directly:

$$\frac{\partial \mathcal{L}}{\partial w_1} = 2(w_2 w_1 x - y) w_2 x$$

No problem. But ***what if the network is huge?*** What if it has millions of parameters? For gradient descent, we need the derivative with respect to every single parameter:

$$\nabla_\theta \mathcal{L} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial w_1} \\ \frac{\partial \mathcal{L}}{\partial w_2} \\ \vdots \\ \frac{\partial \mathcal{L}}{\partial w_N} \end{bmatrix}$$

If $N = 10^6$, then we would have to do this substitution and differentiation a million times. That is **not feasible**.

✔ The solution is an amazing algorithm called **backpropagation**. It is a clever application of the chain rule of calculus (page 100). For example, consider again:

$$w_1 \to h_1 \to h_2 \to h_3 \to \mathcal{L}$$

By the chain rule:

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_1}$$

Now suppose we also need the gradient for a parameter w_2 responsible for h_2:

$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{\partial \mathcal{L}}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial w_2}$$

Look carefully. Part of the calculation is the same:

$$\frac{\partial \mathcal{L}}{\partial w_2} = \underbrace{\frac{\partial \mathcal{L}}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2}}_{\frac{\partial \mathcal{L}}{\partial h_2}} \cdot \frac{\partial h_2}{\partial w_2}$$

So, why calculate that over and over again? Calculate it **once**, save it, and reuse it. That's the essence of backpropagation.

Therefore, it starts at the end of the network, calculates the derivative of the loss with respect to the last layer:

$$\frac{\partial \mathcal{L}}{\partial h_3}$$

Then move one step backward and calculate the derivative of the loss with respect to the second-to-last layer:

$$\frac{\partial \mathcal{L}}{\partial h_2} = \underbrace{\frac{\partial \mathcal{L}}{\partial h_3}}_{\text{cached!}} \cdot \frac{\partial h_3}{\partial h_2}$$

Move backward again and calculate the derivative of the loss with respect to the third-to-last layer:

$$\frac{\partial \mathcal{L}}{\partial h_1} = \underbrace{\frac{\partial \mathcal{L}}{\partial h_2}}_{\text{cached!}} \cdot \frac{\partial h_2}{\partial h_1}$$

And so on, until we reach the first layer.

So there are three different concepts that are easy to mix together:

Calculus → tells us the derivative of each operation
Backpropagation → efficiently combines those derivatives
Gradient descent → uses the resulting gradients to update parameters

A CNN is a composition of differentiable layers:

$$\mathbf{x} \to \text{Conv} \to \text{Activation} \to \text{Pooling} \to \cdots \to \hat{\mathbf{y}}$$

Given a loss function $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$, training aims to find parameters θ minimizing that loss over the training data:

$$\theta^* = \arg\min_{\theta} \mathcal{L}(\theta)$$

Or, when working with a mini-batch[25] (page 193) B:

$$\theta^* = \arg\min_{\theta} \frac{1}{|B|} \sum_{i \in B} \mathcal{L}_i(\theta)$$

Gradient descent is the same idea as before. Once we can compute the gradient of the loss with respect to the parameters $\nabla_\theta \mathcal{L}(\theta)$, the parameters can be updated through gradient descent or one of its stochastic variants:

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta \mathcal{L}(\theta)$$

Where η is the **learning rate**, a hyperparameter that controls how much we change the parameters in response to the estimated error each time the model weights are updated.

So, conceptually, there is no special "*CNN optimizer*". The same machinery used for Feed-Forward Neural Networks applies to CNNs:

$$\text{forward pass} \to \text{loss} \to \text{backpropagation} \to \text{parameter update}$$

[25] A **Mini-Batch** is simply a small **subset of the training dataset** that we process together before making one parameter update. For example, suppose our training set contains $N = 50'000$ images. Instead of feeding all $50'000$ images through the CNN before updating its weights, we might choose a mini-batch size of $B = 100$ images. We run those 100 images through the network, compute their losses, usually average them, and then backpropagate the average loss to update the weights. Finally, we tale the next 100 images, and repeat the process until we have processed all $50'000$ images.

The **difference** is simply that the computational graph now contains CNN-specific **operations** such as **convolution** and **max pooling**.

Why does backpropagation still work for CNNs? First of all, to know how θ_l (the parameters of layer l) affects the loss $\mathcal{L}$, we need to follow the path from θ_l to $\mathcal{L}$. That is all the **chain rule** is doing (and backpropagation relies on it).

Suppose the network has two layers:

$$\mathbf{h}^{(1)} = f^{(1)}(\mathbf{x}; \theta_1), \qquad \mathbf{h}^{(2)} = f^{(2)}(\mathbf{h}^{(1)}; \theta_2)$$

Where:

- $\mathbf{h}^{(1)}$ is the output of layer 1, and $\mathbf{h}^{(2)}$ is the output of layer 2.
- $f^{(1)}$ is the function that layer 1 computes, and $f^{(2)}$ is the function that layer 2 computes.
- θ_1 are the parameters of layer 1, and θ_2 are the parameters of layer 2.
- $\mathbf{x}$ is the input to the network.
- $f^{(1)}(\mathbf{x}; \theta_1)$ means layer 1 takes the input $\mathbf{x}$, applies an operation $f^{(1)}$, using its parameters θ_1, and produces its output $\mathbf{h}^{(1)}$.
- $f^{(2)}(\mathbf{h}^{(1)}; \theta_2)$ means layer 2 takes the output of layer 1 (previously computed), applies an operation $f^{(2)}$, using its parameters θ_2, and produces its output $\mathbf{h}^{(2)}$.

For example, for a **convolutional layer**, $f^{(l)}$ could be a **convolution**.

Since we want to update θ_l during training, we need to compute the gradient of the loss with respect to θ_l:

$$\frac{\partial \mathcal{L}}{\partial \theta_l}$$

Which means: "*if we change the parameters of layer l, who does the final loss change?*". But notice the **problem**: θ_l does not directly produce the loss. Instead, it produces an output $\mathbf{h}^{(l)}$, which produces another output $\mathbf{h}^{(l+1)}$, and so on, until eventually the loss is computed. So we have to follow the entire path.

Example 7: Backpropagation through a simple network

For example, imagine a network:

$$\theta \to h_1 \to h_2 \to \mathcal{L}$$

We want to compute the gradient of the loss with respect to θ: $\dfrac{\partial \mathcal{L}}{\partial \theta}$. But θ affects L through h_1 and h_2. The chain rule tells us:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial \theta}$$

Read it from **right to left**:

$$\theta \xrightarrow{\frac{\partial h_1}{\partial \theta}} h_1 \xrightarrow{\frac{\partial h_2}{\partial h_1}} h_2 \xrightarrow{\frac{\partial \mathcal{L}}{\partial h_2}} \mathcal{L}$$

Where:

- $\frac{\partial h_1}{\partial \theta}$ means *how much h_1 changes if we slightly change θ.*
- When we slightly change theta, h_1 changes, which causes a change in h_2: $\frac{\partial h_2}{\partial h_1}$ means *how much h_2 changes if we slightly change h_1.*
- Finally, when we slightly change h_2, the loss changes: $\frac{\partial \mathcal{L}}{\partial h_2}$ means *how much the loss changes if we slightly change h_2.*

A tiny numerical example makes it clearer. Suppose:

$$h_1 = 3\theta, \quad \frac{\partial h_1}{\partial \theta} = 3$$

And suppose:

$$h_2 = 2h_1, \quad \frac{\partial h_2}{\partial h_1} = 2$$

If we increase θ by:

$$\Delta\theta = 0.1$$

Then h_1 changes by approximately:

$$\Delta h_1 = 3(0.1) = 0.3$$

Then h_2 changes by:

$$\Delta h_2 = 2(0.3) = 0.6$$

Overall:

$$\frac{\partial h_2}{\partial \theta} = 2 \cdot 3 = 6$$

So:

$$\Delta h_2 \approx 6(0.1) = 0.6$$

That's the chain rule: each derivative tells us how one stage reacts to the previous one, and multiplying them tells us the total effect through the chain.

In general, the gradients are propagated backward through the composition:

$$\frac{\partial \mathcal{L}}{\partial \theta_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}} \cdot \frac{\partial \mathbf{h}^{(L)}}{\partial \mathbf{h}^{(L-1)}} \cdot \frac{\partial \mathbf{h}^{(L-1)}}{\partial \mathbf{h}^{(L-2)}} \cdots \frac{\partial \mathbf{h}^{(l)}}{\partial \theta_l}$$

Where:

- The last term $\frac{\partial \mathbf{h}^{(l)}}{\partial \theta_l}$ means "*how does changing parameter θ_l change the output of its own layer?*".

- Then $\frac{\partial \mathbf{h}^{(l+1)}}{\partial \mathbf{h}^{(l)}}$ means "*how does changing* $\mathbf{h}^{(l)}$ *affect the next layer* $l+1$*?*".
- And so on, until we reach the loss $\mathcal{L}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}}$$

Which means: "*how does changing the final network output affect the loss?*".

In other words, the gradient of the loss with respect to the parameters of layer l is the product of the gradients of all subsequent layers. Therefore, the **only requirement for backpropagation to work** is that every layer of the CNN must be differentiable. If every operation in the forward path has a suitable derivative, the chain rule lets us propagate the loss gradient all the way back to the parameters.

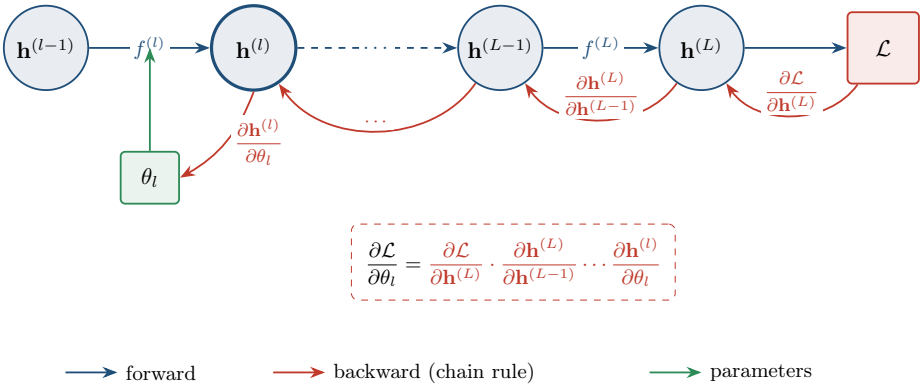


Figure 88: **Backpropagation through a CNN.** The forward pass computes activations from layer l to the loss, while the backward pass applies the chain rule in reverse to obtain $\frac{\partial \mathcal{L}}{\partial \theta_l}$, multiplying the local derivatives of all subsequent layers.

Backpropagation through a convolutional layer

The convolutional activation is computed as:

$$a(r, c, l) = \sum_{u,v,k} w(u, v, k) \cdot x(r + u, c + v, k) + b(l)$$

Where:

- $a(r, c, l)$ is the **activation** at row r, column c, and channel l;
- $w(u, v, k)$ is the **convolutional filter weight** at row u, column v, and input channel k;
- $x(r + u, c + v, k)$ is the **input** at the corresponding position;

- $b(l)$ is the **bias** for channel l.

This tells us how the convolution computes **one output activation**. It is a **forward computation** after all, because it takes the input and produces an output.

After the forward pass, we know the loss $\mathcal{L}$ at the end of the network. But **to train the convolutional filter**, we need to update its weights:

$$w \leftarrow w - \eta \cdot \frac{\partial \mathcal{L}}{\partial w}$$

Therefore, we need to compute the **gradient of the loss** with respect to the convolutional filter weights:

$$\frac{\partial \mathcal{L}}{\partial w}$$

More formally, we want to compute the **gradient of the loss with respect to every convolutional weight**:

$$\frac{\partial \mathcal{L}}{\partial w_l(u, v, k)}$$

In other words, we want to answer the question: "***how would changing this convolutional weight change the final loss?***". But the problem is that the convolutional weight does **not** directly produce the loss. Instead, it produces an activation, which produces another activation, and so on, until eventually the loss is computed:

$$w \to a^{(1)} \to a^{(2)} \to \cdots \to a^{(L)} \to \mathcal{L}$$

So we cannot compute the derivative directly. Instead, we **use the chain rule**:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial a^{(L)}} \cdot \frac{\partial a^{(L)}}{\partial a^{(L-1)}} \cdots \frac{\partial a^{(2)}}{\partial a^{(1)}} \cdot \frac{\partial a^{(1)}}{\partial w}$$

And in Convolutional Neural Networks (CNNs) context, we can write this as:

$$\frac{\partial \mathcal{L}}{\partial w_l(u, v, k)} = \underbrace{\frac{\partial \mathcal{L}}{\partial a(r, c, l)}}_{\substack{\text{effect of activation} \\ \text{on loss}}} \cdot \underbrace{\frac{\partial a(r, c, l)}{\partial w_l(u, v, k)}}_{\substack{\text{effect of weight} \\ \text{on activation}}}$$

Formally, the **effect of the activation on the loss** is:

$$\delta \equiv \frac{\partial \mathcal{L}}{\partial a} \quad \xrightarrow{\text{CNN context}} \quad \delta(r, c, l) = \frac{\partial \mathcal{L}}{\partial a(r, c, l)}$$

And the **derivative of the activation with respect to the convolutional weight** is:

$$\frac{\partial a(r, c, l)}{\partial w_l(u, v, k)} = x(r + u, c + v, k)$$

Finally, we can combine these two pieces to compute the gradient of the loss with respect to the convolutional weight. But the same filter weight is reused at

many spatial positions. Thus its total gradient is the sum of the contributions from **all positions where that parameter was used**:

$$\frac{\partial \mathcal{L}}{\partial w_l(u,v,k)} = \sum_{r,c} \delta(r,c,l) \cdot x(r+u, c+v, k) \tag{187}$$

This is the effect of **weight sharing** during backpropagation.

Deepening: Why is the derivative of the activation with respect to a convolutional weight simply the corresponding input value?

At first glance, this might seem counterintuitive:

$$\frac{\partial a(r,c,l)}{\partial w_l(u,v,k)} = x(r+u, c+v, k)$$

So, imagine that, after expanding the summation of the convolution, the activation looks like:

$$a = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Now ask, *how does a change if we slightly change* w_2*?* In other words, what is:

$$\frac{\partial a}{\partial w_2} = ?$$

When differentiating with respect to w_2, everything else is treated as constant:

$$\frac{\partial}{\partial w_2}(w_1x_1 + w_2x_2 + w_3x_3 + b)$$

Term by term:

$$\frac{\partial (w_1x_1)}{\partial w_2} = 0$$

Because that term doesn't contain w_2;

$$\frac{\partial (w_2x_2)}{\partial w_2} = x_2$$

Because x_2 is constant with respect to w_2; And:

$$\frac{\partial (w_3x_3)}{\partial w_2} = 0$$

Therefore:

$$\frac{\partial a}{\partial w_2} = x_2$$

That's exactly what the formula says: the derivative of the activation with respect to a convolutional weight is simply the corresponding input value.

Example 8: Backpropagation through a 1D convolution

Return to the 1D convolution

$$a_1 = w_1x_1 + w_2x_2 + w_3x_3$$
$$a_2 = w_1x_2 + w_2x_3 + w_3x_4$$
$$a_3 = w_1x_3 + w_2x_4 + w_3x_5$$

Suppose

$$\delta_i = \frac{\partial \mathcal{L}}{\partial a_i}.$$

Since w_1 appears in all three equations:

$$\frac{\partial \mathcal{L}}{\partial w_1} = \delta_1x_1 + \delta_2x_2 + \delta_3x_3$$

Similarly:

$$\frac{\partial \mathcal{L}}{\partial w_2} = \delta_1x_2 + \delta_2x_3 + \delta_3x_4$$

And:

$$\frac{\partial \mathcal{L}}{\partial w_3} = \delta_1x_3 + \delta_2x_4 + \delta_3x_5$$

So each filter coefficient receives information from every spatial location where it was applied. That is the backpropagation counterpart of weight sharing. In other words, the gradient of the loss w.r.t. a convolutional weight is the sum of the contributions from all positions where that parameter was used.

The bias gradient is shared too

For a moment, forget about CNNs. Consider a simple linear combination of three inputs:

$$a = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Differentiate with respect to b:

$$\frac{\partial a}{\partial b} = \frac{\partial}{\partial b}(w_1x_1 + w_2x_2 + w_3x_3 + b)$$

All the terms that don't contain b are constants with respect to b:

$$\frac{\partial(w_1x_1)}{\partial b} = 0$$

And similarly for the others. The only remaining term is:

$$\frac{\partial b}{\partial b} = 1$$

Therefore:

$$\frac{\partial a}{\partial b} = 1$$

It's just the elementary rule:

$$\frac{d}{dx}x = 1$$

Going back to CNN notation:

$$\frac{\partial a(r, c, l)}{\partial b_l} = 1$$

However, we don't care about the derivative of the activation with respect to the bias. For **training**, we need the **derivative of the loss with respect to the bias**:

$$\frac{\partial \mathcal{L}}{\partial b_l}$$

Because that's what tells us how to update b_l. For **one output location**, the **chain rule** gives:

$$\frac{\partial \mathcal{L}}{\partial b_l} = \frac{\partial \mathcal{L}}{\partial a(r, c, l)} \cdot \frac{\partial a(r, c, l)}{\partial b_l} = \delta(r, c, l) \cdot 1 = \delta(r, c, l)$$

Because for each filter l we have **one bias** b_l, that same bias is added at **every spatial position** of the resulting feature map. Therefore, the total gradient of the loss with respect to the bias is the sum of the contributions from all spatial positions:

$$\frac{\partial \mathcal{L}}{\partial b_l} = \sum_{r,c} \delta(r, c, l) \tag{188}$$

Example 9: Why is the bias gradient shared?

Suppose, for simplicity, the output feature map is 2×2. We have:

$$\begin{aligned} a(1,1,l) &= \cdots + b_l \\ a(1,2,l) &= \cdots + b_l \\ a(2,1,l) &= \cdots + b_l \\ a(2,2,l) &= \cdots + b_l \end{aligned}$$

Notice the same bias b_l is added to every activation. Therefore b_l influences the loss through four different activations:

$$\begin{aligned} b_l &\to a(1,1,l) \to \mathcal{L} \\ b_l &\to a(1,2,l) \to \mathcal{L} \\ b_l &\to a(2,1,l) \to \mathcal{L} \\ b_l &\to a(2,2,l) \to \mathcal{L} \end{aligned}$$

So, exactly like with a shared filter weight, all contributions must be added.

Now, let's **compute the gradient of the loss with respect to the bias**. We have:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial b_l} = \frac{\partial \mathcal{L}}{\partial a(1,1,l)} \frac{\partial a(1,1,l)}{\partial b_l} + \frac{\partial \mathcal{L}}{\partial a(1,2,l)} \frac{\partial a(1,2,l)}{\partial b_l} + \\ \frac{\partial \mathcal{L}}{\partial a(2,1,l)} \frac{\partial a(2,1,l)}{\partial b_l} + \frac{\partial \mathcal{L}}{\partial a(2,2,l)} \frac{\partial a(2,2,l)}{\partial b_l} \end{aligned}$$

But every local bias derivative is 1:

$$\frac{\partial a(r,c,l)}{\partial b_l} = 1$$

And every loss derivative is what we're calling δ:

$$\frac{\partial \mathcal{L}}{\partial a(r,c,l)} = \delta(r,c,l)$$

Therefore:

$$\frac{\partial \mathcal{L}}{\partial b_l} = \delta(1,1,l) + \delta(1,2,l) + \delta(2,1,l) + \delta(2,2,l)$$

For an arbitrary feature-map size, we write this compactly as:

$$\frac{\partial \mathcal{L}}{\partial b_l} = \sum_{r,c} \delta(r,c,l)$$

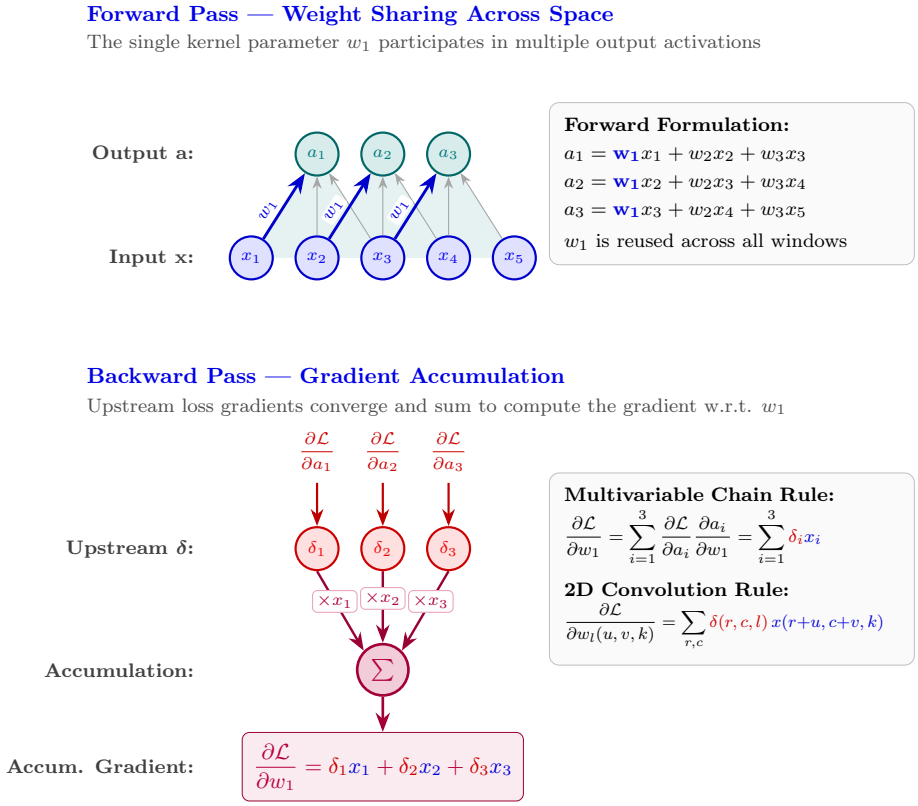


Figure 89: **Weight sharing and gradient accumulation in convolutional layers.** A single kernel parameter is reused across multiple spatial locations during the forward pass; consequently, during backpropagation, its gradient accumulates the contributions from all output activations that depend on that shared parameter.

Key Takeaways 8

CNNs are trained by ordinary gradient-based optimization and backpropagation. Because a CNN is effectively an MLP with sparse and shared connectivity, the same chain-rule machinery applies.

Also, one convolutional filter parameter is used at many spatial positions. So during backpropagation, its gradient is obtained by accumulating contributions from every use of that parameter.

In general, CNNs can be trained by gradient descent and backpropagation exactly like feed-forward neural networks, provided each layer is differentiable. Convolutional weight sharing must be accounted for during differentiation: since the same filter parameter is used at many spatial positions, its gradient accumulates contributions from all those positions.

8.4.2 Backpropagation Through Max Pooling

Max Pooling has **no trainable parameters**, but gradients still need to pass through it so that earlier convolutional layers can be trained. The rule is simple: the gradient is routed only to the input location that produced the maximum activation. For all other inputs in the pooling window, the gradient is zero.

→ Forward Pass

Consider a 2×2 Max Pooling region:

$$X = \begin{bmatrix} 1 & 3 \\ 2 & 5 \end{bmatrix}$$

The max pooling output is:

$$y = \max(X) = \max(1, 3, 2, 5) = 5$$

So during the forward pass, the value 5 is selected. Now the important thing is to remember the **location** of the maximum value, which is at position $(2, 2)$ in this case. This location is crucial because we will route the gradient back to this position during the backward pass.

Derivative of max pooling

Suppose:

$$y = \max(X) = \max\left(x_1, x_2, \ldots, x_j, \ldots, x_n\right)$$

If x_j is the unique maximum, then the derivative of y with respect to x_j is:

$$\frac{\partial y}{\partial x_j} = 1$$

While for every other input x_i that is not the maximum, the derivative is:

$$\frac{\partial y}{\partial x_i} = 0 \quad \text{for } i \neq j$$

Therefore:

$$\frac{\partial y}{\partial x_i} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases} = \begin{cases} 1 & \text{if } x_i \text{ is the selected maximum} \\ 0 & \text{otherwise} \end{cases}$$

← Backpropagating the upstream gradient

Suppose the **loss gradient** (i.e., the gradient of the loss w.r.t. the output of the max pooling layer) arriving from the next layer is:

$$\delta = \frac{\partial L}{\partial y}$$

Using the **chain rule**, we can compute the gradient of the loss with respect to each input x_i in the pooling window:

$$\frac{\partial L}{\partial x_i} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x_i} = \delta \cdot \frac{\partial y}{\partial x_i}$$

Thus, the gradient is routed back only to the input that was the **maximum**, and all other inputs receive a gradient of zero:

$$\frac{\partial L}{\partial x_i} = \begin{cases} \delta & \text{if } x_i \text{ is the selected maximum} \\ 0 & \text{otherwise} \end{cases}$$

In our example, if $\delta = 2$, then:

$$\frac{\partial L}{\partial X} = \begin{bmatrix} 0 & 0 \\ 0 & 2 \end{bmatrix}$$

Because the maximum value was at position $(2, 2)$. So **max pooling** does not split or average the incoming gradient; it simply sends the entire gradient back to the winning input location, while all other locations receive zero gradient.

Example 10: Backpropagation Through Max Pooling

Take again:

$$X = \begin{bmatrix} 1 & 3 \\ 2 & 5 \end{bmatrix}$$

And suppose the gradient arriving from above is:

$$\frac{\partial \mathcal{L}}{\partial y} = 7$$

Because 5 was the maximum, the gradient is routed back to that location:

$$\frac{\partial \mathcal{L}}{\partial X} = \begin{bmatrix} 0 & 0 \\ 0 & 7 \end{bmatrix}$$

Only the location that contributed the forward output receives the gradient. So the forward and backward passes are:

$$\begin{bmatrix} 1 & 3 \\ 2 & \boxed{5} \end{bmatrix} \xrightarrow{\max} 5$$

And:

$$7 \xrightarrow{\text{backward}} \begin{bmatrix} 0 & 0 \\ 0 & \boxed{7} \end{bmatrix}$$

Why the others get zero gradient?

Suppose we slightly change one of the non-maximal inputs:

$$X = \begin{bmatrix} 1 & 3 \\ 2 & 5 \end{bmatrix} \quad \longrightarrow \quad X' = \begin{bmatrix} 1.01 & 3 \\ 2 & 5 \end{bmatrix}$$

As long as the maximum remains at 5, the **output of the max pooling layer does not change**:

$$y = \max(X) = 5 \quad \text{and} \quad y' = \max(X') = 5$$

Therefore locally, the output is **insensitive** to changes in the non-maximal inputs. This means that the derivative of the output with respect to any non-maximal input is zero:

$$\frac{\partial y}{\partial x_i} = 0$$

By contrast, if we **change the maximum** slightly:

$$X'' = \begin{bmatrix} 1 & 3 \\ 2 & 5.01 \end{bmatrix}$$

Then the **output changes**:

$$y'' = \max(X'') = 5.01$$

This means that the output is **sensitive** to changes in the maximal input, and the derivative of the output with respect to the maximal input is non-zero:

$$\frac{\partial y}{\partial x_j} = 1$$

Max pooling behaves like a switch. Max Pooling stores which input won during the forward pass. Then **during backpropagation**, that **stored position acts like a switch**:

$$\text{incoming gradient} \rightarrow \text{winning input only}$$

This is called **Argmax Mask**. For example, taking the previous example:

$$X = \begin{bmatrix} 1 & 3 \\ 2 & 5 \end{bmatrix} \quad \longrightarrow \quad \text{Argmax Mask} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

Then the backward pass is simply:

$$\frac{\partial \mathcal{L}}{\partial X} = \delta M$$

Max pooling has no parameter gradients. Unlike convolution, **Max Pooling has no weights or biases to update**, so there are no parameter gradients to compute:

$$\theta_{\text{pool}} = \emptyset \quad \Longrightarrow \quad \frac{\partial \mathcal{L}}{\partial \theta_{\text{pool}}} = 0$$

The only reason we need its derivative is to continue backpropagation toward earlier activations:

$$\frac{\partial \mathcal{L}}{\partial X} = \delta M$$

In other words, Max Pooling is a non-linear operation that passes gradients back to the winning input location. It does not have any trainable parameters, but it is essential for propagating gradients through the network. Otherwise, the earlier convolutional layers would not receive any gradient signal and could not be trained.

Key Takeaways 9

For a max-pooling output:

$$y = \max_i x_i$$

The local derivative is:

$$\frac{\partial y}{\partial x_i} = \begin{cases} 1, & x_i \text{ is the selected maximum,} \\ 0, & \text{otherwise.} \end{cases}$$

Therefore, if the upstream gradient is:

$$\delta = \frac{\partial \mathcal{L}}{\partial y}$$

Then:

$$\frac{\partial \mathcal{L}}{\partial x_i} = \begin{cases} \delta, & x_i \text{ is the maximum,} \\ 0, & \text{otherwise.} \end{cases}$$

The gradient is only routed through the input pixel that contributed to the output value.

In general, during backpropagation through max pooling, the incoming gradient is routed only to the input activation that produced the maximum during the forward pass. The local derivative is 1 at that position and 0 for every other input in the pooling region.

9 Exams

9.1 2026

9.1.1 July 17

Exercise 1 [1.5 pt]. Select all (and only those) statements correctly defining classification, regression, representation learning, and clustering.

1. A model trained only to reconstruct its input is necessarily a reinforcement learning agent.
2. A machine-health estimator may combine a discrete status label and a continuous risk score.
3. Regression predicts continuous quantities such as price or temperature.
4. Clustering requires the desired class label for every training example.
5. A task is unsupervised only because its input is high-dimensional.
6. Classification predicts one or more discrete labels.
7. Predicting a binary sentiment label from examples is supervised learning.
8. Learning an embedding can be useful even if no semantic class label is available.

Solution 1. Before looking at the answers, reduce everything to **4 definitions**:

- **Classification**

  *Are we predicting a **discrete label/category**?*

  We have an input x, and we want to predict a **discrete label**:

  $$f(x) = \hat{y}, \quad \hat{y} \in \{C_1, C_2, \ldots, C_K\}$$

  For example, given an image, we want to predict whether it contains a cat, a dog, or a bird. The word **discrete** means that the output is a finite set of possible values, rather than a continuous range.

  Also, classification can be binary (two classes, e.g., spam vs. not spam) or multi-class (more than two classes, e.g., cat, dog, bird). In some cases, we may also have **multi-label classification**, where each input can belong to multiple classes simultaneously.

- **Regression**

  *Are we predicting a **continuous numerical value**?*

Again, we have an input x, but now the target is a **continuous quantity**:

$$f(x) = \hat{y}, \quad \hat{y} \in \mathbb{R}$$

For example, given the features of a house (size, number of bedrooms, location), we want to predict its price. The crucial **distinction** from classification is therefore **what the prediction represents**: a discrete category vs. a continuous value.

- **Clustering**

  *Are we finding **groups in unlabeled data**?*

  This is different from classification and regression. We are **not given the correct group for every example**. Instead, we have data:

  $$D = \{x_1, x_2, \ldots, x_N\}$$

  And we want to **discover some grouping** based on regularities or similarities in those data. Conceptually:

  $$\{x_1, x_2, \ldots, x_N\} \to \{\text{cluster}_1, \text{cluster}_2, \ldots, \text{cluster}_K\}$$

  For example, suppose we have customer information but nobody tells us what kinds of customers exist. A clustering algorithm could discover groups corresponding to customers with similar behavior.

- **Representation Learning**

  *Are we learning **features/embeddings/representations** from the data?*

  The primary goal here isn't necessarily to predict a category or numerical value. Instead, we want the model to **learn a useful representation of the input**. Think:

  $$x \to z$$

  Where x is the original input and z is its learned representation. We have seen this on page 309 with word embeddings, where the model learns a vector representation of words that captures semantic meaning, or more generally in autoencoders (page 313), where the model learns a compressed representation of the input data.

These definitions tell us **what task we are trying to solve**.
In addition, we must also consider **learning paradigms** (page 11), which tell us **how the learning signal/information is provided to the model during learning**. In other words, the learning paradigm tells us "*where does the learning signal come from?*". The three main paradigms are:

- **Supervised Learning**: train using inputs **and desired outputs (labels)**:

  $$D = \{(x_1, t_1), (x_2, t_2), \ldots, (x_N, t_N)\}$$

  The target t_i tells the model what it should have predicted for input x_i. The model learns to map inputs to outputs.

Task	Definition
Classification	Discrete label(s)
Regression	Continuous value(s)
Clustering	Discover groups, no predefined labels
Representation Learning	Learn useful features / embeddings / representations

Table 10: Summary of the four tasks: classification, regression, clustering, and representation learning.

- **Unsupervised Learning**: train using the data **without desired outputs (no labels)**:

  $$D = \{x_1, x_2, \ldots, x_N\}$$

  The model exploits regularities/structure in the data to learn useful representations or groupings. There is no explicit feedback on what the correct output should be.

- **Reinforcement Learning**: learning happens through **interaction with an environment**:

  $$\text{state} \rightarrow \text{action} \rightarrow \text{reward/penalty}$$

  The reward provides the learning signal, with the goal of maximizing long-term cumulative reward.

Now, we can analyze each statement in the exercise:

✗ ***A model trained only to reconstruct its input is necessarily a reinforcement learning agent.***

This is **false**. Reconstructing the input means something like:

$$x \rightarrow \text{encoder} \rightarrow z \rightarrow \text{decoder} \rightarrow \hat{x}$$

With the goal of minimizing the difference between x and $\hat{x}$:

$$\hat{x} \approx x$$

That's the typical setup of an **autoencoder**.

Instead, reinforcement learning involves an agent interacting with an environment and receiving rewards or penalties based on its actions. Reconstructing the input does not involve any interaction with an environment or reward signal. Thus, this statement is incorrect.

✓ ***A machine-health estimator may combine a discrete status label and a continuous risk score.***

This one is probably among the options that made us hesitate. First of all, a **Machine-Health Estimator** is simply a model that looks at data from a physical machine and estimates **how healthy the machine is**. For example, imagine an industrial motor equipped with sensors measuring:

$$x = [\text{temperature}, \text{vibration}, \text{pressure}, \text{RPM}, \ldots]$$

A Neural Network receives those measurements and could produce **two outputs**:

$$x \to \begin{cases} \text{status} = \text{“warning”} \\ \text{risk score} = 0.73 \end{cases}$$

These two outputs correspond to two different tasks:

- The **status** is a **discrete label**:

  $$\text{status} \in \{\text{“healthy”}, \text{“warning”}, \text{“failure”}\}$$

  This is a **classification task**, where the model predicts one of several discrete categories.
- The **risk score** is a **continuous value**:

  $$\text{risk score} \in [0, 1]$$

  This is a **regression task**, where the model predicts a continuous quantity.

A Neural Network can simultaneously perform both tasks, making it a multi-task learning model. Therefore, this statement is **true**.

Tip: we shouldn't assume that a model can *only perform one type* of task.

✔ ***Regression predicts continuous quantities such as price or temperature.***

This is simply the definition of regression, so this statement is **true**.

✖ ***Clustering requires the desired class label for every training example.***

This is the **opposite** of the definition of clustering, so this statement is **false**. If we have:

$$D = \{(x_1, t_1), (x_2, t_2), \ldots, (x_N, t_N)\}$$

And y_i tells us the correct class, then we are in a **supervised learning** scenario, not clustering. In clustering we instead have:

$$D = \{x_1, x_2, \ldots, x_N\}$$

And try to discover groups such as:

$$\{x_1, x_2, \ldots, x_N\} \to \{\text{cluster}_1, \text{cluster}_2, \ldots, \text{cluster}_K\}$$

No predefined class labels are required, so this statement is **false**.

✖ ***A task is unsupervised only because its input is high-dimensional.***

Classic distraction. Dimensionality has nothing to do with whether learning is supervised or unsupervised. If we have a $1'000'000$-dimensional image and its label (e.g., “cat” or “dog”), we are in a supervised learning scenario. What matters is **supervision/targets**, not dimensionality. Therefore, this statement is **false**.

✔ ***Classification predicts one or more discrete labels.***

This is **true** because it is the definition of classification. If the classification predicts one label, it is called **single-label classification**. If it predicts multiple labels, it is called **multi-label classification**. In both cases, the output consists of discrete categories.

✔ ***Predicting a binary sentiment label from examples is supervised learning.***

A **Binary Sentiment Label** is a label describing the **sentiment of something using two possible categories**. For example, suppose we want a model to understand whether a movie review is positive or negative:

- "*This movie was fantastic!*" → **positive**
- "*I didn't like this movie at all.*" → **negative**

For example, suppose we want a model to understand whether a movie review is positive or negative:

$$y \in \{\text{positive}, \text{negative}\}$$

So **binary sentiment classification** is a **classification task**: given some text, classify its sentiment into one of two categories, typically **positive** or **negative**. Since we have **labeled examples** (the text and its corresponding sentiment label), this is a **supervised learning** scenario. Therefore, this statement is **true**.

✔ ***Learning an embedding can be useful even if no semantic class label is available.***

Semantic Class Label means a label that gives an example an **explicit human-interpretable category/meaning**. For example, in a dataset of images, a semantic class label could be "cat", "dog", or "car". So the statement means that we might have data x_1, x_2, …, x_N, but nobody has annotated them with meaningful categories such as "dog", "cat", or "car". However, we can still learn an **embedding** $x \to z$, where z is a useful vector representation of the input x. This is often done in **unsupervised learning** scenarios, where we want to learn a representation that captures important features of the data without relying on explicit labels. Therefore, this statement is **true**.

Exercise 2 [1.5 pt]. For each task, select the most appropriate learning paradigm: supervised learning, unsupervised learning, or reinforcement learning.

- A model receives paired sensor vectors and binary alarm labels and must learn to predict the alarm label for new vectors.
- An agent improves a navigation policy by trial and error in a simulator.
- A model receives previous daily temperatures and learns to predict the next day's temperature from labelled examples.
- A controller learns by choosing actions and receiving delayed rewards from an environment.
- A system receives raw customer records and must discover groups with similar behaviour, without pre-defined labels.
- A network maps documents to compact vectors by reconstructing their original bag-of-words representation.

Solution 2. The question give 6 scenarios, and for each one we must choose **Supervised** (S), **Unsupervised** (U), or **Reinforcement Learning** (RL). We provided the definitions of these paradigms in the previous exercise, so we can use them to analyze each scenario.

S. ***A model receives paired sensor vectors and binary alarm labels and must learn to predict the alarm label for new vectors.***

The keyword here is **paired sensor vectors and binary alarm labels**. This means that for each input (sensor vector), we have a corresponding output (alarm label):

$$(x_i, t_i) = (\text{sensor vector}, \text{alarm label})$$

For example:

$$[32°, 0.23, 1200 \text{ RPM}] \rightarrow \text{“NO ALARM”}$$

The model is explicitly given the desired output for each training example. Therefore, this is a **Supervised Learning** scenario. More specifically, the **task** is binary classification (predicting whether an alarm should be triggered or not), but the **learning paradigm** is supervised learning.

RL. ***An agent improves a navigation policy by trial and error in a simulator.***

Here, *agent*, *policy*, and especially *trial and error* are strong indicators for **Reinforcement Learning**. Imagine a robot:

$$\text{state} \xrightarrow{\text{choose action}} \text{move left/right/forward} \rightarrow \text{new state}$$

It tries actions and learns which actions lead to better outcomes.

S. ***A model receives previous daily temperatures and learns to predict the next day's temperature from labelled examples.***

This one contains a trap. We might think: "*it's predicting the future from previous values, so maybe it's unsupervised learning*". No! The decisive words are "*from labelled examples*". For example, the training data could contain:

$$\underbrace{[20, 21, 19, 22, 23]}_{x} \rightarrow \underbrace{24^\circ}_{t}$$

The correct next-day temperature t is provided during training, so this is a **Supervised Learning** scenario. And the **task** is regression because temperature is continuous.

RL. ***A controller learns by choosing actions and receiving delayed rewards from an environment.***

This one should become almost automatic. The keywords are *choosing actions*, *rewards*, and *environment*. That's essentially the definition of **Reinforcement Learning**:

$$\text{Agent} \xrightarrow{a_t} \text{Environment} \xrightarrow{r_t} \text{Agent}$$

The word **delayed** is also relevant, because in reinforcement learning, the goal is concerned with maximizing **long-term cumulative reward**, so an action doesn't necessarily receive its useful reward immediately.

U. ***A system receives raw customer records and must discover groups with similar behaviour, without pre-defined labels.***

This is exactly the clustering example we discussed in the previous exercise. We have:

$$D = \{x_1, x_2, \ldots, x_N\}$$

But no (x_i, t_i) pairs. The system discovers groups:

$$D \rightarrow C_1, C_2, \ldots, C_K$$

The strongest indicator is "*without pre-defined labels*", which means there are no target outputs provided. Therefore, this is a **Unsupervised Learning** scenario. And the **task** is clustering because the goal is to find groups of similar customers.

U. ***A network maps documents to compact vectors by reconstructing their original bag-of-words representation.***

This is probably the least obvious one. The network is essentially doing:

$$x \xrightarrow{\text{encoder}} z \xrightarrow{\text{decoder}} \hat{x}$$

Where x is the original document representation, z is the compact representation (embedding), and $\hat{x}$ is the reconstructed document. And training tries to make $\hat{x} \approx x$. This is an **autoencoder-like reconstruction task**. There are **no semantic labels** (e.g., "sports", "politics", etc.) provided for the documents. Instead, the model learns useful structure/representations from the data through reconstruction. Therefore, this is a **Unsupervised Learning** scenario. And the **task** is representation learning because the goal is to learn a compact vector representation of the documents.

Exercise 3 [1.5 pt]. Which statements correctly describe Hebbian-style perceptron learning as presented in basic neural models? Select all and only those that apply.

1. The weights may be updated online, one sample at a time.
2. The weight update rule is a supervised procedure where target outputs are used.
3. The method can start from a random initialization.
4. The weight update rule is identical to the one used to train a deep network with backpropagation.
5. The update does not depend on the target output.
6. The update can depend on the input value, target value, and learning rate.
7. The model does not receive corrections when correctly classifying a sample.
8. It proves that XOR is linearly separable.

Solution 3. First of all, *what is Hebbian-style perceptron learning?* As introduced on page 47, it is a simple learning rule: the strength of a connection increases according to the simultaneous activation of an input and the **desired target**. The idea comes from the biological principle that "*neurons that fire together, wire together*" (Hebb Donald, 1949). For the perceptron, this means mathematically (page 47):

$$w_i^{(k+1)} = w_i^{(k)} + \Delta w_i = w_i^{(k)} + \eta \cdot x_i \cdot t, \qquad \Delta w_i = \eta \cdot x_i \cdot t$$

Where:

- η is the **learning rate**, controlling how much the weights change at each update.
- x_i is the **input value** for the i-th input.
- t is the **target output** (desired output) for the current training example.

And the algorithm is very simple:

1. **Initialize** the weights w_i randomly (zero or small random values).
2. **Set** the learning rate η (small positive value).
3. For each training example (x, t):
   (a) **Compute the output** y of the perceptron using the **current weights**.
   (b) **Compare** the output y with the target t.
   - If $y \neq t$, **update the weights** using the Hebbian rule:

$$w_i^{(k+1)} = w_i^{(k)} + \eta \cdot x_i \cdot t$$

   - If $y = t$, **no weight update** is performed.

4. **Repeat** until convergence or for a fixed number of epochs.

Now let's analyze each statement in the exercise:

✔ ***The weights may be updated online, one sample at a time.***

This is **true**. The Hebbian learning rule does not specify that we need to wait for a batch of samples. In fact, it can be applied in an online manner, where the weights are updated after each individual training example. For example, if we have a training set of N examples, we can iterate through them one by one and update the weights after each example:

$$w_i^{(k+1)} = w_i^{(k)} + \eta \cdot x_i^{(n)} \cdot t^{(n)}, \quad n = 1, 2, \ldots, N$$

In contrast, batch learning would involve accumulating the weight changes over all examples before applying the update, which is not required by Hebbian learning.

✔ ***The weight update rule is a supervised procedure where target outputs are used.***

This is **true** and it is evident from the weight update formula:

$$\Delta w_i = \eta \cdot x_i \cdot \boxed{t}$$

The **desired target output** t explicitly appears. Therefore the training data contain something like (x_i, t_i) pairs, which is the hallmark of supervised learning.

✔ ***The method can start from a random initialization.***

Similar to the first statement, this is **true**. The Hebbian learning rule does not impose any restrictions on the initial values of the weights. So, we can start with random weights (or even zero weights) and let the learning process adjust them based on the training data.

✘ ***The weight update rule is identical to the one used to train a deep network with backpropagation.***

The Hebbian learning rule is a simple local update rule based on the input and target output. In contrast, deep neural-network backpropagation involves computing gradients of a loss function with respect to the weights, using the chain rule to propagate errors backward through multiple layers. So they are **not identical algorithms**.

✘ ***The update does not depend on the target output.***

This one should be immediate from the equation:

$$\Delta w_i = \eta \cdot x_i \cdot \boxed{t}$$

The target output t is explicitly part of the update rule, so this statement is **false**. Notice also that this answer is the opposite of statement 2, which we already established as true.

✔ ***The update can depend on the input value, target value, and learning rate.***

Again, this is **true** and it is evident from the weight update formula:

$$\Delta w_i = \underbrace{\eta}_{\text{learning rate}} \cdot \underbrace{x_i}_{\text{input}} \cdot \underbrace{t}_{\text{target}}$$

This is the easiest statement if we remember the definition of the Hebbian learning rule.

✔ ***The model does not receive corrections when correctly classifying a sample.***

According to the Hebbian learning rule algorithm, if the perceptron correctly classifies a sample (i.e., the output y matches the target t), then no weight update is performed. This means that the model does not receive any corrections when it is already making correct predictions. Therefore, this statement is **true**. So:

$$\begin{aligned} \text{correct prediction } y = t &\implies \Delta w = 0 \\ \text{incorrect prediction } y \neq t &\implies \Delta w = \eta \cdot x_i \cdot t \quad \text{(apply correction)} \end{aligned}$$

✖ ***It proves that XOR is linearly separable.***

As discussed on page 55, the XOR problem is a classic example of a function that is **not linearly separable**. A single-layer perceptron, which uses the Hebbian learning rule, cannot learn to correctly classify the XOR function. Therefore, this statement is **false**.

Key Takeaways 1: What shoul we actually remember?

For a **misclassified sample**, the Hebbian learning rule updates the weights based on the input, target output, and learning rate:

$$w_i^{(k+1)} = w_i^{(k)} + \eta \cdot x_i^{(n)} \cdot t^{(n)}$$

And three properties of the Hebbian learning rule are:

1. It is a **supervised learning** procedure: target t is known and used in the update.
2. It can be **applied online**, one sample at a time.
3. If the sample is correctly classified, no weight update occurs.

Finally, the Hebbian learning rule does not solve the XOR problem, which is not linearly separable.

Exercise 4 [1 pt]. Consider a perceptron with Sign activation function and $w_0 = -0.5$, $w_1 = 1$, $w_2 = 1$. What outputs are produced for $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$? (Correct answer full grade, wrong answer 0.25 penalization)

- $1, 0, 0, 0$
- $0, 0, 0, 1$
- $0, 1, 0, 1$
- $1, 1, 1, 0$
- $0, 1, 1, 1$

Solution 4. A perceptron computes a weighted sum (page 43):

$$a = w_0 + w_1 x_1 + w_2 x_2 + b$$

And then applies the Sign activation function:

$$y = \text{Sign}(a) = \begin{cases} 1, & \text{if } a \geq 0 \\ 0, & \text{if } a < 0 \end{cases}$$

In this case, we have $w_0 = -0.5$, $w_1 = 1$, $w_2 = 1$, and no bias term b:

$$a = -0.5 + 1 \cdot x_1 + 1 \cdot x_2$$

Then, for each input pair (x_1, x_2), we compute a and then apply the Sign function to get the output y:

- $(0,0)$:
$$a = -0.5 + 1 \cdot 0 + 1 \cdot 0 = -0.5 \implies y = \text{Sign}(-0.5) = 0$$
- $(0,1)$:
$$a = -0.5 + 1 \cdot 0 + 1 \cdot 1 = 0.5 \implies y = \text{Sign}(0.5) = 1$$
- $(1,0)$:
$$a = -0.5 + 1 \cdot 1 + 1 \cdot 0 = 0.5 \implies y = \text{Sign}(0.5) = 1$$
- $(1,1)$:
$$a = -0.5 + 1 \cdot 1 + 1 \cdot 1 = 1.5 \implies y = \text{Sign}(1.5) = 1$$

So, the outputs for the input pairs $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$ are:

$$0, 1, 1, 1$$

Exercise 5 [1.5 pt]. Choose the coherent output-layer and loss design choice. For each task, select "Coherent" or "Not Coherent".

- A reconstruction loss when the target is to reproduce an input vector.
- Softmax over all labels for multi-label classification where several labels may be true at once.
- Single sigmoid output with binary cross-entropy for binary classification.
- Softmax output with categorical cross-entropy for mutually exclusive multi-class classification.
- Independent sigmoid outputs with binary cross-entropy for multi-label classification.
- Linear output with squared-error loss for scalar regression.
- One scalar regression output with binary cross-entropy for an unrestricted real-valued target.

Solution 5. This exercise is basically checking whether the **output activation and the loss function match the task**. The easiest way to solve this is to ask ourselves: "*what kind of output must the network represent?*". Then choose an activation whose output range/structure matches that target, and a loss that makes sense for that output.

- *A reconstruction loss when the target is to reproduce an input vector.*

  First of all, the terminology "***Reconstruction Loss***" sounds like a special loss function, but it is **usually not one specific mathematical loss**. It simply means that the **loss measures how well the model reconstructs its input**. The goal is something like:

  $$x \to \hat{x}$$

  And we want something like:

  $$\hat{x} \approx x$$

  That is exactly what happens in an autoencoder/reconstruction task. Compare the reconstructed vector with the original input vector using a reconstruction loss (e.g., mean squared error). This is a **coherent** design choice.

- *Softmax over all labels for multi-label classification where several labels may be true at once.*

  Softmax (page 81) takes a **vector** of real-valued scores (called logits) and **converts them into a probability distribution**. Mathematically:

  $$\sum_{k=1}^{K} p_k = 1, \quad p_k \in [0, 1]$$

  That creates **competition between classes**. For example $[0.7, 0.2, 0.1]$ means the model is distributing one unit of probability among mutually exclusive alternatives. But in **multi-label classification**, several labels

can be true simultaneously. For example, an image may contain a dog, a person, and a grass. Those **labels should not compete for a total probability of 1**. Therefore, using softmax for multi-label classification is **not coherent**.

- *Single sigmoid output with binary cross-entropy for binary classification.*

  A neural network doing classification has roughly this structure:

  $$\text{input} \to \text{hidden layers} \to \text{output layer}$$

  Suppose the task is "*given an email, decide whether it is **spam or not spam***". That's **binary classification** because there are two possible classes: *spam* or *not spam*. Now the network needs some way to **express its answer**. That's what the **output layer** is for.

  Instead of directly outputting the word "*spam*", we can have **one output neuron** producing a number:

  $$\text{email} \to \text{Neural Network} \to \underbrace{\hat{y} \in [0,1]}_{\text{e.g., } 0.87}$$

  We decide that this number means something like:

  $$\hat{y} = 0.87 = \text{high confidence/probability for class 1 (spam)}$$

  This is the "*single output*" part. But an ordinary neuron initially computes an unrestricted number:

  $$z = w^T x + b \in \mathbb{R}$$

  Which could be anything, e.g., 5.3, -17, 0.4, etc. We want the final output between 0 and 1, so we put a **sigmoid activation function** on that output neuron (page 64):

  $$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} \in (0,1)$$

  So "*single sigmoid output*" means simply that the network has **one output neuron**, and that neuron uses **sigmoid as its activation function**.

  We're now at:

  $$\text{email} \to \text{Neural Network} \to z \to \text{sigmoid} \to \hat{y} \in (0,1)$$

  There is one final problem: **how do we train it?** Suppose the training example tells us the email really is spam: $y = 1$. And the network predicts $\hat{y} = 0.87$. We need a loss function that says how bad the prediction $\hat{y}$ is compared with the correct answer y.

  For this particular binary setup (i.e., one output neuron with sigmoid activation), a suitable loss function is **Binary Cross-Entropy** (page 77). It is defined as:

  $$\mathcal{L}_{\text{BCE}} = -\left[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})\right]$$

  But let's focus on what it **does**.

Model prediction $\hat{y}$	Interpretation	BCE
0.99	Very confidently correct	≈ 0.01
0.9	Correct	≈ 0.11
0.5	Unsure	≈ 0.69
0.1	Wrong	≈ 2.30
0.01	Very confidently wrong	≈ 4.61

So BCE has a desirable property: it penalizes the model more when it is confidently wrong, and less when it is confidently correct. This is exactly what we want for a binary classification task. Therefore, "*single sigmoid output with binary cross-entropy for binary classification*" is a **coherent** design choice.

- *Softmax output with categorical cross-entropy for mutually exclusive multi-class classification.*

  Suppose we want to classify an image as:

  $$\{\text{cat}, \text{dog}, \text{horse}\}$$

  Unlike binary classification, we now have **3 possible classes**. And the question says **mutually exclusive**, which means that each image can belong to **exactly one class**. So instead of the single output neuron we used before, it's natural to have **one output for each class**: (output neuron 1) *cat*, (output neuron 2) *dog*, (output neuron 3) *horse*. Before the final activation, the network might produce arbitrary scores:

  $$z = [z_{\text{cat}}, z_{\text{dog}}, z_{\text{horse}}] \in \mathbb{R}^3 \xrightarrow{\text{for example}} \begin{bmatrix} 1.2 \\ 3.7 \\ 0.4 \end{bmatrix}$$

  These are usually called **logits**.

  Now, we'd like to transform those arbitrary scores into something we can interpret as a probability distribution over the classes. That's exactly what the **softmax activation function** does (page 81):

  $$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \quad i = 1, 2, \ldots, K$$

  Where K is the number of classes. For our example:

  $$\text{softmax}\left(\begin{bmatrix} 1.2 \\ 3.7 \\ 0.4 \end{bmatrix}\right) \approx \begin{bmatrix} 0.08 \\ 0.90 \\ 0.02 \end{bmatrix}$$

  So the network is now saying: "*I think it's a dog with 90% probability, a cat with 8% probability, and a horse with 2% probability*". And crucially, the probabilities sum to 1, which is **exactly what we want for mutually exclusive classes**.

Finally, we need a loss function to train the network. For multi-class classification with mutually exclusive classes, the standard choice is **Categorical Cross-Entropy** (page 83). It is defined as:

$$\mathcal{L}_{\text{CCE}} = -\sum_{i=1}^{K} y_i \cdot \log(\hat{y}_i)$$

Because only one entry of y is 1 (the correct class) and everything else disappears, this simplifies to:

$$\mathcal{L}_{\text{CCE}} = -\log(\hat{y}_{\text{correct class}})$$

This loss function penalizes the model based on how much probability it assigns to the correct class. If the model assigns a high probability to the correct class, the loss is low; if it assigns a low probability, the loss is high.

In our example, if the correct class is *dog*, then:

$$\mathcal{L}_{\text{CCE}} = -\log(0.90) \approx 0.105$$

Which is a low loss, indicating a good prediction. If the correct class were *cat*, then:

$$\mathcal{L}_{\text{CCE}} = -\log(0.08) \approx 2.525$$

Which is a high loss, indicating a poor prediction. Therefore, "*Softmax output with categorical cross-entropy for mutually exclusive multi-class classification*" is a **coherent** design choice.

- *Independent sigmoid outputs with binary cross-entropy for multi-label classification.*

  Suppose we classify the contents of an image using these labels:

  $$\{\text{dog}, \text{person}, \text{car}\}$$

  In the previous **mutually exclusive multi-class** problem, exactly one answer could be correct. But here, **several labels can be correct simultaneously**. For example, an image could contain both a dog and a person:

  $$y = [1, 1, 0] \quad \text{(dog and person are present, car is not)}$$

  This is **multi-label classification**.

  The softmax activation function makes sense if the classes compete with each other, because the output probabilities must sum to 1:

  $$\sum_{k=1}^{K} p_k = 1$$

  But that's exactly what we **don't want** in multi-label classification. An image could very confidently contain both a dog and a person. In fact, if the model predicts:

  $$\hat{y} = [0.9, 0.8, 0.1]$$

That means the model is 90% confident there's a dog, 80% confident there's a person, and 10% confident there's a car. It is perfectly fine because these are **not competing alternatives**. So no softmax is needed.

Instead, we give **each label its own output neuron** and its own sigmoid activation function. For example:

$$\hat{y}_{\text{dog}} = \sigma(z_{\text{dog}}), \quad \hat{y}_{\text{person}} = \sigma(z_{\text{person}}), \quad \hat{y}_{\text{car}} = \sigma(z_{\text{car}})$$

The network might produce:

$$\hat{y} = [\hat{y}_{\text{dog}}, \hat{y}_{\text{person}}, \hat{y}_{\text{car}}] = [0.95, 0.98, 0.03]$$

And we interpret these as independent probabilities for each label. Each sigmoid only cares about its own logit:

$$\hat{y}_k = \sigma(z_k) = \frac{1}{1 + e^{-z_k}}, \quad k = 1, 2, \ldots, K$$

That's what "*independent sigmoid outputs*" means.

Finally, we need a loss function. We just said that Binary Cross-Entropy (BCE) makes sense when we have a binary question (*is this label present or not?*). Well, **each label in a multi-label problem is a binary classification problem**. So we can apply BCE to **each output independently**. The architecture is essentially: several binary classifiers, one for each label.

Thus, "*Independent sigmoid outputs with binary cross-entropy for multi-label classification*" is a **coherent** design choice.

- *Linear output with squared-error loss for scalar regression.*

  Regression means predicting a **continuous numerical value**, rather than a class. For example, given information about a house (size, rooms, location, age, etc.), we want to predict its price.

  With "***scalar regression***", it means we are simply **predicting one number** $y \in \mathbb{R}$ (e.g., the price of a house), rather than a vector of several numbers.

  The simplest way to do this is to have a **linear output neuron**. This wording can be confusing, but it does **not** mean the entire neural network is linear. It simply means that the final output neuron does not have a non-linear activation function applied to it. Instead, it outputs a linear combination of its inputs:

  $$\hat{y} = w^T x + b \in \mathbb{R}$$

  This is exactly what we want for regression: the output can be any real number. In contrast with, for example, a sigmoid output, which is restricted to the range $(0, 1)$. If we're predicting an unrestricted temperature, sigmoid would be a strange choice because we'd artificially restrict the prediction to $0 < \hat{y} < 1$. So for an **unrestricted real-valued regression target**, a linear output is a natural choice.

As always, we need a loss function to train the network. Suppose the true temperature is:

$$y = 20°\text{C}$$

And the network predicts:

$$\hat{y} = 18°\text{C}$$

We need a loss function that measures how far is our numerical prediction $\hat{y}$ from the correct numerical value y. The basic squared error loss is a natural choice (page 72):

$$\mathcal{L} = (y - \hat{y})^2$$

For our example:

$$\mathcal{L} = (20 - 18)^2 = 4$$

If instead the network predicted $\hat{y} = 10°\text{C}$, the loss would be:

$$\mathcal{L} = (20 - 10)^2 = 100$$

So closer numerical predictions result in smaller loss values, and large errors are penalized more strongly because they're squared.

Across many samples, averaging these squared errors gives the familiar **Mean Squared Error (MSE)** loss:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

Again, suppose we're predicting temperature:

$$\text{sensor data} \rightarrow \text{neural network} \rightarrow \hat{y} \in \mathbb{R} \quad \text{(predicted temperature)}$$

The task is scalar regression (predict one continuous number). We use a linear output neuron (allows unrestricted real-valued predictions) and during training we compare the prediction $\hat{y}$ with the true temperature y using squared-error loss (measures numerical prediction error). Therefore:

$$\text{Scalar regression} \rightarrow \text{Linear output} \rightarrow \text{Squared-error loss / MSE}$$

Is a **coherent** design choice.

- *One scalar regression output with binary cross-entropy for an unrestricted real-valued target.*

  The first part of the statement "*one scalar regression output*" means the network has one output neuron producing one number: $\hat{y} \in \mathbb{R}$. And because this is regression, that number represents a **continuous quantity**, not a class. And "*unrestricted real-valued target*" makes this even more explicit: the true value y can be any real number, not just 0 or 1.

  Here, the problem is the loss function. Binary Cross-Entropy (BCE) wants the prediction to approach the **binary target**. For example, if the target $y \in \{0, 1\}$ is $y = 1$, then BCE will penalize the model based on how close $\hat{y}$ is to 1 (i.e., $\hat{y} \rightarrow 1$, then $\mathcal{L}_{\text{BCE}} \rightarrow 0$). But if the target is unrestricted, say $y = 3.14$, then BCE doesn't make sense because it expects a binary

target. The model will be confused: it will try to push $\hat{y}$ toward 0 or 1, but the true value is something else entirely.

A **better choice** for an unrestricted real-valued target is to use a **squared-error loss (MSE)** which directly asks: *how far is our prediction $\hat{y}$ from the correct real value y?* So, "*One scalar regression output with binary cross-entropy for an unrestricted real-valued target*" is **not coherent**.

Exercise 6 [1.5 pt]. Which statements about momentum or adaptive learning-rate methods are correct? Select all that apply and only those.

- They replace the dataset with hand-written rules.
- Momentum can accumulate update directions across iterations.
- They may interact with learning-rate selection.
- They are optimization choices, not definitions of supervised learning.
- They guarantee the global optimum for every neural network.
- They can make gradient descent variants more effective in practice.
- They remove the need for a training objective.
- Adaptive methods can change effective step sizes across layers.

Solution 6. Before analyzing the statements individually, let's establish the **basic idea**.

Where are we in the neural-network pipeline? During training, we have a loss $E(w)$ and backpropagation computes gradients such as:

$$\nabla E(w) = \frac{\partial E}{\partial w}$$

Then **gradient descent** use those gradients to change the weights:

$$w_{k+1} = w_k - \eta \cdot \nabla E(w_k)$$

Where η is the **learning rate**. So conceptually we are in the **optimization step** of training:

$$\text{Forward pass} \rightarrow \text{Loss} \rightarrow \text{Backpropagation} \rightarrow \boxed{\text{Optimizer}} \rightarrow \text{Update weights}$$

What is momentum? Ordinary gradient descent essentially looks at the **current** gradient and move accordingly (i.e., the steepest descent direction). But if the loss surface is **bumpy** (i.e., has many local minima or oscillations), then the current gradient may not be a good indicator of the overall direction to move. Momentum is a technique that **accumulates past gradients** to smooth out the update direction. In other words, it carries information from **previous updates**. Mathematically, momentum can be expressed adding a velocity term v called *inertia*:

$$v_t = \alpha \cdot v_{t-1} - \eta \cdot \nabla E(w_t)$$

Followed by the weight update:

$$w_{t+1} = w_t + v_t$$

The crucial term $\alpha \cdot v_{t-1}$ means that the new movement partially depends on the **previous movement**. Imagine gradient descent repeatedly wants to move in roughly the same direction, let's say rightwards. Momentum builds up movement in that persistent direction. That's the **intuition** behind momentum: the

optimization develops some *inertia* instead of completely forgetting its previous direction at every iteration.

Adaptive learning rates. Now consider another problem. With ordinary gradient descent, we have one learning rate η. But different parameters/layers may behave very differently, because gradient magnitudes vary across layers, and early layers can receive vanishing gradients (i.e., very small gradients, see vanishing gradient problem on page 177). So ideally we **should use separate adaptive learning rates for each parameter**. Methods such as AdaGrad, RMSProp, and Adam modify the effective step size based on gradient history.

So the big-picture distinction is:

- **Momentum**: uses previous update/gradient information to build *inertia*.
- **Adaptive methods**: adapt effective step sizes using gradient history.

And Adam (Adaptive Moment Estimation) combines ideas from both momentum and adaptive learning rates.

Now let's analyze the statements individually:

✗ ***They replace the dataset with hand-written rules.***

This is **false**. The reason it's false is simply that momentum/adaptive methods are **optimization methods**. They affect parameter updates. They don't replace the training data with manually programmed rules. In other words, data remain data, and the optimizer changes how parameters are updated.

✓ ***Momentum can accumulate update directions across iterations.***

This is **true**. As explained above, momentum accumulates information from previous gradients to smooth out the update direction. The velocity term v_t depends on the previous velocity v_{t-1}, which means that the optimizer retains some memory of past updates. This allows it to build up speed in consistent directions and dampen oscillations in noisy gradients.

✓ ***They may interact with learning-rate selection.***

This is **true**. Adaptive methods explicitly affect the **effective step size**, while momentum itself also contains the learning rate η in its update equation. So these optimization choices interact with learning-rate behavior/selection.

✓ ***They are optimization choices, not definitions of supervised learning.***

This is obviously **true**. Supervised learning is a paradigm where the learning signal/target comes from labeled data. Whereas momentum and adaptive methods are **optimization techniques**, so how optimization updates parameters.

✖ ***They guarantee the global optimum for every neural network.***

This is **false**. Neural-Network loss landscapes are generally non-convex, and these methods do **not** provide a universal guarantee that training reaches the global minimum. The word "*guarantee*" is especially suspicious in this context. While momentum and adaptive methods **can help optimization converge faster or escape shallow local minima**, they do not guarantee finding the global optimum.

✔ ***They can make gradient descent variants more effective in practice.***

This is **true**. In practice, momentum and adaptive methods often improve convergence speed and stability of training. Also, notice the cautious wording again: "*can make … more effective*", not "*always make … more effective*". So momentum and adaptive learning-rate methods exist precisely as practical improvements/modifications of basic gradient-based optimization.

✖ ***They remove the need for a training objective.***

This is **false**. The objective is the function we're trying to optimize, typically the loss function $\mathcal{L}(w)$. The optimization goal is to minimize that loss:

$$w^* = \arg\min_w \mathcal{L}(w)$$

Momentum or Adam changes **how we search for good** w. It doesn't eliminate $\mathcal{L}$. In fact, without an objective, there would be no gradient $\nabla_w \mathcal{L}$ for the optimizer to use.

✔ ***Adaptive methods can change effective step sizes across layers.***

This is **true**. Think about ordinary gradient descent:

$$\Delta w = -\eta g$$

Where $g = \nabla E(w)$ is the gradient, and η is the global learning rate. Adaptive methods modify the **effective size of the update** based on gradient history. Consequently, parameters belonging to different layers can receive differently scaled updates. So:

$$\text{different gradient histories} \implies \text{different effective step sizes}$$

And therefore different layers can indeed end up with different effective step sizes. Again, this statement says "*can*", not that the optimizer necessarily assigns exactly one explicit learning rate to each layer.

Exercise 7 [1 pt]. Which statements about backpropagation through time are correct? Select all and only those.

- It cannot suffer from vanishing gradients.
- Long unfolded sequences can behave like very deep computational graphs.
- It is irrelevant to recurrent neural networks.
- It can propagate error signals from later outputs to earlier recurrent states.
- It requires the hidden state to be manually labelled at every step.
- It applies the same chain-rule principle used in ordinary backpropagation.
- It unfolds the recurrent computation over time.
- It trains weights that are shared across time steps.

Solution 7. Before analyzing the statements individually, let's understand the intuition behind backpropagation through time (BPTT).

What problem is BPTT trying to solve? A *vanilla* Recurrent Neural Network (RNN) has a hidden state h_t that is updated at each time step t based on the previous hidden state h_{t-1} and the current input x_t:

$$h_t = f\left(W_x x_t + W_h h_{t-1} + b\right)$$

Which means that the hidden state at time t depends on all previous inputs and hidden states. That's what gives an RNN its **temporal memory**.[26] But now suppose at time $t = 4$ the network makes a bad prediction and produces a loss E_4. That prediction didn't depend only on x_4. Through the recurrent states, it may depend on: x_4, x_3, x_2, and x_1. So the error at $t = 4$ potentially needs to influence computations that happened at earlier time steps.

First we unroll the RNN. The actual RNN conceptually has a recurrent loop (i.e., h_t depends on h_{t-1}). For training, we represent that recurrence across a finite sequence as:

$$h_0 \to h_1 \to h_2 \to h_3 \to h_4$$

With the corresponding inputs:

$$\underbrace{h_0}_{x_0} \to \underbrace{h_1}_{x_1} \to \underbrace{h_2}_{x_2} \to \underbrace{h_3}_{x_3} \to \underbrace{h_4}_{x_4}$$

This is called **unrolling/unfolding the RNN through time**. Unrolling creates copies of the RNN cell for the different time steps, turning the recurrent computation into a DAG on which ordinary backpropagation can operate. But

[26] Recurrent Neural Networks (RNNs) were developed because ordinary Feed-Forward Networks cannot handle sequential data. For instance, to predict the next word in a sentence, we must consider the preceding words. A Feed-Forward Network has no recurrent state/memory connecting successive inputs, so it cannot remember previous words. RNNs solve this by maintaining a hidden state that is updated at each time step, allowing the network to retain information from earlier inputs.

there's an extremely important detail: these aren't independent Neural-Network layers with independent weights. The RNN uses the **same parameters at every time step**. For example:

$$h_1 = f\left(W_x x_1 + W_h h_0 + b\right)$$
$$h_2 = f\left(W_x x_2 + W_h h_1 + b\right)$$
$$h_3 = f\left(W_x x_3 + W_h h_2 + b\right)$$
$$h_4 = f\left(W_x x_4 + W_h h_3 + b\right)$$

Notice that the same W_x, W_h, and b are used at every time step. That's **weight sharing through time**.

Then we do ordinary backpropagation on this unrolled graph. Here there is the central idea of Backpropagation Through Time (BPTT): ordinary backpropagation applied to the RNN unrolled through time. Suppose an error occurs at a later time:

$$h_1 \to h_2 \to h_3 \to h_4 \to E_4$$

During backpropagation, information flows backward:

$$E_4 \to h_4 \to h_3 \to h_2 \to h_1$$

Therefore an error occurring **later in the sequence** can affect gradients associated with **earlier recurrent states**. Mathematically, this is just the **chain rule**, exactly like ordinary backpropagation:

$$\frac{\partial E_t}{\partial h_k} = \frac{\partial E_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial h_{t-1}} \cdot \frac{\partial h_{t-1}}{\partial h_{t-2}} \cdots \frac{\partial h_{k+1}}{\partial h_k}$$

Why do long sequences cause problems? Suppose we unroll an RNN for a long sequence of length T, say $T = 100$ time steps. Computationally, we've created something that behaves somewhat like a network that's **100 layer deep in time**:

$$h_1 \to h_2 \to h_3 \to \cdots \to h_{100}$$

And during Backpropagation Through Time (BPTT) we're repeatedly multiplying gradients through the chain rule:

$$\frac{\partial E_t}{\partial h_k} = \frac{\partial E_t}{\partial h_t} \cdot \prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}}$$

If those factors repeatedly have magnitude below 1:

$$0.5 \cdot 0.5 \cdot 0.5 \cdots \approx 0$$

The **gradient vanishes**. And similarly, if those factors repeatedly have magnitude above 1:

$$2 \cdot 2 \cdot 2 \cdots \approx \infty$$

The **gradient explodes**. That's why vanilla RNNs trained with BPTT have the famous **vanishing/exploding gradient problem**, especially over long

sequences.

RNN has recurrence over time
↓
Unroll it across the sequence
↓
Same weights are reused at every time step
↓
Apply ordinary chain-rule backpropagation
↓
Errors can flow from later to earlier states
↓
Long chains can cause vanishing/exploding gradients

Now let's analyze the statements individually:

✗ ***It cannot suffer from vanishing gradients.***

This is **false**. Actually, BPTT **can very much suffer from vanishing gradients**. When we unfold an RNN:

$$h_1 \to h_2 \to h_3 \to \cdots \to h_T$$

And propagate the gradient backward, the chain rule repeatedly multiplies derivatives:

$$\frac{\partial E_t}{\partial h_k} = \frac{\partial E_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial h_{t-1}} \cdot \frac{\partial h_{t-1}}{\partial h_{t-2}} \cdots \frac{\partial h_{k+1}}{\partial h_k}$$

For a long sequence, if these factors tend to have magnitude below 1, repeated multiplication makes the gradient approach zero.

✓ ***Long unfolded sequences can behave like very deep computational graphs.***

This is **true**. Suppose the RNN processes 100 time steps. When unfolded:

$$h_1 \to h_2 \to h_3 \to \cdots \to h_{100}$$

We now have a computational chain with 100 recurrent applications. It's not 100 independent layers with different weights, the parameters are shared, but from the perspective of **gradient propagation**, the computational graph is extremely deep.

✗ ***It is irrelevant to recurrent neural networks.***

This is **false**. BPTT is specifically the standard backpropagation formulation for recurrent networks. We start with an RNN:

$$h_t = f\left(W_x x_t + W_h h_{t-1} + b\right)$$

Unfold it:

$$h_1 \to h_2 \to h_3 \to \cdots \to h_T$$

And then apply backpropagation through this temporal computational graph.

✔ ***It can propagate error signals from later outputs to earlier recurrent states.***

This is **true**. It is one of the fundamental reasons BPTT exists. Imagine:

$$h_1 \to h_2 \to h_3 \to h_4 \to y_4$$

And the prediction y_4 generates a loss E_4. During backpropagation, gradients travel backward through the dependencies:

$$E_4 \to h_4 \to h_3 \to h_2 \to h_1$$

So an error at a **later time** can generate gradients associated with computations at **earlier time steps**. That's exactly what "*backpropagation through time*" means.

✖ ***It requires the hidden state to be manually labelled at every step.***

This is **false**. The hidden state h_t is an **internal representation computed by the RNN**. They arise from the computation itself, not from any external labels. Depending on the task, we may have targets associated with outputs, but that is completely different from manually labelling every hidden state.

✔ ***It applies the same chain-rule principle used in ordinary backpropagation.***

This is **true**. BPTT is **not a fundamentally different mathematical algorithm** from ordinary backpropagation.

1. Unfold the recurrent computation through time.
2. Obtain a computational graph that is a DAG (Directed Acyclic Graph).
3. Apply the **chain rule backward through that graph**.

✔ ***It unfolds the recurrent computation over time.***

This is **true**. The recurrent representation looks conceptually like a loop: h_t ↻. Because the next computation depends on the previous hidden state. For BPTT, we represent those successive applications explicitly:

$$h_1 \to h_2 \to h_3 \to h_4$$

This is called **unfolding** or **unrolling** the RNN **through time**.

✔ ***It trains weights that are shared across time steps.***

This is **true**. When we draw:

$$h_1 \to h_2 \to h_3 \to h_4$$

It might look as though we have four different neural-network layers. But we don't have:

$$W_h^{(1)}, W_h^{(2)}, W_h^{(3)}, W_h^{(4)}$$

Instead, the **same recurrent parameters** are reused at every time step:

$$h_1 = f\left(W_x x_1 + W_h h_0 + b\right)$$
$$h_2 = f\left(W_x x_2 + W_h h_1 + b\right)$$
$$h_3 = f\left(W_x x_3 + W_h h_2 + b\right)$$
$$h_4 = f\left(W_x x_4 + W_h h_3 + b\right)$$

So the same weights W_x, W_h, and b are shared across time steps. BPTT therefore computes gradient contributions coming from the different time steps for these **shared parameters**.

Exercise 8 [1 pt]. For each of the following statements about neural autoencoders, state if they are true or false.

T/F. The encoder maps from hidden representation to reconstruction.

T/F. Given a sample, its autoencoder's hidden representation can be used as features for another model.

T/F. A reconstruction error can be used as a signal for anomaly detection.

T/F. A denoising autoencoder receives a corrupted input and learns to recover a clean version.

T/F. A sparsity penalty encourages all hidden units to be strongly active for every input.

Solution 8. Let's do a quick theory recap before analyzing the statements individually.

What is an Autoencoder? We encountered this concept briefly earlier when discussing *reconstruction loss* (page 655). A neural autoencoder is trained to approximately **reproduce its input at its output**:

$$x \to \text{Autoencoder} \to \hat{x}$$

With the goal:

$$\hat{x} \approx x$$

But if all we're doing is copying x to $\hat{x}$, why use a neural network at all? The interesting part is what happens **inside** (page 313):

$$x \xrightarrow{\text{Encoder}} h \xrightarrow{\text{Decoder}} \hat{x}$$

Where x is the original input, h is the **hidden/latent representation** (or *latent code*), and $\hat{x}$ is the reconstructed input. For example, imagine x is a large vector $x \in \mathbb{R}^{1000}$. The encoder might compress it into a smaller hidden representation $h \in \mathbb{R}^{50}$. Then the decoder tries to reconstruct the original 1000 values:

$$\mathbb{R}^{1000} \xrightarrow{\text{encoder}} \mathbb{R}^{50} \xrightarrow{\text{decoder}} \mathbb{R}^{1000}$$

And because the network must reconstruct x from this constrained representation, h should capture useful structure in the input.

How is an Autoencoder trained? We compare the input x with the reconstruction $\hat{x}$ using the **reconstruction loss** we discussed earlier (page 655):

$$\mathcal{L}_{\text{rec}}(x, \hat{x})$$

For example, depending on the data, it might be squared error:

$$\mathcal{L}_{\text{rec}}(x, \hat{x}) = \|x - \hat{x}\|^2$$

Training tries to make $\hat{x}$ as close to x as possible: $\hat{x} \approx x$. Notice that we **don't need any semantic class labels** for this. The input itself supplies what must

be reconstructed, and that's why we present autoencoders in the chapter on **unsupervised representation learning**.

Why is h useful? We aren't necessarily interested in the reconstructed output itself. We hope that the hidden representation h:

$$h = \text{Encoder}(x)$$

Becomes a useful representation of the input x. For example:

$$x \in \mathbb{R}^{1000} \rightarrow h \in \mathbb{R}^{50}$$

This means that the 50-dimensional representation h captures the most important information from the original 1000-dimensional input x. Those 50 numbers can potentially capture important patterns in the original 1000-dimensional sample. Then we could take h and give it to another machine-learning model as features. This is **feature extraction** or **representation learning**.

Three variants of Autoencoders. There are many variants of autoencoders, but three common ones are (page 317):

- **Denoising Autoencoder (DAE)**. Instead of giving the network clean x and asking for x, deliberately corrupt the input with noise:

  $$x \rightarrow \text{add noise} \rightarrow \tilde{x}$$

  Then train the network to reconstruct the original clean input x from the noisy version $\tilde{x}$:

  $$\tilde{x} \xrightarrow{\text{Autoencoder}} \hat{x} \approx x$$

  This helps the network learn to ignore noise and focus on the underlying structure of the data.

- **Anomaly Detection Autoencoder (ADA)**. Suppose we train an autoencoder only on **normal machine behavior**. It becomes good at reconstructing normal samples:

  $$x_{\text{normal}} \rightarrow \hat{x}_{\text{normal}}$$

  With:

  $$\|x_{\text{normal}} - \hat{x}_{\text{normal}}\|^2 \approx 0$$

  Now give it something very unusual that differs substantially from its training distribution. It may reconstruct it poorly:

  $$x_{\text{anomaly}} \rightarrow \hat{x}_{\text{anomaly}}$$

  With:

  $$\|x_{\text{anomaly}} - \hat{x}_{\text{anomaly}}\|^2 \gg 0$$

  Therefore the **reconstruction error itself can become an anomaly signal**.

- **Sparse Autoencoder (SAE)**. We add a constraint encouraging only a **small subset of hidden units to be active** for a particular input. Conceptually, instead of:

$$h = [0.8, 0.7, 0.9, 0.6, 0.8, 0.9, \dots]$$

We might encourage something more like:

$$h = [0, 0, 0.8, 0, 0, 0.6, 0, 0, \dots]$$

This is done by adding a **sparsity penalty** to the loss function, which encourages most hidden units to be near zero for any given input. This can help the autoencoder learn more distinct features and avoid trivial solutions.

Now we can analyze the statements individually:

F. ***The encoder maps from hidden representation to reconstruction.***

This is clearly **false**. An autoencoder has:

$$x \xrightarrow{\text{Encoder}} h \xrightarrow{\text{Decoder}} \hat{x}$$

Where x is the input, h is the hidden representation, and $\hat{x}$ is the reconstruction. The encoder maps from the input x to the hidden representation h, not from h to $\hat{x}$. The **decoder** is responsible for mapping from h to $\hat{x}$.

T. ***Given a sample, its autoencoder's hidden representation can be used as features for another model.***

This is **true**. And this is one of the main reasons autoencoders are interesting. The hidden representation h is something the autoencoder has **learned from the data**. Suppose $x \in \mathbb{R}^{1000}$ and the encoder produces $h \in \mathbb{R}^{50}$. After training, we could discard the decoder and use h as a **feature vector** for another machine-learning model:

$$x \xrightarrow{\text{trained encoder}} h \xrightarrow{\text{another model}} y$$

T. ***A reconstruction error can be used as a signal for anomaly detection.***

This is **true**. Imagine training an autoencoder on **normal examples**. It learns to reconstruct normal patterns well:

$$x_{\text{normal}} \to \hat{x}_{\text{normal}} \approx x_{\text{normal}}$$

Therefore:

$$\mathcal{L}_{\text{rec}}(x, \hat{x}) \text{ is small for normal } x$$

Now give it an unusual sample that differs from what it learned. It may reconstruct that sample poorly:

$$\hat{x}_{\text{anomaly}} \not\approx x_{\text{anomaly}}$$

Therefore:

$$\mathcal{L}_{\text{rec}}(x, \hat{x}) \text{ is large for anomalous } x$$

So the reconstruction error itself can serve as an anomaly signal.

T. ***A denoising autoencoder receives a corrupted input and learns to recover a clean version.***

This is **true**. It is the definition of a **denoising autoencoder**. We start with clean data x, we deliberately corrupt it $x \to \tilde{x}$, and then give the corrupted version to the network:

$$\tilde{x} \xrightarrow{\text{Encoder}} h \xrightarrow{\text{Decoder}} \hat{x}$$

But the desired reconstruction is the **original clean** x, so the loss function compares $\hat{x}$ to x:

$$\mathcal{L}_{\text{rec}}(x, \hat{x})$$

F. ***A sparsity penalty encourages all hidden units to be strongly active for every input.***

This is **false** because it's the **opposite of sparsity**. If every hidden unit were strongly active:

$$h = [0.8, 0.7, 0.9, 0.6, 0.8, 0.9, \dots]$$

That's a **dense activation**. Sparsity instead encourages something like:

$$h = [0, 0, 0.8, 0, 0, 0.6, 0, 0, \dots]$$

Only a relative small subset of units should be strongly active. So sparsity penalty encourages many hidden activations to be inactive/small, not make every hidden unit strongly active.

References

[1] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of machine learning research*, 3(Feb):1137–1155, 2003.

[2] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4):303–314, 1989.

[3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

[4] Kurt Hornik. Approximation capabilities of multilayer feedforward networks. *Neural networks*, 4(2):251–257, 1991.

[5] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[6] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr, 2015.

[7] Slava Katz. Estimation of probabilities from sparse data for the language model component of a speech recognizer. *IEEE transactions on acoustics, speech, and signal processing*, 35(3):400–401, 1987.

[8] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.

[9] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[10] Fraser Love. Nntikz: Tikz diagrams for deep learning and neural networks, 2024.

[11] Matteucci Matteo. Artificial neural networks and deep learning. Slides from the HPC-E master's degree course on Politecnico di Milano, 2025-2026.

[12] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[13] Jeffrey Pennington, Richard Socher, and Christopher D Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543, 2014.

[14] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[15] Petar Veličković. Tikz: Complete collection of my pgf/tikz figures, 2018. https://github.com/PetarV-/TikZ.

Index

A

Activation Function 60
Activation Potential 60
ADALINE (Adaptive Linear Neuron) 42
Adam (Adaptive Moment Estimation) 198
Adaptive Learning Rate Methods: Adam (Adaptive Moment Estimation) 198
Adaptive Learning Rate Methods: RMSProp (Root Mean Square Propagation) 198
Anomaly Detection Autoencoder (ADA) 671
Argmax Mask 642
Artificial Neuron 39
Autoregressive (AR) Model 205
Averaging Filter 498

B

Backpropagation 100
Backpropagation Through Time (BPTT) 242
Bag-of-Words (BoW) 320
Batch Gradient Descent 119
Batch Normalization (BN) 189
Bayesian Optimization 161
Beam Search 307
Bias 39
Bias-Variance Trade-off 129
Bidirectional LSTM (BiLSTM) 282
Binary Cross-Entropy 656
Binary Cross-Entropy (BCE, Log Loss) 77
Binary Sentiment Label 648
Bottleneck Problem 300

C

Candidate cell state $\tilde{C}_t$ 269
Categorical Cross-Entropy 301, 658
Categorical Cross-Entropy (CCE) 83, 302
Chain Rule 100
Chain Rule of Probability 332
Channel-First Flattening 428
Channel-Last Flattening 428
CIFAR-10 431
Classification 12, 644
Clustering 15, 645
CNN Data-Driven Feature Vector 565
CNN Latent Representation 565
Column-Wise Flattening 427
Computer Vision (CV) 402
Conditional Language Model 296, 297
Constant Error Carousel (CEC) 265
Context Vector 299

Context Window 334
Continuous Bag-of-Words (CBOW) 362, 367
Core Learning Rule 95
Correaltion 416
Cosine Similarity 383
Cost Function 90
Covariate Shift 188
Cross-Validation 136
Curse of Dimensionality 321

D

Data Augmentation 442
Data Leakage 135
Dead ReLU Problem 180
Decision Boundary 51
Decision Boundary Equation 44
Decoder 299
Decoding Problem 226
Deformation 454
Denoising Autoencoder (DAE) 671
Density Estimation 310
Deterministic Recurrent Systems 231
Dictionary IDs 319
Dimensionality Reduction 310
Distributed Representation 329
Distributional Hypothesis 327
Dropout 173, 174
Dying ReLU Problem 180

E

Early Stopping 152
Embedding Matrix 346
Empirical Risk 193
Encoder 299
Encoder-Decoder Framework 299
End-to-End Learning 488
Error Function $E(w)$ 90
Experience (E) 8
Exploding Gradient Problem 179
Exponential Linear Unit (ELU) 182
Exposure Bias 304

F

Feature 471
Feature Engineering (Traditional ML) 24
Feature Extraction 310
Feature Extraction Network (FEN) 565
Feed-Forward Neural Network (FNN) 57
Feed-Forward Time-Delay Neural Network (TDNN) 207
Filter 416

Fixed Learning Rate 197
Flattening 427
Forget gate f_t 268

G

Gated Recurrent Unit (GRU) 271
GloVe (Global Vectors for Word Representation) 390
Gradient Clipping 257, 262
Gradient Descent 94
Greedy Search 307
Grid Search 159

H

Hand-Crafted Feature 474
He (Kaiming) Initialization 186
Hebbian Learning Rule 47
Hidden Markov Model (HMM) 220
Hold-Out Method 133, 137
Hold-Out Validation 137
Hyperbolic Tangent (tanh) Activation Function 67
Hyperparameter Tuning 155
Hyperparameter Tuning Algorithm 157
Hyperplane 114

I

Identity Filter 498
Image Augmentation 442
Image Classification Problem 425
Independent and Identically Distributed (i.i.d.) 108
Inductive Bias 128, 605
Inductive Hypothesis 128, 130
Information Retrieval (IR) 381
Input gate i_t 269
Inter-Class Variability 455
Internal Covariate Shift 188

K

K-Fold Cross-Validation 142
k-Nearest Neighbours (k-NN) 460
Kalman Filter 215
Katz Smoothing 338
Keras 579
Kernel 416

L

L2 Regularization 164
Label Ambiguity 450
Language Model (LM) 296, 331
Leaky ReLU 181
Learned Features (Deep Learning) 24
Learning Rate Scheduling (decay) 197

Learning Rate Scheduling: 1/t Decay 197
Learning Rate Scheduling: Cosine Annealing 197
Learning Rate Scheduling: Exponential Decay 197
Learning Rate Scheduling: Step Decay 197
Leave-One-Out Cross-Validation (LOOCV) 139
LeNet-5 574
Linear Activation Function 62
Linear Dynamical System (LDS) 212
Linearly Separable 54
Local Representations 329
Logistic Activation Function 64
Long Short-Term Memory Network (LSTM) 264
Loss Function 90

M

Machine-Health Estimator 646
MADALINE (Multiple ADALINE network) 42
Manhattan distance 459
Many-to-Many Problems 290
Many-to-Many Problems (with aligned sequences) 291
Many-to-One Problems 289
Markov Assumption 334
Markov Chain 219
Markov Property 211
Maximum A Posteriori (MAP) 167
Maximum Likelihood Estimation (MLE) 107
Mean Absolute Error 74
Mean Squared Error (MSE) 72, 660
Memory Cell 264
Memoryless Models 203
Mini-Batch 630
Mini-Batch Gradient Descent (MBGD) 194
Model Complexity vs. Error Curve 131
Models with Memory 203
Momentum 198
Motion Blur Filter 502

N

N-gram Language Model 334
Negative Log-Likelihood (NLL) 354
neighbourhood 410
Nested Cross-Validation 145
Net Input 60
Neural Autoencoder 312
Neural Network 41
Neural Sparse Autoencoder 315
No Padding 540

O

Ockham's Razor 124

One-Hot Encoding 80, 319
One-to-Many Problems 288
One-to-One Problems 288
Output gate o_t 270
Overfitting 126

P

Parametric ReLU (PReLU) 181
Patience Parameter 153
Patience Window 153
Perceptron 41, 43
Perceptron Convergence Theorem 120
Perceptual Similarity 456, 463
Performance measure (P) 8
Perplexity 359
Pixel Similarity 456, 463
Pixel-Wise Distances 458
Probabilistic Dynamical Systems 230

R

Random Search 160
Random Subsampling 133
Receptive Field 609
Reconstruction Error 313
Reconstruction Loss 655
Rectified Linear Unit (ReLU) 180
Recurrent Neural Network (RNN) 232
Regression 13, 644
Reinforcement Learning 310, 646
Reinforcement Learning (RL) 20
Representation Learning 310, 645
RMSProp (Root Mean Square Propagation) 198
Row-Wise Flattening 427

S

Scaled Exponential Linear Unit (SELU) 182
Semantic Class Label 648
Semantic Relatedness 322
Semantic Similarity 322, 456
Sentiment Analysis 388
Sequence Labeling 291
Sequence Tagging 291
Sequence-to-Sequence Learning 293
Sigmoid Activation Function 64
Skip-Gram 362
Sliding Window 412
Smoothing 330
Softmax Activation Function 81
Sparse Autoencoder (SAE) 672
Spatial Resolution 510

Special Tokens 305
State Estimation Problem 213
State Propagation 202
Stochastic Gradient Descent (SGD) 119
Stratified Sampling 133
Stride 544
Sum of Squared Errors (SSE) 91
Supervised Learning 12, 87, 309, 645
Symmetric Padding 543

T

t-distributed Stochastic Neighbor Embedding (t-SNE) 466
Task (T) 8
Task, Experience, Performance 8
Teacher Forcing 303
Temporal Dimension 202
Term Frequency-Inverse Document Frequency (TF-IDF) 381
Test Set 134
Threshold Logic Unit (TLU) 41
Token 297
Training 88
Training Dataset 134
Training Set 134
Training vs. Validation Error Curve 151
Transformation Invariance 451
Translation Invariance 605
Truncated Backpropagation Through Time (TBPTT) 244

U

Underfitting 127
Unfolding 427
Universal Approximation Theorem 84, 123
Unsupervised Learning 15, 309, 646

V

Valid Convolution 540
Validation Set 134
Vanishing Gradient Problem 177
Vectorization 427
Viewpoint Change 454
Viterbi Algorithm 226

W

Weight Decay 164
Weight Scaling Rule 174
Weight Update Rule in Hebbian Learning 48
Word Embedding 325
Word2Vec 362

X

Xavier (Glorot) Initialization 186

Z

Zero Padding 541
Zero-Gradient Problem 177
Zero-Probability Problem 337